To the University of Wyoming:

The members of the Committee approve the dissertation of Kegan C. McIlwaine presented on May 1, 2026.

James Caldwell, Chairperson

Tyrrell McAllister, Outside Member

John Hitchcock

Ruben Gamboa

Mike Borowczak, Associate Professor at University of Central Florida

APPROVED:

Ian D. Walker, Head, Department of Electrical Engineering and Computer Science

Daniel Dale, Interim Dean, College of Engineering and Physical Sciences

McIlwaine, Kegan C., Contracts, Simulations, and Sessions: Formal Verification of Compilers, Smart Contracts, and Protocols, Ph.D., Department of Electrical Engineering and Computer Science, May, 2026.

Smart contracts are self-enforcing, self-executing protocols governing interactions between several (potentially distrusting) parties. Smart contracts automate the execution of an agreement between participants, typically using the blockchain. The potential for, and the magnitude of, financial losses motivate the application of the most rigorous correctness guarantees to smart contracts, *i.e.*, formal verification. This dissertation applies concurrent process theory and formal verification across the full smart contract development pipeline, from semantics and compilation to transformation correctness and protocol specification. At the level of language semantics, this dissertation provides a small-step operational semantics for the Marlowe smart contract programming language. This clean semantic foundation is then used to verify the correctness of the Faustus smart contract programming language compiler. Faustus is a smart contract programming language that extends Marlowe with parameterized contract abstractions and an expression language for guarded commands with operators based on CCS. The compiler is verified in stages: first the normalization of the guard expressions is verified, and then the compilation to Marlowe is verified. Faustus contracts are then formalized as a labeled transition system that distinguishes observable and unobservable behavior. This formalization allows a smart contract transformation to be verified using weak bisimulation. The observable behaviors of the original contract are preserved in the transformed contract. Finally, this dissertation presents STILL, a standalone interactive tactic-based theorem prover for dependent session types. Unlike similar provers for linear type theories, STILL allows for delayed linear context partitioning at the tactic level rather than modifying the underlying proof rules. Protocols for the axiom of choice, natural numbers, and a long-running bit counter are verified to demonstrate the properties of the STILL system and its underlying type theory. This dissertation uses simulation, bisimulation, theorem proving, and process extraction to connect implementations to formal specifications. Together, these results show how formal methods based on concurrency and type theory can provide correctness guarantees at each step of protocol specification and smart contract development.

CONTRACTS, SIMULATIONS, AND SESSIONS: FORMAL VERIFICATION OF COMPILERS, SMART CONTRACTS, AND PROTOCOLS

by

Kegan C. McIlwaine, B.S. Computer Science

A dissertation submitted to the
Department of Electrical Engineering and Computer Science
and the
University of Wyoming
in partial fulfillment of the requirements
for the degree of

DOCTOR OF PHILOSOPHY
in
COMPUTER SCIENCE

Laramie, Wyoming
May 2026

Copyright © 2026

by

Kegan C. McIlwaine

I dedicate this work to my family and friends. Thank you for your support.

Contents

List of Figures **ix**

List of Tables **xiii**

List of Computer Programs **xvii**

Acknowledgments **xix**

Chapter 1 Introduction **1**

1.1 Goals and Motivations 1

1.2 Related Work 3

1.2.1 Related Work for Marlowe Small-Step Semantics 3

1.2.2 Related Work for Faustus 5

1.2.3 Related Work for Smart Contract Verification 6

1.2.4 Related Work for Interactive Theorem Proving Using Dependent Session Types 7

1.3 Dissertation Overview and Organization 9

Chapter 2 Formal Semantics for Marlowe **11**

2.1 Contributions in This Chapter 11

2.2 Marlowe 12

2.2.1 The Marlowe State 12

2.2.2 Marlowe Contract Structure 14

2.2.3 Marlowe Input Processing 16

2.2.4 Marlowe Internal Reductions 17
2.2.5 Marlowe Transaction Processing 18
2.3 Formalized Marlowe Semantics 19
2.3.1 Small-Step Reduction Semantics 19
2.3.2 Small-Step Input Processing Semantics 22
2.3.3 The Full Marlowe Small-Step Semantics 23
2.3.4 Verifying the Small-Step Reduction Semantics 24
2.3.5 Verifying the Small-Step Input Processing Semantics 25
2.3.6 Meta-Properties of the Small-Step Semantics 27
2.4 Discussion 29

Chapter 3 Faustus **31**
3.1 Motivations 31
3.2 Contributions in This Chapter 32
3.3 Formalized Faustus Semantics 33
3.3.1 Adding Abstractions 34
3.3.2 Concurrency Operators for Interaction 38
3.3.3 Faustus Internal Reduction Semantics 39
3.3.4 Faustus Input Processing Semantics 41
3.4 Type-Checking Faustus 42
3.4.1 Preservation and Progress for Well-Typed Faustus Contracts 43
3.5 Formalized Faustus Compiler 46
3.5.1 Compiling Concurrency Operators via Normalization 47
3.5.2 Compiling Abstractions 51
3.6 Conciseness of Faustus Programs 56
3.7 Discussion 59

Chapter 4 Verifying Smart Contract Transformations Using Bisimulation **61**
4.1 Theoretical Supports 61
4.1.1 The Calculus of Communicating Systems 61

4.2 Contributions in This Chapter 63
4.3 Equivalent Contracts 64
4.3.1 Faustus Processes 65
4.3.2 Observational Equivalence 70
4.4 Contract Transformations 71
4.4.1 Restriction - Making Observable Actions Unobservable 74
4.4.2 Verifying the Transformation 74
4.5 Discussion 77

Chapter 5 STILL: A Tactic-Based Theorem Prover for Dependent Session Types 79

5.1 Motivation 79
5.2 Theoretical Supports 80
5.2.1 The π-calculus 80
5.2.2 Linear Logic 82
5.2.3 Session Types 85
5.2.4 The Extended Calculus of Constructions 88
5.2.5 Tactic-Based Theorem Provers 92
5.3 Contributions in This Chapter 95
5.4 The STILL Theorem Prover 96
5.4.1 Dependent Session Types 97
5.4.2 Tactics for Session Types 102
5.4.3 Evaluating the System 109
5.4.4 Discussion 116

Chapter 6 Verifying Protocols 119

6.1 Contributions in This Chapter 119
6.2 Using the STILL Theorem Prover 119
6.2.1 Top-Level Commands 120
6.2.2 Proving Theorems 121

6.3 The Axiom of Choice as a Protocol 122
6.4 The Natural Numbers as Concurrent Processes 124
6.5 Bit Counting Processes 127
6.6 Discussion 131

Chapter 7 Conclusion **133**

Appendix A Marlowe Isabelle Formalization **137**

Appendix B Faustus Type-Checking Reference **175**

Appendix C Faustus Isabelle Formalization **181**

Appendix D Faustus Processes and Bisimulation Isabelle Formalization **317**

Appendix E STILL Example Proofs **437**

References **446**

List of Figures

1.1 Simulation diagram showing correctness of an implementation $\mathcal{I}$ with respect to a specification $\mathcal{S}$. 3

1.2 Simulation diagram showing adequacy of an implementation $\mathcal{I}$ with respect to a specification $\mathcal{S}$. 3

1.3 Simulation diagram showing an implementation $\mathcal{I}$ is correct and adequate with respect to a specification $\mathcal{S}$. 4

2.1 Simulation diagrams for verifying the Marlowe small-step semantics. 12

3.1 Marlowe simple escrow contract. 32

3.2 The simulation diagrams for correctness of the Faustus compiler. 33

3.3 Faustus contract snippet demonstrating issue that occurs by allowing lazy `Observation` abstractions with eager `Value` abstractions. 37

3.4 Faustus contract demonstrating shadowing issue. 38

3.5 Faustus simple escrow contract. 57

3.6 Faustus five party multi-signature contract. 58

4.1 The CCS process syntax. 62

4.2 The CCS reaction rules. 62

4.3 A contract (P) and the result (Q) of recursively applying the transformation. 72

4.4 A contract without disjoint guards before and after applying the transformation. 75

5.1 The π-calculus syntax. 80

5.2 The π-calculus rules for free names. 81

5.3 The π-calculus rules for structural congruence. 81
5.4 The π-calculus reduction rules. 82
5.5 Inference rules for classical linear logic. 84
5.6 Inference rules for intuitionistic linear logic. 85
5.7 Type rules for the π-calculus. 87
5.8 The Extended Calculus of Constructions syntax. 89
5.9 Function for computing free names in Extended Calculus of Constructions terms. 89
5.10 Term reduction rules for the Extended Calculus of Constructions. 90
5.11 Type-checking rules for ECC. 91
5.12 Selected rules demonstrating dependency between the session and functional layer. 98
5.13 Type-checking rules for coinductive session types 99
5.14 Type-checking rules for polymorphic session types 101
5.15 Concrete syntax for terms and types in the STILL prover. 102
5.16 Delayed partitioning during proof construction. The proof is constructed by applying tactics to candidate sets at the leaves of a partial derivation. 105
5.17 The architecture of the STILL prover. 108
5.18 Example prompts and commands. 110
5.19 English auction specification and extracted π-calculus term. 114
5.20 Book Auction Protocol 115

B.1 Faustus `Party`, `Payee`, and `ChoiceId` type rules. Implemented in Isabelle with their respective `wellTyped` functions. 175
B.2 Faustus `Value` type rules. Implemented in Isabelle with the `wellTyped-Value` function. 176
B.3 Faustus `Observation` type rules. Implemented in Isabelle with the `well-TypedObservation` function. 176
B.4 Faustus Statement ($\mathcal{S}$) type rules. Implemented in Isabelle with the `well-TypedStatement` function. 177

B.5 Faustus `Action` and `GuardExpression` type rules. Implemented in Isabelle with their respective `wellTyped` functions. 177
B.6 Faustus `Contract` ($\mathcal{C}$) and `Case` type rules. Implemented in Isabelle with the `wellTypedContract` and `wellTypeCase` function. 178
B.7 Faustus context type rules. Implemented in Isabelle as the `wellTypedContext` function. 178
B.8 Faustus state type rules. Implemented in Isabelle as the `wellTypedState` function. 179

List of Tables

5.1 Benchmarks for the STILL prover. 113

List of Computer Programs

2.1 Isabelle type declarations for the Marlowe `State`. 13

2.2 Isabelle type declarations for the Marlowe `Environment`. 14

2.3 Isabelle type declarations for the Marlowe `Contract` and `Case` datatypes. . 15

2.4 Isabelle type declarations for Marlowe `Action`s and `Input`s. 16

2.5 Isabelle type declarations for Marlowe reduction results. 17

2.6 Marlowe small-step semantics 20

2.7 Marlowe small-step semantics for processing input. 23

2.8 The Isabelle inductive definition of the full Marlowe semantics. 24

2.9 A `Reduced` result from the `reduceContractStep` function implies a step in the Marlowe small-step semantics. 25

2.10 A step in the Marlowe small-step semantics implies a corresponding `Reduced` result. 25

2.11 An `Applied` result from `applyCases` implies a step in the Marlowe small-step semantics for input processing. 26

2.12 A step in the Marlowe input processing semantics with an input implies an `Applied` result from the `applyCases` function. 26

2.13 A step in the Marlowe input processing semantics without an input implies a `Reduced` result from the `reduceContractStep` function. 26

2.14 Isabelle lemmas proving the Marlowe small-step semantics are deterministic. 27

2.15 An Isabelle lemma proving that `reductionLoop` corresponds to the reflexive transitive closure of the Marlowe small-step reduction semantics. 28

3.1 Isabelle type declarations for the Faustus *AbstractionInformation* and *FContext* types. 34
3.2 Isabelle type declarations for the Faustus *FStatement*s and *FContract*s. 35
3.3 Isabelle record declaration for the *FState*. 35
3.4 Isabelle type declarations for the Faustus *FParty*, *FObservation*, and *FArgument* constructs. 36
3.5 The abstract syntax for the Faustus *FGuardExpression*. 39
3.6 Isabelle inductive rules added to the Marlowe internal reduction semantics to create the Faustus internal reductions. 40
3.7 The reduction semantics for a single *FGuardExpression* over an input. . . 41
3.8 Isabelle type information for the *AbstractionTypeInformation*, *Type-Context*, and type-checking functions. 42
3.9 Isabelle lemmas proving a well-typed *Party*, *Payee*, *ChoiceId*, *Value*, and *Observation* will evaluate to a meaningful value. 43
3.10 Isabelle lemmas proving a well-typed *Statement* and *Argument list* will evaluate to a meaningful value. 44
3.11 An Isabelle lemma proving that a well-typed Faustus *Contract* only halts when reaching a *When* or *Close*. 45
3.12 The Faustus type preservation lemmas in Isabelle. 45
3.13 The type signatures of the Faustus compiler functions. 46
3.14 Isabelle type information for Faustus guard normalization. 47
3.15 Isabelle lemma proving that a well-typed Faustus contract will be well-typed after normalization is applied. 48
3.16 Isabelle lemmas proving the correctness of the Faustus guard normalization process. 48
3.17 Isabelle lemma proving a well-typed Faustus configuration will succeed in compiling to a Marlowe configuration. 53
3.18 Isabelle lemmas proving the correctness of the Faustus compiler. 53
4.1 Isabelle implementation of the transformation algorithm. 73

6.1 A proof of the axiom of choice using the STILL prover. 123
6.2 Verification of the natural numbers using polymorphic session types to represent inductive types. 125
6.3 Verification of the little endian bit counter protocol in the STILL prover. . . 128
A.1 The Isabelle definition of the Marlowe small-step semantics. 137
C.1 Faustus semantics without guard expressions. 181
C.2 The Faustus type-checker in Isabelle. 308
D.1 Formalization of Faustus processes. 317
D.2 Formalization of Faustus bisimulation and transformation verification. 335
E.1 The verification of an English auction protocol in the STILL theorem prover. 437
E.2 The verification of a bank and ATM client protocol in the STILL theorem prover. 440
E.3 The verification of a cloud server protocol and implementation in the STILL theorem prover. 442
E.4 The verification of a protocol with one thousand tensor applications for stress testing the STILL theorem prover. 443

Acknowledgments

Thank you to Professor Caldwell for meeting multiple times every week throughout the last six years. The numerous topics we covered in our discussions provided inspiration for the projects in this dissertation. I also thank the members of my committee for their support over the years. I would also like to thank the Marlowe team at Input-Output Global for our meetings throughout the years. It was especially useful to talk to a group focusing on formal methods and production-level smart contract deployments.

The work presented in this dissertation was supported by a grant from IOG Singapore Pte. Ltd. with matching funds from the State of Wyoming. Thank you especially to Charles Hoskinson for his support of the University of Wyoming.

Kegan C. McIlwaine

University of Wyoming

May 2026

Chapter 1

Introduction

1.1 Goals and Motivations

The underlying practical motivation that spans the work presented in this dissertation is to provide programmers writing smart contracts with confidence in the correctness of an implementation through every step of development. The work presented in this dissertation began with a simple project: extending the Marlowe smart contract programming language [1] with abstractions that would make it easier to write more complex contracts. This led to the development of a new language, Faustus, that compiles into Marlowe. After verifying the correctness of the Faustus compiler, the next goal was to prove the correctness of a transformation that is applied to smart contracts before they are loaded onto the blockchain. The final goal was to verify the correctness of the programs themselves against a specification of their behaviors.

Smart contracts were initially proposed by Szabo [2] as self-enforcing, self-executing protocols governing interactions between several (potentially distrusting) parties. Smart contracts have become popular with the rise of blockchains and cryptocurrencies. A blockchain is an unforgeable distributed ledger where transactions are added in a cryptographically secure manner. A cryptocurrency is the programmable money used in blockchain transactions. These platforms provide a medium where smart contracts can execute in a trusted and secure manner.

While the distributed consensus algorithms governing blockchains prevent the acceptance of fraudulent transactions, they do not prevent the execution of smart contracts with logical errors or vulnerabilities. Since smart contracts handle monetary transactions, vulnerabilities have severe financial consequences [3–5]. This motivates the application of formal verification to every stage of smart contract development, *i.e.*, verifying that the behavior of an implementation is correct with respect to a specification.

The framework for verifying the correctness of an "implementation" with respect to a "specification" is well understood and is a core piece of concurrent process theory [6, 7]. The specification describes the intended behavior of a system, while the implementation is an artifact that is intended to realize that behavior. A verification of correctness is presented using simulation diagrams with a correspondence relation $\mathcal{R}$ between specification states $\mathcal{P}_i$ and implementation states $\mathcal{Q}_i$, and transition or evaluation relations on configurations using the specification $\mathcal{S}$ and implementation $\mathcal{I}$. These diagrams show the existence of corresponding configurations after evaluation using the implementation or specification. In Figure 1.1, the implementation $\mathcal{I}$ is verified to be correct with respect to the specification $\mathcal{S}$. In Figure 1.2, the implementation $\mathcal{I}$ is verified to realize the behaviors of $\mathcal{S}$.

With just Figure 1.1, the behaviors of the implementation are shown to be behaviors of the specification, but there may be behaviors of the specification that the implementation does not implement. With just Figure 1.2, the implementation realizes all specified behaviors, but there still may be behaviors of the implementation that are not part of the specification. Proving both simulation diagrams hold shows that the implementation is correct and adequate with respect to the specification. The simulation diagram in Figure 1.3 represents the correctness and adequacy of an implementation with respect to a specification.

In this dissertation, the "specification" and "implementation" pairing differs in each chapter. Chapter 2 uses an existing evaluation function as the specification and develops a new small-step semantics as the implementation. Chapter 3 presents a source language semantics as the specification and a small-step semantics of a target compilation language as the implementation. Chapter 4 uses the semantics of an arbitrary Faustus contract as the specification and the semantics of a restricted, transformed version of that contract as the

$$\text{If}\quad \begin{array}{l} \mathcal{P}_1 \\ \mathcal{R} \\ \mathcal{Q}_1 \xrightarrow{\mathcal{I}} \mathcal{Q}_2 \end{array} \qquad \text{then, for some } \mathcal{P}_2 \qquad \begin{array}{l} \mathcal{P}_1 \xrightarrow{\mathcal{S}} \mathcal{P}_2 \\ \mathcal{R} \\ \mathcal{Q}_2 \end{array}$$

Figure 1.1: Simulation diagram showing correctness of an implementation $\mathcal{I}$ with respect to a specification $\mathcal{S}$.

$$\text{If}\quad \begin{array}{l} \mathcal{P}_1 \xrightarrow{\mathcal{S}} \mathcal{P}_2 \\ \mathcal{R} \\ \mathcal{Q}_1 \end{array} \qquad \text{then, for some } \mathcal{Q}_2 \qquad \begin{array}{l} \mathcal{P}_2 \\ \mathcal{R} \\ \mathcal{Q}_1 \xrightarrow{\mathcal{I}} \mathcal{Q}_2 \end{array}$$

Figure 1.2: Simulation diagram showing adequacy of an implementation $\mathcal{I}$ with respect to a specification $\mathcal{S}$.

implementation.

Moving away from simulation diagrams, Chapter 5 and Chapter 6 use session types as the specification and a process calculus term as the implementation. By using a type system that allows protocol behavior to be specified, a well-typed process calculus term is guaranteed to exhibit that behavior. Furthermore, proving that a protocol is valid allows for the extraction of a process calculus term that follows that protocol. We will discuss additional work that can be done to strengthen this guarantee in Chapter 7.

1.2 Related Work

1.2.1 Related Work for Marlowe Small-Step Semantics

Marlowe is one of many works in the field of developing programming languages for specifying and implementing financial contracts. Peyton Jones et al. [8] developed a declarative language for specifying executable financial contracts in Haskell and proved that two syntactically different contracts are equivalent using denotational semantics. This paper is one of

$$
\begin{array}{ccc}
\mathcal{P}_1 & \xrightarrow{\mathcal{S}} & \mathcal{P}_2 \\
\mathcal{R} & & \mathcal{R} \\
\mathcal{Q}_1 & \xrightarrow{\mathcal{I}} & \mathcal{Q}_2
\end{array}
$$

Figure 1.3: Simulation diagram showing an implementation $\mathcal{I}$ is correct and adequate with respect to a specification $\mathcal{S}$.

the few that focuses on proving the equivalence of contracts. Andersen et al. [9] expanded on that work with their Contract Specification Language (CSL). Then Henglein et al. [10] provided a static analysis framework for verifying properties of CSL contracts, such as fairness and party participation. In a separate piece of work, Seijas and Thompson [1] developed Marlowe, a Domain Specific Language (DSL) for writing financial contracts on the Cardano blockchain [11]. All Marlowe contracts are total – they are all guaranteed to terminate. Marlowe includes a reference evaluator in the Isabelle/HOL theorem prover which has been used to prove that all Marlowe contracts terminate and pay out any locked funds. Seijas et al. [12, 13] also developed a static analysis tool that can verify all possible executions of a Marlowe contract never result in an error.

The Marlowe small-step semantics presented in this dissertation aim to provide a proof-friendly semantics aligned with the preexisting evaluator. Similarly, the KEVM by Hildenbrandt et al. [14] provides a formal semantics of the Ethereum Virtual Machine (EVM) [15] in the $\mathbb{K}$ Framework [16]. Resembling the work presented here, the KEVM provides a formal semantics of a preexisting system. However, the verification of the formal semantics is done by running the evaluator extracted from the $\mathbb{K}$ Framework against the existing EVM test suite. Other higher-level smart contract languages have had their semantics formalized against preexisting implementations. One example is the work on Mi-Cho-Coq by Bernardo et al. [17] which presents semantics for the Michelson language on the Tezos blockchain [18]. Another is the semantics for Findel, another DSL for writing financial smart contracts, presented by Arusoaie [19]. A third is a denotational semantics for Solidity [20] presented by Marmsoler et al. [21, 22]. Bernardo et al. do not verify their semantics against the existing

evaluator, while Marmsoler et al. verify their semantics against a reference implementation of Solidity and Arusoaie tests their semantics against the existing Findel test suite. The verification of the Marlowe small-step semantics presented in Chapter 2 differs by proving a Marlowe contract executed in the new semantics simulates the contract executed in the original evaluator and vice versa.

1.2.2 Related Work for Faustus

Faustus extends Marlowe, so the development of Marlowe provides the closest related work to Faustus. The design of Marlowe aims to provide a foundation for exhaustive static analysis and formal reasoning. Developments around Marlowe include a web-based editor with templates for common contract structures [12], efficient static analysis of smart contracts [13], and a formal evaluator in Isabelle/HOL [1]. These tools aimed at improving the usability of Marlowe provide the motivation for the Faustus extensions that improve the expressiveness of smart contracts through abstractions and guarded commands.

The verification of the Faustus compiler in this dissertation proves the preservation of behaviors between the source and target semantics. CompCert by Leroy [23] is the canonical reference in this field. CompCert is implemented in Rocq and proves the preservation of semantics through multiple passes by proving simulation relations between each layer of operational semantics. This multiple layer simulation argument is followed by the structure of the Faustus compiler verification. The work presented in this dissertation was also guided by the work of Nipkow and Klein [24] who present the formalization of a programming language using Isabelle/HOL.

Faustus guard expressions are inspired by the choice operator in the Calculus of Communicating Systems (CCS) [25]. In the original presentation of CCS by Milner, the choice operator is nondeterministic. In Faustus, the semantics of matching guards to inputs is deterministic. The semantics of guard expressions prioritize the first guard in the list of guarded commands and the left side of the operand if multiple choices in a single guard match the input. This technique of adding priority to the choice operator in CCS can also be found in work by Camilleri and Winskel [26] that adds the priority choice operator to

CCS. Faustus goes further: it does not include a nondeterministic choice operator and it includes a priority interleaving operator that prioritizes the left operand when attempting transitions. The normalization step of the Faustus compiler is based on the CCS Expansion Law presented by Milner [27] which shows that parallel composition of processes can be rewritten as a sequence of choices. The Faustus normalization step adjusts the Expansion Law to account for the lack of nondeterministic operators and explicitly tracks the priority of each operand of the guard expression during normalization. Faustus guard expressions are also closely related to the guarded command language by Dijkstra [28]. Guarded commands as presented by Dijkstra use boolean guards and feature nondeterminism. Faustus guard expressions differ by matching on user input and prioritizing the first enabled guard in the list of guarded commands.

1.2.3 Related Work for Smart Contract Verification

Formal verification has previously been used to prove individual smart contracts satisfy certain properties. A recent survey of these efforts can be found in Tolmach et al. [29]. These techniques verify the correctness of a smart contract with respect to a formal specification. For example, tools have been developed to verify unannotated common smart contracts against their canonical forms [30], automated tools have been developed to verify liquidity in BitML [31] and Solidity contracts [20, 32, 33], and verified super-optimization techniques have been applied to Ethereum smart contracts [34].

Nelaturu et al. [35] have developed a methodology for optimizing the cost of smart contracts, and synthesized bisimulation relations to verify the correctness of the optimizations. Their techniques are limited by the constraints on the inputs to the automated tools they use in their system, *e.g.* the programs must be deterministic and the kinds of loops their system can operate on are restricted.

Similar to the work described in this dissertation, Sergey and Hobor [36] aim to provide an analogy between smart contracts and concurrent processes. Differing from previous works of low-level verification, Sergey and Hobor argue that smart contract programmers and verifiers will work more efficiently and be less error prone by adopting the perspective of

smart contracts as concurrent processes.

Building on the perspective of viewing smart contracts as concurrent processes, Qu et al. [37] presented a method for modeling and verifying smart contracts using Communicating Sequential Processes (CSP) [38]. Their technique is used to verify that a smart contract is safe against particular attacks. In their methodology, smart contracts that are written in Solidity are translated into CSP. This step introduces a gap between the actual code and the model, as they do not verify the correctness of the translation.

Another related field of work is the development of smart contract languages based on process algebra. Rholang, based on the ρ-calculus, was developed as the native smart contract language for the RChain blockchain [39]. Then Bartoletti et al. [31] developed the process calculus BitML as a DSL for writing Bitcoin smart contracts. Rholang executes natively on the RChain blockchain, while BitML is compiled to Bitcoin transactions. Finally, another process calculus, Illum, was recently developed by Bartoletti et al. [40] as an intermediate language for compiling higher level languages into transactions on the Bitcoin and Cardano blockchains.

1.2.4 Related Work for Interactive Theorem Proving Using Dependent Session Types

Implementations of dependent session types are most closely related to the work presented in Chapter 5 and Chapter 6. In particular, the work by Ciccone and Padovani [41] presents a dependently typed π-calculus in Agda. Their system also includes a layer for a functional type system that is separate from the session type layer in the same way the system presented by Toninho et al. [42] does. However, it does not include the multiplicative operators, so context partitioning is not addressed in their work. Other implementations of dependent session types can be found in the RAST [43] and Granule [44] programming languages. RAST provides type level dependency with arithmetic refinements, and Granule provides type level dependency on natural number inputs.

Due to the Curry-Howard correspondence identified between linear logic and session types, previous work on interactive theorem proving for linear logic is also closely related to

the work presented in this dissertation. A system for interactive development of intuitionistic linear logic has been implemented in Isabelle by Kalvala and De Paiva [45], and a deep embedding of intuitionistic linear logic in Isabelle has been developed by Smola and Fleuriot [46]. Standalone provers have also been implemented for linear logic. These systems include the Janet theorem prover developed by Trifunovski [47], and the Click and Collect theorem prover by Callies and Laurent [48]. All of these systems have modified the inference rules in the system to allow for delayed context partitioning, except for the Click and Collect prover which requires the partition to be provided at the moment a multiplicative tactic is applied.

Also tangentially related to the STILL theorem prover is Idris 2 [49], which is built on Quantitative Type Theory [50, 51]. While it is not directly related to session types or linear logic, Idris 2 presents a dependently typed language and proof environment that must track resources as they are consumed. However, Quantitative Type Theory is not a theory of session types (there is no context partitioning for the $\otimes$ rule), and Idris 2 is not a tactic-based theorem prover, so the relation between STILL and Idris 2 is marginal.

Proof search systems for linear logic and type-checking implementations of session types are also related to our work on delayed context partitioning. Proof search systems include the work by Hodas and Miller [52] and Cervesato et al. [53] on efficient resource management for the Lolli prover. The work of Harland et al. [54] on the Lygon theorem prover also identifies the problem of finding the correct context partition for the multiplicative rules. Allais [55] presents a type-checker for a linear λ-calculus using leftovers. Zalakain and Dardha [56] use the same techniques to implement a type-checker for a π-calculus. The resource management calculus provided by Cervesato et al. [53], and the leftover type system presented by Allais [55] both result in systems that return the resources that were not consumed in one branch of the premises of a multiplicative rule. These are systems that behave similarly to the way the STILL prover hides resources; but the delayed partitioning is encoded in the calculus/type system, while the STILL prover delays the choice at the tactic level and validates the resulting partitions at the kernel level.

1.3 Dissertation Overview and Organization

The accomplishments of this dissertation are the following:

Marlowe semantics. A formal small-step operational semantics for the Marlowe smart contract programming language is presented in Chapter 2. This semantics is developed in Isabelle and verified against the preexisting Marlowe evaluation functions. The new formal semantics provide a cleaner semantic model for verification of the work presented in later chapters.

Faustus. This new smart contract programming language is presented in Chapter 3. Faustus is a smart contract programming language that uses operators from the Calculus of Communicating Systems (CCS) [25] to define the interactions between the smart contracts and the participants. The Faustus development includes a concrete semantics, a formal small-step operational semantics, a type-checker with verified soundness and progress properties, and a verified compiler to Marlowe.

Transformation verification. This dissertation describes a methodology for formalizing smart contracts as concurrent processes with a Labeled Transition System (LTS) that differentiates between observable and unobservable interactions in Chapter 4. Through this formalization, a common transformation that is applied to smart contracts is verified by demonstrating a weak bisimulation between an arbitrary original Faustus contract and the contract resulting from applying the transformation. The bisimulation is possible due to the addition of a restriction operator in the formal operational semantics that turns specific observable transitions into unobservable transitions.

STILL theorem prover. The STILL theorem prover for dependent session types is presented in Chapter 5. STILL is a tactic-based interactive theorem prover that implements dependent session types [42] with polymorphism [57] and coinduction [58] along with an implementation of the Extended Calculus of Constructions [59] for the functional system. The STILL prover also provides delayed linear context partitioning that

allows for more natural proof development where the user does not need to provide or prove the existence of a valid partition.

Formalized protocols. This dissertation presents the formalization of selected protocols in Chapter 6 using the STILL prover. The protocols demonstrate the features of the underlying type theory that STILL implements. The Σ-types in ECC allow for the communication of a proof separately from the witness. Polymorphic session types provide the ability to represent inductive session types. Finally, coinductive session types provide the ability to verify long-running processes with changing network topology.

Chapter 2

Formal Semantics for Marlowe

2.1 Contributions in This Chapter

In this chapter, we define a small-step semantics for the Marlowe smart contract programming language. Marlowe is a preexisting language with evaluators implemented in Isabelle and Haskell. The most significant part of the effort is a proof that the existing implementation of the Marlowe evaluator is correct with respect to the small-step semantics we define here. The agreement is proved separately for internal reductions, input processing, and timeout behavior. The functions `reduceContractStep`, `applyCases`, and `fixInterval` define the behavior of the evaluator.

$$\texttt{reduceContractStep} :: \texttt{Environment} \Rightarrow \texttt{State} \Rightarrow \texttt{Contract} \Rightarrow \texttt{ReduceResult}$$
$$\texttt{applyCases} :: \texttt{Environment} \Rightarrow \texttt{State} \Rightarrow \texttt{Input} \Rightarrow \texttt{Case list} \Rightarrow \texttt{ApplyResult}$$
$$\texttt{fixInterval} :: \texttt{TimeInterval} \Rightarrow \texttt{State} \Rightarrow \texttt{IntervalResult}$$

Our proof shows that the Marlowe evaluator agrees with the small-step semantics, in the sense that the three simulation diagrams in Figure 2.1 hold where $\mathcal{ID}$ is the identity relation on `Configuration`s. For the purposes of the diagrams, we use an overloaded operator $\uparrow$ that lifts a `ReduceResult`, an `ApplyResult`, or an `IntervalResult` to a `Configuration option`. In the case that one of these three evaluation functions fails, $\uparrow$ returns `None`. We use τ to denote internal reduction steps, λ to denote steps that process user input, and Δ to denote a time interval. The proofs that these simulation diagrams hold

$$
\begin{tikzcd}
M_1 \arrow[r, "\uparrow \texttt{reduceContractStep}"] & M_2 \\
\mathcal{ID} & \mathcal{ID} \\
S_1 \arrow[r, "\xrightarrow{\tau}_m"] & S_2
\end{tikzcd}
\tag{2.1}
$$

$$
\begin{tikzcd}
M_1 \arrow[r, "\uparrow \texttt{fixInterval}\,\Delta\ \texttt{>>=}\ \uparrow \texttt{applyCases}\,\lambda"] & M_2 \\
\mathcal{ID} & \mathcal{ID} \\
S_1 \arrow[r, "\xrightarrow{(\lambda,\Delta)}_m"] & S_2
\end{tikzcd}
\tag{2.2}
$$

$$
\begin{tikzcd}
M_1 \arrow[r, "\uparrow \texttt{fixInterval}\,\Delta\ \texttt{>>=}\ \uparrow \texttt{reduceContractStep}"] & M_2 \\
\mathcal{ID} & \mathcal{ID} \\
S_1 \arrow[r, "\xrightarrow{(\tau,\Delta)}_m"] & S_2
\end{tikzcd}
\tag{2.3}
$$

Figure 2.1: Simulation diagrams for verifying the Marlowe small-step semantics.

have been formalized in the Isabelle/HOL [60] theorem prover.

2.2 Marlowe

Marlowe is a deeply embedded domain-specific language (eDSL) for writing and analyzing financial contracts on the blockchain [1]. Early versions of Marlowe provided seven small `Contract` constructors that could be composed to form larger contracts. As the language strives for simplicity, the number of those constructors has been reduced to six in the most recent version of core Marlowe. We follow the version of Marlowe presented in [12], with the addition of `Assert` as in [61].

2.2.1 The Marlowe State

The blockchain provides a computing context where token transfers are a central concern. The behavior of programs on the blockchain may not be simple to intuit from the syntax alone, because evaluation depends on data on the ledger. For this reason, we introduce the

Marlowe *State*, which represents the datum associated with the contract's on-chain script address (together with the value locked at that address), before presenting the abstract syntax. The meaning of each *Contract* constructor will then be described by how it transforms the *State*. Listing 2.1 shows the type declaration for the Marlowe *State* along with additional type declarations for *Party*, *Token*, *ChoiceId*, *Accounts*, and *POSIXTime*:

Listing 2.1: Isabelle type declarations for the Marlowe *State*.

```
type_synonym POSIXTime = int

type_synonym PubKey = ByteString

type_synonym Ada = int
type_synonym CurrencySymbol = ByteString
type_synonym TokenName = ByteString

type_synonym ChoiceName = ByteString
type_synonym NumAccount = int
type_synonym Timeout = POSIXTime
type_synonym Money = Ada
type_synonym ChosenNum = int

datatype Party = PubKey ByteString
               | Role TokenName

type_synonym AccountId = Party
datatype Token = Token CurrencySymbol TokenName
datatype ChoiceId = ChoiceId ChoiceName Party
type_synonym Accounts = "((AccountId × Token) × Money) list"

record State = accounts :: Accounts
               choices :: "(ChoiceId × ChosenNum) list"
               boundValues :: "(ValueId × int) list"
               minTime :: POSIXTime
```

The *State* uses association lists as the internal structure for storing key–value mappings. For example, the *((Party × Token) × Money) list* type represents account balances for party–token pairs.

The *accounts* field of the *State* tracks the internal account balances for the contract.

Marlowe contracts receive deposits into the accounts and make payments from them to wallets on the blockchain. Money can also be transferred between internal Marlowe accounts. When a Marlowe contract is closed, all accounts are emptied, and the contents are sent to the wallets associated with the party.

The `choices` field of the `State` tracks choices that have been made earlier in the execution of the contract. Choices are made by parties interacting with the contract. A `ChoiceId` contains a string for the `ChoiceName` and the `Party` that can make the choice. This allows parties to make multiple choices, and for choices to be made by multiple parties. Choices are tracked so that they may be referenced later in the contract.

The `boundValues` field of the `State` tracks the variables that have been bound earlier during the execution of the contract. The only form of declaration available in Marlowe binds integer expressions, which are eagerly evaluated. Bound variables are tracked so that they may be referenced later in the contract.

The `minTime` field of the `State` tracks the minimum timestamp that could be used in the next evaluation step. As a Marlowe contract executes, each step is allowed to take place within a time interval. These intervals may overlap, but the `minTime` is used to trim overlaps to avoid the possibility of a contract moving backward in time. Internally, `POSIXTime` is a type synonym for `int`. The time interval is defined by the `Environment` record in Isabelle:

Listing 2.2: Isabelle type declarations for the Marlowe `Environment`.

```
type_synonym TimeInterval = "POSIXTime × POSIXTime"
record Environment = timeInterval :: TimeInterval
```

2.2.2 Marlowe Contract Structure

As described by Seijas et al. [12], Marlowe is a deeply embedded DSL. The DSL is intended to be host language and transaction model [62] agnostic, but is currently implemented in Haskell to interface with the Cardano blockchain. The `Contract` datatype defines the main building blocks of Marlowe programs. A fully defined Marlowe `Contract` is a tree structure constructed at run-time in the host language (Haskell or Isabelle) that Marlowe is embedded

in. Each branch in the *Contract* tree structure fully defines a possible execution path for the program, with no backtracking. The constructors for the mutually recursive *Case* and *Contract* datatypes can be seen in the following Isabelle definition:

Listing 2.3: Isabelle type declarations for the Marlowe *Contract* and *Case* datatypes.

```
datatype Case = Case Action Contract
and Contract = Close
             | Pay AccountId Payee Token Value Contract
             | If Observation Contract Contract
             | When "Case list" Timeout Contract
             | Let ValueId Value Contract
             | Assert Observation Contract
```

Note that contracts are presented in continuation passing style. The *Close* constructor refunds money to owners of all the accounts that still exist in the contract state. Since this is the end of a contract, there is no continuation for this constructor.

The *Pay* constructor sends money from the account with the given *AccountId* to the *Payee* which can be a participant or account in the contract. The currency and amount are given in the *Token* and *Value* parameters respectively. *Value*s are arithmetic operations that evaluate to integers. Additionally, *Value*s can reference choices made earlier by participants. Evaluation continues with the *Contract* argument.

The *If* constructor allows branching. *Observation*s are *Boolean* expressions in Marlowe, and (like *Value*s) *Observation*s can reference earlier choices. If the *Observation* is true at the time of execution, then the program continues with the first *Contract*. Otherwise the program continues with the second *Contract*.

The *When* constructor allows processing user input. Each *Case* in the list of *Case*s specifies the *Action* (user input) required to evaluate its continuation. Evaluation pauses at a *When* if there is no valid user input. The semantics of processing user input are discussed in Section 2.2.3.

The *Let* constructor binds a *Value* to a *ValueId* for later use. The program continues with the *Contract* argument, and uses the integer result of the eagerly evaluated *Value* where requested.

Finally, the *Assert* constructor evaluates an *Observation* and then continues with

the *Contract* argument. If the *Observation* evaluates to false, then the contract emits a warning indicating an assertion failed. The continuation *Contract* executes regardless of the assertion result.

2.2.3 Marlowe Input Processing

The behavior of a Marlowe contract may depend on input from parties involved in the contract. These parties are identified by their public-key identifiers or by a role token associated with a specific wallet on the blockchain. User input is formalized by Seijas et al. [12] as a list of transactions, each of which contains a list of inputs. The logic for applying the inputs is defined by the *applyCases* function. When evaluation of a contract reaches a *When* constructor, it pauses unless the next input is a valid choice, deposit, or notification, or unless the timeout has passed at the time the current transaction was submitted. If an input matches the *Action* of some *Case*, then evaluation continues with the corresponding *Contract*. If the timeout has been reached, then evaluation continues with the *Contract* argument in the *When* constructor. Users can submit three types of *Input* corresponding to three types of *Action*:

Listing 2.4: Isabelle type declarations for Marlowe *Action*s and *Input*s.

```
datatype Action = Deposit AccountId Party Token Value
                | Choice ChoiceId "Bound list"
                | Notify Observation
datatype Input = IDeposit AccountId Party Token Money
               | IChoice ChoiceId ChosenNum
               | INotify
```

The *IDeposit* input indicates the user is depositing a specified amount of a token into an internal account. On the blockchain, the contract receives the money and holds it in the specified account. The party in the deposit input is associated with the user making the input. Users may have multiple roles in the contract, so they indicate the role making the deposit in the input. The *Deposit* action specifies the expected account, party, token, and amount of a deposit. In order for an *IDeposit* input to match a *Deposit* action, the four arguments in the input must be equal to their respective arguments in the action at the time

the deposit input is submitted.

The *IChoice* input indicates the user is making a choice. The *ChoiceId* holds information about the party making the choice, and the specific choice being made (represented as a string). The supplied *ChosenNum* (an integer) is recorded and may be referenced later in the contract. The *Choice* action specifies the expected party and choice name, and it also specifies the acceptable bounds for the *ChosenNum*. In order for an *IChoice* input to match a *Choice* action, the *ChoiceId*s must be equal, and the *ChosenNum* supplied in the input must be within the bounds of the action.

The *INotify* input indicates that an *Observation* (boolean expression) in a *Notify* action evaluates to true. The first *Notify* action to evaluate to true in the *Case* list is the one that matches if multiple evaluate to true.

2.2.4 Marlowe Internal Reductions

The internal reductions — evaluation steps that do not consume user input — of Marlowe contracts are defined in [12] using an evaluator function called *reduceContractStep*, which performs one step of execution through the *Contract*. This function takes a time interval, contract state information, and a Marlowe contract, and returns the result of a single step of evaluation. The time interval represents a valid start and end time during which the evaluation takes place on the blockchain. The result type is shown in the following Isabelle datatype definitions:

Listing 2.5: Isabelle type declarations for Marlowe reduction results.

```
datatype Payment = Payment AccountId Payee Token Money
datatype ReduceEffect = ReduceNoPayment
                      | ReduceWithPayment Payment
datatype ReduceWarning = ReduceNoWarning
                       | ReduceNonPositivePay AccountId Payee Token Money
                       | ReducePartialPay AccountId Payee Token Money Money
                       | ReduceShadowing ValueId int int
                       | ReduceAssertionFailed
datatype ReduceStepResult = Reduced ReduceWarning ReduceEffect State Contract
                          | NotReduced
                          | AmbiguousTimeIntervalReductionError
```

A return value of `Reduced` indicates a successful reduction by `reduceContractStep`. The result includes the warning, payment, new state, and contract continuation information. The payment information indicates what internal account the payment is coming from, and where the payment is going. Payments can be sent to other internal accounts or to external wallets on the blockchain. The warning indicates whether a `Pay` contract attempts to pay a negative amount, or an amount greater than the money in the account. In the case of a negative payment, no money is sent or received. In the case of a partial payment, all of the money in the account is sent. In addition to the payment-related warnings, Marlowe emits warnings for variable shadowing and for failed assertions.

A return value of `NotReduced` indicates there is no evaluation that can be performed on the contract. This means the contract is either closed or waiting for user input at a `When` constructor. Additional user input is required to progress the contract if it is not closed in this configuration.

A return value of `AmbiguousTimeIntervalReductionError` means that a `When` timeout occurs within the current time interval. In this case, the evaluator cannot determine whether the `When` should time out or wait for additional user input. This error is intended to be propagated to the blockchain smart contract validator and will prevent the transaction from being accepted on the blockchain.

A function called `reductionLoop` repeatedly attempts internal reductions by calling `reduceContractStep` on its returned state and continuation until no more internal reductions are possible or an error occurs. If the contract does not error and has no more internal reductions, it is in a quiescent state. The function `reduceContractUntilQuiescent` is used throughout the rest of the description of the semantics, and is a wrapper for the initial call to `reductionLoop`.

2.2.5 Marlowe Transaction Processing

A transaction on the blockchain that executes a Marlowe contract includes an execution time interval and inputs for the contract to process. The time interval is trimmed according to the `minTime` in the state of the contract, and then all inputs are applied. After each input

is applied as described in Section 2.2.3, the contract performs internal reductions until it reaches a quiescent state as described in Section 2.2.4. This is repeated until all inputs are processed. If any error is returned, the transaction is rejected and no changes are applied to the contract.

Given a list of transactions, the execution trace of a Marlowe contract can be followed by applying each transaction to the result of the previous one. Traces are finite. Due to the finite structure and the semantics of Marlowe contracts, each contract has a maximum number of transactions that can be applied before it is guaranteed to close [12]. The maximum timeout on the `When` contracts inside the contract structure also defines the maximum amount of time that the contract will keep funds locked. After that time is reached, tokens will be returned to the corresponding parties according to the timeout logic of the contract.

2.3 Formalized Marlowe Semantics

This section describes the small-step semantics for Marlowe in the style of Nipkow and Klein [24], and a formal verification of the semantics against the Marlowe `reduceContractStep` and `applyCases` functions defined by Seijas et al. [12]. With these new small-step semantics, the techniques for formalizing the semantics of programming languages provided by Nipkow and Klein [24] can be followed in a step-by-step manner.

2.3.1 Small-Step Reduction Semantics

The small-step reduction semantics define valid transitions between two Marlowe `Configurations` using the $\rightarrow$ infix operator. A `Configuration` is a 5-tuple type that holds the current `Contract`, `State`, `Environment`, `ReduceWarning list`, and `Payment list` information of a Marlowe program. The `ReduceWarning list` is a list of warnings that have been issued throughout the execution of the contract, up to the current `Configuration`. Similarly, the Payment list is a list of payments made by the execution of the contract, up to the current `Configuration`. Note that these are the accumulated warnings and payments over multiple steps of evaluation, not just the ones emitted dur-

ing the current step. The inductive rules that define the Marlowe small-step semantics are presented in Listing 2.6.

Listing 2.6: Marlowe small-step semantics

```
inductive
  small_step_reduce :: "Configuration ⇒ Configuration ⇒ bool" (infix "→" 55)
where
CloseRefund:  "refundOne (accounts s) = Some ((party, token, money), newAccount) ⟹
  (Close, s, env, warns, payments) →
  (Close, (s⦇accounts := newAccount⦈), env, warns @ [ReduceNoWarning], payments @ [Payment party
       (SemanticsTypes.Party party) token money])" |
PayNonPositive: "evalValue env s val ≤ 0 ⟹
  (Pay accId payee token val cont, s, env, warns, payments) →
  (cont, s, env, warns @ [ReduceNonPositivePay accId payee token (evalValue env s val)], payments
       )" |
PayPositivePartialWithPayment: "⟦evalValue env s val > 0;
  evalValue env s val > moneyInAccount accId token (accounts s);
  updateMoneyInAccount accId token 0 (accounts s) = newAccs;
  moneyInAccount accId token (accounts s) = moneyToPay;
  giveMoney accId payee token (moneyToPay) newAccs = (payment, finalAccs);
  payment = ReduceWithPayment somePayment⟧ ⟹
  (Pay accId payee token val cont, s, env, warns, payments) →
  ((cont, s⦇accounts := finalAccs⦈), env, warns @ [ReducePartialPay accId payee token moneyToPay (
       evalValue env s val)], payments @ [somePayment]))" |
PayPositivePartialWithoutPayment: "⟦evalValue env s val > 0;
  evalValue env s val > moneyInAccount accId token (accounts s);
  updateMoneyInAccount accId token 0 (accounts s) = newAccs;
  moneyInAccount accId token (accounts s) = moneyToPay;
  giveMoney accId payee token (moneyToPay) newAccs = (payment, finalAccs);
  payment = ReduceNoPayment⟧ ⟹
  (Pay accId payee token val cont, s, env, warns, payments) →
  ((cont, s⦇accounts := finalAccs⦈), env, warns @ [ReducePartialPay accId payee token moneyToPay (
       evalValue env s val)], payments))" |
PayPositiveFullWithPayment: "⟦evalValue env s val > 0;
  evalValue env s val ≤ moneyInAccount accId token (accounts s);
  moneyInAccount accId token (accounts s) = moneyInAcc;
  moneyInAcc - (evalValue env s val) = newBalance;
  updateMoneyInAccount accId token newBalance (accounts s) = newAccs;
  giveMoney accId payee token (evalValue env s val) newAccs = (payment, finalAccs);
  payment = ReduceWithPayment somePayment⟧ ⟹
  (Pay accId payee token val cont, s, env, warns, payments) →
  (cont, s⦇accounts := finalAccs⦈), env, warns @ [ReduceNoWarning], payments @ [somePayment])" |
PayPositiveFullWithoutPayment: "⟦evalValue env s val > 0;
```

```
  evalValue env s val ≤ moneyInAccount accId token (accounts s);
  moneyInAccount accId token (accounts s) = moneyInAcc;
  moneyInAcc - (evalValue env s val) = newBalance;
  updateMoneyInAccount accId token newBalance (accounts s) = newAccs;
  giveMoney accId payee token (evalValue env s val) newAccs = (payment, finalAccs);
  payment = ReduceNoPayment⟧ ⟹
  (Pay accId payee token val cont, s, env, warns, payments) →
  (cont, s⦇accounts := finalAccs⦈, env, warns @ [ReduceNoWarning], payments)" |
IfTrue: "evalObservation env s obs ⟹
  (If obs cont1 cont2, s, env, warns, payments) →
  (cont1, s, env, warns @ [ReduceNoWarning], payments)" |
IfFalse: "¬evalObservation env s obs ⟹
  (If obs cont1 cont2, s, env, warns, payments) →
  (cont2, s, env, warns @ [ReduceNoWarning], payments)" |
WhenTimeout: "⟦timeInterval env = (startSlot, endSlot);
  endSlot ≥ timeout;
  startSlot ≥ timeout⟧ ⟹
  (When cases timeout cont, s, env, warns, payments) →
  (cont, s, env, warns @ [ReduceNoWarning], payments)" |
LetShadow: "lookup valId (boundValues s) = Some oldVal ⟹
  (Let valId val cont, s, env, warns, payments) →
  (cont, s⦇ boundValues := MList.insert valId (evalValue env s val) (boundValues s)⦈, env, warns
       @ [ReduceShadowing valId oldVal (evalValue env s val)], payments)" |
LetNoShadow: "lookup valId (boundValues s) = None ⟹
  (Let valId val cont, s, env, warns, payments) →
  (cont, s⦇ boundValues := MList.insert valId (evalValue env s val) (boundValues s)⦈, env, warns
       @ [ReduceNoWarning], payments)" |
AssertTrue: "evalObservation env s obs ⟹
  (Assert obs cont, s, env, warns, payments) →
  (cont, s, env, warns @ [ReduceNoWarning], payments)" |
AssertFalse: "¬evalObservation env s obs ⟹
  (Assert obs cont, s, env, warns, payments) →
  (cont, s, env, warns @ [ReduceAssertionFailed], payments)"
```

Each inductive rule represents a reduction rule for a Marlowe configuration. The rule begins with a name (*e.g.*, `CloseRefund`) and then is followed by the premises that must hold for the rule to be applied. The conclusion of the rule is a transition $\rightarrow$ between configurations. For example, the `PayNonPositive` rule states that if the value in the payment evaluates to a non-positive result, then the `Pay` contract reduces to its continuation contract with no changes to the state, payments, or environment, but a `ReduceNonPositivePay` warning is added to the list of warnings in the resulting configuration.

Note that none of the reduction rules modify the environment/time interval. This reflects the fact that time cannot change while the contract performs internal reductions within a transaction. Also, the `When` can only transition to its timeout continuation as a reduction without processing input. The next section describes how transitions into case continuations occur when processing user input.

2.3.2 Small-Step Input Processing Semantics

The small-step input processing semantics define valid transitions between two Marlowe `MConfig`s using the $\rightarrow_i$ operator[1]. The `MConfig` differs from the `Configuration` type by using a `TransactionWarning list` rather than a `ReduceWarning list`. This change is made because `applyCases` outputs an `ApplyWarning`, and the `applyAllLoop` evaluation function that applies all inputs converts that warning to a `TransactionWarning list` in the evaluator. Transitions are labeled with an optional input and a time interval. Labels represent transactions on the blockchain that advance the Marlowe contract. The inductive rules that define the semantics of input processing are built from inductive rules for applying an input to a single action, a single case, and a list of cases which are presented in Appendix A. From those rules, the small-step input processing semantics are defined in Listing 2.7 along with the type definition for `MConfig`.

There are two possible transitions that can occur when a label is processed. The first transition is valid when the time interval does not time out the current `When` contract, an input is provided, and the list of cases has a valid transition to a new contract–state pair when given that input. This corresponds to the input matching a `Case` in the `When` contract. The second transition is valid when no input is provided and the time interval times out the current `When` contract. This corresponds to the real-world time changing and the `When` contract timing out. Since the ledger state can only change as part of a transaction, the time interval without an input corresponds to submitting a transaction that provides only time information. Providing an input indicates that the input is intended to match a `Case` in the current `When`.

[1]The $\rightarrow_i$ operator is subscripted with an i to prevent Isabelle parsing errors that occur if $\rightarrow$ is overloaded.

Listing 2.7: Marlowe small-step semantics for processing input.

```
type_synonym MConfig = "Contract ⋆ State ⋆ Environment ⋆ (TransactionWarning list) ⋆ (Payment
     list)"
inductive small_step_input_marlowe :: "MConfig ⇒ InputLabel ⇒ MConfig ⇒ bool" ("_ →i _ _" [55,
      0, 55]) where
MLambdaReceive: "⟦
  fixInterval (lower, upper) state = IntervalTrimmed newTimeEnv newTimeState;
  timeInterval newTimeEnv = (realLower, realUpper);
  realLower < t;
  realUpper < t;
  inp ⊢ (cases, newTimeState, newTimeEnv) →casesm (newC, newState, warn);
  i = (Some inp, lower, upper) ⟧ ⟹
  (When cases t c, state, env, warnings, payments) →ii (newC, newState, newTimeEnv, warnings @ (
       convertApplyWarning warn), payments)" |
CCSTimeout: "⟦
  fixInterval (lower, upper) state = IntervalTrimmed newTimeEnv newTimeState;
  timeInterval newTimeEnv = (realLower, realUpper);
  realLower ≥ t;
  realUpper ≥ t;
  i = (None, lower, upper) ⟧ ⟹
  (When cases t c, state, env, warnings, payments) →ii (c, newTimeState, newTimeEnv, warnings,
       payments)"
```

2.3.3 The Full Marlowe Small-Step Semantics

So far we have defined two small-step semantics for Marlowe. One small-step semantics applies to internal reductions that do not process any user input. The other small-step semantics applies to reductions that process user input. These two semantics are now combined with a special label τ that denotes internal reduction steps, and with λ being a constructor that contains user input information. As part of this combination, an additional intermediate internal reduction relation denoted with $\rightarrow_m^{\mathcal{T}}$ has been defined that allows for the conversion of `ReduceWarning` to `TransactionWarning`.

These full semantics provide a single relation between Marlowe configurations that defines the behavior of Marlowe contracts. The labels on the transitions themselves indicate whether a step of computation is an internal reduction or an input processing step. We now proceed to verify these semantics against the two preexisting evaluation functions.

Listing 2.8: The Isabelle inductive definition of the full Marlowe semantics.

```
datatype FullSymbol = Tau ("τ")
  | MLambda InputLabel ("λ_")
type_synonym TransitionRelation = "MConfig ⇒ FullSymbol ⇒ MConfig ⇒ bool"
inductive internal_reaction :: "MConfig ⇒ MConfig ⇒ bool" ("_ →m𝒯 _" [55, 55]) where
SmallStepReduce: "(c, s, env, [], payments) → (newC, newS, newEnv, newWarnings, newPayments) ⟹
  (c, s, env, warnings, payments) →m𝒯 (newC, newS, newEnv, warnings @ (convertReduceWarnings
       newWarnings), newPayments)"
inductive full_semantics_marlowe :: "TransitionRelation" ("_ →m_ _" [55, 0, 55]) where
InternalFull: "p1 →m𝒯 p2 ⟹ full_semantics_marlowe p1 τ p2" |
ReceiveFull: "⟦
  p1 →i (Some i, lower, upper) p2⟧ ⟹
  full_semantics_marlowe p1 (λ(Some i, lower, upper)) p2" |
TimeoutFull: "⟦p1 →i (None, lower, upper) p2⟧ ⟹ full_semantics_marlowe p1 (λ(None, lower, upper
     )) p2"
```

2.3.4 Verifying the Small-Step Reduction Semantics

The *reduceContractStep* function has been verified to agree with the newly presented small-step reduction semantics. In other words, it has been shown that there is a valid small-step transition if and only if the Marlowe evaluator returns a *Reduced* result. The other evaluator results either indicate quiescence or signal an error. They do not advance the contract by an internal reduction step.

The lemma in Listing 2.9 proves that if the Marlowe evaluator gives a *Reduced* result, either with or without a payment output, there is a valid small-step transition in the new small-step semantics. The *convertReduceEffect* function converts a reduce effect to the empty list if there is no payment or a singleton list with the payment if one exists. The *convertReduceWarnings* function returns the list of warnings converted to their *TransactionWarning* counterparts with *ReduceNoWarning* filtered out.

The proof for this lemma is done by case analysis on the *Contract* argument to *reduceContractStep* (called *contract*).

The next lemma in Listing 2.10 proves that if there is a valid small-step transition in our small-step semantics for Marlowe, then the Marlowe evaluator returns the corresponding *Reduced* result.

The proof for this lemma is done by case analysis on the *contract* variable.

Listing 2.9: A *Reduced* result from the *reduceContractStep* function implies a step in the Marlowe small-step semantics.

```
lemma reduceContractStepImpSmallStepReduction:
assumes "reduceContractStep env state contract = Reduced warning paymentReduceResult newState
     continueContract"
shows "(contract, state, env, initialWarnings, initialPayments) →mτ (continueContract, newState,
      env, initialWarnings @ convertReduceWarnings [warning], initialPayments @
     convertReduceEffect paymentReduceResult)"
```

Listing 2.10: A step in the Marlowe small-step semantics implies a corresponding *Reduced* result.

```
lemma smallStepReductionImpReduceContractStep:
assumes "(contract, state, env, initialWarnings, initialPayments) →mτ (continueContract,
     newState, newEnv, newWarnings, newPayments)"
shows "∃ reduceWarning paymentReduceResult . reduceContractStep env state contract = Reduced
     reduceWarning paymentReduceResult newState continueContract ∧
      initialWarnings @ convertReduceWarnings [reduceWarning] = newWarnings ∧
      initialPayments @ convertReduceEffect paymentReduceResult = newPayments"
```

Listing 2.6 presents the small-step reduction semantics for Marlowe, and the lemmas in this section proved that there is a valid small-step transition if and only if the Marlowe evaluator gives a *Reduced* result. This shows simulation diagram 2.1 holds in Figure 2.1.

2.3.5 Verifying the Small-Step Input Processing Semantics

The small-step semantics defined in Listing 2.7 correspond to the *applyCases* function, which processes inputs on a paused *When* contract, when the optional input is included. Otherwise, they correspond to the *reduceContractStep* advancing to the timeout continuation. There are three lemmas that are required to prove this correspondence. The first lemma in Listing 2.11 shows that the *applyCases* function returning an *Applied* result implies there is a corresponding step in the Marlowe small-step input processing semantics. The *applyCases* function assumes a valid time interval, while the input processing semantics do not, so the lemma assumes that the time interval has been correctly trimmed before *applyCases* is called.

Listing 2.11: An *Applied* result from *applyCases* implies a step in the Marlowe small-step semantics for input processing.

```
lemma applyCasesImpliesMarloweSmallStep:
  assumes "fixInterval (lower, upper) state1 = IntervalTrimmed newTimeEnv1 newTimeState1 ∧
    timeInterval newTimeEnv1 = (realLower, realUpper) ∧
    realLower < t ∧
    realUpper < t ∧
    applyCases newTimeEnv1 newTimeState1 input cases = Applied warning newState newCont"
  shows "(When cases t c, state1, env, warnings, payments) →mλ(Some input, lower, upper) (
      newCont, newState, newTimeEnv1, warnings @ (convertApplyWarning warning), payments)"
```

The next two lemmas verify the two cases of the Marlowe small-step input processing semantics. The first verifies that if an input is received, then a step implies the *applyCases* function will return the corresponding *Applied* result. The second verifies that a step without an input implies the *reduceContractStep* function returns a *Reduced* result corresponding to the timeout logic.

Listing 2.12: A step in the Marlowe input processing semantics with an input implies an *Applied* result from the *applyCases* function.

```
lemma marloweSmallStepImpliesApplyCases:
  assumes "((When cases t c, state1, env, warnings, payments) →mλ(Some input, lower, upper) (
      newCont, newState, newTimeEnv1, warnings', payments)) ∧
    fixInterval (lower, upper) state1 = IntervalTrimmed newTimeEnv1 newTimeState1"
  shows "∃warning . applyCases newTimeEnv1 newTimeState1 input cases = Applied warning newState
      newCont ∧ warnings' = warnings @ (convertApplyWarning warning)"
```

Listing 2.13: A step in the Marlowe input processing semantics without an input implies a *Reduced* result from the *reduceContractStep* function.

```
lemma marloweSmallStepTimeoutImpliesReduceStep:
  assumes "((When cases t c, state1, env, warnings, payments) →mλ(None, lower, upper) (newCont,
      newState, newTimeEnv1, warnings, payments)) ∧
    fixInterval (lower, upper) state1 = IntervalTrimmed newTimeEnv1 newTimeState1"
  shows "reduceContractStep newTimeEnv1 newTimeState1 (When cases t c) = Reduced ReduceNoWarning
      ReduceNoPayment newState newCont"
```

With these three lemmas, the Marlowe small-step semantics for input processing are shown to correspond to the preexisting *applyCases* and *reduceContractStep* functions. Together, these lemmas show that the simulation diagrams 2.2 and 2.3 in Figure 2.1

hold.

2.3.6 Meta-Properties of the Small-Step Semantics

The Marlowe evaluation functions already have the following meta-properties proved [1, 12]:

- Account balances in the contract are always greater than zero, and are deleted when they reach zero.
- A Marlowe contract correctly preserves the amount of money in the contract.
- The `reduceContractStep` function only returns `NotReduced` in a quiescent state.
- The maximum time left in the contract always decreases.

The small-step semantics maintain these meta-properties due to the correspondence proved between the `reduceContractStep` and `applyCases` evaluation functions and the small-step semantics. Two additional properties are provided to strengthen the correspondence between the small-step semantics and the other evaluation functions. First, we prove that the small-step semantics are deterministic. This is done with one lemma showing the small-step reduction semantics are deterministic and another lemma showing the small-step input processing semantics are deterministic (Listing 2.14). Second, we define a relation that performs all internal reductions $\Downarrow_m$ and show it corresponds to the `reductionLoop` evaluation function. Specifically, we prove that if the `reductionLoop` reaches a quiescent state, then the reflexive transitive closure of the small-step reduction relation can reach the same state (Listing 2.15).

Listing 2.14: Isabelle lemmas proving the Marlowe small-step semantics are deterministic.

```
lemma deterministicMarloweReduction:
  "cs → cs' ⟹ cs → cs'' ⟹ cs'' = cs'"

lemma deterministicSmallStepInput:
"f →ᵢμ f' ⟹
  f →ᵢμ f'' ⟹
  f' = f''"
```

Listing 2.15: An Isabelle lemma proving that *reductionLoop* corresponds to the reflexive transitive closure of the Marlowe small-step reduction semantics.

```
inductive small_step_reduce_all_marlowe :: "Configuration ⇒ Configuration ⇒ bool" (infix  "⇓m"
     55) where
FinalStep: "⟦f1 → f2; finalMarlowe f2⟧ ⟹ f1 ⇓m f2" |
AddStep: "⟦f1 → f2; f2 ⇓m f3⟧ ⟹ f1 ⇓m f3"
lemma reduceAllWarningsPaymentsMatchReductionLoop:
"(contract, state, env, initialWarnings, initialPayments) ⇓m (continueContract, newState, newEnv,
      newWarnings, newPayments) ⟹
  reductionLoop reduced env state contract initialWarningsEval initialPaymentsEval =
       ContractQuiescent True newWarningsEval newPaymentsEval newState continueContract ⟹
  (∃addedWarnings addedPayments . newWarnings = initialWarnings @ addedWarnings ∧
    newPayments = initialPayments @ addedPayments ∧
    newWarningsEval = (rev initialWarningsEval) @ convertSemanticsWarningsToEvalWarnings
         addedWarnings ∧
    newPaymentsEval = (rev initialPaymentsEval) @ addedPayments)"
lemma reductionLoopIsAllSmallStepReductions:
"reductionLoop reducedInit env state contract initialWarningsEval initialPaymentsEval =
     ContractQuiescent reducedRes newWarningsEval newPaymentsEval newState continueContract ⟶
  (∃newWarnings newPayments . (contract, state, env, warnings, payments) ⇓m (continueContract,
       newState, env, newWarnings, newPayments) ∨ finalMarlowe (contract, state, env, warnings,
       payments))"
```

The purpose of proving these properties is to ensure that the small-step semantics do not permit behaviors that are absent from the preexisting Marlowe evaluation functions. The correspondences proved with the *reduceContractStep* and *applyCases* functions prove single-step behaviors are preserved, but additional behaviors arising from multi-step evaluation functions must also be accounted for.

We also note that, at a qualitative level, the Isabelle system user experience is enhanced through the use of inductive relations rather than semantics defined by evaluation functions. The *reductionLoop* function applies the *reduceContractStep* function multiple times, but using the inductive relation that Isabelle automatically derives does not provide the base case as a subgoal. The base case must be constructed by the user by applying case analysis to the result of the first call to *reduceContractStep*. Additionally, applying the "auto" or "simp" tactic may erroneously unfold the inner calls to *reductionLoop*. In such cases, the user must identify the issue, and use the *simp del:*

`reductionLoop.simps` option when applying the tactics. The $\Downarrow_m$ relation avoids these issues by providing an explicit base case where a single step to a final configuration occurs. This allows Isabelle to automatically split the base case and the inductive step when attempting a proof by induction.

2.4 Discussion

The small-step semantics presented here provide semantics for Marlowe contracts that are not tied to an evaluation function. Since the new small-step semantics are correct and adequate with respect to the preexisting evaluation functions, they can serve as a reference to future evaluator implementations. Further work is required to verify the small-step semantics against the Marlowe transaction processing functions. This will require defining a relation between Marlowe configurations over lists of inputs, similar to the experiment relation defined for processes in the Calculus of Communicating Systems, as discussed in Chapter 4. This dissertation uses the Marlowe small-step semantics from this chapter to verify the correctness of the Faustus compiler in Chapter 3.

Chapter 3

Faustus

3.1 Motivations

Marlowe programs frequently contain duplicate code that would benefit from `Contract` abstractions. For an example of this issue, we will examine a simple escrow contract written in Marlowe that can be found in Figure 3.1. In the escrow contract, "alice" deposits 450 tokens into their account. Then "alice" or "bob" can choose to attempt to claim the funds by making the "claim" choice. Finally, "carol" chooses to honor the withdrawal by sending 1 for either the "agree" or the "disagree" choice.

In this contract there are two pieces of similar code that would benefit from abstraction. Whenever "alice" or "bob" have made a claim to the money after it is deposited, "carol" will choose to agree or disagree with the claimant, and the money is sent to the claimant or nonclaimant based on whether "carol" agrees or disagrees. If a programmer needs to make a change to the program that targets the general behavior of processing a claim, they must remember to modify two parts of the code. We will show that the abstractions in Faustus presented in this chapter will remove the need to duplicate the logic for processing a claim.

```
When [
  Case (Deposit (Role "alice") (Role "alice") (Token "" "") (Constant 450))
    (When [
      Case (Choice (ChoiceId "claim" (Role "bob")) [Bound 1 1])
        (When [
          Case (Choice (ChoiceId "agree" (Role "carol")) [Bound 1 1])
            (Pay (Role "alice") (Party (Role "bob")) (Token "" "") (Constant 450) Close),
          Case (Choice (ChoiceId "disagree" (Role "carol")) [Bound 1 1])
            (Pay (Role "alice") (Party (Role "alice")) (Token "" "") (Constant 450) Close)
        ] 100 Close),
      Case (Choice (ChoiceId "claim" (Role "alice")) [Bound 1 1])
        (When [
          Case (Choice (ChoiceId "agree" (Role "carol")) [Bound 1 1])
            (Pay (Role "alice") (Party (Role "alice")) (Token "" "") (Constant 450) Close),
          Case (Choice (ChoiceId "disagree" (Role "carol")) [Bound 1 1])
            (Pay (Role "alice") (Party (Role "bob")) (Token "" "") (Constant 450) Close)
        ] 100 Close)
    ] 100 Close)
] 90 Close
```

Figure 3.1: Marlowe simple escrow contract.

3.2 Contributions in This Chapter

In this chapter, we present the Faustus smart contract programming language. Faustus compiles to the embedded domain specific language (eDSL) Marlowe [1, 12, 13, 63]. Marlowe code runs on the Cardano blockchain [11]. Marlowe was designed to support static analysis of smart contracts; Faustus is designed to make writing smart contracts significantly easier than it is to write Marlowe.

The main purpose of Faustus is to extend the Marlowe smart contract programming language with features that allow programmers to write code that is easier to understand and maintain. The next goal of Faustus is to preserve the useful termination and liquidity guarantees that exist for Marlowe contracts [12]. To these ends, Faustus contains parameterized abstractions of the Contract type that can be used elsewhere in the program. Faustus also contains sequencing, choice, and interleaving operators inspired by Milner's Calculus of Communicating Systems (CCS) [25] for expressing the common combinatorial interaction patterns. The expressions using these operators are added to the `Case` constructor in the place of the `Action`. Additionally, Faustus programs can be compiled into Marlowe programs that preserve the behavior of the original Faustus program, thus the guarantees of liquidity and termination of Marlowe programs are also enjoyed by Faustus programs.

The compiler verification is done in multiple stages. First, we verify a normalization

step from the full version of Faustus with guard expressions in the cases to a version of Faustus without guard expressions. This normalization step is represented in the simulation diagrams using the $\mathcal{N}$ relation between Faustus configurations $\mathcal{F}_i$ and intermediate Faustus configurations $\mathcal{I}_i$. Second, we verify a compiler from this intermediate version of Faustus to Marlowe. This compilation step is represented using the $\mathcal{C}$ relation between intermediate Faustus configurations and Marlowe configurations $\mathcal{M}_i$. Each level of verification is performed using the input processing semantics $\xrightarrow{\lambda}$ and the reflexive closure of the $\Downarrow$ relation from Section 2.3.6 that applies all available $\xrightarrow{\tau}$ steps. The proofs show that the simulation diagrams in Figure 3.2 hold[1].

$$
\begin{array}{ccc}
\mathcal{F}_1 & \xrightarrow{\lambda} & \mathcal{F}_2 \\
\mathcal{N} & & \mathcal{N} \\
\mathcal{I}_1 & \xrightarrow{\lambda}_{fir} & \mathcal{I}_2 \\
\mathcal{C} & & \mathcal{C} \\
\mathcal{M}_1 & \xrightarrow{\lambda}_{m} & \mathcal{M}_2
\end{array}
\qquad
\begin{array}{ccc}
\mathcal{F}_1 & \Downarrow^{=} & \mathcal{F}_2 \\
\mathcal{N} & & \mathcal{N} \\
\mathcal{I}_1 & \Downarrow^{=}_{fir} & \mathcal{I}_2 \\
\mathcal{C} & & \mathcal{C} \\
\mathcal{M}_1 & \Downarrow^{=}_{m} & \mathcal{M}_2
\end{array}
$$

Figure 3.2: The simulation diagrams for correctness of the Faustus compiler.

3.3 Formalized Faustus Semantics

The Faustus abstract syntax is defined as an extension to Marlowe. In addition to the Marlowe `Value` abstractions, Faustus includes abstractions for the types `Observation`, `Party`, and `Contract`. Additionally, `Contract` abstractions may be parameterized by any number of `Value`, `Party`, and `Observation` arguments. The type-checker verifies that calls to a parameterized abstraction include the correct number and types of arguments.

We follow the techniques of Schmidt [64] to add these new abstractions. An `FConfiguration` is an extension of the Marlowe `Configuration` that adds the `FContext` for tracking declaration information. It is the 6-tuple:

[1]Here we use λ to denote an observable transition, and we use $\Downarrow^{=}$ to indicate the reflexive closure of the $\Downarrow$ relation between configurations that applies all internal steps if they are available.

(*FContract*, *FContext*, *FState*, *Environment*, *ReduceWarning list*, *Payment list*)

The new *FContext* and *AbstractionInformation* types are defined as follows:

Listing 3.1: Isabelle type declarations for the Faustus *AbstractionInformation* and *FContext* types.

```
datatype AbstractionInformation = ValueAbstraction Identifier
  | ObservationAbstraction Identifier
  | PartyAbstraction Identifier
  | ContractAbstraction "Identifier × FParameter list × FContract"
type_synonym FContext = "AbstractionInformation list"
```

The *FContext* type is a list of *AbstractionInformation*; it corresponds to the environment that holds identifier-meaning information from Schmidt [64]. *Abstraction-Information* holds an identifier-meaning pair. Since identifiers are directly mapped to values in the *FState*, the *FContext* only holds *Identifiers* for bound *Value*, *Observation*, and *Party* entries. For *Contract* abstractions, the *FContext* holds parameter information and the body of the *Contract* that is bound to an *Identifier*.

In this section, when modifications are made to a language construct that result in a new datatype definition, that new datatype will replace the old datatype in other language constructs as needed. For example, the *FParty* definition is created, and then all Marlowe *Party* parameters are replaced with *FParty* parameters in Marlowe language constructs.

3.3.1 Adding Abstractions

The *Contract* definition is extended to include support for *Party*, *Observation*, and *Contract* declarations. Additionally, a new syntactic category of statements is introduced that contains the syntactical constructs for reassigning declared variables, and also the `Pay` and `Assert` statements found in the Marlowe `Contract` datatype. Statements will also be used in the new *Case* guard expressions described in Section 3.3.2. These changes result in new datatypes called *FContract*, *FCase*, *FStatement*, and *FParameter* which can be found in Listing 3.2.

For the Faustus state record, *FState*, a new *boundParties* field is added that holds the *Identifier* × *Party* pair. This list is used like a map or a dictionary, though for

now, reasoning about lists is better supported in Isabelle. Since *Observation* constructs are mutually recursive with *Value* constructs, the bound *Observation* information is stored in the *boundValues* field that already exists in the Marlowe *State* record. The new *FState* record can be found in Listing 3.3.

Finally, the ability to invoke previously bound identifiers in the *Party*, *Observation*, and *Contract* definitions is added. This results in the *FContract* datatype from Listing 3.2, and the creation of the new *FParty*, *FObservation*, *FValue*, and *FArgument* datatypes in Listing 3.4.

Listing 3.2: Isabelle type declarations for the Faustus *FStatement*s and *FContract*s.

```
datatype FParameter = ValueParameter Identifier
  | ObservationParameter Identifier
  | PartyParameter Identifier
datatype FStatement = Pay FAccountId FPayee Token FValue
  | Assert FObservation
  | ReassignVal Identifier FValue
  | ReassignObservation Identifier FObservation
  | ReassignParty Identifier FParty
datatype FContract = Close
             | StatementCont FStatement FContract
             | If FObservation FContract FContract
             | When "FCase list" Timeout FContract
             | Let Identifier FValue FContract
             | LetObservation Identifier FObservation FContract
             | LetParty Identifier FParty FContract
             | LetC Identifier "FParameter list" FContract FContract
             | UseC Identifier "FArgument list"
and FCase = Case FGuardExpression FContract
```

Listing 3.3: Isabelle record declaration for the *FState*.

```
record FState = accounts :: Accounts
  choices :: "(ChoiceId × ChosenNum) list"
  boundValues :: "(Identifier × int) list"
  boundParties :: "(Identifier × SemanticsTypes.Party) list"
  minTime :: POSIXTime
```

Listing 3.4: Isabelle type declarations for the Faustus *FParty*, *FObservation*, and *FArgument* constructs.

```
datatype FParty = ConstantPubKey ByteString
  | UseParty Identifier
  | Role TokenName
datatype FValue = AvailableMoney FAccountId Token
               | Constant int
               | NegValue FValue
               | AddValue FValue FValue
               | SubValue FValue FValue
               | MulValue FValue FValue
               | DivValue FValue FValue
               | ChoiceValue FChoiceId
               | TimeIntervalStart
               | TimeIntervalEnd
               | UseValue Identifier
               | Cond FObservation FValue FValue
and FObservation = AndObs FObservation FObservation
                     | OrObs FObservation FObservation
                     | NotObs FObservation
                     | ChoseSomething FChoiceId
                     | ValueGE FValue FValue
                     | ValueGT FValue FValue
                     | ValueLT FValue FValue
                     | ValueLE FValue FValue
                     | ValueEQ FValue FValue
                     | UseObservation Identifier
                     | TrueObs
                     | FalseObs
datatype FArgument = ValueArgument FValue
  | ObservationArgument FObservation
  | PartyArgument FParty
```

When the declaration of a *Party* (or an *Observation*) is encountered in the execution of a program, the body of the abstraction is eagerly evaluated and the identifier and result pair is stored in the *FState* for later use. The eager evaluation of these primitive types matches the behavior of the *Value* abstractions. Making *Observation* declarations behave lazily would give two different evaluation strategies to mutually defined constructs, and allow for seemingly impossible declarations that are demonstrated in Figure 3.3. To simplify verification of the compiler (in Section 3.5) *Observation* (Boolean) values are

```
// assume start = 0
var a = start < 10 ? 2 : 0;   // eagerly evaluated value set to 2
var b = start < 10;           // lazily evaluated observation
when {}
after 11 -> {                 // start is now > 10
    if b                      // b evaluates to false here
    then assert a == 2; close
    else assert a == 0; close // So the assert fails!
}
```

Figure 3.3: Faustus contract snippet demonstrating issue that occurs by allowing lazy `Observation` abstractions with eager `Value` abstractions.

stored as integer values using the existing `boundValue` field of the `State`. This is done by evaluating an `Observation` and storing the constant 1 for `true` and 0 for `false`.

A `Contract` declaration is lazily evaluated to allow for invocation of the body as a procedure. Eager evaluation would not make sense, because a `Close` contract inside the body of a declaration would close the contract if it were evaluated at the point of declaration, and the rest of the contract would not be executed. When a `Contract` declaration is encountered it stores the `Parameter` information and body at the left end (front) of the `FContext` list. Since the `FContext` is a list, only items in the list occurring to the right of a `Contract` binding can be used to resolve identifiers to their values when evaluating the body of that `Contract`.

For the invocation of abstractions, when a bound `Party` (or `Observation`) is used, the `Identifier` is looked up in `boundParties` (or `boundValues`). That value is then used in the evaluation of the program. This follows the implementation of Marlowe `Value` abstractions.

A bound contract is invoked via abstract syntax of the form `(UseC` cid `[`$e_1, \cdots, e_k$`])`. To evaluate this contract, identifier cid is looked up in the current `FContext`, yielding a triple of the form $(cid, [(ty_1, id_1), \cdots, (ty_k, id_k)], c)$. Let $fctx$ be the tail of the remainder of the current context obtained by dropping all entries up until the one with cid is found. The program is well-typed, so it is known that $e_1 : ty_1, \cdots, e_k : ty_k$, *i.e.*, each expression $e_1, \cdots, e_k$ is well-typed. A new `FContext` is constructed (call it $fctx'$) by starting with $fctx$,

```
var a = 5; // boundValues = [(a,5)]
contract b()
{
    @"alice" pays !bob (a, ""); close // Expect a to be 5 here.
}
var a = true; // boundValues = [(a,1)]
b(); // a is now 1, so alice will pay 1
```

Figure 3.4: Faustus contract demonstrating shadowing issue.

iteratively evaluating each argument ($e_i \mapsto v_i$) in the argument list and adding the entry (id_i, v_i) to $fctx'$. The body of the invoked contract c is evaluated using the new context $fctx'$. This mechanism prevents recursion (which must be avoided to compile to Marlowe) and enforces lexical scoping. Only names that were bound before the `Contract` *cid* was declared will be available in the $fctx'$ when evaluating the body.

Since `Observation` and `Value` abstractions are stored in the same field of the `FState`, the type-checker prevents variable shadowing. The example contract snippet in Figure 3.4 demonstrates the issue that occurs if shadowing is allowed. Since `Observation` abstractions need to use runtime information, the only place they can be stored after eager evaluation is in the `boundValues` field of the `FState`. Otherwise Faustus contracts could not be compiled to Marlowe contracts. We believe that treating `boundValues` as a store, and mapping variable names to store locations similar to the way Schmidt describes variable declarations [64] will allow for variable shadowing without unintended side effects. The formalization of such a change is outside the scope of this work, but a prototype implementation can be found in the Faustus repository [65].

3.3.2 Concurrency Operators for Interaction

Marlowe programs are fully expanded instances of a contract abstract syntax tree. This causes even simple contracts to become very large. One cause of the blow-up in contract size is accounting for sequences of user interaction. Marlowe requires branches for all possible orders of the interactions.

To address this verbosity, Faustus includes the sequencing (`->`), choice (`<+>`), and

interleaving (<⋆>) operators from Milner's CCS [25]. With these operators, user interaction can be described in a non-linear fashion. The sequencing operator allows the programmer to define actions that happen before and after one another. The choice operator allows the programmer to specify that only one of the action sequences must happen. The interleaving operator allows the programmer to specify that the valid sequences of actions in the two expressions on either side can occur in any order with respect to the sequences in the other expression. Guarded commands also allow the evaluation of an *FStatement list* after a guard has been evaluated. In the abstract syntax, these operators are added to the `Case` datatype as *FGuardExpression* in the place of the *Action* to result in the new `FCase` datatype (Listing 3.2). The abstract syntax for the guard expressions is defined as follows:

Listing 3.5: The abstract syntax for the Faustus *FGuardExpression*.

```
datatype FGuardExpression = ActionGuard FAction
  | GuardThenGuard FGuardExpression FGuardExpression
  | GuardStmtsGuard FGuardExpression "FStatement list"
  | DisjointGuard FGuardExpression FGuardExpression
  | InterleavedGuard FGuardExpression FGuardExpression
```

3.3.3 Faustus Internal Reduction Semantics

The Faustus small-step semantics for internal reductions are denoted with the $\rightarrow_{f2}$ operator. The full rules can be derived starting from the Marlowe small-step semantics for internal reductions (Section 2.3.1) through the following steps:

1. Rules for contract constructors that have been moved to the *FStatement* datatype are wrapped by the *StatementCont* constructor.

2. Add additional premises to all rules to evaluate any *FParty* to some concrete *Party* that is used in the evaluation of the rest of the rule (*e.g.* the reduction rules for a *Pay* statement evaluate the *FParty* before determining where the payment comes from and where it will be sent).

3. Add the rules for declaration, reassignment, and evaluation of the new abstractions that can be found in Listing 3.6.

Readers interested in the full definition of the Faustus small-step semantics can find them in Appendix C.

Listing 3.6: Isabelle inductive rules added to the Marlowe internal reduction semantics to create the Faustus internal reductions.

```
ReassignValShadow: "⟦lookup valId (boundValues s) = Some oldVal; evalFValue env s val = Some res⟧
      ⟹
  (StatementCont (ReassignVal valId val) cont, ctx, s, env, warns, payments) →f2
  (cont, ValueAbstraction valId#ctx, s⦇ boundValues := MList.insert valId res (boundValues s)⦈,
       env, warns @ [ReduceShadowing valId oldVal res], payments)" |
ReassignValNoShadow: "⟦lookup valId (boundValues s) = None; evalFValue env s val = Some res⟧ ⟹
  (StatementCont (ReassignVal valId val) cont, ctx, s, env, warns, payments) →f2
  (cont, ValueAbstraction valId#ctx, s⦇ boundValues := MList.insert valId res (boundValues s)⦈,
       env, warns @ [ReduceNoWarning], payments)" |
LetObservation: "⟦evalFObservation e s obs = Some res⟧ ⟹
  (LetObservation oId obs c, cx, s, e, ws, ps) →f2
  (c, ObservationAbstraction oId#cx, s⦇boundValues := MList.insert oId (if res then 1 else 0) (
       boundValues s)⦈, e, ws @ [ReduceNoWarning], ps)" |
ReassignObservation: "⟦evalFObservation e s obs = Some res⟧ ⟹
  (StatementCont (ReassignObservation oId obs) c, cx, s, e, ws, ps) →f2
  (c, ObservationAbstraction oId#cx, s⦇boundValues := MList.insert oId (if res then 1 else 0) (
       boundValues s)⦈, e, ws @ [ReduceNoWarning], ps)" |
LetParty: "⟦evalFParty s pk = Some res⟧ ⟹
  (LetParty pId pk c, cx, s, e, ws, ps) →f2
  (c, PartyAbstraction pId#cx, s⦇ boundParties := MList.insert pId res (boundParties s)⦈, e, ws @
        [ReduceNoWarning], ps)" |
ReassignParty: "⟦evalFParty s pk = Some res⟧ ⟹
  (StatementCont (ReassignParty pId pk) c, cx, s, e, ws, ps) →f2
  (c, PartyAbstraction pId#cx, s⦇ boundParties := MList.insert pId res (boundParties s)⦈, e, ws @
        [ReduceNoWarning], ps)" |
LetC: "(LetC cid params boundCon cont, ctx, s, env, warns, payments) →f2
  (cont, ContractAbstraction (cid, params, boundCon)#ctx, s, env, warns @ [ReduceNoWarning],
       payments)" |
UseCFound: "⟦lookupContractIdAbsInformation ctx cid = Some (params, bodyCont, innerCtx);
  evalFArguments env s params args = Some newState⟧ ⟹
  (UseC cid args, ctx, s, env, warns, payments) →f2
  (bodyCont, (paramsToFContext params)@innerCtx, newState, env, warns @ [ReduceNoWarning],
       payments)"
```

3.3.4 Faustus Input Processing Semantics

We begin describing the Faustus input processing semantics with the semantics for a single *FGuardExpression*. These rules will then be used to describe the input processing semantics for a *FCase list*, and finally a small-step input processing semantics between Faustus configurations labeled by an input in the same manner as the Marlowe small-step input processing semantics in Section 2.2.3. The semantics for a single *FGuardExpression* are given by the following Isabelle inductive relation:

Listing 3.7: The reduction semantics for a single *FGuardExpression* over an input.

```
type_synonym FGuardConfig = "FGuardExpression * FState * Environment"
type_synonym FGuardConfigRes = "(FStatement list * FGuardExpression option * FState *
     ApplyWarning)"
inductive small_step_reduce_guard :: "Input ⇒ FGuardConfig ⇒ FGuardConfigRes ⇒ bool" ("_ ⊢ _
     →g_f2 _" [55, 0, 55]) where
ActionGuardStep:  "⟦g = ActionGuard a;
  ins ⊢ (a, s, env) →a_f2 newS⟧ ⟹
  ins ⊢ (g, s, env) →g_f2 ([], None, newS)" |
GuardThenGuardCompleteFirstGuardStep:  "⟦g = GuardThenGuard g1 g2;
  ins ⊢ (g1, s, env) →g_f2 (stmts, None, newS)⟧ ⟹
  ins ⊢ (g, s, env) →g_f2 (stmts, Some g2, newS)" |
GuardThenGuardIncompleteGuard:  "⟦g = GuardThenGuard g1 g2;
  ins ⊢ (g1, s, env) →g_f2 (stmts, Some nextG1, newS)⟧ ⟹
  ins ⊢ (g, s, env) →g_f2 (stmts, Some (GuardThenGuard nextG1 g2), newS)" |
GuardStmtsGuardCompleteGuardStep: "⟦g = GuardStmtsGuard g1 stmts;
  ins ⊢ (g1, s, env) →g_f2 (newStmts, None, newS)⟧ ⟹
  ins ⊢ (g, s, env) →g_f2 (newStmts @ stmts, None, newS)" |
GuardStmtsGuardIncompleteGuard: "⟦g = GuardStmtsGuard g1 stmts;
  ins ⊢ (g1, s, env) →g_f2 (newStmts, Some nextG1, newS)⟧ ⟹
  ins ⊢ (g, s, env) →g_f2 (newStmts, Some (GuardStmtsGuard nextG1 stmts), newS)" |
DisjointGuardMatchesG1:  "⟦(ins ⊢ (g1, s, env) →g_f2 g1Res)⟧ ⟹
  ins ⊢ (DisjointGuard g1 g2, s, env) →g_f2 g1Res" |
DisjointGuardMatchesG2:  "⟦computeGuardStep ins (g1, s, env) = None;
  (ins ⊢ (g2, s, env) →g_f2 g2Res)⟧ ⟹
  ins ⊢ (DisjointGuard g1 g2, s, env) →g_f2 g2Res" |
InterleavedGuardMatchesG1Complete:  "⟦(ins ⊢ (g1, s, env) →g_f2 (stmts, None, newS))⟧ ⟹
  ins ⊢ (InterleavedGuard g1 g2, s, env) →g_f2 (stmts, Some g2, newS)" |
InterleavedGuardMatchesG1Incomplete:  "⟦(ins ⊢ (g1, s, env) →g_f2 (stmts, Some innerG1, newS))⟧
     ⟹
  ins ⊢ (InterleavedGuard g1 g2, s, env) →g_f2 (stmts, Some (InterleavedGuard innerG1 g2), newS)"
        |
```

```
InterleavedGuardMatchesG2Complete: "⟦(ins ⊢ (g2, s, env) →gf2 (stmts, None, newS));
  computeGuardStep ins (g1, s, env) = None⟧ ⟹
  ins ⊢ (InterleavedGuard g1 g2, s, env) →gf2 (stmts, Some g1, newS)" |
InterleavedGuardMatchesG2Incomplete:  "⟦(ins ⊢ (g2, s, env) →gf2 (stmts, Some innerG2, newS));
  computeGuardStep ins (g1, s, env) = None⟧ ⟹
  ins ⊢ (InterleavedGuard g1 g2, s, env) →gf2 (stmts, Some (InterleavedGuard g1 innerG2), newS)"
```

A single guard expression reduction of the form $i \vdash (g, s, env) \rightarrow^{g}_{f2} (stmts, g', s')$ can be read as "given an input i then a guard expression g in state s and environment env reduces to the statements $stmts$ possibly followed by another guard expression in g' in state s'". Note that the interleaving and choice guards prioritize the left operand in order to maintain a deterministic semantics that can be used in the verification of the compiler. Since Marlowe has deterministic `Case` evaluation in its input processing semantics, Faustus must also maintain deterministic `FGuardExpression` and `FCase` evaluation in its input processing semantics, otherwise nondeterministic steps in the Faustus semantics would not have corresponding steps in the Marlowe semantics.

3.4 Type-Checking Faustus

Marlowe notably does not have a type-checker, as all syntactically correct Marlowe programs will execute as described by Seijas and Thompson [1]. Unbound value names are handled by using the default value 0 (zero). Type-checkers are typically defined by a collection of inductive rules. Nipkow and Klein demonstrate how to implement such rules in Isabelle [24]. Based on, but modifying, their model, the Faustus type-checker is implemented as a series of functions that type-check the `FContract` and other pieces of the syntax tree recursively along with the `FState` and `FContext`. The types of these functions and the `TypeContext` are:

Listing 3.8: Isabelle type information for the `AbstractionTypeInformation`, `TypeContext`, and type-checking functions.

```
datatype AbstractionTypeInformation = ValueType
  | ObservationType
  | PartyType
  | ContractType "FParameter list"
```

```
type_synonym TypeContext = "(Identifier × AbstractionTypeInformation) list"
fun paramsToTypeContext :: "FParameter list ⇒ TypeContext"
fun wellTypedParty :: "TypeContext ⇒ FParty ⇒ bool"
fun wellTypedPayee :: "TypeContext ⇒ FPayee ⇒ bool"
fun wellTypedChoiceId :: "TypeContext ⇒ FChoiceId ⇒ bool"
fun wellTypedValue :: "TypeContext ⇒ FValue ⇒ bool" and
  wellTypedObservation :: "TypeContext ⇒ FObservation ⇒ bool"
fun wellTypedArgument :: "TypeContext ⇒ FParameter ⇒ FArgument ⇒ bool"
fun wellTypedArguments :: "TypeContext ⇒ FParameter list ⇒ FArgument list ⇒ bool"
fun wellTypedState :: "TypeContext ⇒ FState ⇒ bool"
fun wellTypedStatement :: "TypeContext ⇒ FStatement ⇒ bool"
fun wellTypedStatements :: "TypeContext ⇒ FStatement list ⇒ bool"
fun wellTypedAction :: "TypeContext ⇒ FAction ⇒ bool"
fun wellTypedGuardExpression :: "TypeContext ⇒ FGuardExpression ⇒ bool"
fun wellTypedContract :: "TypeContext ⇒ FContract ⇒ bool" and
  wellTypedCases :: "TypeContext ⇒ FCase list ⇒ bool"
fun wellTypedContext :: "TypeContext ⇒ FContext ⇒ bool"
```

The type-checking rules for Faustus can be found in Figures B.1, B.2, B.3, B.4, B.5, B.6, B.7, and B.8 in Appendix B, and the Isabelle definition of the type-checker can be found in Appendix C.

3.4.1 Preservation and Progress for Well-Typed Faustus Contracts

From our definition of the Faustus type-checker, we can prove properties of the execution of well-typed Faustus contracts. We first show that a well-typed *Party*, *Payee*, *ChoiceId*, *Value*, and *Observation* will evaluate to a meaningful value in the following lemmas:

Listing 3.9: Isabelle lemmas proving a well-typed *Party*, *Payee*, *ChoiceId*, *Value*, and *Observation* will evaluate to a meaningful value.

```
lemma wellTypedPartyEvaluates:
"wellTypedParty tyCtx pk ⟹
  wellTypedState tyCtx s ⟹
  evalFParty s pk ≠ None"
lemma wellTypedPayeeEvaluates:
"wellTypedPayee tyCtx payee ⟹
  wellTypedState tyCtx s ⟹
  evalFPayee s payee ≠ None"
lemma wellTypedChoiceIdEvaluates:
"wellTypedChoiceId tyCtx choiceId ⟹
```

```
  wellTypedState tyCtx s ⟹
  evalFChoiceId s choiceId ≠ None"
lemma wellTypedValueObservationEvaluates:
"wellTypedValue tyCtx val ⟹
  wellTypedState tyCtx s ⟹
  evalFValue env s val ≠ None"
"wellTypedObservation tyCtx obs ⟹
  wellTypedState tyCtx s ⟹
  evalFObservation env s obs ≠ None"
```

After proving the previous lemmas, we use them to show that a well-typed *Statement* and *Argument list* will evaluate to a meaningful value.

Listing 3.10: Isabelle lemmas proving a well-typed *Statement* and *Argument list* will evaluate to a meaningful value.

```
lemma wellTypedPayContinues:
  assumes "wellTypedContract tyCtx (StatementCont (Pay accId payee token amt) cont) ∧
  wellTypedContext tyCtx ctx ∧
  wellTypedState tyCtx s"
  shows "(∃ newS newW newP .(StatementCont (Pay accId payee token amt) cont, ctx, s, e, w, p) →f
      2 (cont, ctx, newS, e, newW, newP))"
lemma wellTypedAssertContinues:
  assumes "wellTypedContract tyCtx (StatementCont (Assert obs) cont) ∧
  wellTypedContext tyCtx ctx ∧
  wellTypedState tyCtx s"
  shows "(∃ newW newP .(StatementCont (Assert obs) cont, ctx, s, e, w, p) →f2 (cont, ctx, s, e,
      newW, newP))"
lemma wellTypedReassignValContinues:
  assumes "wellTypedContract tyCtx (StatementCont (ReassignVal vId val) cont) ∧
  wellTypedContext tyCtx ctx ∧
  wellTypedState tyCtx s"
  shows "(∃ newS newW newP .(StatementCont (ReassignVal vId val) cont, ctx, s, e, w, p) →f2 (
      cont, (ValueAbstraction vId)#ctx, newS, e, newW, newP))"
lemma wellTypedReassignPkContinues:
  assumes "wellTypedContract tyCtx (StatementCont (ReassignParty pkId pk) cont) ∧
  wellTypedContext tyCtx ctx ∧
  wellTypedState tyCtx s"
  shows "(∃ newS newW newP .(StatementCont (ReassignParty pkId pk) cont, ctx, s, e, w, p) →f2 (
      cont, (PartyAbstraction pkId)#ctx, newS, e, newW, newP))"
lemma wellTypedArgumentsExecute:
"wellTypedArguments tyCtx params args ⟹
  wellTypedState tyCtx s ⟹
  evalFArguments e s params args ≠ None"
```

The proofs for the lemmas are done by using the induction rules defined by the type-checker. With a definition for *FConfigurations* in a final state, the following lemma proves that a Faustus small-step reduction on a well-typed *FContract* only halts on *Close* and *When* contracts.

Listing 3.11: An Isabelle lemma proving that a well-typed Faustus *Contract* only halts when reaching a *When* or *Close*.

```
definition "finalF cs ⟷ ¬(∃cs'. cs →f2 cs')"
lemma finalFD: "(finalF (c1, cx, s, e, w, p) ∧ (wellTypedContract tcx c1) ∧ wellTypedContext tcx
     cx ∧ wellTypedState tcx s) ⟹ c1 = Close ∨ (∃ cs t c2 . c1 = When cs t c2)"
```

The proof for this lemma is done by case analysis on the *c1* variable. We are able to show that all contracts except *Close* and *When* will reduce.

The next lemmas prove that a well-typed *FContract* will continue to be well-typed after a small-step reduction and after processing input.

Listing 3.12: The Faustus type preservation lemmas in Isabelle.

```
lemma faustusTypePreservation:
assumes
  "((c1, cx1, s1, e1, w1, p1) →𝒯 (c2, cx2, s2, e2, w2, p2)) ∧
  wellTypedContract tcx1 c1 ∧
  wellTypedContext tcx1 cx1 ∧
  wellTypedState tcx1 s1"
shows
  "(∃tcx2 . wellTypedContract tcx2 c2 ∧
  wellTypedContext tcx2 cx2 ∧
  wellTypedState tcx2 s2)"
lemma faustusTypePreservationInput:
assumes
  "((c1, cx1, s1, e1, w1, p1) →(i, lower, upper) (c2, cx2, s2, e2, w2, p2)) ∧
  wellTypedContract tcx1 c1 ∧
  wellTypedContext tcx1 cx1 ∧
  wellTypedState tcx1 s1"
shows
  "wellTypedContract tcx1 c2 ∧
  wellTypedContext tcx1 cx2 ∧
  wellTypedState tcx1 s2"
```

The proof for the first lemma is done by cases of the *c1* variable. By going through small-step reduction rules, we construct a type context so that the *c2*, *cx2*, and *s2* are

well-typed.

The proof for the second lemma is done by proving the type context does not need to be modified to type-check the result of processing input. Specifically, the result of reducing guard expressions with statements is shown to be well-typed under the same type context.

In this section we have defined the type-checker for *FContracts*. The type-checker allows us to avoid halting on the *UseParty*, *UseValue*, *UseObservation*, and *UseC* constructs when running small-step reductions. Note that *Value* abstractions exist in Marlowe, but Marlowe does not have a type-checker, so the Marlowe evaluator can encounter an identifier that has not previously been declared. The Faustus type-checker can be used on Marlowe contracts to avoid this error, because Marlowe is a subset of Faustus. In the next section we present the Faustus compiler and show that well-typed *FContracts* will compile.

3.5 Formalized Faustus Compiler

In this section we present the compiler for Faustus. As demonstrated by Nipkow and Klein [24], but extended for parameterized abstractions, the compiler is defined as a series of functions that translate a Faustus program into a Marlowe program in Appendix C. Isabelle proofs show that the semantics of the original Faustus program are preserved in the compiled Marlowe program. In particular, we show that the behaviors of a well-typed Faustus contract correspond to the behaviors of the compiled Marlowe contract through the normalization and compilation relations. The semantics of the resulting Marlowe contract are correct and adequate with respect to the semantics of the original Faustus contract. This verifies that the simulation diagrams in Figure 3.2 hold. The type signatures of the compilation functions are:

Listing 3.13: The type signatures of the Faustus compiler functions.

```
fun compileFParty :: "FParty ⇒ FState ⇒ Party option"
fun compileFPayee :: "FPayee ⇒ FState ⇒ Payee option"
fun compileFChoiceId :: "FChoiceId ⇒ FState ⇒ ChoiceId option"
fun compileFValue :: "FValue ⇒ FState ⇒ Value option" and
```

```
  compileFObservation :: "FObservation ⇒ FState ⇒ Observation option"
fun compileAction :: "FAction ⇒ FState ⇒ SemanticsTypes.Action option"
fun compileArguments :: "FParameter list ⇒ FArgument list ⇒ FState ⇒ Contract ⇒ Contract
     option"
fun compileStmtToIR :: "FStatement ⇒ FContractIR ⇒ FContractIR"
fun compileStmtsToIR :: "FStatement list ⇒ FContractIR ⇒ FContractIR"
fun compileGuardExpressionToIR :: "FGuardExpression ⇒ FContractIR ⇒ int ⇒ FContractIR ⇒
     FCaseIR list"
fun normalizeContract :: "FContract ⇒ FContractIR"
  and normalizeCases :: "FCase list ⇒ int ⇒ FContractIR ⇒ FCaseIR list"
```

3.5.1 Compiling Concurrency Operators via Normalization

In order to verify the Faustus compiler, the guard expressions in the Faustus `When` construct must be translated to a list of `FCase` constructs with single `FAction` guards that preserve the behavior of the guard expression. Additionally, the statements in the guard expressions must be compiled to Faustus `FContract` constructs. This is the *normalization* step of Faustus compilation. First, the guard expressions are normalized by translating them into normalized guard expressions with no interleaving; then the normalized guard expressions are compiled to an intermediate Faustus `FContractIR` construct. This intermediate representation (IR) of the Faustus contract is a `FContract` with no guard expressions. The function type signatures for the Faustus normalization process are given below with the datatype definition of normalized guard expressions.

Listing 3.14: Isabelle type information for Faustus guard normalization.

```
type_synonym NormalizedGuardNode = "(FAction × FStatement list)"
datatype NormalizedGuardExpression =
  NormalizedGuardNode NormalizedGuardNode
  | NormalizedGuardThenGuard NormalizedGuardNode NormalizedGuardExpression
  | NormalizedDisjointGuard NormalizedGuardExpression NormalizedGuardExpression
fun normalizeGuardExpression :: "FGuardExpression ⇒ NormalizedGuardExpression"
fun compileNormalizedGuardNodeToIR :: "NormalizedGuardNode ⇒ FContractIR ⇒ FCaseIR"
fun compileNormalizedGuardExpressionToIR :: "NormalizedGuardExpression ⇒ FContractIR ⇒ int ⇒
     FContractIR ⇒ FCaseIR list"
fun compileGuardExpressionToIR :: "FGuardExpression ⇒ FContractIR ⇒ int ⇒ FContractIR ⇒
     FCaseIR list"
fun normalizeContract :: "FContract ⇒ FContractIR"
  and normalizeCases :: "FCase list ⇒ int ⇒ FContractIR ⇒ FCaseIR list"
```

Note that all Faustus contracts can be normalized. However, the compilation to Marlowe can fail after normalization, so we also prove that a well-typed Faustus configuration will still be well-typed after the normalization process.

Listing 3.15: Isabelle lemma proving that a well-typed Faustus contract will be well-typed after normalization is applied.

```
lemma wellTypedCompilesToWellTyped:
"(∀tyCtx . (wellTypedContract tyCtx contract ⟶
  (FaustusSemantics.wellTypedContract tyCtx (normalizeContract contract))))"
"(∀tyCtx .(wellTypedCases tyCtx cases ⟶
  (∀t tcont. FaustusSemantics.wellTypedContract tyCtx tcont ⟶ FaustusSemantics.wellTypedCases
       tyCtx (normalizeCases cases t tcont))))"
```

After defining the process of normalization, we define a relation between original Faustus configurations and normalized Faustus configurations and prove that the semantics of a normalized contract are correct and adequate with respect to the semantics of the original Faustus contract. The first step of this process is defining reduction rules for the normalized guard expressions; then we prove a step occurs in the original guard expression if and only if a step occurs in the normalized guard expression; and finally, we prove that a step in the original Faustus contract occurs if and only if a step occurs in the normalized Faustus contract.

Listing 3.16: Isabelle lemmas proving the correctness of the Faustus guard normalization process.

```
inductive small_step_reduce_normalized_guard :: "Input ⇒ FNormalizedGuardConfig ⇒
     FNormalizedGuardConfigRes ⇒ bool" ("_ ⊢ _ →lg_f2 _" [55, 0, 55]) where
NormalizedGuardNodeStep:  "⟦g = NormalizedGuardNode (a, stmts);
  ins ⊢ (a, s, env) →a_f2 newS⟧ ⟹
  ins ⊢ (g, s, env) →lg_f2 (stmts, None, newS)" |
NormalizedGuardThenGuardStep:  "⟦g = NormalizedGuardThenGuard (a, stmts) g2;
  ins ⊢ (a, s, env) →a_f2 newS⟧ ⟹
  ins ⊢ (g, s, env) →lg_f2 (stmts, Some g2, newS)" |
NormalizedDisjointGuardMatchesG1:  "⟦(ins ⊢ (g1, s, env) →lg_f2 g1Res)⟧ ⟹
  ins ⊢ (NormalizedDisjointGuard g1 g2, s, env) →lg_f2 g1Res" |
NormalizedDisjointGuardMatchesG2:  "⟦computeNormalizedGuardStep ins (g1, s, env) = None;
  (ins ⊢ (g2, s, env) →lg_f2 g2Res)⟧ ⟹
  ins ⊢ (NormalizedDisjointGuard g1 g2, s, env) →lg_f2 g2Res"
lemma finalGuardImpliesFinalNormalizedGuard:
```

```
"finalGuard ins (g, state, env) ⟹ finalNormalizedGuard ins (normalizeGuardExpression g, state,
     env)"
lemma guardStepImpliesNormalizedGuardStep:
"ins ⊢ (g, state, env) →g_f2 (stmts, None, state') ⟹
   ins ⊢ ((normalizeGuardExpression g), state, env) →lg_f2 (stmts, None, state')"
lemma guardStepImpliesNormalizedGuardStepGeneral:
"ins ⊢ (g, state, env) →g_f2 (stmts, gCont, state') ⟹
   (∃lgCont . ins ⊢ ((normalizeGuardExpression g), state, env) →lg_f2 (stmts, lgCont, state') ∧
    (gCont = None ⟶ lgCont = None) ∧
    (gCont = Some innerGCont ⟶ lgCont = Some (normalizeGuardExpression innerGCont)))"
lemma normalizedGuardExpressionsStepImpliesCasesStep:
"ins ⊢ (g, state, env) →lg_f2 (stmts, nextGuard, state') ⟹
  (∃casesRes . ins ⊢ (normalizedGuardExpressionToCases g cont t tcont, state, env) →cases_f2 (
       casesRes, state'))"
lemma casesStepImpliesNormalizedGuardExpressionsStep:
"(ins ⊢ (normalizedGuardExpressionToCases g cont t tcont, state, env) →cases_f2 (casesRes, state
     ')) ⟹
lemma finalGuardExpressionImpliesFinalCases:
"finalGuard ins (g, state, env) ⟹
  finalFCases ins (normalizedGuardExpressionToCases (normalizeGuardExpression g) cont t tcont,
       state, env)"
lemma casesStepImpliesGuardExpressionsStep:
"ins ⊢ (normalizedGuardExpressionToCases (normalizeGuardExpression g) cont t tcont, state, env)
     →cases_f2 (casesRes, state') ⟹
  (∃stmts nextGuard . ins ⊢ (g, state, env) →g_f2 (stmts, nextGuard, state'))"
definition normalize_relation :: "(FConfig × FIRConfig) set" where
"normalize_relation =
  {((fc, fctx, fstate, fenv, fwarnings, fpayments), (irc, irctx, irstate, irenv, irwarnings,
       irpayments)) |
    fc fctx fstate fenv fwarnings fpayments irc irctx irstate irenv irwarnings irpayments .
    ∃tyCtx . (wellTypedContract tyCtx fc ∧
    wellTypedContext tyCtx fctx ∧
    wellTypedState tyCtx fstate ∧
    normalizeContract fc = irc ∧
    normalizeCtx fctx = irctx ∧
    fstate = irstate ∧
    fenv = irenv ∧
    fwarnings = irwarnings ∧
    fpayments = irpayments)}"
lemma smallStepInternalImpliesFIRSmallStepInternalNormalizeRelation:
"((fc, fctx, fstate, fenv, fwarnings, fpayments), (irc, irctx, irstate, irenv, irwarnings,
     irpayments)) ∈ normalize_relation ⟹
  (fc, fctx, fstate, fenv, fwarnings, fpayments) →𝒯 (fc', fctx', fstate', fenv', fwarnings',
       fpayments') ⟹
```

```
  ∃irc' irctx' irstate' irenv' irwarnings' irpayments'.  (irc, irctx, irstate, irenv, irwarnings,
        irpayments) →fir𝒯 (irc', irctx', irstate', irenv', irwarnings', irpayments') ∧
    ((fc', fctx', fstate', fenv', fwarnings', fpayments'), (irc', irctx', irstate', irenv',
         irwarnings', irpayments')) ∈ normalize_relation"
lemma smallStepInternalFIRImpliesSmallStepInternalNormalizeRelation:
"((fc, fctx, fstate, fenv, fwarnings, fpayments), (irc, irctx, irstate, irenv, irwarnings,
     irpayments)) ∈ normalize_relation ⟹
  (irc, irctx, irstate, irenv, irwarnings, irpayments) →fir𝒯 (irc', irctx', irstate', irenv',
       irwarnings', irpayments') ⟹
  ∃fc' fctx' fstate' fenv' fwarnings' fpayments' tyCtx'. (fc, fctx, fstate, fenv, fwarnings,
       fpayments) →𝒯 (fc', fctx', fstate', fenv', fwarnings', fpayments') ∧
    ((fc', fctx', fstate', fenv', fwarnings', fpayments'), (irc', irctx', irstate', irenv',
         irwarnings', irpayments')) ∈ normalize_relation"
lemma smallStepInputImpliesFIRSmallStepInputNormalizeRelation:
"((fc, fctx, fstate, fenv, fwarnings, fpayments), (irc, irctx, irstate, irenv, irwarnings,
     irpayments)) ∈ normalize_relation ⟹
  ((fc, fctx, fstate, fenv, fwarnings, fpayments) →(Some i, lower, upper) (fc', fctx', fstate',
       fenv', fwarnings', fpayments')) ⟹
  ∃irc' irctx' irstate' irenv' irwarnings' irpayments' tyCtx'.
    ((irc, irctx, irstate, irenv, irwarnings, irpayments) →firi(Some i, lower, upper) (irc',
         irctx', irstate', irenv', irwarnings', irpayments')) ∧
    ((fc', fctx', fstate', fenv', fwarnings', fpayments'), (irc', irctx', irstate', irenv',
         irwarnings', irpayments')) ∈ normalize_relation"
lemma smallStepInputFIRImpliesSmallStepInputNormalizeRelation:
"((fc, fctx, fstate, fenv, fwarnings, fpayments), (irc, irctx, irstate, irenv, irwarnings,
     irpayments)) ∈ normalize_relation ⟹
  ((irc, irctx, irstate, irenv, irwarnings, irpayments) →firi(Some i, lower, upper) (irc', irctx
       ', irstate', irenv', irwarnings', irpayments')) ⟹
  ∃fc' fctx' fstate' fenv' fwarnings' fpayments' tyCtx' .
    ((fc, fctx, fstate, fenv, fwarnings, fpayments) →(Some i, lower, upper) (fc', fctx', fstate
         ', fenv', fwarnings', fpayments')) ∧
    ((fc', fctx', fstate', fenv', fwarnings', fpayments'), (irc', irctx', irstate', irenv',
         irwarnings', irpayments')) ∈ normalize_relation"
lemma smallStepTimeoutImpliesFIRSmallStepTimeoutNormalizeRelation:
"((fc, fctx, fstate, fenv, fwarnings, fpayments), (irc, irctx, irstate, irenv, irwarnings,
     irpayments)) ∈ normalize_relation ⟹
  ((fc, fctx, fstate, fenv, fwarnings, fpayments) →(None, lower, upper) (fc', fctx', fstate',
       fenv', fwarnings', fpayments')) ⟹
  ∃irc' irctx' irstate' irenv' irwarnings' irpayments' tyCtx'.
    ((irc, irctx, irstate, irenv, irwarnings, irpayments) →firi(None, lower, upper) (irc', irctx
         ', irstate', irenv', irwarnings', irpayments')) ∧
    ((fc', fctx', fstate', fenv', fwarnings', fpayments'), (irc', irctx', irstate', irenv',
         irwarnings', irpayments')) ∈ normalize_relation"
lemma smallStepTimeoutFIRImpliesSmallStepTimeoutNormalizeRelation:
```

```
"((fc, fctx, fstate, fenv, fwarnings, fpayments), (irc, irctx, irstate, irenv, irwarnings,
    irpayments)) ∈ normalize_relation ⟹
  ((irc, irctx, irstate, irenv, irwarnings, irpayments) →firi (None, lower, upper) (irc', irctx',
      irstate', irenv', irwarnings', irpayments')) ⟹
  ∃fc' fctx' fstate' fenv' fwarnings' fpayments' tyCtx' .
    ((fc, fctx, fstate, fenv, fwarnings, fpayments) →(None, lower, upper) (fc', fctx', fstate',
        fenv', fwarnings', fpayments')) ∧
    ((fc', fctx', fstate', fenv', fwarnings', fpayments'), (irc', irctx', irstate', irenv',
        irwarnings', irpayments')) ∈ normalize_relation"
```

The completion of these proofs verifies the top part of the simulation diagrams from Figure 3.2:

$$
\begin{array}{ccc}
\mathcal{F}_1 & \xrightarrow{\lambda} & \mathcal{F}_2 \\
\mathcal{N} & & \mathcal{N} \\
\mathcal{I}_1 & \xrightarrow{\lambda}_{fir} & \mathcal{I}_2
\end{array}
\qquad
\begin{array}{cc}
\mathcal{F}_1 \Downarrow^{=} & \mathcal{F}_2 \\
\mathcal{N} & \mathcal{N} \\
\mathcal{I}_1 \Downarrow^{=}_{fir} & \mathcal{I}_2
\end{array}
$$

The simulation diagrams presented allow for stuttering on internal reductions. A stronger step-by-step bisimulation is shown for the normalization relation that can be seen below[2]. We will explain the need for stuttering between Faustus and Marlowe contracts in the next section.

$$
\begin{array}{ccc}
\mathcal{F}_1 & \xrightarrow{\alpha} & \mathcal{F}_2 \\
\mathcal{N} & & \mathcal{N} \\
\mathcal{I}_1 & \xrightarrow{\alpha}_{fir} & \mathcal{I}_2
\end{array}
$$

3.5.2 Compiling Abstractions

After normalization, Faustus programs may contain abstractions of three syntactic classes that do not have abstractions in Marlowe. This stage of the compiler translates an `FContractIR`, `FContext`, and `FState` at any point of execution into a Marlowe `Contract`. The `FState` is also translated into a Marlowe `State` by dropping the `boundParties` field in the record. As previously described, Marlowe only supports abstractions for `Values`, and there is no way to jump to a previously evaluated part of the program. This means that

[2]We use α here to denote an internal reduction or input processing.

the usual strategies for compilation to assembly languages will not work for the Faustus compiler, and we must detail our strategies for each of the three syntactic classes with added abstractions.

For `Party` abstractions we note that a `Party` can only contain constant values, and there are no operations that can modify a `Party` abstraction once it is declared. We use this knowledge to evaluate all Faustus `FParty`s during compilation. The `FState` holds all `Party` abstraction information during compilation. Additional lemmas can be found in Appendix C which prove that all Faustus compilation functions only rely on the `boundParty` information in the `FState`.

Next we note that `Observation` abstractions can be used in `Value` expressions, and there are operations on the two that depend on runtime information. We have also defined our `Observation` abstractions to be eagerly evaluated to match the behavior of the existing `Value` abstractions. This means that we cannot evaluate `Observation` expressions during compilation, nor can we replace all invocations inline with the body of the abstractions. The results of the expressions depend on the program state as it runs. Instead we convert all `FObservation` declarations into Marlowe `Value` declarations through the conditional ternary operator, called `Cond` in Marlowe, that evaluate to 1 or 0 if the `FObservation` evaluates to true or false respectively. Then all Faustus `FObservation` invocations are translated into Marlowe `Value` invocations on the left side of a greater than 0 expression.

Finally we note that `Contract` abstractions are lazily evaluated. So we compile `Contract` declarations storing the `FContract` body of the abstraction along with the parameter information in the `FContext`. Then `Contract` invocations are compiled by compiling the arguments and parameters as declarations of their corresponding types, then compiling the body using the tail of the `FContext` to prevent recursion, and placing the argument declarations before the compiled Marlowe `Contract` body.

To begin the compiler verification, we show that well-typed Faustus contracts compile to Marlowe contracts in Listing 3.17. This lemma is required to prevent the definition of a compiler that never produces any output. Compilation of non-well-typed Faustus contracts may fail due to the use of abstractions that have not been declared.

Listing 3.17: Isabelle lemma proving a well-typed Faustus configuration will succeed in compiling to a Marlowe configuration.

```
lemma wellTypedCompiles:
"(∀tyCtx . (wellTypedContract tyCtx contract ⟶
  wellTypedState tyCtx state ⟶
  wellTypedContext tyCtx ctx ⟶
  (∃mCont . compile contract ctx state = Some mCont)))"
"(∀tyCtx .(wellTypedCases tyCtx cases ⟶
  wellTypedState tyCtx state ⟶
  wellTypedContext tyCtx ctx ⟶
  (∃mCases. compileCases cases ctx state = Some mCases)))"
```

Through the following Isabelle lemmas we show that the semantics of the compiled Marlowe contract are adequate and correct with respect to the semantics of the original Faustus program. Recall that this is better understood through the simulation diagrams in Figure 3.2 in Section 3.2. That is, we define a relation between well-typed Faustus IR configurations and the compiled Marlowe configurations, and show the relation holds as both configurations execute. Due to the compilation strategies of the Faustus declarations, the internal reductions between Faustus and Marlowe do not exactly match, and one may stutter while the other performs internal reductions. For example, a `Party` declaration is evaluated at compile time, so the Faustus contract evaluates the declaration while the corresponding Marlowe contract has no internal reductions to perform. Additionally, a Faustus contract that evaluates a `Value` argument as part of a `UseC` contract would then have no corresponding internal reductions to perform while the Marlowe contract evaluates the resulting `Value` declaration.

Listing 3.18: Isabelle lemmas proving the correctness of the Faustus compiler.

```
definition compile_relation :: "(FIRConfig × MConfig) set" where
"compile_relation =
  {((fc, fctx, fstate, fenv, fwarnings, fpayments), (mc, mstate, menv, mwarnings, mpayments)) |
    fc fctx fstate fenv fwarnings fpayments mc mstate menv mwarnings mpayments .
    ∃tyCtx . (wellTypedContract tyCtx fc ∧
    wellTypedContext tyCtx fctx ∧
    wellTypedState tyCtx fstate ∧
    compile fc fctx fstate = Some mc ∧
    fStateToMState fstate = mstate ∧
```

```
    fenv = menv ∧
    fpayments = mpayments)}"
lemma reduceAllFIRImpliesMarloweReduceAllCompilerRelation:
"((fc, fctx, fstate, fenv, fwarnings, fpayments), (mc, mstate, menv, mwarnings, mpayments)) ∈
     compile_relation ⟹
  (fc, fctx, fstate, fenv, [], fpayments) ⇓fir (newFContract, newCtx, newState, newEnv,
       newReduceWarnings, newPayments) ⟹
  ∃mc' mstate' menv' mwarnings' mtwarnings' mpayments' tyCtx' .  (mc, mstate, menv, [], mpayments
       ) ⇓m (mc', mstate', menv', mwarnings', mpayments') ∧
    ((newFContract, newCtx, newState, newEnv, fwarnings @ convertReduceWarnings newReduceWarnings
         , newPayments), (mc', mstate', menv', mwarnings @ convertReduceWarnings mwarnings',
         mpayments')) ∈ compile_relation ∨
    ((newFContract, newCtx, newState, newEnv, fwarnings @ convertReduceWarnings newReduceWarnings
         , newPayments), (mc, mstate, menv, mwarnings, mpayments)) ∈ compile_relation"
lemma reduceAllMarloweImpliesFIRReduceAllCompilerRelation:
"((fc, fctx, fstate, fenv, fwarnings, fpayments), (mc, mstate, menv, mwarnings, mpayments)) ∈
     compile_relation ⟹
  (mc, mstate, menv, [], mpayments) ⇓m (mc', mstate', menv', mwarnings', mpayments') ⟹
  ∃fc' fctx' fstate' fenv' fwarnings' fpayments' tyCtx'. (fc, fctx, fstate, fenv, [], fpayments)
       ⇓fir (fc', fctx', fstate', fenv', fwarnings', fpayments') ∧
    ((fc', fctx', fstate', fenv', fwarnings @ convertReduceWarnings fwarnings', fpayments'), (mc
         ', mstate', menv', mwarnings @ convertReduceWarnings mwarnings', mpayments')) ∈
         compile_relation"
lemma reduceAllReflFIRImpliesMarloweReduceAllReflCompilerRelation:
"((fc, fctx, fstate, fenv, fwarnings, fpayments), (mc, mstate, menv, mwarnings, mpayments)) ∈
     compile_relation ⟹
  (fc, fctx, fstate, fenv, [], fpayments) ⇓fir= (newFContract, newCtx, newState, newEnv,
       newReduceWarnings, newPayments) ⟹
  ∃mc' mstate' menv' mwarnings' mtwarnings' mpayments' tyCtx' .  (mc, mstate, menv, [], mpayments
       ) ⇓m= (mc', mstate', menv', mwarnings', mpayments') ∧
    ((newFContract, newCtx, newState, newEnv, fwarnings @ convertReduceWarnings newReduceWarnings
         , newPayments), (mc', mstate', menv', mwarnings @ convertReduceWarnings mwarnings',
         mpayments')) ∈ compile_relation"
lemma reduceAllReflMarloweImpliesFIRReduceAllReflCompilerRelation:
"((fc, fctx, fstate, fenv, fwarnings, fpayments), (mc, mstate, menv, mwarnings, mpayments)) ∈
     compile_relation ⟹
  (mc, mstate, menv, [], mpayments) ⇓m= (mc', mstate', menv', mwarnings', mpayments') ⟹
  ∃fc' fctx' fstate' fenv' fwarnings' fpayments' tyCtx'. (fc, fctx, fstate, fenv, [], fpayments)
       ⇓fir= (fc', fctx', fstate', fenv', fwarnings', fpayments') ∧
    ((fc', fctx', fstate', fenv', fwarnings @ convertReduceWarnings fwarnings', fpayments'), (mc
         ', mstate', menv', mwarnings @ convertReduceWarnings mwarnings', mpayments')) ∈
         compile_relation"
lemma smallStepInputFIRImpliesMarloweSmallStepInputCompilerRelation:
"((fc, fctx, fstate, fenv, fwarnings, fpayments), (mc, mstate, menv, mwarnings, mpayments)) ∈
```

```
    compile_relation ⟹
  (fc, fctx, fstate, fenv, fwarnings, fpayments) →firλ(Some i, lower, upper) (newFContract,
      newCtx, newState, newEnv, newWarnings, newPayments) ⟹
  (∃mc' mstate' menv' mwarnings' mpayments' tyCtx' .
    ((mc, mstate, menv, mwarnings, mpayments) →mλ(Some i, lower, upper)  (mc', mstate', menv',
        mwarnings', mpayments')) ∧
    ((newFContract, newCtx, newState, newEnv, newReduceWarnings, newPayments), (mc', mstate',
        menv', mwarnings', mpayments')) ∈ compile_relation)"
lemma smallStepInputMarloweImpliesSmallStepInputFIRCompilerRelation:
"((When fcases t fc, fctx, fstate, fenv, fwarnings, fpayments), (mc, mstate, menv, mwarnings,
    mpayments)) ∈ compile_relation ⟹
  ((mc, mstate, menv, mwarnings, mpayments) →mλ(Some i, lower, upper)  (mc', mstate', menv',
      mwarnings', mpayments')) ⟹
  ∃fc' fctx' fstate' fenv' fwarnings' fpayments' tyCtx' .
    ((When fcases t fc, fctx, fstate, fenv, fwarnings, fpayments) →firλ(Some i, lower, upper) (
        fc', fctx', fstate', fenv', fwarnings', fpayments')) ∧
    ((fc', fctx', fstate', fenv', fwarnings', fpayments'), (mc', mstate', menv', mwarnings',
        mpayments')) ∈ compile_relation"
lemma smallStepTimeoutFIRImpliesMarloweSmallStepInputCompilerRelation:
"((fc, fctx, fstate, fenv, fwarnings, fpayments), (mc, mstate, menv, mwarnings, mpayments)) ∈
    compile_relation ⟹
  (fc, fctx, fstate, fenv, fwarnings, fpayments) →firλ(None, lower, upper) (newFContract, newCtx
      , newState, newEnv, newWarnings, newPayments) ⟹
  (∃mc' mstate' menv' mwarnings' mpayments' tyCtx' .
    ((mc, mstate, menv, mwarnings, mpayments) →mλ(None, lower, upper)  (mc', mstate', menv',
        mwarnings', mpayments')) ∧
    ((newFContract, newCtx, newState, newEnv, newReduceWarnings, newPayments), (mc', mstate',
        menv', mwarnings', mpayments')) ∈ compile_relation)"
lemma smallStepTimeoutMarloweImpliesSmallStepTimeoutFIRCompilerRelation:
"((When fcases t fc, fctx, fstate, fenv, fwarnings, fpayments), (mc, mstate, menv, mwarnings,
    mpayments)) ∈ compile_relation ⟹
  ((mc, mstate, menv, mwarnings, mpayments) →mλ(None, lower, upper)  (mc', mstate', menv',
      mwarnings', mpayments')) ⟹
  ∃fc' fctx' fstate' fenv' fwarnings' fpayments' tyCtx' .
    ((When fcases t fc, fctx, fstate, fenv, fwarnings, fpayments) →firλ(None, lower, upper) (fc
        ', fctx', fstate', fenv', fwarnings', fpayments')) ∧
    ((fc', fctx', fstate', fenv', fwarnings', fpayments'), (mc', mstate', menv', mwarnings',
        mpayments')) ∈ compile_relation"
```

With these lemmas, we verify the bottom part of the simulation diagrams from Figure 3.2. For input processing this means that given a well-typed Faustus configuration and the resulting compiled Marlowe configuration, the Faustus configuration will take a step to another well-typed Faustus configuration if and only if the compiled Marlowe configuration will

take a step to another Marlowe configuration such that the resulting Faustus configuration compiles to the resulting Marlowe configuration. For internal reductions this means that given a well-typed Faustus configuration and the resulting compiled Marlowe configuration, the Faustus configuration will stutter or do all internal reductions if and only if the compiled Marlowe configuration will stutter or do all internal reductions such that the resulting Faustus configuration compiles to the resulting Marlowe configuration.

$$\begin{array}{ccc} \mathcal{I}_1 & \xrightarrow{\lambda}_{fir} & \mathcal{I}_2 \\ \mathcal{C} & & \mathcal{C} \\ \mathcal{M}_1 & \xrightarrow{\lambda}_{m} & \mathcal{M}_2 \end{array} \qquad \begin{array}{ccc} \mathcal{I}_1 & \Downarrow^{=}_{fir} & \mathcal{I}_2 \\ \mathcal{C} & & \mathcal{C} \\ \mathcal{M}_1 & \Downarrow^{=}_{m} & \mathcal{M}_2 \end{array}$$

Finally, we note that any property that holds for the execution of all Marlowe programs (*e.g.* guaranteed termination or money preservation proved by Seijas et al. [12]) holds for all well-typed Faustus programs. This follows from the fact that a well-typed Faustus program will compile to a Marlowe program that preserves the semantics of the Faustus program.

3.6 Conciseness of Faustus Programs

In addition to the formalization of Faustus in Isabelle/HOL, we have developed an implementation of the language in Haskell. The purpose of the Faustus Haskell implementation is to provide a compiler for Faustus that targets the Marlowe Haskell implementation. The implementation provides a parser for a concrete syntax, a type-checker, and a compiler. The concrete syntax is provided with the goal of allowing the development of readable and maintainable smart contracts that do not rely on knowledge of Haskell. Marlowe contracts are written using Haskell datatype constructors which require every continuation and inner expression to be wrapped with parentheses. We argue that such syntax does not allow for readable and maintainable code. Providing a concrete syntax for Faustus solves this issue.

The Faustus concrete syntax follows most of the structure of the abstract syntax tree discussed in Section 2.2 for the Marlowe base and Section 3.3 for the Faustus extensions. Infix operators are used for `Value` and `Observation` expressions where expected (*e.g.*, + for addition and < for less than). Guard expressions also use infix <⋆> for interleaving,

```
contract check(party p1, party p2) {
    when {
        "carol" chooses "agree" -> {
            @"alice" pays !p1 (450, "");
            close
        },
        "carol" chooses "disagree" -> {
            @"alice" pays !p2 (450, "");
            close
        }
    } after 100 -> { close }
};
when {
    "alice" deposits 450 "" into @"alice" -> {
        when {
                "bob" chooses "claim" -> {
                    check("bob", "alice")
                },
                "alice" chooses "claim" -> {
                    check("alice", "bob")
                }
        } after 100 -> { close }
    }
} after 90 -> { close }
```

Figure 3.5: Faustus simple escrow contract.

<+> for choice, and -> for sequencing. Finally, declarations use the `var` keyword except for *`Contract`* abstractions which use the `contract` keyword, and `:=` is used for reassignment.

Using the Haskell implementation of Faustus, we can analyze the size of Faustus contracts against the resulting Marlowe contract. For an example of code size savings, we will examine the simple escrow contract written in Marlowe that can be found in Figure 3.1. Recall that in Section 3.1 we have identified two pieces of similar code after "alice" or "bob" have made a claim to the money after it is deposited. No matter who makes the claim, "carol" will choose to agree or disagree with the claimant, and the money is sent to the claimant or nonclaimant based on "carol's" choice.

With these observations, we can rewrite this contract in Faustus. The new version of the contract can be found in Figure 3.5. We move the code after "bob" or "alice" make a claim into a *`Contract`* abstraction, "check" that takes the claimant *`Party`* as a parameter "p1" and the nonclaimant as "p2". So when "alice" makes a claim we call "check" with the arguments "alice" and "bob", and when "bob" makes a claim we call "check" with the arguments "bob" and "alice".

Even in this simple example, the Faustus program is more maintainable. The sections

```
when {
    "alice" deposits 1000 "" into @"carol" -> {
        var votes = 0;
        when {
                ("voter1" chooses "agree" -> votes := votes + 1 <+>
                    "voter1" chooses not "agree") <*>
                ("voter2" chooses "agree" -> votes := votes + 1 <+>
                    "voter2" chooses not "agree") <*>
                ("voter3" chooses "agree" -> votes := votes + 1 <+>
                    "voter3" chooses not "agree") <*>
                ("voter4" chooses "agree" -> votes := votes + 1 <+>
                    "voter4" chooses not "agree") <*>
                ("voter5" chooses "agree" -> votes := votes + 1 <+>
                    "voter5" chooses not "agree") -> {
                    if votes >= 3
                    then @"carol" pays !"bob" (1000, ""); close
                    else @"carol" pays !"alice" (1000, ""); close
                }
        } after 100 -> { if votes >= 3
                    then @"carol" pays !"bob" (1000, ""); close
                    else @"carol" pays !"alice" (1000, ""); close }
    }
} after 90 -> { close }
```

Figure 3.6: Faustus five party multi-signature contract.

of code that are meant to have the same behavior in Marlowe are captured in a single abstraction in Faustus as expected. By better following the DRY principle from Hunt and Thomas [66] programmers will not be required to find all duplicate pieces of code in a Faustus program if the behavior of a program needs to be changed.

Larger multi-signature contracts show the expressive power of Faustus. A multi-signature contract that allows five people to vote in any order can be written in 23 lines of Faustus code. In Marlowe, the compiled contract is 138,253 lines of code when pretty printed according to the preexisting Marlowe pretty printing function. These savings come from the use of the concurrency operators in the Faustus program. There are 6,331 possible sequences of actions that can be taken after "alice" deposits money into "carol's" account. Each timeout cuts a sequence short, but this means the Marlowe pretty printer uses about 20 lines per sequence to print the `Case list` and `Pay` contract. We note that compressing the Marlowe contract using a zip compression tool such as 7zip [67] compresses the contract to 46.7KB, while the Faustus contract source file takes up only 873 bytes of space.

3.7 Discussion

In this chapter we have presented the Faustus smart contract programming language, which provides concurrency operators and constructs for handling abstractions (declarations and invocations) of `Party`, `Observation`, and `Contract` types. We have provided a small-step semantics to include rules for the new constructs. With the new rules, we can only continue execution of the `UseValue`, `UseObservation`, `UseParty`, and `UseC` constructs if the corresponding `Identifier` has been bound. The Faustus type-checker ensures well-typed Faustus contracts do not encounter these halting conditions. Finally, the Faustus compiler has been verified to ensure that a well-typed Faustus contract will compile to a Marlowe contract with correct and adequate behavior. This means evaluating a well-typed contract according to the Faustus semantics and then compiling to Marlowe gives the same result as compiling to Marlowe and evaluating the result according to the Marlowe semantics. As a result, Faustus contracts can be compiled into Marlowe contracts that run on any blockchain that has a Marlowe evaluator.

Even though Faustus has the same computational expressiveness as Marlowe, Faustus contracts are more readable and maintainable than their Marlowe counterparts. Marlowe contracts contain many instances of duplicate contract logic, and require knowledge of Haskell to generate them. With the verification of the Faustus compiler, smart contract programmers can write programs with the same runtime guarantees that all Marlowe contracts have with access to more ergonomically expressive concurrency operators and abstractions for more constructs than integer expressions.

There is still more work to be done to expand the number of features that Faustus supports. The next step is a formalization that uses the `boundValues` field of the `State` as a store to allow for variable shadowing. Once that indirection is in place, we believe a formalization of function abstractions for `Value` and `Observation` types would be possible by inlining the function bodies, and similar techniques could be used for abstractions of every single syntactical construct. Preliminary experiments in the Haskell implementation have shown promise in the ability to implement these extensions.

Chapter 4

Verifying Smart Contract Transformations Using Bisimulation

4.1 Theoretical Supports

4.1.1 The Calculus of Communicating Systems

The Calculus of Communicating Systems (CCS) is a process algebra developed by Milner [25, 68, 69] for describing concurrent systems and their interactions. A system is described in CCS by its possible interactions with its environment and the internal steps that arise from interactions with other concurrent components of the system. This allows for the study of the observable behavior that a system exhibits over time.

The semantics of CCS processes are understood through labeled transitions. The label of a transition is called an *action*. An action is an element of an infinite set of names and co-names. A name is usually denoted with letters $a, b, ...$, but may also be chosen to be easy to read (*e.g.*, tea, coffee, token). A co-name is a name with a line over it (*e.g.*, $\overline{\text{tea}}$, $\overline{\text{coffee}}$, $\overline{\text{token}}$). There is an infinite set of names $\mathcal{N}$ and co-names $\overline{\mathcal{N}}$ which make up the set of labels $\mathcal{L} = \mathcal{N} \cup \overline{\mathcal{N}}$. For example, the transition $P \xrightarrow{\alpha} Q$ indicates that process P transitions to process Q on the action α.

An important feature of CCS is the distinction between *observable* and *unobservable*

behavior. Observable actions represent process interactions that an external observer can detect. Unobservable actions are distinguished with the label τ and represent an internal step that an external observer cannot detect. A τ transition arises when two parallel components synchronize on complementary actions, say a and $\overline{a}$. The distinction between observable and unobservable behavior leads to equivalence notions that abstract away internal computation and compare processes up to their observable behavior.

The syntax for CCS processes is defined in Figure 4.1, and the transition rules are given in Figure 4.2. Note that M and N denote zero or more alternatives that are discarded.

$$
\begin{array}{rcll}
\alpha & ::= & \text{labels} & \\
P, Q & ::= & 0 & \text{Halt} \\
 & & \alpha.P & \text{Sequencing} \\
 & & \alpha_1.P + \alpha_2.Q & \text{Choice between } P \text{ and } Q \\
 & & P|Q & \text{Parallel composition of } P \text{ and } Q \\
 & & P\backslash x & \text{Restrict } x \text{ in } P \text{ where } x \in \mathcal{N}
\end{array}
$$

Figure 4.1: The CCS process syntax.

$$
\text{SUM: } M + \alpha.P + N \xrightarrow{\alpha} P \qquad \text{REACT: } \frac{P \xrightarrow{\alpha} P' \qquad Q \xrightarrow{\overline{\alpha}} Q'}{P|Q \xrightarrow{\tau} P'|Q'}
$$

$$
\text{L-PAR: } \frac{P \xrightarrow{\alpha} P'}{P|Q \xrightarrow{\alpha} P'|Q} \qquad \text{R-PAR: } \frac{Q \xrightarrow{\alpha} Q'}{P|Q \xrightarrow{\alpha} P|Q'}
$$

$$
\text{RES: } \frac{P \xrightarrow{\alpha} P'}{P\backslash a \xrightarrow{\alpha} P'\backslash a} \text{ if } \alpha \notin \{a, \overline{a}\}
$$

Figure 4.2: The CCS reaction rules.

The processes and labeled transitions we use in this chapter do not require process abstraction that is commonly found in other presentations of CCS. The SUM transition indicates a transition on α from $\alpha.P$ to P that discards any choices that may exist. The REACT transition represents two processes P and Q that transition to P' and Q' respectively on a name and corresponding co-name. The system composing the processes in parallel takes an unobservable step. The L-PAR and R-PAR transitions represent parallel processes taking

independent steps. The RES transition represents an observable transition in a restricted process. The transition remains observable as long as the action is not restricted.

The distinction between observable and unobservable transitions allows for the definition of observational equivalence. Two processes with different internal behavior may be considered equivalent when a weak bisimulation exists between them. A weak simulation is a binary relation $\mathcal{S}$ between processes P and Q that holds under the following two conditions:

1. If $P \rightarrow P'$ then there exists Q' such that $Q \rightarrow^* Q'$ and $P'\mathcal{S}Q'$
   where $\rightarrow$ is a single τ-transition, and $\rightarrow^*$ is zero or more τ-transitions.

2. If $P \xrightarrow{\lambda} P'$ then there exists Q' such that $Q \overset{\lambda}{\Rightarrow} Q'$ and $P'\mathcal{S}Q'$
   where $\lambda \neq \tau$ and $\overset{\lambda}{\Rightarrow}$ is a λ-transition with zero or more τ-transitions before and after.

A weak bisimulation is a binary relation $\mathcal{S}$ where both it and its converse $\mathcal{S}^{-1}$ are weak simulations. Two processes are called observationally equivalent if there exists a weak bisimulation between them.

In this chapter we do not embed Faustus into CCS, but adopt the viewpoint of labeled transitions directly on the operational semantics of Faustus. From this perspective, the observable labels of the system are deposits, choices, and notifications, while internal reduction steps correspond to τ-transitions. In addition, this chapter uses a notion of restriction inspired by CCS where a set of actions may be designated as unobservable so that transitions labeled with those actions become τ-transitions. In practice, restriction in Faustus represents a transaction submission application that will synchronize on the restricted Faustus action.

4.2 Contributions in This Chapter

This chapter describes a methodology for verifying equivalence of Faustus contracts based on bisimulation [7, 68] from the perspective of parties interacting with the contract. The work presented in this chapter differs from previous work in the following ways:

1. We have developed a methodology based on Milner's Calculus of Communicating Systems (CCS) [25,68,69] to prove the *observational equivalence* of contracts, *i.e.* to prove

that Faustus contracts are semantically equivalent from the perspective of the parties interacting with the contract. Other work that we are aware of is: limited by the types of contracts that can be verified; verifies properties of individual contracts; or develops tools to analyze safety and liveness properties of specified contracts.

2. In Faustus, reasoning using CCS-based techniques is done directly on well-formed code of the smart contract. There is no translation from the smart contract code to a separate model for verification. In many methodologies the first step is an unverified translation from a contract/program to a model suitable for formal analysis.

3. In our formal model, Faustus processes include a set of actions that, when executed, are hidden from users. Following Milner [68], we call this collection a restriction set. The behavior of a Faustus process with a nonempty restriction set corresponds to a parallel composition of processes. In the process defined by the parallel composition, actions in the restriction set are unobservable.

4. Our methodology for verifying two processes are equivalent is to prove that the two are observationally equivalent by exhibiting the appropriate bisimulation between the two. We describe a program (contract) transformation, and apply this methodology to formally prove that an arbitrary contract and the resulting transformed contract are observationally equivalent.

5. The work described here has been formalized in the Isabelle/HOL interactive theorem prover. We provide proofs for theorems and lemmas in this chapter where possible, and give a more informal outline if the proof relies on theorems outside of the scope of this work.

4.3 Equivalent Contracts

A central question that arises when thinking about computational systems is, when do two systems have equivalent behavior? Quoting Milner [27], *"Until we know what constitutes*

similarity or difference of behavior we cannot claim to know what 'behavior' means - and if that is the case then we have no precise way of explaining what our systems do."

Thinking from first principles to answer what it might mean for a pair of processes, P and Q, to be equivalent[1], if we are very clever, we might come up with something like:

> *Processes P and Q are* observationally equivalent *if for every sequence of actions (say s), applying s to P results in state P', then there is a state Q' that results from applying s to Q, and P' and Q' are themselves equivalent; and vice versa.*

This idea has served as the basis of a whole host of formalisms for reasoning about processes described as labeled transition systems.

It is convenient to use some notation for this relation. We write $P \approx Q$ to mean that P and Q are observationally equivalent. To express the idea that a sequence of actions (say s) is *applied* to a process P yielding P', we write $P \overset{s}{\Rightarrow} P'$. With this notation we can make the definition more precise as follows:

$P \approx Q$ if, and only if, for all sequences of actions s

i.) Whenever $P \overset{s}{\Rightarrow} P'$ then, for some Q', $Q \overset{s}{\Rightarrow} Q'$ and $P' \approx Q'$.

ii.) Whenever $Q \overset{s}{\Rightarrow} Q'$ then, for some P', $P \overset{s}{\Rightarrow} P'$ and $P' \approx Q'$.

In the rest of this section, we describe the formalization of this idea in a form suitable for proving the equivalence of Faustus processes.

4.3.1 Faustus Processes

The operational semantics of Faustus is formalized in Isabelle/HOL [72] and can be found in Chapter 3. The small step semantics of Faustus are given as an inductive relation over configurations which consist of a Faustus contract together with the Faustus state. The small step semantics define an evaluator for Faustus. Configurations correspond to CCS processes. Transitions in the small step semantics are either observable, and labeled with an action; or

[1] This is exactly what Milner [6, 70], Hoare [71], and Park [7] did.

unobservable, and labeled with a τ. Additionally, Faustus processes contain a set of actions that become restricted and unobservable to the user. This formalization serves as the basis of our development of observational equivalence.

Definition 4.1 (Blockchain Primitives)**.** $\boldsymbol{String}$ is the set of all strings. $\boldsymbol{Party}$ is the set of all blockchain addresses. $\boldsymbol{Curr}$ is the set of all blockchain currencies or kinds of tokens.

Definition 4.2 (Labeled Transition System)**.** A labeled transition system (LTS) is a triple $\langle \mathcal{S}, \mathcal{L}, \Sigma \rangle$ where $\mathcal{S}$ is a set of states, $\mathcal{L}$ is a set of labels, and $\Sigma \subseteq (\mathcal{S} \times \mathcal{L} \times \mathcal{S})$ is a transition relation (see *e.g.* [68]).

We now provide definitions with the end result of defining an LTS for Faustus processes. The states of the Faustus LTS will be the contract, its execution state, and a restriction set. The transition relation is defined by the operational semantics of Faustus. The labels will be actions paired with a time window when they are generated.

Definition 4.3 (Faustus Execution $\boldsymbol{State}$)**.**

$$\boldsymbol{State} = \langle \boldsymbol{Context}, \boldsymbol{Env}, \boldsymbol{Acct}, \boldsymbol{Choice}, (\mathbb{Z}, \mathbb{Z}), [\boldsymbol{Payment}], [\boldsymbol{Warning}] \rangle$$

Where $\boldsymbol{Context}$ is a typing context (a map of variable names to types); $\boldsymbol{Env}$ is an environment (a map of variable names to their meanings); $\boldsymbol{Acct}$ are account balances (a map of elements of $(\boldsymbol{Party}, \boldsymbol{Curr})$ pairs to integers); $\boldsymbol{Choice}$ are the choices that have been made (a map of choice name and party pairs to integers); the start and end times of the current transaction window are given by a pair of integers interpreted as POSIX Times; $[\boldsymbol{Payment}]$ is the history of payments; and $[\boldsymbol{Warning}]$ is the history of emitted warnings. We use variables $\{\sigma, \sigma_1, \sigma_2, \cdots\}$ to denote arbitrary execution states.

In practice, we use a small-step semantics to define the transition relation used to evaluate Faustus processes (see Definition 4.8). The small step semantics manage the actions in the restriction set to make them unobservable to parties observing the progress of a running contract.

There are three ways to interact with a contract running on the blockchain. They are the choice, deposit, and notify actions.

A choice action allows a party to provide an integer value for a specific choice, labeled by a string. For each choice action $\langle \mathsf{choice}, p, c, x \rangle$ encountered during the evaluation of a contract, the choice-party pair is mapped to the integer value x in the execution state (see Definition 4.3). This data can be referred to later, both as part of the logic of the contract itself and by other parties, to guide future behavior. The party and choice in the user action must match a party and choice specified in a *choice guard* of the current when. In a choice guard, choice values are specified to be in a list of range values, $[[y_1, z_1], \cdots [y_k, z_k]]$. A guard is triggered if, for the particular choice value x, $y_i \leq x \leq z_i$ for some $i \in \{1..k\}$.

A deposit action $\langle \mathsf{deposit}, p_1, x, c, p_2 \rangle$ specifies that party p_1 (recall, strictly speaking, parties are addresses on the blockchain[2]) deposits x tokens of currency c into party p_2's account in the contract. During the execution of a contract, the contract itself holds the deposited funds. These funds may be paid back out of the contract to a party (an address on the blockchain) as specified by the logic of the contract. Deposit actions of the form $\langle \mathsf{deposit}, p_1, x, c, p_2 \rangle$ are only successful if the parties, amount of the deposit and the currency match the deposit guard exactly. Funds internal to a contract are tracked in the $\boldsymbol{Acct}$ map of the execution state.

The notify action allows parties to *notify* a contract that it can proceed with execution. Notify guards are of the form notify b -> $\{\mathcal{C}\}$ where b is a Boolean expression and $\mathcal{C}$ is the next contract to evaluate if the expression b evaluates to true in the current execution state. For a notify action to trigger a notify guard, the expression b must evaluate to true.

Definition 4.4 (User Actions)**.** There are three types of user actions; choice, deposit, and notify. We define the sets $\boldsymbol{Choice}$, the set of all choice actions; $\boldsymbol{Dep}$, the set of all deposit actions; and $\boldsymbol{Notif}$, the set of all notify actions as follows:

$$\begin{aligned}
\boldsymbol{Choice} &\stackrel{\text{def}}{=} \{\langle \mathsf{choice}, p, c, x \rangle \mid p \in \boldsymbol{Party}, c \in \boldsymbol{String}, x \in \mathbb{Z}\} \\
\boldsymbol{Dep} &\stackrel{\text{def}}{=} \{\langle \mathsf{deposit}, p_1, x, c, p_2 \rangle \mid p_1 \in \boldsymbol{Party}, x \in \mathbb{Z}, c \in \boldsymbol{Curr}, p_2 \in \boldsymbol{Party}\} \\
\boldsymbol{Notif} &\stackrel{\text{def}}{=} \{\mathsf{notify}\}
\end{aligned}$$

We define $\boldsymbol{Act} \stackrel{\text{def}}{=} \boldsymbol{Choice} \cup \boldsymbol{Dep} \cup \boldsymbol{Notif}$.

[2]Cardano is based on an extended UTXO (Unspent Transaction Outputs) [73] model. Faustus uses an account based model built on top of the EUTXO model of the Cardano blockchain.

Faustus processes are triples containing the code of the contract that is executing, the state of the executing contract, and a *restriction set*, a collection of actions that are *unobservable*[3].

Definition 4.5 (Faustus Processes)**.** Let $\boldsymbol{Contract}$ be the set of all well-formed Faustus contracts. Then we define the set of Faustus processes, $\boldsymbol{Proc}$, to be the set of all pairs of Faustus contracts and execution states restricted by an action set $\mathcal{A}$:

$$\boldsymbol{Proc} \stackrel{\text{def}}{=} \{\langle \mathcal{C}, \sigma \rangle \backslash \mathcal{A} \mid \mathcal{C} \in \boldsymbol{Contract}, \sigma \in \boldsymbol{State}, \mathcal{A} \subseteq \boldsymbol{Act}\}$$

Note that we write $\langle \mathcal{C}, \sigma \rangle$ when $\mathcal{A}$ is empty instead of $\langle \mathcal{C}, \sigma \rangle \backslash \{\}$, also, we write P (or P_1, P', Q, $\cdots$ etc.) to denote arbitrary elements of $\boldsymbol{Contract} \times \boldsymbol{State}$ if otherwise unspecified.

Full transition labels include an action and a time window indicated by a start t_1 and end time t_2, we use $\mathit{tt} = \langle t_1, t_2 \rangle$ to denote an arbitrary time window $\langle t_1, t_2 \rangle$. Due to the latency between a transaction being submitted and evaluated on the blockchain, actions are expected to be processed within the provided time window tt, and are rejected otherwise. The times in tt are generated externally and provided in the stream of input to a running process. Typically, the time window provided allows more than enough time to be evaluated by the blockchain. These times are used to update the start and end time components of the $\boldsymbol{State}$ according to the Faustus semantics.

Definition 4.6 (Labels)**.** When a Faustus process makes a transition, that transition is labeled to indicate the external/user action, a, and a time window, tt, during which the transaction containing the action is processed. The ϵ action allows a transition where time changes without an explicit user action. The τ label indicates an unobservable transition; a transition that does not require an external action. We define the set of all labels, $\boldsymbol{Label}$, as follows:

$$\boldsymbol{Label} \stackrel{\text{def}}{=} \{\langle a, \mathit{tt} \rangle \mid a \in \boldsymbol{Act} \cup \{\epsilon\},\ \mathit{tt} \in \mathbb{Z} \times \mathbb{Z}\} \ \cup \{\tau\}$$

[3]For readers familiar with Milner and Hoare style process algebras this is similar to restriction.

Definition 4.7 (Composition of Transition Relations)**.** If $\mathcal{R} \subseteq A \times C$ and $\mathcal{S} \subseteq C \times B$ we write $\mathcal{R} \cdot \mathcal{S}$ to denote the relation on $A \times B$ constructed by (ordinary) composition of relations.

$$\mathcal{R} \cdot \mathcal{S} \stackrel{\text{def}}{=} \{\langle x, y\rangle \in (A \times B) \,|\, \exists z \in C.\, x\mathcal{R}z \wedge z\mathcal{S}y\}$$

Definition 4.8 (Faustus Transition Relation)**.** We write $\langle \mathcal{C}, \sigma\rangle \backslash \mathcal{A} \xrightarrow{\lambda} \langle \mathcal{C}', \sigma'\rangle \backslash \mathcal{A}'$ to denote transitions restricted by $\mathcal{A}$ where $\mathcal{A}, \mathcal{A}' \subseteq \boldsymbol{Act}$. Under the small step semantics of Faustus[4], contract $\mathcal{C}$ in state σ with restricted actions $\mathcal{A}$ transitions on input (label) λ to contract $\mathcal{C}'$, with state σ', and restriction set $\mathcal{A}'$. Labels $\langle a, \mathit{tt}\rangle \in \mathcal{A} \times \mathbb{Z}^2$ are hidden in the sense that transitions made on labels containing actions in $\mathcal{A}$ are not observable[5]. When $\mathcal{A} = \mathcal{A}'$ (and it always will be in this chapter) we write $\langle \mathcal{C}, \sigma\rangle \backslash \mathcal{A} \xrightarrow{\lambda} \langle \mathcal{C}', \sigma'\rangle \backslash \mathcal{A}$. In the case where $P\backslash\mathcal{A} \xrightarrow{\lambda_1} R_1\backslash\mathcal{A}$, $R_1\backslash\mathcal{A} \xrightarrow{\lambda_2} R_2\backslash\mathcal{A}$, and so on, up to $R_{k-1}\backslash\mathcal{A} \xrightarrow{\lambda_k} Q\backslash\mathcal{A}$, we write $P\backslash\mathcal{A} \xrightarrow{\lambda_1 \cdots \lambda_k} Q\backslash\mathcal{A}$ (instead of $P\backslash\mathcal{A}(\xrightarrow{\lambda_1} \cdots \xrightarrow{\lambda_k})Q\backslash\mathcal{A}$) to denote a transition in the relation formed by the composition of $\xrightarrow{\lambda_1} \cdots \xrightarrow{\lambda_k}$. In the case where there is a vector of actions, $\vec{\alpha} = \langle \alpha_1, \cdots, \alpha_k\rangle$, occurring sequentially with the same time window, we write $P\backslash\mathcal{A} \xrightarrow{\langle \vec{\alpha}, \mathit{tt}\rangle} Q\backslash\mathcal{A}$ instead of $P\backslash\mathcal{A} \xrightarrow{\langle \alpha_1, \mathit{tt}\rangle \cdots \langle \alpha_k, \mathit{tt}\rangle} Q\backslash\mathcal{A}$. The full transition relation is defined by the following equations:

1.) $P\backslash\mathcal{A} \xrightarrow{\langle b, \mathit{tt}\rangle}_0 Q\backslash\mathcal{A} \stackrel{\text{def}}{=} \{\langle P\backslash\mathcal{A}, \langle b, \mathit{tt}\rangle, Q\backslash\mathcal{A}\rangle \,|\, b \notin \mathcal{A} \wedge P \xrightarrow{\lambda} Q\}$
2.) $P\backslash\mathcal{A} \xrightarrow{\langle b, \mathit{tt}\rangle}_k Q\backslash\mathcal{A} \stackrel{\text{def}}{=} \{\langle P\backslash\mathcal{A}, \langle b, \mathit{tt}\rangle, Q\backslash\mathcal{A}\rangle \,|\, k > 0 \wedge b \neq \epsilon \wedge b \notin \mathcal{A} \wedge \exists \vec{\alpha} \in \mathcal{A}^k. P \xrightarrow{\langle \vec{\alpha} \cdot b, \mathit{tt}\rangle} Q\}$
3.) $P\backslash\mathcal{A} \xrightarrow{\langle b, \mathit{tt}\rangle} Q\backslash\mathcal{A} \stackrel{\text{def}}{=} \bigcup_{k \in \mathbb{N}} \{P\backslash\mathcal{A} \xrightarrow{\langle b, \mathit{tt}\rangle}_k Q\backslash\mathcal{A}\}$
4.) $P\backslash\mathcal{A} \xrightarrow{\tau} Q\backslash\mathcal{A} \quad \stackrel{\text{def}}{=} \{\langle P\backslash\mathcal{A}, \tau, Q\backslash\mathcal{A}\rangle \,|\, P \xrightarrow{\tau} Q\}$

Note that $\xrightarrow{\lambda}^*$ denotes the reflexive transitive closure of $\xrightarrow{\lambda}$, *i.e.* it is the relation defined by the composition of any number of $\xrightarrow{\lambda}$ relations including 0.

The Faustus transition relation allows an arbitrary number of hidden actions from the restriction set to occur before an observable transition. In this way, the restriction set not only prevents the users from sending actions in the set, but it also corresponds to a process

[4]Formally, in the Faustus semantics, $\langle \mathcal{C}, \sigma\rangle \xrightarrow{\lambda} \langle \mathcal{C}', \sigma'\rangle$ is defined by $[\![\Gamma, \mathcal{A} \vdash \mathcal{C} : \mathsf{contract}]\!]\, \sigma\, \lambda = \langle \mathcal{C}', \sigma'\rangle$.

[5]In CCS [68] unobservable τ-transitions arise by the synchronization of a name (a label) say λ and its co-name $\bar{\lambda}$.

running concurrently with the Faustus smart contract that can send any of the actions in the restriction set. Regular τ-transitions and timeout transitions remain unchanged by the restriction set.

Definition 4.9 (Faustus Labeled Transition System)**.** The Faustus LTS is defined as the triple $\langle \boldsymbol{Proc}, \boldsymbol{Label}, \rightarrow \rangle$. The states of the Faustus transition relation are elements of $\boldsymbol{Proc}$. The labels of the Faustus transition system are elements of $\boldsymbol{Label}$. The transition relation $\rightarrow$ is defined in Definition 4.8, and given by the small step semantics of Faustus; the details of which have been formalized in Isabelle/HOL [72].

Typically, users interact with contracts using an empty set of restricted actions. Non-empty sets of restricted actions allow for traversing different contract structures through multiple transactions without requiring additional user interaction. This will be explained in more detail in Section 4.4.

With the definition of the Faustus transition relation, we also define the experiment relation between Faustus processes on sequences of labels.

Definition 4.10 (Experiment Relation)**.** Let $s = \lambda_1 \cdots \lambda_n \in \boldsymbol{Label}^*$. We define the experiment relation $\overset{s}{\Longrightarrow}$ as follows:

$$\Longrightarrow \overset{\texttt{def}}{=} \xrightarrow{\tau}{}^*$$

$$\overset{s}{\Longrightarrow} \overset{\texttt{def}}{=} \Longrightarrow \cdot \xrightarrow{\lambda_1} \cdot \Longrightarrow \cdots \Longrightarrow \cdot \xrightarrow{\lambda_n} \cdot \Longrightarrow$$

Thus, $\overset{s}{\Longrightarrow}$ is the behavior that allows an arbitrary number of τ actions before, between, and after the observable actions in s.

4.3.2 Observational Equivalence

A *bisimulation* is a binary relation between states of processes. It identifies states that cannot be distinguished from one another by any sequence of observable actions. From the perspective of a user interacting with two bisimilar processes, the same inputs result in the same outputs for both processes. The user cannot tell the difference between the two processes by interacting with them. If two processes, P and Q, are bisimilar, we write $P \approx Q$. It turns out that $\approx$ is an equivalence relation [69].

Definition 4.11 (Weak Simulation)**.** Let $\mathscr{F} = \langle \boldsymbol{Proc}, \boldsymbol{Label}, \rightarrow \rangle$ be the Faustus LTS, and let $\mathcal{R} \subseteq (\boldsymbol{Proc} \times \boldsymbol{Proc})$ be a binary relation, and $s \in \boldsymbol{Label}^*$. $\mathcal{R}$ is a *weak simulation* over $\mathscr{F}$ when the following property holds:

If $(P_1, Q_1) \in \mathcal{R}$ and $P_1 \overset{s}{\Rightarrow} P_2$ then there exists $Q_2 \in \boldsymbol{Proc}$ such that $Q_1 \overset{s}{\Rightarrow} Q_2$ and $(P_2, Q_2) \in \mathcal{R}$

This definition leads to the definitions of weak bisimulation and observational equivalence.

Definition 4.12 (Weak Bisimulation, Observational Equivalence)**.** Let $\mathscr{F} = \langle \boldsymbol{Proc}, \boldsymbol{Label}, \rightarrow \rangle$ be the Faustus LTS, and let $\mathcal{R} \subseteq (\boldsymbol{Proc} \times \boldsymbol{Proc})$ be a binary relation. Then $\mathcal{R}$ is a *weak bisimulation* over $\mathscr{F}$, if it and its converse are both weak simulations. We say P and Q are *observationally equivalent*, written $P \approx Q$, if there exists a weak bisimulation $\mathcal{R}$ such that $(P, Q) \in \mathcal{R}$.

Processes are called observationally equivalent when there is a bisimulation between them. The number of unobservable transitions may differ when evaluating two observationally equivalent processes on the same observable transitions. We have proved in Isabelle/HOL that bisimulations on Faustus processes are equivalence relations, and that simulations are closed under union. Appendix C contains the Isabelle formalization of Faustus processes and bisimulation. The proofs of these properties are well understood, and can be found in multiple works by Milner [25, 27, 68, 69].

With the definition of observational equivalence, we can now prove transformations of Faustus contracts do not modify their observed behaviors. The next section describes a common transformation that is applied to Faustus contracts, and proves the result of the transformation is observationally equivalent to the original.

4.4 Contract Transformations

In each round of computation on the blockchain, Faustus programs run until they pause for external input at a `when` contract, or are closed. The computation time between each pause may take longer than the maximum computation time allowed per transaction on the

P

```
when {a1 -> c1,
      ⋮
      an -> cn
} after t -> { close }
```

Q $[\mathtt{k} < \mathtt{n}]$

```
when {a1 -> c1
      ⋮
      ak -> ck
      p chooses fresh within [0, 0] -> {
          when {ak1 -> ck1
                ⋮
                an -> cn
          } after t -> { close }
    }
} after t -> { close }
```

Figure 4.3: A contract (P) and the result (Q) of recursively applying the transformation.

blockchain, leading to out of gas errors [74]. If an out of gas error occurs, the contract will revert back to its state prior to that round of computation. Reverting to the prior state can result in code that never progresses while still generating gas costs.

Consider the contract on the left side of Figure 4.3. Assume that in each round of computation there is only enough time to check $k + 1$ guards before an out of gas error occurs. Then there are $n - (k + 1)$ guard continuations that are unreachable when running the contract on the blockchain. On the right side of Figure 4.3, the sequence of n guards has been replaced by a length $k + 1$ sequence of guards, the last of which is a fresh choice guard that cascades to a when containing the rest of the transformed contract. We have colored this section blue to indicate it will be recursively transformed.

To allow every guarded command to be reachable, programmers apply this transformation where the list of guarded commands is truncated, and moved into a sub-contract that runs after a "fresh" choice action is taken. The key aspect is selecting a choice name that does not appear in the original contract, *i.e.* it is fresh, hence (p chooses *fresh* within $[0, 0]$)[6] is a choice guard that does not occur in the original contract. The result of the transformation applied to P in Figure 4.3 is Q in the same figure. As we will see below, the "fresh" choice allows for a transition that leads directly to a when contract. Since when contracts pause execution until the next round of blockchain computation, the out of gas error is avoided,

[6]The range `[0,0]` simply means `p` must choose `0`.

and the rest of the guard continuations can be reached in the next round of computation.

This type of transformation has been informally described by Bush [75] as a best practice for Marlowe contracts with complex logic. The Isabelle function *sc* in Listing 4.1, which takes a *ChoiceName* and acts on the Faustus abstract syntax tree from Chapter 3, formally describes the transformation.

Listing 4.1: Isabelle implementation of the transformation algorithm.

```
function sc :: "ChoiceName ⇒ FContract ⇒ FContract" where
"sc cn (When cases t cont) = (
  let
    newCases = map (λ(Case g c) ⇒ (Case g (sc cn c))) cases;
    newCont = sc cn cont
  in
  if (rcmdk > 1 ∧ length newCases > rcmdk)
  then (When (take rcmdk newCases @
    [Case (ActionGuard (Choice (FChoiceId cn advP) [(0, 0)]))
      (sc cn (When (drop rcmdk cases) t cont))])
    t newCont)
  else (When newCases t newCont))" |
"sc cn (Close) = Close" |
"sc cn (StatementCont s c) = (StatementCont s (sc cn c))" |
"sc cn (If e c1 c2) = (If e (sc cn c1) (sc cn c2))" |
"sc cn (Let i v c) = (Let i v (sc cn c))" |
"sc cn (LetObservation i v c) = (LetObservation i v (sc cn c))" |
"sc cn (LetParty i v c) = (LetParty i v (sc cn c))" |
"sc cn (LetC i p b c) = (LetC i p (sc cn b) (sc cn c))" |
"sc cn (UseC i a) = (UseC i a)"
```

Definition 4.13 (Transformation Constants)**.** The transformation function in Listing 4.1 has two constants associated with it: *advP*, meaning "advancing party"; and *rcmdk*, meaning "recommended k". The first one, *advP*, is an arbitrary ***Party*** that will be allowed to send signals containing the $\langle \mathsf{choice}, p, c, 0\rangle$ action that traverses the cascading when contracts; and *rcmdk* is a natural number representing the maximum number of guarded commands before a split into cascading when contracts should be performed. These constants can take on any values of the appropriate type.

Evaluation of different guard expressions may take different amounts of time; thus, experimentation may be required to determine the safe value for *rcmdk* for a specific contract.

Definition 4.14 (Transformed Contracts)**.** Given a contract C, and choice name c, we use the notation $\ulcorner C \urcorner_c$ to denote the result of applying the transformation (`sc c`) to C. We also overload the notation for $\ulcorner \langle C, \sigma \rangle \urcorner_c$ to denote application of the transformation to C as well as all contracts mentioned in the environment in σ. See Definition 4.3 for the definition of the environment in σ.

In the example from Figure 4.3, $Q = \ulcorner P \urcorner_{fresh}$. To verify this transformation, we must show that there is a weak bisimulation between them for an arbitrary restriction set $\mathcal{A}$, *i.e.* that $\langle P, \sigma \rangle \backslash \mathcal{A} \approx \ulcorner \langle P, \sigma \rangle \urcorner_{fresh} \backslash (\mathcal{A} \cup \{\langle \mathsf{choice}, p, fresh, 0 \rangle\})$.

4.4.1 Restriction - Making Observable Actions Unobservable

The strategy of the transformation is to transform a when contract having a long list of guarded commands into a new contract having a structure of cascading when contracts whose guarded command lists are length less than or equal to $k + 1$. To accomplish this, we introduce a choice action with a choice name that does not occur in the original contract called c. The guard in the contract takes the form (p chooses c within $[0, 0]$), so that inputs of the form $\langle \langle \mathsf{choice}, p, c, 0 \rangle, tt \rangle$ induce a transition.

Definition 4.15 (Restricted Choice Action)**.** Given a choice name $c \in$ ***String*** we define the choice action, $\alpha_c \stackrel{\text{def}}{=} \langle \mathsf{choice}, p, c, 0 \rangle$.

The action α_c is used to traverse the cascading when structure introduced by the transformation. As long as c does not occur as a choice name in P, α_c can safely be used to internally transition along the chain of cascading whens introduced by sc in $\ulcorner P \urcorner_c$ (see Listing 4.1). Adding α_c to the restriction set hides those transitions when evaluating $\ulcorner P \urcorner_c$. This allows us to show that there is a bisimulation between $P \backslash \mathcal{A}$ and $\ulcorner P \urcorner_c \backslash (\mathcal{A} \cup \{\alpha_c\})$.

4.4.2 Verifying the Transformation

To be a candidate for the transformation requires a kind of predictability in the behavior of the contract to be transformed. Specifically, the guards in every when having more than k

```
when {
    a -> c1,
    a -> c2
} after t -> { close }
```

```
when {
    a -> c1,
    p chooses fresh within [0, 0] -> {
        when {
            a -> c2
        } after t -> { close }
    }
} after t -> { close }
```

Figure 4.4: A contract without disjoint guards before and after applying the transformation.

guarded commands must be disjoint, in the sense that at most one guard evaluates to true in any state-action pair.

The example contracts in Figure 4.4 demonstrate the issue that occurs if the guards are not disjoint. In the original contract on the left there is no way to get to the `c2` continuation. This is because the guards in a `when` are evaluated in the order they appear in the list. If the guarded command (`a->c1`) is triggered, then (`a->c2`) would have been triggered as well. (`a->c1`) will always continue to `c1` before (`a->c2`) is ever evaluated. On the other hand, the transformed contract on the right can reach both `c1` and `c2`. If the label $\langle a, \not{t} \rangle$ reaches `c1`, then the sequence $\langle \alpha_{fresh} \cdot a, \not{t} \rangle$ reaches `c2`, which was previously unreachable. Also note that α_{fresh} is hidden, so to any observer, the labels $\langle \alpha_{fresh} \cdot a, \not{t} \rangle$ and $\langle a, \not{t} \rangle$ appear to be identical. As this example shows, the transformed contract may exhibit more behaviors than the original contract, and there is no evaluation of the original contract that will simulate those behaviors. By requiring the guards to be disjoint, we guarantee that there is no unreachable code uncovered by the transformation. In practice, most contracts are written to have disjoint guards. For example, the escrow contract in Chapter 3 has disjoint guards since all choice guards in the `when` are for different parties or allow different choices of values for the vote, and there is only one deposit guard.

Definition 4.16 (Disjoint Guards)**.** A contract-state pair $\langle C, \sigma \rangle$ is *disjoint* if, for each contract in $\langle C, \sigma \rangle$ of the form (`when` gs `after` t `->` $cont$), the list of guarded commands gs is disjoint. gs is disjoint when, for all l and r such that $gs = l$ ++ r, the following two properties hold for all actions in all execution states:

i.) If l contains a guard that evaluates to true then all guards in r evaluate to false.

ii.) If r contains a guard that evaluates to true then all guards in l evaluate to false.

Next, we define a binary relation between original processes and their transformed counterparts, and prove it is a bisimulation.

Definition 4.17 (Split Guards Relation)**.** For all $c \in \boldsymbol{String}$ let $\mathbb{SG}_c$ be the following binary relation over $\boldsymbol{Proc}$:

$$\begin{aligned}\mathbb{SG}_c \stackrel{\text{def}}{=} \{ & \langle \langle C, \sigma_1 \rangle \backslash \mathcal{A}, \ulcorner \langle C, \sigma_2 \rangle_c \urcorner \backslash (\mathcal{A} \cup \{\alpha_c\}) \rangle \\ & \mid c \text{ fresh in } \langle C, \sigma_1 \rangle, \text{ disjoint } \langle C, \sigma_1 \rangle, \\ & \quad \sigma_1 = \langle \Gamma,\, e,\, accts,\, choices_1,\, t,\, ps,\, asrt \rangle, \\ & \quad \sigma_2 = \langle \Gamma,\, e,\, accts,\, choices_2,\, t,\, ps,\, asrt \rangle, \\ & \quad \forall x \in \boldsymbol{String}, p \in \boldsymbol{Party} \,.\, x \neq c \text{ implies } choices_1[(x,p)] = choices_2[(x,p)] \ \}\end{aligned}$$

The binary relation $\mathbb{SG}_c$ is a relation in $\boldsymbol{Proc} \times \boldsymbol{Proc}$, where the second element is the result of applying the transformation function (`sc c`) to the first element. Also note, the original contract must have disjoint guarded command lists; and the Faustus execution states in the two elements of the relation are equivalent up to the choices for the new choice name c. We will use $\mathbb{SG}_c$ to show a bisimulation between an original contract and a transformed contract under the condition that the choice name used in the transformation is not in the original contract.

Lemma 4.1. *For all $c \in \boldsymbol{String}$, $\mathbb{SG}_c$ is a weak simulation over $(\boldsymbol{Proc}, \boldsymbol{Label}, \rightarrow)$.*

Lemma 4.2. *For all $c \in \boldsymbol{String}$, $\mathbb{SG}_c^{-1}$ is a weak simulation over $(\boldsymbol{Proc}, \boldsymbol{Label}, \rightarrow)$.*

Proof. The proofs of Lemmas 4.1 and 4.2 are done in Isabelle/HOL by induction on the structure of the experiment relation ($\Rightarrow$), and by cases on the structure of the transition relation ($\longrightarrow$). Assume membership in the relation ($\mathbb{SG}_c$ or $\mathbb{SG}_c^{-1}$) and an arbitrary transition for the first member of the relation. Then for each case of the transition relation, we show the second member of the relation will transition on an experiment containing the same label to another element of $\boldsymbol{Proc}$ that maintains membership in the relation ($\mathbb{SG}_c$ or $\mathbb{SG}_c^{-1}$). $\square$

Theorem 4.3 (Transformation Verification)**.** *Let* $\langle C, \sigma_1 \rangle \backslash \mathcal{A}, \ulcorner \langle C, \sigma_2 \rangle_c \urcorner \backslash (\mathcal{A} \cup \{\alpha_c\}) \in \boldsymbol{Proc}$ *be states in the Faustus LTS* $(\boldsymbol{Proc}, \boldsymbol{Label}, \rightarrow)$*, for some arbitrary* $c \in \boldsymbol{String}$ *such that:*

i.) *c is fresh for C and σ_1,* i.e. *c does not occur as a choice name in either.*

ii.) *Guarded command lists in $\langle C, \sigma_1 \rangle$ are disjoint.*

iii.) $\sigma_1 = \langle \Gamma,\ e,\ accts,\ chs_1,\ t,\ ps,\ asrt \rangle$.

iv.) $\sigma_2 = \langle \Gamma,\ e,\ accts,\ chs_2,\ t,\ ps,\ asrt \rangle$.

v.) $\forall x \in \boldsymbol{String}, r \in \boldsymbol{Party}. x \neq c$ *implies* $chs_1[(x, r)] = chs_2[(x, r)]$.

Then $\langle C, \sigma_1 \rangle \backslash \mathcal{A} \approx \ulcorner \langle C, \sigma_2 \rangle_c \urcorner \backslash (\mathcal{A} \cup \{\alpha_c\})$.

Proof. Assume an arbitrary $c \in \boldsymbol{String}$, $\langle C, \sigma_1 \rangle \backslash \mathcal{A}, \ulcorner \langle C, \sigma_2 \rangle_c \urcorner \backslash (\mathcal{A} \cup \{\alpha_c\}) \in \boldsymbol{Proc}$, and that (***i.***) through (***v.***) hold. From Lemma 4.1, Lemma 4.2, and Definition 4.12 of bisimulation, we can conclude that $\mathbb{SG}_c$ is a bisimulation. Then by the definition of $\mathbb{SG}_c$, we know that $\langle \langle C, \sigma_1 \rangle \backslash \mathcal{A}, \ulcorner \langle C, \sigma_2 \rangle_c \urcorner \backslash (\mathcal{A} \cup \{\alpha_c\}) \rangle \in \mathbb{SG}_c$. Thus $\langle C, \sigma_1 \rangle \backslash \mathcal{A} \approx \ulcorner \langle C, \sigma_2 \rangle_c \urcorner \backslash (\mathcal{A} \cup \{\alpha_c\})$. □

With Theorem 4.3 we have verified that contracts have the same observable behavior before and after the transformation is applied. Programmers can safely apply the transformation to their contracts using the initial execution state and empty restriction set before the contracts are loaded onto the blockchain. The conditions required to guarantee that the bisimulation is valid are: *i.)* the choice name used in the transformation is fresh, and *ii.)* the guarded command lists in the original contract are all disjoint.

4.5 Discussion

In this chapter we have described an Isabelle formalization of a bisimulation verifying that smart contracts written in Faustus and their transformed instances are observationally equivalent. The formalization introduces a notion of hidden actions in a way that differs from Milner's CCS [68]. Also, Hoare's CSP [38] has a restriction operator, but it differs in a

number of ways; perhaps most significantly, his restriction sets enumerate the allowable actions.

There are a few clear ways this work can be continued. The first is to apply this methodology to many more transformations. First among them is a formalization of a transformation, in Isabelle, that transforms an arbitrary guarded command list into one that is disjoint. A strategy for this transformation and the changes that would be needed to be made to the Faustus semantics are described in Chapter 7. Since the verification of the transformation in this chapter requires the guarded command lists in the contract to be disjoint, it would be useful for programmers to have more contracts that the transformation can be applied to.

Also, we believe these techniques can be applied to other smart contract programming languages (perhaps Solidity [20]). In Faustus, labels for the transition relation are clearly given in the syntax of the contract as guards in the guarded commands of a `when` contract. In Solidity, the `require` statements act like guards. Marmsoler and Brucker [21] have formalized the semantics of Solidity in Isabelle/HOL, which would serve as a good starting point.

Another area of investigation is applying the additional methodologies used for Timed CCS. We include time information as part of the labels in the Faustus LTS and use that information to update the ***State*** time information according to the Faustus semantics. Timed CCS variants use clock signals as labels in process transitions [76, 77]. In order to show a bisimulation between Faustus processes, the ***State*** information must be included in the relation. By moving the time information out of the ***State*** and using the clock labels that timed CCS variants use, it should reduce the complexity of the relations required to verify smart contract transformations.

Chapter 5

STILL: A Tactic-Based Theorem Prover for Dependent Session Types

5.1 Motivation

The work presented in the previous chapters provides a methodology for formalizing smart contract programming languages and verifying smart contract transformations using those formalizations. These techniques are useful for ensuring that the smart contract a programmer writes will behave as written, but they do not prevent the programmer from introducing logical errors of their own. A verified compiler or transformation can only preserve the errors that already exist in the contract.

When a smart contract contains bugs, it fails to adhere to some behavioral specification. Three observations lead us to the motivation for the work presented in this chapter: *i.)* type systems provide the ability to specify the behavior of a program, *ii.)* smart contracts can be modeled as a process calculus, and *iii.)* session types are the type system for process calculi. This chapter presents STILL, a tactic-based theorem prover for dependent session types with π-calculus term extractions. The purpose of STILL is to provide a system for specifying process behaviors, proving the validity of those specifications, and extracting processes from those proofs that follow those specifications.

In this chapter, we first provide supporting background on each piece of the theory

that STILL implements for readers who are unfamiliar. This includes background on the π-calculus which builds upon the theory of CCS from Section 4.1.1, background on linear logic and session types, background on the Extended Calculus of Constructions, and a basic introduction to implementing tactics for tactic-based theorem provers. After introducing the necessary background knowledge, we present the contributions of the STILL prover.

5.2 Theoretical Supports

5.2.1 The π-calculus

Milner's π-calculus provides a calculus for concurrent processes that allows for the mobility of communication channels [27]. Similar to CCS, processes in the π-calculus are defined by the actions they can perform and the way those actions are composed. Unlike CCS, which has processes that only send and receive signals for actions with no extra information, processes in the π-calculus send and receive communication channels. This allows the process communication topology to change during execution.

The version of the π-calculus that is discussed in this work is a π-calculus with linking, term passing, and binary guarded choice [42, 78]. Given an infinite set of names (*e.g.* u, v, x, y, z), π-calculus processes are defined by the process constructors in Figure 5.1.

P, Q	$::=$	0	Halt
		$P \mid Q$	Parallel composition of P and Q
		$(\nu\ x)P$	Restrict x in P
		$x[y].P$	Send y along x and continue with P
		$x[N].P$	Send the term N along x and continue with P
		$x(y).P$	Receive y along x and continue with P
		$!x(y).P$	Replicate, receive y along x, and continue with P
		$x.\texttt{inl}; P$	Send the choice `inl` along x, and continue with P
		$x.\texttt{inr}; P$	Send the choice `inr` along x, and continue with P
		$x.\texttt{case}(P, Q)$	Choose between P and Q along x
		$[y \leftrightarrow x]$	Link y and x
		$[x \leftarrow N]$	Send term N along x

Figure 5.1: The π-calculus syntax.

The term language used in this work is the λ-calculus defined by the Extended Calculus of Constructions [59] that can be found in Section 5.2.4. The function that returns free names in processes (`fn`) is defined in Figure 5.2.

$$
\begin{array}{lll}
\texttt{fn}(0) & = \{\} & \\
\texttt{fn}(P|Q) & = \texttt{fn}(P) \cup \texttt{fn}(Q) & \\
\texttt{fn}((\nu\ x)P) & = \texttt{fn}(P) - \{x\} & \\
\texttt{fn}(x[y].P) & = \{x, y\} \cup \texttt{fn}(P) & \\
\texttt{fn}(x[N].P) & = \{x\} \cup \texttt{fn}(P) \cup \texttt{fn}(N) & \text{see Section 5.2.4 for free names in } N \\
\texttt{fn}(x(y).P) & = \{x\} \cup \texttt{fn}(P) - \{y\} & \\
\texttt{fn}(!x(y).P) & = \{x\} \cup \texttt{fn}(P) - \{y\} & \\
\texttt{fn}(x.\texttt{inl}; P) & = \{x\} \cup \texttt{fn}(P) & \\
\texttt{fn}(x.\texttt{inr}; P) & = \{x\} \cup \texttt{fn}(P) & \\
\texttt{fn}(x.\texttt{case}(P, Q)) & = \{x\} \cup \texttt{fn}(P) \cup \texttt{fn}(Q) & \\
\texttt{fn}([x \leftrightarrow y]) & = \{x, y\} & \\
\texttt{fn}([x \leftarrow N]) & = \{x\} \cup \texttt{fn}(N) & \text{see Section 5.2.4 for free names in } N
\end{array}
$$

Figure 5.2: The π-calculus rules for free names.

Processes can evolve by applying two sets of rules. The first set of rules consists of the structural congruence rules. Processes can be structurally congruent ($\equiv$) which is the least congruence defined by the rules in Figure 5.3 [42]. The second set of rules are the reduction rules defined using the $\rightarrow$ operator. These define the internal reductions of this extended π-calculus, and can be found in Figure 5.4 [42].

1. Alpha-conversion
2. $P|0 \equiv P$, $P|Q \equiv Q|P$, $P|(Q|R) \equiv (P|Q)|R$
3. $(\nu\ x)(P|Q) \equiv P|(\nu\ x)Q$ if $x \notin \texttt{fn}(P)$
   $(\nu\ x)0 \equiv 0$, $(\nu\ x)(\nu\ y)P \equiv (\nu\ y)(\nu\ x)P$
4. $[y \leftrightarrow x] \equiv [x \leftrightarrow y]$

Figure 5.3: The π-calculus rules for structural congruence.

Due to the ability to send and receive channels in the π-calculus, it is possible to send and receive channels that have been restricted using the $(\nu\ x)P$ process constructor. For example, the process $z(a).P|(\nu\ x)z[x].Q$ sends the private x over channel z to $z(a).P$ so

$$
\begin{array}{ll}
x[y].Q|x(z).P \to Q|P\{y/z\} & x[y].Q|!x(z).P \to Q|P\{y/z\}|!x(z).P \\
x[N].Q|x(z).P \to Q|P\{N/z\} & (\nu x)([x \leftrightarrow y]|P) \to P\{y/x\} \\
(\nu x)([x \leftarrow N]|P) \to P\{N/x\} & x.\mathtt{inl}; P|x.\mathtt{case}(Q,R) \to P|Q \\
x.\mathtt{inr}; P|x.\mathtt{case}(Q,R) \to P|R & Q \to Q' \Rightarrow P|Q \to P|Q' \\
P \to Q \Rightarrow (\nu x)P \to (\nu x)Q & P \equiv P', P' \to Q, Q \equiv Q' \Rightarrow P \to Q'
\end{array}
$$

Figure 5.4: The π-calculus reduction rules.

that it may continue with $P\{x/a\}$. Both processes need access to the private x now, so the scope expands to encompass $(\nu\ x)(P\{x/a\}|Q)$. This capability of the π-calculus is known as *scope extrusion*, and it allows a process to create a new channel and control which processes may engage in communication along that channel. This will become significant in the next sections where we will use session types to specify the behavior of π-calculus processes.

5.2.2 Linear Logic

This section provides background on the logical foundation that the work in this chapter is based on. Since Girard's seminal publication [79], linear logic has provided a logical foundation for diverse computational systems, especially those that include constraints on resources. Linear logic arises when disallowing weakening and contraction rules in classical logic. Then, formerly equivalent ways of writing the rules for the connectives give rise to the *multiplicative* and *additive* connectives. For example, the two possible rules for $\wedge$ in classical logic are:

$$
\frac{\vdash A, C \qquad \vdash B, D}{\vdash A \wedge B, C, D} \qquad \frac{\vdash A, C \qquad \vdash B, C}{\vdash A \wedge B, C}
$$

These are no longer equivalent in linear logic, and now give the linear logic rules for multiplicative conjunction $\otimes$ and additive conjunction & respectively. This process can be followed to obtain the rules for multiplicative $\parr$ and additive $\oplus$ disjunction from the following rules:

$$
\frac{\vdash A, B, C}{\vdash A \vee B, C} \qquad \frac{\vdash A, C}{\vdash A \vee B, C} \qquad \frac{\vdash B, C}{\vdash A \vee B, C}
$$

Now that the multiplicative and additive connectives have been defined, we find that they admit the neutral elements 1,$\perp$,$\top$, and 0:

$$1 \otimes A = A \quad \perp \parr A = A$$
$$\top \,\&\, A = A \quad 0 \oplus A = A$$

This allows us to define the properties of linear negation for the logic so far $(\cdot)^\perp$ using De Morgan's Laws:

$$1^\perp = \perp \qquad \perp^\perp = 1 \qquad \top^\perp = 0 \qquad 0^\perp = \top$$
$$(A^\perp)^\perp = A$$
$$(A \otimes B)^\perp = A^\perp \parr B^\perp \quad (A \parr B)^\perp = A^\perp \otimes B^\perp$$
$$(A \,\&\, B)^\perp = A^\perp \oplus B^\perp \quad (A \oplus B)^\perp = A^\perp \,\&\, B^\perp$$

Finally, we note that some form of weakening and contraction is desired to match the expressiveness of other logics. For this, the exponential *why not* ? and *of course* ! operators are defined. A proposition of the form $?A$ allows for the usual contraction and weakening rules we expect in other forms of logic. Its negation, a proposition of the form $!A$ can only be proved by other propositions of the form $?B$. In the realm of communication protocols, a proposition of the form $!A$ is the protocol of a server that repeatedly accepts connections using protocol A, and a proposition of the form $?A$ is the protocol of a collection of clients that may each request a connection using protocol A. The full set of rules for classical linear logic can be found in Figure 5.5. We use $\Delta, \Delta_1, \Delta_2$ to denote sets of propositions, and $?\Delta$ to denote sets where all propositions have the form $?A$.

The names of the connectives used in linear logic are *tensor* ($\otimes$), *with* (&), *par* ($\parr$), *plus* ($\oplus$), *of course* (!), and *why not* (?). Linear implication $\multimap$ can be defined using *par*: $A \multimap B = A^\perp \parr B$.

Intuitionistic Linear Logic

Intuitionistic linear logic (ILL) is a fragment of linear logic that supports a single conclusion and two-sided sequents [80, 81]. These sequents have the form $\Gamma; \Delta \vdash A$. At the leftmost

$$\frac{}{\vdash A, A^{\perp}}\,\text{Ax} \qquad \frac{\vdash A, \Delta_1 \qquad \vdash A^{\perp}, \Delta_2}{\vdash \Delta_1, \Delta_2}\,\text{Cut}$$

$$\frac{\vdash A, \Delta_1 \qquad \vdash B, \Delta_2}{\vdash A \otimes B, \Delta_1, \Delta_2}\,\otimes \qquad \frac{\vdash A, C \qquad \vdash B, C}{\vdash A \,\&\, B, C}\,\&$$

$$\frac{\vdash A, B, \Delta}{\vdash A \parr B, \Delta}\,\parr \qquad \frac{\vdash A, \Delta}{\vdash A \oplus B, \Delta}\,\oplus_1 \qquad \frac{\vdash B, \Delta}{\vdash A \oplus B, \Delta}\,\oplus_2$$

$$\frac{\vdash A, ?\Delta}{\vdash !A, ?\Delta}\,! \qquad \frac{\vdash A, \Delta}{\vdash ?A, \Delta}\,? \qquad \frac{\vdash \Delta}{\vdash ?A, \Delta}\,\text{Weaken} \qquad \frac{\vdash ?A, ?A, \Delta}{\vdash ?A, \Delta}\,\text{Contract}$$

$$\frac{}{\vdash 1}\,1 \qquad \frac{\vdash \Delta}{\vdash \bot, \Delta}\,\bot \qquad \frac{}{\vdash \top, \Delta}\,\top \qquad \text{(No rule for 0)}$$

Figure 5.5: Inference rules for classical linear logic.

position is the context Γ that tracks unrestricted (sometimes called intuitionistic) assumptions. These are assumptions that can be used zero or more times in the proof. After Γ is the context Δ that tracks linear assumptions. These are assumptions that must be used exactly once in the proof. To the right of the $\vdash$ is a proposition A which represents the conclusion.

The grammar for formulas in intuitionistic linear logic is given by the following rules:

$$A, B ::= 1 \mid \top \mid 0 \mid A \otimes B \mid A \multimap B \mid A \,\&\, B \mid A \oplus B \mid !A$$

The first noticeable difference of this system of linear logic is the omission of the par $\parr$ connective. There is no possible right-side introduction rule that can be defined for par. However, this system does define linear implication $\multimap$ as a primitive connective. Similarly, the why not ? connective is omitted, as weakening and contraction on the right side would no longer be allowed. Finally, the omission of par and why not means that linear negation $(\cdot)^{\perp}$ must also be omitted.

The rules in Figure 5.6 define the rules for this system. We use $\cdot$ to denote an empty context, and ; separates the unrestricted context from the linear context. The left side rules for a connective follow naturally from the right side rules of its dual. The rule for ! on the left changes, and moves the replicating linear assumption to the unrestricted context. An additional rule called copy is added that copies unrestricted assumptions to the linear context. The copy rule is necessary as most other rules of the system are only concerned with the linear context.

$$\frac{}{\Gamma; A \vdash A}\,\text{Ax} \qquad \frac{}{\Gamma; \Delta, 0 \vdash A}\,0\text{L} \qquad \frac{}{\Gamma; \Delta \vdash \top}\,\top\text{R} \qquad \frac{\Gamma; \Delta \vdash A}{\Gamma; \Delta, 1 \vdash A}\,1\text{L} \qquad \frac{}{\Gamma; \cdot \vdash 1}\,1\text{R}$$

$$\frac{\Gamma; \Delta_1 \vdash A \qquad \Gamma; \Delta_2 \vdash B}{\Gamma; \Delta_1, \Delta_2 \vdash A \otimes B}\,\otimes\text{R} \qquad \frac{\Gamma; \Delta, A, B \vdash C}{\Gamma; \Delta, A \otimes B \vdash C}\,\otimes\text{L}$$

$$\frac{\Gamma; \Delta, A \vdash B}{\Gamma; \Delta \vdash A \multimap B}\,\multimap\text{R} \qquad \frac{\Gamma; \Delta_1 \vdash A \qquad \Gamma; \Delta_2, B \vdash C}{\Gamma; \Delta_1, \Delta_2, A \multimap B \vdash C}\,\multimap\text{L}$$

$$\frac{\Gamma; \Delta \vdash A \qquad \Gamma; \Delta \vdash B}{\Gamma; \Delta \vdash A \,\&\, B}\,\&\text{R} \qquad \frac{\Gamma; \Delta, A \vdash C}{\Gamma; \Delta, A \,\&\, B \vdash C}\,\&\text{L1} \qquad \frac{\Gamma; \Delta, B \vdash C}{\Gamma; \Delta, A \,\&\, B \vdash C}\,\&\text{L2}$$

$$\frac{\Gamma; \Delta \vdash A}{\Gamma; \Delta \vdash A \oplus B}\,\oplus\text{R1} \qquad \frac{\Gamma; \Delta \vdash B}{\Gamma; \Delta \vdash A \oplus B}\,\oplus\text{R2} \qquad \frac{\Gamma; \Delta, A \vdash C \qquad \Gamma; \Delta, B \vdash C}{\Gamma; \Delta, A \oplus B \vdash C}\,\oplus\text{L}$$

$$\frac{\Gamma; \cdot \vdash A}{\Gamma; \cdot \vdash !A}\,!\text{R} \qquad \frac{\Gamma, A; \Delta \vdash B}{\Gamma; \Delta, !A \vdash B}\,!\text{L} \qquad \frac{\Gamma, A; \Delta, A \vdash B}{\Gamma, A; \Delta \vdash B}\,\text{copy}$$

$$\frac{\Gamma; \Delta_1 \vdash A \qquad \Gamma; \Delta_2, A \vdash B}{\Gamma; \Delta_1, \Delta_2 \vdash B}\,\text{Cut} \qquad \frac{\Gamma; \cdot \vdash A \qquad \Gamma, A; \Delta \vdash B}{\Gamma; \Delta \vdash B}\,\text{Cut!}$$

$$\frac{\Gamma; \Delta \vdash B}{\Gamma, A; \Delta \vdash B}\,\text{Weakening} \qquad \frac{\Gamma, A, A; \Delta \vdash B}{\Gamma, A; \Delta \vdash B}\,\text{Contraction}$$

Figure 5.6: Inference rules for intuitionistic linear logic.

Also note that weakening and contraction are allowed in the unrestricted context. Tracking the unrestricted assumptions in this way is required, because allowing weakening and contraction on assumptions of the form $!A$ causes issues in proof reduction [82].

5.2.3 Session Types

Session types were initially introduced by Honda as a typing discipline for the structure of communication between processes [83]. A session type specifies which messages may be exchanged along a channel, the order the messages are sent, what choices may be made, and when communication terminates. Session types aim to prevent behavioral communication errors by making the protocol part of the type. In the π-calculus, session types assign each channel a protocol that constrains its use. A well-typed process is guaranteed to use each channel according to the protocol described by its session type.

More than two decades after Girard presented linear logic, Caires and Pfenning [78] identified a Curry-Howard correspondence between session types and intuitionistic linear

logic. This was later expanded upon by Wadler [84], who established a Curry-Howard correspondence between session types and classical linear logic. For the purpose of our work we present the type system of Caires and Pfenning (plus the Id/Ax rule with linking from Wadler [84]) in Figure 5.7, which corresponds to the intuitionistic linear logic system in Figure 5.6 without the rules for 0 and $\top$. Type judgments in this system have the form $\Gamma; \Delta \vdash P :: z : A$. The contexts Γ and Δ now consist of channel names assigned to session types which are written $x : A$ to mean channel x has session type A. Γ is the unrestricted context, and Δ is the linear context. The active channel is z which is typed with session type A, and the process being typed is P. The names of channels in Γ, Δ, and $\{z\}$ must be pairwise disjoint, and combining contexts is only allowed if they contain distinct names.

From the process terms that are typed by those rules, we can describe the meaning of the session types. For example, the rule for $\otimes$ on the right types a process that offers protocol A and B concurrently through channel z. The process offers this protocol by sending a fresh channel along z that will perform protocol A in process P with ambient channels in Δ_1, and then z continues to perform protocol B in process Q with ambient channels in Δ_2. Since Δ_1 and Δ_2 are combined, they must have distinct names. This means that processes P and Q will not interfere with the communication in the other process.

Typing Processes

A well-typed process has compatible endpoints. The first thing to demonstrate in this system of session types is the verification of a client that consumes a server protocol. Assuming a server protocol exists called *ServerProto*, then we can verify a client process *clientP* that consumes the server protocol on channel s by deriving the type judgment:

$$\cdot; s : \mathit{ServerProto} \vdash \mathit{clientP} :: - : 1$$

where $-$ is a channel that does not appear free in *clientP*. This works because the rule 1R allows typing the halted process with any channel of type 1 with an empty linear context. The linearity of the session types requires that *ServerProto* is correctly handled in *clientP* on channel s.

$$\frac{}{\Gamma; x : A \vdash [x \leftrightarrow z] :: z : A}\,\mathrm{Id} \qquad \frac{\Gamma; \Delta \vdash P :: z : A}{\Gamma; \Delta, x : 1 \vdash P :: z : A}\,1\mathrm{L} \qquad \frac{}{\Gamma; \cdot \vdash 0 :: z : 1}\,1\mathrm{R}$$

$$\frac{\Gamma; \Delta_1 \vdash P :: y : A \qquad \Gamma; \Delta_2 \vdash Q :: z : B}{\Gamma; \Delta_1, \Delta_2 \vdash (\nu y) z[y].(P|Q) :: z : A \otimes B}\,\otimes\mathrm{R} \qquad \frac{\Gamma; \Delta, y : A, x : B \vdash P :: z : C}{\Gamma; \Delta, x : A \otimes B \vdash x(y).P :: z : C}\,\otimes\mathrm{L}$$

$$\frac{\Gamma; \Delta, x : A \vdash P :: z : B}{\Gamma; \Delta \vdash z(x).P :: z : A \multimap B}\,\multimap\mathrm{R} \qquad \frac{\Gamma; \Delta_1 \vdash P :: y : A \qquad \Gamma; \Delta_2, x : B \vdash Q :: z : C}{\Gamma; \Delta_1, \Delta_2, x : A \multimap B \vdash (\nu y) x[y].(P|Q) :: z : C}\,\multimap\mathrm{L}$$

$$\frac{\Gamma; \Delta \vdash P :: z : A \qquad \Gamma; \Delta \vdash Q :: z : B}{\Gamma; \Delta \vdash z.\mathtt{case}(P, Q) :: z : A \,\&\, B}\,\&\mathrm{R}$$

$$\frac{\Gamma; \Delta, x : A \vdash P :: z : C}{\Gamma; \Delta, x : A \,\&\, B \vdash x.\mathtt{inl}; P :: z : C}\,\&\mathrm{L1} \qquad \frac{\Gamma; \Delta, x : B \vdash P :: z : C}{\Gamma; \Delta, x : A \,\&\, B \vdash x.\mathtt{inr}; P :: z : C}\,\&\mathrm{L2}$$

$$\frac{\Gamma; \Delta \vdash P :: z : A}{\Gamma; \Delta \vdash z.\mathtt{inl}; P :: z : A \oplus B}\,\oplus\mathrm{R1} \qquad \frac{\Gamma; \Delta \vdash P :: z : B}{\Gamma; \Delta \vdash z.\mathtt{inr}; P :: z : A \oplus B}\,\oplus\mathrm{R2}$$

$$\frac{\Gamma; \Delta, x : A \vdash P :: z : C \qquad \Gamma; \Delta, x : B \vdash Q :: z : C}{\Gamma; \Delta, x : A \oplus B \vdash x.\mathtt{case}(P, Q) :: z : C}\,\oplus\mathrm{L}$$

$$\frac{\Gamma; \cdot \vdash P :: x : A}{\Gamma; \cdot \vdash !z(x).P :: z : !A}\,!\mathrm{R} \qquad \frac{\Gamma, u : A; \Delta \vdash P\{u/x\} :: z : B}{\Gamma; \Delta, x : !A \vdash P :: z : B}\,!\mathrm{L}$$

$$\frac{\Gamma, u : A; \Delta, y : A \vdash P :: z : B}{\Gamma, u : A; \Delta \vdash (\nu y) u[y].P :: z : B}\,\mathrm{copy}$$

$$\frac{\Gamma; \Delta_1 \vdash P :: x : A \qquad \Gamma; \Delta_2, x : A \vdash Q :: z : B}{\Gamma; \Delta_1, \Delta_2 \vdash (\nu x)(P|Q) :: z : B}\,\mathrm{Cut}$$

$$\frac{\Gamma; \cdot \vdash P :: x : A \qquad \Gamma, u : A; \Delta \vdash Q :: z : B}{\Gamma; \Delta \vdash (\nu u)((!u(x).P)|Q) :: z : B}\,\mathrm{Cut!}$$

$$\frac{\Gamma; \Delta \vdash P :: z : B}{\Gamma, u : A; \Delta \vdash P :: z : B}\,\mathrm{Weakening} \qquad \frac{\Gamma, u : A, v : A; \Delta \vdash P :: z : B}{\Gamma, w : A; \Delta \vdash P\{w/u, w/v\} :: z : B}\,\mathrm{Contraction}$$

Figure 5.7: Type rules for the π-calculus.

Now if we have a server process that is meant to implement the server protocol, we can verify that it has the correct type by deriving the type judgment:

$$\cdot; \cdot \vdash \mathit{SrvBody} :: s : \mathit{ServerProto}$$

And we can create a replicating version of the protocol with the following type judgment:

$$\cdot; \cdot \vdash !c(s).\mathit{SrvBody} :: c : !\mathit{ServerProto}$$

Finally, assume we have two clients that consume this replicating protocol on channel c called *AClient* and *BClient*, then we can create a complete system with the two clients and the server. The complete system has the following type judgment:

$$\cdot; \cdot \vdash (\nu c)(!c(s).\mathit{SrvBody} | \mathit{AClient} | \mathit{BClient}) :: - : 1$$

These different techniques using session types to type-check a system described by the π-calculus demonstrate the power of session types. By deriving a type of 1, a system shows that it will fully consume any ambient channels. In the case where there are no ambient channels, it shows that the internal interactions are free from deadlock [78]. So far, we have discussed type-checking processes that are known, but we have not verified the correctness of a protocol described by a session type. In this chapter we will present a tactic-based theorem prover for dependent session types with the ability to validate protocols and extract the processes that implement those valid protocols.

5.2.4 The Extended Calculus of Constructions

The Extended Calculus of Constructions (ECC) was developed by Luo [59] to provide type universes, based on Martin-Löf's type theory [85], to the Calculus of Constructions [86]. The Calculus of Constructions is a dependently typed λ-calculus that supports Curry-Howard: propositions are types and proofs are terms. Kinds (types of types) in the Calculus of Constructions are given by the impredicative universe of propositions *Prop* and a single predicative type universe *Type*. ECC adopts the Martin-Löf approach and uses a hierarchy of predicative type universes $\mathit{Type}_0, \mathit{Type}_1, \mathit{Type}_2, ...$, but extends it with the impredicative

universe of propositions *Prop* at the bottom. The calculus also provides full cumulativity, where each element of each type universe is included in the higher level type universes.

In addition to the stratification of cumulative type universes, ECC includes dependent Σ-types. A term with the type $\Sigma x : A.P(x)$ is a pair containing a witness $x : A$ and the proof of $P(x)$. Inconsistencies that appear for impredicative Σ-types are avoided by requiring Σ-types to live in the predicative type universes. Finally, the terms in ECC are defined as dependently-typed λ-calculus terms with type annotations on pairs that allow for a simple type inference algorithm [59]. The syntax for ECC terms can be found in Figure 5.8, the function that returns free names can be found in Figure 5.9, and the reduction rules can be found in Figure 5.10.

$$
\begin{array}{lcl}
\mathit{Variable} & ::= & x, y, z \\
\mathit{Constant} & ::= & \mathit{Prop} \mid \mathit{Type}_j \text{ where } j \in \mathbb{N} \\
\mathit{Terms}(M, N, P, Q) & ::= & \mathit{Variable} \mid \mathit{Constant} \\
 & & \mid \Pi x : M.N \mid \lambda x : M.N, \\
 & & \mid \Sigma x : M.N \mid (M, N) : (\Sigma x : P.Q) \\
 & & \mid M\ N \mid \mathtt{fst}(M) \mid \mathtt{snd}(M)
\end{array}
$$

Figure 5.8: The Extended Calculus of Constructions syntax.

$$
\begin{array}{lcl}
\mathtt{fn}(x) & = & \{x\} \\
\mathtt{fn}(\mathit{Type}_j) & = & \{\} \\
\mathtt{fn}(\mathit{Prop}) & = & \{\} \\
\mathtt{fn}(\Pi x : M.N) & = & \mathtt{fn}(M) \cup (\mathtt{fn}(N) - \{x\}) \\
\mathtt{fn}(\lambda x : M.N) & = & \mathtt{fn}(M) \cup (\mathtt{fn}(N) - \{x\}) \\
\mathtt{fn}(\Sigma x : M.N) & = & \mathtt{fn}(M) \cup (\mathtt{fn}(N) - \{x\}) \\
\mathtt{fn}((M, N) : \Sigma x : P.Q) & = & \mathtt{fn}(M) \cup \mathtt{fn}(N) \cup \mathtt{fn}(\Sigma x : P.Q) \\
\mathtt{fn}(M\ N) & = & \mathtt{fn}(M) \cup \mathtt{fn}(N) \\
\mathtt{fn}(\mathtt{fst}(M)) & = & \mathtt{fn}(M) \\
\mathtt{fn}(\mathtt{snd}(M)) & = & \mathtt{fn}(M)
\end{array}
$$

Figure 5.9: Function for computing free names in Extended Calculus of Constructions terms.

The left side of a reduction rule provides the redex, and the right side provides the contractum. A term is in normal form if, and only if, it does not contain a redex. If M

$$
\begin{array}{rcl}
(\lambda x : A.M)N & \rightarrow_\beta & [N/x]M \\
\texttt{fst}((M,N) : \Sigma x : A.B) & \rightarrow_\sigma & M \\
\texttt{snd}((M,N) : \Sigma x : A.B) & \rightarrow_\sigma & N
\end{array}
$$

Figure 5.10: Term reduction rules for the Extended Calculus of Constructions.

contains a redex, and replacing that redex with its contractum gives M', then M one-step reduces to M'. Terms are convertible $M \simeq N$ if, and only if, N is obtained from M by a series of reductions and reverse reductions.

With the conversion relation $\simeq$ defined between terms, the cumulativity relation ($\preceq$) can be defined between terms:

1. If $A \simeq B$ then $A \preceq B$.
2. If $A \preceq B$ and $B \preceq A$, then $A \simeq B$
3. If $A \preceq B$ and $B \preceq C$, then $A \preceq C$
4. $\mathit{Prop} \preceq \mathit{Type}_0 \preceq \mathit{Type}_1 \preceq , \cdots ,$
5. If $A \simeq B$ and $C \preceq D$, then $\Pi x : A.C \preceq \Pi x : B.D$
6. If $A \preceq B$ and $C \preceq D$, then $\Sigma x : A.C \preceq \Sigma x : B.D$
7. $A \prec B$ if, and only if, $A \preceq B$ and $A \not\simeq B$

The cumulativity relation provides a partial order between terms with respect to conversion.

Contexts in ECC are finite sequences containing type assignments of the form $x : M$, which means that variable x has type M. Type judgments have the form $\Gamma \vdash M : N$ where Γ is a context, M is the term being type-checked, and N is the type given to M. An empty context is denoted with "$\cdot$". The set of free names in a context is defined by:

$$
\begin{array}{lcl}
\texttt{fn}(\cdot) & = & \{\} \\
\texttt{fn}(\Gamma, x : M) & = & \texttt{fn}(\Gamma) \cup \texttt{fn}(M) \cup \{x\}
\end{array}
$$

And the rules for type-checking can be found in Figure 5.11.

(Ax) $$\frac{}{\cdot \text{ valid}}$$

(C) $$\frac{\Gamma \vdash M : \mathit{Type}_j}{\Gamma, x : M \text{ valid}}\ x \notin \mathtt{fn}(\Gamma)$$

(P) $$\frac{\Gamma \text{ valid}}{\Gamma \vdash \mathit{Prop} : \mathit{Type}_0}$$

(T) $$\frac{\Gamma \text{ valid}}{\Gamma \vdash \mathit{Type}_j : \mathit{Type}_{j+1}}$$

(Var) $$\frac{\Gamma, x : M, \Gamma' \text{ valid}}{\Gamma, x : M, \Gamma' \vdash x : M}$$

(Π1) $$\frac{\Gamma, x : M \vdash N : \mathit{Prop}}{\Gamma \vdash \Pi x : M.N : \mathit{Prop}}$$

(Π2) $$\frac{\Gamma \vdash M : \mathit{Type}_j \qquad \Gamma, x : M \vdash N : \mathit{Type}_j}{\Gamma \vdash \Pi x : M.N : \mathit{Type}_j}$$

(λ) $$\frac{\Gamma, x : M \vdash N : P}{\Gamma \vdash \lambda x : M.N : \Pi x : M.P}$$

(app) $$\frac{\Gamma \vdash M : \Pi x : P.Q \qquad \Gamma \vdash N : P}{\Gamma \vdash MN : [N/x]Q}$$

(Σ) $$\frac{\Gamma \vdash M : \mathit{Type}_j \qquad \Gamma, x : M \vdash N : \mathit{Type}_j}{\Gamma \vdash \Sigma x : M.N : \mathit{Type}_j}$$

(pair) $$\frac{\Gamma \vdash M : P \qquad \Gamma \vdash N : [M/x]Q \qquad \Gamma, x : P \vdash Q : \mathit{Type}_j}{\Gamma \vdash ((M, N) : \Sigma x : P.Q) : \Sigma x : P.Q}$$

(`fst`) $$\frac{\Gamma \vdash M : \Sigma x : P.Q}{\Gamma \vdash \mathtt{fst}(M) : P}$$

(`snd`) $$\frac{\Gamma \vdash M : \Sigma x : P.Q}{\Gamma \vdash \mathtt{snd}(M) : [\mathtt{fst}(M)/x]Q}$$

(⪯) $$\frac{\Gamma \vdash M : P \qquad \Gamma \vdash P' : \mathit{Type}_j}{\Gamma \vdash M : P'}\ P \preceq P'$$

Figure 5.11: Type-checking rules for ECC.

For the purposes of this dissertation, ECC provides the functional system of the STILL theorem prover that is implemented in this chapter. The system is simple enough to build for an initial implementation of the theorem prover, and it is expressive enough to allow for more interesting proofs that are discussed in Chapter 6. The system of dependent session types that is implemented in the STILL theorem prover can be limited in expressiveness by the choice of the functional system. This chapter presents the implementation details of ECC as part of the presentation of the implementation of the whole STILL theorem prover.

5.2.5 Tactic-Based Theorem Provers

Interactive theorem provers are systems for developing machine-checked formal proofs in a logical framework. Fully automatic theorem provers attempt to complete proofs without user guidance, while interactive provers support proof development where the user guides the proof. Even in interactive provers, automatic proving techniques may still be available. This approach allows for the flexibility required to complete complicated proofs.

A large class of interactive theorem provers is tactic-based. Proof construction in tactic-based theorem provers is performed by repeatedly transforming a proof state into simpler subgoals until the proof is complete. The user applies tactics that perform proof steps such as introducing assumptions, decomposing propositions, and applying lemmas. This allows proofs to be developed in a goal-directed manner that is more consistent with how humans usually write proofs while still preserving the precise logical foundations.

Trusted Kernels

Interactive theorem provers are typically built on a core kernel that implements the logic of the system. Some examples of logics and the systems that use them are higher-order logic in Isabelle [60, 87], Martin-Löf type theory in NuPRL [88, 89], or the Calculus of Constructions in Rocq [86, 90]. These systems rely on small kernels that implement only primitive inference rules for the logic or dependent type theory that the system works in. All proof procedures must reduce to kernel-checked derivations for a theorem to be considered proved.

Outside of the trusted kernel, the user interacts with the system through higher-level

proof languages and tactics. A separate system of interaction allows for complicated proof construction procedures to be implemented without affecting the soundness of the system. A proof object created by these procedures must still be accepted by the kernel. The trust of the system relies on the kernel, while the usability of the system relies on the interaction layer.

Goal-Directed Proving

Constructing a proof from the axioms to derive a theorem is forward reasoning. Tactic-based theorem proving uses backward reasoning where the proof is constructed bottom-up. The user begins with the theorem as the first goal, and repeatedly reduces it to simpler proof obligations. The proof state tracks these obligations as a list of subgoals with justifications for each discharged subgoal.

A subgoal typically contains a context of assumptions and a target proposition to prove. This is typically written $\Gamma \vdash P$ where Γ is the context, and P is the goal proposition. Applying a tactic to this goal returns zero or more new subgoals. In the case the tactic succeeds and returns zero subgoals, the proof is complete. Otherwise, tactics may be applied to the newly created subgoals to further develop the proof.

A tactic that is applied to a goal returns a justification in addition to the list of subgoals. The justification is a function that takes a list of proof objects and returns a proof object. If the tactic that is applied to $\Gamma \vdash P$ is valid, then its justification will prove $\Gamma \vdash P$ if the proof objects provided to it conclude with the subgoals that were output by the tactic [91].

In addition to individual tactics, tactic-based systems provide tacticals: operators that combine tactics into more expressive proof procedures. Common tactical combinators include sequencing, alternative, and repetition. The sequencing tactical takes two tactics: it applies the first tactic and then the second tactic to all subgoals resulting from the first tactic. The alternative tactical also takes two tactics: it attempts the first tactic and applies the second tactic if the first one fails. The repetition tactical takes a single tactic and applies it repeatedly until it fails. Tacticals allow for combining primitive tactics into more expressive proof strategies.

Primitive Tactics and Program Extraction

Tactics can be implemented by following an inference rule from the bottom-up as described by Gordon et al. [91]. By the Curry-Howard isomorphism we know that propositions correspond to types and proofs correspond to programs. With this knowledge we can demonstrate how to implement a primitive tactic for the λ type-checking rule from ECC [59].

$$\frac{\Gamma, x : M \vdash N : P}{\Gamma \vdash \lambda x : M.N : \Pi x : M.P}$$

Recall that types are propositions, so the goal we are trying to prove is the type of the term. The tactic that corresponds to this rule must first verify that the goal has the form $\Pi x : M.P$. In the case that the goal has the correct form, the tactic returns a subgoal list with the single element $\Gamma, x : M \vdash N : P$ and a justification that will add the application of the λ inference rule to the proof object it is provided. This process can be followed for all inference rules to implement their corresponding primitive tactics.

Once the tactics complete a proof object that is accepted by the kernel, it is possible to extract a program from the proof. The extraction is given by the inference rules of the system. Working again with the λ rule, the term that is being type-checked is the term that can be extracted from the proof. If the upper part of the derivation extracts a term N, then the full derivation will extract the term $\lambda x : M.N$. In this way tactic-based provers also create programs that are correct-by-construction.

Also note that the Curry-Howard isomorphism goes both ways. This means that providing a term that has the correct type is the same as developing a proof for that same goal. In most systems of dependent type theories the user can provide a term of the correct type or apply tactics to prove a proposition. In this chapter we present the implementation of a tactic-based prover for dependent session types based on intuitionistic linear logic that extracts π-calculus terms that communicate dependently typed terms in ECC.

5.3 Contributions in This Chapter

This chapter describes STILL, a tactic-based theorem prover for dependent session types with coinduction and session type polymorphism. STILL stands for Session Types and Intuitionistic Linear Logic. The underlying type system of STILL is based on the dependent session type system presented in [42], with extensions that add polymorphic session types [57] and coinductive session types [58]. This theory of dependent session types intentionally leaves the functional system unspecified to maintain generality, so we have implemented the Extended Calculus of Constructions [59] as the functional system due to its simplicity and the potential it holds for future extensions.

The work presented here differs from previous work in the following ways:

- To our knowledge, the STILL prover is the first standalone (*i.e.*, not embedded in another theorem prover) tactic-based prover for a dependent session type system. Previous work on dependent session types by Ciccone and Padovani [41] does not present a standalone system, but instead embeds the system within the Agda programming language.

- Primitive tactics are provided that delay context partitioning without modifying the rules of the underlying type theory. This is achieved with a nondeterministic representation of context partitions called a *subgoal candidate set*. A traversal of the tree of subgoal candidate sets is used to hide resources that have been reserved in other branches. Validations reify these trees into proofs. Previous works identify the need for delayed partitioning in both tactic-based implementations and in proof search algorithms [45, 47, 52, 53]. Unlike those works, our implementation does not modify the proof rules of the systems we base our implementation on [42, 57, 58]. We count this as a significant advantage because of the considerable effort that goes into proving soundness, progress, subject reduction, consistency, normalization, and cut elimination when modifications are made.

- The STILL prover includes a separate tactic engine and kernel for developing proofs and type-checking terms in the Extended Calculus of Constructions [59] that integrates

into the overall system. Other interactive systems with two layers of logic include the Abella interactive theorem prover [92], and the Hybrid tool [93] within Isabelle/HOL and Rocq. These systems have a logical layer for specification and a logical layer for reasoning about specifications. STILL differs from the other two-layer systems by providing two object-level dependent type systems inside a standalone theorem prover. The session type layer provides the ability to specify properties of protocols, and the dependently typed functional layer provides the ability to specify properties of the values communicated by those protocols.

- Extracted proof terms are π-calculus terms rather than λ-calculus terms. Prior works that support program extraction from linear type systems either provide a type-checker for the π-calculus [41, 56] or are designed to type-check and extract terms of a linear λ-calculus [44, 47, 50, 51].

- The STILL prover implements theoretical extensions for coinductive and polymorphic session types within the dependent session type system developed by Toninho, Caires, and their colleagues [42, 57, 58].

This chapter is organized as follows. Section 5.4.1 describes the underlying theory of dependent session types with polymorphism and coinduction that is implemented by STILL. Section 5.4.2 describes the architecture of the system, which includes the implementation of tactics with delayed context partitioning and the trusted computing base. Section 5.4.3 evaluates the usability of the system and demonstrates the application of the prover with example specifications, example extractions of π-calculus terms, and benchmarks for several types of proofs. Section 5.4.4 concludes with discussion about the future applications of the STILL prover and the techniques that can be applied to future systems.

5.4 The STILL Theorem Prover

Toninho et al. [42] have developed a theory of *dependent* session types, based on intuitionistic linear logic, which extends a version of the term-passing π-calculus with polymorphism (type

variables) and kinds (a type of types). While work on developing dependent session types continues [94], the work on applying these powerful theories has not gained as much traction in practice as one might have expected.

A motivation for the work described in this chapter is the belief that specifications are easier to write, and more comprehensible, when they are written in richly expressive logics of the kind just mentioned. Implementations of other powerful logical systems [60,89,90,95–100] have been extraordinarily successful. We are unaware of similar systems for the theory of dependent session types.

5.4.1 Dependent Session Types

The theory of dependent session types developed by Toninho et al. [42] uses session types for the communication layer, and a dependent functional type system for the computation layer[1]. Types may be used to specify the behaviors of protocols and properties of the data being sent and received. This allows users to naturally describe the interactions between concurrent processes together with the internal computations that determine communicated values.

A sequent in this system has the form $\Psi; \Gamma; \Delta \vdash P :: z : C$. To the left of $\vdash$ are three (finite) contexts mapping variables to types. The three contexts together track typing judgments of variables and their usage restrictions. The empty context is denoted by the symbol "$\cdot$".

Ψ is the functional context. The variables in this context have types that are used in typing functional terms in the system. Functional terms are used to define values that can be sent and received across channels. In derivations, variables in Ψ are used to prove that values transmitted in the protocol have the correct type.

Γ is the unrestricted context. Variables in Γ have session types that correspond to exponential propositions in linear logic. These variables denote channel names with their session behaviors. These channels can be replicated zero or more times. They are used

[1]Readers interested in the full set of proof rules can find them on page 9 of [42].

to spawn channels with fresh names in the linear context. In a way, the unrestricted context denotes shared channels acting as servers that accept zero or more connections.

Δ is the linear context. It binds linear session behaviors to variables/channels. Each variable in the linear context must be used exactly once in a proof. The linear context denotes channels used to send and receive signals that must behave as specified by their session type.

In a sequent of the form $\Psi; \Gamma; \Delta \vdash P :: z : C$, P is a term in an enriched value-passing π-calculus. In STILL, P is extracted from a proof that dependent session type C (using channel z) is inhabited. Under the Curry-Howard interpretation, P is a proof term verifying $z : C$ under the assumptions in Ψ, Γ, and Δ. Note that variable names in Γ, Δ, and $\{z\}$ must be pairwise disjoint.

Selected rules that illustrate dependency in this system of dependent session types can be found in Figure 5.12. These rules demonstrate how communication of values extends the functional context, and how the protocol itself depends on the values being communicated. Sequents of the form $\Psi \vdash M : \tau$ indicate type derivations in the functional layer. Types in the term language τ are referenced at the session type level using $\$\tau$. The $\forall$ and $\exists$ rules provide quantification over values and indicate channels that receive and send terms, respectively.

$$\frac{\Psi \vdash M : \tau}{\Psi; \Gamma; \cdot \vdash [z \leftarrow M] :: z : \$\tau}\ \$\mathrm{R} \qquad \frac{\Psi, x : \tau; \Gamma; \Delta \vdash P :: z : C}{\Psi; \Gamma; \Delta, x : \$\tau \vdash P :: z : C}\ \$\mathrm{L}$$

$$\frac{\Psi, y : \tau; \Gamma; x : A, \Delta \vdash P :: z : C}{\Psi; \Gamma; \Delta, x : \exists y : \tau.A \vdash x(y).P :: z : C}\ \exists\mathrm{L} \qquad \frac{\Psi \vdash N : \tau \quad \Psi; \Gamma; \Delta \vdash P :: z : A\{N/x\}}{\Psi; \Gamma; \Delta \vdash z\langle N\rangle.P :: z : \exists x : \tau.A}\ \exists\mathrm{R}$$

$$\frac{\Psi, x : \tau; \Gamma; \Delta \vdash P :: z : A}{\Psi; \Gamma; \Delta \vdash z(x).P :: z : \forall x : \tau.A}\ \forall\mathrm{R} \qquad \frac{\Psi \vdash N : \tau \quad \Psi; \Gamma; \Delta, x : A\{N/y\} \vdash P :: z : C}{\Psi; \Gamma; \Delta, x : \forall y : \tau.A \vdash x\langle N\rangle.P :: z : C}\ \forall\mathrm{L}$$

Figure 5.12: Selected rules demonstrating dependency between the session and functional layer.

$$\frac{\Psi;\Gamma;\Delta, c : A\{\nu X.A/X\} \vdash_\eta P :: d : D}{\Psi;\Gamma;\Delta, c : \nu X.A \vdash_\eta P :: d : D}\ \nu\mathrm{L}$$

$$\frac{\Psi;\Gamma;\Delta \vdash_{\eta'} P :: c : A \qquad \eta' = \eta[X(\overline{y}) \mapsto \Psi\{\overline{y}/\overline{z}\};\Gamma\{\overline{y}/\overline{z}\};\Delta\{\overline{y}/\overline{z}\} \vdash c\{\overline{y}/\overline{z}\} : Y]}{\Psi;\Gamma;\Delta \vdash_\eta (\texttt{corec}\ X(\overline{y}).P\{\overline{y}/\overline{z}\})\ \overline{z} :: c : \nu Y.A}\ \nu\mathrm{R}$$

$$\frac{\eta(X(\overline{y})) = \Psi;\Gamma;\Delta \vdash d : Y \qquad \rho = \{\overline{z}/\overline{y}\}}{\rho(\Psi);\rho(\Gamma);\rho(\Delta) \vdash_\eta X(\overline{z}) :: \rho(d) : Y}\ \mathrm{VAR}$$

Figure 5.13: Type-checking rules for coinductive session types

Coinduction

Toninho et al. [58] extend the logical theory of intuitionistic session types with coinductive session types for corecursive processes[2]. Coinductive session types provide a system for validating protocols that run indefinitely. The rules implemented for the STILL prover (specifically νR) differ slightly from the rules described by Toninho et al. The modification allows the parameters and initial arguments to use different names, whereas the original rules require the parameters and initial arguments to have the same names.

With the addition of coinductive session types, sequents are extended with a mapping of corecursive process variables to typing contexts. As such, sequents now have the form $\Psi;\Gamma;\Delta \vdash_\eta P :: z : C$, where η is the new mapping. Type-checking rules for the system of dependent session types described above are extended with η, but are otherwise left unchanged. We write $\overline{y}$ to denote lists of variables (or channels). Figure 5.13 presents the new rules for coinductive session types.

The first rule, νL, simply allows for unrolling a coinductive type. The next rule, νR, defines the type-checking rule for corecursive process definitions. The body of the definition, P, may refer to the declared variable X. The binding information, η, tracks the context information at the point where the corecursive declaration is made. Last, the VAR rule ensures that corecursive calls are made with the same typing information after the parameter names in the contexts are renamed to the argument names.

Our modification to the rules presented in [58] is made to the νR rule by applying

[2]Additional proof rules for the system can be found on page 6 of [58].

the renaming $\{\overline{y}/\overline{z}\}$ to the information inserted into η. This renaming allows for different parameter and argument names; otherwise, the renaming in the VAR rule would only be valid when the initial arguments and parameters have the same names.

Coinductive session types are essential for a usable session type system. In session type systems without coinduction, it is still possible for long running protocols to be type-checked using the rules for replication [78]; but it is not possible to type-check a long running protocol that changes over time and calls itself. Many of the processes that require coinduction to be well-typed are simple concurrent processes described in Milner's book on the π-calculus [27], *e.g.*, bit counters and semaphores.

Polymorphism

As defined up to this point, polymorphism exists in the functional layer of the system, but the session type layer does not allow for polymorphism. In another piece of work, Caires et al. [57] use second-order intuitionistic linear logic to define polymorphic session types. Based on this work, we have implemented the rules in Figure 5.14 in STILL (omitting the straightforward well-formedness rules for conciseness)[3]. These rules also include the syntactic extensions to the π-calculus that allow processes to send and receive session types. Following along with the work by Caires et al., a new context, Ω, is added for tracking type variables and checking well-formed session types. Sequents are extended with the new context, and are written $\Omega; \Psi; \Gamma; \Delta \vdash_\eta P :: z : C$. Type-checking rules for the system described so far are also extended with the new context, but are otherwise left unchanged.

As demonstrated by Toninho and Yoshida [101], these rules for the quantification over session types make it possible to express inductive and coinductive session types. An inductive session type $\mu X.F(X)$ can be encoded as $\forall X.!(F(X) \multimap X)$, and a coinductive type $\nu X.F(X)$ can be encoded as $\exists X.!(X \multimap F(X)) \otimes X$. For example, the session type for natural numbers would be $\forall X.!((1 \oplus X \multimap X) \multimap X)$. Example proofs in Chapter 6 demonstrate validating and extracting π-calculus terms for inductive session types using polymorphism.

[3]The full set of proof rules for this system can be found on page 7 of [57].

$$\frac{\Omega, X; \Psi; \Gamma; \Delta \vdash_\eta P :: z : A}{\Omega; \Psi; \Gamma; \Delta \vdash_\eta z(X).P :: z : \forall X.A} \forall 2\mathrm{R}$$

$$\frac{\Omega \vdash_\eta B \ \texttt{stype} \qquad \Omega; \Psi; \Gamma; \Delta, x : A\{B/X\} \vdash_\eta P :: z : C}{\Omega; \Psi; \Gamma; \Delta, x : \forall X.A \vdash_\eta x\langle B\rangle.P :: z : C} \forall 2\mathrm{L}$$

$$\frac{\Omega \vdash_\eta B \ \texttt{stype} \qquad \Omega; \Psi; \Gamma; \Delta \vdash_\eta P :: z : A\{B/X\}}{\Omega; \Psi; \Gamma; \Delta \vdash_\eta z\langle B\rangle.P :: z : \exists X.A} \exists 2\mathrm{R}$$

$$\frac{\Omega, X; \Psi; \Gamma; \Delta, x : A \vdash_\eta P :: z : C}{\Omega; \Psi; \Gamma; \Delta, x : \exists X.A \vdash_\eta x(X).P :: z : C} \exists 2\mathrm{L}$$

Figure 5.14: Type-checking rules for polymorphic session types

Putting It All Together

The STILL theorem prover uses the system described in this section with sequents of the form $\Omega; \Psi; \Gamma; \Delta \vdash_\eta P :: z : C$. Ω tracks session type variables that can be used in session types in Γ, Δ, η, and C. Ψ, the functional context, contains type judgments for variables that can be used in terms in the functional layer. Γ, the unrestricted context, contains channels with replicating session protocols. Δ, the linear context, tracks linear channels. η, the binding map, tracks the binding information for corecursive processes. To the right of the turnstile is the process being type-checked, P, followed by the channel implementing the session behavior in the process, z. The rightmost element of the sequent, C, is the session type indicating the protocol of the channel. Throughout the rest of the chapter, the process term may be omitted for readability. Such sequents will have the form: $\Omega; \Psi; \Gamma; \Delta \vdash_\eta z : C$.

The concrete syntax for functional terms, session types, and processes used in the STILL theorem prover are presented in Figure 5.15. The syntax and semantics for the session type terms and process terms are described in detail in [42, 57, 58, 78] — soundness and type preservation are also proved. The syntax, semantics, and computational properties of the functional terms are from Luo's Extended Calculus of Constructions [59].

```
n        ::= (natural numbers)
x,y,z    ::= (variable names)
xs,ys,zs ::= x,xs | ϵ

-- Functional terms
M,N,S,T  ::= Pi x : M . N | Sigma x : M . N | lambda x : M . N
         | (M,N) : Sigma x : S . T | fst M | snd M | M N | Prop | Type n | x

-- Session types
A,B      ::= 1 | A -o B | A * B | A & B | A + B | nu x . A | !A | x
         | $M | forall x : M . A | exists x : M . A
         | forall x : stype . A | exists x : stype . A

-- Process terms
P,Q      ::= P|Q | (nu x:A) P | x[y].P | x[M].P | x[A].P | x.inl;P | x.inr;P
         | x.case(P, Q) | x(y).P | !x(y).P
         | x <-> y | [x <- M] | corec x(ys) P (zs) | x(ys)
```

Figure 5.15: Concrete syntax for terms and types in the STILL prover.

5.4.2 Tactics for Session Types

Tactics were initially described by Gordon et al. [91] as pure functions with the following types:

```
tactic = goal -> ([goal], validation)
validation = [theorem] -> theorem
```

As Gordon et al. state, primitive tactics can be written for logical systems by inverting the inference rules, *i.e.*, reading them bottom-up. Recall that Section 5.2.5 demonstrates how to create a primitive tactic for the λ rule in ECC. A *valid tactic* [91] has the property that if the list of theorems given to the validation function proves the list of goals returned as the result of the tactic, then the theorem output from the validation function will prove the goal originally given to the tactic. Now consider the rule for the tensor operator in the system described in Section 5.4.1:

$$\frac{\Omega; \Psi; \Gamma; \Delta_1 \vdash_\eta y : A \qquad \Omega; \Psi; \Gamma; \Delta_2 \vdash_\eta z : B}{\Omega; \Psi; \Gamma; \Delta_1, \Delta_2 \vdash_\eta z : A \otimes B} \otimes\mathrm{R}$$

Applying the inference rule requires two proofs with disjoint linear assumptions Δ_1 and Δ_2. A naive approach to writing a valid tactic for the rule requires the user to provide a partition of the assumptions when the tactic is applied. The tactic would then output the two subgoals with those user-provided partitions as the respective assumptions in each

subgoal. However, this requires the user to have knowledge of exactly how the proof will partition the assumptions. Such a system would require the user to have a clear plan for resource usage in at least one branch. Otherwise, the user may pick the wrong partition, making it impossible to complete the proofs of the subgoals.

Delaying Partition Choices

In order to delay the partitioning decision, other automated and interactive systems often modify the rules of the logic. Modifications include adding constraints that must be solved to give the desired partition [47], having the sequent indicate the used and unused assumptions in the context that can be used in other branches [53, 55, 56, 102], or creating backchaining rules that delete resources in the linear context [52]. These modifications still result in proof rules that contain unknown partition results at the top of the inference rules. Constraint solving is deferred, and the leftover resource information is not known until the full proof branch is complete.

To demonstrate the issue, consider a modification to the tensor right rule based on the resource management calculus presented by Cervesato et al. [53]:

$$\frac{\Omega; \Psi; \Gamma; \Delta^I \backslash \Delta' \vdash_\eta y : A \qquad \Omega; \Psi; \Gamma; \Delta' \backslash \Delta^O \vdash_\eta z : B}{\Omega; \Psi; \Gamma; \Delta^I \backslash \Delta^O \vdash_\eta z : A \otimes B} \otimes\mathrm{R}$$

In this system, the context $\Delta^I \backslash \Delta^O$ indicates that the linear resources in Δ^I are available to prove the goal, and the resources that were not used in the proof are in Δ^O. While this more closely resembles the desired behavior of delaying the context partition for as long as possible, implementing a tactic that delays the partitioning is still not straightforward. A simple tactic that follows the rule from the bottom-up creates two subgoals of the form $\Omega; \Psi; \Gamma; \Delta^I \backslash \Delta' \vdash_\eta y : A$ and $\Omega; \Psi; \Gamma; \Delta' \backslash \Delta^O \vdash_\eta z : B$, and the user would be expected to provide Δ' and Δ^O just as they would be expected to provide Δ_1 and Δ_2 for the tactic based on the tensor operator in intuitionistic linear logic. Modifying the notation in this way does not delay the context partition choice.

STILL tactics take as input and output a nondeterministic representation of subgoals. We call this structure, which encodes all possible partitions of the linear context, a *subgoal*

candidate set. Order of the subgoals is not significant. Conceptually[4], STILL tactics have the following type:

```
subgoalCandidates = [goal]
tactic = subgoalCandidates -> ([subgoalCandidates], validation)
validation = [theorem] -> theorem
```

As tactics are applied, invalid subgoal candidates are pruned from the sets. If the collection of subgoal candidate sets is empty, that shows that the derivation tree is complete — or — there is no well-formed derivation tree resulting from the tactics as applied. Which of these two holds can tentatively be identified if the last rule applied was an axiom rule. But whether the derivation is a valid proof can only be determined by running the validation.

Figure 5.16 demonstrates an example of proof development beginning with the singleton subgoal candidate set $\{\cdot;\cdot;\cdot;a : A, b : B \vdash_\eta z : B \otimes A\}$. Arrows between each step of development indicate application of the tactic the arrow is labeled with. For readability of the figure we use *Can* to generate all subgoal candidates from a single subgoal. For example, $Can(\cdot;\cdot;\cdot;a : A, b : B \vdash_\eta z : A)$ generates four subgoal candidates: one for each set in the power set of the linear context.

Applying the `Id b` tactic reserves `b`, and eliminates subgoal candidates in the other branch that use `b`. Applying the `Id a` tactic completes the proof. The STILL prover checks the full derivation before accepting the theorem. Following this simple proof demonstrates how the partition of the linear context is determined as the proof is constructed. The choices are not baked into the rules, and a partitioning choice is not explicitly made by the user.

Representing Subgoal Candidates

Treating tactics nondeterministically avoids the need to modify the system of dependent session types at the core of the STILL theorem prover. However, enumerating and tracking all subgoal candidates individually requires storing 2^n subgoals in each candidate set for

[4]In the implementation of STILL, the state monad is used to represent proof states which allows for an imperative style of sharing which is crucial for constructing and evaluating validations.

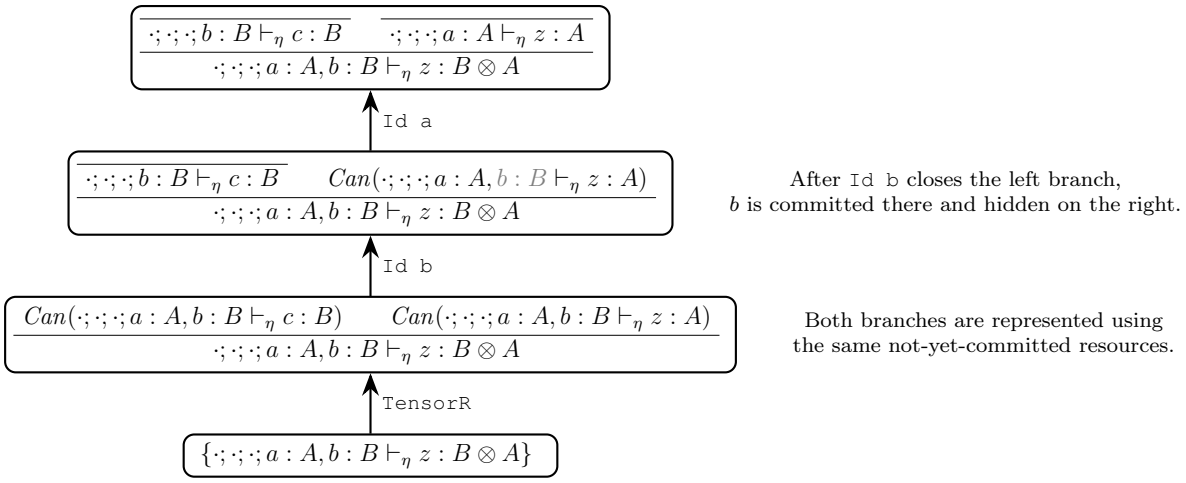

Figure 5.16: Delayed partitioning during proof construction. The proof is constructed by applying tactics to candidate sets at the leaves of a partial derivation.

every application of the `TensorR` tactic, where n is the number of elements in the linear context. This quickly becomes infeasible, even for modestly sized proofs.

The subgoal candidates in a set only differ in the choice of partitions for the linear context. For that reason, the STILL prover represents subgoal candidate sets deterministically as a single sequent, except for the linear context, which is stored as all currently unclaimed resources. For example, the `TensorR` tactic for the tensor right rule outputs the subgoal candidates for the left and right operand with the full linear context available on both sides. Section 5.4.2 and Section 5.4.2 describe how linear resources are reserved in each branch of the proof.

Applying Tactics

We now move from theoretical foundations to the concrete implementation. The STILL prover is implemented in Haskell and uses the state monad to implement tactics in a comparable manner to the techniques described by Martin et al. [103], and by Martin and Gibbons [104]. The proof state holds information about tactic history, variable usage, and open goals (subgoal candidate sets). The user can then interact with the system by selecting the current candidate set, applying tactics to the selected candidate set, or viewing other state information.

The proof state of the STILL prover contains a map of names to subgoal candidate sets, together with a list of the names of the open candidate sets. Every candidate set, except the initial one, is the result of a tactic application from a previous subgoal. The subgoal candidate set information also contains reserved resource names, a reference to the previous subgoal candidate set, references to the next subgoal candidate sets, and the branch type of the applied tactic if the subgoal candidate has been expanded.

When a tactic is applied to a subgoal candidate set, it updates the candidate set information with any resources it reserved in the linear context, and creates the next subgoal candidate sets and the justification that will run when the proof is complete. Tactics create subgoal candidate sets according to their corresponding inference rule as described in Section 5.4.2. Those new subgoal candidate sets are then added to the candidate set map in the state and the list of open candidate sets. Subgoal candidate set references to their parent and children subgoal candidate sets are also created by the application of a tactic. This creates a tree structure from the initial goal to the current open candidate sets.

The corresponding inference rule for a tactic that creates multiple candidate sets may require the linear context for the proofs of those goals to be disjoint or equal. This branch type distinction corresponds to inference rules for multiplicative and additive connectives, respectively. When a tactic is applied to a candidate set, the candidate set is marked as expanded with the corresponding branch type of the tactic (or as a trunk expansion if fewer than two subgoal candidate sets are created).

Tracking Resources in the Subgoal Candidate Tree

Due to tactics outputting sequents with all unreserved linear resources as the representation of subgoal candidate sets, all resources would appear available in the linear context when the user views the candidate sets on opposite sides of a multiplicative branch. This introduces a new problem for users: they might use resources that have already been used elsewhere in the proof. If the two candidate sets that use the same resource are on opposite sides of the proof of a multiplicative operator, the full proof will fail. This problem is solved in the STILL theorem prover by tracking the reserved resources in the candidate set tree.

When the user views the sequent representing a candidate set, the system traverses the subgoal candidate set tree and collects all resource names that are not available to that candidate set. Those resources are then hidden and treated as unavailable. The recursive traversal that collects the resources to hide has the following steps:

1. Take a subgoal candidate set as input.
2. Initialize an empty set of resource names, R.
3. For the current subgoal candidate set, look up the parent subgoal candidate set.
4. If the previous subgoal candidate set was expanded by a multiplicative tactic, then get all reserved resource names in branches other than the current branch (*i.e.*, the parent's other children) and add them to R.
5. Recursively apply this algorithm with the parent subgoal candidate set as input and union the result with R.
6. Return R.

By applying this algorithm and hiding resources that are returned, the resources that have been used on the opposite sides of a multiplicative branch are automatically tracked. When a resource is in the set returned by the algorithm for a given candidate set input, that indicates there is a multiplicative inference rule (*e.g.*, the rule for tensor on the right) in the path from the root goal to the subgoal candidate set in the proof derivation that will have that resource allocated to the other branch. Preventing the use of those resources in other branches prevents the construction of proofs with invalid partitions.

The Trusted Computing Base

The STILL theorem prover is organized into separate modules for the type system kernels and tactic implementations. The architecture diagram of the STILL prover can be seen in Figure 5.17 where nodes represent system components (typically a single Haskell module). Arrows between nodes indicate the source node references the target node. Nodes colored in

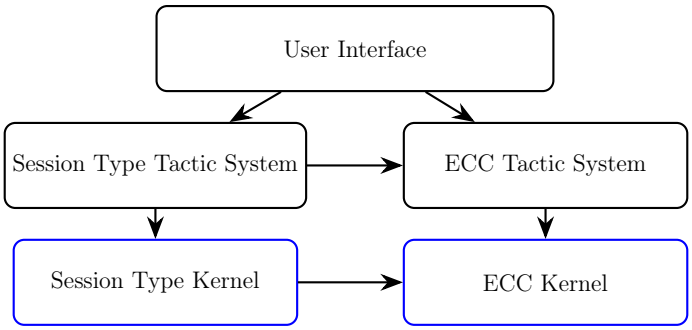

Figure 5.17: The architecture of the STILL prover.

blue represent the trusted computing base of the system. These are the two kernels for proof checking: one for the system of dependent session types described in Section 5.4.1, and one for the Extended Calculus of Constructions (ECC) [59]. These kernels check the correctness of proof objects in the two systems. The dependent session type kernel invokes the ECC kernel when it checks an inference rule that contains a functional proof object. Assuming the correctness of the implementation of the kernels, acceptance of a proof object corresponds to a valid derivation of the concluding sequent (including the π/λ-calculus term extraction).

Both kernels contain a Haskell datatype definition for a proof object whose constructors correspond to attempted inference rule applications for the system, and a validation function that determines whether the proof object should be accepted or not, *i.e.*, each inference rule is applied correctly. This includes well-formedness checking of polymorphic session types and checking corecursive calls match the recorded typing environment after substitution. The validation functions in both kernels rely on algorithms for capture-avoiding substitution, α-renaming, and fresh name generation. The ECC kernel also relies on a normalization algorithm for determining convertibility and universe cumulativity between two terms.

The dependent session type kernel also checks the multiplicative rule applications for disjoint linear contexts between the premises. Proof objects produced by the tactic system that attempt to use non-disjoint partitions will be caught when the proof object is checked. We note that under the correct implementation of the tactic system, the user should not be able to construct these invalid proofs interactively. The traversal algorithm and resource hiding prevent the use of resources in both premises of a multiplicative inference rule. How-

ever, the kernel still checks for disjoint linear contexts, and rejects proof objects with invalid multiplicative rule applications.

While we attempt to implement the tactic systems correctly, they may be unsound. For that reason, the kernels reside in modules that do not reference the tactics or prover state for either system. Trust in the STILL prover does not rely on the soundness of the tactic implementations, the subgoal candidate tree, the subgoal candidate sets, or resource hiding algorithms. Only kernel validation grants theorem status.

5.4.3 Evaluating the System

This section evaluates the usefulness of the STILL prover with the resource hiding algorithm and two tactic engines. There are three modes in which the system can run: the script mode that runs proof scripts, the file watch mode that reruns the proof script as the file is updated, and the REPL mode for experimentation. The script running mode can optionally return benchmarks for the scripts that are discussed in Section 5.4.3. The resource hiding algorithm is used in the file watch and REPL modes as the user develops proofs.

Usefulness of Resource Hiding

The examples in Figure 5.18 show two REPL sessions. One session has two different types of resources (`a:A` and `b:B`), and the other has multiple resources of the same type (`A`). These examples demonstrate how both subgoal candidate sets receive the unclaimed resources in the linear context after the `TensorR` tactic is applied. A context partitioning has not been determined at that point.

After the `TensorR` tactic creates multiplicative branches in the candidate set tree, resource hiding is demonstrated with the application of the `IdA`[5] tactic to reserve a resource on one branch of the candidate set tree (contexts are printed as $\Omega; \Psi; \Gamma; \Delta$ and left blank if empty). Before `IdA` is applied, both `a` and `b` are available in the linear context of the sequents representing both open subgoal candidate sets. After the tactic is applied, only one

[5]The automatic version of the `Id` tactic. Automatic versions of tactics that use resources in the contexts are provided in addition to versions that allow specifying which resource to use.

```
*?c>> ; ; ; a:A, b:B |- z:B ⊗ A
π> apply TensorR
Tensor right side tactic applied.
 ?d>> ; ; ; a:A, b:B |- c:B
*?e>> ; ; ; a:A, b:B |- z:A
π> apply IdA
IdA tactic applied.
*?d>> ; ; ; b:B |- c:B
π>
```

```
*?c>> ; ; ; a:A, b:A |- z:A ⊗ A
π> apply TensorR
Tensor right side tactic applied.
 ?d>> ; ; ; a:A, b:A |- c:A
*?e>> ; ; ; a:A, b:A |- z:A
π> apply IdA
IdA tactic applied.
*?d>> ; ; ; b:A |- c:A
π>
```

Figure 5.18: Example prompts and commands.

resource is available in the open candidate set, because the other is reserved on the opposite side of a multiplicative branch in the subgoal tree.

The example on the left follows the proof development in Figure 5.16. The example on the right is provided as a demonstration of a proof where both branches have the ability to use the same resources in the development of their proofs. The STILL tactic engine ensures that the user does not have to track which resource of type `A` has been used to prove subgoal `?e`. This example generalizes to a larger proof with more complex steps, during which the user would quickly lose track of which resources have been used in either branch without the hiding algorithm.

The Functional Subsystem

Developing proofs in the STILL prover requires interacting with the functional subsystem. When the user applies a tactic for session types that require discharging a functional typing subgoal (*e.g.,* the tactic for $R in Figure 5.12), the tactic is suspended pending the completion of the proof of the initialized functional subgoal. When a tactic is applied to the functional subgoal, the system checks if the proof is complete, and if so, the original session type tactic is resumed, expanding candidate sets and reserving resources.

Users see subgoals for an in-progress functional proof associated with a session type subgoal with names written as `<sessionSubgoal>.<functionalSubgoal>`. Tactics

are applied to functional subgoals using the `apply` command in the same way they are applied to session type subgoals. Tactic expressions in the session type and functional systems also have the same tactical operators. In this way, STILL provides a seamless transition between the two tactic engines during proof development.

Computing Type Derivations in ECC

Type derivations with the rules defined in Figure 5.11 quickly grow in size. For example, the derivation for a polymorphic identity function is:

$$
\begin{array}{c}
\hline \cdot \text{ valid} \\
\hline \cdot \vdash \mathit{Type}_1 : \mathit{Type}_2 \\
\hline A : \mathit{Type}_1 \vdash \text{valid} \\
\hline A : \mathit{Type}_1 \vdash A : \mathit{Type}_1 \\
\hline A : \mathit{Type}_1, x : A \text{ valid} \\
\hline A : \mathit{Type}_1, x : A \vdash x : A \\
\hline A : \mathit{Type}_1 \vdash \lambda x : A.x : \Pi x : A.A \\
\hline \cdot \vdash \lambda A : \mathit{Type}_1.\lambda x : A.x : \Pi A : \mathit{Type}_1.\Pi x : A.A
\end{array}
$$

Then creating a term that takes a proposition, and returns that proposition with the identity function gives the following type derivation:

$$
\begin{array}{c}
\hline \cdot \text{ valid} \\
\hline \cdot \vdash \mathit{Prop} : \mathit{Type}_0 \\
\hline P : \mathit{Prop} \text{ valid} \\
\hline P : \mathit{Prop} \vdash \mathit{Type}_1 : \mathit{Type}_2 \\
\hline P : \mathit{Prop}, A : \mathit{Type}_1 \text{ valid} \\
\hline P : \mathit{Prop}, A : \mathit{Type}_1 \vdash A : \mathit{Type}_1 \\
\hline P : \mathit{Prop}, A : \mathit{Type}_1, x : A \text{ valid} \\
\hline P : \mathit{Prop}, A : \mathit{Type}_1, x : A \vdash x : A \\
\hline P : \mathit{Prop}, A : \mathit{Type}_1 \vdash \lambda x : A.x : \Pi x : A.A \\
\hline P : \mathit{Prop} \vdash \lambda A : \mathit{Type}_1.\lambda x : A.x : \Pi A : \mathit{Type}_1.\Pi x : A.A
\end{array}
\quad
\begin{array}{c}
\hline \cdot \text{ valid} \\
\hline \cdot \vdash \mathit{Prop} : \mathit{Type}_0 \\
\hline P : \mathit{Prop} \text{ valid} \\
\hline P : \mathit{Prop} \vdash \mathit{Prop} : \mathit{Type}_0
\end{array}
\quad
\begin{array}{c}
\hline \cdot \text{ valid} \\
\hline \cdot \vdash \mathit{Prop} : \mathit{Type}_0 \\
\hline P : \mathit{Prop} \text{ valid} \\
\hline P : \mathit{Prop} \vdash \mathit{Type}_1 : \mathit{Type}_2
\end{array}
\quad
\begin{array}{c}
\hline \cdot \text{ valid} \\
\hline \cdot \vdash \mathit{Prop} : \mathit{Type}_0 \\
\hline P : \mathit{Prop} \text{ valid} \\
\hline P : \mathit{Prop} \vdash P : \mathit{Prop}
\end{array}
$$

$$
\begin{array}{c}
\hline P : \mathit{Prop} \vdash \mathit{Prop} : \mathit{Type}_1 \\
\hline P : \mathit{Prop} \vdash (\lambda A : \mathit{Type}_1.\lambda x : A.x)\ \mathit{Prop} : \Pi x : \mathit{Prop}.\mathit{Prop} \\
\hline P : \mathit{Prop} \vdash (\lambda A : \mathit{Type}_1.\lambda x : A.x)\ \mathit{Prop}\ P : \mathit{Prop} \\
\hline \cdot \vdash \lambda P : \mathit{Prop}.(\lambda A : \mathit{Type}_1.\lambda x : A.x)\ \mathit{Prop}\ P : \Pi P : \mathit{Prop}.\mathit{Prop}
\end{array}
$$

This results in four derivations of the judgment $P : \mathit{Prop}$ valid which are colored in red. The size of these type derivations becomes unmanageable to quickly verify for the proofs we wish to evaluate.

In order to address this issue, we use a technique from the Rocq theorem prover [90]. Rather than require every derivation to contain a subderivation that shows context validity, we instead expose an API from the ECC kernel that enforces context validity for all

functional contexts. The result is enforcement of context validity as the proof derivation is constructed. With that change, the C, T, and Var rules all become axioms since the context validity is enforced by the kernel and type system. Concretely, a Haskell term of type `FunctionalContext` can only be constructed and extended using the exposed API from the ECC kernel. With these changes, the type derivation for the term now avoids the duplicate derivations for context validity.

$$
\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{P : Prop, A : Type_1, x : A \vdash x : A}}{P : Prop, A : Type_1 \vdash \lambda x : A.x : \Pi x : A.A}}{P : Prop \vdash \lambda A : Type_1.\lambda x : A.x : \Pi A : Type_1.\Pi x : A.A} \quad \dfrac{\dfrac{}{P : Prop \vdash Prop : Type_0} \quad \dfrac{}{P : Prop \vdash Type_1 : Type_2}}{P : Prop \vdash Prop : Type_1}}{P : Prop \vdash (\lambda A : Type_1.\lambda x : A.x)\ Prop : \Pi x : Prop.Prop} \quad \dfrac{}{P : Prop \vdash P : Prop}}{P : Prop \vdash (\lambda A : Type_1.\lambda x : A.x)\ Prop\ P : Prop}}{\cdot \vdash \lambda P : Prop.(\lambda A : Type_1.\lambda x : A.x)\ Prop\ P : \Pi P : Prop.Prop}
$$

Example Proofs and Benchmarks

The STILL source code comes with example proofs that demonstrate its features. The bit counter proof can also be found in Chapter 6 along with additional proofs outside of the test suite provided here. The rest of the proofs in this test suite can also be found in Appendix E. The proof scripts can be executed by running the command: `./still Proofs/*.still`, and the benchmark table for the proof scripts can be generated with the benchmark argument: `./still benchmark Proofs/*.still`. Our benchmark results from running the examples[6] are presented in Table 5.1. The time column represents an average time over five executions of the script. A single execution includes the time to read the file from the disk, parse the file, execute the commands, and print the final message to the command line. The final message is printed to the command line to ensure full evaluation of the proof script due to the lazy nature of Haskell; otherwise, the function that returns the final proof state may not be fully executed and the benchmarks would not correctly evaluate the system. The examples in the table have been chosen to exercise the system features together, to stress test the traversal algorithm, and to assess the faithfulness of the implementation of the theory presented by Toninho, Caires, and their colleagues [42, 57, 58].

In Table 5.1, the Bank, CloudServer, and Counter modules assess the faithfulness of the implementation of the theory of dependent session types with polymorphism and coinduction.

[6]Benchmarks were taken on a 2019 MacBook Pro with a 2.4 GHz 8-Core Intel Core i9 processor and 32 GB 2667 MHz DDR4 memory.

Module	Theorems	Total Subgoals Total Inference Rules	Max Subgoals Max Inference Rules	Time (s)
Auction	3	104 189	38 115	0.009079
Bank	2	51 675	30 497	0.006347
CloudServer	2	25 25	14 14	0.000794
Counter	3	72 198	32 128	0.003192
LargeMul	1	3997 2999	3997 2999	3.536581

Table 5.1: Benchmarks for the STILL prover.

The Bank module validates a bank and ATM client protocol from [42]. The CloudServer module validates a polymorphic cloud server protocol, and a file conversion instance of the protocol from [57]. The Counter module validates a coinductive protocol for a bit counter which includes processes that store bits and are spawned as more storage is required from [58].

The LargeMul module stress tests the resource hiding traversal algorithm in the worst-case scenario: traversing the entire subgoal tree for every tactic application. It contains an expression with 1000 variables combined using the tensor operator. Implication of the same expression, but reversed, allows the proposition to be proved, *i.e.*, `(a * ... * l38)` implies `(l38 * ... * a)`. This module demonstrates that the traversal algorithm for reserving and hiding resources does not impede proof checking and development in a meaningful way. The proof of the proposition also demonstrates the power of the subgoal candidate sets. A single tactic expression `ImpliesR; TensorLA+; (TensorR; (IdA <|> Skip))+` proves the entire proposition.

Additional experiments testing how the performance scales can be found in the STILL source code [105]. Tensor expressions like those in the LargeMul module are evaluated with sizes from 10 to 1000, and the results show quadratic scaling. These are worst-case benchmarks as the tensor operator expands subgoals with multiplicative branches in the subgoal tree, so every subgoal must be traversed to hide resources for each tactic application. Further work is required to determine if performance can be improved.

```
theorem english:
  "forall ItemType:stype. forall bid:Type 1. forall zeroB:bid.
    forall validNewB:(Pi b1:bid. Pi b2:bid. Type 1).
      forall currency:(Pi b:bid.Type 1).ItemType -o
        (forall b:bid. $currency b -o 1) -o forall firstB:bid.
          forall firstValid:validNewB zeroB firstB. $currency firstB -o
            (ItemType + $currency firstB -o 1) -o
              nu Y. (forall newB:bid.forall nextValid:validNewB firstB newB.
                $currency newB -o (ItemType + $currency newB -o 1) -o Y) & 1"

z(ItemType).z(bid).z(zeroB).z(validNewB).z(currency).z(a).z(c).
  z(firstB).z(firstValid).z(d).z(e).
    (corec AUC(item, ownerC, curB, winnerC, oldB, lastB, curValid).z.case(
      inl: z(newB).z(nextValid).z(h).z(i).((ν j) winnerC[j].
        (j.inr;[curB <-> j] | AUC(item, ownerC, h, i, lastB, newB, nextValid))),
      inr: ownerC[lastB].((ν f) ownerC[f].
        ([curB <-> f] | (ν g) winnerC[g].(g.inl;[item <-> g] | 0)))))
    (a, c, d, e, zeroB, firstB, firstValid)
```

Figure 5.19: English auction specification and extracted π-calculus term.

The Auction Specification

Figure 5.19 presents the English auction protocol from the Auctions module (omitting the proof for space), and demonstrates the power of dependent session types alongside coinductive and polymorphic session types. The auction protocol begins with the seller providing the type of the item being auctioned. The item type has type `stype`, so it represents a service being offered (including ones that may transfer a tangible object). Then the protocol introduces a bid type, a bid validation type family, and a currency type family. The bid validation type family represents a validation parameterized by the old bid and the current bid. A term of the type `validNewB b1 b2` is a proof that `b2` is a valid new bid when checked against the previous bid `b1`. The currency dependent type family represents the currency being provided, and is parameterized by the amount of the currency. With this, the auction is polymorphic with respect to the item being offered, the validation of the bids, and the type of the currency. Finally, before bidding begins, the item is sent to the auction process, and the seller provides a channel that will receive the winning bid.

After the auction begins, the first bid amount can be provided along with a validation of the first bid against the zero bid. After that, the correct amount of currency must be provided. The current highest bidder also provides a channel that can either receive the item in the case that they win, or their money back in the case that their bid is beaten. After the

$$
\begin{aligned}
\mathit{postOffice} &\triangleq \forall \mathit{itemObjectType} : \mathit{Type}\ 1.\forall \mathit{item} : \mathit{itemObjectType}.\$\mathit{delivery}\ \mathit{itemObjectType}\ \mathit{item} \\
\mathit{bookSeller} &\triangleq \exists \mathit{startPrice} : \mathit{int}.\exists \mathit{bidValid} : (\Pi \mathit{b1} : \mathit{int}.\Pi \mathit{b2} : \mathit{int}.\mathit{Type}\ 1). \\
&\quad \exists \mathit{bookForSale} : \mathit{book}.(\forall b : \mathit{int}.\$\mathit{usd}\ b \multimap 1) \\
\mathit{bookBuyer} &\triangleq \forall \mathit{validator} : (\Pi \mathit{b1} : \mathit{int}.\Pi \mathit{b2} : \mathit{int}.\mathit{Type}\ 1).\exists \mathit{buyerAmount} : \mathit{int}.\forall \mathit{curPrice} : \mathit{int}. \\
&\quad \exists \mathit{buyerValid} : \mathit{validator}\ \mathit{curPrice}\ \mathit{buyerAmount}.\$(\mathit{usd}\ \mathit{buyerAmount}) \otimes \\
&\qquad (\forall \mathit{bookBought} : \mathit{book}.(\$\mathit{usd}\ \mathit{buyerAmount} \oplus \$\mathit{delivery}\ \mathit{book}\ \mathit{bookBought}) \multimap 1) \\
\cdot; \mathit{book} : \mathit{Type}\ 1, , \cdots ,, \mathit{usd} : \mathit{Type}\ 1; \cdot; \mathit{bookSeller}, \mathit{bookBuyer}, \mathit{auction}, \mathit{postOffice} &\vdash . \ z : 1
\end{aligned}
$$

Figure 5.20: Book Auction Protocol

first bid is fully processed, the protocol enters a coinductive session that repeats the process of receiving a bid, or terminates once a termination signal is given.

The linearity of the session types ensures that once another bid is received, the money is returned to the previous highest bidder. The dependent session types ensure that bidders only receive their money back, because the refund channel is typed with the current bid amount. No other amount of currency can be sent back to the bidder. Additionally, the seller can only receive the highest bid at the end of the auction, because the coinductive session type requires that channels that exist at the start of the auction remain available as the auction continues (modulo renaming according to the VAR rule). Finally, when the auction receives the signal to terminate (*i.e.*, enters the right side of the & connective), the item must be sent to the highest bidder, because that is the only channel that remains that can receive the item, and the seller receives the money from the highest bid, because that is the only channel that remains that can receive money.

The π-calculus process extracted from the proof of the validity of the auction protocol can also be seen in Figure 5.19. It has been formatted for readability and to save space. It enjoys the properties explained above, which are guaranteed by the dependent session types: bids must be validated, losers receive their money back, the winner receives the item, and the seller of the item receives the money from the winning bid.

Additionally, a verification that a book auction protocol satisfies the specification of the English auction is included in the Auction module. It validates the protocol in Figure 5.20. In the book auction, the post office creates deliveries for items, the book seller offers a book

for sale with a bid validator, and the book buyer provides a valid bid with the expectation of the delivery of the book if they win. The types for the book, bids, currency, and delivery are assumed in the functional context. The validation of the protocol corresponds to the creation of a process that enforces the correct interaction between the other four processes in order to complete the auction. This validation demonstrates the use of coinductive, polymorphic, and dependent session types together. Even though the book seller only offers a book for sale, the post office process can be used to create the delivery that the book buyer expects. Finally, the coinductive auction protocol on the left of the turnstile must terminate (according to the rule for 1 on the right). The termination of the auction ensures that the items and money are transferred correctly.

5.4.4 Discussion

To our knowledge, the STILL prover is the first standalone tactic-based interactive theorem prover for dependent session types. The system provides a concrete implementation of the theoretical system presented by Toninho, Caires, and their colleagues over multiple works [42, 57, 58], but there are similar theoretical systems. Griffith and Gunter [106] and Ciccone and Padovani [41] have both presented dependently typed π-calculi with a separate dependently typed functional layer. The system developed by Ciccone and Padovani is embedded in Agda. Additionally, the Session Abstract Machine (SAM) [107] provides a type-checker and interpreter for a process calculus using classical linear logic. The SAM also contains a simple functional layer with operations on integers, Booleans, and strings.

The STILL prover provides π-calculus process extractions from completed proofs. Additional work is required to translate these π-calculus processes with value-passing, type-passing, and corecursion into forms that can be run on existing systems that use process calculi. There are at least two approaches: to extract π-calculus terms from STILL proofs in a form that can be evaluated, or to translate the π-calculus terms used by STILL into an existing executable form. Target systems are SAM processes, ProVerif processes [108], or blockchain smart contract languages based on process calculi [40, 109, 110]. Having an executable form of STILL extractions would allow for simulation, further verification, and

deployment of the extracted processes. For example, a translation of the English auction process in Figure 5.19 into a smart contract would provide the protocol's guarantees of liquidity, refunding losing bids, and asset transfer. Formal verification of the translations would be required to ensure the preservation of the invariants given by the session type of the process.

Previous implementations of theorem provers for linear logic and session types have managed the context partitioning in one of two ways. The first is to require the user to provide the partition when refining a multiplicative goal, and continuing the proof using that choice [48]. The second is to modify the rules of the logic or type system to allow the choice of the partition to be delayed [53, 55, 56]. The STILL prover implements a third technique that models all partition choices nondeterministically using subgoal candidate sets. Users of the STILL theorem prover never explicitly provide the partition. The partitions are represented using subgoal candidate sets as the proof is constructed, and finally checked at completion. With this technique, a new session type system did not need to be re-developed and re-verified. We expect that the same tactic implementation techniques can be used to implement other flavors of session types and linear logic in their own tactic-based theorem provers without changing the underlying proof rules.

Chapter 6

Verifying Protocols

6.1 Contributions in This Chapter

In this chapter, we present multiple protocols with verifications carried out in the STILL theorem prover that was introduced in Chapter 5. This chapter begins with an introduction to using the STILL theorem prover. The aim is to provide a tutorial for anyone unfamiliar with tactic-based theorem proving. From there, we move to verifying the following protocols and extracting the processes for them:

1. The axiom of choice.
2. The natural numbers.
3. Bit counting processes.

6.2 Using the STILL Theorem Prover

The STILL theorem prover is a standalone application for developing proofs using dependent session types. It provides a server mode that allows for proof state inspection inside text editors. We provide a brief tutorial on proof development assuming access to the STILL extension for Visual Studio Code and the STILL prover. More details on obtaining the

STILL prover and the Visual Studio Code extension can be found in their respective code repositories[1].

A proof begins inside a text file with the `.still` extension. Before any commands, the module name must be declared with `module MODULE_NAME begin` or with imports `module MODULE_NAME imports [FILE_NAME,...] FILE_NAME begin`. After the module is declared, the proof script may have commands for declaring theorems or assumptions.

6.2.1 Top-Level Commands

A theorem is declared using the `theorem` keyword followed by the name of the theorem, a colon, and then the proposition wrapped in quotes. Optional linear assumptions can be supplied using the `consumes` keyword after the theorem name. Declaring a theorem begins a proof starting with the proposition and any assumptions that have been made before the theorem. The command has the form:

```
theorem THEOREM_NAME [consumes ["PROPOSITION"...]]: "PROPOSITION"
```

An assumption is declared using the `assume` keyword. It is used for declaring functional assumptions. This is useful when the user wants to assume the existence of a type, such as natural numbers, without including the information in their propositions. The type being assumed must be well-formed so that the global functional context is valid, otherwise the command will fail. Assumptions have the form:

```
assume IDENTIFIER is "FUNCTIONAL_TERM"
```

The STILL prover also allows for the assumption of variables as session types. These assumptions are useful for providing a name to an assumed protocol whose internal structure is irrelevant to the theorem. A session type assumption has the form:

```
assume IDENTIFIER is stype
```

[1] The STILL source code can be found at: https://github.com/STILL-prover/STILL. The extension source code can be found at: https://github.com/STILL-prover/STILL-extension

In addition to assumptions, the STILL prover also allows for type synonyms at the session type level. These are declared using the `stype` keyword. After declaration, the identifier may be used in the proposition of a theorem and it will be replaced with the proposition it was assigned to. Session type declarations have the form:

```
stype IDENTIFIER = "PROPOSITION"
```

There are a few more top-level commands that are useful for users, but they do not modify the proof state in any way. These commands are:

`print_theorems`	Print the theorems that have been proved at that point in the script.
`extract THEOREM_NAME`	Extract the π-calculus process that implements the protocol verified by the theorem.
`help`	Display help text for the tactics.

6.2.2 Proving Theorems

Once a proof begins, the proof state shows the current open goals and the resources available to those goals. Tactics are applied to the selected goal using the `apply` command followed by a tactic. Tactics can be found in the documentation of the `help` command, but usually follow naming conventions to make them easy to remember. For example, the tactic that corresponds to the Id rule in the theory of dependent session types is called `Id`. Furthermore, tactics for rules that require an assumption in one of the contexts take the selection of the assumption as a parameter. So the full application of the `Id` tactic would have the form:

```
apply Id IDENTIFIER
```

These rules also have an additional form of tactic that automatically searches for the required proposition in the assumptions. The tactics that implement this behavior end with `A`. In our Id example, applying this tactic has the form:

```
apply IdA
```

The STILL prover also provides a simple tactical language for composing tactics. Given two tactics `T1` and `T2`, a tactic may be created that tries `T1` and then, if that fails, tries `T2` using the expression `T1 <|> T2`; a tactic may also be created that applies `T1` and then `T2` to all resulting subgoals with the expression `T1;T2`; finally, a tactic that repeatedly applies `T1` until it fails can be created using `T1+`.

Once all the subgoals have been discharged, the proof object must be verified for the theorem to be considered proved. This is accomplished using the `done` command. The `done` command builds the proof object from the justifications of the tactics that have been applied, and verifies the resulting proof object. If the verification succeeds, the theorem is added to the collection of proved theorems that can be viewed using `print_theorems`, and the process can be extracted using the `extract` command.

6.3 The Axiom of Choice as a Protocol

The axiom of choice has been formalized in Martin-Löf Type Theory [88]. Using the STILL theorem prover, the axiom of choice can be formalized as a protocol using dependent session types, and the corresponding process can be extracted from the proof. This proof demonstrates how the strong Σ-types in ECC allow for the witness of a proof to be transmitted before the proof component. The proposition that encodes the axiom of choice uses two types, *A* and *B*, and a relation between elements of *A* and *B* called *P*. Given those three pieces of data, if there is a function that takes any *x* of type *A* and returns a term of type *Sigma y:B. P x y*, then there exists a function *f* from *A* to *B* such that, for every *x:A*, the proposition *P x (f x)* holds. The proof is given in the STILL proof script in Listing 6.1.

The first tactic application in the proof introduces all of the assumptions using the *Intros* tactic. Next, functional terms in the linear context are moved to the functional context using the *FTermLA* tactic. The *ExistsR* tactic begins the functional proof of *Pi v:A.B*, which is proved by providing the term *\v:A. fst (a v)*. This is the witness for *f*. Then the proof continues to prove *forall x:A.$(P x (f x))* by introducing *x* as

an assumption and completing the functional proof of `P x (f x)` through simplification and providing the correct term.

After the proof is complete, the process is extracted and placed in a comment at the bottom of the proof script. The process receives the types `A` and `B`, and the relation between their elements `P`. Then `z` receives the term of type `Pi x:A.Sigma y:B.P x y`. With those values communicated, `z` sends the term for `f:(Pi v:A.B)`. Finally, `z` will receive any element of `A` and return a proof that `P x (f x)` holds.

Listing 6.1: A proof of the axiom of choice using the STILL prover.

```
module AxiomOfChoice begin
theorem AC: "forall A:Type 1.
    forall B:Type 1.
    forall P:(Pi x:A. Pi y:B. Type 1).
    $(Pi x:A.Sigma y:B.P x y) -o
    exists f:(Pi v:A.B).
    forall x:A.
    $(P x (f x))"
apply Intros
apply FTermLA
apply ExistsR
apply Exact "\v:A. fst (a v)"
apply Intros
apply FTermR
apply Simp
apply Exact "(snd (a x))"
done

extract AC
-- z(A).z(B).z(P).z(a).z[(\v : A. proj1 (a v))].z(x).[ z <- proj2 (a x) ]
```

This proof demonstrates how Σ-types in ECC can be used to communicate proofs in the π-calculus. Using the `Exact` tactic, two terms are provided in the proof of the existence of `f` and the property `forall x:A.$(P x (f x))`. In the extracted term, the value for `f` is communicated before any proof of the validity of `f`. It is only after another term `x` is received that a proof of `P x (f x)` is communicated. Without Σ-types, this process would not type-check. With an impredicative encoding of the dependent sum, which is possible in the Calculus of Constructions [86, 111], the computational content of the proof component

cannot be accessed and sent after the witness.

6.4 The Natural Numbers as Concurrent Processes

In the second-order λ-calculus (System F [112,113]), recursive types can be defined in terms of universal quantification [114, 115]. Types defined as a least fixpoint are given by the equation:

$$\mu X.F(X) = \forall X.(F(X) \to X) \to X$$

Where F has only positive occurrences of X (X is only found on the left side of an even number of function arrows). Negative occurrences of X in a recursive type introduce terms with no normal form.

The polymorphic type $\forall X.(F\ X \to X) \to X$ forms an initial F-algebra when paired with a function we will call *in* with type $F\ T \to T$ where T is $\forall X.(F\ X \to X) \to X$. For T to be initial, there must be a unique map from T to every other object in the category. We will call this unique map *fold* and it has the type $\forall X.(F\ X \to X) \to T \to X$. These developments provide a representation of inductive datatypes using a categorical justification.

In a similar way, polymorphic session types can be used to represent recursive session types. This technique is presented by Toninho and Yoshida [101] based on encodings of recursive types in the second-order λ-calculus. In this section we will demonstrate how to use this representation to extract processes that represent the natural numbers.

First, Toninho and Yoshida provide the type judgments for fold and the initial algebra map. For natural numbers, the fold process and initial algebra map have the following type judgments:

```
stype nat     =  "forall X : stype.  !(1 + X -o X)  -o X"
stype foldT   =  "forall X : stype.  !(1 + X -o X)  -o nat -o X"
stype inNatT  =  "1 + nat -o nat"
```

Together, these allow us to define the constructors `zero` and `succ`. The protocol verifications for `zero` and `succ` use the process extracted from the verification of `inNatT` in cut, matching the representation presented by Toninho and Yoshida [116]. The following

proof script verifies the natural number protocols discussed here with additional proofs for the natural numbers *one* and *two*:

Listing 6.2: Verification of the natural numbers using polymorphic session types to represent inductive types.

```
-- Recursive Types for Free by Wadler 1990 https://homepages.inf.ed.ac.uk/wadler/papers/free-
     rectypes/free-rectypes.txt
module SNat begin

stype nat = "forall X : stype. !(1 + X -o X) -o X"

-- Right eliminator .
theorem foldNat: "forall X : stype. !(1 + X -o X) -o nat -o X"
apply Intros
apply BangLA
apply Forall2L b "X"
apply ImpliesL b
apply IdA
apply BangR
apply Copy c
apply IdA
done

extract foldNat
-- z(X).z(a).z(b).b[X].((nu d) b[d].(!d(e).((nu f) a[f].[f <-> e]) | [b <-> z]))

-- Initial algebra structure
theorem inNat: "(1 + nat) -o nat"
apply ImpliesR
apply Forall2R
apply ImpliesR
apply BangLA
apply Copy c
apply ImpliesL d
apply IdA
apply PlusLA
apply Forall2L a "X"
apply ImpliesLA
apply PlusR2
apply IdA
apply BangR
apply Copy c
apply IdA
```

```
apply PlusR1
apply IdA
done

extract inNat
-- z(a).z(X).z(b).((nu d) b[d].((nu e) d[e].(a.case(inl: e.inl;[a <-> e], inr: a[X].((nu f) a[f
     ].(!f(g).((nu h) b[h].[h <-> g]) | e.inr;[a <-> e])) ) | [d <-> z])))

theorem zero: "nat"
apply CutTheorem inNat
apply ImpliesLA
apply IdA
apply PlusR1
apply UnitR
done

extract zero
-- (nu i) (i(a).i(X).i(b).((nu d) b[d].((nu e) d[e].(a.case(inl: e.inl;[a <-> e], inr: a[X].((nu
     f) a[f].(!f(g).((nu h) b[h].[h <-> g]) | e.inr;[a <-> e])) ) | [d <-> i]))) | (nu a) i[a].(a
     .inl;0 | [i <-> z]))

theorem succ: "nat -o nat"
apply CutTheorem inNat
apply ImpliesR
apply ImpliesL i
defer
apply PlusR2
apply IdA
apply IdA
done

extract succ
-- (nu i) (i(a).i(X).i(b).((nu d) b[d].((nu e) d[e].(a.case(inl: e.inl;[a <-> e], inr: a[X].((nu
     f) a[f].(!f(g).((nu h) b[h].[h <-> g]) | e.inr;[a <-> e])) ) | [d <-> i]))) | z(a).((nu b) i
     [b].(b.inr;[a <-> b] | [i <-> z])))

theorem one: "nat"
apply CutTheorem zero
apply CutTheorem succ
apply ImpliesLA
apply Id k
apply IdA
done
```

```
extract one
-- (nu j) ((nu i) (i(a).i(X).i(b).((nu d) b[d].((nu e) d[e].(a.case(inl: e.inl;[a <-> e], inr: a[
     X].((nu f) a[f].(!f(g).((nu h) b[h].[h <-> g]) | e.inr;[a <-> e])) ) | [d <-> i]))) | (nu a)
      i[a].(a.inl;0 | [i <-> j])) | (nu k) ((nu i) (i(a).i(X).i(b).((nu d) b[d].((nu e) d[e].(a.
     case(inl: e.inl;[a <-> e], inr: a[X].((nu f) a[f].(!f(g).((nu h) b[h].[h <-> g]) | e.inr;[a
     <-> e])) ) | [d <-> i]))) | k(a).((nu b) i[b].(b.inr;[a <-> b] | [i <-> k]))) | (nu a) k[a
     ].([j <-> a] | [k <-> z])))

theorem two: "nat"
apply CutTheorem one
apply CutTheorem succ
apply ImpliesLA
apply Id j
apply IdA
done
```

The proofs of later theorems in this script rely heavily on the previous theorems. The most extensive proof is of *inNat*, and the subsequent proofs all rely on it either directly or indirectly through other theorems. This script demonstrates the usefulness of the *CutTheorem* tactic that allows the reuse of previously proved theorems. Each subsequent theorem would be larger than the last if the propositions had to be reproved in each theorem. This can be seen in the process extractions given in the comment after the *extract* command for each theorem. Note that when a theorem is applied to a proof, the free names in the derivation are renamed to avoid name conflicts. Even though all proofs are done using the active channel *z*, the theorems can still be used later in the proof script, and the extracted processes will also have the channels renamed appropriately.

6.5 Bit Counting Processes

The STILL prover allows for the specification of long-running systems with nodes that spawn and terminate according to session protocols. This can be demonstrated with the little endian bit counter presented by Toninho et al. [58]. The bit counter is made up of *Node* processes that are spawned if more space is required for the bit counter. Each *Node* holds a bit value and a channel for communicating to its neighboring node. This forms a network of communicating processes.

In addition to the *Node* processes, the full *Counter* is defined as a process. The *Counter* can be queried for the total value, incremented, and halted. When a signal is sent to the *Counter*, that signal is propagated along the *Node* network to the end, and the return signal is propagated back. A special node called *epsilon* represents the initial counter without any bits of storage. The following proof script first verifies the protocol for each piece of the counter, and then uses those proofs to verify the protocol of the full counter as a long-running process with a changing network topology:

Listing 6.3: Verification of the little endian bit counter protocol in the STILL prover.

```
module CounterSession begin

stype epsilonT = "forall int:Type 1.nu X. ((forall m:int.exists n:int.X) & (X + X) & 1)"
stype NodeT = "forall int:Type 1.forall b:int.forall compute:(Pi m:int.Pi b:int.int). (nu X. ((
     forall m:int.exists n:int.X) & (X + X) & 1)) -o (nu X. ((forall m:int.exists n:int.X) & (X +
      X) & 1))"
stype CounterT = "forall int:Type 1.forall compute:(Pi m:int.Pi b:int.int).forall zero:int.forall
      one:int.nu X. (X & X & 1)"
theorem epsilon: "epsilonT"
apply Intros
apply NuR X () ()
apply WithR
apply UnitR
apply WithR
apply PlusR1
apply TyVar X ()
apply ForallR
apply ExistsR
apply Exact "m"
apply Weaken m
apply TyVar X ()
done

-- CImpl -o CImpl
theorem Node: "NodeT"
apply Intros
apply NuR N (x) (a)
apply WithR
apply NuLA
apply WithL2A
apply UnitLA
apply UnitR
```

```
apply WithR
apply NuLA
apply WithL1A
apply WithL2A
apply PlusLA
apply PlusR2
apply TyVar N (a)
apply PlusR1
apply TyVar N (a)
apply NuLA
apply WithL1A
apply WithL1A
apply ForallR
apply ForallLA
apply Exact "compute m b"
apply ExistsLA
apply ExistsR
apply Exact "n"
apply Weaken m
apply Weaken n
apply TyVar N (a)
done

-- Counter nu X. {val:int /\ X, inc:X, halt:1}
theorem Counter: "CounterT"
apply CutTheorem epsilon
apply Intros
apply ForallLA
apply Exact "int"
apply NuR COORD (x) (a)
apply WithR
apply NuLA
apply WithL2A
apply UnitLA
apply UnitR
apply WithR
apply NuLA
apply WithL1A
apply WithL2A
apply PlusLA
apply TyVar COORD (a)
apply CutTheorem Node
apply ForallLA
apply Exact "int"
```

```
apply ForallLA
apply Exact "one"
apply ForallLA
apply Exact "compute"
apply ImpliesLA
defer
apply IdA
defer
apply TyVar COORD (c)
apply NuLA
apply WithL1A
apply WithL1A
apply ForallLA
apply Exact "zero"
apply ExistsLA
apply Weaken n
apply TyVar COORD (a)
done

extract Counter
-- (nu a) (a(int).(corec X().a.case(inl: a.case(inl: a(m).a[m].X(), inr: a.inl;X() ), inr: 0 ))
     () | z(int).z(compute).z(zero).z(one).a[int].(corec COORD(x).z.case(inl: z.case(inl: x.inl;x
     .inl;x[zero].x(n).COORD(x), inr: x.inl;x.inr;x.case(inl: (nu c) (c(int).c(b).c(compute).c(a)
     .(corec N(x).c.case(inl: c.case(inl: x.inl;x.inl;c(m).x[((compute m) b)].x(n).c[n].N(x), inr
     : x.inl;x.inr;x.case(inl: c.inl;N(x), inr: c.inr;N(x) ) ), inr: x.inr;0 )) (a) | c[int].c[
     one].c[compute].((nu d) c[d].([x <-> d] | COORD(c)))), inr: COORD(x) ) ), inr: x.inr;0 )) (a
     ))
```

The extracted process for the *Counter* begins with *epsilon* and a *COORD* process running concurrently. When *COORD* receives a signal to increment (*inl* then *inr*), it signals the most recently created *Node* to increment. Then, if *COORD* receives a carry signal (*inl*), a new node *N* is spawned and given the former last created *Node* as its neighbor, and the *COORD* process continues with its pointer to the last created *Node* updated. There is no limit to the number of *Node* processes in the network.

For the other signals that query the value or halt the counter, the *COORD* sends the signal to the last created *Node*. The signal is sent to the other *Node*s, and, in the case that the value is queried, the *Node* network returns the value of the counter.

6.6 Discussion

In this chapter, we have presented a variety of protocol verifications. The protocols presented here are selected to demonstrate the features available in the system of dependent session types that the STILL prover implements. The axiom of choice demonstrates the power of dependent Σ types in ECC, the natural numbers demonstrate the power of representing inductive session types using polymorphic session types, and the bit counter demonstrates the ability to verify long-running processes with changing topology. Additional protocol verifications can be found in the STILL source code repository.

Although the underlying theory that the STILL prover is based on is very powerful, improvements can still be made to the system. The proofs developed for the bit counter `Node` and `Counter` processes rely on a function to compute the total value of the counter based on the value sent by the neighboring node. This is assumed to be $2 * m + b$ by Toninho et al. [58], but it requires integers to be implemented as a primitive type, a feature that the STILL prover does not provide. Additionally, the extracted processes quickly become unreadable due to the process terms being fully expanded. Further work is required to improve extraction readability and to add useful primitive types to the STILL prover.

Another avenue of research worth investigating is the use of artificial intelligence (AI) to assist with the development of protocol verifications. Early experiments using ChatGPT [117] to assist with a proof that demonstrates the Needham-Schroeder man-in-the-middle attack [118] can be found in the source code of the STILL theorem prover [105]. However, additional work is required to improve the workflow used in these experiments. Currently, the user requests a protocol verification and provides the source code of the prover. The agent then attempts a proof without receiving feedback from the system during proof development. As a result, the user must repeatedly prompt the agent to fix errors and refine the proof. Further work is also needed to develop methodologies for assessing the correctness, reproducibility, and practical usefulness of proofs produced with AI assistance.

Chapter 7

Conclusion

The work presented in this dissertation represents the application of concurrent process theory to each stage of software development. For the final stage of software development, running the compiled program, we present the verification of the Marlowe small-step operational semantics using forward and backward simulation in Chapter 2. For the compilation and code writing steps, we present the verification of the Faustus compiler using forward and backward simulation for a language with guard expressions based on CCS operators in Chapter 3. For the code optimization step that lies between code writing and compilation, we present the transformation verification using bisimulation in Chapter 4. Finally, for the specification step before writing any code, we present the STILL theorem prover for specifying protocols using dependent session types and extracting process calculus terms from verifications of those protocols in Chapter 5 and Chapter 6. Each aspect of this research leaves many important topics to explore.

The verification of the Marlowe small-step semantics against the evaluator contains a gap between the evaluator in Isabelle and the evaluator implemented in Haskell. This gap also exists between the Faustus implementations in Isabelle and Haskell. There are two approaches that would close this gap. First, Haskell evaluators can be extracted from existing evaluators in Isabelle, which would provide partial guarantees of correctness [119]. Second, the formalizations can be ported to a system that allows for certified program extraction to Haskell, such as Agda [99]. Both of these approaches would require adjusting the formalized

evaluators to match all of the features of the Haskell evaluators, such as storing the hash of `Case` continuations rather than the whole `Contract`, which is expanded only when matching actions are provided. The Faustus Haskell implementation also uses a parser for its concrete syntax that would need to be implemented in the formalized code.

The formalization of Faustus can also be extended with abstractions for additional syntactic classes. Some of them, like the `Bound` and `POSIXTime` classes, would need to be evaluated at compile-time. Others, like `Case` and `Action`, would need to be evaluated at run-time in some way. Additionally, all abstractions could be parameterized. The compilation strategies would include inlining function bodies or using a `Contract` to adjust the data in the compiled Marlowe contract as needed before evaluating the abstraction of some other syntactic class. A prototype of these strategies can be found in the Faustus source code [65].

The verification of the transformation relies on the assumption of disjoint guards in the `Case` list. A transformation that makes all `Case` lists into `Case` lists with disjoint guards could be verified using the same techniques if the semantics of the input processing for the `Deposit` action is adjusted. The current semantics allow deposits of zero tokens when the value of a `Deposit` action evaluates to a non-positive result. If this behavior is prohibited, then it is possible to adjust any two `Action`s so that the first is prioritized over the second if they were to match on the same input. However, adjusting the Faustus semantics in this way would mean there is no backward simulation from the compiled Marlowe contract to the original Faustus contract, because the Marlowe contract would still allow deposits when the value of a `Deposit` action evaluates to a non-positive result. This means both the Marlowe and Faustus semantics would need to be modified along with their Haskell implementations to match the new semantics.

Finally, the STILL prover would benefit from a formalization of its trusted kernels that implement the system of dependent session types and the Extended Calculus of Constructions. There is a possible gap between the behavior described by the dependent session type protocols and the behavior of the processes that inhabit those session types. A verification of the correctness of the type-checking algorithms and program extractions would ensure

the extracted terms behave as the verified protocols describe. Further work also remains to translate the protocol extractions into a more usable form — possibly a form that is executable as a smart contract — and to add useful primitive data types and proof libraries so that users are not required to verify protocols using the most basic axioms and rules.

Appendix A

Marlowe Isabelle Formalization

Listing A.1: The Isabelle definition of the Marlowe small-step semantics.

```
theory SmallStep
imports Main Util.MList Util.SList Core.ListTools Core.Semantics "HOL-Library.Product_Lexorder" "
     HOL.Product_Type" "HOL-IMP.Star"
begin

type_synonym Configuration = "Contract * State * Environment * (ReduceWarning list) * (Payment
     list)"

inductive
  small_step_reduce :: "Configuration ⇒ Configuration ⇒ bool" (infix "→" 55)
where
CloseRefund:  "refundOne (accounts s) = Some ((party, token, money), newAccount) ⟹
  (Close, s, env, warns, payments) →
  (Close, (s⦇accounts := newAccount⦈), env, warns @ [ReduceNoWarning], payments @ [Payment party
       (SemanticsTypes.Party party) token money])" |
PayNonPositive: "evalValue env s val ≤ 0 ⟹
  (Pay accId payee token val cont, s, env, warns, payments) →
  (cont, s, env, warns @ [ReduceNonPositivePay accId payee token (evalValue env s val)], payments
       )" |
PayPositivePartialWithPayment: "⟦evalValue env s val > 0;
  evalValue env s val > moneyInAccount accId token (accounts s);
  updateMoneyInAccount accId token 0 (accounts s) = newAccs;
  moneyInAccount accId token (accounts s) = moneyToPay;
  giveMoney accId payee token (moneyToPay) newAccs = (payment, finalAccs);
  payment = ReduceWithPayment somePayment⟧ ⟹
  (Pay accId payee token val cont, s, env, warns, payments) →
  ((cont, s⦇accounts := finalAccs⦈, env, warns @ [ReducePartialPay accId payee token moneyToPay (
```

```
      evalValue env s val)], payments @ [somePayment]))" |
PayPositivePartialWithoutPayment: "⟦evalValue env s val > 0;
  evalValue env s val > moneyInAccount accId token (accounts s);
  updateMoneyInAccount accId token 0 (accounts s) = newAccs;
  moneyInAccount accId token (accounts s) = moneyToPay;
  giveMoney accId payee token (moneyToPay) newAccs = (payment, finalAccs);
  payment = ReduceNoPayment⟧ ⟹
  (Pay accId payee token val cont, s, env, warns, payments) →
  ((cont, s⦇accounts := finalAccs⦈, env, warns @ [ReducePartialPay accId payee token moneyToPay (
      evalValue env s val)], payments))" |
PayPositiveFullWithPayment: "⟦evalValue env s val > 0;
  evalValue env s val ≤ moneyInAccount accId token (accounts s);
  moneyInAccount accId token (accounts s) = moneyInAcc;
  moneyInAcc - (evalValue env s val) = newBalance;
  updateMoneyInAccount accId token newBalance (accounts s) = newAccs;
  giveMoney accId payee token (evalValue env s val) newAccs = (payment, finalAccs);
  payment = ReduceWithPayment somePayment⟧ ⟹
  (Pay accId payee token val cont, s, env, warns, payments) →
  (cont, s⦇accounts := finalAccs⦈, env, warns @ [ReduceNoWarning], payments @ [somePayment])" |
PayPositiveFullWithoutPayment: "⟦evalValue env s val > 0;
  evalValue env s val ≤ moneyInAccount accId token (accounts s);
  moneyInAccount accId token (accounts s) = moneyInAcc;
  moneyInAcc - (evalValue env s val) = newBalance;
  updateMoneyInAccount accId token newBalance (accounts s) = newAccs;
  giveMoney accId payee token (evalValue env s val) newAccs = (payment, finalAccs);
  payment = ReduceNoPayment⟧ ⟹
  (Pay accId payee token val cont, s, env, warns, payments) →
  (cont, s⦇accounts := finalAccs⦈, env, warns @ [ReduceNoWarning], payments)" |
IfTrue: "evalObservation env s obs ⟹
  (If obs cont1 cont2, s, env, warns, payments) →
  (cont1, s, env, warns @ [ReduceNoWarning], payments)" |
IfFalse: "¬evalObservation env s obs ⟹
  (If obs cont1 cont2, s, env, warns, payments) →
  (cont2, s, env, warns @ [ReduceNoWarning], payments)" |
WhenTimeout: "⟦timeInterval env = (startSlot, endSlot);
  endSlot ≥ timeout;
  startSlot ≥ timeout⟧ ⟹
  (When cases timeout cont, s, env, warns, payments) →
  (cont, s, env, warns @ [ReduceNoWarning], payments)" |
LetShadow: "lookup valId (boundValues s) = Some oldVal ⟹
  (Let valId val cont, s, env, warns, payments) →
  (cont, s⦇ boundValues := MList.insert valId (evalValue env s val) (boundValues s)⦈, env, warns
      @ [ReduceShadowing valId oldVal (evalValue env s val)], payments)" |
LetNoShadow: "lookup valId (boundValues s) = None ⟹
```

```
  (Let valId val cont, s, env, warns, payments) →
  (cont, s⦇ boundValues := MList.insert valId (evalValue env s val) (boundValues s)⦈, env, warns
       @ [ReduceNoWarning], payments)" |
AssertTrue: "evalObservation env s obs ⟹
  (Assert obs cont, s, env, warns, payments) →
  (cont, s, env, warns @ [ReduceNoWarning], payments)" |
AssertFalse: "¬evalObservation env s obs ⟹
  (Assert obs cont, s, env, warns, payments) →
  (cont, s, env, warns @ [ReduceAssertionFailed], payments)"

abbreviation
  small_step_reduces :: "Configuration ⇒ Configuration ⇒ bool" (infix "→*" 55)
  where "x →* y == star small_step_reduce x y"

(*Small Step Induction from FaustusSemantics*)
thm small_step_reduce.induct
lemmas small_step_reduce_induct = small_step_reduce.induct[split_format(complete)]
declare small_step_reduce.intros[simp,intro]

(*Needed to use Elimination rules for smallStepReductionImpReduceStep lemma*)
inductive_cases CloseE[elim]: "(Close, s, env, warns, payments) → ct"
thm CloseE
inductive_cases PayE[elim!]: "(Pay accId payee token val cont, s, env, warns, payments) → ct"
thm PayE
inductive_cases IfE[elim!]: "(If obs cont1 cont2, s, env, warns, payments) → ct"
thm IfE
inductive_cases WhenE[elim!]: "(When cases timeout cont, s, env, warns, payments) → ct"
thm WhenE
inductive_cases LetE[elim!]: "(Let valId val cont, s, env, warns, payments) → ct"
thm LetE
inductive_cases AssertE[elim!]: "(Assert obs cont, s, env, warns, payments) → ct"
thm AssertE

(*Warning Lemmas*)
lemma smallStepWarningsAreArbitrary:
"(c,  s, e, w, p) → (c', s', e', w', p') ⟹
  (∀w'' . ∃w''' . (c, s, e, w'', p) → (c', s', e', w''', p'))"
proof (induction c s e w p c' s' e' w' p' rule: small_step_reduce_induct)
  case (CloseRefund s party token money newAccount env warns payments)
  then show ?case by auto
next
  case (PayNonPositive env s val accId payee token cont warns payments)
  then show ?case by auto
next
```

```
  case (PayPositivePartialWithPayment env s val accId token newAccs moneyToPay payee payment
        finalAccs somePayment cont warns payments)
  then show ?case apply auto
    apply (cases "lookup (accId, token) (accounts s)", auto)
    using PayPositivePartialWithPayment.hyps(5) apply force
    using PayPositivePartialWithPayment.hyps(5) by force
next
  case (PayPositivePartialWithoutPayment env s val accId token newAccs moneyToPay payee payment
        finalAccs cont warns payments)
  then show ?case by auto
next
  case (PayPositiveFullWithPayment env s val accId token moneyInAcc newBalance newAccs payee
        payment finalAccs somePayment cont warns payments)
  then show ?case apply auto
    apply (cases "lookup (accId, token) (accounts s)", auto)
    apply (cases "moneyInAcc = evalValue env s val", auto)
     apply fastforce
    by fastforce
next
  case (PayPositiveFullWithoutPayment env s val accId token moneyInAcc newBalance newAccs payee
        payment finalAccs cont warns payments)
  then show ?case by auto
next
  case (IfTrue env s obs cont1 cont2 warns payments)
  then show ?case by auto
next
  case (IfFalse env s obs cont1 cont2 warns payments)
  then show ?case by auto
next
  case (WhenTimeout env startSlot endSlot timeout cases cont s warns payments)
  then show ?case by auto
next
  case (LetShadow valId s oldVal val cont env warns payments)
  then show ?case by auto
next
  case (LetNoShadow valId s val cont env warns payments)
  then show ?case by auto
next
  case (AssertTrue env s obs cont warns payments)
  then show ?case by auto
next
  case (AssertFalse env s obs cont warns payments)
  then show ?case by auto
qed
```

```
lemma smallStepStarWarningsAreArbitrary:
  "(c, s, e, w, p) →* (c', s', e', w'', p') ⟹
    (∀w'' . ∃w''' . (c, s, e, w'', p) →* (c', s', e', w''', p'))"
apply (induction rule: star.induct[of "small_step_reduce", split_format(complete)], auto)
  by (meson smallStepWarningsAreArbitrary star.step)

definition "finalMarlowe cs ⟷ ¬(∃cs'. cs → cs')"

lemma finalD:
"finalMarlowe (contract, s, e, w, p) ⟹
  contract = Close ∨ (∃ cases timeout continue . contract = When cases timeout continue)"
  apply (auto simp add: finalMarlowe_def)
  apply (cases contract, auto)
     apply (meson PayNonPositive PayPositiveFullWithPayment PayPositiveFullWithoutPayment
          PayPositivePartialWithPayment PayPositivePartialWithoutPayment giveMoney.elims not_less
          )
    apply (meson IfFalse IfTrue)
   apply (meson LetShadow option.exhaust_sel small_step_reduce.intros(11))
  by (meson small_step_reduce.intros(12) small_step_reduce.intros(13))

(*BEGIN proof of Reduce Step is a Small Step Reduction in Marlowe*)
lemma reduceStepIsSmallStepReduction:
assumes "reduceContractStep env state contract = Reduced warning paymentReduceResult newState
     continueContract"
shows "∃ appendPayment .(contract, state, env, initialWarnings, initialPayments) → (
     continueContract, newState, env, initialWarnings @ [warning], initialPayments @
     appendPayment)"
proof (cases contract paymentReduceResult rule: Contract.exhaust[case_product ReduceEffect.
     exhaust])
  case Close_ReduceNoPayment
  then show ?thesis using assms apply auto
    by (cases "refundOne (accounts state)", auto)
next
  case (Close_ReduceWithPayment reducePayment)
  then show ?thesis using assms apply auto
    by (cases "refundOne (accounts state)", auto)
next
  case (Pay_ReduceNoPayment accId payee token val cont)
  then show ?thesis using assms apply auto
    apply (cases "evalValue env state val ≤ 0", auto)
     apply (metis PayNonPositive append_Nil2)
    subgoal premises ps proof (cases "lookup (accId, token) (accounts state)")
      case None
```

```
      moreover have "(min 0 (evalValue env state val)) = 0" using ps(4) by auto
      ultimately show ?thesis using ps apply auto
        subgoal premises ps proof (cases "giveMoney accId payee token 0 (MList.delete (accId,
             token) (accounts state))")
          case (Pair payment finalAccs)
          moreover from ps have "(Contract.Pay accId payee token val continueContract, state, env
               ,
                     initialWarnings, initialPayments) →
                   (continueContract, state(accounts := finalAccs), env,
                    initialWarnings @ [ReducePartialPay accId payee token 0 (evalValue env state
                         val)],
                    initialPayments @ [])" using ps calculation by auto
          ultimately show ?thesis using ps by blast
        qed
        done
    next
      case (Some availableBal)
      moreover have "availableBal ≤ evalValue env state val ⟹ min availableBal (evalValue env
           state val) = availableBal" by simp
      moreover have "¬(availableBal ≤ evalValue env state val) ⟹ min availableBal (evalValue
           env state val) = (evalValue env state val)" by simp
      ultimately show ?thesis using ps apply auto
        apply (cases "availableBal ≤ evalValue env state val", auto)
        subgoal premises ps proof (cases "giveMoney accId payee token availableBal (MList.delete
             (accId, token) (accounts state))")
          case (Pair payment finalAccs)
          then show ?thesis using ps apply auto
            by (cases "availableBal < (evalValue env state val)", auto)
        qed
        subgoal premises ps proof (cases "giveMoney accId payee token (evalValue env state val) (
             MList.insert (accId, token) (availableBal - (evalValue env state val)) (accounts
             state))")
          case (Pair payment finalAccs)
          then show ?thesis using ps by auto
        qed
        done
    qed
    done
next
  case (Pay_ReduceWithPayment accId payee token val cont payment)
  then show ?thesis using assms apply auto
    apply (cases "evalValue env state val ≤ 0", auto)
    subgoal premises ps proof (cases "lookup (accId, token) (accounts state)")
      case None
```

```
    moreover from ps have "min 0 (evalValue env state val) = 0" by simp
    ultimately show ?thesis using ps apply auto
      subgoal premises ps proof (cases "giveMoney accId payee token 0 (MList.delete (accId,
           token) (accounts state))")
        case (Pair reduceEffect finalAccs)
        moreover from ps calculation have "(Contract.Pay accId payee token val continueContract
             , state, env,
                              initialWarnings, initialPayments) →
                             (continueContract, state(accounts := finalAccs), env,
                              initialWarnings @ [ReducePartialPay accId payee token 0 (evalValue
                                   env state val)],
                              initialPayments @ [payment])" by auto
        (*by auto before elimination rules*)
        ultimately show ?thesis using ps by (auto, meson, metis)
      qed
      done
  next
    case (Some bal)
    then show ?thesis using ps apply auto
      apply (cases "bal ≤ evalValue env state val", auto)
      subgoal premises ps proof -
        from ps have "min bal (evalValue env state val) = bal" by simp
        then show ?thesis using ps apply auto
          subgoal premises ps proof (cases "giveMoney accId payee token bal (MList.delete (
               accId, token) (accounts state))")
            case (Pair reduceEffect finalAccs)
            then show ?thesis using ps apply auto
              apply (cases "bal < evalValue env state val", auto)
              subgoal premises ps proof -
                from ps have "(Contract.Pay accId payee token val continueContract, state, env,
                                initialWarnings, initialPayments) →
                               (continueContract, state(accounts := finalAccs), env,
                                initialWarnings @ [ReducePartialPay accId payee token bal (
                                     evalValue env state val)],
                                initialPayments @ [payment])" by simp
                then show ?thesis using ps by meson
              qed
              subgoal premises ps proof -
                from ps have "(Contract.Pay accId payee token val continueContract, state, env,
                                initialWarnings, initialPayments) →
                               (continueContract, state(accounts := finalAccs), env,
                                initialWarnings @ [ReduceNoWarning], initialPayments @ [payment])
                                     " by simp
                then show ?thesis using ps by meson
```

```
              qed
              done
          qed
          done
      qed
      subgoal premises ps proof -
        from ps have "min bal (evalValue env state val) = evalValue env state val" by simp
        then show ?thesis using ps apply auto
          subgoal premises ps proof (cases "giveMoney accId payee token (evalValue env state
               val) (MList.insert (accId, token) (bal - (evalValue env state val)) (accounts
               state))")
            case (Pair reduceEffect finalAccs)
            moreover from ps calculation have "(Contract.Pay accId payee token val
                 continueContract, state, env,
                                initialWarnings, initialPayments) →
                               (continueContract, state(accounts := finalAccs), env,
                                initialWarnings @ [ReduceNoWarning], initialPayments @ [payment
                                     ])" by auto
            (*used calculation by auto before elimination rules*)
            ultimately show ?thesis using ps by force
          qed
          done
      qed
      done
  qed
  done

next
  case (If_ReduceNoPayment x31 x32 x33)
  then show ?thesis using assms apply auto
     apply (metis IfTrue append_Nil2)
    by (metis IfFalse append.right_neutral)
next
  case (If_ReduceWithPayment x31 x32 x33 x2)
  then show ?thesis using assms by auto
next
  case (When_ReduceNoPayment x41 timeout x43)
  then show ?thesis using assms apply auto
    subgoal premises ps proof (cases "timeInterval env")
      case (Pair ss es)
      then show ?thesis using ps apply auto
        apply (cases "es < timeout", auto)
        apply (cases "timeout ≤ ss", auto)
        by (smt WhenTimeout append_Nil2)
```

```
    qed
    done
next
  case (When_ReduceWithPayment x41 x42 x43 x2)
  then show ?thesis using assms apply auto
  by (smt ReduceStepResult.distinct(1) ReduceStepResult.distinct(3) ReduceStepResult.inject
       Pair_inject ReduceEffect.distinct(1) fixInterval.cases old.prod.case)
next
  case (Let_ReduceNoPayment vid x52 x53)
  then show ?thesis using assms apply auto
    apply (cases "lookup vid (boundValues state)", auto)
     apply (metis LetNoShadow append_Nil2)
    by (metis (no_types, lifting) ReduceStepResult.inject LetShadow State.unfold_congs(3)
         append_Nil2)
next
  case (Let_ReduceWithPayment vid x52 x53 x2)
  then show ?thesis using assms apply auto
    apply (cases "lookup vid (boundValues state)", auto)
    by (meson ReduceStepResult.inject ReduceEffect.distinct(1))
next
  case (Assert_ReduceNoPayment x61 x62)
  then show ?thesis using assms apply auto
     apply (metis AssertTrue append.right_neutral)
    by (metis AssertFalse append_Nil2)
next
  case (Assert_ReduceWithPayment x61 x62 x2)
  then show ?thesis using assms by auto
next
qed

lemma reduceStepIsSmallStepReductionNoPayment:
assumes
  "reduceContractStep e s1 c1 = Reduced w
    ReduceNoPayment s2 c2"
shows
  "(c1, s1, e, ws, ps) → (c2, s2, e, ws @ [w], ps)"
proof (cases c1)
  case Close
  then show ?thesis using assms apply auto
    by (cases "refundOne (accounts s1)", auto)
next
  case (Pay accId payee token val cont)
  then show ?thesis using assms apply auto
    apply (cases "evalValue e s1 val ≤ 0", auto)
```

```
    subgoal premises ps proof (cases "lookup (accId, token) (accounts s1)")
      case None
      moreover have "(min 0 (evalValue e s1 val)) = 0" using ps by auto
      ultimately show ?thesis using ps apply auto
        subgoal premises ps proof (cases "giveMoney accId payee token 0 (MList.delete (accId,
             token) (accounts s1))")
          case (Pair payment finalAccs)
          moreover from ps have "(Contract.Pay accId payee token val c2, s1, e,
                     ws, ps) →
                    (c2, s1(accounts := finalAccs), e,
                     ws @ [ReducePartialPay accId payee token 0 (evalValue e s1 val)],
                     ps @ [])" using ps calculation by auto
          ultimately show ?thesis using ps by simp
        qed
        done
    next
      case (Some availableBal)
      moreover have "availableBal ≤ evalValue e s1 val ⟹ min availableBal (evalValue e s1 val)
           = availableBal" by simp
      moreover have "¬(availableBal ≤ evalValue e s1 val) ⟹ min availableBal (evalValue e s1
           val) = (evalValue e s1 val)" by simp
      ultimately show ?thesis using ps apply auto
        apply (cases "availableBal ≤ evalValue e s1 val", auto)
        subgoal premises ps proof (cases "giveMoney accId payee token availableBal (MList.delete
             (accId, token) (accounts s1))")
          case (Pair payment finalAccs)
          then show ?thesis using ps by (cases "availableBal < (evalValue e s1 val)", auto)
        qed
        subgoal premises ps proof (cases "giveMoney accId payee token (evalValue e s1 val) (MList
             .insert (accId, token) (availableBal - (evalValue e s1 val)) (accounts s1))")
          case (Pair payment finalAccs)
          then show ?thesis using ps by auto
        qed
        done
    qed
    done
next
  case (If x31 x32 x33)
  then show ?thesis using assms by auto
next
  case (When x41 timeout x43)
  then show ?thesis using assms apply auto
    subgoal premises ps proof (cases "timeInterval e")
      case (Pair ss es)
```

```
      then show ?thesis using ps apply auto
        apply (cases "es < timeout", auto)
        by (cases "timeout ≤ ss", auto)
    qed
    done
next
  case (Let vid x52 x53)
  then show ?thesis using assms apply auto
    apply (cases "lookup vid (boundValues s1)", auto)
    by (metis (no_types, lifting) ReduceStepResult.inject LetShadow State.unfold_congs(3))
next
  case (Assert x61 x62)
  then show ?thesis using assms by auto
qed

lemma reduceStepIsSmallStepReductionWithPayment:
  assumes
    "reduceContractStep e s1 c1 = Reduced w
      (ReduceWithPayment p) s2 c2"
  shows
    "(c1, s1, e, ws, ps) → (c2, s2, e, ws @ [w], ps @ [p])"
proof (cases c1)
  case (Close)
  then show ?thesis using assms by (cases "refundOne (accounts s1)", auto)
next
  case (Pay accId payee token val cont)
  then show ?thesis using assms apply auto
    apply (cases "evalValue e s1 val ≤ 0", auto)
    subgoal premises ps proof (cases "lookup (accId, token) (accounts s1)")
      case None
      moreover from ps have "min 0 (evalValue e s1 val) = 0" by simp
      ultimately show ?thesis using ps apply auto
        subgoal premises ps proof (cases "giveMoney accId payee token 0 (MList.delete (accId,
             token) (accounts s1))")
          case (Pair reduceEffect finalAccs)
          moreover from ps calculation have "(Contract.Pay accId payee token val c2, s1, e,
                                ws, ps) →
                               (c2, s1⦇accounts := finalAccs⦈, e,
                                ws @ [ReducePartialPay accId payee token 0 (evalValue e s1 val)],
                                ps @ [p])" by auto
          ultimately show ?thesis using ps by auto
        qed
        done
    next
```

```
      case (Some bal)
      then show ?thesis using ps apply auto
        apply (cases "bal ≤ evalValue e s1 val", auto)
        subgoal premises ps proof -
          from ps have "min bal (evalValue e s1 val) = bal" by simp
          then show ?thesis using ps apply auto
            subgoal premises ps proof (cases "giveMoney accId payee token bal (MList.delete (
                 accId, token) (accounts s1))")
              case (Pair reduceEffect finalAccs)
              then show ?thesis using ps by (cases "bal < evalValue e s1 val", auto)
            qed
            done
        qed
        subgoal premises ps proof -
          from ps have "min bal (evalValue e s1 val) = evalValue e s1 val" by simp
          then show ?thesis using ps apply auto
            subgoal premises ps proof (cases "giveMoney accId payee token (evalValue e s1 val) (
                 MList.insert (accId, token) (bal - (evalValue e s1 val)) (accounts s1))")
              case (Pair reduceEffect finalAccs)
              moreover from ps calculation have "(Contract.Pay accId payee token val c2, s1, e,
                                   ws, ps) →
                                  (c2, s1(accounts := finalAccs), e,
                                   ws @ [ReduceNoWarning], ps @ [p])" by auto
              ultimately show ?thesis using ps by (auto)
            qed
            done
        qed
        done
    qed
    done
next
  case (If x31 x32 x33)
  then show ?thesis using assms by auto
next
  case (When x41 x42 x43)
  then show ?thesis using assms apply auto
  by (smt ReduceStepResult.distinct(1) ReduceStepResult.distinct(3) ReduceStepResult.inject
       Pair_inject ReduceEffect.distinct(1) fixInterval.cases old.prod.case)
next
  case (Let vid x52 x53)
  then show ?thesis using assms apply auto
    apply (cases "lookup vid (boundValues s1)", auto)
    by (meson ReduceStepResult.inject ReduceEffect.distinct(1))
next
```

```
  case (Assert x61 x62)
  then show ?thesis using assms by auto
qed

(*BEGIN proof Small Step in Marlowe implies that there is a Reduced Step*)

lemma smallStepReductionAddsOneWarning:
assumes "cs = (contract, state, env, initialWarnings, initialPayments) ∧
    cs' = (continueContract, s2, newEnv, newWarnings, newPayments) ∧
    cs → cs'"
shows "∃newWarning . newWarnings = initialWarnings @ [newWarning]"
proof (cases contract)
  case Close
  then show ?thesis using assms by auto
next
  case (Pay accId payee token val cont)
  then show ?thesis using assms by auto
next
  case (If x31 x32 x33)
  then show ?thesis using assms by auto
next
  case (When cases timeout cont)
  then show ?thesis using assms by auto
next
  case (Let x51 x52 x53)
  then show ?thesis using assms by auto
next
  case (Assert x61 x62)
  then show ?thesis using assms by auto
qed

lemma smallStepReductionImpReduceStep:
assumes "cs = (contract, state, env, initialWarnings, initialPayments) ∧
    cs' = (continueContract, newState, newEnv, initialWarnings @ [warning], newPayments) ∧
    cs → cs'"
shows "∃ paymentReduceResult . reduceContractStep env state contract = Reduced warning
     paymentReduceResult newState continueContract"
proof (cases contract)
  case Close
  then show ?thesis using assms by auto
next
  case (Pay accId payee token val cont)
  then show ?thesis using assms apply (cases payee, auto)
           apply metis
```

```
          apply metis
         apply metis
        apply metis
       apply metis
      apply metis
     apply metis
    by metis
next
  case (If x31 x32 x33)
  then show ?thesis using assms by auto
next
  case (When cases timeout cont)
  then show ?thesis using assms by auto
next
  case (Let x51 x52 x53)
  then show ?thesis using assms apply auto
    by metis
next
  case (Assert x61 x62)
  then show ?thesis using assms by auto
qed

lemma smallStepReductionGivesOnePaymentMax:
assumes "cs = (contract, state, env, initialWarnings, initialPayments) ∧
    cs' = (continueContract, newState, newEnv, initialWarnings @ [warning], newPayments) ∧
    cs → cs'"
shows "newPayments = initialPayments ∨ (∃newPayment . newPayments = initialPayments @ [newPayment
     ])"
proof (cases contract)
  case Close
  then show ?thesis using assms by auto
next
  case (Pay x21 x22 x23 x24 x25)
  then show ?thesis using assms by auto
next
  case (If x31 x32 x33)
  then show ?thesis using assms by auto
next
  case (When x41 x42 x43)
  then show ?thesis using assms by auto
next
  case (Let x51 x52 x53)
  then show ?thesis using assms by auto
next
```

```
  case (Assert x61 x62)
  then show ?thesis using assms by auto
qed

lemma smallStepReductionImpReduceStepNoPayment:
  assumes
    "(c1, s1, e1, ws, ps) → (c2, s2, e2, ws @ [w], ps)"
  shows
    "reduceContractStep e1 s1 c1 = Reduced w
      ReduceNoPayment s2 c2"
proof (cases c1)
  case Close
  then show ?thesis using assms by auto
next
  case (Pay accId payee token val cont)
  then show ?thesis using assms by auto
next
  case (If x31 x32 x33)
  then show ?thesis using assms by auto
next
  case (When cases timeout cont)
  then show ?thesis using assms by auto
next
  case (Let x51 x52 x53)
  then show ?thesis using assms apply auto
    by metis
next
  case (Assert x61 x62)
  then show ?thesis using assms by auto
qed

lemma smallStepReductionImpReduceStepWithPayment:
assumes
  "(c1, s1, e1, ws, ps) → (c2, s2, e2, ws @ [w], ps @ [p])"
shows
  "reduceContractStep e1 s1 c1 = Reduced w
    (ReduceWithPayment p) s2 c2"
proof (cases c1)
  case Close
  then show ?thesis using assms by auto
next
  case (Pay accId payee token val cont)
  then show ?thesis using assms apply (cases payee, auto)
          apply metis
```

```
          apply metis
         apply metis
        apply metis
       apply metis
      apply metis
     apply metis
    by metis
next
  case (If x31 x32 x33)
  then show ?thesis using assms by auto
next
  case (When cases timeout cont)
  then show ?thesis using assms by auto
next
  case (Let x51 x52 x53)
  then show ?thesis using assms by auto
next
  case (Assert x61 x62)
  then show ?thesis using assms by auto
qed

type_synonym ActionConfig = "Action * State * Environment"

type_synonym ActionRes = "State"

inductive small_step_reduce_action_marlowe :: "Input ⇒ ActionConfig ⇒ ActionRes ⇒ bool" ("_ ⊢
     _ →a_m _" [55, 0, 55]) where
DepositPositiveActionStep:  "⟦evalValue env s aValue = money⟧ ⟹
  (IDeposit accId party token money) ⊢ (Deposit accId party token aValue, s, env) →a_m
  (s⦇accounts := addMoneyToAccount accId token money (accounts s)⦈)" |
ChoiceActionStep:  "⟦inBounds choice bounds⟧ ⟹
  (IChoice choiceId choice) ⊢ (Choice choiceId bounds, s, env) →a_m
  (s⦇ choices := MList.insert choiceId choice (choices s) ⦈)" |
NotifyActionStep:  "⟦evalObservation env s obs = True⟧ ⟹
  INotify ⊢ (Notify obs, s, env) →a_m (s)"

inductive_cases DepositActionE[elim]: "ins ⊢ (Deposit fAccId fParty fToken fValue, s, env) →a_m
     aRes"
inductive_cases ChoiceActionE[elim]: "ins ⊢ (Choice fChoiceId bounds, s, env) →a_m aRes"
inductive_cases NotifyActionE[elim]: "ins ⊢ (Notify obs, s, env) →a_m aRes"

lemma deterministicActionSemantics:
"ins ⊢ a →a_m a' ⟹
ins ⊢ a →a_m a'' ⟹
```

```
a' = a'''"
  by (cases ins a a' rule: small_step_reduce_action_marlowe.cases;
      cases ins a a'' rule: small_step_reduce_action_marlowe.cases;
      auto split: option.split_asm)

fun computeActionStepMarlowe :: "Input ⇒ ActionConfig ⇒ ActionRes option" where
"computeActionStepMarlowe (IDeposit iAccId iParty iToken iMoney) (Deposit aAccId aParty aToken
     aValue, s, env) = (
  if (evalValue env s aValue = iMoney ∧ iAccId = aAccId ∧ iParty = aParty ∧ iToken = aToken)
  then (Some (s ⦇ accounts := addMoneyToAccount iAccId iToken iMoney (accounts s) ⦈))
  else None)" |
"computeActionStepMarlowe (IChoice iChoiceId choice) (Choice aChoiceId bounds, s, env) = (
  if (inBounds choice bounds ∧ iChoiceId = aChoiceId)
  then Some (s⦇ choices := MList.insert aChoiceId choice (choices s) ⦈)
  else None)" |
"computeActionStepMarlowe (INotify) (Notify obs, s, env) = (
  if evalObservation env s obs = True
  then Some (s)
  else None)" |
"computeActionStepMarlowe INotify _ = None" |
"computeActionStepMarlowe _ _ = None"

lemma computeActionStepMarloweIsSmallStep:
"computeActionStepMarlowe ins a = Some res ⟷ ins ⊢ a →a_m res"
  apply auto
  apply (cases "(ins, a)" rule: computeActionStepMarlowe.cases)
  using DepositPositiveActionStep NotifyActionStep ChoiceActionStep apply (auto split: option.
       split_asm if_split_asm simp del: inBounds.simps moneyInAccount.simps addMoneyToAccount.
       simps)
  by (cases ins a res rule: small_step_reduce_action_marlowe.cases, auto)

type_synonym MCaseConfig = "Case * State * Environment"
type_synonym MCaseConfigRes = "Contract * State"

inductive small_step_reduce_case :: "Input ⇒ MCaseConfig ⇒ MCaseConfigRes ⇒ bool" ("_ ⊢ _ →ca
     se_m _" [55, 0, 55]) where
CaseActionMatchStep:  "⟦ins ⊢ (a, s, env) →a_m newS⟧ ⟹
  ins ⊢ (Case a c, s, env) →case_m (c, newS)"

inductive_cases MCaseStepE: "ins ⊢ (Case g c, s, env) →case_m res"

fun computeCaseStepMarlowe :: "Input ⇒ MCaseConfig ⇒ MCaseConfigRes option" where
"computeCaseStepMarlowe ins (Case a c, s, env) = (case computeActionStepMarlowe ins (a, s, env)
     of
```

```
  Some newS ⇒ Some (c, newS)
  | None ⇒ None)"

lemma computeCaseStepMarloweResImpliesReduceStep:
"computeCaseStepMarlowe ins c = Some res ⟹
  ins ⊢ c →case_m res"
  apply (cases "(ins, c)" rule: computeCaseStepMarlowe.cases)
  by (auto split: option.split_asm simp add: CaseActionMatchStep
       computeActionStepMarloweIsSmallStep)

lemma reduceCaseStepImpliesComputeCaseStepRes:
"ins ⊢ c →case_m res ⟹
  computeCaseStepMarlowe ins c = Some res"
  apply (induction ins c res rule: small_step_reduce_case.induct)
  by (auto split: option.split simp add: computeActionStepMarloweIsSmallStep
       deterministicActionSemantics)

type_synonym MCasesConfig = "Case list * State * Environment"
type_synonym MCasesConfigRes = "Contract * State"

inductive small_step_reduce_cases_marlowe :: "Input ⇒ MCasesConfig ⇒ MCasesConfigRes ⇒ bool"
     ("_ ⊢ _ →cases_m _" [55, 0, 55]) where
CasesHeadMatches: "ins ⊢ (c, s, env) →case_m (newC, newS) ⟹
  ins ⊢ (c#rest, s, env) →cases_m (newC, newS)" |
CasesTailMatches: "⟦ins ⊢ (rest, s, env) →cases_m (newC, newS);
  (∀res . ¬(ins ⊢ (c, s, env) →case_m res))⟧ ⟹
  ins ⊢ (c#rest, s, env) →cases_m (newC, newS)"

inductive_cases SingleCaseCasesStepE: " ins ⊢ ([c], state, env) →cases_m res"
inductive_cases ConsCasesStepE: "ins ⊢ (c#cs, state, env) →cases_m res"

definition "finalCasesM ins cConfig ⟷ (∀res . ¬(ins ⊢ cConfig →cases_m res))"

fun computeCasesStepMarlowe :: "Input ⇒ MCasesConfig ⇒ MCasesConfigRes option" where
"computeCasesStepMarlowe ins ([], _) = None" |
"computeCasesStepMarlowe ins (c#cs, s, env) = (case computeCaseStepMarlowe ins (c, s, env) of
  Some res ⇒ Some res
  | None ⇒ computeCasesStepMarlowe ins (cs, s, env))"

lemma computeCasesStepMarloweResImpliesReduceStep:
"computeCasesStepMarlowe ins c = Some res ⟹
  ins ⊢ c →cases_m res"
  apply (induction ins c arbitrary: res rule: computeCasesStepMarlowe.induct)
   apply (auto split: option.split_asm simp add: reduceCaseStepImpliesComputeCaseStepRes
```

```
        CasesTailMatches)
   apply (smt (verit, best) CasesTailMatches option.distinct(1)
        reduceCaseStepImpliesComputeCaseStepRes)
  by (simp add: CasesHeadMatches computeCaseStepMarloweResImpliesReduceStep)

lemma reduceCasesStepImpliesComputeCaseStepRes:
"ins ⊢ c →cases_m res ⟹
  computeCasesStepMarlowe ins c = Some res"
  apply (induction ins c res rule: small_step_reduce_cases_marlowe.induct)
   apply (simp add: reduceCaseStepImpliesComputeCaseStepRes)
  by (metis computeCaseStepMarloweResImpliesReduceStep computeCasesStepMarlowe.simps(2) option.
       exhaust option.simps(4))

lemma deterministicCaseSemantics:
"ins ⊢ c →case_m c' ⟹
  ins ⊢ c →case_m c'' ⟹
  c' = c''"
  by (metis option.inject reduceCaseStepImpliesComputeCaseStepRes)

lemma deterministicCasesSemantics:
"ins ⊢ c →cases_m c' ⟹
  ins ⊢ c →cases_m c'' ⟹
  c' = c''"
  by (metis option.inject reduceCasesStepImpliesComputeCaseStepRes)

lemma casesStepImpliesAppendCasesStep:
"ins ⊢ (c1, state, env) →cases_m (casesRes, state') ⟹
  ins ⊢ (c1 @ c2, state, env) →cases_m (casesRes, state')"
  apply (induction ins c1 state env casesRes state' rule: small_step_reduce_cases_marlowe.induct[
       split_format(complete)])
  by (simp_all add: CasesHeadMatches CasesTailMatches)

lemma finalCasesMTailNoDef:
"¬ (∃res . ins ⊢ (c#c1, state, env) →cases_m res) ⟹
  (∃innerRes . ins ⊢ (c1, state, env) →cases_m innerRes) ⟹
  False"
  apply (cases "(∃innerRes2 . ins ⊢ (c, state, env) →case_m innerRes2)")
   apply (simp add: computeCasesStepMarloweResImpliesReduceStep
        reduceCaseStepImpliesComputeCaseStepRes)
  subgoal premises ps proof -
    obtain innerRes2 where "ins ⊢ (c, state, env) →case_m innerRes2" using ps by blast
    then have "ins ⊢ (c # c1, state, env) →cases_m innerRes2" using CasesHeadMatches ps
         deterministicCaseSemantics by blast
    then show ?thesis using ps \<open>ins ⊢ (c, state, env) →case_m innerRes2\<close>
```

```
        deterministicCaseSemantics by blast
  qed
  subgoal premises ps proof -
    obtain innerRes where "ins ⊢ (c1, state, env) →cases_m innerRes" using ps by blast
    then have "ins ⊢ (c # c1, state, env) →cases_m innerRes" using ps CasesTailMatches by (auto,
        meson)
    then show ?thesis using ps by blast
  qed
  done

lemma finalCasesMTail:
"finalCasesM ins (c#c1, state, env) ⟹
  finalCasesM ins (c1, state, env)"
  by (meson finalCasesMTailNoDef finalCasesM_def)

lemma finalCasesMAppend:
"finalCasesM ins (c1, state, env) ⟹
  finalCasesM ins (c2, state, env) ⟹
  finalCasesM ins (c1 @ c2, state, env)"
  apply (auto simp add: finalCasesM_def)
proof (induction c1 arbitrary:c2)
  case Nil
  then show ?case by auto
next
  case (Cons firstCase c1)
  moreover have "finalCasesM ins (firstCase#c1, state, env)" using Cons calculation
      finalCasesM_def by auto
  moreover have "finalCasesM ins (c1, state, env)" using Cons calculation finalCasesMTail by auto
  moreover have "finalCasesM ins (c2, state, env)" using Cons calculation finalCasesM_def by auto
  moreover have "finalCasesM ins (c1 @ c2, state, env)" using calculation finalCasesM_def by auto
  ultimately show ?case using Cons apply auto
    by (metis (no_types, opaque_lifting) computeCasesStepMarlowe.simps(2)
        computeCasesStepMarloweResImpliesReduceStep not_Some_eq option.simps(4) option.simps(5)
        reduceCasesStepImpliesComputeCaseStepRes)
qed

lemma casesStepAppendFinalCases:
"finalCasesM ins (c2, state, env) ⟹
  ins ⊢ (c1, state, env) →cases_m res ⟹
  ins ⊢ (c2 @ c1, state, env) →cases_m res"
proof (induction c2 arbitrary: c1)
  case Nil
  then show ?case by auto
next
```

```
  case (Cons a c2)
  moreover have "finalCasesM ins (c2, state, env)" using calculation finalCasesMTail by auto
  moreover have "ins ⊢ (c2 @ c1, state, env) →cases_m res" using calculation by auto
  ultimately show ?case apply (auto simp add: finalCasesM_def)
    by (metis CasesHeadMatches CasesTailMatches surj_pair)
qed

thm small_step_reduce_cases_marlowe.induct[split_format(complete)]

lemma casesStepDropFinalCases:
"ins ⊢ (c1 @ c2, state, env) →cases_m (newC, newState) ⟹
  finalCasesM ins (c2, state, env) ⟹
  ins ⊢ (c1, state, env) →cases_m (newC, newState)"
proof (induction ins "c1@c2" state env newC newState arbitrary: c1 c2 rule:
     small_step_reduce_cases_marlowe.induct[split_format(complete)])
  case (CasesHeadMatches ins c s env newC newS rest)
  then show ?case by (smt (verit, ccfv_threshold) append_eq_Cons_conv finalCasesM_def
       small_step_reduce_cases_marlowe.CasesHeadMatches)
next
  case (CasesTailMatches ins rest s env newC newS c)
  then show ?case by (smt (verit, ccfv_SIG) append_eq_Cons_conv append_self_conv2 finalCasesMTail
        small_step_reduce_cases_marlowe.CasesTailMatches)
qed

lemma fixIntervalOnlyTimeDependent:
"minTime state1 = minTime state2 ⟹
    fixInterval (lower, upper) state1 = IntervalTrimmed newTimeEnv1 newTimeState1 ⟹
    fixInterval (lower, upper) state2 = IntervalTrimmed newTimeEnv2 newTimeState2 ⟹
    minTime newTimeState1 = minTime newTimeState2 ∧ newTimeEnv1 = newTimeEnv2"
  subgoal premises ps proof -
    from ps obtain newLower where new_lower_1: "newLower = max lower (minTime state1)" by auto
    from ps new_lower_1 have new_lower_2: "newLower = max lower (minTime state2)" by auto
    from ps new_lower_1 have "newTimeState1 = state1⦇minTime := newLower⦈" apply auto
      by (smt (verit) IntervalResult.inject(1) IntervalResult.simps(4))
    moreover from ps(3) new_lower_2 have "newTimeState2 = state2⦇minTime := newLower⦈" apply auto
      by (smt (verit, best) IntervalResult.inject(1) IntervalResult.simps(4))
    moreover from ps new_lower_1 new_lower_2 have "newTimeEnv1 = newTimeEnv2" apply auto
      by (smt (verit, best) IntervalResult.inject(1))
    ultimately show ?thesis by auto
  qed
  done

type_synonym MInput = "Input option × int × int"
```

```
(* MTau transitions = internal_reaction freshChoiceName *)
datatype MFullSymbol  = MTau ("mτ")
  | MLambda MInput ("λm_")

type_synonym MConfig = "Contract * State * Environment * (TransactionWarning list) * (Payment
     list)"

type_synonym Proc = "MConfig"

type_synonym TransitionRelation = "MConfig ⇒ MFullSymbol ⇒ MConfig ⇒ bool"

inductive small_step_input_marlowe :: "MConfig ⇒ MInput ⇒ MConfig ⇒ bool" ("_ →m_ _" [55, 0,
     55]) where
MLambdaReceive: "⟦
  fixInterval (lower, upper) state = IntervalTrimmed newTimeEnv newTimeState;
  timeInterval newTimeEnv = (realLower, realUpper);
  realLower < t;
  realUpper < t;
  inp ⊢ (cases, newTimeState, newTimeEnv) →casesm (newC, newState);
  i = (Some inp, lower, upper) ⟧ ⟹
  (When cases t c, state, env, warnings, payments) →mi (newC, newState, newTimeEnv, warnings,
       payments)" |
CCSTimeout: "⟦
  fixInterval (lower, upper) state = IntervalTrimmed newTimeEnv newTimeState;
  timeInterval newTimeEnv = (realLower, realUpper);
  realLower ≥ t;
  realUpper ≥ t;
  i = (None, lower, upper) ⟧ ⟹
  (When cases t c, state, env, warnings, payments) →mi (c, newTimeState, newTimeEnv, warnings,
       payments)"

inductive_cases ReceiveE[elim]: "p1 →mμ p1'"

lemma deterministicSmallStepInput:
"f →mμ f' ⟹
  f →mμ f'' ⟹
  f' = f''"
proof (induction f μ f' arbitrary: f'' rule: small_step_input_marlowe.induct)
  case (MLambdaReceive lower upper state newTimeEnv newTimeState realLower realUpper t inp cases
       newC newState i c env warnings payments f'')
  show ?case using MLambdaReceive(7) MLambdaReceive(1-6) apply (induction "(When cases t c, state
       , env, warnings, payments)" i f'' rule: small_step_input_marlowe.induct)
     apply auto
    by (metis Pair_inject deterministicCasesSemantics)+
```

```
qed auto

(* Reactions have already been defined in small_step_reduce *)
inductive internal_reaction :: "MConfig ⇒ MConfig ⇒ bool" ("_ →m𝒯 _" [55, 55]) where
SmallStepReduce: "(c, s, env, warnings, payments) → (newC, newS, newEnv, newWarnings,
     newPayments) ⟹
  (c, s, env, (convertReduceWarnings warnings), payments) →m𝒯 (newC, newS, newEnv, (
       convertReduceWarnings newWarnings), newPayments)"

abbreviation small_step_reacts_ccs :: "MConfig ⇒ MConfig ⇒ bool" ("_ →m*𝒯 _" [55, 55]) where
"x →m*𝒯 y == star internal_reaction x y"

inductive_cases ReactE: "p1 →m𝒯 p1'"
thm ReactE

inductive full_semantics_marlowe :: "TransitionRelation" ("_ −_→m _" [55, 0, 55]) where
InternalFull: "p1 →m𝒯 p2 ⟹ full_semantics_marlowe p1 mτ p2" |
ReceiveFull: "⟦
  p1 →m(Some i, lower, upper) p2⟧ ⟹
  full_semantics_marlowe p1 (λm(Some i, lower, upper)) p2" |
TimeoutFull: "⟦p1 →m(None, lower, upper) p2⟧ ⟹ full_semantics_marlowe p1 (λm(None, lower,
     upper)) p2"

inductive full_nd_semantics_marlowe :: "TransitionRelation" ("_ —_→m _" [55, 0, 55]) where
InternalFullND: "p1 →m𝒯 p2 ⟹ full_nd_semantics_marlowe p1 mτ p2" |
ReceiveFullND: "⟦p1 →m(Some i, lower, upper) p2⟧ ⟹
  full_nd_semantics_marlowe p1 (λm(Some i, lower, upper)) p2" |
TimeoutFullND: "⟦p1 →m(None, lower, upper) p2⟧ ⟹ full_nd_semantics_marlowe p1 (λm(None, lower,
      upper)) p2"

lemma full_nd_simulates_original:
"p1 −i→m p2 ⟹
  p1 —i→m p2"
proof (induction p1 i p2 rule: full_semantics_marlowe.induct)
  case (InternalFull p1 p2)
  then show ?case by (simp add: InternalFullND)
next
  case (ReceiveFull i p1 lower upper p2)
  then show ?case by (simp add: ReceiveFullND)
next
  case (TimeoutFull p1 lower upper p2)
  then show ?case by (simp add: TimeoutFullND)
qed
```

```
lemma full_semantics_marlowe_react_is_equivalent_to_internal_reaction:
"p1 →mT p2 ⟷ full_semantics_marlowe p1 mτ p2"
  apply (auto simp add: InternalFull)
  using full_semantics_marlowe.cases by blast

lemma full_nd_semantics_marlowe_react_is_equivalent_to_internal_reaction:
"p1 →mT p2 ⟷ full_nd_semantics_marlowe p1 mτ p2"
  apply (auto simp add: InternalFullND)
  using full_nd_semantics_marlowe.cases by blast

(* ADV = (a.(!a.ADV + !c.!a.ADV))
  CON = (c.a...0)
 new a,c (ADV | CON)
*)
lemma full_semantics_marlowe_receive_empty_set_is_equivalent_to_base_small_step:
"(p1 →mμ p2) ⟷ (full_semantics_marlowe p1 (λmμ) p2)"
  apply auto
   apply (induction p1 μ p2 rule: small_step_input_marlowe.induct, auto)
    apply (simp add: ReceiveFull MLambdaReceive)
   apply (simp add: TimeoutFull small_step_input_marlowe.CCSTimeout)
  by (smt (verit, best) MFullSymbol.distinct(1) MFullSymbol.inject empty_iff
       full_semantics_marlowe.simps)

lemma full_nd_semantics_marlowe_receive_empty_set_is_equivalent_to_base_small_step:
"(p1 →mμ p2) ⟷ (full_nd_semantics_marlowe p1 (λmμ) p2)"
  apply auto
   apply (induction p1 μ p2 rule: small_step_input_marlowe.induct, auto)
    apply (simp add: ReceiveFullND MLambdaReceive)
   apply (simp add: TimeoutFullND small_step_input_marlowe.CCSTimeout)
  by (smt (verit, best) MFullSymbol.distinct(1) MFullSymbol.inject empty_iff
       full_nd_semantics_marlowe.simps)

inductive experiment_relation_marlowe :: "MConfig ⇒ MInput list ⇒ MConfig ⇒ bool" ("_ =_⇒m _"
      [55, 0, 55]) where
BaseExperiment: "⟦c1 →m*T c2 ⟧ ⟹
  c1 =[]⇒m c2" |
StepExperiment: "⟦c1 →m*T c2;
  c2 →mi c3;
  c3 →m*T c4;
  c4 =rest⇒m c5 ⟧ ⟹
   c1 =(i#rest)⇒m c5"

inductive full_nd_experiment_relation_marlowe :: "MConfig ⇒ MFullSymbol list ⇒ MConfig ⇒ bool"
      ("_ ==_⇒m _" [55, 0, 55]) where
```

```
BaseExperimentND: "⟦c1 →m*𝒯 c2 ⟧ ⟹
  c1 ==[]⇒m c2" |
StepExperimentND: "⟦c1 →m*𝒯 c2;
  c2 —μ→m c3;
  c3 →m*𝒯 c4;
  c4 ==rest⇒m c5 ⟧ ⟹
   c1 ==(μ#rest)⇒m c5"

lemma single_full_nd_receive_in_experiment:
"full_nd_semantics_marlowe p1 μ p2 ⟹
  p1 ==[μ]⇒m p2"
  using BaseExperimentND StepExperimentND by blast

inductive general_experiment_relation_marlowe :: "('process ⇒ 'process ⇒ bool) ⇒
  ('process ⇒ 'label ⇒ 'process ⇒ bool) ⇒
  'process => 'label list ⇒ 'process ⇒ bool" ("_, _ ⊢ _ =_⇒m _" [55, 55, 0, 0, 55]) where
GeneralBaseExperiment: "⟦star (general_reaction) p1 p2⟧ ⟹
  general_reaction, general_receive ⊢ p1 =[]⇒m p2" |
GeneralStepExperiment: "⟦star (general_reaction) p1 p2;
  general_receive p2 inp p3;
  star general_reaction p3 p4;
  general_reaction, general_receive ⊢ p4 =rest⇒m p5 ⟧ ⟹
  general_reaction, general_receive ⊢ p1 =(inp#rest)⇒m  p5"

lemma single_receive_in_experiment:
"receive p1 inp p2 ⟹
  reaction, receive ⊢ p1 =[inp]⇒m p2"
  by (meson GeneralBaseExperiment GeneralStepExperiment star.refl)

lemma marloweExperimentInGeneral:
"experiment_relation_marlowe c1 inp c2 = general_experiment_relation_marlowe (internal_reaction)
     (small_step_input_marlowe) c1 inp c2"
  apply auto
   apply (induction  c1 inp c2 rule: experiment_relation_marlowe.induct, auto simp add:
        GeneralBaseExperiment GeneralStepExperiment)
  by (induction internal_reaction small_step_input_marlowe c1 inp c2 rule:
       general_experiment_relation_marlowe.induct, auto simp add: BaseExperiment StepExperiment)

inductive_cases EmptyExperimentE: "c1 =[]⇒m c2"
inductive_cases GeneralEmptyExperimentE: "general_experiment_relation_marlowe reaction receive []
      c1 c2"
thm EmptyExperimentE

lemma emptyExperimentComposition:
```

```
"reaction, receive ⊢ c1 =[]⇒m c2 ⟹
  reaction, receive ⊢ c2 =[]⇒m c3 ⟹
  reaction, receive ⊢ c1 =[]⇒m c3"
  by (smt (verit, best) GeneralBaseExperiment general_experiment_relation_marlowe.cases list.
       distinct(1) star_trans)

lemma emptyNDExperimentComposition:
"c1 ==[]⇒m c2 ⟹
  c2 ==[]⇒m c3 ⟹
  c1 ==[]⇒m c3"
  by (smt (verit) full_nd_experiment_relation_marlowe.simps list.discI star_trans)

lemma reactionsPrefixNDExperiment:
"c2 ==e⇒m c3 ⟹
  star internal_reaction c1 c2 ⟹
  c1 ==e⇒m c3"
proof (induction c2 e c3 arbitrary: c1 rule: full_nd_experiment_relation_marlowe.induct)
  case (BaseExperimentND c1 c2)
  then show ?case by (meson full_nd_experiment_relation_marlowe.BaseExperimentND star_trans)
next
  case (StepExperimentND c1' c2 μ c3 c4 rest c5)
  moreover have "star internal_reaction c1 c2" using calculation star_trans by metis
  ultimately show ?case using full_nd_experiment_relation_marlowe.StepExperimentND by blast
qed

lemma reactionsPrefixExperiment:
"reaction, receive ⊢ c2 =e⇒m c3 ⟹
  star reaction c1 c2 ⟹
  reaction, receive ⊢ c1 =e⇒m c3"
proof (induction reaction receive c2 e c3 arbitrary: c1 rule: general_experiment_relation_marlowe
     .induct)
  case (GeneralBaseExperiment c2 c3)
  then show ?case using emptyExperimentComposition general_experiment_relation_marlowe.
       GeneralBaseExperiment by metis
next
  case (GeneralStepExperiment reaction c2 c3 receive inp c4 c5 rest c6 c1)
  moreover have "star reaction c1 c3" using calculation star_trans by metis
  ultimately show ?case using general_experiment_relation_marlowe.GeneralStepExperiment by metis
qed

lemma fullNDExperimentComposition:
"c1 ==e1⇒m c2 ⟹
  c2 ==e2⇒m c3 ⟹
  c1 ==(e1 @ e2)⇒m c3"
```

```
proof (induction c1 e1 c2 arbitrary: c3 rule: full_nd_experiment_relation_marlowe.induct)
  case (BaseExperimentND c1 c2)
  then show ?case proof (induction c1 c2 rule: star.induct[of internal_reaction])
    case (refl c1)
    then show ?case by auto
  next
    case (step c1 c2 c2')
    moreover have "star internal_reaction c1 c2'" using calculation by (simp add: star.step)
    ultimately show ?case using reactionsPrefixNDExperiment by blast
  qed
next
  case (StepExperimentND c1 c2 μ innerC3 c4 rest c5)
  moreover have "c4 ==(rest @ e2)⇒m c3" using calculation(5,6) by auto
  moreover have "innerC3 ==(rest @ e2)⇒m c3" using calculation reactionsPrefixNDExperiment by
       blast
  moreover have "c2 ==(μ # rest @ e2)⇒m c3" using calculation
       full_nd_experiment_relation_marlowe.StepExperimentND star.refl by (metis
       full_nd_experiment_relation_marlowe.StepExperimentND)
  ultimately show ?case using reactionsPrefixExperiment by (simp add:
       full_nd_experiment_relation_marlowe.StepExperimentND)
qed

lemma generalExperimentComposition:
"reaction, receive ⊢ c1 =e1⇒m c2 ⟹
  reaction, receive ⊢ c2 =e2⇒m c3 ⟹
  reaction, receive ⊢ c1 =(e1 @ e2)⇒m c3"
proof (induction reaction receive c1 e1 c2 rule: general_experiment_relation_marlowe.induct)
  case (GeneralBaseExperiment reaction c1 c2 receive)
  then show ?case proof (induction c1 c2 rule: star.induct[of reaction])
    case (refl c1)
    then show ?case by auto
  next
    case (step c1 c2 c2')
    moreover have "star reaction c1 c2'" using calculation by (simp add: star.step)
    ultimately show ?case by (metis star_step1 reactionsPrefixExperiment)
  qed
next
  case (GeneralStepExperiment reaction c1 c2 receive inp innerC3 c4 rest c5)
  moreover have "general_experiment_relation_marlowe reaction receive c4 (rest @ e2) c3" using
       calculation by auto
  moreover have "general_experiment_relation_marlowe reaction receive innerC3 (rest @ e2) c3"
       using calculation reactionsPrefixExperiment by auto
  moreover have "general_experiment_relation_marlowe reaction receive c2 (inp # rest @ e2) c3"
       using calculation experiment_relation_marlowe.StepExperiment star.refl by (metis
```

```
      general_experiment_relation_marlowe.GeneralStepExperiment)
  ultimately show ?case using reactionsPrefixExperiment by auto
qed


(*
  The weak_sim_marlowe and weak_sim_marlowe_simple definitions are given in
  Robin Milner's book "Communicating and Mobile Systems: The π-calculus".
  They are said to be equivalent, and we show that here.
*)

(* c1 is weakly simulated by c2 if there is a binary relation from c1 to c2 where this property
     holds. *)
definition weak_sim_marlowe :: "MConfig rel ⇒ bool" where
"weak_sim_marlowe s ⟷ (∀c1 c2 . (c1, c2) ∈ s ⟶ (∀e c1' . c1 =e⇒m c1' ⟶ (∃c2' . c2 =e⇒m c2
     ' ∧ (c1', c2') ∈ s)))"

definition weak_nd_sim_marlowe :: "Proc rel ⇒ bool" where
"weak_nd_sim_marlowe s ⟷ (∀c1 c2 . ((c1), (c2)) ∈ s ⟶ (∀e c1' . (c1 ==e⇒m c1') ⟶ (∃c2' . (
     c2 ==e⇒m c2') ∧ ((c1'), (c2')) ∈ s)))"

definition weak_nd_sim_marlowe_simple :: "Proc rel ⇒ bool" where
"weak_nd_sim_marlowe_simple s ⟷ (∀c1 c2 . ((c1), (c2)) ∈ s ⟶
  (∀c1' . (c1 —mτ→m c1') ⟶ (∃c2' . c2 →m*𝒯 c2' ∧ ((c1'), (c2')) ∈ s)) ∧
  (∀μ c1' . (c1 —μ→m c1') ⟶ (∃c2' . (c2 ==[μ]⇒m c2') ∧ ((c1'), (c2')) ∈ s)))"

definition observation_nd_bisimulation_marlowe :: "Proc rel ⇒ bool" where
"observation_nd_bisimulation_marlowe s ⟷ weak_nd_sim_marlowe_simple s ∧
     weak_nd_sim_marlowe_simple (s⁻¹)"

definition observable_equivalence_marlowe :: "Proc ⇒ Proc ⇒ bool" (infix "\<approx>" 55) where
"observable_equivalence_marlowe p1 p2 ⟷ (∃r. observation_nd_bisimulation_marlowe r ∧ (p1, p2) ∈
      r)"


(*
  Simulations with arbitrary transition rules.
  Must be provided internal reaction rules and observable transition rules.
*)
definition general_weak_sim_marlowe :: "('process ⇒ 'process ⇒ bool) ⇒
  ('process ⇒ 'label ⇒ 'process ⇒ bool) ⇒
  'process rel ⇒ bool" where
"general_weak_sim_marlowe reaction receive s ⟷ (∀c1 c2 . (c1, c2) ∈ s ⟶
  (∀e c1' . reaction, receive ⊢ c1 =e⇒m c1' ⟶
    (∃c2' . reaction, receive ⊢ c2 =e⇒m c2' ∧ (c1', c2') ∈ s)))"
```

```
definition weak_sim_marlowe_simple :: "MConfig rel ⇒ bool" where
"weak_sim_marlowe_simple s ⟷ (∀c1 c2 . (c1, c2) ∈ s ⟶
  (∀c1' . c1 →m𝒯 c1' ⟶ (∃c2' . c2 →m*𝒯 c2' ∧ (c1', c2') ∈ s)) ∧
  (∀i c1' . c1 →mi c1' ⟶ (∃c2' . c2 =[i]⇒m c2' ∧ (c1', c2') ∈ s)))"

definition general_weak_sim_marlowe_simple :: "('process ⇒ 'process ⇒ bool) ⇒
  ('process ⇒ 'label ⇒ 'process ⇒ bool) ⇒
  'process rel ⇒ bool" where
"general_weak_sim_marlowe_simple reaction receive s ⟷ (∀c1 c2 . (c1, c2) ∈ s ⟶
  (∀c1' . reaction c1 c1' ⟶ (∃c2' . star reaction c2 c2' ∧ (c1', c2') ∈ s)) ∧
  (∀i c1' . receive c1 i c1' ⟶ (∃c2' . reaction, receive ⊢ c2 =[i]⇒m c2' ∧ (c1', c2') ∈ s)))"

definition observation_bisimulation_marlowe :: "('process ⇒ 'process ⇒ bool) ⇒
  ('process ⇒ 'label ⇒ 'process ⇒ bool) ⇒
  'process rel ⇒ bool" where
"observation_bisimulation_marlowe reaction receive s ⟷ general_weak_sim_marlowe_simple reaction
      receive s ∧ general_weak_sim_marlowe_simple reaction receive (s⁻¹)"

lemma concreteWeakSimDefInGeneralDef:
"weak_sim_marlowe s = general_weak_sim_marlowe internal_reaction small_step_input_marlowe s"
"weak_sim_marlowe_simple s = general_weak_sim_marlowe_simple internal_reaction
     small_step_input_marlowe s"
   apply (simp_all add: weak_sim_marlowe_def general_weak_sim_marlowe_def
        weak_sim_marlowe_simple_def general_weak_sim_marlowe_simple_def)
  using marloweExperimentInGeneral by presburger+

lemma initialReactionInEmptyExperimentGeneral:
"reaction c1 c1' ⟹ (reaction, receive ⊢ c1 =[]⇒m c1')"
  by (simp add: GeneralBaseExperiment)

lemma simpleWeakSimulationGeneralReaction:
"star reaction c1 c2 ⟹
  general_weak_sim_marlowe_simple reaction receive s ⟹
  (c1, d1) ∈ s ⟹
  ∃d2 . star reaction d1 d2 ∧ (c2, d2) ∈ s"
  apply (induction c1 c2 arbitrary: d1 rule: star.induct[of reaction])
   apply blast
  by (smt (verit, best) star_trans general_weak_sim_marlowe_simple_def)

lemma simpleWeakNDSimulationGeneralReaction:
"star internal_reaction c1 c2 ⟹
  weak_nd_sim_marlowe_simple s ⟹
  ((c1), (d1)) ∈ s ⟹
```

```
  ∃d2 . star internal_reaction d1 d2 ∧ ((c2), (d2)) ∈ s"
proof (induction c1 c2 arbitrary: d1 rule: star.induct[of internal_reaction])
  case (refl x)
  then show ?case by blast
next
  case (step x y z)
  moreover obtain d2' where "((y), d2') ∈ s" using calculation by (metis
       full_nd_semantics_marlowe_react_is_equivalent_to_internal_reaction
       weak_nd_sim_marlowe_simple_def)
  ultimately show ?case by (smt (z3) InternalFullND star_trans weak_nd_sim_marlowe_simple_def)
qed

lemma simpleWeakSimulationImpliesWeakSimulation_aux:
"reaction, receive ⊢ c1 =e⇒m c1' ⟹
  general_weak_sim_marlowe_simple reaction receive s ⟹
  (c1, c2) ∈ s ⟹
  ∃c2'. reaction, receive ⊢ c2 =e⇒m c2' ∧ (c1', c2') ∈ s"
proof (induction reaction receive "c1" e "c1'" arbitrary: c2 rule:
     general_experiment_relation_marlowe.induct)
  case (GeneralBaseExperiment reaction c1 c1' receive)
  then show ?case proof (induction c1 c1' arbitrary: c2 rule: star.induct[of reaction])
    case (refl x)
    then show ?case by (metis star.refl general_experiment_relation_marlowe.GeneralBaseExperiment
         )
  next
    case (step x y z)
    moreover obtain c2y where "star reaction c2 c2y ∧ (y, c2y) ∈ s" using
         general_weak_sim_marlowe_simple_def calculation by metis
    moreover obtain c2' where "reaction, receive ⊢ c2y =[]⇒m c2' ∧ (z, c2') ∈ s" using
         calculation by blast
    moreover have "reaction, receive ⊢ c2 =[]⇒m c2y" using calculation
         general_experiment_relation_marlowe.GeneralBaseExperiment by metis
    moreover have "reaction, receive ⊢ c2 =[]⇒m c2'" using calculation
         emptyExperimentComposition by metis
    ultimately show ?case by blast
  qed
next
  case (GeneralStepExperiment reaction c1 c2 receive inp c3 c4 rest c5 d1)
  moreover obtain d2 where "star reaction d1 d2 ∧ (c2, d2) ∈ s" using calculation(1,6,7)
       simpleWeakSimulationGeneralReaction weak_sim_marlowe_simple_def by metis
  moreover obtain d3 where "general_experiment_relation_marlowe reaction receive d2 [inp]  d3 ∧ (
       c3, d3) ∈ s" using general_weak_sim_marlowe_simple_def calculation by metis
  moreover obtain d4 where "star reaction d3 d4 ∧ (c4, d4) ∈ s" using calculation(3,6,9)
       simpleWeakSimulationGeneralReaction weak_sim_marlowe_simple_def by metis
```

```
  moreover obtain d5 where "general_experiment_relation_marlowe reaction receive d4 rest d5 ∧ (c5
       , d5) ∈ s" using calculation by blast
  ultimately show ?case by (metis (mono_tags, opaque_lifting) append_Cons append_Nil
       generalExperimentComposition reactionsPrefixExperiment)
qed

lemma simpleWeakNDSimulationImpliesWeakNDSimulation_aux:
"(c1 ==e⇒m c1') ⟹
  weak_nd_sim_marlowe_simple s ⟹
  ((c1), (c2)) ∈ s ⟹
  ∃c2'. (c2 ==e⇒m c2') ∧ ((c1'), (c2')) ∈ s"
proof (induction "c1" e "c1'" arbitrary: c2 rule: full_nd_experiment_relation_marlowe.induct)
  case (BaseExperimentND c1 c1')
  then show ?case proof (induction c1 c1' arbitrary: c2 rule: star.induct[])
    case (refl x)
    then show ?case using full_nd_experiment_relation_marlowe.BaseExperimentND by blast
  next
    case (step x y z)
    moreover obtain c2y where "star internal_reaction c2 c2y ∧ ((y), (c2y)) ∈ s" using
         weak_nd_sim_marlowe_simple_def calculation InternalFullND by presburger
    moreover obtain c2' where "(c2y ==[]⇒m c2') ∧ ((z), (c2')) ∈ s" using calculation by blast
    moreover have "c2 ==[]⇒m c2y" using calculation full_nd_experiment_relation_marlowe.
         BaseExperimentND by blast
    moreover have "c2 ==[]⇒m c2'" using calculation emptyNDExperimentComposition by metis
    ultimately show ?case by blast
  qed
next
  case (StepExperimentND c1 c1' μ c1'' c1''' rest c1'''')
  moreover obtain c2' where "star internal_reaction c2 c2' ∧ ((c1'), (c2')) ∈ s" using
       calculation(1,6,7) weak_nd_sim_marlowe_simple_def by (meson
       simpleWeakNDSimulationGeneralReaction)
  moreover obtain c2'' where "(c2' ==[μ]⇒m c2'') ∧ ((c1''), (c2'')) ∈ s" using
       weak_nd_sim_marlowe_simple_def calculation by metis
  moreover obtain c2''' where "star internal_reaction c2'' c2''' ∧ ((c1'''), (c2''')) ∈ s" using
       calculation(3,6,9) simpleWeakNDSimulationGeneralReaction weak_nd_sim_marlowe_simple_def by
        metis
  moreover obtain c2'''' where "(c2''' ==rest⇒m c2'''') ∧ ((c1''''), c2'''') ∈ s" using
       calculation by blast
  moreover have "c2 ==μ#rest⇒m c2''''" by (metis (full_types) append_Cons append_Nil calculation
       (10,11,8,9) fullNDExperimentComposition reactionsPrefixNDExperiment)
  ultimately show ?case by blast
qed

lemma simpleWeakSimulationImpliesWeakSimulation:
```

```
"general_weak_sim_marlowe_simple reaction receive s ⟶ general_weak_sim_marlowe reaction receive
      s"
  by (smt (verit, ccfv_SIG) simpleWeakSimulationImpliesWeakSimulation_aux
       general_weak_sim_marlowe_def general_weak_sim_marlowe_simple_def)

lemma simpleWeakNDSimulationImpliesWeakNDSimulation:
"weak_nd_sim_marlowe_simple s ⟶ weak_nd_sim_marlowe s"
  using simpleWeakNDSimulationImpliesWeakNDSimulation_aux weak_nd_sim_marlowe_def by presburger

lemma weakSimulationImpliesSimpleWeakSimulation:
"general_weak_sim_marlowe reaction receive s ⟶ general_weak_sim_marlowe_simple reaction receive
      s"
  apply (auto simp add: general_weak_sim_marlowe_def general_weak_sim_marlowe_simple_def)
   apply (smt (verit, best) general_experiment_relation_marlowe.cases
        initialReactionInEmptyExperimentGeneral neq_Nil_conv)
  by (meson GeneralBaseExperiment GeneralStepExperiment star.refl)

lemma weakNDSimulationImpliesSimpleWeakNDSimulation:
"weak_nd_sim_marlowe s ⟶ weak_nd_sim_marlowe_simple s"
  apply (auto simp add: weak_nd_sim_marlowe_def weak_nd_sim_marlowe_simple_def)
   apply (metis BaseExperimentND full_nd_experiment_relation_marlowe.cases
        full_nd_semantics_marlowe_react_is_equivalent_to_internal_reaction list.distinct(1)
        star_step1)
  using single_full_nd_receive_in_experiment by presburger

lemma equivalenceIsWeakSimulation:
"general_weak_sim_marlowe_simple reaction receive Id"
  by (simp add: general_weak_sim_marlowe_def weakSimulationImpliesSimpleWeakSimulation)

inductive_cases FullSemanticsMLambdaE[elim]: "(c1, state1, env2, warnings2, payments2) −λm(a, aa,
      b)→m (c1', state1', env1', warnings1', payments1')"
inductive_cases FullSemanticsMTauE[elim]: "(c1, state1, env2, warnings2, payments2) −mτ→m (c1',
     state1', env1', warnings1', payments1')"
inductive_cases ReceiveWhenSomeElim: "(When cases t cont, state1, env1, warnings1, payments1) →m
     (Some inp, lower, upper) (c1', state1', env1', warnings1', payments1')"

lemma small_step_receive_time_change_before_or_during:
"(When cases t cont, newTimeState, newTimeEnv, warnings, payments) →m(i, lower, upper) (c1',
     state1', env1', warnings1', payments1') ⟹
  fixInterval (lower, upper) state = IntervalTrimmed newTimeEnv newTimeState ⟹
  (When cases t cont, state, env, warnings, payments) →m(i, lower, upper) (c1', state1', env1',
       warnings1', payments1')"
proof (induction "When cases t cont" newTimeState newTimeEnv warnings payments i lower upper c1'
     state1' env1' warnings1' payments1' arbitrary: state rule: small_step_input_marlowe.induct[
```

```
    split_format(complete)])
  case (MLambdaReceive lower upper state1 newTimeEnv newTimeState realLower realUpper inp newC
      newState i1 lower1 upper1 env warnings payments arbState)
  moreover have "lower1 = lower ∧ upper1 = upper" using calculation by auto
  moreover have "state1 = newTimeState" using calculation apply auto
    apply (cases "lower < (State.minTime state1)", auto)
     apply (cases "upper < lower", auto)
     apply (cases "upper < (State.minTime state1)", auto)
    apply (cases "upper < lower", auto)
    apply (cases "realUpper < (State.minTime arbState)", auto)
    by (cases "realLower < (State.minTime arbState)", auto)
  moreover have "env = newTimeEnv" using calculation apply auto
    by (cases "upper < lower";
        cases "upper < minTime newTimeState";
        cases "lower < (State.minTime newTimeState)";
        cases "lower < (State.minTime arbState)";
        cases "realUpper < (State.minTime arbState)";
        auto)
  ultimately show ?case apply (auto simp del: fixInterval.simps)
    using small_step_input_marlowe.MLambdaReceive by presburger
next
  case (CCSTimeout lower upper state1 newTimeEnv newTimeState realLower realUpper i1 lower1
      upper1 env warnings payments arbState)
  moreover have "lower1 = lower ∧ upper1 = upper" using calculation by auto
  moreover have "state1 = newTimeState" using calculation apply auto
    apply (cases "lower < (State.minTime state1)", auto)
     apply (cases "upper < lower", auto)
     apply (cases "upper < (State.minTime state1)", auto)
    apply (cases "upper < lower", auto)
    apply (cases "realUpper < (State.minTime arbState)", auto)
    by (cases "realLower < (State.minTime arbState)", auto)
  moreover have "env = newTimeEnv" using calculation apply auto
    by (cases "upper < lower";
        cases "upper < minTime newTimeState";
        cases "lower < (State.minTime newTimeState)";
        cases "lower < (State.minTime arbState)";
        cases "realUpper < (State.minTime arbState)";
        auto)
  ultimately show ?case using small_step_input_marlowe.CCSTimeout by auto
qed


lemma full_nd_semantics_marlowe_time_change_before_or_during:
"(When cases t cont, newTimeState, newTimeEnv, warnings, payments) ——λm(i, lower, upper)→m (c1',
```

```
      state1', env1', warnings1', payments1') ⟹
  fixInterval (lower, upper) state = IntervalTrimmed newTimeEnv newTimeState ⟹
  (When cases t cont, state, env, warnings, payments) —λm(i, lower, upper)→m (c1', state1',
       env1', warnings1', payments1')"
proof (induction "(When cases t cont, newTimeState, newTimeEnv, warnings, payments)" "λm(i, lower
     , upper)" "(c1', state1', env1', warnings1', payments1')" rule: full_nd_semantics_marlowe.
     induct)
  case (ReceiveFullND i)
  then show ?case apply auto
    using ReceiveFullND.prems full_nd_semantics_marlowe.ReceiveFullND
         small_step_receive_time_change_before_or_during by presburger
next
  case TimeoutFullND
  then show ?case by (meson full_nd_semantics_marlowe.TimeoutFullND
       small_step_receive_time_change_before_or_during)
qed

inductive_cases FullNDInternalE: "c1 —mτ→m c2"

lemma applyCasesImpliesComputeCasesStepMarlowe:
"applyCases env state input cases = Applied warning newState newCont ⟹
  computeCasesStepMarlowe input (cases, state, env) = Some (newCont, newState)"
  apply (induction env state input cases rule: applyCases.induct)
  by auto

lemma computeCasesStepMarloweImpliesApplyCases:
"computeCasesStepMarlowe input (cases, state, env) = Some (newCont, newState) ⟹
  ∃warning . applyCases env state input cases = Applied warning newState newCont"
  apply (induction env state input cases rule: applyCases.induct)
  by auto

lemma applyCasesImpliesMarloweSmallStepInput:
  assumes "fixInterval (lower, upper) state1 = IntervalTrimmed newTimeEnv1 newTimeState1 ∧
    timeInterval newTimeEnv1 = (realLower, realUpper) ∧
    realLower < t ∧
    realUpper < t ∧
    applyCases newTimeEnv1 newTimeState1 input cases = Applied warning newState newCont"
  shows "(When cases t c, state1, env, warnings, payments) →m(Some input, lower, upper) (newCont
       , newState, newTimeEnv1, warnings, payments)"
proof -
  from assms have "computeCasesStepMarlowe input (cases, newTimeState1, newTimeEnv1) = Some (
       newCont, newState)" using applyCasesImpliesComputeCasesStepMarlowe by presburger
  moreover have "input ⊢ (cases, newTimeState1, newTimeEnv1) →cases_m (newCont, newState)" using
       calculation computeCasesStepMarloweResImpliesReduceStep by blast
```

```
  ultimately show ?thesis using small_step_input_marlowe.MLambdaReceive[of lower upper state1
        newTimeEnv1 newTimeState1 realLower realUpper t input cases newCont newState _ _ env
        warnings payments] using assms by blast
qed

lemma marloweSmallStepInputImpliesApplyCases:
  assumes "((When cases t c, state1, env, warnings, payments) →m(Some input, lower, upper) (
        newCont, newState, newTimeEnv1, warnings, payments)) ∧
    fixInterval (lower, upper) state1 = IntervalTrimmed newTimeEnv1 newTimeState1"
  shows "∃warning . applyCases newTimeEnv1 newTimeState1 input cases = Applied warning newState
        newCont"
proof -
  from assms have "input ⊢ (cases, newTimeState1, newTimeEnv1) →cases_m (newCont, newState)" by
        auto
  moreover have "computeCasesStepMarlowe input (cases, newTimeState1, newTimeEnv1) = Some (
        newCont, newState)" using calculation reduceCasesStepImpliesComputeCaseStepRes by blast
  ultimately show ?thesis using computeCasesStepMarloweImpliesApplyCases by blast
qed

lemma marloweSmallStepInputTimeoutImpliesReduceStep:
  assumes "((When cases t c, state1, env, warnings, payments) →m(None, lower, upper) (newCont,
        newState, newTimeEnv1, warnings, payments)) ∧
    fixInterval (lower, upper) state1 = IntervalTrimmed newTimeEnv1 newTimeState1"
  shows "reduceContractStep newTimeEnv1 newTimeState1 (When cases t c) = Reduced ReduceNoWarning
        ReduceNoPayment newState newCont"
proof -
  show ?thesis using assms by auto
qed

lemma reductionLoopImpliesSmallStepStarMarlowe:
"reductionLoop reduced env state contract warnings payments =
  ContractQuiescent reducedRes newWarnings newPayments newState newContract ⟹
  (∀smallStepWarnings. ∃ newSmallStepWarnings .
    (contract, state, env, smallStepWarnings, rev payments) →⋆ (newContract, newState, env,
        newSmallStepWarnings, newPayments))"
proof (induction reduced env state contract warnings payments rule: reductionLoop.induct)
  case (1 reduced env state contract warnings payments)
  then show ?case apply (simp only:)
    subgoal premises ps proof (cases "reduceContractStep env state contract")
      case (Reduced newW newP newS newC)
      moreover from Reduced reduceStepIsSmallStepReductionNoPayment[of env state contract newW
          newS newC _ _] have "newP = ReduceNoPayment ⟶ (∀smallStepWarnings . ∃
          newSmallStepWarnings . (contract, state, env, smallStepWarnings, rev payments) → (
          newC, newS, env, newSmallStepWarnings, rev payments))" by auto
```

```
    moreover from Reduced reduceStepIsSmallStepReductionWithPayment[of env state contract newW
        _ newS newC _ "rev payments"] have "(∀ p . newP = ReduceWithPayment p ⟶ (∀
        smallStepWarnings . ∃ newSmallStepWarnings . (contract, state, env, smallStepWarnings,
         rev payments) → (newC, newS, env, newSmallStepWarnings, (rev payments) @ [p])))" by
        auto
    ultimately show ?thesis using ps apply (auto split: if_split_asm simp only:)
      subgoal premises ps2 proof (cases newP)
        case ReduceNoPayment
        then show ?thesis using ps ps2 by (auto simp only:)
      next
        case (ReduceWithPayment innerP)
        moreover have rd: "reductionLoop True env newS newC (if newW = ReduceNoWarning then
              warnings else newW # warnings) (case newP of ReduceNoPayment ⇒ payments |
              ReduceWithPayment payment ⇒ payment # payments) = ContractQuiescent reducedRes
              newWarnings newPayments newState newContract" by (metis (no_types, lifting) "1.
              prems" ReduceEffect.simps(5) ReduceStepResult.simps(8) Reduced calculation
              reductionLoop.simps)
        moreover obtain x where "x = (if newW = ReduceNoWarning then warnings else newW #
              warnings)" by auto
        moreover obtain xa where "xa = (case newP of ReduceNoPayment ⇒ payments |
              ReduceWithPayment payment ⇒ payment # payments)" by auto
        moreover have innerStar: "∀smallStepWarnings. ∃newSmallStepWarnings . (newC, newS, env
              , smallStepWarnings, rev (case newP of ReduceNoPayment ⇒ payments |
              ReduceWithPayment payment ⇒ payment # payments)) →* (newContract, newState, env
              , newSmallStepWarnings, newPayments)" using ReduceWithPayment rd ps2(3)[of newW
              newP newS newC x xa] using calculation(3) calculation(4) by presburger
        moreover have revP: "rev (case newP of ReduceNoPayment ⇒ payments | ReduceWithPayment
               payment ⇒ payment # payments) = (rev (payments) @ [innerP])" using
              ReduceWithPayment by auto
        moreover have "∀smallStepWarnings. ∃newSmallStepWarnings . (newC, newS, env,
              smallStepWarnings, (rev payments) @ [innerP]) →* (newContract, newState, env,
              newSmallStepWarnings, newPayments)" using revP innerStar by presburger
        ultimately show ?thesis by (smt (verit) Reduced \<open>\<And>ws pb. reduceContractStep
               env state contract = Reduced newW (ReduceWithPayment pb) newS newC ⟹ (contract
              , state, env, ws, rev payments) → (newC, newS, env, ws @ [newW], rev payments @
              [pb])\<close> star.step)
      qed
      subgoal premises ps2 proof (cases newP)
        case ReduceNoPayment
        moreover have rd: "reductionLoop True env newS newC (if newW = ReduceNoWarning then
              warnings else newW # warnings) (case newP of ReduceNoPayment ⇒ payments |
              ReduceWithPayment payment ⇒ payment # payments) = ContractQuiescent reducedRes
              newWarnings newPayments newState newContract" by (metis (no_types, lifting) "1.
              prems" ReduceEffect.simps(5) ReduceStepResult.simps(8) Reduced calculation
```

```
        reductionLoop.simps)
    moreover obtain x where "x = (if newW = ReduceNoWarning then warnings else newW #
        warnings)" by auto
    moreover obtain xa where "xa = (case newP of ReduceNoPayment ⇒ payments |
        ReduceWithPayment payment ⇒ payment # payments)" by auto
    moreover have innerStar: "∀smallStepWarnings. ∃newSmallStepWarnings . (newC, newS, env
        , smallStepWarnings, rev (case newP of ReduceNoPayment ⇒ payments |
        ReduceWithPayment payment ⇒ payment # payments)) →⋆ (newContract, newState, env
        , newSmallStepWarnings, newPayments)" using ReduceNoPayment rd ps2(3)[of newW
        newP newS newC x xa] using calculation(3) calculation(4) by presburger
    moreover have revP: "rev (case newP of ReduceNoPayment ⇒ payments | ReduceWithPayment
         payment ⇒ payment # payments) = (rev (payments))" using ReduceNoPayment by auto
    moreover have "∀smallStepWarnings. ∃newSmallStepWarnings . (newC, newS, env,
        smallStepWarnings, (rev payments)) →⋆ (newContract, newState, env,
        newSmallStepWarnings, newPayments)" using revP innerStar by presburger
    ultimately show ?thesis by (meson ps2(5) star.step)
  next
    case (ReduceWithPayment innerP)
    moreover have rd: "reductionLoop True env newS newC (if newW = ReduceNoWarning then
        warnings else newW # warnings) (case newP of ReduceNoPayment ⇒ payments |
        ReduceWithPayment payment ⇒ payment # payments) = ContractQuiescent reducedRes
        newWarnings newPayments newState newContract" by (metis (no_types, lifting) "1.
        prems" ReduceEffect.simps(5) ReduceStepResult.simps(8) Reduced calculation
        reductionLoop.simps)
    moreover obtain x where "x = (if newW = ReduceNoWarning then warnings else newW #
        warnings)" by auto
    moreover obtain xa where "xa = (case newP of ReduceNoPayment ⇒ payments |
        ReduceWithPayment payment ⇒ payment # payments)" by auto
    moreover have innerStar: "∀smallStepWarnings. ∃newSmallStepWarnings . (newC, newS, env
        , smallStepWarnings, rev (case newP of ReduceNoPayment ⇒ payments |
        ReduceWithPayment payment ⇒ payment # payments)) →⋆ (newContract, newState, env
        , newSmallStepWarnings, newPayments)" using ReduceWithPayment rd ps2(3)[of newW
        newP newS newC x xa] using calculation(3) calculation(4) by presburger
    moreover have revP: "rev (case newP of ReduceNoPayment ⇒ payments | ReduceWithPayment
         payment ⇒ payment # payments) = (rev (payments) @ [innerP])" using
        ReduceWithPayment by auto
    moreover have "∀smallStepWarnings. ∃newSmallStepWarnings . (newC, newS, env,
        smallStepWarnings, (rev payments) @ [innerP]) →⋆ (newContract, newState, env,
        newSmallStepWarnings, newPayments)" using revP innerStar by presburger
    ultimately show ?thesis by (smt (verit) Reduced \<open>\<And>ws pb. reduceContractStep
         env state contract = Reduced newW (ReduceWithPayment pb) newS newC ⟹ (contract
        , state, env, ws, rev payments) → (newC, newS, env, ws @ [newW], rev payments @
        [pb])\<close> star.step)
  qed
```

```
        done
    next
      case NotReduced
      then show ?thesis using ps by auto
    next
      case AmbiguousTimeIntervalReductionError
      then show ?thesis using ps by auto
    qed
    done
qed

lemma deterministicMarloweReduction:
  "cs → cs' ⟹ cs → cs'' ⟹ cs'' = cs'"
  by (induction arbitrary: cs'' rule: small_step_reduce.induct, auto)

end
```

Appendix B

Faustus Type-Checking Reference

This appendix is meant to provide the numerous type-checking rules for Faustus without disrupting the flow of the chapter. Note that the type context Γ is represented as a list, and we use : , the list cons operator, to add type information to the type context.

$$\frac{}{\Gamma \vdash \mathit{ConstantPubKey}\ b :: \mathit{Party}} \qquad \frac{}{\Gamma \vdash \mathit{Role}\ t :: \mathit{Party}}$$

$$\frac{}{\Gamma \vdash \mathit{UseParty}\ i :: \mathit{Party}}\ (i, \mathit{Party}) \in \Gamma$$

$$\frac{\Gamma \vdash p :: \mathit{Party}}{\Gamma \vdash \mathit{Account}\ p :: \mathit{Payee}} \qquad \frac{\Gamma \vdash p :: \mathit{Party}}{\Gamma \vdash \mathit{Party}\ p :: \mathit{Payee}}$$

$$\frac{\Gamma \vdash p :: \mathit{Party}}{\Gamma \vdash \mathit{FChoiceId}\ n\ p :: \mathit{ChoiceId}}$$

Figure B.1: Faustus `Party`, `Payee`, and `ChoiceId` type rules. Implemented in Isabelle with their respective `wellTyped` functions.

$$\frac{\Gamma \vdash p :: \mathit{Party}}{\Gamma \vdash \mathit{AvailableMoney}\ p\ t :: \mathit{Value}} \qquad \frac{}{\Gamma \vdash \mathit{Constant}\ i :: \mathit{Value}}$$

$$\frac{}{\Gamma \vdash \mathit{TimeIntervalStart} :: \mathit{Value}} \qquad \frac{}{\Gamma \vdash \mathit{TimeIntervalEnd} :: \mathit{Value}}$$

$$\frac{}{\Gamma \vdash \mathit{UseValue}\ i :: \mathit{Value}}\ (i, \mathit{Value}) \in \Gamma$$

$$\frac{\Gamma \vdash c :: \mathit{ChoiceId}}{\Gamma \vdash \mathit{ChoiceValue}\ c :: \mathit{Value}} \qquad \frac{\Gamma \vdash v :: \mathit{Value}}{\Gamma \vdash \mathit{NegValue}\ v :: \mathit{Value}}$$

$$\frac{\Gamma \vdash v1 :: \mathit{Value} \quad \Gamma \vdash v2 :: \mathit{Value}}{\Gamma \vdash \mathit{AddValue}\ v1\ v2 :: \mathit{Value}} \qquad \frac{\Gamma \vdash v1 :: \mathit{Value} \quad \Gamma \vdash v2 :: \mathit{Value}}{\Gamma \vdash \mathit{SubValue}\ v1\ v2 :: \mathit{Value}}$$

$$\frac{\Gamma \vdash v1 :: \mathit{Value} \quad \Gamma \vdash v2 :: \mathit{Value}}{\Gamma \vdash \mathit{MulValue}\ v1\ v2 :: \mathit{Value}} \qquad \frac{\Gamma \vdash v1 :: \mathit{Value} \quad \Gamma \vdash v2 :: \mathit{Value}}{\Gamma \vdash \mathit{DivValue}\ v1\ v2 :: \mathit{Value}}$$

$$\frac{\Gamma \vdash o :: \mathit{Observation} \quad \Gamma \vdash v1 :: \mathit{Value} \quad \Gamma \vdash v2 :: \mathit{Value}}{\Gamma \vdash \mathit{Cond}\ o\ v1\ v2 :: \mathit{Value}}$$

Figure B.2: Faustus `Value` type rules. Implemented in Isabelle with the `wellTypedValue` function.

$$\frac{}{\Gamma \vdash \mathit{TrueObs} :: \mathit{Observation}} \qquad \frac{}{\Gamma \vdash \mathit{FalseObs} :: \mathit{Observation}}$$

$$\frac{}{\Gamma \vdash \mathit{UseObservation}\ i :: \mathit{Observation}}\ (i, \mathit{Observation}) \in \Gamma$$

$$\frac{\Gamma \vdash c :: \mathit{ChoiceId}}{\Gamma \vdash \mathit{ChoseSomething}\ c :: \mathit{Observation}} \qquad \frac{\Gamma \vdash o :: \mathit{Observation}}{\Gamma \vdash \mathit{NotObs}\ o :: \mathit{Observation}}$$

$$\frac{\Gamma \vdash o1 :: \mathit{Observation} \quad \Gamma \vdash o2 :: \mathit{Observation}}{\Gamma \vdash \mathit{AndObs}\ o1\ o2 :: \mathit{Observation}}$$

$$\frac{\Gamma \vdash o1 :: \mathit{Observation} \quad \Gamma \vdash o2 :: \mathit{Observation}}{\Gamma \vdash \mathit{OrObs}\ o1\ o2 :: \mathit{Observation}}$$

$$\frac{\Gamma \vdash v1 :: \mathit{Value} \quad \Gamma \vdash v2 :: \mathit{Value}}{\Gamma \vdash \mathit{ValueGE}\ v1\ v2 :: \mathit{Observation}} \qquad \frac{\Gamma \vdash v1 :: \mathit{Value} \quad \Gamma \vdash v2 :: \mathit{Value}}{\Gamma \vdash \mathit{ValueGT}\ v1\ v2 :: \mathit{Observation}}$$

$$\frac{\Gamma \vdash v1 :: \mathit{Value} \quad \Gamma \vdash v2 :: \mathit{Value}}{\Gamma \vdash \mathit{ValueLT}\ v1\ v2 :: \mathit{Observation}} \qquad \frac{\Gamma \vdash v1 :: \mathit{Value} \quad \Gamma \vdash v2 :: \mathit{Value}}{\Gamma \vdash \mathit{ValueLE}\ v1\ v2 :: \mathit{Observation}}$$

$$\frac{\Gamma \vdash v1 :: \mathit{Value} \quad \Gamma \vdash v2 :: \mathit{Value}}{\Gamma \vdash \mathit{ValueEQ}\ v1\ v2 :: \mathit{Observation}}$$

Figure B.3: Faustus `Observation` type rules. Implemented in Isabelle with the `wellTypedObservation` function.

$$\frac{\Gamma \vdash pk :: Party \quad \Gamma \vdash py :: Payee \quad \Gamma \vdash v :: Value}{\Gamma \vdash Pay\ pk\ py\ t\ v :: \mathcal{S}}$$

$$\frac{\Gamma \vdash v :: Value}{\Gamma \vdash ReassignValue\ i\ v :: \mathcal{S}}\ (i, Value) \in \Gamma$$

$$\frac{\Gamma \vdash o :: Observation}{\Gamma \vdash ReassignObservation\ i\ o :: \mathcal{S}}\ (i, Observation) \in \Gamma$$

$$\frac{\Gamma \vdash p :: Party}{\Gamma \vdash ReassignParty\ i\ p :: \mathcal{S}}\ (i, Party) \in \Gamma$$

$$\frac{}{\Gamma \vdash [] :: \mathcal{S}\ list} \qquad \frac{\Gamma \vdash s :: \mathcal{S} \quad \Gamma \vdash ss :: \mathcal{S}\ list}{\Gamma \vdash s : ss :: \mathcal{S}\ list}$$

Figure B.4: Faustus Statement ($\mathcal{S}$) type rules. Implemented in Isabelle with the `well-TypedStatement` function.

$$\frac{\Gamma \vdash p1 :: Party \quad \Gamma \vdash p2 :: Party \quad \Gamma \vdash v :: Value}{\Gamma \vdash Deposit\ p1\ p2\ t\ v :: Action}$$

$$\frac{\Gamma \vdash c :: ChoiceId}{\Gamma \vdash Choice\ c\ b :: Action} \qquad \frac{\Gamma \vdash o :: Observation}{\Gamma \vdash Notify\ o :: Action}$$

$$\frac{\Gamma \vdash a :: Action}{\Gamma \vdash ActionGuard\ a :: GuardExpression}$$

$$\frac{\Gamma \vdash g1 :: GuardExpression \quad \Gamma \vdash g2 :: GuardExpression}{\Gamma \vdash GuardThenGuard\ g1\ g2 :: GuardExpression}$$

$$\frac{\Gamma \vdash g1 :: GuardExpression \quad \Gamma \vdash g2 :: GuardExpression}{\Gamma \vdash DisjointGuard\ g1\ g2 :: GuardExpression}$$

$$\frac{\Gamma \vdash g1 :: GuardExpression \quad \Gamma \vdash g2 :: GuardExpression}{\Gamma \vdash InterleavedGuard\ g1\ g2 :: GuardExpression}$$

$$\frac{\Gamma \vdash g :: GuardExpression \quad \Gamma \vdash s :: \mathcal{S}\ list}{\Gamma \vdash GuardStmtsGuard\ g\ s :: GuardExpression}$$

Figure B.5: Faustus `Action` and `GuardExpression` type rules. Implemented in Isabelle with their respective `wellTyped` functions.

$$\frac{}{\Gamma \vdash \mathit{Close} :: \mathcal{C}} \qquad \frac{\Gamma \vdash \mathit{obs} :: \mathit{Observation} \quad \Gamma \vdash \mathit{c1} :: \mathcal{C} \quad \Gamma \vdash \mathit{c2} :: \mathcal{C}}{\Gamma \vdash \mathit{If}\ \mathit{obs}\ \mathit{c1}\ \mathit{c2} :: \mathcal{C}}$$

$$\frac{\Gamma \vdash s :: \mathcal{S} \quad \Gamma \vdash c :: \mathcal{C}}{\Gamma \vdash \mathit{StatementCont}\ s\ c :: \mathcal{C}}$$

$$\frac{\Gamma \vdash \mathit{cs} :: \mathit{Case\ list} \quad \Gamma \vdash c :: \mathcal{C}}{\Gamma \vdash \mathit{When}\ \mathit{cs}\ t\ c :: \mathcal{C}}$$

$$\frac{\Gamma \vdash v :: \mathit{Value} \quad (i, Value) : \Gamma \vdash c :: \mathcal{C}}{\Gamma \vdash \mathit{Let}\ i\ v\ c :: \mathcal{C}}\ i \notin \Gamma$$

$$\frac{\Gamma \vdash o :: \mathit{Observation} \quad (i, \mathit{Observation}) : \Gamma \vdash c :: \mathcal{C}}{\Gamma \vdash \mathit{LetObservation}\ i\ o\ c :: \mathcal{C}}\ i \notin \Gamma$$

$$\frac{\Gamma \vdash p :: \mathit{Party} \quad (i, \mathit{Party}) : \Gamma \vdash c :: \mathcal{C}}{\Gamma \vdash \mathit{LetParty}\ i\ p\ c :: \mathcal{C}}\ i \notin \Gamma$$

$$\frac{(\mathit{paramsToTypeContext}\ p) : \Gamma \vdash \mathit{c1} :: \mathcal{C} \quad (i, C, p) : \Gamma \vdash \mathit{c2} :: \mathcal{C}}{\Gamma \vdash \mathit{LetC}\ i\ p\ \mathit{c1}\ \mathit{c2} :: \mathcal{C}}\ i \notin \Gamma \text{ and } \forall \mathit{pid} \in p.\mathit{pid} \notin \Gamma$$

$$\frac{}{(i, \mathcal{C}, []) : \Gamma \vdash \mathit{UseC}\ i\ [] :: \mathcal{C}} \qquad \frac{\Gamma \vdash \mathit{UseC}\ i\ a :: C}{(k, \mathcal{C}, p) : \Gamma \vdash \mathit{UseC}\ i\ a :: \mathcal{C}}\ k \neq i$$

$$\frac{\Gamma \vdash v :: \mathit{Value} \quad (i, \mathcal{C}, p) : \Gamma \vdash \mathit{UseC}\ i\ a :: \mathcal{C}}{(i, \mathcal{C}, \mathit{ValueParameter}\ k : p) : \Gamma \vdash \mathit{UseC}\ i\ (\mathit{ValueArgument}\ v{:}a) :: \mathcal{C}}$$

$$\frac{\Gamma \vdash o :: \mathit{Observation} \quad (i, \mathcal{C}, p) : \Gamma \vdash \mathit{UseC}\ i\ a :: \mathcal{C}}{(i, \mathcal{C}, \mathit{ObservationParameter}\ k : p) : \Gamma \vdash \mathit{UseC}\ i\ (\mathit{ObservationArgument}\ o{:}a) :: \mathcal{C}}$$

$$\frac{\Gamma \vdash \mathit{pk} :: \mathit{Party} \quad (i, \mathcal{C}, p) : \Gamma \vdash \mathit{UseC}\ i\ a :: \mathcal{C}}{(i, \mathcal{C}, \mathit{PartyParameter}\ k : p) : \Gamma \vdash \mathit{UseC}\ i\ (\mathit{PartyArgument}\ \mathit{pk}{:}a) :: \mathcal{C}}$$

$$\frac{}{\Gamma \vdash [] :: \mathit{Case\ list}} \qquad \frac{\Gamma \vdash g :: \mathit{GuardExpression} \quad \Gamma \vdash c :: \mathcal{C} \quad \Gamma \vdash \mathit{cs} :: \mathit{Case\ list}}{\Gamma \vdash \mathit{Case}\ g\ c : \mathit{cs} :: \mathit{Case\ list}}$$

Figure B.6: Faustus `Contract` ($\mathcal{C}$) and `Case` type rules. Implemented in Isabelle with the `wellTypedContract` and `wellTypeCase` function.

$$\frac{}{[] \vdash [] :: \mathit{Context}} \qquad \frac{\Gamma \vdash C :: \mathit{Context}}{(i, \mathit{PartyType}) : \Gamma \vdash \mathit{PartyAbstraction}\ i : C :: \mathit{Context}}$$

$$\frac{\Gamma \vdash C :: \mathit{Context}}{(i, \mathit{ValueType}) : \Gamma \vdash \mathit{ValueAbstraction}\ i : C :: \mathit{Context}}$$

$$\frac{\Gamma \vdash C :: \mathit{Context}}{(i, \mathit{ObservationType}) : \Gamma \vdash \mathit{ObservationAbstraction}\ i : C :: \mathit{Context}}$$

$$\frac{\Gamma \vdash C :: \mathit{Context} \quad (\mathit{paramsToTypeContext}\ p) : \Gamma \vdash c :: \mathit{Contract}}{(i, \mathit{ContractType}\ p) : \Gamma \vdash \mathit{ContractAbstraction}\ (i, p, c) : C :: \mathit{Context}}$$

Figure B.7: Faustus context type rules. Implemented in Isabelle as the `wellTyped-Context` function.

$$\frac{}{[] \vdash S :: State} \qquad \frac{\Gamma \vdash S :: State}{(i, ContractType\ p) : \Gamma \vdash S :: State}$$

$$\frac{\Gamma \vdash S :: State \quad member\ i\ (boundValues\ S)}{(i, ValueType) : \Gamma \vdash S :: State}$$

$$\frac{\Gamma \vdash S :: State \quad member\ i\ (boundValues\ S)}{(i, ObservationType) : \Gamma \vdash S :: State}$$

$$\frac{\Gamma \vdash S :: State \quad member\ i\ (boundParties\ S)}{(i, PartyType) : \Gamma \vdash S :: State}$$

Figure B.8: Faustus state type rules. Implemented in Isabelle as the `wellTypedState` function.

Appendix C

Faustus Isabelle Formalization

The Faustus semantics and type-checker without guarded commands are defined in Isabelle with the following code.

Listing C.1: Faustus semantics without guard expressions.

```
theory FaustusSemantics
imports Main Util.MList Util.SList Core.ListTools Core.Semantics "HOL-Library.Product_Lexorder" "
     HOL.Product_Type" "HOL-IMP.Star" FaustusTypes SmallStep
begin

(* Type rules require the identifier is most recently associated with the correct type in the
     stack.*)
fun wellTypedParty :: "TypeContext ⇒ FParty ⇒ bool" where
"wellTypedParty tyCtx (ConstantPubKey pk) = True" |
"wellTypedParty tyCtx (Role role) = True" |
"wellTypedParty [] (UseParty pkId) = False" |
"wellTypedParty ((boundPkId, ty)#rest) (UseParty pkId) = ((boundPkId = pkId ∧ ty = PubKeyType) ∨
     (boundPkId ≠ pkId ∧ wellTypedParty rest (UseParty pkId)))"

fun wellTypedPayee :: "TypeContext ⇒ FPayee ⇒ bool" where
"wellTypedPayee tyCtx (Account pk) = wellTypedParty tyCtx pk" |
"wellTypedPayee tyCtx (Party pk) = wellTypedParty tyCtx pk"

fun wellTypedChoiceId :: "TypeContext ⇒ FChoiceId ⇒ bool" where
"wellTypedChoiceId tyCtx (FChoiceId cname pk) = wellTypedParty tyCtx pk"

fun wellTypedValue :: "TypeContext ⇒ FValue ⇒ bool" and
  wellTypedObservation :: "TypeContext ⇒ FObservation ⇒ bool" where
"wellTypedValue tyCtx (AvailableMoney pk token) = wellTypedParty tyCtx pk" |
```

```
"wellTypedValue tyCtx (Constant integer) = True" |
"wellTypedValue tyCtx (NegValue val) = wellTypedValue tyCtx val" |
"wellTypedValue tyCtx (AddValue lhs rhs) = ((wellTypedValue tyCtx lhs) ∧ (wellTypedValue tyCtx
     rhs))" |
"wellTypedValue tyCtx (SubValue lhs rhs) = ((wellTypedValue tyCtx lhs) ∧ (wellTypedValue tyCtx
     rhs))" |
"wellTypedValue tyCtx (MulValue lhs rhs) = ((wellTypedValue tyCtx lhs) ∧ (wellTypedValue tyCtx
     rhs))" |
"wellTypedValue tyCtx (DivValue lhs rhs) = ((wellTypedValue tyCtx lhs) ∧ (wellTypedValue tyCtx
     rhs))" |
"wellTypedValue tyCtx (ChoiceValue choId) = (wellTypedChoiceId tyCtx choId)" |
"wellTypedValue tyCtx (TimeIntervalStart) = True" |
"wellTypedValue tyCtx (TimeIntervalEnd) = True" |
"wellTypedValue [] (UseValue valId) = False" |
"wellTypedValue ((boundValId, ty)#rest) (UseValue valId) = ((boundValId = valId ∧ ty = ValueType)
      ∨ (boundValId ≠ valId ∧ wellTypedValue rest (UseValue valId)))" |
"wellTypedValue tyCtx (Cond cond thn els) = (wellTypedObservation tyCtx cond ∧ wellTypedValue
     tyCtx thn ∧ wellTypedValue tyCtx els)" |
"wellTypedObservation tyCtx (AndObs lhs rhs) = ((wellTypedObservation tyCtx lhs) ∧ (
     wellTypedObservation tyCtx rhs))" |
"wellTypedObservation tyCtx (OrObs lhs rhs) = ((wellTypedObservation tyCtx lhs) ∧ (
     wellTypedObservation tyCtx rhs))" |
"wellTypedObservation tyCtx (NotObs subObs) = wellTypedObservation tyCtx subObs" |
"wellTypedObservation tyCtx (ChoseSomething choId) = (wellTypedChoiceId tyCtx choId)" |
"wellTypedObservation tyCtx (ValueGE lhs rhs) = ((wellTypedValue tyCtx lhs) ∧ (wellTypedValue
     tyCtx rhs))" |
"wellTypedObservation tyCtx (ValueGT lhs rhs) = ((wellTypedValue tyCtx lhs) ∧ (wellTypedValue
     tyCtx rhs))" |
"wellTypedObservation tyCtx (ValueLT lhs rhs) = ((wellTypedValue tyCtx lhs) ∧ (wellTypedValue
     tyCtx rhs))" |
"wellTypedObservation tyCtx (ValueLE lhs rhs) = ((wellTypedValue tyCtx lhs) ∧ (wellTypedValue
     tyCtx rhs))" |
"wellTypedObservation tyCtx (ValueEQ lhs rhs) = ((wellTypedValue tyCtx lhs) ∧ (wellTypedValue
     tyCtx rhs))" |
"wellTypedObservation [] (UseObservation obsId) = False" |
"wellTypedObservation ((boundObsId, ty)#rest) (UseObservation obsId) = ((boundObsId = obsId ∧ ty
     = ObservationType) ∨ (boundObsId ≠ obsId ∧ wellTypedObservation rest (UseObservation obsId))
     )" |
"wellTypedObservation tyCtx TrueObs = True" |
"wellTypedObservation tyCtx FalseObs = True"

fun wellTypedArgument :: "TypeContext ⇒ FParameter ⇒ FArgument ⇒ bool" where
"wellTypedArgument tyCtx (ValueParameter vid) (ValueArgument val) = wellTypedValue tyCtx val" |
```

```
"wellTypedArgument tyCtx (ObservationParameter obsId) (ObservationArgument obs) =
     wellTypedObservation tyCtx obs" |
"wellTypedArgument tyCtx (PubKeyParameter pkId) (PubKeyArgument pk) = wellTypedParty tyCtx pk" |
"wellTypedArgument tyCtx p a = False"

fun wellTypedArguments :: "TypeContext ⇒ FParameter list ⇒ FArgument list ⇒ bool" where
"wellTypedArguments tyCtx [] [] = True" |
"wellTypedArguments tyCtx p [] = False" |
"wellTypedArguments tyCtx [] a = False" |
"wellTypedArguments tyCtx (firstParam#restParams) (firstArg#restArgs) = (wellTypedArgument tyCtx
     firstParam firstArg ∧ wellTypedArguments tyCtx restParams restArgs)"

fun lookupContractIdParamTypeInformation :: "TypeContext ⇒ Identifier ⇒ (FParameter list ×
     TypeContext) option" where
"lookupContractIdParamTypeInformation [] cid = None" |
"lookupContractIdParamTypeInformation ((i, ty)#restTy) cid = (if i = cid
  then case ty of
    ContractType params ⇒ Some (params, restTy)
    | _ ⇒ None
  else lookupContractIdParamTypeInformation restTy cid)"

fun wellTypedContract :: "TypeContext ⇒ FContract ⇒ bool" and
  wellTypedCases :: "TypeContext ⇒ FCase list ⇒ bool" where
"wellTypedContract tyCtx Close = True" |
"wellTypedContract tyCtx (Pay accId payee token val cont) =
  (wellTypedParty tyCtx accId ∧ wellTypedPayee tyCtx payee ∧ wellTypedValue tyCtx val ∧
       wellTypedContract tyCtx cont)" |
"wellTypedContract tyCtx (If obs cont1 cont2) =
  (wellTypedObservation tyCtx obs ∧ wellTypedContract tyCtx cont1 ∧ wellTypedContract tyCtx cont2
       )" |
"wellTypedContract tyCtx (When cases t cont) = (wellTypedCases tyCtx cases ∧ wellTypedContract
     tyCtx cont)" |
"wellTypedContract tyCtx (Assert obs cont) = (wellTypedObservation tyCtx obs ∧ wellTypedContract
     tyCtx cont)" |
"wellTypedContract tyCtx (Let valId val cont) = (wellTypedValue tyCtx val ∧ wellTypedContract ((
     valId, ValueType)#tyCtx) cont)" |
"wellTypedContract tyCtx (LetObservation obsId obs cont) = (wellTypedObservation tyCtx obs ∧
     wellTypedContract ((obsId, ObservationType)#tyCtx) cont)" |
"wellTypedContract tyCtx (LetPubKey pkId pk cont) = (wellTypedParty tyCtx pk ∧ wellTypedContract
     ((pkId, PubKeyType)#tyCtx) cont)" |
"wellTypedContract tyCtx (LetC cid params body cont) = (wellTypedContract ((paramsToTypeContext
     params)@tyCtx) body ∧ wellTypedContract ((cid, ContractType params)#tyCtx) cont)" |
"wellTypedContract tyCtx (UseC cid args) = (case lookupContractIdParamTypeInformation tyCtx cid
     of
```

```
  Some (params, innerTyCtx) ⇒ wellTypedArguments tyCtx params args
  | _ ⇒ False)" |
"wellTypedCases tyCtx [] = True" |
"wellTypedCases tyCtx ((Case (Deposit fromPk toPk token value) c)#rest) =
  ((wellTypedParty tyCtx fromPk) ∧ (wellTypedParty tyCtx toPk) ∧ (wellTypedValue tyCtx value) ∧ (
       wellTypedContract tyCtx c) ∧ (wellTypedCases tyCtx rest))" |
"wellTypedCases tyCtx ((Case (Choice choiceId bounds) c)#rest) = ((wellTypedChoiceId tyCtx
     choiceId) ∧ (wellTypedContract tyCtx c) ∧ (wellTypedCases tyCtx rest))" |
"wellTypedCases tyCtx ((Case (Notify observation) c)#rest) = ((wellTypedObservation tyCtx
     observation) ∧ (wellTypedContract tyCtx c) ∧ (wellTypedCases tyCtx rest))"


(* Execution context and type context must match through the course of a program.
  If the execution context is larger, then there is not enough type information to type the
       program.
  If the type context is larger, then there is not enough meaning information to run a well-typed
        program.*)
fun wellTypedContext :: "TypeContext ⇒ FContext ⇒ bool" where
"wellTypedContext [] [] = True" |
"wellTypedContext tyCtx [] = False" |
"wellTypedContext [] ctx = False" |
"wellTypedContext ((tyValId, ValueType)#restTypes) ((ValueAbstraction absValId)#restAbstractions)
      =
  (tyValId = absValId ∧ wellTypedContext restTypes restAbstractions)" |
"wellTypedContext ((tyPkId, PubKeyType)#restTypes) ((PubKeyAbstraction absPkId)#restAbstractions)
      =
  (tyPkId = absPkId ∧ wellTypedContext restTypes restAbstractions)" |
"wellTypedContext ((tyObsId, ObservationType)#restTypes) ((ObservationAbstraction absObsId)#
     restAbstractions) =
  (tyObsId = absObsId ∧ wellTypedContext restTypes restAbstractions)" |
"wellTypedContext ((tyCid, ContractType tyParams)#restTypes) ((ContractAbstraction (absCid,
     absParams, absBody))#restAbstractions) =
  (tyCid = absCid ∧ tyParams = absParams ∧ wellTypedContract ((paramsToTypeContext absParams)
       @restTypes) absBody ∧ wellTypedContext restTypes restAbstractions)" |
"wellTypedContext (someType#restTypes) (someAbs#restAbs) = False"

lemma consValueWellTypedContext:
"wellTypedContext tyCtx ctx ⟷
  wellTypedContext ((valId, ValueType)#tyCtx) (ValueAbstraction valId#ctx)"
  by simp

lemma consObservationWellTypedContext:
"wellTypedContext tyCtx ctx ⟷
  wellTypedContext ((obsId, ObservationType)#tyCtx) (ObservationAbstraction obsId#ctx)"
```

```
  by simp

lemma consPubKeyWellTypedContext:
"wellTypedContext tyCtx ctx ⟷
  wellTypedContext ((pkId, PubKeyType)#tyCtx) (PubKeyAbstraction pkId#ctx)"
  by simp

lemma consContractWellTypedContext:
"wellTypedContext tyCtx ctx ∧ wellTypedContract ((paramsToTypeContext params)@tyCtx) bodyCont ⟷
  wellTypedContext ((cid, ContractType params)#tyCtx) (ContractAbstraction (cid, params, bodyCont
      )#ctx)"
  by auto

lemma paramsWellTypedContext:
"wellTypedContext (paramsToTypeContext params) (paramsToFContext params)"
proof (induction params)
  case Nil
  then show ?case by auto
next
  case (Cons firstParam params)
  then show ?case by (cases firstParam, auto)
qed

lemma wellTypedPartyAppendTypes:
"wellTypedParty tyCtx pk ⟹
  wellTypedParty (tyCtx @ tyCtx2) pk"
proof (induction tyCtx)
  case Nil
  then show ?case by (cases pk, auto)
next
  case (Cons firstType tyCtx)
  then show ?case by (cases pk firstType rule: FParty.exhaust[case_product prod.exhaust], auto)
qed

lemma wellTypedPayeeAppendTypes:
"wellTypedPayee tyCtx payee ⟹
  wellTypedPayee (tyCtx @ tyCtx2) payee"
proof (induction tyCtx)
  case Nil
  then show ?case
    by (metis wellTypedPayee.elims(2) wellTypedPayee.simps(1) wellTypedPayee.simps(2)
        wellTypedPartyAppendTypes)
next
  case (Cons firstType tyCtx)
```

```
  then show ?case by (metis FPayee.exhaust wellTypedPayee.simps(1) wellTypedPayee.simps(2)
        wellTypedPartyAppendTypes)
qed

lemma wellTypedChoiceIdAppendTypes:
"wellTypedChoiceId tyCtx choiceId ⟹
  wellTypedChoiceId (tyCtx @ tyCtx2) choiceId"
  apply (induction tyCtx, auto)
   apply (cases choiceId, auto)
  using wellTypedPartyAppendTypes apply fastforce
  by (metis append_Cons wellTypedChoiceId.elims(3) wellTypedChoiceId.simps
        wellTypedPartyAppendTypes)

lemma wellTypedValueAppendTypes:
"(∀ tyCtx2 . wellTypedValue tyCtx val ⟶
  wellTypedValue (tyCtx @ tyCtx2) val)"
"(∀ tyCtx2 . wellTypedObservation tyCtx obs ⟶
  wellTypedObservation (tyCtx @ tyCtx2) obs)"
  by (induction rule: wellTypedValue_wellTypedObservation.induct, auto simp add:
        wellTypedChoiceIdAppendTypes wellTypedPartyAppendTypes)

lemma wellTypedArgumentAppendTypes:
"(∀ tyCtx2 . wellTypedArgument tyCtx param arg ⟶
  wellTypedArgument (tyCtx @ tyCtx2) param arg)"
  by (induction rule: wellTypedArgument.induct, auto simp add: wellTypedValueAppendTypes
        wellTypedPartyAppendTypes)

lemma wellTypedArgumentsAppendTypes:
"(∀ tyCtx2 . wellTypedArguments tyCtx params args ⟶
  wellTypedArguments (tyCtx @ tyCtx2) params args)"
  by (induction rule: wellTypedArguments.induct, auto simp add: wellTypedArgumentAppendTypes)

lemma lookupCidAppendTypes:
"(∀ tyCtx2 restTy. lookupContractIdParamTypeInformation tyCtx cid = Some (params, restTy) ⟶
  lookupContractIdParamTypeInformation (tyCtx@tyCtx2) cid = Some (params, restTy@tyCtx2))"
proof (induction rule: lookupContractIdParamTypeInformation.induct)
  case (1 cid)
  then show ?case by auto
next
  case (2 i ty restTy cid)
  then show ?case by (cases ty, auto)
qed

lemma wellTypedContractAppendTypes:
```

```
"(∀ tyCtx2 . wellTypedContract tyCtx cont ⟶
  wellTypedContract (tyCtx @ tyCtx2) cont)"
"(∀ tyCtx2 . wellTypedCases tyCtx cases ⟶
  wellTypedCases (tyCtx @ tyCtx2) cases)"
proof (induction rule: wellTypedContract_wellTypedCases.induct)
  case (1 tyCtx)
  then show ?case by auto
next
  case (2 tyCtx accId payee token val cont)
  then show ?case by (simp add: wellTypedPayeeAppendTypes wellTypedPartyAppendTypes
        wellTypedValueAppendTypes(1))
next
  case (3 tyCtx obs cont1 cont2)
  then show ?case by (simp add: wellTypedValueAppendTypes(2))
next
  case (4 tyCtx cases t cont)
  then show ?case by simp
next
  case (5 tyCtx obs cont)
  then show ?case by (simp add: wellTypedValueAppendTypes(2))
next
  case (6 tyCtx valId val cont)
  then show ?case by (simp add: wellTypedValueAppendTypes(1))
next
  case (7 tyCtx obsId obs cont)
  then show ?case by (simp add: wellTypedValueAppendTypes(2))
next
  case (8 tyCtx pkId pk cont)
  then show ?case by (simp add: wellTypedPartyAppendTypes)
next
  case (9 tyCtx cid params body cont)
  then show ?case by auto
next
  case (10 tyCtx cid args)
  then show ?case by (cases "lookupContractIdParamTypeInformation tyCtx cid", auto simp add:
        lookupCidAppendTypes wellTypedArgumentsAppendTypes)
next
  case (11 tyCtx)
  then show ?case by simp
next
  case (12 tyCtx fromPk toPk token val c rest)
  then show ?case by (simp add: wellTypedPartyAppendTypes wellTypedValueAppendTypes(1))
next
  case (13 tyCtx choiceId bounds c rest)
```

```
  then show ?case by (simp add: wellTypedChoiceIdAppendTypes)
next
  case (14 tyCtx observation c rest)
  then show ?case by (simp add: wellTypedValueAppendTypes(2))
qed

lemma concatWellTypedContexts:
"wellTypedContext tyCtx1 ctx1 ⟹
  wellTypedContext tyCtx2 ctx2 ⟹
  wellTypedContext (tyCtx2 @ tyCtx1) (ctx2 @ ctx1)"
  apply (induction tyCtx2 ctx2 rule: wellTypedContext.induct, auto)
  using wellTypedContractAppendTypes(1) by fastforce

(* If an identifier has a type, it should be in the state. *)
fun wellTypedState :: "TypeContext ⇒ FState ⇒ bool" where
"wellTypedState [] s = True" |
"wellTypedState ((valId, ValueType)#restTypes) s = (member valId (boundValues s) ∧ wellTypedState
      restTypes s)" |
"wellTypedState ((obsId, ObservationType)#restTypes) s = (member obsId (boundValues s) ∧
     wellTypedState restTypes s)" |
"wellTypedState ((pkId, PubKeyType)#restTypes) s = (member pkId (boundPubKeys s) ∧ wellTypedState
      restTypes s)" |
"wellTypedState ((cid, ContractType params)#restTypes) s = wellTypedState restTypes s"

lemma insertValueWellTypedStateIsWellTyped:
"wellTypedState tyCtx s ⟹
  wellTypedState tyCtx (s⦇boundValues := MList.insert vid val (boundValues s)⦈)"
proof (induction tyCtx)
  case Nil
  then show ?case by auto
next
  case (Cons firstType tyCtx)
  then show ?case proof (cases firstType)
    case (Pair identifier ty)
    then show ?thesis using Cons apply (cases ty, auto)
       apply (metis MList.insert_lookup_Some insert_lookup_different)
      by (metis MList.insert_lookup_Some insert_lookup_different)
  qed
qed

lemma insertAppendValueWellTypedStateIsWellTyped:
"wellTypedState tyCtx s ⟹
  wellTypedState ((vid, ValueType)#tyCtx) (s⦇boundValues := MList.insert vid val (boundValues s)⦈
       )"
```

```
proof (induction tyCtx)
  case Nil
  then show ?case by (simp add: MList.insert_lookup_Some)
next
  case (Cons firstTy tyCtx)
  then show ?case proof (cases firstTy)
    case (Pair i ty)
    then show ?thesis using Cons.IH Cons.prems insertValueWellTypedStateIsWellTyped
         wellTypedState.elims(2) by fastforce
  qed
qed

lemma insertAppendObservationWellTypedStateIsWellTyped:
"wellTypedState tyCtx s ⟹
  wellTypedState ((vid, ObservationType)#tyCtx) (s⦇boundValues := MList.insert vid val (
       boundValues s)⦈)"
proof (induction tyCtx)
  case Nil
  then show ?case by (simp add: MList.insert_lookup_Some)
next
  case (Cons firstTy tyCtx)
  then show ?case proof (cases firstTy)
    case (Pair i ty)
    then show ?thesis using Cons.IH Cons.prems insertValueWellTypedStateIsWellTyped
         wellTypedState.elims(2) by fastforce
  qed
qed

lemma insertPubKeyWellTypedStateIsWellTyped:
"wellTypedState tyCtx s ⟹
  wellTypedState tyCtx (s⦇boundPubKeys := MList.insert pkId pk (boundPubKeys s)⦈)"
proof (induction tyCtx)
  case Nil
  then show ?case by auto
next
case (Cons firstType tyCtx)
  then show ?case proof (cases firstType)
    case (Pair identifier ty)
    then show ?thesis using Cons apply (cases ty, auto)
      by (metis MList.insert_lookup_Some insert_lookup_different)
  qed
qed

lemma insertAppendPubKeyWellTypedStateIsWellTyped:
```

```
"wellTypedState tyCtx s ⟹
  wellTypedState ((vid, PubKeyType)#tyCtx) (s⦇boundPubKeys := MList.insert vid val (boundPubKeys
      s)⦈)"
proof (induction tyCtx)
  case Nil
  then show ?case by (simp add: MList.insert_lookup_Some)
next
  case (Cons firstTy tyCtx)
  then show ?case proof (cases firstTy)
    case (Pair i ty)
    then show ?thesis using Cons.IH Cons.prems insertPubKeyWellTypedStateIsWellTyped
         wellTypedState.elims by fastforce
  qed
qed

lemma wellTypedStateAccountsArbitrary:
"wellTypedState tyCtx s ⟹
  wellTypedState tyCtx (s⦇accounts := newAccounts⦈)"
proof (induction tyCtx)
  case Nil
  then show ?case by auto
next
  case (Cons firstType tyCtx)
  then show ?case proof (cases firstType)
    case (Pair identifier ty)
    then show ?thesis using Cons by (cases ty, auto)
  qed
qed

lemma wellTypedStateCombineTypeContexts:
"wellTypedState tyCtx s ⟹
  wellTypedState tyCtx2 s ⟹
  wellTypedState (tyCtx @ tyCtx2) s"
proof (induction tyCtx)
  case Nil
  then show ?case by auto
next
  case (Cons firstType tyCtx)
  then show ?case proof (cases firstType)
    case (Pair identifier ty)
    then show ?thesis using Cons by (cases ty, auto)
  qed
qed
```

```
fun evalFParty :: "FState ⇒ FParty ⇒ Party option" where
"evalFParty state (ConstantPubKey pk) = Some (SemanticsTypes.PubKey pk)"|
"evalFParty state (Role role) = Some (SemanticsTypes.Role role)" |
"evalFParty state (UseParty pkId) = lookup pkId (boundPubKeys state)"

lemma evalFParty_SamePubKeyDomainImpliesExecution:
"∀evaluatedPubKey. ∃otherEvaluatedPubKey.
  (∀pubKeyId . member pubKeyId (boundPubKeys s) = member pubKeyId (boundPubKeys otherState)) ⟶
  evalFParty s pk = Some evaluatedPubKey ⟶
  evalFParty otherState pk = Some otherEvaluatedPubKey"
  apply auto
  subgoal premises ps proof (cases pk)
    case (ConstantPubKey x1)
    then show ?thesis by auto
  next
    case (UseParty x2)
    then show ?thesis using ps by auto
  next
    case (Role x3)
    then show ?thesis using ps by auto
  qed
  done

lemma wellTypedPartyEvaluates:
"wellTypedParty tyCtx pk ⟹
  wellTypedState tyCtx s ⟹
  evalFParty s pk ≠ None"
proof (induction rule: wellTypedParty.induct)
  case (1 tyCtx pk)
  then show ?case by auto
next
  case (2 pkId)
  then show ?case by auto
next
  case (3 pkId)
  then show ?case by auto
next
  case (4 boundPkId ty rest pkId)
  then show ?case by (cases ty, auto)
qed

fun evalFPayee :: "FState ⇒ FPayee ⇒ Payee option" where
"evalFPayee state (Account acc) = (case (evalFParty state acc) of
  None ⇒ None
```

```
  | Some pk ⇒ Some (SemanticsTypes.Account pk))"|
"evalFPayee state (Party p) = (case (evalFParty state p) of
  None ⇒ None
  | Some pk ⇒ Some (SemanticsTypes.Party pk))"

lemma wellTypedPayeeEvaluates:
"wellTypedPayee tyCtx payee ⟹
  wellTypedState tyCtx s ⟹
  evalFPayee s payee ≠ None"
  by (metis (no_types, lifting) evalFPayee.simps option.case_eq_if option.simps(3) wellTypedPayee
       .elims(2) wellTypedPartyEvaluates)

fun evalFChoiceId :: "FState ⇒ FChoiceId ⇒ ChoiceId option" where
"evalFChoiceId state (FChoiceId cname pk) = (case evalFParty state pk of
  None ⇒ None
  | Some pkResult ⇒ Some (ChoiceId cname pkResult))"

lemma wellTypedChoiceIdEvaluates:
"wellTypedChoiceId tyCtx choiceId ⟹
  wellTypedState tyCtx s ⟹
  evalFChoiceId s choiceId ≠ None"
  by (metis evalFChoiceId.simps option.case_eq_if option.distinct(1) wellTypedChoiceId.elims(2)
       wellTypedPartyEvaluates)

fun evalFValue :: "Environment ⇒ FState ⇒ FValue ⇒ int option" and
  evalFObservation :: "Environment ⇒ FState ⇒ FObservation ⇒ bool option" where
"evalFValue env state (AvailableMoney fPubKey token) =
  (case (evalFParty state fPubKey) of
  None ⇒ None
  | Some accId ⇒ Some (findWithDefault 0 (accId, token) (accounts state)))" |
"evalFValue env state (Constant integer) = Some integer" |
"evalFValue env state (NegValue val) = (case (evalFValue env state val) of
  None ⇒ None
  | Some res ⇒ Some (uminus res))" |
"evalFValue env state (AddValue lhs rhs) = (case (evalFValue env state lhs, evalFValue env state
     rhs) of
  (None, _) ⇒ None
  | (_, None) ⇒ None
  | (Some res1, Some res2) ⇒ Some (res1 + res2))" |
"evalFValue env state (SubValue lhs rhs) = (case (evalFValue env state lhs, evalFValue env state
     rhs) of
  (None, _) ⇒ None
  | (_, None) ⇒ None
  | (Some res1, Some res2) ⇒ Some (res1 - res2))" |
```

```
"evalFValue env state (MulValue lhs rhs) = (case (evalFValue env state lhs, evalFValue env state
     rhs) of
  (None, _) ⇒ None
  | (_, None) ⇒ None
  | (Some res1, Some res2) ⇒ Some (res1 * res2))" |
"evalFValue env state (DivValue lhs rhs) = (case (evalFValue env state lhs, evalFValue env state
     rhs) of
  (None, _) ⇒ None
  | (_, None) ⇒ None
  | (Some n, Some d) ⇒ if d = 0 then Some 0
       else Some (n quot d))" |
"evalFValue env state (ChoiceValue choId) = (case evalFChoiceId state choId of
  None ⇒ None
  | Some evalChoId ⇒ Some (findWithDefault 0 evalChoId (choices state)))" |
"evalFValue env state (TimeIntervalStart) = Some (fst (timeInterval env))" |
"evalFValue env state (TimeIntervalEnd) = Some (snd (timeInterval env))" |
"evalFValue env state (UseValue valId) =
    MList.lookup valId (boundValues state)" |
"evalFValue env state (Cond cond thn els) =
  (case (evalFObservation env state cond, evalFValue env state thn, evalFValue env state els) of
  (None, _, _) ⇒ None
  | (_, None, _) ⇒ None
  | (_, _, None) ⇒ None
  | (Some condRes, Some thenRes, Some elseRes) ⇒
    Some (if condRes then thenRes else elseRes))" |
"evalFObservation env state (AndObs lhs rhs) = (case (evalFObservation env state lhs,
     evalFObservation env state rhs) of
  (None, _) ⇒ None
  | (_, None) ⇒  None
  | (Some res1, Some res2) ⇒ Some (res1 ∧ res2))" |
"evalFObservation env state (OrObs lhs rhs) = (case (evalFObservation env state lhs,
     evalFObservation env state rhs) of
  (None, _) ⇒ None
  | (_, None) ⇒  None
  | (Some res1, Some res2) ⇒ Some (res1 ∨ res2))" |
"evalFObservation env state (NotObs subObs) = (case evalFObservation env state subObs of
  None ⇒ None
  | Some res ⇒ Some (¬res))" |
"evalFObservation env state (ChoseSomething choId) = (case evalFChoiceId state choId of
  None ⇒ None
  | Some evalChoId ⇒ Some (member evalChoId (choices state)))" |
"evalFObservation env state (ValueGE lhs rhs) = (case (evalFValue env state lhs, evalFValue env
     state rhs) of
  (None, _) ⇒ None
```

```
  | (_, None) ⇒ None
  | (Some res1, Some res2) ⇒ Some (res1 ≥ res2))" |
"evalFObservation env state (ValueGT lhs rhs) = (case (evalFValue env state lhs, evalFValue env
     state rhs) of
  (None, _) ⇒ None
  | (_, None) ⇒ None
  | (Some res1, Some res2) ⇒ Some (res1 > res2))" |
"evalFObservation env state (ValueLT lhs rhs) = (case (evalFValue env state lhs, evalFValue env
     state rhs) of
  (None, _) ⇒ None
  | (_, None) ⇒ None
  | (Some res1, Some res2) ⇒ Some (res1 < res2))" |
"evalFObservation env state (ValueLE lhs rhs) = (case (evalFValue env state lhs, evalFValue env
     state rhs) of
  (None, _) ⇒ None
  | (_, None) ⇒ None
  | (Some res1, Some res2) ⇒ Some (res1 ≤ res2))" |
"evalFObservation env state (ValueEQ lhs rhs) = (case (evalFValue env state lhs, evalFValue env
     state rhs) of
  (None, _) ⇒ None
  | (_, None) ⇒ None
  | (Some res1, Some res2) ⇒ Some (res1 = res2))" |
"evalFObservation env state (UseObservation obsId) = (case MList.lookup obsId (boundValues state)
      of
  None ⇒ None
  | Some v ⇒ Some (v ≠ 0))" |
"evalFObservation env state TrueObs = Some True" |
"evalFObservation env state FalseObs = Some False"

lemma evalFValue_InsertDifferentValueStillExecutes:
"∀evaluatedValue. ∃otherEvaluatedValue.
  (∀valId . member valId (boundValues s) = member valId (boundValues otherState)) ⟶
  (∀pubKeyId . member pubKeyId (boundPubKeys s) = member pubKeyId (boundPubKeys otherState)) ⟶
  evalFValue e s val = Some evaluatedValue ⟶
  evalFValue e otherState val = Some otherEvaluatedValue"
"∀evaluatedObservation. ∃otherEvaluatedObservation.
  (∀valId . member valId (boundValues s) = member valId (boundValues otherState)) ⟶
  (∀pubKeyId . member pubKeyId (boundPubKeys s) = member pubKeyId (boundPubKeys otherState)) ⟶
  evalFObservation e s obs = Some evaluatedObservation ⟶
  evalFObservation e otherState obs = Some otherEvaluatedObservation"
proof (induction rule: evalFValue_evalFObservation.induct)
  case (1 env state fPubKey token)
  then show ?case apply auto
    apply (cases fPubKey, auto)
```

```
    by (split option.split, auto)
next
  case (2 env state integer)
  then show ?case by auto
next
  case (3 env state val)
  then show ?case by auto
next
  case (4 env state lhs rhs)
  then show ?case apply auto
    apply (split option.split, auto)
    by (split option.split_asm, auto)
next
  case (5 env state lhs rhs)
  then show ?case apply auto
    apply (split option.split, auto)
    by (split option.split_asm, auto)
next
  case (6 env state lhs rhs)
  then show ?case apply auto
    apply (split option.split, auto)
    by (split option.split_asm, auto)
next
  case (7 env state lhs rhs)
  then show ?case apply auto
    by (split option.split_asm, auto)
next
  case (8 env state choId)
  then show ?case apply (auto split: option.split)
    by (smt FParty.exhaust evalFParty.simps evalFChoiceId.elims evalFChoiceId.simps option.
          case_eq_if option.distinct option.exhaust_sel option.simps wellTypedChoiceId.simps
          wellTypedChoiceIdEvaluates wellTypedParty.simps wellTypedState.simps)
next
  case (9 env state)
  then show ?case by auto
next
  case (10 env state)
  then show ?case by auto
next
  case (11 env state valId)
  then show ?case by auto
next
  case (12 env state cond thn els)
  then show ?case apply auto
```

```
        apply (metis (no_types, lifting) option.case_eq_if)
       apply (metis (no_types, lifting) option.case_eq_if)
      apply (metis (no_types, lifting) option.case_eq_if)
     apply (metis (no_types, lifting) option.case_eq_if)
    by (metis (no_types, lifting) option.case_eq_if)
next
  case (13 env state lhs rhs)
  then show ?case apply auto
    by (metis (no_types, lifting) option.case_eq_if)
next
  case (14 env state lhs rhs)
  then show ?case apply auto
    by (metis (no_types, lifting) option.case_eq_if)
next
  case (15 env state subObs)
  then show ?case by auto
next
  case (16 env state choId)
  then show ?case apply (auto split: option.split)
    by (smt FParty.exhaust evalFParty.simps evalFChoiceId.elims evalFChoiceId.simps option.
          case_eq_if option.distinct(1) option.exhaust_sel option.simps(4) wellTypedChoiceId.simps
           wellTypedChoiceIdEvaluates wellTypedParty.simps wellTypedState.simps(1))
next
  case (17 env state lhs rhs)
  then show ?case apply (simp)
    apply (split option.split, simp)
    apply (split option.split, simp)
    apply (split option.split, simp)
    apply (split option.split, simp)
    by auto
next
  case (18 env state lhs rhs)
  then show ?case apply auto
    apply (split option.split, auto)
    by (split option.split_asm, auto)
next
  case (19 env state lhs rhs)
  then show ?case apply auto
    apply (split option.split, auto)
    by (split option.split_asm, auto)
next
  case (20 env state lhs rhs)
  then show ?case apply auto
    apply (split option.split, auto)
```

```
    by (split option.split_asm, auto)
next
  case (21 env state lhs rhs)
  then show ?case apply auto
    apply (split option.split, auto)
    by (split option.split_asm, auto)
next
  case (22 env state obsId)
  then show ?case apply auto
    by (split option.split, auto)
next
  case (23 env state)
  then show ?case by auto
next
  case (24 env state)
  then show ?case by auto
qed

lemma wellTypedValueObservationEvaluates:
"wellTypedValue tyCtx val ⟹
  wellTypedState tyCtx s ⟹
  evalFValue env s val ≠ None"
"wellTypedObservation tyCtx obs ⟹
  wellTypedState tyCtx s ⟹
  evalFObservation env s obs ≠ None"
proof (induction rule: wellTypedValue_wellTypedObservation.induct)
  case (1 tyCtx pk token)
  then show ?case by (simp add: option.case_eq_if wellTypedPartyEvaluates)
next
  case (2 tyCtx integer)
  then show ?case by simp
next
  case (3 tyCtx val)
  then show ?case by auto
next
  case (4 tyCtx lhs rhs)
  then show ?case by auto
next
  case (5 tyCtx lhs rhs)
  then show ?case by auto
next
  case (6 tyCtx lhs rhs)
  then show ?case by auto
next
```

```
  case (7 tyCtx lhs rhs)
  then show ?case by auto
next
  case (8 tyCtx choId)
  then show ?case by (auto simp add: wellTypedChoiceIdEvaluates split: option.split)
next
  case (9 tyCtx)
  then show ?case by auto
next
  case (10 tyCtx)
  then show ?case by auto
next
  case (11 valId)
  then show ?case by auto
next
  case (12 boundValId ty rest valId)
  then show ?case by (cases ty, auto)
next
  case (13 tyCtx cond thn els)
  then show ?case by auto
next
  case (14 tyCtx lhs rhs)
  then show ?case by auto
next
  case (15 tyCtx lhs rhs)
  then show ?case by auto
next
  case (16 tyCtx subObs)
  then show ?case by auto
next
  case (17 tyCtx choId)
  then show ?case by (auto simp add: wellTypedChoiceIdEvaluates split: option.split)
next
  case (18 tyCtx lhs rhs)
  then show ?case by auto
next
  case (19 tyCtx lhs rhs)
  then show ?case by auto
next
  case (20 tyCtx lhs rhs)
  then show ?case by auto
next
  case (21 tyCtx lhs rhs)
  then show ?case by auto
```

```
next
  case (22 tyCtx lhs rhs)
  then show ?case by auto
next
  case (23 obsId)
  then show ?case by auto
next
  case (24 boundObsId ty rest obsId)
  then show ?case by (cases ty, auto)
next
  case (25 tyCtx)
  then show ?case by auto
next
  case (26 tyCtx)
  then show ?case by auto
qed

(* Needs to pass modified state to maintain parity with compiled Marlowe contract. Evaluate
     sequentially left to right with previous results available to next arguments. *)
fun evalFArguments :: "Environment ⇒ FState ⇒ FParameter list ⇒ FArgument list ⇒ FState
     option" where
"evalFArguments e s [] [] = Some s" |
"evalFArguments e s ((ValueParameter vid)#restParams) ((ValueArgument val)#restArgs) = (case
     evalFValue e s val of
  None ⇒ None
  | Some res ⇒ evalFArguments e (s⦇boundValues := MList.insert vid res (boundValues s)⦈)
       restParams restArgs)" |
"evalFArguments e s ((ObservationParameter obsId)#restParams) ((ObservationArgument obs)#restArgs
     ) = (case evalFObservation e s obs of
  None ⇒ None
  | Some res ⇒ evalFArguments e (s⦇boundValues := MList.insert obsId (if res then 1 else 0) (
       boundValues s)⦈) restParams restArgs)" |
"evalFArguments e s ((PubKeyParameter pkId)#restParams) ((PubKeyArgument pk)#restArgs) = (case
     evalFParty s pk of
  None ⇒ None
  | Some res ⇒ evalFArguments e (s⦇boundPubKeys := MList.insert pkId res (boundPubKeys s)⦈)
       restParams restArgs)" |
"evalFArguments e s (firstParam#restParams) (firstArg#restArgs) = None" |
"evalFArguments e s params [] = None" |
"evalFArguments e s [] args = None"

lemma evalFArguments_OnlyBoundPubKeysChangeBoundPubKeys:
"∀evaluatedState otherState otherEvaluatedState .
  evalFArguments e s params args = Some evaluatedState ⟶
```

```
  evalFArguments e (otherState(boundPubKeys := boundPubKeys s)) params args = Some
        otherEvaluatedState ⟶
  boundPubKeys evaluatedState = boundPubKeys otherEvaluatedState"
proof (induction rule: evalFArguments.induct)
  case (1 e s)
  then show ?case by auto
next
  case (2 e s vid restParams val restArgs)
  then show ?case apply auto
    apply (cases "evalFValue e s val", auto)
    apply (split option.split_asm, auto)
    by (metis FState.surjective FState.update_convs(3) FState.update_convs(4))
next
  case (3 e s obsId restParams obs restArgs)
  then show ?case apply auto
    apply (split option.split_asm, simp)
    apply (split option.split_asm, simp)
    apply (cases "the (evalFObservation e s obs)", auto)
     apply (metis FState.surjective FState.update_convs(3) FState.update_convs(4))
    by (metis FState.surjective FState.update_convs(3) FState.update_convs(4))
next
  case (4 e s pkId restParams pk restArgs)
  then show ?case apply (cases pk)
     apply simp
    apply auto
    apply (split option.split_asm, simp)
    apply (split option.split_asm)
     apply blast
    by (metis (no_types) FState.surjective FState.update_convs(4) option.inject)
next
  case ("5_1" e s v restParams va restArgs)
  then show ?case by auto
next
  case ("5_2" e s v restParams va restArgs)
  then show ?case by auto
next
  case ("5_3" e s v restParams va restArgs)
  then show ?case by auto
next
  case ("5_4" e s v restParams va restArgs)
  then show ?case by auto
next
  case ("5_5" e s va restParams v restArgs)
  then show ?case by auto
```

```
next
  case ("5_6" e s va restParams v restArgs)
  then show ?case by auto
next
  case ("5_7" e s va restParams v restArgs)
  then show ?case by auto
next
  case ("5_8" e s va restParams v restArgs)
  then show ?case by auto
next
  case (6 e s v va)
  then show ?case by auto
next
  case (7 e s v va)
  then show ?case by auto
qed

lemma wellTypedArgumentsExecute:
"wellTypedArguments tyCtx params args ⟹
  wellTypedState tyCtx s ⟹
  evalFArguments e s params args ≠ None"
proof (induction arbitrary: s rule: wellTypedArguments.induct)
  case (1 tyCtx)
  then show ?case by auto
next
  case (2 tyCtx v va)
  then show ?case by auto
next
  case (3 tyCtx v va)
  then show ?case by auto
next
  case (4 tyCtx firstParam restParams firstArg restArgs)
  then show ?case apply auto
    subgoal premises ps proof (cases firstParam firstArg rule: FParameter.exhaust[case_product
         FArgument.exhaust])
      case (ValueParameter_ValueArgument valId val)
      then show ?thesis using ps apply auto
        by (cases "evalFValue e s val", auto simp add: wellTypedValueObservationEvaluates(1)
             insertValueWellTypedStateIsWellTyped ps)
    next
      case (ValueParameter_ObservationArgument x1 x2)
      then show ?thesis using ps by auto
    next
      case (ValueParameter_PubKeyArgument x1 x3)
```

```
      then show ?thesis using ps by auto
    next
      case (ObservationParameter_ValueArgument x2 x1)
      then show ?thesis using ps by auto
    next
      case (ObservationParameter_ObservationArgument obsId obs)
      then show ?thesis using ps apply auto
        apply (cases "evalFObservation e s obs", auto)
        using wellTypedValueObservationEvaluates(2) insertValueWellTypedStateIsWellTyped by auto
    next
      case (ObservationParameter_PubKeyArgument x2 x3)
      then show ?thesis using ps by auto
    next
      case (PubKeyParameter_ValueArgument x3 x1)
      then show ?thesis using ps by auto
    next
      case (PubKeyParameter_ObservationArgument x3 x2)
      then show ?thesis using ps by auto
    next
      case (PubKeyParameter_PubKeyArgument pkId pk)
      then show ?thesis using ps apply auto
        apply (cases "evalFParty s pk", auto)
        using wellTypedPartyEvaluates insertPubKeyWellTypedStateIsWellTyped by auto
    qed
    done
qed

lemma wellTypedArgumentsOnlyAddIdsToState:
"(wellTypedState tyCtx s ⟶
  wellTypedArguments tyCtx params args ⟶
  Some newS = evalFArguments e s params args ⟶
  wellTypedState tyCtx newS)"
proof (induction arbitrary: newS rule: evalFArguments.induct)
  case (1 e s)
  then show ?case by auto
next
  case (2 e s vid restParams val restArgs)
  then show ?case apply auto
    apply (cases "evalFValue e s val", auto)
    by (cases "evalFArguments e s restParams restArgs", auto simp add:
         insertValueWellTypedStateIsWellTyped)
next
  case (3 e s obsId restParams obs restArgs)
  then show ?case apply auto
```

```
    by (cases "evalFObservation e s obs", auto simp add: insertValueWellTypedStateIsWellTyped)
next
  case (4 e s pkId restParams pk restArgs)
  then show ?case apply auto
    by (cases "evalFParty s pk", auto simp add: insertPubKeyWellTypedStateIsWellTyped)
next
  case ("5_1" e s v restParams va restArgs)
  then show ?case by auto
next
  case ("5_2" e s v restParams va restArgs)
  then show ?case by auto
next
  case ("5_3" e s v restParams va restArgs)
  then show ?case by auto
next
  case ("5_4" e s v restParams va restArgs)
  then show ?case by auto
next
  case ("5_5" e s va restParams v restArgs)
  then show ?case by auto
next
  case ("5_6" e s va restParams v restArgs)
  then show ?case by auto
next
  case ("5_7" e s va restParams v restArgs)
  then show ?case by auto
next
  case ("5_8" e s va restParams v restArgs)
  then show ?case by auto
next
  case (6 e s v va)
  then show ?case by auto
next
  case (7 e s v va)
  then show ?case by auto
qed

lemma wellTypedArgumentsLookupPreviousBoundValueIds:
"(∀newS vid boundVal. ∃otherBoundVal. Some newS = evalFArguments e s params args ⟶
  MList.lookup vid (boundValues s) = Some boundVal ⟶
  MList.lookup vid (boundValues newS) = Some otherBoundVal)"
proof (induction rule: evalFArguments.induct)
  case (1 e s)
  then show ?case by auto
```

```
next
  case (2 e s vid restParams val restArgs)
  then show ?case apply (cases "evalFValue e s val", auto)
    by (metis MList.insert_lookup_Some insert_lookup_different)
next
  case (3 e s obsId restParams obs restArgs)
  then show ?case apply auto
    apply (cases "evalFObservation e s obs", auto)
    by (metis MList.insert_lookup_Some insert_lookup_different)
next
  case (4 e s pkId restParams pk restArgs)
  then show ?case by (cases "evalFParty s pk", auto)
next
  case ("5_1" e s v restParams va restArgs)
  then show ?case by auto
next
  case ("5_2" e s v restParams va restArgs)
  then show ?case by auto
next
  case ("5_3" e s v restParams va restArgs)
  then show ?case by auto
next
  case ("5_4" e s v restParams va restArgs)
  then show ?case by auto
next
  case ("5_5" e s va restParams v restArgs)
  then show ?case by auto
next
  case ("5_6" e s va restParams v restArgs)
  then show ?case by auto
next
  case ("5_7" e s va restParams v restArgs)
  then show ?case by auto
next
  case ("5_8" e s va restParams v restArgs)
  then show ?case by auto
next
  case (6 e s v va)
  then show ?case by auto
next
  case (7 e s v va)
  then show ?case by auto
qed
```

```
lemma wellTypedArgumentsLookupPreviousBoundPubKeyIds:
"(∀newS pkId boundVal. ∃otherBoundVal. Some newS = evalFArguments e s params args ⟶
  MList.lookup pkId (boundPubKeys s) = Some boundVal ⟶
  MList.lookup pkId (boundPubKeys newS) = Some otherBoundVal)"
proof (induction rule: evalFArguments.induct)
  case (1 e s)
  then show ?case by auto
next
  case (2 e s vid restParams val restArgs)
  then show ?case by (cases "evalFValue e s val", auto)
next
  case (3 e s obsId restParams obs restArgs)
  then show ?case by (cases "evalFObservation e s obs", auto)
next
  case (4 e s pkId restParams pk restArgs)
  then show ?case apply (cases "evalFParty s pk", auto)
    by (metis MList.insert_lookup_Some insert_lookup_different)
next
  case ("5_1" e s v restParams va restArgs)
  then show ?case by auto
next
  case ("5_2" e s v restParams va restArgs)
  then show ?case by auto
next
  case ("5_3" e s v restParams va restArgs)
  then show ?case by auto
next
  case ("5_4" e s v restParams va restArgs)
  then show ?case by auto
next
  case ("5_5" e s va restParams v restArgs)
  then show ?case by auto
next
  case ("5_6" e s va restParams v restArgs)
  then show ?case by auto
next
  case ("5_7" e s va restParams v restArgs)
  then show ?case by auto
next
  case ("5_8" e s va restParams v restArgs)
  then show ?case by auto
next
  case (6 e s v va)
  then show ?case by auto
```

```
next
  case (7 e s v va)
  then show ?case by auto
qed


lemma wellTypedArgumentsAddParamsToState:
"(∀ newS . Some newS = evalFArguments e s params args ⟶ wellTypedState (paramsToTypeContext
     params) newS)"
proof (induction rule: evalFArguments.induct)
  case (1 e s)
  then show ?case by auto
next
  case (2 e s vid restParams val restArgs)
  then show ?case apply auto
     apply (cases "evalFValue e s val", auto)
    using MList.insert_lookup_Some wellTypedArgumentsLookupPreviousBoundValueIds apply fastforce
    by (cases "evalFValue e s val", auto)
next
  case (3 e s obsId restParams obs restArgs)
  then show ?case apply auto
     apply (cases "evalFObservation e s obs", auto)
    using MList.insert_lookup_Some wellTypedArgumentsLookupPreviousBoundValueIds apply fastforce
    by (cases "evalFObservation e s obs", auto)
next
  case (4 e s pkId restParams pk restArgs)
  then show ?case apply auto
     apply (cases "evalFParty s pk", auto)
    using MList.insert_lookup_Some wellTypedArgumentsLookupPreviousBoundPubKeyIds apply fastforce
    by (cases "evalFParty s pk", auto)
next
  case ("5_1" e s v restParams va restArgs)
  then show ?case by auto
next
  case ("5_2" e s v restParams va restArgs)
  then show ?case by auto
next
  case ("5_3" e s v restParams va restArgs)
  then show ?case by auto
next
  case ("5_4" e s v restParams va restArgs)
  then show ?case by auto
next
  case ("5_5" e s va restParams v restArgs)
```

```
  then show ?case by auto
next
  case ("5_6" e s va restParams v restArgs)
  then show ?case by auto
next
  case ("5_7" e s va restParams v restArgs)
  then show ?case by auto
next
  case ("5_8" e s va restParams v restArgs)
  then show ?case by auto
next
  case (6 e s v va)
  then show ?case by auto
next
  case (7 e s v va)
  then show ?case by auto
qed

lemma wellTypedArgumentsResultWellTypedState:
"wellTypedArguments restTyCtx params args ⟹
    wellTypedState restTyCtx s ⟹
    Some newS = evalFArguments e s params args ⟹ wellTypedState (paramsToTypeContext params @
         restTyCtx) newS"
  by (metis wellTypedArgumentsAddParamsToState wellTypedArgumentsOnlyAddIdsToState
       wellTypedStateCombineTypeContexts)

fun lookupContractIdAbsInformation :: "FContext ⇒ Identifier ⇒ (FParameter list × FContract ×
     FContext) option" where
"lookupContractIdAbsInformation [] cid = None" |
"lookupContractIdAbsInformation (ContractAbstraction (boundCid, params, bodyCont)#restAbs) cid =
     (if boundCid = cid
  then Some (params, bodyCont, restAbs)
  else lookupContractIdAbsInformation restAbs cid)" |
"lookupContractIdAbsInformation (ValueAbstraction i#restAbs) cid = (if i = cid then None else
     lookupContractIdAbsInformation restAbs cid)" |
"lookupContractIdAbsInformation (ObservationAbstraction i#restAbs) cid = (if i = cid then None
     else lookupContractIdAbsInformation restAbs cid)" |
"lookupContractIdAbsInformation (PubKeyAbstraction i#restAbs) cid = (if i = cid then None else
     lookupContractIdAbsInformation restAbs cid)"

lemma lookupContractTypeWellTypedContext:
"(∀cid params bodyCont restCtx. ∃restTy . wellTypedContext tyCtx ctx ⟶
  lookupContractIdAbsInformation ctx cid = Some (params, bodyCont, restCtx) ⟶
  lookupContractIdParamTypeInformation tyCtx cid = Some (params, restTy))"
```

```
  by (induction tyCtx ctx rule: wellTypedContext.induct, auto)

lemma lookupContractAbstractionWellTypedContext:
"(∀cid params restTy. ∃bodyCont restCtx . wellTypedContext tyCtx ctx ⟶
  lookupContractIdParamTypeInformation tyCtx cid = Some (params, restTy) ⟶
  lookupContractIdAbsInformation ctx cid = Some (params, bodyCont, restCtx))"
  by (induction tyCtx ctx rule: wellTypedContext.induct, auto)

lemma lookupContractRestIsWellTyped:
"(∀cid params restTy bodyCont restCtx . wellTypedContext tyCtx ctx ⟶
  lookupContractIdParamTypeInformation tyCtx cid = Some (params, restTy) ⟶
  lookupContractIdAbsInformation ctx cid = Some (params, bodyCont, restCtx) ⟶
  wellTypedContext restTy restCtx)"
  by (induction tyCtx ctx rule: wellTypedContext.induct, auto)

lemma lookupContractFoundContractIsWellTyped:
"(∀cid params restTy bodyCont restCtx . wellTypedContext tyCtx ctx ⟶
  lookupContractIdParamTypeInformation tyCtx cid = Some (params, restTy) ⟶
  lookupContractIdAbsInformation ctx cid = Some (params, bodyCont, restCtx) ⟶
  wellTypedContract (paramsToTypeContext params @ restTy) bodyCont)"
  by (induction tyCtx ctx rule: wellTypedContext.induct, auto)

lemma lookupContractStateStaysWellTyped:
"(∀cid params restTy restCtx .
  wellTypedState tyCtx s ⟶
  lookupContractIdParamTypeInformation tyCtx cid = Some (params, restTy) ⟶
  wellTypedState restTy s)"
  by (induction tyCtx s rule: wellTypedState.induct, auto)

type_synonym FConfiguration = "FContract * FContext * FState * Environment * (ReduceWarning list)
      * (Payment list)"

inductive
  small_step_reduce :: "FConfiguration ⇒ FConfiguration ⇒ bool" (infix "→f" 55)
where
CloseRefund:  "refundOne (accounts s) = Some ((party, token, money), newAccount) ⟹
  (Close, ctx, s, env, warns, payments) →f
  (Close, ctx, (s(accounts := newAccount)), env, warns @ [ReduceNoWarning], payments @ [Payment
       party (SemanticsTypes.Party party) token money])" |
PayNonPositive: "⟦evalFValue env s val = Some res; res ≤ 0; Some fromAcc = (evalFParty s accId);
     Some toPayee = (evalFPayee s payee)⟧ ⟹
  (Pay accId payee token val cont, ctx, s, env, warns, payments) →f
  (cont, ctx, s, env, warns @ [ReduceNonPositivePay fromAcc toPayee token res], payments)" |
PayPositivePartialWithPayment: "⟦evalFValue env s val = Some res;
```

```
  res > 0;
  Some fromAcc = (evalFParty s accId);
  Some toPayee = (evalFPayee s payee);
  res > moneyInAccount fromAcc token (accounts s);
  updateMoneyInAccount fromAcc token 0 (accounts s) = newAccs;
  moneyInAccount fromAcc token (accounts s) = moneyToPay;
  giveMoney fromAcc toPayee token (moneyToPay) newAccs = (payment, finalAccs);
  payment = ReduceWithPayment somePayment⟧ ⟹
  (Pay accId payee token val cont, ctx, s, env, warns, payments) →f
  ((cont, ctx, s⦇accounts := finalAccs⦈, env, warns @ [ReducePartialPay fromAcc toPayee token
      moneyToPay res], payments @ [somePayment]))" |
PayPositivePartialWithoutPayment: "⟦evalFValue env s val = Some res;
  res > 0;
  Some fromAcc = (evalFParty s accId);
  Some toPayee = (evalFPayee s payee);
  res > moneyInAccount fromAcc token (accounts s);
  updateMoneyInAccount fromAcc token 0 (accounts s) = newAccs;
  moneyInAccount fromAcc token (accounts s) = moneyToPay;
  giveMoney fromAcc toPayee token (moneyToPay) newAccs = (payment, finalAccs);
  payment = ReduceNoPayment⟧ ⟹
  (Pay accId payee token val cont, ctx, s, env, warns, payments) →f
  ((cont, ctx, s⦇accounts := finalAccs⦈, env, warns @ [ReducePartialPay fromAcc toPayee token
      moneyToPay res], payments))" |
PayPositiveFullWithPayment: "⟦evalFValue env s val = Some res;
  res > 0;
  Some fromAcc = (evalFParty s accId);
  Some toPayee = (evalFPayee s payee);
  res ≤ moneyInAccount fromAcc token (accounts s);
  moneyInAccount fromAcc token (accounts s) = moneyInAcc;
  moneyInAcc - res = newBalance;
  updateMoneyInAccount fromAcc token newBalance (accounts s) = newAccs;
  giveMoney fromAcc toPayee token res newAccs = (payment, finalAccs);
  payment = ReduceWithPayment somePayment⟧ ⟹
  (Pay accId payee token val cont, ctx, s, env, warns, payments) →f
  (cont, ctx, s⦇accounts := finalAccs⦈, env, warns @ [ReduceNoWarning], payments @ [somePayment])
      " |
PayPositiveFullWithoutPayment: "⟦evalFValue env s val = Some res;
  res > 0;
  Some fromAcc = (evalFParty s accId);
  Some toPayee = (evalFPayee s payee);
  res ≤ moneyInAccount fromAcc token (accounts s);
  moneyInAccount fromAcc token (accounts s) = moneyInAcc;
  moneyInAcc - res = newBalance;
  updateMoneyInAccount fromAcc token newBalance (accounts s) = newAccs;
```

```
  giveMoney fromAcc toPayee token res newAccs = (payment, finalAccs);
  payment = ReduceNoPayment⟧ ⟹
  (Pay accId payee token val cont, ctx, s, env, warns, payments) →f
  (cont, ctx, s⦇accounts := finalAccs⦈, env, warns @ [ReduceNoWarning], payments)" |
IfTrue: "evalFObservation env s obs = Some True ⟹
  (If obs cont1 cont2, ctx, s, env, warns, payments) →f
  (cont1, ctx, s, env, warns @ [ReduceNoWarning], payments)" |
IfFalse: "evalFObservation env s obs = Some False ⟹
  (If obs cont1 cont2, ctx, s, env, warns, payments) →f
  (cont2, ctx, s, env, warns @ [ReduceNoWarning], payments)" |
WhenTimeout: "⟦timeInterval env = (startSlot, endSlot);
  endSlot ≥ timeout;
  startSlot ≥ timeout⟧ ⟹
  (When cases timeout cont, ctx, s, env, warns, payments) →f
  (cont, ctx, s, env, warns @ [ReduceNoWarning], payments)" |
LetShadow: "⟦lookup valId (boundValues s) = Some oldVal; evalFValue env s val = Some res⟧ ⟹
  (Let valId val cont, ctx, s, env, warns, payments) →f
  (cont, ValueAbstraction valId#ctx, s⦇ boundValues := MList.insert valId res (boundValues s)⦈,
       env, warns @ [ReduceShadowing valId oldVal res], payments)" |
LetNoShadow: "⟦lookup valId (boundValues s) = None; evalFValue env s val = Some res⟧ ⟹
  (Let valId val cont, ctx, s, env, warns, payments) →f
  (cont, ValueAbstraction valId#ctx, s⦇ boundValues := MList.insert valId res (boundValues s)⦈,
       env, warns @ [ReduceNoWarning], payments)" |
LetObservation: "⟦evalFObservation e s obs = Some res⟧ ⟹
  (LetObservation oId obs c, cx, s, e, ws, ps) →f
  (c, ObservationAbstraction oId#cx, s⦇boundValues := MList.insert oId (if res then 1 else 0) (
       boundValues s)⦈, e, ws @ [ReduceNoWarning], ps)" |
LetPubKey: "⟦evalFParty s pk = Some res⟧ ⟹
  (LetPubKey pId pk c, cx, s, e, ws, ps) →f
  (c, PubKeyAbstraction pId#cx, s⦇ boundPubKeys := MList.insert pId res (boundPubKeys s)⦈, e, ws
       @ [ReduceNoWarning], ps)" |
AssertTrue: "evalFObservation env s obs = Some True ⟹
  (Assert obs cont, ctx, s, env, warns, payments) →f
  (cont, ctx, s, env, warns @ [ReduceNoWarning], payments)" |
AssertFalse: "evalFObservation env s obs = Some False ⟹
  (Assert obs cont, ctx, s, env, warns, payments) →f
  (cont, ctx, s, env, warns @ [ReduceAssertionFailed], payments)" |
LetC: "(LetC cid params boundCon cont, ctx, s, env, warns, payments) →f
  (cont, ContractAbstraction (cid, params, boundCon)#ctx, s, env, warns @ [ReduceNoWarning],
       payments)" |
UseCFound: "⟦lookupContractIdAbsInformation ctx cid = Some (params, bodyCont, innerCtx);
  evalFArguments env s params args = Some newState⟧ ⟹
  (UseC cid args, ctx, s, env, warns, payments) →f
```

```
  (bodyCont, (paramsToFContext params)@innerCtx, newState, env, warns @ [ReduceNoWarning],
      payments)"

abbreviation
  small_step_reduces :: "FConfiguration ⇒ FConfiguration ⇒ bool" (infix "→f*" 55)
where "x →f* y == star small_step_reduce x y"

thm small_step_reduce.induct
lemmas small_step_reduce_induct = small_step_reduce.induct[split_format(complete)]
declare small_step_reduce.intros[simp,intro]

inductive_cases CloseE[elim!]: "(Close, ctx, s, env, warns, payments) →f ct"
thm CloseE
inductive_cases PayE[elim!]: "(Pay accId payee token val cont, ctx, s, env, warns, payments) →f
     ct"
thm PayE
inductive_cases IfE[elim!]: "(If obs cont1 cont2, ctx, s, env, warns, payments) →f ct"
thm IfE
inductive_cases WhenE[elim!]: "(When cases timeout cont, ctx, s, env, warns, payments) →f ct"
thm WhenE
inductive_cases LetE[elim!]: "(Let valId val cont, ctx, s, env, warns, payments) →f ct"
thm LetE
inductive_cases LetObservationE[elim!]: "(LetObservation obsId obs cont, ctx, s, env, warns,
     payments) →f ct"
thm LetObservationE
inductive_cases LetPubKeyE[elim!]: "(LetPubKey pkId pk cont, ctx, s, env, warns, payments) →f ct
     "
thm LetPubKeyE
inductive_cases AssertE[elim!]: "(Assert obs cont, ctx, s, env, warns, payments) →f ct"
thm AssertE
inductive_cases LetCE[elim!]: "(LetC cid boundCon params cont, ctx, s, env, warns, payments) →f
     ct"
thm LetCE
inductive_cases UseCE[elim!]: "(UseC cid args, ctx, s, env, warns, payments) →f ct"
thm UseCE

lemma deterministic:
  "cs →f cs' ⟹ cs →f cs'' ⟹ cs'' = cs'"
proof(induction arbitrary: cs'' rule: small_step_reduce.induct)
  case (PayNonPositive env s val res fromAcc accId toPayee payee token cont ctx warns payments)
  then show ?case apply auto
     apply (cases "evalFParty s accId", auto)
    by (cases "evalFPayee s payee", auto)
next
```

```
  case (PayPositivePartialWithPayment env s val res fromAcc accId toPayee payee token newAccs
        moneyToPay payment finalAccs somePayment cont ctx warns payments)
  then show ?case apply (auto)
    by (cases "evalFParty s accId"; cases "evalFPayee s payee", auto)+
next
  case (PayPositiveFullWithPayment env s val res fromAcc accId toPayee payee token moneyInAcc
        newBalance newAccs payment finalAccs somePayment cont ctx warns payments)
  then show ?case apply (auto)
    by (cases "evalFParty s accId"; cases "evalFPayee s payee", auto)+
qed auto

lemma smallStepWarningsAreArbitrary:
"(c, bc, s, e, w, p) →f (c', bc', s', e', w', p') ⟹
  (∀w'' . ∃w''' . (c, bc, s, e, w'', p) →f (c', bc', s', e', w''', p'))"
proof (induction c bc s e w p c' bc' s' e' w' p' rule: small_step_reduce_induct)
  case (CloseRefund s party token money newAccount ctx env warns payments)
  then show ?case by auto
next
  case (PayNonPositive env s val res fromAcc accId toPayee payee token cont ctx warns payments)
  then show ?case by auto
next
  case (PayPositivePartialWithPayment env s val res fromAcc accId toPayee payee token newAccs
        moneyToPay payment finalAccs somePayment cont ctx warns payments)
  then show ?case by fastforce
next
  case (PayPositivePartialWithoutPayment env s val res fromAcc accId toPayee payee token newAccs
        moneyToPay payment finalAccs cont ctx warns payments)
  then show ?case by fastforce
next
  case (PayPositiveFullWithPayment env s val res fromAcc accId toPayee payee token moneyInAcc
        newBalance newAccs payment finalAccs somePayment cont ctx warns payments)
  then show ?case by fastforce
next
  case (PayPositiveFullWithoutPayment env s val res fromAcc accId toPayee payee token moneyInAcc
        newBalance newAccs payment finalAccs cont ctx warns payments)
  then show ?case by fastforce
next
  case (IfTrue env s obs cont1 cont2 ctx warns payments)
  then show ?case by auto
next
  case (IfFalse env s obs cont1 cont2 ctx warns payments)
  then show ?case by auto
next
  case (WhenTimeout env startSlot endSlot timeout cases cont ctx s warns payments)
```

```
  then show ?case by auto
next
  case (LetShadow valId s oldVal env val res cont ctx warns payments)
  then show ?case by auto
next
  case (LetNoShadow valId s env val res cont ctx warns payments)
  then show ?case by auto
next
  case (LetObservation env s obs res obsId cont ctx warns payments)
  then show ?case by fastforce
next
  case (LetPubKey s pk res pkId cont ctx env warns payments)
  then show ?case by auto
next
  case (AssertTrue env s obs cont ctx warns payments)
  then show ?case by auto
next
  case (AssertFalse env s obs cont ctx warns payments)
  then show ?case by auto
next
  case (LetC cid params boundCon cont ctx s env warns payments)
  then show ?case by auto
next
  case (UseCFound ctx cid params bodyCont innerCtx newState env s args warns payments)
  then show ?case by auto
qed

lemma smallStepStarWarningsAreArbitrary:
  "(c, bc, s, e, w, p) →f* (c', bc', s', e', w'', p') ⟹
    (∀w'' . ∃w''' . (c, bc, s, e, w'', p) →f* (c', bc', s', e', w''', p'))"
  apply (induction rule: star.induct[of "small_step_reduce", split_format(complete)], auto)
  by (meson smallStepWarningsAreArbitrary star.step)

definition "final cs ⟷ ¬(∃cs'. cs →f cs')"

lemma wellTypedPayContinues:
  assumes "wellTypedContract tyCtx (Pay accId payee token amt cont) ∧
  wellTypedContext tyCtx ctx ∧
  wellTypedState tyCtx s"
  shows "(∃ newS newW newP .(Pay accId payee token amt cont, ctx, s, e, w, p) →f (cont, ctx,
       newS, e, newW, newP))"
proof -
  obtain res where "evalFValue e s amt = Some res" using assms apply auto
    by (meson option.exhaust wellTypedValueObservationEvaluates(1))
```

```
  show ?thesis proof (cases "res ≤ 0")
    case True
    then show ?thesis using assms apply auto
      using \<open>evalFValue e s amt = Some res\<close> wellTypedPayeeEvaluates
           wellTypedPartyEvaluates by fastforce
  next
    case False
    then show ?thesis proof -
      obtain fromAcc where "evalFParty s accId = Some fromAcc" using assms apply auto
        by (meson option.exhaust_sel wellTypedPartyEvaluates)
      moreover obtain toPayee where "evalFPayee s payee = Some toPayee" using assms apply auto
        by (meson option.exhaust wellTypedPayeeEvaluates)
      ultimately show ?thesis proof (cases "res > moneyInAccount fromAcc token (accounts s)")
        case True
        then show ?thesis using assms apply auto
          by (metis FaustusSemantics.small_step_reduce.PayNonPositive FaustusSemantics.
               small_step_reduce.PayPositiveFullWithPayment FaustusSemantics.small_step_reduce.
               PayPositivePartialWithPayment  \<open>\<And>thesis. (\<And>res. evalFValue e s amt
                = Some res ⟹ thesis) ⟹ thesis\<close> \<open>evalFPayee s payee = Some
               toPayee\<close> \<open>evalFParty s accId = Some fromAcc\<close> giveMoney.elims
               not_less)
      next
        case False
        then show ?thesis using assms apply auto
          by (metis False FaustusSemantics.small_step_reduce.PayNonPositive FaustusSemantics.
               small_step_reduce.PayPositiveFullWithPayment \<open>evalFPayee s payee = Some
               toPayee\<close> \<open>evalFParty s accId = Some fromAcc\<close> \<open>evalFValue
                e s amt = Some res\<close> giveMoney.elims not_less)
      qed
    qed
  qed
qed

lemma wellTypedAssertContinues:
  assumes "wellTypedContract tyCtx (Assert obs cont) ∧
  wellTypedContext tyCtx ctx ∧
  wellTypedState tyCtx s"
  shows "(∃ newW newP .(Assert obs cont, ctx, s, e, w, p) →f (cont, ctx, s, e, newW, newP))"
proof -
  obtain res where "evalFObservation e s obs = Some res" using assms apply auto
    by (meson option.exhaust wellTypedValueObservationEvaluates(2))
  show ?thesis proof (cases res)
    case True
    then show ?thesis using assms \<open>evalFObservation e s obs = Some res\<close> by force
```

```
  next
    case False
    then show ?thesis using assms \<open>evalFObservation e s obs = Some res\<close> by force
  qed
qed

lemma wellTypedUseCContinues:
  assumes "wellTypedContract tyCtx (UseC cid args) ∧
  wellTypedContext tyCtx ctx ∧
  wellTypedState tyCtx s"
  shows "(∃ newCont newCtx newS newW .(UseC cid args, ctx, s, e, w, p) →f (newCont, newCtx, newS
       , e, newW, p))"
  using assms apply (cases "lookupContractIdParamTypeInformation tyCtx cid", auto)
  by (metis UseCFound lookupContractAbstractionWellTypedContext option.exhaust_sel
       wellTypedArgumentsExecute)

(* A well typed final configuration only has Close or When contracts. *)
lemma finalD: "(final (c1, cx, s, e, w, p) ∧ (wellTypedContract tcx c1) ∧ wellTypedContext tcx cx
      ∧ wellTypedState tcx s) ⟹ c1 = Close ∨ (∃ cs t c2 . c1 = When cs t c2)"
  apply (auto simp add: final_def)
  apply (cases c1, auto)
         apply (meson wellTypedContract.simps(2) wellTypedPayContinues)
        apply (smt FaustusSemantics.small_step_reduce.IfFalse FaustusSemantics.small_step_reduce.
             intros(7) option.exhaust_sel wellTypedValueObservationEvaluates(2))
       apply (metis FaustusSemantics.small_step_reduce.LetNoShadow FaustusSemantics.
            small_step_reduce.LetShadow not_Some_eq wellTypedValueObservationEvaluates(1))
      apply (meson FaustusSemantics.small_step_reduce.intros(12) option.exhaust_sel
           wellTypedValueObservationEvaluates(2))
     apply (meson FaustusSemantics.small_step_reduce.intros(13) option.exhaust_sel
          wellTypedPartyEvaluates)
    apply (meson wellTypedAssertContinues wellTypedContract.simps)
   apply blast
  by (metis wellTypedContract.simps(10) wellTypedUseCContinues)

lemma insertGivesSameDomainAsConsCompared:
"set (map fst m) \<subseteq> set c ⟹ set (map fst (MList.insert newKey (newVal) (m))) \<
     subseteq> set ((newKey)#c)"
  by (induction m rule: MList.insert.induct, auto)

lemma typePreservationUseC:
  assumes"(UseC cid args, ContractAbstraction (cid, params, newContract) # innerCtx, s, e, w, p)
       →f (newContract, newCtx, newS, newE, newW, newP) ∧
    wellTypedContract ((cid, ContractType params) # restTyCtx) (UseC cid args) ∧
```

```
    wellTypedContext ((cid, ContractType params) # restTyCtx) (ContractAbstraction (cid, params,
         newContract) # innerCtx) ∧
    wellTypedState ((cid, ContractType params) # restTyCtx) s"
  shows "wellTypedContract (paramsToTypeContext params@restTyCtx) newContract ∧
    wellTypedContext (paramsToTypeContext params@restTyCtx) newCtx ∧
    wellTypedState (paramsToTypeContext params@restTyCtx) newS"
  using assms apply auto
  using concatWellTypedContexts paramsWellTypedContext apply blast
  by (metis wellTypedArgumentsAddParamsToState wellTypedArgumentsOnlyAddIdsToState wellTypedState
       .simps(5) wellTypedStateCombineTypeContexts)

(* A single step of reduction maintains type correctness. *)
lemma typePreservation:
assumes
  "(c1, cx1, s1, e1, w1, p1) →f (c2, cx2, s2, e2, w2, p2) ∧
  wellTypedContract tcx1 c1 ∧
  wellTypedContext tcx1 cx1 ∧
  wellTypedState tcx1 s1"
shows
  "(∃tcx2 . wellTypedContract tcx2 c2 ∧
  wellTypedContext tcx2 cx2 ∧
  wellTypedState tcx2 s2)"
  using assms apply (cases c1, auto)
  using wellTypedStateAccountsArbitrary apply blast
  using wellTypedStateAccountsArbitrary apply blast
  using wellTypedStateAccountsArbitrary apply blast
  using wellTypedStateAccountsArbitrary apply blast
  using consValueWellTypedContext insertAppendValueWellTypedStateIsWellTyped apply blast
  subgoal premises ps proof -
    obtain vid val where "c1 = Let vid val c2" using ps by auto
    moreover have "wellTypedContext ((vid, ValueType)#tcx1) cx2" using calculation ps assms by
         auto
    moreover have "wellTypedState ((vid, ValueType)#tcx1) s2" using calculation ps assms apply (
         auto simp add: MList.insert_lookup_Some)
      using insertValueWellTypedStateIsWellTyped by blast
    ultimately show ?thesis using ps by blast
  qed
  subgoal premises ps proof -
    obtain vid obs where "c1 = LetObservation vid obs c2" using ps by auto
    moreover have "wellTypedContext ((vid, ObservationType)#tcx1) cx2" using calculation ps assms
          by auto
    moreover have "wellTypedState ((vid, ObservationType)#tcx1) s2" using calculation ps assms
         apply simp
      by (metis MList.insert_lookup_Some insertValueWellTypedStateIsWellTyped)
```

```
    ultimately show ?thesis using ps by blast
  qed
  subgoal premises ps proof -
    obtain obsId obs where "c1 = LetObservation obsId obs c2" using ps by auto
    moreover have "wellTypedContext ((obsId, ObservationType)#tcx1) cx2" using calculation ps
          assms by auto
    moreover have "wellTypedState ((obsId, ObservationType)#tcx1) s2" using calculation ps assms
          apply simp
      apply (meson calculation ps assms insertValueWellTypedStateIsWellTyped MList.
            insert_lookup_Some)
      using insertValueWellTypedStateIsWellTyped by blast
    ultimately show ?thesis using ps by blast
  qed
  subgoal premises ps proof -
    obtain pkId pk where "c1 = LetPubKey pkId pk c2" using ps by auto
    moreover have "wellTypedContext ((pkId, PubKeyType)#tcx1) cx2" using calculation ps assms by
          auto
    moreover have "wellTypedState ((pkId, PubKeyType)#tcx1) s2" using calculation ps assms apply
          (auto simp add: MList.insert_lookup_Some)
      using insertPubKeyWellTypedStateIsWellTyped by blast
    ultimately show ?thesis using ps by blast
  qed
  subgoal premises ps proof -
    obtain cid args where "c1 = UseC cid args" using ps by auto
    moreover obtain params restTy where "lookupContractIdParamTypeInformation tcx1 cid = Some (
          params, restTy)" using ps assms calculation by (cases "
          lookupContractIdParamTypeInformation tcx1 cid", auto)
    moreover obtain  innerCtx where "lookupContractIdAbsInformation cx1 cid = Some (params, c2,
          innerCtx)" using ps assms calculation by (metis FContract.inject(9) Pair_inject
          lookupContractTypeWellTypedContext option.inject)
    moreover have "wellTypedContext restTy innerCtx" using calculation(2) calculation(3)
          lookupContractRestIsWellTyped ps(5) by blast
    moreover have "wellTypedContext (paramsToTypeContext params @ restTy) ((paramsToFContext
          params) @ innerCtx)" by (simp add: calculation(4) concatWellTypedContexts
          paramsWellTypedContext)
    ultimately show ?thesis using ps assms apply auto
      by (metis assms lookupContractFoundContractIsWellTyped lookupContractStateStaysWellTyped
            wellTypedArgumentsAddParamsToState wellTypedArgumentsOnlyAddIdsToState
            wellTypedStateCombineTypeContexts)
  qed
  done

(* Many steps of reduction maintains type correctness. *)
lemma typePreservationManySteps:
```

```
assumes"(contract, ctx, s, e, w, p) →f* (newContract, newCtx, newS, newE, newW, newP) ∧
    wellTypedContract tyCtx contract ∧
    wellTypedContext tyCtx ctx ∧
    wellTypedState tyCtx s"
shows "(∃newTyCtx . wellTypedContract newTyCtx newContract ∧
    wellTypedContext newTyCtx newCtx ∧
    wellTypedState newTyCtx newS)"
  using assms apply auto
  subgoal proof (induction arbitrary: tyCtx rule:star.induct[of small_step_reduce, split_format(
      complete)])
    case (refl)
    then show ?case by auto
  next
    case (step)
    then show ?case using typePreservation by blast
  qed
  done

(* A well typed contract will be able to continue unless in a final state. *)
lemma contractProgress:
  assumes "wellTypedContract tyCtx contract ∧
    wellTypedContext tyCtx ctx ∧
    wellTypedState tyCtx s ∧
    ¬(final (contract, ctx, s, e, w, p))"
  shows "∃cs'. (contract, ctx, s, e, w, p) →f cs'"
  using assms by (auto simp add: final_def)

(* A well typed program can get to a final state. *)
lemma typeSoundness:
  assumes "(contract, ctx, s, env, warns, payments) →f* (newContract, newCtx, newS, newEnv,
      newWarns, newPayments) ∧
          ¬(final (newContract, newCtx, newS, newEnv, newWarns, newPayments)) ∧
          wellTypedContract tyCtx contract ∧
          wellTypedContext tyCtx ctx ∧
          wellTypedState tyCtx s"
  shows "∃cs''. (newContract, newCtx, newS, newEnv, newWarns, newPayments) →f cs''"
  using assms by (auto simp add: final_def)

fun compileFParty :: "FParty ⇒ FState ⇒ Party option" where
"compileFParty (UseParty pkId) fs = MList.lookup pkId (boundPubKeys fs)" |
"compileFParty (ConstantPubKey pk) s = Some (PubKey pk)" |
"compileFParty (Role role) s = Some (SemanticsTypes.Role role)"

fun compileFPayee :: "FPayee ⇒ FState ⇒ Payee option" where
```

```
"compileFPayee (Account pk) fs = (case compileFParty pk fs of
  Some mPk ⇒ Some (SemanticsTypes.Account mPk)
  | None ⇒ None)" |
"compileFPayee (Party pk) fs = (case compileFParty pk fs of
  Some mPk ⇒ Some (SemanticsTypes.Party mPk)
  | None ⇒ None)"

fun compileFChoiceId :: "FChoiceId ⇒ FState ⇒ ChoiceId option" where
"compileFChoiceId (FChoiceId cname pk) fs = (case compileFParty pk fs of
  Some mPk ⇒ Some (ChoiceId cname mPk)
  | None ⇒ None)"

(* Convert observation abstractions to values. 0 is false and 1 is true. *)
fun compileFValue :: "FValue ⇒ FState ⇒ Value option" and
  compileFObservation :: "FObservation ⇒ FState ⇒ Observation option" where
"compileFValue (AvailableMoney fAccountId token) s = (case compileFParty fAccountId s of
  Some mPk ⇒ Some (SemanticsTypes.AvailableMoney mPk token)
  | None ⇒ None)" |
"compileFValue (Constant v) s = Some (SemanticsTypes.Constant v)" |
"compileFValue (NegValue v) s = (case compileFValue v s of
  Some mv ⇒ Some (SemanticsTypes.NegValue mv)
  | None ⇒ None)" |
"compileFValue (AddValue lhs rhs) s = (case (compileFValue lhs s, compileFValue rhs s) of
  (Some compLhs, Some compRhs) ⇒ Some (SemanticsTypes.AddValue compLhs compRhs)
  | _ ⇒ None)" |
"compileFValue (SubValue lhs rhs) s = (case (compileFValue lhs s, compileFValue rhs s) of
  (Some compLhs, Some compRhs) ⇒ Some (SemanticsTypes.SubValue compLhs compRhs)
  | _ ⇒ None)" |
"compileFValue (MulValue lhs rhs) s = (case (compileFValue lhs s, compileFValue rhs s) of
  (Some compLhs, Some compRhs) ⇒ Some (SemanticsTypes.MulValue compLhs compRhs)
  | _ ⇒ None)" |
"compileFValue (DivValue lhs rhs) s = (case (compileFValue lhs s, compileFValue rhs s) of
  (Some compLhs, Some compRhs) ⇒ Some (SemanticsTypes.DivValue compLhs compRhs)
  | _ ⇒ None)" |
"compileFValue (ChoiceValue choiceId) s = (case compileFChoiceId choiceId s of
  Some compChoiceId ⇒ Some (SemanticsTypes.ChoiceValue compChoiceId)
  | None ⇒ None)" |
"compileFValue TimeIntervalStart s = Some SemanticsTypes.TimeIntervalStart" |
"compileFValue TimeIntervalEnd s = Some SemanticsTypes.TimeIntervalEnd" |
"compileFValue (UseValue valId) s = Some (SemanticsTypes.UseValue valId)" |
"compileFValue (Cond obs trueVal falseVal) s = (case (compileFObservation obs s, compileFValue
     trueVal s, compileFValue falseVal s) of
  (Some compObs, Some compTrueVal, Some compFalseVal) ⇒ Some (SemanticsTypes.Cond compObs
       compTrueVal compFalseVal)
```

```
  | _ ⇒ None)" |
"compileFObservation (AndObs lhs rhs) s = (case (compileFObservation lhs s, compileFObservation
     rhs s) of
  (Some compLhs, Some compRhs) ⇒ Some (SemanticsTypes.AndObs compLhs compRhs)
  | _ ⇒ None)" |
"compileFObservation (OrObs lhs rhs) s = (case (compileFObservation lhs s, compileFObservation
     rhs s) of
  (Some compLhs, Some compRhs) ⇒ Some (SemanticsTypes.OrObs compLhs compRhs)
  | _ ⇒ None)" |
"compileFObservation (NotObs obs) s = (case compileFObservation obs s of
  Some compObs ⇒ Some (SemanticsTypes.NotObs compObs)
  | None ⇒ None)" |
"compileFObservation (ValueGE lhs rhs) s = (case (compileFValue lhs s, compileFValue rhs s) of
  (Some compLhs, Some compRhs) ⇒ Some (SemanticsTypes.ValueGE compLhs compRhs)
  | _ ⇒ None)" |
"compileFObservation (ValueGT lhs rhs) s = (case (compileFValue lhs s, compileFValue rhs s) of
  (Some compLhs, Some compRhs) ⇒ Some (SemanticsTypes.ValueGT compLhs compRhs)
  | _ ⇒ None)" |
"compileFObservation (ValueLT lhs rhs) s = (case (compileFValue lhs s, compileFValue rhs s) of
  (Some compLhs, Some compRhs) ⇒ Some (SemanticsTypes.ValueLT compLhs compRhs)
  | _ ⇒ None)" |
"compileFObservation (ValueLE lhs rhs) s = (case (compileFValue lhs s, compileFValue rhs s) of
  (Some compLhs, Some compRhs) ⇒ Some (SemanticsTypes.ValueLE compLhs compRhs)
  | _ ⇒ None)" |
"compileFObservation (ValueEQ lhs rhs) s = (case (compileFValue lhs s, compileFValue rhs s) of
  (Some compLhs, Some compRhs) ⇒ Some (SemanticsTypes.ValueEQ compLhs compRhs)
  | _ ⇒ None)" |
"compileFObservation (UseObservation obsId) s = Some (SemanticsTypes.NotObs (SemanticsTypes.
     ValueEQ (SemanticsTypes.UseValue obsId) (SemanticsTypes.Constant 0)))" |
"compileFObservation (ChoseSomething choiceId) s = (case compileFChoiceId choiceId s of
  Some compChoiceId ⇒ Some (SemanticsTypes.ChoseSomething compChoiceId)
  | None ⇒ None)" |
"compileFObservation TrueObs s = Some SemanticsTypes.TrueObs" |
"compileFObservation FalseObs s = Some SemanticsTypes.FalseObs"

fun compileAction :: "FAction ⇒ FState ⇒ SemanticsTypes.Action option" where
"compileAction (Deposit fromPk toPk token value) s = (case (compileFParty fromPk s, compileFParty
      toPk s, compileFValue value s) of
  (Some compiledFromPk, Some compiledToPk, Some compiledValue) ⇒ Some (SemanticsTypes.Deposit
       compiledFromPk compiledToPk token compiledValue)
  | _ ⇒ None)" |
"compileAction (Choice choiceId bounds) s = (case compileFChoiceId choiceId s of
  Some compChoiceId ⇒ Some (SemanticsTypes.Choice compChoiceId bounds)
  | None ⇒ None)" |
```

```
"compileAction (Notify observation) s = (case compileFObservation observation s of
  Some compiledObservation ⇒ Some (SemanticsTypes.Notify compiledObservation)
  | None ⇒ None)"

fun argumentsToCompilerState :: "FParameter list ⇒ FArgument list ⇒ FState ⇒ FState option"
     where
"argumentsToCompilerState [] [] fs = Some fs" |
"argumentsToCompilerState params [] fs = None" |
"argumentsToCompilerState [] args fs = None" |
"argumentsToCompilerState (PubKeyParameter pkId#restParams) (PubKeyArgument pk#restArgs) fs =
  (case compileFParty pk fs of
    Some pkCompiled ⇒ argumentsToCompilerState restParams restArgs (fs⦇boundPubKeys := MList.
         insert pkId pkCompiled (boundPubKeys fs)⦈)
    | None ⇒ None)" |
"argumentsToCompilerState (ValueParameter valId#restParams) (ValueArgument val#restArgs) ctx =
     argumentsToCompilerState restParams restArgs (ctx⦇boundValues := MList.insert valId 0 (
     boundValues ctx)⦈)" |
"argumentsToCompilerState (ObservationParameter obsId#restParams) (ObservationArgument obs#
     restArgs) ctx = argumentsToCompilerState restParams restArgs (ctx⦇boundValues := MList.
     insert obsId 0 (boundValues ctx)⦈)" |
"argumentsToCompilerState (firstParam#restParams) (firstArg#restArgs) ctx = None"

(* Creates Marlowe contract that executes parameters sequentially. *)
fun compileArguments :: "FParameter list ⇒ FArgument list ⇒ FState ⇒ Contract ⇒ Contract
     option" where
"compileArguments [] [] fs cont = Some cont" |
"compileArguments params [] fs cont = None" |
"compileArguments [] args fs cont = None" |
"compileArguments (ValueParameter valId#restParams) (ValueArgument val#restArgs) fs cont =
  (case compileFValue val fs of
  Some valCompiled ⇒ (case compileArguments restParams restArgs (fs⦇boundValues := MList.insert
       valId 0 (boundValues fs)⦈) cont of
    Some compiledRest ⇒ Some (SemanticsTypes.Let valId valCompiled compiledRest)
    | None ⇒ None)
  | None ⇒ None)" |
"compileArguments (ObservationParameter obsId#restParams) (ObservationArgument obs#restArgs) fs
     cont =
  (case compileFObservation obs fs of
  Some obsCompiled ⇒ (case compileArguments restParams restArgs (fs⦇boundValues := MList.insert
       obsId 0 (boundValues fs)⦈) cont of
    Some compiledRest ⇒ Some (SemanticsTypes.Let obsId (SemanticsTypes.Cond obsCompiled (
         SemanticsTypes.Constant 1) (SemanticsTypes.Constant 0)) compiledRest)
    | None ⇒ None)
  | None ⇒ None)" |
```

```
"compileArguments (PubKeyParameter pkId#restParams) (PubKeyArgument pk#restArgs) fs cont =
  (case compileFParty pk fs of
  Some pkCompiled ⇒ compileArguments restParams restArgs (fs(boundPubKeys := MList.insert pkId
       pkCompiled (boundPubKeys fs))) cont
  | None ⇒ None)" |
"compileArguments (firstParam#restParams) (firstArg#restArgs) ctx cont = None"

fun abstractionInformationSize :: "AbstractionInformation ⇒ nat" where
"abstractionInformationSize (PubKeyAbstraction pkId) = 0" |
"abstractionInformationSize (ValueAbstraction valId) = 0" |
"abstractionInformationSize (ObservationAbstraction obsId) = 0" |
"abstractionInformationSize (ContractAbstraction (cid, params, bodyCont)) = size params +
     fContractSize bodyCont"

lemma abstractionInformationSize_GE0:
"(abstractionInformationSize a) ≥ 0"
  by (cases a, auto simp add: fContractSize_GT0)

lemma paramsToFContextNoSize:
"paramsToFContext params = newCtx ⟹
  fold (+) (map (abstractionInformationSize) newCtx) 0 = 0"
  by (induction arbitrary: newCtx rule: paramsToFContext.induct, auto)

lemma abstractionInformationSizeFold0:
"fold (+) (map (abstractionInformationSize \<circ> snd) ctx) 0 = 0 ⟹
 fold (+) (map (abstractionInformationSize \<circ> snd) (ctx@ctx2)) 0 = fold (+) (map (
      abstractionInformationSize \<circ> snd) ctx2) 0"
  by auto

lemma lookupContractCompilerContextSmaller:
"lookupContractIdAbsInformation ctx cid = Some (params, bodyCont, innerCtx) ⟹
  1 + fold (+) (map (abstractionInformationSize) ctx) 0 > fContractSize bodyCont + fold (+) (map
       (abstractionInformationSize) (innerCtx)) 0"
  apply (induction rule: lookupContractIdAbsInformation.induct, auto)
     apply (smt add.commute add.right_neutral fold_plus_sum_list_rev less_add_Suc1 less_trans
          nat_add_left_cancel_less not_less_eq not_less_iff_gr_or_eq option.inject prod.inject)
    apply (meson option.distinct(1))
   apply (meson option.distinct(1))
  by (meson option.distinct(1))

function compile :: "FContract ⇒ FContext ⇒ FState ⇒ Contract option"
  and compileCases :: "FCase list ⇒ FContext ⇒ FState ⇒ SemanticsTypes.Case list option" where
"compile Close ctx state = Some SemanticsTypes.Close"|
```

```
"compile (Pay accId payee tok val cont) ctx state = (let innerCompilation = compile cont ctx
     state in
  let compiledAccIdOption = compileFParty accId state in
  let compiledPayeeOption = compileFPayee payee state in
  let compiledValueOption = compileFValue val state in
  case (innerCompilation, compiledAccIdOption, compiledPayeeOption, compiledValueOption) of
    (Some innerCompiled, Some compiledAccId, Some compiledPayee, Some compiledValue) ⇒ Some (
         SemanticsTypes.Pay compiledAccId compiledPayee tok compiledValue innerCompiled)
  | _ ⇒ None)"|
"compile (If obs trueCont falseCont) ctx state = (let trueCompilation = compile trueCont ctx
     state in
  let falseCompilation = compile falseCont ctx state in
  let obsCompilation = compileFObservation obs state in
  case (trueCompilation, falseCompilation, obsCompilation) of
    (Some trueCompiled, Some falseCompiled, Some obsCompiled) ⇒ Some (SemanticsTypes.If
         obsCompiled trueCompiled falseCompiled)
  | _ ⇒ None)"|
"compile (When cases t tCont) ctx state = (
    let compiledT = compile tCont ctx state in
    let newCases = compileCases cases ctx state in
    case (compiledT, newCases) of
      (Some compiledTCont, Some compiledCases) ⇒ Some (SemanticsTypes.When compiledCases t
           compiledTCont)
    | _ ⇒ None)"|
"compile (Let valId val cont) ctx state = (let innerCompilation = compile cont ((ValueAbstraction
      valId)#ctx) (state⦇boundValues := MList.insert valId 0 (boundValues state)⦈) in
  let valCompilation = compileFValue val state in
  case (innerCompilation, valCompilation) of
    (Some innerCompiled, Some valCompiled) ⇒ Some (SemanticsTypes.Let valId valCompiled
         innerCompiled)
  | _ ⇒ None)"|
"compile (LetObservation obsId obs cont) ctx state = (let innerCompilation = compile cont ((
     ObservationAbstraction obsId)#ctx) (state⦇boundValues := MList.insert obsId 0 (boundValues
     state)⦈) in
  let obsCompilation = compileFObservation obs state in
  case (innerCompilation, obsCompilation) of
    (Some innerCompiled, Some obsCompiled) ⇒ Some (SemanticsTypes.Let obsId (SemanticsTypes.Cond
          obsCompiled (SemanticsTypes.Constant 1) (SemanticsTypes.Constant 0)) innerCompiled)
  | _ ⇒ None)"|
"compile (LetPubKey pkId pk cont) ctx state = (case compileFParty pk state of
  Some pkCompiled ⇒ compile cont ((PubKeyAbstraction pkId)#ctx) (state⦇boundPubKeys := MList.
       insert pkId pkCompiled (boundPubKeys state)⦈)
  | None ⇒ None)"|
"compile (Assert obs cont) ctx state = (let innerCompilation = compile cont ctx state in
```

```
  let obsCompilation = compileFObservation obs state in
  case (innerCompilation, obsCompilation) of
    (Some innerCompiled, Some obsCompiled) ⇒ Some (SemanticsTypes.Assert obsCompiled
         innerCompiled)
  | _ ⇒ None)"|
"compile (LetC cid params boundCon cont) ctx state = compile cont ((ContractAbstraction (cid,
     params, boundCon))#ctx) state" |
"compile (UseC cid args) ctx state = (case lookupContractIdAbsInformation ctx cid of
  Some (params, bodyCont, innerCtx) ⇒ (case argumentsToCompilerState params args state of
    Some newState ⇒ (case compile bodyCont ((paramsToFContext params)@innerCtx) newState of
      Some bodyCompiled ⇒ compileArguments params args state bodyCompiled
      | None ⇒ None)
    | None ⇒ None)
  | None ⇒ None)" |
"compileCases ((Case a c) # rest) ctx state =
  (case (compile c ctx state, compileCases rest ctx state, compileAction a state) of
    (Some compiledCaseCont, Some compiledCases, Some compiledAction) ⇒ Some ((SemanticsTypes.
         Case compiledAction compiledCaseCont) # compiledCases)
    | _ ⇒ None)" |
"compileCases [] ctx state = Some []"
  by pat_completeness auto

termination
  apply (relation "measures [(λx::((FContract × FContext × FState) + (FCase list × FContext ×
       FState)) . case x of
    Inl (contract, ctx, state) ⇒ (fContractSize contract) + (fold (+) (map (
         abstractionInformationSize) ctx) 0)
    | Inr (cases, ctx, state) ⇒ (fCasesSize cases) + (fold (+) (map (abstractionInformationSize)
          ctx) 0)),
    (λx::((FContract × FContext × FState) + (FCase list × FContext × FState)) . case x of
    Inl (contract, ctx, state) ⇒ length ctx
    | Inr (cases, ctx, state) ⇒ length ctx)]")
              apply (auto)[9]
  using paramsToFContextNoSize lookupContractCompilerContextSmaller apply fastforce
     apply (auto simp add: fold_plus_sum_list_rev fContractSize_GT0
          abstractionInformationSize_GE0)[1]
    apply (auto)[1]
  using paramsToFContextNoSize lookupContractCompilerContextSmaller apply (fastforce+)[2]
  by (auto simp add: fold_plus_sum_list_rev fContractSize_GT0 abstractionInformationSize_GE0)+

lemma compileCorrectness_Close:
"compile Close ctx state = Some (SemanticsTypes.Close)"
  by auto
```

```
lemma compileCorrectness_Pay:
"compile (Pay accId payee tok val cont) ctx state = Some (mCont) ⟹
  ∃innerMCont compiledAccId compiledPayee compiledVal .compile cont ctx state = Some (innerMCont)
        ∧
  compileFParty accId state = Some compiledAccId ∧
  compileFPayee payee state = Some compiledPayee ∧
  compileFValue val state = Some compiledVal ∧
  mCont = (SemanticsTypes.Pay compiledAccId compiledPayee tok compiledVal innerMCont)"
  apply auto
  apply (cases "compile cont ctx state", auto)
  apply (cases "compileFParty accId state", auto)
  apply (cases "compileFPayee payee state", auto)
  by (cases "compileFValue val state", auto)

lemma compileCorrectness_If:
"compile (If obs trueCont falseCont) ctx state = Some (mCont) ⟹
  ∃trueMCont falseMCont compiledObs. compile trueCont ctx state = Some (trueMCont) ∧
  compile falseCont ctx state = Some (falseMCont) ∧
  compileFObservation obs state = Some compiledObs ∧
  (mCont = SemanticsTypes.If compiledObs trueMCont falseMCont)"
  apply auto
  apply (cases "compile trueCont ctx state", auto)
  apply (cases "compile falseCont ctx state", auto)
  by (cases "compileFObservation obs state", auto)

lemma compileCorrectness_When:
"compile (When cases t tCont) ctx state = Some (mCont) ⟹
∃mcs mTCont. compile tCont ctx state = Some (mTCont) ∧
  mCont = (SemanticsTypes.When mcs t mTCont)"
  apply auto
  apply (cases "compile tCont ctx state", auto)
  by(cases "compileCases cases ctx state", auto)

lemma compileCorrectness_Let:
"compile (Let vid val cont) ctx state = Some (mCont) ⟹
  ∃innerMCont valId compiledVal . compile cont ((ValueAbstraction vid)#ctx) (state⦇FState.
       boundValues := MList.insert vid 0 (FState.boundValues state)⦈) = Some (innerMCont) ∧
  compileFValue val state = Some compiledVal ∧
  mCont = (SemanticsTypes.Let valId compiledVal innerMCont)"
  apply auto
  apply (cases "compile cont ((ValueAbstraction vid)#ctx) (state⦇FState.boundValues := MList.
       insert vid 0 (FState.boundValues state)⦈)", auto)
  by (cases "compileFValue val state", auto)
```

```
lemma compileCorrectness_LetObservation:
"compile (LetObservation obsId obs cont) ctx state = Some (mCont) ⟹
  ∃innerMCont valId compiledObs . compile cont ((ObservationAbstraction obsId)#ctx) (state⦇FState
       .boundValues := MList.insert obsId 0 (FState.boundValues state)⦈) = Some (innerMCont) ∧
  compileFObservation obs state = Some compiledObs ∧
  mCont = (SemanticsTypes.Let valId (SemanticsTypes.Cond compiledObs (SemanticsTypes.Constant 1)
       (SemanticsTypes.Constant 0)) innerMCont)"
  apply auto
  apply (cases "compile cont ((ObservationAbstraction obsId)#ctx) (state⦇FState.boundValues :=
       MList.insert obsId 0 (FState.boundValues state)⦈)", auto)
  by (cases "compileFObservation obs state", auto)

lemma compileCorrectness_LetPubKey:
"compile (LetPubKey pkId pk cont) ctx state = Some (mCont) ⟹
  ∃innerMCont valId compiledPk . compile cont ((PubKeyAbstraction pkId)#ctx) (state⦇boundPubKeys
       := MList.insert pkId compiledPk (boundPubKeys state)⦈) = Some (innerMCont) ∧
  compileFParty pk state = Some compiledPk ∧
  mCont = innerMCont"
  apply auto
  by (cases "compileFParty pk state", auto)

lemma compileCorrectness_Assert:
"compile (Assert obs cont) ctx state = Some (mCont) ⟹
  ∃innerMCont compiledObs. compile cont ctx state = Some (innerMCont) ∧
  compileFObservation obs state = Some compiledObs ∧
  mCont = (SemanticsTypes.Assert compiledObs innerMCont)"
  apply auto
  apply (cases "compile cont ctx state", auto)
  by (cases "compileFObservation obs state", auto)

lemma compileCorrectness_LetC:
"compile (LetC cid params boundCon cont) ctx state = Some (mCont) ⟹
  ∃innerMCont . compile cont ((ContractAbstraction (cid, params, boundCon))#ctx) state = Some (
       innerMCont) ∧
  mCont = innerMCont"
  by auto

lemma compileCorrectness_UseC:
"compile (UseC cid args) ctx state = Some (mCont) ⟹
  ∃params bodyCont innerCtx newState bodyCompiled innerMCont . lookupContractIdAbsInformation ctx
        cid = Some (params, bodyCont, innerCtx) ∧
  argumentsToCompilerState params args state = Some newState ∧
  compile bodyCont ((paramsToFContext params)@innerCtx) newState = Some bodyCompiled ∧
  compileArguments params args state bodyCompiled = Some (innerMCont) ∧
```

```
  mCont = innerMCont"
  by (auto split: option.split_asm)

lemma wellTypedPartyCompiles:
"(∃compiledPk. wellTypedParty tyCtx pk ⟶
  wellTypedState tyCtx state ⟶
  compileFParty pk state = Some (compiledPk))"
proof (induction rule: wellTypedState.induct)
  case (1 s)
  then show ?case by (cases pk, auto)
next
  case (2 valId restTypes s)
  then show ?case by (cases pk, auto)
next
  case (3 obsId restTypes s)
  then show ?case by (cases pk, auto)
next
  case (4 pkId restTypes s)
  then show ?case by (cases pk, auto)
next
  case (5 cid params restTypes s)
  then show ?case by (cases pk, auto)
qed

lemma wellTypedPayeeCompiles:
"wellTypedPayee tyCtx payee ⟶
  wellTypedState tyCtx state ⟶
  (∃compiledPayee . compileFPayee payee state = Some (compiledPayee))"
  apply (cases payee, auto)
  using wellTypedPartyCompiles apply force
  using wellTypedPartyCompiles by fastforce

lemma wellTypedChoiceIdCompiles:
"wellTypedChoiceId tyCtx choiceId ⟶
  wellTypedState tyCtx state ⟶
  (∃compiledChoiceId . compileFChoiceId choiceId state = Some (compiledChoiceId))"
  apply (cases choiceId, auto split: option.split)
  using wellTypedPartyCompiles by fastforce

lemma wellTypedValueObservationCompiles:
"wellTypedValue tyCtx val ⟶
  wellTypedState tyCtx state ⟶
  (∃compiledVal . compileFValue val state = Some compiledVal)"
"wellTypedObservation tyCtx obs ⟶
```

```
  wellTypedState tyCtx state ⟶
  (∃compiledObs . compileFObservation obs state = Some compiledObs)"
   apply (induction rule: compileFValue_compileFObservation.induct, auto split: option.split)
  using wellTypedPartyCompiles apply fastforce
  using wellTypedChoiceIdCompiles apply fastforce
  using wellTypedChoiceIdCompiles by fastforce

lemma wellTypedArgumentsCompilerContext:
"(∀tyCtx . ∃newState . wellTypedState tyCtx state ⟶
  wellTypedArguments tyCtx params args ⟶
  argumentsToCompilerState params args state = Some newState)"
  apply (induction rule: argumentsToCompilerState.induct, auto)
    apply (metis (mono_tags, lifting) FState.fold_congs(4) insertPubKeyWellTypedStateIsWellTyped
         option.simps(5) wellTypedPartyCompiles)
  using insertValueWellTypedStateIsWellTyped apply blast
  using insertValueWellTypedStateIsWellTyped by blast

lemma argumentsCompilerContextLookupPreviousBoundValueIds:
"(∀newS vid boundVal. ∃otherBoundVal. Some newS = argumentsToCompilerState params args state ⟶
  MList.lookup vid (boundValues state) = Some boundVal ⟶
  MList.lookup vid (boundValues newS) = Some otherBoundVal)"
proof (induction rule: argumentsToCompilerState.induct)
  case (1 fs)
  then show ?case by auto
next
  case (2 v va fs)
  then show ?case by auto
next
  case (3 v va fs)
  then show ?case by auto
next
  case (4 pkId restParams pk restArgs fs)
  then show ?case by (cases "compileFParty pk fs", auto)
next
  case (5 valId restParams val restArgs ctx)
  then show ?case apply auto
    by (metis MList.insert_lookup_Some insert_lookup_different)
next
  case (6 obsId restParams obs restArgs ctx)
  then show ?case apply auto
    by (metis MList.insert_lookup_Some insert_lookup_different)
next
  case ("7_1" v restParams va restArgs ctx)
  then show ?case by auto
```

```
next
  case ("7_2" v restParams va restArgs ctx)
  then show ?case by auto
next
  case ("7_3" v restParams va restArgs ctx)
  then show ?case by auto
next
  case ("7_4" v restParams va restArgs ctx)
  then show ?case by auto
next
  case ("7_5" va restParams v restArgs ctx)
  then show ?case by auto
next
  case ("7_6" va restParams v restArgs ctx)
  then show ?case by auto
next
  case ("7_7" va restParams v restArgs ctx)
  then show ?case by auto
next
  case ("7_8" va restParams v restArgs ctx)
  then show ?case by auto
qed

lemma argumentsCompilerContextLookupPreviousBoundPubKeysIds:
"(∀newS pkId boundVal. ∃otherBoundVal. Some newS = argumentsToCompilerState params args state ⟶
  MList.lookup pkId (boundPubKeys state) = Some boundVal ⟶
  MList.lookup pkId (boundPubKeys newS) = Some otherBoundVal)"
proof (induction rule: argumentsToCompilerState.induct)
  case (1 fs)
  then show ?case by auto
next
  case (2 v va fs)
  then show ?case by auto
next
  case (3 v va fs)
  then show ?case by auto
next
  case (4 pkId restParams pk restArgs fs)
  then show ?case apply (cases "compileFParty pk fs", auto)
    by (metis MList.insert_lookup_Some insert_lookup_different)
next
  case (5 valId restParams val restArgs ctx)
  then show ?case by auto
next
```

```
  case (6 obsId restParams obs restArgs ctx)
  then show ?case by auto
next
  case ("7_1" v restParams va restArgs ctx)
  then show ?case by auto
next
  case ("7_2" v restParams va restArgs ctx)
  then show ?case by auto
next
  case ("7_3" v restParams va restArgs ctx)
  then show ?case by auto
next
  case ("7_4" v restParams va restArgs ctx)
  then show ?case by auto
next
  case ("7_5" va restParams v restArgs ctx)
  then show ?case by auto
next
  case ("7_6" va restParams v restArgs ctx)
  then show ?case by auto
next
  case ("7_7" va restParams v restArgs ctx)
  then show ?case by auto
next
  case ("7_8" va restParams v restArgs ctx)
  then show ?case by auto
qed

lemma wellTypedArgumentsCompiles:
"(∀tyCtx innerCont ctx. ∃mCont. wellTypedArguments tyCtx params args ⟶
  wellTypedState tyCtx state ⟶
  compileArguments params args state innerCont = Some mCont)"
  apply (induction rule: compileArguments.induct, auto)
  using wellTypedValueObservationCompiles(1) apply (split option.split, auto)
  using insertValueWellTypedStateIsWellTyped apply (split option.split, auto)
  using wellTypedValueObservationCompiles(2) apply (split option.split, auto)
  apply (split option.split, auto)
  apply (split option.split, auto)
  using wellTypedPartyCompiles apply fastforce
  by (simp add: insertPubKeyWellTypedStateIsWellTyped)

lemma wellTypedArgumentsAddsParamsToCompilerState:
"(∀newState . argumentsToCompilerState params args state = Some newState ⟶ wellTypedState (
     paramsToTypeContext params) newState)"
```

```
proof (induction rule: argumentsToCompilerState.induct)
case (1 fs)
  then show ?case by auto
next
  case (2 v va fs)
  then show ?case by auto
next
  case (3 v va fs)
  then show ?case by auto
next
  case (4 pkId restParams pk restArgs fs)
  moreover have "∀a . ∃pk . lookup pkId (boundPubKeys (fs⦇boundPubKeys := MList.insert pkId a (
       boundPubKeys fs)⦈)) = Some pk" by (simp add: MList.insert_lookup_Some)
  ultimately show ?case apply (cases "compileFParty pk fs", auto)
    by (metis \<open>∀a. ∃pk. lookup pkId (boundPubKeys (fs⦇boundPubKeys := MList.insert pkId a (
         boundPubKeys fs)⦈)) = Some pk\<close>
         argumentsCompilerContextLookupPreviousBoundPubKeysIds)
next
  case (5 valId restParams val restArgs ctx)
  moreover have "∃y. lookup valId (FState.boundValues (ctx⦇FState.boundValues := MList.insert
       valId 0 (FState.boundValues ctx)⦈)) = Some y" by (simp add: MList.insert_lookup_Some)
  ultimately show ?case apply auto
    by (metis \<open>∃y. lookup valId (FState.boundValues (ctx⦇FState.boundValues := MList.insert
          valId 0 (FState.boundValues ctx)⦈)) = Some y\<close>
         argumentsCompilerContextLookupPreviousBoundValueIds)
next
  case (6 obsId restParams obs restArgs ctx)
  moreover have "∃y. lookup obsId (FState.boundValues (ctx⦇FState.boundValues := MList.insert
       obsId 0 (FState.boundValues ctx)⦈)) = Some y" by (simp add: MList.insert_lookup_Some)
  ultimately show ?case apply auto
    by (metis \<open>∃y. lookup obsId (FState.boundValues (ctx⦇FState.boundValues := MList.insert
          obsId 0 (FState.boundValues ctx)⦈)) = Some y\<close>
         argumentsCompilerContextLookupPreviousBoundValueIds)
next
  case ("7_1" v restParams va restArgs ctx)
  then show ?case by auto
next
  case ("7_2" v restParams va restArgs ctx)
  then show ?case by auto
next
  case ("7_3" v restParams va restArgs ctx)
  then show ?case by auto
next
  case ("7_4" v restParams va restArgs ctx)
```

```
  then show ?case by auto
next
  case ("7_5" va restParams v restArgs ctx)
  then show ?case by auto
next
  case ("7_6" va restParams v restArgs ctx)
  then show ?case by auto
next
  case ("7_7" va restParams v restArgs ctx)
  then show ?case by auto
next
  case ("7_8" va restParams v restArgs ctx)
  then show ?case by auto
qed

lemma wellTypedArgumentsOnlyAddIdsToCompilerState:
"(wellTypedState tyCtx s ⟶
  wellTypedArguments tyCtx params args ⟶
  Some newS = argumentsToCompilerState params args s ⟶
  wellTypedState tyCtx newS)"
proof (induction arbitrary: newS rule: argumentsToCompilerState.induct)
  case (1 fs)
  then show ?case by auto
next
  case (2 v va fs)
  then show ?case by auto
next
  case (3 v va fs)
  then show ?case by auto
next
  case (4 pkId restParams pk restArgs fs)
  then show ?case apply auto
    by (cases "compileFParty pk fs", auto simp add: insertPubKeyWellTypedStateIsWellTyped)
next
  case (5 valId restParams val restArgs ctx)
  then show ?case apply (cases "argumentsToCompilerState restParams restArgs ctx", auto simp add:
        MList.insert_lookup_Some)
    by (auto simp add: insertValueWellTypedStateIsWellTyped)
next
  case (6 obsId restParams obs restArgs ctx)
  then show ?case apply (cases "argumentsToCompilerState restParams restArgs ctx", auto simp add:
        MList.insert_lookup_Some)
    by (cases "argumentsToCompilerState restParams restArgs ctx", auto simp add:
         insertValueWellTypedStateIsWellTyped)
```

```
next
  case ("7_1" v restParams va restArgs ctx)
  then show ?case by auto
next
  case ("7_2" v restParams va restArgs ctx)
  then show ?case by auto
next
  case ("7_3" v restParams va restArgs ctx)
  then show ?case by auto
next
  case ("7_4" v restParams va restArgs ctx)
  then show ?case by auto
next
  case ("7_5" va restParams v restArgs ctx)
  then show ?case by auto
next
  case ("7_6" va restParams v restArgs ctx)
  then show ?case by auto
next
  case ("7_7" va restParams v restArgs ctx)
  then show ?case by auto
next
  case ("7_8" va restParams v restArgs ctx)
  then show ?case by auto
qed

lemma wellTypedCompiles_UseC_Inductive:
  assumes "lookupContractIdParamTypeInformation tyCtx cid = Some (x, restTy) ∧
  argumentsToCompilerState params args state = Some argCompilerState ∧
  lookupContractIdAbsInformation ctx cid = Some (params, bodyCont, restC) ∧
  (∀tyCtx.
        wellTypedContract tyCtx bodyCont ⟶
        wellTypedState tyCtx argCompilerState ⟶
        wellTypedContext tyCtx (paramsToFContext params @ restC) ⟶
        (∃mCont. compile bodyCont (paramsToFContext params @ restC) argCompilerState = Some mCont
             )) ∧
  wellTypedState tyCtx state ∧
  wellTypedContext tyCtx ctx ∧
  wellTypedArguments tyCtx x args"
  shows"∃mCont.
    (case compile bodyCont (paramsToFContext params @ restC) argCompilerState of None ⇒ None |
         Some x ⇒ compileArguments params args state x) =
    Some mCont"
proof -
```

```
  have "x = params" using lookupContractAbstractionWellTypedContext assms by fastforce
  moreover have "wellTypedContract (paramsToTypeContext params @ restTy) bodyCont" using
        calculation lookupContractFoundContractIsWellTyped assms by auto
  moreover have "wellTypedState (paramsToTypeContext params @ restTy) argCompilerState" by (metis
         assms calculation(1) lookupContractStateStaysWellTyped
        wellTypedArgumentsAddsParamsToCompilerState wellTypedArgumentsOnlyAddIdsToCompilerState
        wellTypedStateCombineTypeContexts)
  moreover have "wellTypedContext (paramsToTypeContext params @ restTy) (paramsToFContext params
        @ restC)" using lookupContractRestIsWellTyped assms calculation concatWellTypedContexts
        paramsWellTypedContext by blast
  moreover obtain bodyMCont where "compile bodyCont (paramsToFContext params @ restC)
        argCompilerState = Some bodyMCont" using calculation assms by auto
  ultimately show ?thesis using assms wellTypedArgumentsCompiles
        wellTypedArgumentsOnlyAddIdsToCompilerState by auto
qed

(* A well typed contract/cases will compile
  to a Marlowe contract/cases *)
lemma wellTypedCompiles:
"(∀tyCtx . (wellTypedContract tyCtx contract ⟶
  wellTypedState tyCtx state ⟶
  wellTypedContext tyCtx ctx ⟶
  (∃mCont . compile contract ctx state = Some mCont)))"
"(∀tyCtx .(wellTypedCases tyCtx cases ⟶
  wellTypedState tyCtx state ⟶
  wellTypedContext tyCtx ctx ⟶
  (∃mCases. compileCases cases ctx state = Some mCases)))"
proof (induction rule: compile_compileCases.induct)
  case (1 ctx)
  then show ?case by auto
next
  case (2 accId payee tok val cont ctx)
  then show ?case using wellTypedPayeeCompiles wellTypedPartyCompiles
        wellTypedValueObservationCompiles(1) by fastforce
next
  case (3 obs trueCont falseCont ctx)
  then show ?case using wellTypedValueObservationCompiles(2) by fastforce
next
  case (4 cases t tCont ctx)
  then show ?case by fastforce
next
  case (5 vid val cont ctx state)
  then show ?case apply auto
```

```
    using wellTypedValueObservationCompiles insertValueWellTypedStateIsWellTyped
         insertAppendValueWellTypedStateIsWellTyped by fastforce
next
  case (6 obsId obs cont ctx)
  then show ?case using wellTypedValueObservationCompiles(2)
       insertAppendValueWellTypedStateIsWellTyped by fastforce
next
  case (7 pkId pk cont ctx)
  then show ?case using wellTypedPartyCompiles insertAppendPubKeyWellTypedStateIsWellTyped by
       fastforce
next
  case (8 obs cont ctx)
  then show ?case using wellTypedValueObservationCompiles(2) by fastforce
next
  case (9 cid params boundCon cont ctx)
  then show ?case by auto
next
  case (10 cid args ctx state)
  then show ?case apply auto
    subgoal premises ps proof(cases "lookupContractIdAbsInformation ctx cid")
      case None
      then show ?thesis using ps apply auto
        by (smt case_optionE case_prodE lookupContractAbstractionWellTypedContext option.distinct
             (1))
    next
      case (Some compilerCidInfo)
      then show ?thesis using ps apply auto
        subgoal premises ps proof (cases compilerCidInfo)
          case (fields params bodyCont restC)
          then show ?thesis using ps apply auto
            subgoal premises ps proof (cases "argumentsToCompilerState params args state")
              case None
              then show ?thesis using lookupContractTypeWellTypedContext ps
                   wellTypedArgumentsCompilerContext by fastforce
            next
              case (Some argCompilerState)
              moreover obtain tyCtx where "(case lookupContractIdParamTypeInformation tyCtx cid
                   of None ⇒ False | Some (params, innerTyCtx) ⇒ wellTypedArguments tyCtx
                   params args) ∧ wellTypedState tyCtx state ∧ wellTypedContext tyCtx ctx" using
                   ps by auto
              ultimately show ?thesis using ps apply auto
                subgoal premises ps proof (cases "lookupContractIdParamTypeInformation tyCtx cid
                     ")
                  case None
```

```
                then show ?thesis using ps by auto
              next
                case (Some contractIdParamTypeInfo)
                then show ?thesis using ps wellTypedCompiles_UseC_Inductive by auto
              qed
              done
          qed
          done
      qed
      done
    qed
    done
next
  case (11 act cont rest ctx)
  then show ?case apply auto
    subgoal premises ps proof (cases act)
      case (Deposit x11 x12 x13 x14)
      then show ?thesis using ps apply auto
        by (smt case_prod_conv compileAction.simps(1) option.case_eq_if option.distinct(1)
             wellTypedPartyCompiles wellTypedValueObservationCompiles(1))
    next
      case (Choice x21 x22)
      then show ?thesis using ps wellTypedChoiceIdCompiles by fastforce
    next
      case (Notify obs)
      then show ?thesis using ps apply auto
        by (smt case_prod_conv compileAction.simps(3) option.case_eq_if option.distinct(1)
             wellTypedPartyCompiles wellTypedValueObservationCompiles(2))
    qed
    done
next
  case (12 ctx)
  then show ?case by auto
qed

lemma compilePubKeyAccountsArbitrary:
"compileFParty pk state = Some compiledPk ⟶
  compileFParty pk (state(accounts := newAccs)) = Some compiledPk"
  by (cases pk, auto)

lemma compilePayeeAccountsArbitrary:
"compileFPayee pk state = Some compiledPk ⟶
  compileFPayee pk (state(accounts := newAccs)) = Some compiledPk"
proof (cases pk)
```

```
  case (Account acc)
  then show ?thesis by (cases acc, auto)
next
  case (Party p)
  then show ?thesis by (cases p, auto)
qed

lemma compileChoiceIdAccountsArbitrary:
"compileFChoiceId choiceId state = Some compiledChoiceId ⟶
  compileFChoiceId choiceId (state⦇accounts := newAccs⦈) = Some compiledChoiceId"
  by (cases choiceId, auto split: option.split simp add: compilePubKeyAccountsArbitrary)

lemma compileValueAccountsArbitrary:
"(∀compiledVal newAccs. compileFValue val state = Some compiledVal ⟶
  compileFValue val (state⦇accounts := newAccs⦈) = Some compiledVal)"
"(∀compiledObs newAccs. compileFObservation obs state = Some compiledObs ⟶
  compileFObservation obs (state⦇accounts := newAccs⦈) = Some compiledObs)"
proof (induction rule: compileFValue_compileFObservation.induct)
  case (1 fPubKey token s)
  then show ?case using compilePubKeyAccountsArbitrary by (cases "compileFParty fPubKey s" "
       compileFParty fPubKey (s⦇FState.accounts := newAccs⦈)" rule: option.exhaust[case_product
       option.exhaust], auto)
next
  case (2 v s)
  then show ?case by auto
next
  case (3 v s)
  then show ?case by (cases "compileFValue v s", auto)
next
  case (4 lhs rhs s)
  then show ?case by (cases "compileFValue lhs s" "compileFValue rhs s" rule: option.exhaust[
       case_product option.exhaust], auto)
next
  case (5 lhs rhs s)
  then show ?case by (cases "compileFValue lhs s" "compileFValue rhs s" rule: option.exhaust[
       case_product option.exhaust], auto)
next
  case (6 lhs rhs s)
  then show ?case by (cases "compileFValue lhs s" "compileFValue rhs s" rule: option.exhaust[
       case_product option.exhaust], auto)
next
  case (7 lhs rhs s)
  then show ?case by (cases "compileFValue lhs s" "compileFValue rhs s" rule: option.exhaust[
       case_product option.exhaust], auto)
```

```
next
  case (8 choiceId s)
  then show ?case by (auto split: option.split simp add: compileChoiceIdAccountsArbitrary)
next
  case (9 s)
  then show ?case by auto
next
  case (10 s)
  then show ?case by auto
next
  case (11 valId s)
  then show ?case by auto
next
  case (12 obs trueVal falseVal s)
  then show ?case by (cases "compileFObservation obs s" "compileFValue trueVal s" "compileFValue
        falseVal s" rule: option.exhaust[case_product option.exhaust[case_product option.exhaust
        ]], auto)
next
  case (13 lhs rhs s)
  then show ?case by (cases "compileFObservation lhs s" "compileFObservation rhs s" rule: option.
        exhaust[case_product option.exhaust], auto)
next
  case (14 lhs rhs s)
  then show ?case by (cases "compileFObservation lhs s" "compileFObservation rhs s" rule: option.
        exhaust[case_product option.exhaust], auto)
next
  case (15 obs s)
  then show ?case by (cases "compileFObservation obs s", auto)
next
  case (16 lhs rhs s)
  then show ?case by (cases "compileFValue lhs s" "compileFValue rhs s" rule: option.exhaust[
        case_product option.exhaust], auto)
next
  case (17 lhs rhs s)
then show ?case by (cases "compileFValue lhs s" "compileFValue rhs s" rule: option.exhaust[
      case_product option.exhaust], auto)
next
  case (18 lhs rhs s)
  then show ?case by (cases "compileFValue lhs s" "compileFValue rhs s" rule: option.exhaust[
        case_product option.exhaust], auto)
next
  case (19 lhs rhs s)
  then show ?case by (cases "compileFValue lhs s" "compileFValue rhs s" rule: option.exhaust[
        case_product option.exhaust], auto)
```

```
next
  case (20 lhs rhs s)
  then show ?case by (cases "compileFValue lhs s" "compileFValue rhs s" rule: option.exhaust[
        case_product option.exhaust], auto)
next
  case (21 obsId s)
  then show ?case by auto
next
  case (22 choiceId s)
  then show ?case by (auto split: option.split simp add: compileChoiceIdAccountsArbitrary)
next
  case (23 s)
  then show ?case by auto
next
  case (24 s)
  then show ?case by auto
qed

lemma stateModificationOrderArbitrary:
"state⦇FState.boundValues := MList.insert valId 0 (FState.boundValues state), FState.accounts :=
      newAccs⦈ =
  state⦇FState.accounts := newAccs, FState.boundValues := MList.insert valId 0 (FState.
        boundValues state)⦈"
"state⦇FState.accounts := newAccs, boundPubKeys := MList.insert pkId a (boundPubKeys state)⦈ =
  state⦇boundPubKeys := MList.insert pkId a (boundPubKeys state), FState.accounts := newAccs⦈"
"state⦇FState.boundValues := newVals, boundPubKeys := newPubKeys⦈ =
  state⦇boundPubKeys := newPubKeys, FState.boundValues := newVals⦈"
  by auto

lemma argumentsToContextAccountsArbitrary:
"(∀newAccs newState . argumentsToCompilerState params args state = Some newState ⟶
  argumentsToCompilerState params args (state⦇FState.accounts := newAccs⦈) = Some (newState⦇
        FState.accounts := newAccs⦈))"
proof (induction rule: argumentsToCompilerState.induct)
  case (1 fs)
  then show ?case by auto
next
  case (2 v va fs)
  then show ?case by auto
next
  case (3 v va fs)
  then show ?case by auto
next
  case (4 pkId restParams pk restArgs fs)
```

```
  then show ?case apply (cases "compileFParty pk fs", auto)
    by (simp add: stateModificationOrderArbitrary compilePubKeyAccountsArbitrary)
next
  case (5 valId restParams val restArgs ctx)
  then show ?case by (simp add: stateModificationOrderArbitrary compilePubKeyAccountsArbitrary)
next
  case (6 obsId restParams obs restArgs ctx)
  then show ?case by (simp add: stateModificationOrderArbitrary compilePubKeyAccountsArbitrary)
next
  case ("7_1" v restParams va restArgs ctx)
  then show ?case by auto
next
  case ("7_2" v restParams va restArgs ctx)
  then show ?case by auto
next
  case ("7_3" v restParams va restArgs ctx)
  then show ?case by auto
next
  case ("7_4" v restParams va restArgs ctx)
  then show ?case by auto
next
  case ("7_5" va restParams v restArgs ctx)
  then show ?case by auto
next
  case ("7_6" va restParams v restArgs ctx)
  then show ?case by auto
next
  case ("7_7" va restParams v restArgs ctx)
  then show ?case by auto
next
  case ("7_8" va restParams v restArgs ctx)
  then show ?case by auto
qed

lemma compileArgumentsAccountsArbitrary:
"(∀newAccs compiledContract . compileArguments params args argCtx bodyCont = Some
     compiledContract ⟶
  compileArguments params args (argCtx(accounts := newAccs)) bodyCont = Some compiledContract)"
proof (induction rule: compileArguments.induct)
  case (1 fs cont)
  then show ?case by auto
next
  case (2 v va fs cont)
  then show ?case by auto
```

```
next
  case (3 v va fs cont)
  then show ?case by auto
next
  case (4 valId restParams val restArgs fs cont)
  then show ?case apply (cases "compileFValue val fs", auto)
    apply (cases "compileArguments restParams restArgs (fs⦇FState.boundValues := MList.insert
         valId 0 (FState.boundValues fs)⦈) cont", auto)
    by (simp add: stateModificationOrderArbitrary compileValueAccountsArbitrary)
next
  case (5 obsId restParams obs restArgs fs cont)
  then show ?case apply (cases "compileFObservation obs fs", auto)
    apply (cases "compileArguments restParams restArgs (fs⦇FState.boundValues := MList.insert
         obsId 0 (FState.boundValues fs)⦈) cont", auto)
    by (simp add: stateModificationOrderArbitrary compileValueAccountsArbitrary)
next
  case (6 pkId restParams pk restArgs fs cont)
  then show ?case by (cases "compileFParty pk fs", auto simp add: stateModificationOrderArbitrary
        compilePubKeyAccountsArbitrary)
next
  case ("7_1")
  then show ?case by auto
next
  case ("7_2")
  then show ?case by auto
next
  case ("7_3")
  then show ?case by auto
next
  case ("7_4")
  then show ?case by auto
next
  case ("7_5")
  then show ?case by auto
next
  case ("7_6")
  then show ?case by auto
next
  case ("7_7")
  then show ?case by auto
next
  case ("7_8")
  then show ?case by auto
qed
```

```
lemma compileActionAccountsArbitrary:
"∀newAccs compiledAct . compileAction act state = Some compiledAct ⟶
  compileAction act (state⦇accounts := newAccs⦈) = Some compiledAct"
proof (cases act)
  case (Deposit x11 x12 x13 x14)
  then show ?thesis apply (cases "compileFParty x11 state", auto)
    apply (cases "compileFParty x12 state", auto)
    apply (cases "compileFValue x14 state", auto)
    by (simp add: stateModificationOrderArbitrary compileValueAccountsArbitrary
         compilePubKeyAccountsArbitrary)
next
  case (Choice x21 x22)
  then show ?thesis by (auto split: option.split simp add: compileChoiceIdAccountsArbitrary)
next
  case (Notify x3)
  then show ?thesis apply (cases "compileFObservation x3 state", auto)
    by (simp add: stateModificationOrderArbitrary compileValueAccountsArbitrary
         compilePubKeyAccountsArbitrary)
qed

lemma compileAccountsArbitrary:
"(∀newAccs compiledContract . compile contract ctx state = Some compiledContract ⟶
  compile contract ctx (state⦇accounts := newAccs⦈) = Some compiledContract)"
"(∀newAccs compiledCases . compileCases cases ctx state = Some compiledCases ⟶
  compileCases cases ctx (state⦇accounts := newAccs⦈) = Some compiledCases)"
proof (induction rule: compile_compileCases.induct)
  case (1 ctx state)
  then show ?case by auto
next
  case (2 accId payee tok val cont ctx state)
  then show ?case apply (cases "compile cont ctx state", auto)
    apply (cases "compileFParty accId state", auto)
    apply (cases "compileFPayee payee state", auto)
    apply (cases "compileFValue val state", auto)
    by (simp add: compilePayeeAccountsArbitrary compilePubKeyAccountsArbitrary
         compileValueAccountsArbitrary(1))
next
  case (3 obs trueCont falseCont ctx state)
  then show ?case apply (cases "compile trueCont ctx state" "compile falseCont ctx state" rule:
       option.exhaust[case_product option.exhaust], auto)
    apply (cases "compileFObservation obs state", auto)
    by (simp add: compileValueAccountsArbitrary(2))
next
```

```
  case (4 cases t tCont ctx state)
  then show ?case by (cases "compile tCont ctx state" "compileCases cases ctx state" rule: option
       .exhaust[case_product option.exhaust], auto)
next
  case (5 valId val cont ctx state)
  then show ?case apply (cases "compile cont (ValueAbstraction valId # ctx) (state⦇FState.
       boundValues := MList.insert valId 0 (FState.boundValues state)⦈)", auto)
    apply (cases "compileFValue val state", auto)
    by (simp add: stateModificationOrderArbitrary compileValueAccountsArbitrary)
next
  case (6 obsId obs cont ctx state)
  then show ?case apply (cases "compile cont (ObservationAbstraction obsId # ctx) (state⦇FState.
       boundValues := MList.insert obsId 0 (FState.boundValues state)⦈)", auto)
    apply (cases "compileFObservation obs state", auto)
    by (simp add: stateModificationOrderArbitrary compileValueAccountsArbitrary)
next
  case (7 pkId pk cont ctx state)
  then show ?case apply (cases "compileFParty pk state", auto)
    by (simp add: stateModificationOrderArbitrary compilePubKeyAccountsArbitrary)
next
  case (8 obs cont ctx state)
  then show ?case apply (cases "compile cont ctx state", auto)
    apply (cases "compileFObservation obs state", auto)
    by (simp add: stateModificationOrderArbitrary compileValueAccountsArbitrary)
next
  case (9 cid params boundCon cont ctx state)
  then show ?case by auto
next
  case (10 cid args ctx state)
  then show ?case apply auto
    subgoal premises ps proof (cases "lookupContractIdAbsInformation ctx cid")
      case None
      then show ?thesis using ps by auto
    next
      case (Some paramsBodyInnerCtx)
      then show ?thesis using ps apply auto
        subgoal premises ps proof (cases paramsBodyInnerCtx)
          case (fields params bodyCont innerCtx)
          then show ?thesis using ps apply auto
            subgoal premises ps proof (cases "argumentsToCompilerState params args state")
              case None
              then show ?thesis using ps by auto
            next
              case (Some argCtx)
```

```
              then show ?thesis using ps apply auto
                apply (cases "compile bodyCont (paramsToFContext params @ innerCtx) argCtx", auto
                     )
                by (simp add: argumentsToContextAccountsArbitrary
                     compileArgumentsAccountsArbitrary)
            qed
            done
        qed
        done
    qed
    done
next
  case (11 a c rest ctx state)
  then show ?case apply auto
    apply (cases "compile c ctx state", auto)
    apply (cases "compileCases rest ctx state", auto)
    apply (cases "compileAction a state", auto)
    by (simp add: stateModificationOrderArbitrary compileActionAccountsArbitrary)
next
  case (12 ctx state)
  then show ?case by auto
qed

lemma compilePubKeyOnlyBoundPubKeysMatter:
"∀compiledPk . compileFParty pk state = Some compiledPk ⟶
  compileFParty pk (newState⦇boundPubKeys := (boundPubKeys state)⦈) = Some compiledPk"
  by (cases pk, auto)

lemma compilePayeeOnlyBoundPubKeysMatter:
"∀compiledPk . compileFPayee payee state = Some compiledPk ⟶
  compileFPayee payee (newState⦇boundPubKeys := (boundPubKeys state)⦈) = Some compiledPk"
proof (cases payee)
  case (Account x1)
  then show ?thesis by (cases x1, auto)
next
  case (Party x2)
  then show ?thesis by (cases x2, auto)
qed

lemma compileChoiceIdOnlyBoundPubKeysMatter:
"∀compiledChoiceId . compileFChoiceId choiceId state = Some compiledChoiceId ⟶
  compileFChoiceId choiceId (newState⦇boundPubKeys := (boundPubKeys state)⦈) = Some
       compiledChoiceId"
  by (cases choiceId, auto split: option.split simp add: compilePubKeyOnlyBoundPubKeysMatter)
```

```
lemma compileValueOnlyBoundPubKeysMatter:
"∀compiledVal newVals newState. compileFValue val state = Some compiledVal ⟶
  compileFValue val (newState⦇boundPubKeys := (boundPubKeys state)⦈) = Some compiledVal"
"∀compiledObs newVals newState. compileFObservation obs state = Some compiledObs ⟶
  compileFObservation obs (newState⦇boundPubKeys := (boundPubKeys state)⦈) = Some compiledObs"
proof (induction rule: compileFValue_compileFObservation.induct)
  case (1 fPubKey token s)
  then show ?case apply auto
    by (cases "compileFParty fPubKey s", auto simp add: compilePubKeyOnlyBoundPubKeysMatter)
next
  case (2 v s)
  then show ?case by auto
next
  case (3 v s)
  then show ?case by (cases "compileFValue v s", auto)
next
  case (4 lhs rhs s)
  then show ?case by (cases "compileFValue lhs s" "compileFValue rhs s" rule: option.exhaust[
        case_product option.exhaust], auto)
next
  case (5 lhs rhs s)
  then show ?case by (cases "compileFValue lhs s" "compileFValue rhs s" rule: option.exhaust[
        case_product option.exhaust], auto)
next
  case (6 lhs rhs s)
  then show ?case by (cases "compileFValue lhs s" "compileFValue rhs s" rule: option.exhaust[
        case_product option.exhaust], auto)
next
  case (7 lhs rhs s)
  then show ?case by (cases "compileFValue lhs s" "compileFValue rhs s" rule: option.exhaust[
        case_product option.exhaust], auto)
next
  case (8 choiceId s)
  then show ?case by (auto split: option.split simp add: compileChoiceIdOnlyBoundPubKeysMatter)
next
  case (9 s)
  then show ?case by auto
next
  case (10 s)
  then show ?case by auto
next
  case (11 valId s)
  then show ?case by auto
```

```
next
  case (12 obs trueVal falseVal s)
  then show ?case by (cases "compileFObservation obs s" "compileFValue trueVal s" "compileFValue
        falseVal s" rule: option.exhaust[case_product option.exhaust[case_product option.exhaust
        ]], auto)
next
  case (13 lhs rhs s)
  then show ?case by (cases "compileFObservation lhs s" "compileFObservation rhs s" rule: option.
        exhaust[case_product option.exhaust], auto)
next
  case (14 lhs rhs s)
  then show ?case by (cases "compileFObservation lhs s" "compileFObservation rhs s" rule: option.
        exhaust[case_product option.exhaust], auto)
next
  case (15 obs s)
  then show ?case by (cases "compileFObservation obs s", auto)
next
  case (16 lhs rhs s)
  then show ?case by (cases "compileFValue lhs s" "compileFValue rhs s" rule: option.exhaust[
        case_product option.exhaust], auto)
next
  case (17 lhs rhs s)
  then show ?case by (cases "compileFValue lhs s" "compileFValue rhs s" rule: option.exhaust[
        case_product option.exhaust], auto)
next
  case (18 lhs rhs s)
  then show ?case by (cases "compileFValue lhs s" "compileFValue rhs s" rule: option.exhaust[
        case_product option.exhaust], auto)
next
  case (19 lhs rhs s)
  then show ?case by (cases "compileFValue lhs s" "compileFValue rhs s" rule: option.exhaust[
        case_product option.exhaust], auto)
next
  case (20 lhs rhs s)
  then show ?case by (cases "compileFValue lhs s" "compileFValue rhs s" rule: option.exhaust[
        case_product option.exhaust], auto)
next
  case (21 obsId s)
  then show ?case by auto
next
  case (22 choiceId s)
  then show ?case by (auto split: option.split simp add: compileChoiceIdOnlyBoundPubKeysMatter)
next
  case (23 s)
```

```
  then show ?case by auto
next
  case (24 s)
  then show ?case by auto
qed

lemma argumentsToContextOnlyBoundPubKeysMatter:
"(∀newVals argState newState . ∃modifiedState . argumentsToCompilerState params args state = Some
      argState ⟶
  argumentsToCompilerState params args (newState⦇boundPubKeys := (boundPubKeys state)⦈) = Some
       modifiedState)"
proof (induction rule: argumentsToCompilerState.induct)
  case (1 fs)
  then show ?case by auto
next
  case (2 v va fs)
  then show ?case by auto
next
  case (3 v va fs)
  then show ?case by auto
next
  case (4 pkId restParams pk restArgs fs)
  then show ?case apply auto
    apply (cases "compileFParty pk fs", auto)
    by (split option.split, auto simp add: stateModificationOrderArbitrary
         compilePubKeyOnlyBoundPubKeysMatter option.case_eq_if)
next
  case (5 valId restParams val restArgs ctx)
  then show ?case apply auto
    by (metis stateModificationOrderArbitrary(3))
next
  case (6 obsId restParams obs restArgs ctx)
  then show ?case apply auto
    by (metis stateModificationOrderArbitrary(3))
next
  case ("7_1" v restParams va restArgs ctx)
  then show ?case by auto
next
  case ("7_2" v restParams va restArgs ctx)
  then show ?case by auto
next
  case ("7_3" v restParams va restArgs ctx)
  then show ?case by auto
next
```

```
  case ("7_4" v restParams va restArgs ctx)
  then show ?case by auto
next
  case ("7_5" va restParams v restArgs ctx)
  then show ?case by auto
next
  case ("7_6" va restParams v restArgs ctx)
  then show ?case by auto
next
  case ("7_7" va restParams v restArgs ctx)
  then show ?case by auto
next
  case ("7_8" va restParams v restArgs ctx)
  then show ?case by auto
qed

lemma argumentsToContextBoundPubKeysDeterministic:
"(∀newVals argState newState modifiedArgState . argumentsToCompilerState params args state = Some
      argState ⟶
  argumentsToCompilerState params args (newState(|boundPubKeys := (boundPubKeys state)|)) = Some
       modifiedArgState ⟶
  (boundPubKeys argState = boundPubKeys modifiedArgState))"
proof (induction rule: argumentsToCompilerState.induct)
  case (1 fs)
  then show ?case by auto
next
  case (2 v va fs)
  then show ?case by auto
next
  case (3 v va fs)
  then show ?case by auto
next
  case (4 pkId restParams pk restArgs fs)
  then show ?case apply auto
    apply (cases "compileFParty pk fs")
     apply simp
    apply (split option.split_asm)
    apply simp
    by (simp add: compilePubKeyOnlyBoundPubKeysMatter)
next
  case (5 valId restParams val restArgs fs)
  then show ?case apply auto
    by (metis stateModificationOrderArbitrary(3))
next
```

```
  case (6 obsId restParams obs restArgs fs)
  then show ?case apply auto
    by (metis stateModificationOrderArbitrary(3))
next
  case ("7_1" v restParams va restArgs ctx)
  then show ?case by auto
next
  case ("7_2" v restParams va restArgs ctx)
  then show ?case by auto
next
  case ("7_3" v restParams va restArgs ctx)
  then show ?case by auto
next
  case ("7_4" v restParams va restArgs ctx)
  then show ?case by auto
next
  case ("7_5" va restParams v restArgs ctx)
  then show ?case by auto
next
  case ("7_6" va restParams v restArgs ctx)
  then show ?case by auto
next
  case ("7_7" va restParams v restArgs ctx)
  then show ?case by auto
next
  case ("7_8" va restParams v restArgs ctx)
  then show ?case by auto
qed

lemma argumentsToContextBoundPubKeysSameAsEvaluateArguments:
"(∀argState evalState newState modifiedArgState tyCtx . wellTypedState tyCtx state ⟶
  wellTypedArguments tyCtx params args ⟶
  argumentsToCompilerState params args state = Some argState ⟶
  evalFArguments env state params args = Some evalState ⟶
  boundPubKeys argState = boundPubKeys evalState)"
proof (induction rule: argumentsToCompilerState.induct)
  case (1 fs)
  then show ?case by auto
next
  case (2 v va fs)
  then show ?case by auto
next
  case (3 v va fs)
  then show ?case by auto
```

```
next
  case (4 pkId restParams pk restArgs fs)
  then show ?case apply auto
    apply (cases "compileFParty pk fs", auto)
    apply (cases "evalFParty fs pk", auto)
    by (metis compileFParty.elims evalFParty.simps insertPubKeyWellTypedStateIsWellTyped option.
         inject)
next
  case (5 valId restParams val restArgs fs)
  then show ?case apply auto
    subgoal premises ps proof (cases "evalFValue env fs val")
      case None
      then show ?thesis using ps by auto
    next
      case (Some a)
      moreover obtain evaluatedArgs where "evalFArguments env (fs⦇FState.boundValues := MList.
           insert valId a (FState.boundValues fs)⦈) restParams restArgs = Some evaluatedArgs"
           using ps calculation by auto
      moreover obtain tyCtx where "wellTypedState tyCtx fs ∧ wellTypedArguments tyCtx restParams
           restArgs" using ps by auto
      moreover have "wellTypedState tyCtx (fs⦇boundValues := MList.insert valId 0 (boundValues fs
           )⦈)" using ps calculation insertValueWellTypedStateIsWellTyped by auto
      moreover obtain otherEvaluatedArgs where "(evalFArguments env (fs⦇boundValues:=MList.insert
            valId 0 (boundValues fs)⦈) restParams restArgs = Some otherEvaluatedArgs)" using ps
           calculation wellTypedArgumentsExecute by fastforce
      moreover have "(fs⦇boundValues:=MList.insert valId 0 (boundValues fs)⦈) = (fs⦇boundValues:=
           MList.insert valId 0 (boundValues fs)⦈)⦇boundPubKeys := boundPubKeys (fs⦇FState.
           boundValues := MList.insert valId a (FState.boundValues fs)⦈)⦈" using ps by auto
      moreover have "boundPubKeys otherEvaluatedArgs = boundPubKeys evaluatedArgs" using ps
           calculation evalFArguments_OnlyBoundPubKeysChangeBoundPubKeys apply auto
        by (metis calculation(6))
      ultimately show ?thesis using ps by auto
    qed
    done
next
  case (6 obsId restParams obs restArgs fs)
  then show ?case apply auto
    subgoal premises ps proof (cases "evalFObservation env fs obs")
      case None
      then show ?thesis using ps by auto
    next
      case (Some a)
      moreover obtain evaluatedArgs where "evalFArguments env (fs⦇FState.boundValues := MList.
           insert obsId (if a then 1 else 0) (FState.boundValues fs)⦈) restParams restArgs = Some
```

```
            evaluatedArgs" using ps calculation by auto
      moreover obtain tyCtx where "wellTypedState tyCtx fs ∧ wellTypedArguments tyCtx restParams
           restArgs" using ps by auto
      moreover have "wellTypedState tyCtx (fs⦇boundValues := MList.insert obsId 0 (boundValues fs
           )⦈)" using ps calculation insertValueWellTypedStateIsWellTyped by auto
      moreover obtain otherEvaluatedArgs where "(evalFArguments env (fs⦇boundValues:=MList.insert
            obsId 0 (boundValues fs)⦈) restParams restArgs = Some otherEvaluatedArgs)" using ps
           calculation wellTypedArgumentsExecute by fastforce
      moreover have "(fs⦇boundValues:=MList.insert obsId 0 (boundValues fs)⦈) = (fs⦇boundValues:=
           MList.insert obsId 0 (boundValues fs)⦈)⦇boundPubKeys := boundPubKeys (fs⦇FState.
           boundValues := MList.insert obsId (if a then 1 else 0) (FState.boundValues fs)⦈)⦈"
           using ps by auto
      moreover have "boundPubKeys otherEvaluatedArgs = boundPubKeys evaluatedArgs" using ps
           calculation evalFArguments_OnlyBoundPubKeysChangeBoundPubKeys apply auto
        by (metis calculation(6))
      ultimately show ?thesis using ps by auto
    qed
    done
next
  case ("7_1" v restParams va restArgs ctx)
  then show ?case by auto
next
  case ("7_2" v restParams va restArgs ctx)
  then show ?case by auto
next
  case ("7_3" v restParams va restArgs ctx)
  then show ?case by auto
next
  case ("7_4" v restParams va restArgs ctx)
  then show ?case by auto
next
  case ("7_5" va restParams v restArgs ctx)
  then show ?case by auto
next
  case ("7_6" va restParams v restArgs ctx)
  then show ?case by auto
next
  case ("7_7" va restParams v restArgs ctx)
  then show ?case by auto
next
  case ("7_8" va restParams v restArgs ctx)
  then show ?case by auto
qed
```

```
lemma compileArgumentsOnlyBoundPubKeysMatter:
"(∀newState compiledContract . compileArguments params args state bodyCont = Some
     compiledContract ⟶
  compileArguments params args (newState⦇boundPubKeys := (boundPubKeys state)⦈) bodyCont = Some
       compiledContract)"
proof (induction rule: compileArguments.induct)
  case (1 fs cont)
  then show ?case by auto
next
  case (2 v va fs cont)
  then show ?case by auto
next
  case (3 v va fs cont)
  then show ?case by auto
next
  case (4 valId restParams val restArgs fs cont)
  then show ?case apply (cases "compileFValue val fs", auto)
    apply (split option.split_asm, auto)
    apply (split option.split, auto simp add: stateModificationOrderArbitrary
         compileValueOnlyBoundPubKeysMatter)
    apply (split option.split, auto simp add: stateModificationOrderArbitrary
         compileValueOnlyBoundPubKeysMatter)
     apply (metis stateModificationOrderArbitrary(3))
    by (metis option.sel stateModificationOrderArbitrary(3))
next
  case (5 obsId restParams obs restArgs fs cont)
  then show ?case apply (cases "compileFObservation obs fs", auto)
    apply (split option.split_asm, auto)
    apply (split option.split, auto simp add: stateModificationOrderArbitrary
         compileValueOnlyBoundPubKeysMatter)
    apply (split option.split, auto simp add: stateModificationOrderArbitrary
         compileValueOnlyBoundPubKeysMatter)
     apply (metis stateModificationOrderArbitrary(3))
    by (metis option.sel stateModificationOrderArbitrary(3))
next
  case (6 pkId restParams pk restArgs fs cont)
  then show ?case apply auto
    apply (split option.split_asm)
    apply simp
    apply (split option.split)
    using compilePubKeyOnlyBoundPubKeysMatter by fastforce
next
  case ("7_1")
  then show ?case by auto
```

```
next
  case ("7_2")
  then show ?case by auto
next
  case ("7_3")
  then show ?case by auto
next
  case ("7_4")
  then show ?case by auto
next
  case ("7_5")
  then show ?case by auto
next
  case ("7_6")
  then show ?case by auto
next
  case ("7_7")
  then show ?case by auto
next
  case ("7_8")
  then show ?case by auto
qed

lemma compileActionOnlyBoundPubKeysMatter:
"∀compiledAction newState . compileAction action state = Some compiledAction ⟶
  compileAction action (newState⦇FState.boundPubKeys := (boundPubKeys state)⦈) = Some
       compiledAction"
proof (induction rule: compileAction.induct)
  case (1 fromPk toPk token "value" s)
  then show ?case apply (cases "compileFParty fromPk s" "compileFParty toPk s" "compileFValue
       value s" rule: option.exhaust[case_product option.exhaust[case_product option.exhaust]],
       auto)
    by (simp add: compilePubKeyOnlyBoundPubKeysMatter compileValueOnlyBoundPubKeysMatter)
next
  case (2 choiceId bounds s)
  then show ?case by (auto split: option.split simp add: compileChoiceIdOnlyBoundPubKeysMatter)
next
  case (3 observation s)
  then show ?case apply (cases "compileFObservation observation s", auto)
    by (simp add: compilePubKeyOnlyBoundPubKeysMatter compileValueOnlyBoundPubKeysMatter)
qed

lemma compileContractOnlyBoundPubKeysMatter_UseC_Inductive:
"argumentsToCompilerState params args state = Some argCtx ⟹
```

```
        paramsBodyInnerCtx = (params, bodyCont, innerCtx) ⟹
        lookupContractIdAbsInformation ctx cid = Some (params, bodyCont, innerCtx) ⟹
        (argumentsToCompilerState params args state = Some argCtx ⟹
            ∀marloweContract.
               compile bodyCont (paramsToFContext params @ innerCtx) argCtx = Some
                    marloweContract ⟶
               (∀newState. compile bodyCont (paramsToFContext params @ innerCtx) (newState⦇
                    boundPubKeys := boundPubKeys argCtx⦈) = Some marloweContract)) ⟹
        compileArguments params args state compiledBodyCont = Some arbMarloweContract ⟹
        compile bodyCont (paramsToFContext params @ innerCtx) argCtx = Some compiledBodyCont
             ⟹
        argumentsToCompilerState params args (arbNewState⦇boundPubKeys := boundPubKeys state⦈)
             = Some x2 ⟹
        (case compile bodyCont (paramsToFContext params @ innerCtx) x2 of None ⇒ None
         | Some x ⇒ compileArguments params args (arbNewState⦇boundPubKeys := boundPubKeys
              state⦈) x) =
        Some arbMarloweContract"
  apply auto
  subgoal premises ps proof -
    have "boundPubKeys x2 = boundPubKeys argCtx" using ps
         argumentsToContextBoundPubKeysDeterministic by presburger
    moreover have "x2 = x2⦇boundPubKeys := boundPubKeys argCtx⦈" using ps calculation by auto
    moreover have "compile bodyCont (paramsToFContext params @ innerCtx) x2 = compile bodyCont (
         paramsToFContext params @ innerCtx) (x2⦇boundPubKeys := boundPubKeys argCtx⦈)" using ps
         calculation by (metis calculation(2))
    moreover have "compile bodyCont (paramsToFContext params @ innerCtx) x2 = Some
         compiledBodyCont" using ps by (simp add: calculation(3))
    moreover have "compileArguments params args (arbNewState⦇boundPubKeys := boundPubKeys state⦈)
          compiledBodyCont = Some arbMarloweContract" by (metis ps(5)
         compileArgumentsOnlyBoundPubKeysMatter)
    ultimately show ?thesis by (metis option.simps(5))
  qed
  done

lemma compileContractOnlyBoundPubKeysMatter:
"∀marloweContract newState . compile fContract ctx state = Some marloweContract ⟶
  compile fContract ctx (newState⦇FState.boundPubKeys := (boundPubKeys state)⦈) = Some
       marloweContract"
"∀marloweCases newState . compileCases fCases ctx state = Some marloweCases ⟶
  compileCases fCases ctx (newState⦇FState.boundPubKeys := (boundPubKeys state)⦈) = Some
       marloweCases"
proof (induction rule: compile_compileCases.induct)
  case (1 ctx state)
  then show ?case by auto
```

```
next
  case (2 accId payee tok val cont ctx state)
  then show ?case apply (cases "compile cont ctx state", auto)
    apply (cases "compileFParty accId state", auto)
    apply (cases "compileFPayee payee state", auto)
    apply (cases "compileFValue val state", auto)
    by (simp add: compilePayeeOnlyBoundPubKeysMatter compilePubKeyOnlyBoundPubKeysMatter
         compileValueOnlyBoundPubKeysMatter(1))
next
  case (3 obs trueCont falseCont ctx state)
  then show ?case apply (cases "compile trueCont ctx state" "compile falseCont ctx state" rule:
       option.exhaust[case_product option.exhaust], auto)
    by (cases "compileFObservation obs state", auto simp add: compileValueOnlyBoundPubKeysMatter
         (2))
next
  case (4 cases t tCont ctx state)
  then show ?case by (cases "compile tCont ctx state" "compileCases cases ctx state" rule: option
       .exhaust[case_product option.exhaust], auto)
next
  case (5 valId val cont ctx state)
  then show ?case apply (cases "compile cont (ValueAbstraction valId # ctx) (state⦇FState.
       boundValues := MList.insert valId 0 (FState.boundValues state)⦈)", auto)
    apply (cases "compileFValue val state", auto)
    by (metis option.simps(5) stateModificationOrderArbitrary(3)
         compileValueOnlyBoundPubKeysMatter(1))
next
  case (6 obsId obs cont ctx state)
  then show ?case apply (cases "compile cont (ObservationAbstraction obsId # ctx) (state⦇FState.
       boundValues := MList.insert obsId 0 (FState.boundValues state)⦈)", auto)
    apply (cases "compileFObservation obs state", auto)
    by (metis option.simps(5) stateModificationOrderArbitrary(3)
         compileValueOnlyBoundPubKeysMatter(2))
next
  case (7 pkId pk cont ctx state)
  then show ?case apply (simp (no_asm_simp) only: compile.simps)
    subgoal premises ps proof (cases "compileFParty pk state")
      case None
      then show ?thesis by auto
    next
      case (Some pkCompiled)
      then show ?thesis using ps apply (cases "compile cont (PubKeyAbstraction pkId # ctx) (state
           ⦇boundPubKeys := MList.insert pkId pkCompiled (boundPubKeys state)⦈)", simp (
           no_asm_simp))
        using compilePubKeyOnlyBoundPubKeysMatter stateModificationOrderArbitrary by fastforce
```

```
    qed
    done
next
  case (8 obs cont ctx state)
  then show ?case apply auto
    apply (cases "compile cont ctx state", auto)
    by (metis compileValueOnlyBoundPubKeysMatter(2) option.discI option.disc_eq_case(1) option.
         split_sel)
next
  case (9 cid params boundCon cont ctx state)
  then show ?case by auto
next
  case (10 cid args ctx state)
  then show ?case apply auto
    subgoal premises ps proof (cases "lookupContractIdAbsInformation ctx cid")
      case None
      then show ?thesis using ps by auto
    next
      case (Some paramsBodyInnerCtx)
      then show ?thesis using ps apply auto
        subgoal premises ps proof (cases paramsBodyInnerCtx)
          case (fields params bodyCont innerCtx)
          then show ?thesis using ps apply auto
            subgoal premises ps proof (cases "argumentsToCompilerState params args state")
              case None
              then show ?thesis using ps by auto
            next
              case (Some argCtx)
              moreover obtain compiledBodyCont where "compile bodyCont (paramsToFContext params @
                    innerCtx) argCtx = Some compiledBodyCont" using calculation ps by fastforce
              ultimately show ?thesis using ps apply auto
                apply (cases "compile bodyCont (paramsToFContext params @ innerCtx) argCtx", auto
                     )
                apply (split option.split, auto)
                using argumentsToContextOnlyBoundPubKeysMatter
                     argumentsToContextBoundPubKeysDeterministic apply (auto)[1]
                using compileContractOnlyBoundPubKeysMatter_UseC_Inductive by fastforce
            qed
            done
        qed
        done
    qed
    done
next
```

```
  case (11 a c rest ctx state)
  then show ?case apply (cases "compile c ctx state" "compileCases rest ctx state" "compileAction
        a state" rule: option.exhaust[case_product option.exhaust[case_product option.exhaust]],
       auto)
    by (simp add: compileActionOnlyBoundPubKeysMatter)
next
  case (12 ctx state)
  then show ?case by auto
qed

(* Problem: What to do about shadowed ids of different types. Should we say ill typed in well
     typed?*)
fun fStateToMState :: "FState ⇒ State" where
"fStateToMState fs = ⦇State.accounts = accounts fs,
  State.choices = choices fs,
  State.boundValues = boundValues fs,
  State.minTime = minTime fs⦈"

lemma fStateToMStatePreservationOfBoundValues:
"boundValues state = State.boundValues (fStateToMState state)"
  by auto

lemma preservationOfPubKeyEvaluation:
"(∀tyCtx env res. wellTypedParty tyCtx fPubKey ⟶
  wellTypedState tyCtx faustusState ⟶
  evalFParty faustusState fPubKey = compileFParty fPubKey faustusState)"
  by (cases fPubKey, auto)

lemma preservationOfPayeeEvaluation:
"(∀tyCtx env res. wellTypedPayee tyCtx faustusPayee ⟶
  wellTypedState tyCtx faustusState ⟶
  evalFPayee faustusState faustusPayee = compileFPayee faustusPayee faustusState)"
  by (cases faustusPayee, auto split: option.split simp add: preservationOfPubKeyEvaluation)

lemma preservationOfChoiceIdEvaluation:
"(∀tyCtx env res. wellTypedChoiceId tyCtx faustusChoiceId ⟶
  wellTypedState tyCtx faustusState ⟶
  evalFChoiceId faustusState faustusChoiceId = compileFChoiceId faustusChoiceId faustusState)"
  by (cases faustusChoiceId, auto split: option.split simp add: preservationOfPubKeyEvaluation)

lemma preservationOfValueEvaluation:
"(∀tyCtx res. wellTypedValue tyCtx fValue ⟶
  wellTypedState tyCtx faustusState ⟶
  evalFValue env faustusState fValue = Some res ⟶
```

```
  evalValue env (fStateToMState faustusState) (the (compileFValue fValue faustusState)) = res)"
"(∀tyCtx res. wellTypedObservation tyCtx fObservation ⟶
  wellTypedState tyCtx faustusState ⟶
  evalFObservation env faustusState fObservation = Some res ⟶
  evalObservation env (fStateToMState faustusState) (the (compileFObservation fObservation
       faustusState)) = res)"
proof (induction rule: evalFValue_evalFObservation.induct)
  case (1 env state fPubKey token)
  then show ?case apply auto
    by (smt wellTypedPartyCompiles wellTypedPartyEvaluates FParty.exhaust State.select_convs(1)
         compileFParty.simps evalFParty.simps evalValue.simps(1) findWithDefault.simps option.
         simps(5) option.the_def)
next
  case (2 env state integer)
  then show ?case by auto
next
  case (3 env state val)
  then show ?case apply auto
    by (metis (no_types, lifting) evalValue.simps(3) option.case_eq_if option.collapse option.
         discI option.sel wellTypedValueObservationCompiles(1))
next
  case (4 env state lhs rhs)
  then show ?case apply auto
    by (smt evalValue.simps(4) not_None_eq option.case_eq_if option.exhaust_sel option.inject
         wellTypedValueObservationCompiles(1) wellTypedValueObservationEvaluates(1))
next
  case (5 env state lhs rhs)
  then show ?case apply auto
  by (smt evalValue.simps(5) is_none_code(1) is_none_code(2) option.case_eq_if option.sel option.
       split_sel wellTypedValueObservationCompiles(1))
next
  case (6 env state lhs rhs)
  then show ?case apply auto
    by (smt evalValue.simps(6) is_none_code(2) option.case_eq_if option.discI option.sel option.
         split_sel wellTypedValueObservationCompiles(1))
next
  case (7 env state lhs rhs)
  then show ?case apply auto
    by (smt evalValue.simps is_none_code(2) option.case_eq_if option.discI option.sel option.
         split_sel wellTypedValueObservationCompiles)
next
  case (8 env state choId)
  then show ?case by (auto split: option.split simp add: preservationOfChoiceIdEvaluation)
next
```

```
  case (9 env state)
  then show ?case by auto
next
  case (10 env state)
  then show ?case by auto
next
  case (11 env state valId)
  then show ?case by auto
next
  case (12 env state cond thn els)
  then show ?case apply auto
    by (smt evalValue.simps(12) option.case_eq_if option.collapse option.discI option.sel
         wellTypedValueObservationCompiles(1) wellTypedValueObservationCompiles(2))
next
  case (13 env state lhs rhs)
  then show ?case apply auto
     apply (smt evalObservation.simps(1) option.case_eq_if option.collapse option.discI option.
          sel wellTypedValueObservationCompiles(2))
    by (smt evalObservation.simps(1) is_none_code(2) option.case_eq_if option.sel option.
         split_sel)
next
  case (14 env state lhs rhs)
  then show ?case apply auto
     apply (smt evalObservation.simps(2) option.case_eq_if option.collapse option.discI option.
          sel)
    by (smt evalObservation.simps(2) option.case_eq_if option.distinct(1) option.exhaust option.
         sel option.simps(5) wellTypedValueObservationCompiles(2))
next
  case (15 env state subObs)
  then show ?case apply auto
     apply (metis (no_types, lifting) evalObservation.simps(3) is_none_code(2) is_none_simps(1)
          option.case_eq_if option.collapse option.sel wellTypedValueObservationCompiles(2))
    by (metis evalObservation.simps(3) option.case_eq_if option.collapse option.discI option.sel
         wellTypedValueObservationCompiles(2))
next
  case (16 env state choId)
  then show ?case by (auto split: option.split simp add: preservationOfChoiceIdEvaluation)
next
  case (17 env state lhs rhs)
  then show ?case apply auto
     apply (smt evalObservation.simps(5) is_none_code(2) is_none_simps(1) option.sel option.simps
          (5) option.split_sel wellTypedValueObservationCompiles(1))
    by (smt evalObservation.simps(5) is_none_code(2) is_none_simps(1) option.sel option.simps(5)
         option.split_sel wellTypedValueObservationCompiles(1))
```

```
next
  case (18 env state lhs rhs)
  then show ?case apply auto
     apply (smt evalObservation.simps(6) is_none_code(2) is_none_simps(1) option.sel option.simps
          (5) option.split_sel wellTypedValueObservationCompiles(1))
    by (smt evalObservation.simps(6) is_none_code(2) is_none_simps(1) option.sel option.simps(5)
         option.split_sel wellTypedValueObservationCompiles(1))
next
  case (19 env state lhs rhs)
  then show ?case apply auto
    apply (smt evalObservation.simps(7) is_none_code(2) is_none_simps(1) option.sel option.simps
         (5) option.split_sel wellTypedValueObservationCompiles(1))
    by (smt evalObservation.simps(7) is_none_code(2) is_none_simps(1) option.sel option.simps(5)
         option.split_sel wellTypedValueObservationCompiles(1))
next
  case (20 env state lhs rhs)
  then show ?case apply auto
     apply (smt evalObservation.simps(8) is_none_code(2) is_none_simps(1) option.sel option.simps
          (5) option.split_sel wellTypedValueObservationCompiles(1))
    by (smt evalObservation.simps(8) is_none_code(2) is_none_simps(1) option.sel option.simps(5)
         option.split_sel wellTypedValueObservationCompiles(1))
next
  case (21 env state lhs rhs)
  then show ?case apply auto
     apply (smt evalObservation.simps(9) is_none_code(2) is_none_simps(1) option.sel option.simps
          (5) option.split_sel wellTypedValueObservationCompiles(1))
    by (smt evalObservation.simps(9) is_none_code(2) is_none_simps(1) option.sel option.simps(5)
         option.split_sel wellTypedValueObservationCompiles(1))
next
  case (22 env state obsId)
  then show ?case apply auto
     apply (smt Collect_empty_eq None_eq_map_option_iff Option.is_none_def all_not_in_conv bind.
          bind_lunit bind_eq_None_conv bind_eq_Some_conv case_map_option case_optionE
          combine_options_def combine_options_simps(3) iszero_0 iszero_def map_option_eq_Some
          mem_Collect_eq not_None_eq not_Some_eq not_iszero_1 odd_nonzero option.case_distrib
          option.case_eq_if option.discI option.disc_eq_case(2) option.exhaust option.exhaust_sel
           option.inject option.map_ident option.map_sel option.sel option.simps(15) option.simps
          (4) option.simps(5) option.split_sel_asm option.the_def ospec split_option_all
          split_option_ex zero_neq_one)
    by (smt option.case_eq_if option.exhaust_sel option.simps(5) option.the_def)
next
  case (23 env state)
  then show ?case by auto
next
```

```
  case (24 env state)
  then show ?case by auto
qed

lemma preservationOfValueEvaluation_MarloweImpliesFaustus:
"(∀tyCtx res. wellTypedValue tyCtx fValue ⟶
  wellTypedState tyCtx faustusState ⟶
  evalValue env (fStateToMState faustusState) (the (compileFValue fValue faustusState)) = res ⟶
  evalFValue env faustusState fValue = Some res)"
"(∀tyCtx res. wellTypedObservation tyCtx fObservation ⟶
  wellTypedState tyCtx faustusState ⟶
  evalObservation env (fStateToMState faustusState) (the (compileFObservation fObservation
       faustusState)) = res ⟶
  evalFObservation env faustusState fObservation = Some res)"
proof (induction rule: evalFValue_evalFObservation.induct)
  case (1 env state fPubKey token)
  then show ?case apply auto
    apply (cases "evalFParty state fPubKey", auto)
     apply (simp add: wellTypedPartyEvaluates)
    apply (cases "compileFParty fPubKey state", auto)
    using wellTypedPartyCompiles apply fastforce
    by (metis FParty.exhaust compileFParty.simps evalFParty.simps option.simps(5))
next
  case (2 env state integer)
  then show ?case by auto
next
  case (3 env state val)
  then show ?case apply auto
    apply (cases "compileFValue val state", auto)
    using wellTypedValueObservationCompiles by fastforce
next
  case (4 env state lhs rhs)
  then show ?case apply auto
    apply (cases "compileFValue lhs state", auto)
    using wellTypedValueObservationCompiles apply fastforce
    apply (cases "compileFValue rhs state", auto)
    using wellTypedValueObservationCompiles by fastforce
next
  case (5 env state lhs rhs)
  then show ?case apply auto
    apply (cases "compileFValue lhs state", auto)
    using wellTypedValueObservationCompiles apply fastforce
    apply (cases "compileFValue rhs state", auto)
    using wellTypedValueObservationCompiles by fastforce
```

```
next
  case (6 env state lhs rhs)
  then show ?case apply auto
    apply (cases "compileFValue lhs state", auto)
    using wellTypedValueObservationCompiles apply fastforce
    apply (cases "compileFValue rhs state", auto)
    using wellTypedValueObservationCompiles by fastforce
next
  case (7 env state lhs rhs)
  then show ?case apply (cases "compileFValue lhs state" "compileFValue rhs state" rule: option.
       exhaust[case_product option.exhaust], auto)
    using wellTypedValueObservationCompiles apply fastforce
            apply (cases "compileFValue rhs state", auto)
            apply (metis not_Some_eq wellTypedValueObservationCompiles(1))
           apply (metis option.discI wellTypedValueObservationCompiles(1))
          apply (metis option.discI wellTypedValueObservationCompiles(1))
         apply (metis option.discI wellTypedValueObservationCompiles(1))
        apply (metis option.discI wellTypedValueObservationCompiles(1))
       apply (metis option.discI wellTypedValueObservationCompiles(1))
      apply (metis option.discI wellTypedValueObservationCompiles(1))
     apply (metis option.discI wellTypedValueObservationCompiles(1))
    by (metis option.discI wellTypedValueObservationCompiles(1))
next
  case (8 env state choId)
  then show ?case apply (auto split: option.split simp add: preservationOfChoiceIdEvaluation)
    using wellTypedChoiceIdCompiles by fastforce
next
  case (9 env state)
  then show ?case by auto
next
  case (10 env state)
  then show ?case by auto
next
  case (11 env state valId)
  then show ?case apply auto
    apply (cases "lookup valId (FState.boundValues state)", auto)
    using wellTypedValueObservationEvaluates(1) by fastforce
next
  case (12 env state cond thn els)
  then show ?case apply auto
     apply (smt evalValue.simps(12) option.sel option.simps(5) wellTypedValueObservationCompiles
          (1) wellTypedValueObservationCompiles(2))
```

```
    by (smt evalValue.simps(12) option.sel option.simps(5) wellTypedValueObservationCompiles(1)
         wellTypedValueObservationCompiles(2))
next
  case (13 env state lhs rhs)
  then show ?case apply auto
      apply (smt evalObservation.simps(1) option.sel option.simps(5)
           wellTypedValueObservationCompiles(1) wellTypedValueObservationCompiles(2))
     apply (smt evalObservation.simps(1) option.sel option.simps(5)
          wellTypedValueObservationCompiles(1) wellTypedValueObservationCompiles(2))
    by (smt evalObservation.simps(1) option.sel option.simps(5) wellTypedValueObservationCompiles
         (1) wellTypedValueObservationCompiles(2))
next
  case (14 env state lhs rhs)
  then show ?case apply auto
      apply (smt evalObservation.simps(2) option.sel option.simps(5)
           wellTypedValueObservationCompiles(1) wellTypedValueObservationCompiles(2))
     apply (smt evalObservation.simps(2) option.sel option.simps(5)
          wellTypedValueObservationCompiles(1) wellTypedValueObservationCompiles(2))
    by (smt evalObservation.simps(2) option.sel option.simps(5) wellTypedValueObservationCompiles
         (1) wellTypedValueObservationCompiles(2))
next
  case (15 env state subObs)
  then show ?case apply auto
     apply (smt evalObservation.simps(3) option.sel option.simps(5)
          wellTypedValueObservationCompiles(1) wellTypedValueObservationCompiles(2))
    by (smt evalObservation.simps(3) option.sel option.simps(5) wellTypedValueObservationCompiles
         (1) wellTypedValueObservationCompiles(2))
next
  case (16 env state choId)
  then show ?case apply (auto split: option.split simp add: preservationOfChoiceIdEvaluation)
    using wellTypedChoiceIdCompiles by fastforce
next
  case (17 env state lhs rhs)
  then show ?case apply auto
     apply (smt evalObservation.simps(5) option.sel option.simps(5)
          wellTypedValueObservationCompiles(1) wellTypedValueObservationCompiles(2))
    by (smt evalObservation.simps(5) option.sel option.simps(5) wellTypedValueObservationCompiles
         (1) wellTypedValueObservationCompiles(2))
next
  case (18 env state lhs rhs)
  then show ?case apply auto
     apply (smt evalObservation.simps(6) option.sel option.simps(5)
          wellTypedValueObservationCompiles(1) wellTypedValueObservationCompiles(2))
```

```
    by (smt evalObservation.simps(6) option.sel option.simps(5) wellTypedValueObservationCompiles
         (1) wellTypedValueObservationCompiles(2))
next
  case (19 env state lhs rhs)
  then show ?case apply auto
     apply (smt evalObservation.simps(7) option.sel option.simps(5)
          wellTypedValueObservationCompiles(1) wellTypedValueObservationCompiles(2))
    by (smt evalObservation.simps(7) option.sel option.simps(5) wellTypedValueObservationCompiles
         (1) wellTypedValueObservationCompiles(2))
next
  case (20 env state lhs rhs)
  then show ?case apply auto
     apply (smt evalObservation.simps(8) option.sel option.simps(5)
          wellTypedValueObservationCompiles(1) wellTypedValueObservationCompiles(2))
    by (smt evalObservation.simps(8) option.sel option.simps(5) wellTypedValueObservationCompiles
         (1) wellTypedValueObservationCompiles(2))
next
  case (21 env state lhs rhs)
  then show ?case apply auto
     apply (smt evalObservation.simps(9) option.sel option.simps(5)
          wellTypedValueObservationCompiles(1) wellTypedValueObservationCompiles(2))
    by (smt evalObservation.simps(9) option.sel option.simps(5) wellTypedValueObservationCompiles
         (1) wellTypedValueObservationCompiles(2))
next
  case (22 env state obsId)
  then show ?case apply auto
    apply (cases "lookup obsId (FState.boundValues state)", auto)
    using wellTypedValueObservationEvaluates(2) by fastforce
next
  case (23 env state)
  then show ?case by auto
next
  case (24 env state)
  then show ?case by auto
qed

lemma preservationOfArgumentEvaluation_ValueParameter_Induct:
"(\<And>arbEvalVal. evaluatedVal = arbEvalVal ⟹
               ∀newState compiledBody compiledFullContract warnings payments tyCtx.
                  wellTypedArguments tyCtx restParams restArgs ⟶
                  wellTypedState tyCtx (s⦇FState.boundValues := MList.insert vid arbEvalVal (
                       FState.boundValues s)⦈) ⟶
                  evalFArguments e (s⦇FState.boundValues := MList.insert vid arbEvalVal (FState.
                       boundValues s)⦈) restParams restArgs =
```

```
                Some newState ⟶
                compileArguments restParams restArgs (s⦇FState.boundValues := MList.insert vid
                     arbEvalVal (FState.boundValues s)⦈) compiledBody =
                Some compiledFullContract ⟶
                (∃newMWarnings.
                    (compiledFullContract,
                     ⦇State.accounts = FState.accounts s, choices = FState.choices s,
                        boundValues = MList.insert vid arbEvalVal (FState.boundValues s),
                             minTime = FState.minTime s⦈,
                     e, warnings, payments) →*
                    (compiledBody,
                     ⦇State.accounts = FState.accounts newState, choices = FState.choices
                          newState,
                        boundValues = FState.boundValues newState, minTime = FState.minTime
                             newState⦈,
                     e, newMWarnings, payments))) ⟹
     wellTypedValue tyCtx val ⟹
     wellTypedArguments tyCtx restParams restArgs ⟹
     wellTypedState tyCtx s ⟹
     evalFArguments e (s⦇FState.boundValues := MList.insert vid evaluatedVal (FState.
          boundValues s)⦈) restParams restArgs = Some newState ⟹
     (case compileArguments restParams restArgs (s⦇FState.boundValues := MList.insert vid 0 (
          FState.boundValues s)⦈) compiledBody of
      None ⇒ None | Some compiledRest ⇒ Some (Contract.Let vid compiledVal compiledRest)) =
     Some compiledFullContract ⟹
     evalFValue e s val = Some evaluatedVal ⟹
     compileFValue val s = Some compiledVal ⟹
     ∃newMWarnings.
        (compiledFullContract,
         ⦇State.accounts = FState.accounts s, choices = FState.choices s, boundValues = FState.
              boundValues s,
            minTime = FState.minTime s⦈,
         e, warnings, payments) →*
        (compiledBody,
         ⦇State.accounts = FState.accounts newState, choices = FState.choices newState,
              boundValues = FState.boundValues newState,
            minTime = FState.minTime newState⦈,
         e, newMWarnings, payments)"
subgoal premises ps proof-
  have "evalValue e (fStateToMState s) compiledVal = evaluatedVal" using ps
       preservationOfValueEvaluation(1) by fastforce
  moreover obtain tyCtx where "wellTypedValue tyCtx val ∧ wellTypedArguments tyCtx restParams
       restArgs ∧ wellTypedState tyCtx s" using ps by auto
```

```
moreover have "wellTypedState tyCtx (s⦇FState.boundValues := MList.insert vid evaluatedVal (
     FState.boundValues s)⦈)" using ps calculation insertValueWellTypedStateIsWellTyped by
     auto
moreover have "wellTypedState tyCtx (s⦇FState.boundValues := MList.insert vid 0 (FState.
     boundValues s)⦈)" using ps calculation insertValueWellTypedStateIsWellTyped by auto
moreover obtain newState where "evalFArguments e (s⦇FState.boundValues := MList.insert vid
     evaluatedVal (FState.boundValues s)⦈) restParams restArgs = Some newState" using ps by
     auto
moreover obtain compiledRest where "compileArguments restParams restArgs (s⦇FState.
     boundValues := MList.insert vid 0 (FState.boundValues s)⦈) compiledBody = Some
     compiledRest" using ps calculation wellTypedArgumentsCompiles apply auto
  by blast
moreover have "s⦇FState.boundValues := MList.insert vid evaluatedVal (FState.boundValues s)⦈
     = (s⦇FState.boundValues := MList.insert vid evaluatedVal (FState.boundValues s)⦈)⦇
     boundPubKeys:=(boundPubKeys (s⦇FState.boundValues := MList.insert vid 0 (FState.
     boundValues s)⦈))⦈" by auto
moreover have "compileArguments restParams restArgs (s⦇FState.boundValues := MList.insert vid
      evaluatedVal (FState.boundValues s)⦈) compiledBody = Some compiledRest" using ps
     calculation apply auto
  by (metis calculation(7) compileArgumentsOnlyBoundPubKeysMatter)
moreover have "(∀warnings payments. ∃newMWarnings. (compiledRest, ⦇State.accounts = FState.
     accounts s, choices = FState.choices s, boundValues = MList.insert vid evaluatedVal (
     FState.boundValues s), minTime = FState.minTime s⦈, e, warnings, payments) →* (
     compiledBody, fStateToMState newState, e, newMWarnings, payments))" using ps calculation
      by auto
moreover have "(∀warnings payments. ∃newMWarnings. (SemanticsTypes.Let vid compiledVal
     compiledRest, fStateToMState s, e, warnings, payments) → (compiledRest, (fStateToMState
      s)⦇State.boundValues := MList.insert vid evaluatedVal (State.boundValues (
     fStateToMState s))⦈, e, newMWarnings, payments))" using ps calculation apply (cases "
     lookup vid (State.boundValues (fStateToMState s))")
  using SmallStep.LetNoShadow apply blast
  using SmallStep.LetShadow by blast
moreover have "(fStateToMState s)⦇State.boundValues := MList.insert vid evaluatedVal (State.
     boundValues (fStateToMState s))⦈ = ⦇State.accounts = FState.accounts s, choices = FState
     .choices s, boundValues = MList.insert vid evaluatedVal (FState.boundValues s), minTime
     = FState.minTime s⦈" by auto
moreover have "(∃newMWarnings. (SemanticsTypes.Let vid compiledVal compiledRest,
     fStateToMState s, e, warnings, payments) → (compiledRest, ⦇State.accounts = FState.
     accounts s, choices = FState.choices s, boundValues = MList.insert vid evaluatedVal (
     FState.boundValues s), minTime = FState.minTime s⦈, e, newMWarnings, payments))" using
     calculation by auto
ultimately show ?thesis using ps star.step ps SmallStep.smallStepStarWarningsAreArbitrary
     apply auto
```

```
      by (smt \<open>∃newMWarnings. (Contract.Let vid compiledVal compiledRest, fStateToMState s,
            e, warnings, payments) → (compiledRest, ⦇State.accounts = FState.accounts s, choices
            = FState.choices s, boundValues = MList.insert vid evaluatedVal (FState.boundValues s
           ), minTime = FState.minTime s⦈, e, newMWarnings, payments)\<close> \<open>∀warnings
           payments. ∃newMWarnings. (compiledRest, ⦇State.accounts = FState.accounts s, choices =
            FState.choices s, boundValues = MList.insert vid evaluatedVal (FState.boundValues s),
            minTime = FState.minTime s⦈, e, warnings, payments) →* (compiledBody, fStateToMState
            newState, e, newMWarnings, payments)\<close> fStateToMState.simps star.step)
  qed
  done

lemma preservationOfArgumentEvaluation_ObservationParameter_Induct:
"(\<And>arbEvalObs. evaluatedObs = arbEvalObs ⟹
              ∀newState compiledBody compiledFullContract warnings payments tyCtx.
                 wellTypedArguments tyCtx restParams restArgs ⟶
                 wellTypedState tyCtx (s⦇FState.boundValues := MList.insert obsId (if arbEvalObs
                       then 1 else 0) (FState.boundValues s)⦈) ⟶
                 evalFArguments e (s⦇FState.boundValues := MList.insert obsId (if arbEvalObs
                      then 1 else 0) (FState.boundValues s)⦈)
                  restParams restArgs =
                 Some newState ⟶
                 compileArguments restParams restArgs
                  (s⦇FState.boundValues := MList.insert obsId (if arbEvalObs then 1 else 0) (
                       FState.boundValues s)⦈) compiledBody =
                 Some compiledFullContract ⟶
                 (∃newMWarnings.
                     (compiledFullContract,
                      ⦇State.accounts = FState.accounts s, choices = FState.choices s,
                         boundValues = MList.insert obsId (if arbEvalObs then 1 else 0) (FState.
                              boundValues s), minTime = FState.minTime s⦈,
                      e, warnings, payments) →*
                     (compiledBody,
                      ⦇State.accounts = FState.accounts newState, choices = FState.choices
                           newState,
                         boundValues = FState.boundValues newState, minTime = FState.minTime
                              newState⦈,
                      e, newMWarnings, payments))) ⟹
      evalFObservation e s obs = Some evaluatedObs ⟹
      compileFObservation obs s = Some compiledObs ⟹
      wellTypedObservation tyCtx obs ⟹
      wellTypedArguments tyCtx restParams restArgs ⟹
      wellTypedState tyCtx s ⟹
      evalFArguments e (s⦇FState.boundValues := MList.insert obsId (if evaluatedObs then 1 else
            0) (FState.boundValues s)⦈) restParams
```

```
      restArgs =
     Some newState ⟹
     (case compileArguments restParams restArgs (s⦇FState.boundValues := MList.insert obsId 0 (
          FState.boundValues s)⦈) compiledBody of
      None ⇒ None | Some compiledRest ⇒ Some (Contract.Let obsId (Value.Cond compiledObs (
           Value.Constant 1) (Value.Constant 0)) compiledRest)) =
     Some compiledFullContract ⟹
     ∃newMWarnings.
        (compiledFullContract,
         ⦇State.accounts = FState.accounts s, choices = FState.choices s, boundValues = FState.
              boundValues s,
            minTime = FState.minTime s⦈,
         e, warnings, payments) →*
        (compiledBody,
         ⦇State.accounts = FState.accounts newState, choices = FState.choices newState,
              boundValues = FState.boundValues newState,
            minTime = FState.minTime newState⦈,
         e, newMWarnings, payments)"
apply (cases evaluatedObs, auto)
subgoal premises ps proof-
  have "evaluatedObs = True" using ps by auto
  moreover have "evalObservation e (fStateToMState s) compiledObs = True" using ps
       preservationOfValueEvaluation(2) by fastforce
  moreover have "evalValue e (fStateToMState s) (Value.Cond compiledObs (Value.Constant 1) (
       Value.Constant 0)) = 1" using ps calculation by auto
  moreover obtain tyCtx where "wellTypedObservation tyCtx obs ∧ wellTypedArguments tyCtx
       restParams restArgs ∧ wellTypedState tyCtx s" using ps by auto
  moreover have "wellTypedState tyCtx (s⦇FState.boundValues := MList.insert obsId 1 (FState.
       boundValues s)⦈)" using ps calculation insertValueWellTypedStateIsWellTyped by auto
  moreover have "wellTypedState tyCtx (s⦇FState.boundValues := MList.insert obsId 0 (FState.
       boundValues s)⦈)" using ps calculation insertValueWellTypedStateIsWellTyped by auto
  moreover obtain newState where "evalFArguments e (s⦇FState.boundValues := MList.insert obsId
       1 (FState.boundValues s)⦈) restParams restArgs = Some newState" using ps by auto
  moreover obtain compiledRest where "compileArguments restParams restArgs (s⦇FState.
       boundValues := MList.insert obsId 0 (FState.boundValues s)⦈) compiledBody = Some
       compiledRest" using ps calculation wellTypedArgumentsCompiles by fastforce
  moreover have "s⦇FState.boundValues := MList.insert obsId 1 (FState.boundValues s)⦈ = (s⦇
       FState.boundValues := MList.insert obsId 1 (FState.boundValues s)⦈)⦇boundPubKeys:=(
       boundPubKeys (s⦇FState.boundValues := MList.insert obsId 0 (FState.boundValues s)⦈))⦈"
       by auto
  moreover have "s⦇FState.boundValues := MList.insert obsId 1 (FState.boundValues s)⦈ = (s⦇
       FState.boundValues := MList.insert obsId 1 (FState.boundValues s)⦈)⦇boundPubKeys:=(
       boundPubKeys (s⦇FState.boundValues := MList.insert obsId 0 (FState.boundValues s)⦈))⦈"
       by auto
```

```
  moreover have "compileArguments restParams restArgs (s⦇FState.boundValues := MList.insert
      obsId 1 (FState.boundValues s)⦈) compiledBody = Some compiledRest" using ps calculation
      apply auto
    by (metis calculation(9) compileArgumentsOnlyBoundPubKeysMatter)
  moreover have "(∀warnings payments. ∃newMWarnings. (compiledRest, ⦇State.accounts = FState.
      accounts s, choices = FState.choices s, boundValues = MList.insert obsId 1 (FState.
      boundValues s), minTime = FState.minTime s⦈, e, warnings, payments) →* (compiledBody,
      fStateToMState newState, e, newMWarnings, payments))" using ps calculation by auto
  moreover have "(∀warnings payments. ∃newMWarnings. (Contract.Let obsId (Value.Cond
      compiledObs (Value.Constant 1) (Value.Constant 0)) compiledRest, fStateToMState s, e,
      warnings, payments) → (compiledRest, (fStateToMState s)⦇State.boundValues := MList.
      insert obsId 1 (State.boundValues (fStateToMState s))⦈, e, newMWarnings, payments))"
      using calculation(3) apply (cases "lookup obsId (State.boundValues (fStateToMState s))")
    using SmallStep.LetNoShadow apply metis
    using SmallStep.LetShadow by metis
  moreover have "(fStateToMState s)⦇State.boundValues := MList.insert obsId 1 (State.
      boundValues (fStateToMState s))⦈ = ⦇State.accounts = FState.accounts s, choices = FState
      .choices s, boundValues = MList.insert obsId 1 (FState.boundValues s), minTime = FState.
      minTime s⦈" by auto
  moreover have "(∃newMWarnings. (SemanticsTypes.Let obsId (Value.Cond compiledObs (Value.
      Constant 1) (Value.Constant 0)) compiledRest, fStateToMState s, e, warnings, payments)
      → (compiledRest, ⦇State.accounts = FState.accounts s, choices = FState.choices s,
      boundValues = MList.insert obsId 1 (FState.boundValues s), minTime = FState.minTime s⦈,
      e, newMWarnings, payments))" using calculation by auto
  ultimately show ?thesis using ps star.step ps SmallStep.smallStepStarWarningsAreArbitrary
      apply auto
    by (smt \<open>∃newMWarnings. (Contract.Let obsId (Value.Cond compiledObs (Value.Constant
        1) (Value.Constant 0)) compiledRest, fStateToMState s, e, warnings, payments) → (
        compiledRest, ⦇State.accounts = FState.accounts s, choices = FState.choices s,
        boundValues = MList.insert obsId 1 (FState.boundValues s), minTime = FState.minTime s⦈
        , e, newMWarnings, payments)\<close> \<open>∀warnings payments. ∃newMWarnings. (
        compiledRest, ⦇State.accounts = FState.accounts s, choices = FState.choices s,
        boundValues = MList.insert obsId 1 (FState.boundValues s), minTime = FState.minTime s⦈
        , e, warnings, payments) →* (compiledBody, fStateToMState newState, e, newMWarnings,
        payments)\<close> fStateToMState.simps star.step)
qed
subgoal premises ps proof-
  have "evaluatedObs = False" using ps by auto
  moreover have "evalObservation e (fStateToMState s) compiledObs = False" using ps
      preservationOfValueEvaluation(2) by fastforce
  moreover have "evalValue e (fStateToMState s) (Value.Cond compiledObs (Value.Constant 1) (
      Value.Constant 0)) = 0" using ps calculation by auto
  moreover obtain tyCtx where "wellTypedObservation tyCtx obs ∧ wellTypedArguments tyCtx
      restParams restArgs ∧ wellTypedState tyCtx s" using ps by auto
```

```
moreover have "wellTypedState tyCtx (s⦇FState.boundValues := MList.insert obsId 0 (FState.
     boundValues s)⦈)" using ps calculation insertValueWellTypedStateIsWellTyped by auto
moreover obtain newState where "evalFArguments e (s⦇FState.boundValues := MList.insert obsId
     0 (FState.boundValues s)⦈) restParams restArgs = Some newState" using ps by auto
moreover obtain compiledRest where "compileArguments restParams restArgs (s⦇FState.
     boundValues := MList.insert obsId 0 (FState.boundValues s)⦈) compiledBody = Some
     compiledRest" using ps calculation wellTypedArgumentsCompiles by fastforce
moreover have "s⦇FState.boundValues := MList.insert obsId 0 (FState.boundValues s)⦈ = (s⦇
     FState.boundValues := MList.insert obsId 0 (FState.boundValues s)⦈)⦇boundPubKeys:=(
     boundPubKeys (s⦇FState.boundValues := MList.insert obsId 0 (FState.boundValues s)⦈))⦈"
     by auto
moreover have "s⦇FState.boundValues := MList.insert obsId 0 (FState.boundValues s)⦈ = (s⦇
     FState.boundValues := MList.insert obsId 0 (FState.boundValues s)⦈)⦇boundPubKeys:=(
     boundPubKeys (s⦇FState.boundValues := MList.insert obsId 0 (FState.boundValues s)⦈))⦈"
     by auto
moreover have "compileArguments restParams restArgs (s⦇FState.boundValues := MList.insert
     obsId 0 (FState.boundValues s)⦈) compiledBody = Some compiledRest" using ps calculation
     by auto
moreover have "(∀warnings payments. ∃newMWarnings. (compiledRest, ⦇State.accounts = FState.
     accounts s, choices = FState.choices s, boundValues = MList.insert obsId 0 (FState.
     boundValues s), minTime = FState.minTime s⦈, e, warnings, payments) →⋆ (compiledBody,
     fStateToMState newState, e, newMWarnings, payments))" using ps calculation by auto
moreover have "(∀warnings payments. ∃newMWarnings. (Contract.Let obsId (Value.Cond
     compiledObs (Value.Constant 1) (Value.Constant 0)) compiledRest, fStateToMState s, e,
     warnings, payments) → (compiledRest, (fStateToMState s)⦇State.boundValues := MList.
     insert obsId 0 (State.boundValues (fStateToMState s))⦈, e, newMWarnings, payments))"
     using calculation(3) apply (cases "lookup obsId (State.boundValues (fStateToMState s))")
  using SmallStep.LetNoShadow apply metis
  using SmallStep.LetShadow by metis
moreover have "(fStateToMState s)⦇State.boundValues := MList.insert obsId 0 (State.
     boundValues (fStateToMState s))⦈ = ⦇State.accounts = FState.accounts s, choices = FState
     .choices s, boundValues = MList.insert obsId 0 (FState.boundValues s), minTime = FState.
     minTime s⦈" by auto
moreover have "(∃newMWarnings. (SemanticsTypes.Let obsId (Value.Cond compiledObs (Value.
     Constant 1) (Value.Constant 0)) compiledRest, fStateToMState s, e, warnings, payments)
     → (compiledRest, ⦇State.accounts = FState.accounts s, choices = FState.choices s,
     boundValues = MList.insert obsId 0 (FState.boundValues s), minTime = FState.minTime s⦈,
     e, newMWarnings, payments))" using calculation by auto
ultimately show ?thesis using ps star.step ps SmallStep.smallStepStarWarningsAreArbitrary
     apply auto
  by (smt \<open>∃newMWarnings. (Contract.Let obsId (Value.Cond compiledObs (Value.Constant
       1) (Value.Constant 0)) compiledRest, fStateToMState s, e, warnings, payments) → (
       compiledRest, ⦇State.accounts = FState.accounts s, choices = FState.choices s,
       boundValues = MList.insert obsId 0 (FState.boundValues s), minTime = FState.minTime s⦈
```

```
          , e, newMWarnings, payments)\<close> \<open>∀warnings payments. ∃newMWarnings. (
          compiledRest, ⦇State.accounts = FState.accounts s, choices = FState.choices s,
          boundValues = MList.insert obsId 0 (FState.boundValues s), minTime = FState.minTime s⦈
          , e, warnings, payments) →* (compiledBody, fStateToMState newState, e, newMWarnings,
          payments)\<close> fStateToMState.simps star.step)
  qed
  done

(* EvalFArguments will give same state as executing compiled Marlowe to body. *)
lemma preservationOfArgumentEvaluation:
"(∀newState compiledBody compiledFullContract warnings payments tyCtx argCtx. wellTypedArguments
     tyCtx params args ⟶
  wellTypedState tyCtx state ⟶
  evalFArguments env state params args = Some newState ⟶
  compileArguments params args state compiledBody = Some compiledFullContract ⟶
  (∃newMWarnings . (compiledFullContract, fStateToMState state, env, warnings, payments) →*
  (compiledBody, fStateToMState newState, env, newMWarnings, payments)))"
proof (induction rule: evalFArguments.induct)
  case (1 e s)
  then show ?case by auto
next
  case (2 e s vid restParams val restArgs)
  then show ?case using preservationOfArgumentEvaluation_ValueParameter_Induct by (cases "
       evalFValue e s val" "compileFValue val s" rule: option.exhaust[case_product option.exhaust
       ], auto)
next
  case (3 e s obsId restParams obs restArgs)
  then show ?case using preservationOfArgumentEvaluation_ObservationParameter_Induct by (cases "
       evalFObservation e s obs" "compileFObservation obs s" rule: option.exhaust[case_product
       option.exhaust], auto split del: if_split)
next
  case (4 e s pkId restParams pk restArgs)
  then show ?case apply auto
    apply (cases "evalFParty s pk", auto)
    apply (cases "compileFParty pk s", auto)
    by (simp add: insertPubKeyWellTypedStateIsWellTyped preservationOfPubKeyEvaluation)
next
  case ("5_1" e s v restParams va restArgs)
  then show ?case by auto
next
  case ("5_2" e s v restParams va restArgs)
  then show ?case by auto
next
  case ("5_3" e s v restParams va restArgs)
```

```
  then show ?case by auto
next
  case ("5_4" e s v restParams va restArgs)
  then show ?case by auto
next
  case ("5_5" e s va restParams v restArgs)
  then show ?case by auto
next
  case ("5_6" e s va restParams v restArgs)
  then show ?case by auto
next
  case ("5_7" e s va restParams v restArgs)
  then show ?case by auto
next
  case ("5_8" e s va restParams v restArgs)
  then show ?case by auto
next
  case (6 e s v va)
  then show ?case by auto
next
  case (7 e s v va)
  then show ?case by auto
qed

(* Zero or more well typed Faustus steps correspond
  to zero or more Marlowe steps. *)
lemma preservationOfSemantics_FaustusStepImpMarloweSteps:
"(fContract, ctx, state, env, warnings, payments) →f (newFContract, newCtx, newState, newEnv,
     newWarnings, newPayments) ⟹
  wellTypedContract tyCtx fContract ⟹
  wellTypedContext tyCtx ctx ⟹
  wellTypedState tyCtx state ⟹
  compile fContract ctx state = Some marloweContract ⟹
  compile newFContract newCtx newState = Some newMarloweContract ⟹
  (∃newMWarnings . (marloweContract, fStateToMState state, env, warnings, payments) →* (
       newMarloweContract, fStateToMState newState, newEnv, newMWarnings, newPayments))"
proof (induction "(fContract, ctx, state, env, warnings, payments)" "(newFContract, newCtx,
     newState, newEnv, newWarnings, newPayments)" rule: small_step_reduce.induct)
  case (CloseRefund party token money newAccount)
  moreover have "refundOne (State.accounts (fStateToMState state)) = Some ((party, token, money),
        newAccount)" using calculation by auto
  moreover have "⦇State.accounts = newAccount, choices = FState.choices state, boundValues =
       FState.boundValues state, minTime = FState.minTime state⦈ = (fStateToMState state)⦇State.
       accounts := newAccount⦈" using calculation by auto
```

```
  moreover have "(Contract.Close, fStateToMState state, newEnv, warnings, payments) → (Contract.
       Close, (fStateToMState state)⦇State.accounts := newAccount⦈, newEnv, warnings @ [
       ReduceNoWarning], payments @ [Payment party (SemanticsTypes.Party party) token money])"
       using calculation SmallStep.CloseRefund by blast
  ultimately show ?case by fastforce
next
  case (PayNonPositive val res fromAcc accId toPayee payee token)
  moreover obtain compiledAccId where "compileFParty accId newState = Some compiledAccId" using
       calculation by fastforce
  moreover obtain compiledPayee where "compileFPayee payee newState = Some compiledPayee" using
       calculation by fastforce
  moreover obtain compiledFValue where "compileFValue val newState = Some compiledFValue" using
       calculation by fastforce
  ultimately show ?case apply auto
    using preservationOfValueEvaluation(1) by fastforce
next
  case (PayPositivePartialWithPayment val res fromAcc accId toPayee payee token newAccs
       moneyToPay payment finalAccs somePayment)
  moreover obtain newCompiledCont where "compile newFContract newCtx state = Some newCompiledCont
       " using calculation wellTypedCompiles typePreservation by fastforce
  moreover have "compile newFContract newCtx newState = Some newCompiledCont" using calculation
       wellTypedCompiles typePreservation compileAccountsArbitrary by fastforce
  moreover obtain compiledAccId where "compileFParty accId state = Some compiledAccId" using
       calculation by fastforce
  moreover obtain compiledPayee where "compileFPayee payee state = Some compiledPayee" using
       calculation by fastforce
  moreover obtain compiledFValue where "compileFValue val state = Some compiledFValue" using
       calculation by fastforce
  moreover have "evalValue env (fStateToMState state) compiledFValue > 0" using calculation
       preservationOfValueEvaluation option.sel by fastforce
  moreover have "evalValue env (fStateToMState state) compiledFValue > moneyInAccount
       compiledAccId token (State.accounts (fStateToMState state))" using calculation
       preservationOfValueEvaluation preservationOfPubKeyEvaluation option.sel by (auto, smt)
  moreover have "updateMoneyInAccount compiledAccId token 0 (State.accounts (fStateToMState state
       )) = newAccs" using calculation preservationOfPubKeyEvaluation option.sel by auto
  moreover have "moneyInAccount compiledAccId token (State.accounts (fStateToMState state)) =
       moneyToPay" using calculation apply (auto split: option.split)
      apply (metis compileFParty.simps evalFParty.elims option.distinct(1) option.sel)
     apply (metis compileFParty.simps evalFParty.elims option.distinct(1) option.sel)
    by (metis calculation(18) calculation(23) calculation(3) option.inject
         preservationOfPubKeyEvaluation)
  moreover have "toPayee = compiledPayee" using calculation wellTypedPartyEvaluates apply auto
    apply (cases payee, simp)
```

```
    apply (metis calculation(4) evalFPayee.simps(1) option.inject preservationOfPubKeyEvaluation
        )
    by (metis compileFPayee.simps(2) compileFParty.simps evalFPayee.simps(2) evalFParty.simps
        option.inject wellTypedPayee.simps(2) wellTypedParty.elims(2))
  moreover have "giveMoney compiledAccId compiledPayee token moneyToPay newAccs = (payment,
      finalAccs)" using calculation apply auto
    by (metis calculation(18) calculation(23) calculation(3) option.inject
        preservationOfPubKeyEvaluation)
  moreover have "compile fContract ctx state = Some (Contract.Pay compiledAccId compiledPayee
      token compiledFValue newCompiledCont)" using calculation apply auto
    by (metis option.sel option.simps(5))
  moreover have "(Contract.Pay compiledAccId compiledPayee token compiledFValue newCompiledCont,
      fStateToMState state, env, warnings, payments) → (newCompiledCont, (fStateToMState state)
      (State.accounts := finalAccs), newEnv, warnings @ [ReducePartialPay compiledAccId
      compiledPayee token moneyToPay (evalValue env (fStateToMState state) compiledFValue)],
      payments @ [somePayment])" using calculation by blast
  moreover have "((fStateToMState state)(State.accounts := finalAccs)) = fStateToMState newState"
       using calculation by fastforce
  ultimately show ?case by (simp, meson star_step1)
next
  case (PayPositivePartialWithoutPayment val res fromAcc accId toPayee payee token newAccs
      moneyToPay payment finalAccs)
  moreover obtain newCompiledCont where "compile newFContract newCtx state = Some newCompiledCont
      " using calculation wellTypedCompiles typePreservation by fastforce
  moreover have "compile newFContract newCtx newState = Some newCompiledCont" using calculation
      wellTypedCompiles typePreservation compileAccountsArbitrary by fastforce
  moreover obtain compiledAccId where "compileFParty accId state = Some compiledAccId" using
      calculation by fastforce
  moreover obtain compiledPayee where "compileFPayee payee state = Some compiledPayee" using
      calculation by fastforce
  moreover obtain compiledFValue where "compileFValue val state = Some compiledFValue" using
      calculation by fastforce
  moreover have "evalValue env (fStateToMState state) compiledFValue > 0" using calculation
      preservationOfValueEvaluation option.sel by fastforce
  moreover have "evalValue env (fStateToMState state) compiledFValue > moneyInAccount
      compiledAccId token (State.accounts (fStateToMState state))" using calculation
      preservationOfValueEvaluation preservationOfPubKeyEvaluation option.sel by auto
  moreover have "updateMoneyInAccount compiledAccId token 0 (State.accounts (fStateToMState state
      )) = newAccs" using calculation preservationOfPubKeyEvaluation option.sel by auto
  moreover have "moneyInAccount compiledAccId token (State.accounts (fStateToMState state)) =
      moneyToPay" using calculation by (auto split: option.split)
  moreover have "giveMoney compiledAccId compiledPayee token moneyToPay newAccs = (payment,
      finalAccs)" using calculation preservationOfPubKeyEvaluation by auto
```

```
  moreover have "compile fContract ctx state = Some (Contract.Pay compiledAccId compiledPayee
       token compiledFValue newCompiledCont)" using calculation by auto
  moreover have "(Contract.Pay compiledAccId compiledPayee token compiledFValue newCompiledCont,
       fStateToMState state, env, warnings, payments) → (newCompiledCont, (fStateToMState state)
       ⦇State.accounts := finalAccs⦈, newEnv, warnings @ [ReducePartialPay compiledAccId
       compiledPayee token moneyToPay (evalValue env (fStateToMState state) compiledFValue)],
       payments)" using calculation by blast
  moreover have "((fStateToMState state)⦇State.accounts := finalAccs⦈) = fStateToMState newState"
        using calculation by fastforce
  ultimately show ?case by auto
next
  case (PayPositiveFullWithPayment val res fromAcc accId toPayee payee token moneyInAcc
       newBalance newAccs payment finalAccs somePayment)
  moreover obtain newCompiledCont where "compile newFContract newCtx state = Some newCompiledCont
       " using calculation wellTypedCompiles typePreservation by fastforce
  moreover have "compile newFContract newCtx newState = Some newCompiledCont" using calculation
       wellTypedCompiles typePreservation compileAccountsArbitrary by fastforce
  moreover obtain compiledAccId where "compileFParty accId state = Some compiledAccId" using
       calculation by fastforce
  moreover obtain compiledPayee where "compileFPayee payee state = Some compiledPayee" using
       calculation by fastforce
  moreover obtain compiledFValue where "compileFValue val state = Some compiledFValue" using
       calculation by fastforce
  moreover have "evalValue env (fStateToMState state) compiledFValue = res" using calculation
       preservationOfValueEvaluation option.sel by fastforce
  moreover have "evalValue env (fStateToMState state) compiledFValue ≤ moneyInAccount
       compiledAccId token (State.accounts (fStateToMState state))" using calculation
       preservationOfValueEvaluation preservationOfPubKeyEvaluation option.sel by auto
  moreover have "updateMoneyInAccount compiledAccId token newBalance (State.accounts (
       fStateToMState state)) = newAccs" using calculation preservationOfPubKeyEvaluation option.
       sel by auto
  moreover have "moneyInAccount compiledAccId token (State.accounts (fStateToMState state)) =
       moneyInAcc" using calculation apply simp
    by (metis compileFParty.simps evalFParty.elims option.sel)
  moreover have "giveMoney compiledAccId compiledPayee token res newAccs = (payment, finalAccs)"
       using calculation preservationOfPayeeEvaluation apply auto
    by (metis calculation(19) calculation(24) calculation(3) option.inject
        preservationOfPubKeyEvaluation)
  moreover have "compile fContract ctx state = Some (Contract.Pay compiledAccId compiledPayee
       token compiledFValue newCompiledCont)" using calculation apply auto
    by (smt (verit, best) calculation(20) compileCorrectness_Pay option.inject)
  moreover have "(Contract.Pay compiledAccId compiledPayee token compiledFValue newCompiledCont,
       fStateToMState state, env, warnings, payments) → (newCompiledCont, (fStateToMState state)
       ⦇State.accounts := finalAccs⦈, newEnv, warnings @ [ReduceNoWarning], payments @ [
```

```
      somePayment])" using calculation by blast
  moreover have "((fStateToMState state)⦇State.accounts := finalAccs⦈) = fStateToMState newState"
       using calculation by fastforce
  ultimately show ?case by (simp, meson star_step1)
next
  case (PayPositiveFullWithoutPayment val res fromAcc accId toPayee payee token moneyInAcc
      newBalance newAccs payment finalAccs)
  moreover obtain newCompiledCont where "compile newFContract newCtx state = Some newCompiledCont
      " using calculation wellTypedCompiles typePreservation by fastforce
  moreover have "compile newFContract newCtx newState = Some newCompiledCont" using calculation
      wellTypedCompiles typePreservation compileAccountsArbitrary by fastforce
  moreover obtain compiledAccId where "compileFParty accId state = Some compiledAccId" using
      calculation by fastforce
  moreover obtain compiledPayee where "compileFPayee payee state = Some compiledPayee" using
      calculation by fastforce
  moreover obtain compiledFValue where "compileFValue val state = Some compiledFValue" using
      calculation by fastforce
  moreover have "evalValue env (fStateToMState state) compiledFValue = res" using calculation
      preservationOfValueEvaluation option.sel by fastforce
  moreover have "evalValue env (fStateToMState state) compiledFValue ≤ moneyInAccount
      compiledAccId token (State.accounts (fStateToMState state))" using calculation
      preservationOfValueEvaluation preservationOfPubKeyEvaluation option.sel by auto
  moreover have "updateMoneyInAccount compiledAccId token newBalance (State.accounts (
      fStateToMState state)) = newAccs" using calculation preservationOfPubKeyEvaluation option.
      sel by auto
  moreover have "moneyInAccount compiledAccId token (State.accounts (fStateToMState state)) =
      moneyInAcc" using calculation preservationOfPubKeyEvaluation by auto
  moreover have "toPayee = compiledPayee" using calculation preservationOfPayeeEvaluation
      wellTypedPartyEvaluates by auto
  moreover have "giveMoney compiledAccId compiledPayee token res newAccs = (payment, finalAccs)"
      using calculation preservationOfPubKeyEvaluation by auto
  moreover have "compile fContract ctx state = Some (Contract.Pay compiledAccId compiledPayee
      token compiledFValue newCompiledCont)" using calculation by auto
  moreover have "(Contract.Pay compiledAccId compiledPayee token compiledFValue newCompiledCont,
      fStateToMState state, env, warnings, payments) → (newCompiledCont, (fStateToMState state)
      ⦇State.accounts := finalAccs⦈, newEnv, warnings @ [ReduceNoWarning], payments)" using
      calculation by blast
  moreover have "((fStateToMState state)⦇State.accounts := finalAccs⦈) = fStateToMState newState"
       using calculation by fastforce
  ultimately show ?case by auto
next
  case (IfTrue obs cont2)
  then show ?case apply (cases "compile cont2 newCtx newState", auto)
```

```
    using preservationOfValueEvaluation(2) wellTypedValueObservationCompiles(2) wellTypedCompiles
          by fastforce
next
  case (IfFalse obs cont1)
  then show ?case using preservationOfValueEvaluation(2) wellTypedValueObservationCompiles(2)
       wellTypedCompiles by fastforce
next
  case (WhenTimeout startSlot endSlot timeout cases)
  then show ?case apply auto
    by (cases "compileCases cases newCtx newState", auto)
next
  case (LetShadow valId oldVal val res)
  moreover have "MList.lookup valId (State.boundValues (fStateToMState state)) = Some oldVal"
       using calculation by auto
  moreover obtain compiledVal where "compileFValue val state = Some compiledVal" using
       calculation wellTypedValueObservationCompiles(1) by fastforce
  moreover obtain compiledCont where "compile newFContract newCtx (state⦇FState.boundValues :=
       MList.insert valId 0 (FState.boundValues state)⦈) = Some compiledCont" using calculation
       wellTypedCompiles by fastforce
  moreover have "newState = newState⦇boundPubKeys := boundPubKeys (state⦇FState.boundValues :=
       MList.insert valId 0 (FState.boundValues state)⦈)⦈" using calculation by auto
  moreover have "compile newFContract newCtx newState = Some compiledCont" by (metis
       compileContractOnlyBoundPubKeysMatter(1) calculation(13) calculation(16) calculation(17))
  moreover have "fStateToMState newState = (fStateToMState state)⦇State.boundValues := MList.
       insert valId (res) (boundValues state)⦈" using calculation by auto
  moreover have "fStateToMState newState = (fStateToMState state)⦇State.boundValues := MList.
       insert valId (res) (State.boundValues (fStateToMState state))⦈" using calculation by auto
  moreover have "(evalValue env (fStateToMState state) compiledVal) = res" using calculation
       preservationOfValueEvaluation by fastforce
  moreover have "(SemanticsTypes.Let valId compiledVal compiledCont, (fStateToMState state), env,
        warnings, payments) →
    (compiledCont, (fStateToMState state)⦇State.boundValues := MList.insert valId (res) (State.
         boundValues (fStateToMState state))⦈, env, warnings @ [ReduceShadowing valId oldVal (res
         )], payments)" using calculation by blast
  moreover have "(SemanticsTypes.Let valId compiledVal compiledCont, (fStateToMState state), env,
        warnings, payments) →
    (compiledCont, (fStateToMState newState), env, warnings @ [ReduceShadowing valId oldVal (res)
         ], payments)" using calculation by auto
  ultimately show ?case apply auto
    by (metis (no_types, lifting) State.update_convs(3) \<open>(Contract.Let valId compiledVal
         compiledCont, fStateToMState state, env, warnings, payments) → (compiledCont,
         fStateToMState newState, env, warnings @ [ReduceShadowing valId oldVal res], payments)\<
         close> \<open>fStateToMState newState = fStateToMState state ⦇State.boundValues := MList
         .insert valId res (State.boundValues (fStateToMState state))⦈\<close> fStateToMState.
```

```
        simps fStateToMStatePreservationOfBoundValues star_step1)
next
  case (LetNoShadow valId val res)
  moreover have "MList.lookup valId (State.boundValues (fStateToMState state)) = None" using
       calculation by auto
  moreover obtain compiledVal where "compileFValue val state = Some compiledVal" using
       calculation wellTypedValueObservationCompiles(1) by fastforce
  moreover obtain compiledCont where "compile newFContract newCtx (state⦇FState.boundValues :=
       MList.insert valId 0 (FState.boundValues state)⦈) = Some compiledCont" using calculation
       wellTypedCompiles by fastforce
  moreover have "newState = newState⦇boundPubKeys := boundPubKeys (state⦇FState.boundValues :=
       MList.insert valId 0 (FState.boundValues state)⦈)⦈" using calculation by auto
  moreover have "compile newFContract newCtx newState = Some compiledCont" by (metis
       compileContractOnlyBoundPubKeysMatter(1) calculation(13) calculation(16) calculation(17))
  moreover have "fStateToMState newState = (fStateToMState state)⦇State.boundValues := MList.
       insert valId (res) (boundValues state)⦈" using calculation by auto
  moreover have "fStateToMState newState = (fStateToMState state)⦇State.boundValues := MList.
       insert valId (res) (State.boundValues (fStateToMState state))⦈" using calculation by auto
  moreover have "(evalValue env (fStateToMState state) compiledVal) = res" using calculation
       preservationOfValueEvaluation by fastforce
  moreover have "(SemanticsTypes.Let valId compiledVal compiledCont, (fStateToMState state), env,
        warnings, payments) →
    (compiledCont, (fStateToMState state)⦇State.boundValues := MList.insert valId (res) (State.
         boundValues (fStateToMState state))⦈, env, warnings @ [ReduceNoWarning], payments)"
         using calculation by blast
  moreover have "(SemanticsTypes.Let valId compiledVal compiledCont, (fStateToMState state), env,
        warnings, payments) →
    (compiledCont, (fStateToMState newState), env, warnings @ [ReduceNoWarning], payments)" using
          calculation by auto
  ultimately show ?case apply auto
    by (metis (no_types, lifting) State.update_convs(3) \<open>(Contract.Let valId compiledVal
         compiledCont, fStateToMState state, env, warnings, payments) → (compiledCont,
         fStateToMState newState, env, warnings @ [ReduceNoWarning], payments)\<close> \<open>
         fStateToMState newState = fStateToMState state ⦇State.boundValues := MList.insert valId
         res (State.boundValues (fStateToMState state))⦈\<close> fStateToMState.simps
         fStateToMStatePreservationOfBoundValues star_step1)
next
  case (LetObservation obs res oId)
  then show ?case apply auto
    subgoal premises ps proof (cases "MList.lookup oId (State.boundValues (fStateToMState state))
         ")
      case None
      moreover obtain compiledObs where "compileFObservation obs state = Some compiledObs" using
           calculation ps wellTypedValueObservationCompiles(2) by fastforce
```

```
    moreover obtain compiledCont where "compile newFContract newCtx (state⦇FState.boundValues
         := MList.insert oId 0 (FState.boundValues state)⦈) = Some compiledCont" using ps
         calculation wellTypedCompiles by fastforce
    moreover have "newState = newState⦇boundPubKeys := boundPubKeys (state⦇FState.boundValues
         := MList.insert oId 0 (FState.boundValues state)⦈)⦈" using ps calculation by auto
    moreover have "compile newFContract newCtx newState = Some compiledCont" by (metis
         compileContractOnlyBoundPubKeysMatter(1) calculation(4) calculation(3))
    moreover have "fStateToMState newState = (fStateToMState state)⦇State.boundValues := MList.
         insert oId 1 (boundValues state)⦈" using ps calculation by auto
    moreover have "fStateToMState newState = (fStateToMState state)⦇State.boundValues := MList.
         insert oId 1 (State.boundValues (fStateToMState state))⦈" using calculation by auto
    moreover have "(evalValue env (fStateToMState state) (Value.Cond compiledObs (Value.
         Constant 1) (Value.Constant 0))) = 1" using ps calculation
         preservationOfValueEvaluation by fastforce
    moreover have "(evalValue env (fStateToMState state) (Value.Cond compiledObs (Value.
         Constant 1) (Value.Constant 0))) = 1" using calculation preservationOfValueEvaluation
         by fastforce
    moreover have "(SemanticsTypes.Let oId (Value.Cond compiledObs (Value.Constant 1) (Value.
         Constant 0)) compiledCont, (fStateToMState state), env, warnings, payments) → (
         compiledCont, (fStateToMState state)⦇State.boundValues := MList.insert oId 1 (State.
         boundValues (fStateToMState state))⦈, env, warnings @ [ReduceNoWarning], payments)" by
          (metis calculation(1) calculation(8) calculation(9) SmallStep.LetNoShadow)
    moreover have "(SemanticsTypes.Let oId (Value.Cond compiledObs (Value.Constant 1) (Value.
         Constant 0)) compiledCont, (fStateToMState state), env, warnings, payments) → (
         compiledCont, (fStateToMState newState), env, warnings @ [ReduceNoWarning], payments)"
          using calculation by auto
    ultimately show ?thesis apply auto
      by (metis (no_types, lifting) LetObservation.prems(5) \<open>(Contract.Let oId (Value.
           Cond compiledObs (Value.Constant 1) (Value.Constant 0)) compiledCont, fStateToMState
            state, env, warnings, payments) → (compiledCont, fStateToMState state ⦇State.
           boundValues := MList.insert oId 1 (State.boundValues (fStateToMState state))⦈, env,
           warnings @ [ReduceNoWarning], payments)\<close> \<open>fStateToMState newState =
           fStateToMState state ⦇State.boundValues := MList.insert oId 1 (State.boundValues (
           fStateToMState state))⦈\<close> fStateToMState.simps option.inject option.simps(5)
           ps(2) ps(3) ps(6) ps(9) star_step1)
  next
    case (Some x2)
    moreover obtain compiledObs where "compileFObservation obs state = Some compiledObs" using
         calculation ps wellTypedValueObservationCompiles(2) by fastforce
    moreover obtain compiledCont where "compile newFContract newCtx (state⦇FState.boundValues
         := MList.insert oId 0 (FState.boundValues state)⦈) = Some compiledCont" using ps
         calculation wellTypedCompiles by fastforce
    moreover have "newState = newState⦇boundPubKeys := boundPubKeys (state⦇FState.boundValues
         := MList.insert oId 0 (FState.boundValues state)⦈)⦈" using ps calculation by auto
```

```
  moreover have "compile newFContract newCtx newState = Some compiledCont" by (metis
       compileContractOnlyBoundPubKeysMatter(1) calculation(3) calculation(4))
  moreover have "fStateToMState newState = (fStateToMState state)⦇State.boundValues := MList.
       insert oId 1 (boundValues state)⦈" using ps calculation by auto
  moreover have "fStateToMState newState = (fStateToMState state)⦇State.boundValues := MList.
       insert oId 1 (State.boundValues (fStateToMState state))⦈" using calculation by auto
  moreover have "(evalValue env (fStateToMState state) (Value.Cond compiledObs (Value.
       Constant 1) (Value.Constant 0))) = 1" using ps calculation
       preservationOfValueEvaluation by fastforce
  moreover have "(evalValue env (fStateToMState state) (Value.Cond compiledObs (Value.
       Constant 1) (Value.Constant 0))) = 1" using calculation preservationOfValueEvaluation
       by fastforce
  moreover have "∃newMWarning. (SemanticsTypes.Let oId (Value.Cond compiledObs (Value.
       Constant 1) (Value.Constant 0)) compiledCont, (fStateToMState state), env, warnings,
       payments) → (compiledCont, (fStateToMState state)⦇State.boundValues := MList.insert
       oId 1 (State.boundValues (fStateToMState state))⦈, env, warnings @ newMWarning,
       payments)" by (metis calculation(1) calculation(8) calculation(9) SmallStep.LetShadow)
  moreover have "∃newMWarning. (SemanticsTypes.Let oId (Value.Cond compiledObs (Value.
       Constant 1) (Value.Constant 0)) compiledCont, (fStateToMState state), env, warnings,
       payments) → (compiledCont, (fStateToMState newState), env, warnings @ newMWarning,
       payments)" using calculation by auto
  ultimately show ?thesis apply auto
    by (metis (no_types, lifting) LetObservation.prems(5) \<open>∃newMWarning. (Contract.Let
         oId (Value.Cond compiledObs (Value.Constant 1) (Value.Constant 0)) compiledCont,
         fStateToMState state, env, warnings, payments) → (compiledCont, fStateToMState
         state ⦇State.boundValues := MList.insert oId 1 (State.boundValues (fStateToMState
         state))⦈, env, warnings @ newMWarning, payments)\<close> \<open>fStateToMState
         newState = fStateToMState state ⦇State.boundValues := MList.insert oId 1 (State.
         boundValues (fStateToMState state))⦈\<close> fStateToMState.simps option.inject
         option.simps(5) ps(2) ps(3) ps(6) ps(9) star_step1)
qed
subgoal premises ps proof (cases "MList.lookup oId (State.boundValues (fStateToMState state))
     ")
  case (None)
  moreover obtain compiledObs where "compileFObservation obs state = Some compiledObs" using
       calculation ps wellTypedValueObservationCompiles(2) by fastforce
  moreover obtain compiledCont where "compile newFContract newCtx (state⦇FState.boundValues
       := MList.insert oId 0 (FState.boundValues state)⦈) = Some compiledCont" using ps
       calculation wellTypedCompiles by fastforce
  moreover have "newState = newState⦇boundPubKeys := boundPubKeys (state⦇FState.boundValues
       := MList.insert oId 0 (FState.boundValues state)⦈)⦈" using ps calculation by auto
  moreover have "compile newFContract newCtx newState = Some compiledCont" by (metis
       compileContractOnlyBoundPubKeysMatter(1) calculation(4) calculation(3))
```

```
    moreover have "fStateToMState newState = (fStateToMState state)⦇State.boundValues := MList.
         insert oId 0 (boundValues state)⦈" using ps calculation by auto
    moreover have "fStateToMState newState = (fStateToMState state)⦇State.boundValues := MList.
         insert oId 0 (State.boundValues (fStateToMState state))⦈" using calculation by auto
    moreover have "(evalValue env (fStateToMState state) (Value.Cond compiledObs (Value.
         Constant 1) (Value.Constant 0))) = 0" using ps calculation
         preservationOfValueEvaluation by fastforce
    moreover have "(evalValue env (fStateToMState state) (Value.Cond compiledObs (Value.
         Constant 1) (Value.Constant 0))) = 0" using calculation preservationOfValueEvaluation
         by fastforce
    moreover have "∃newMWarning. (SemanticsTypes.Let oId (Value.Cond compiledObs (Value.
         Constant 1) (Value.Constant 0)) compiledCont, (fStateToMState state), env, warnings,
         payments) → (compiledCont, (fStateToMState state)⦇State.boundValues := MList.insert
         oId 0 (State.boundValues (fStateToMState state))⦈, env, warnings @ newMWarning,
         payments)" by (metis calculation(1) calculation(8) calculation(9) SmallStep.
         LetNoShadow)
    moreover have "∃newMWarning. (SemanticsTypes.Let oId (Value.Cond compiledObs (Value.
         Constant 1) (Value.Constant 0)) compiledCont, (fStateToMState state), env, warnings,
         payments) → (compiledCont, (fStateToMState newState), env, warnings @ newMWarning,
         payments)" using calculation by auto
    ultimately show ?thesis apply (auto)
      by (smt LetObservation.prems(5) \<open>∃newMWarning. (Contract.Let oId (Value.Cond
           compiledObs (Value.Constant 1) (Value.Constant 0)) compiledCont, fStateToMState
           state, env, warnings, payments) → (compiledCont, fStateToMState newState, env,
           warnings @ newMWarning, payments)\<close> fStateToMState.simps option.sel option.
           simps(5) ps(2) ps(3) ps(6) star_step1)
  next
    case (Some x2)
    moreover obtain compiledObs where "compileFObservation obs state = Some compiledObs" using
         calculation ps wellTypedValueObservationCompiles(2) by fastforce
    moreover obtain compiledCont where "compile newFContract newCtx (state⦇FState.boundValues
         := MList.insert oId 0 (FState.boundValues state)⦈) = Some compiledCont" using ps
         calculation wellTypedCompiles by fastforce
    moreover have "newState = newState⦇boundPubKeys := boundPubKeys (state⦇FState.boundValues
         := MList.insert oId 0 (FState.boundValues state)⦈)⦈" using ps calculation by auto
    moreover have "compile newFContract newCtx newState = Some compiledCont" by (metis
         compileContractOnlyBoundPubKeysMatter(1) calculation(3) calculation(4))
    moreover have "fStateToMState newState = (fStateToMState state)⦇State.boundValues := MList.
         insert oId 0 (boundValues state)⦈" using ps calculation by auto
    moreover have "fStateToMState newState = (fStateToMState state)⦇State.boundValues := MList.
         insert oId 0 (State.boundValues (fStateToMState state))⦈" using calculation by auto
    moreover have "(evalValue env (fStateToMState state) (Value.Cond compiledObs (Value.
         Constant 1) (Value.Constant 0))) = 0" using ps calculation
         preservationOfValueEvaluation by fastforce
```

```
      moreover have "(evalValue env (fStateToMState state) (Value.Cond compiledObs (Value.
           Constant 1) (Value.Constant 0))) = 0" using calculation preservationOfValueEvaluation
           by fastforce
      moreover have "∃newMWarning. (SemanticsTypes.Let oId (Value.Cond compiledObs (Value.
           Constant 1) (Value.Constant 0)) compiledCont, (fStateToMState state), env, warnings,
           payments) → (compiledCont, (fStateToMState state)⦇State.boundValues := MList.insert
           oId 0 (State.boundValues (fStateToMState state))⦈, env, warnings @ newMWarning,
           payments)" by (metis calculation(1) calculation(8) calculation(9) SmallStep.LetShadow)
      moreover have "∃newMWarning. (SemanticsTypes.Let oId (Value.Cond compiledObs (Value.
           Constant 1) (Value.Constant 0)) compiledCont, (fStateToMState state), env, warnings,
           payments) → (compiledCont, (fStateToMState newState), env, warnings @ newMWarning,
           payments)" using calculation by auto
      ultimately show ?thesis apply auto
        by (metis (mono_tags) LetObservation.prems(5) \<open>∃newMWarning. (Contract.Let oId (
             Value.Cond compiledObs (Value.Constant 1) (Value.Constant 0)) compiledCont,
             fStateToMState state, env, warnings, payments) → (compiledCont, fStateToMState
             newState, env, warnings @ newMWarning, payments)\<close> fStateToMState.simps option
             .sel option.simps(5) ps(2) ps(3) ps(6) star_step1)
    qed
    done
next
  case (LetPubKey pk res pId)
  then show ?case using star.refl preservationOfPubKeyEvaluation by fastforce
next
  case (AssertTrue obs)
  then show ?case using preservationOfValueEvaluation(2) wellTypedValueObservationCompiles by
       fastforce
next
  case (AssertFalse obs)
  then show ?case using preservationOfValueEvaluation(2) wellTypedValueObservationCompiles by
       fastforce
next
  case (LetC cid params boundCon)
  then show ?case by fastforce
next
  case (UseCFound cid params innerCtx args)
  moreover obtain innerTyCtx where "lookupContractIdParamTypeInformation tyCtx cid = Some (params
       , innerTyCtx)" using calculation lookupContractTypeWellTypedContext by fastforce
  moreover have "wellTypedArguments tyCtx params args" using calculation by auto
  moreover obtain argState where "argumentsToCompilerState params args state = Some argState"
       using calculation wellTypedArgumentsCompiles by fastforce
  moreover have "boundPubKeys argState = boundPubKeys newState" using calculation
       argumentsToContextBoundPubKeysSameAsEvaluateArguments by auto
```

```
  moreover have "newState⦇boundPubKeys := boundPubKeys argState⦈ = newState" using calculation by
        auto
  moreover obtain innerTyCtx where "lookupContractIdParamTypeInformation tyCtx cid = Some (params
       , innerTyCtx)" using calculation lookupContractTypeWellTypedContext by blast
  moreover obtain compiledBody where "compile newFContract newCtx argState = Some compiledBody"
       using calculation wellTypedCompiles typePreservation by fastforce
  moreover have "newState = newState⦇boundPubKeys := (boundPubKeys argState)⦈" using calculation
       by auto
  moreover have "compile newFContract newCtx newState = Some compiledBody" using calculation by (
       metis compileContractOnlyBoundPubKeysMatter(1))
  moreover have "compileArguments params args state compiledBody = Some marloweContract" using
       calculation by auto
  moreover have "∃newMWarnings . (marloweContract, fStateToMState state, env, warnings, payments)
        →*
    (compiledBody, fStateToMState newState, env, newMWarnings, payments)" using calculation
         preservationOfArgumentEvaluation by auto
  ultimately show ?case by auto
qed

(* If a well typed Faustus contract executes to a certain state in any number of steps,
  then the compiled Marlowe contract will execute to the same state.*)
lemma preservationOfSemantics_FaustusStepsImpMarloweSteps:
"(fContract, ctx, state, env, warnings, payments) →f* (newFContract, newCtx, newState, newEnv,
     newWarnings, newPayments) ⟹
  wellTypedContract tyCtx fContract ⟹
  wellTypedContext tyCtx ctx ⟹
  wellTypedState tyCtx state ⟹
  (∃newMWarnings . (the (compile fContract ctx state), fStateToMState state, env, warnings,
       payments) →* (the (compile newFContract newCtx newState), fStateToMState newState, newEnv
       , newMWarnings, newPayments))"
proof (induction arbitrary: tyCtx rule: star.induct[of small_step_reduce, split_format(complete)
     ])
  case (refl fc ctx a a a b)
  then show ?case by auto
next
  case (step fCont ctx s e w p newFCont newCtx newS newE newW newP finalFCont finalCtx finalS
       finalE finalW finalP)
  moreover obtain newTyCtx where "wellTypedContract newTyCtx newFCont ∧ wellTypedContext newTyCtx
        newCtx ∧ wellTypedState newTyCtx newS" using typePreservation step by blast
  moreover obtain firstMarloweContract where "compile fCont ctx s = Some firstMarloweContract"
       using step wellTypedCompiles by blast
  moreover obtain secondMarloweContract finalMarloweContract fmw where "compile newFCont newCtx
       newS = Some secondMarloweContract" and
```

```
    "(secondMarloweContract, fStateToMState newS, newE, newW, newP) →* (finalMarloweContract,
         fStateToMState finalS, finalE, fmw, finalP)" using step calculation wellTypedCompiles by
          (metis option.sel)
  moreover obtain secondMarloweWarnings where "(firstMarloweContract, fStateToMState s, e, w, p)
       →* (secondMarloweContract, fStateToMState newS, newE, secondMarloweWarnings, newP)" using
        step calculation preservationOfSemantics_FaustusStepImpMarloweSteps by presburger
  moreover obtain finalMarloweWarnings where "(secondMarloweContract, fStateToMState newS, newE,
       secondMarloweWarnings, newP) →* (finalMarloweContract, fStateToMState finalS, finalE,
       finalMarloweWarnings, finalP)" using step calculation SmallStep.
       smallStepStarWarningsAreArbitrary by blast
  ultimately show ?case using step apply auto
    by (meson SmallStep.smallStepStarWarningsAreArbitrary star_trans)
qed

lemma preservationOfSemantics_HaltedFaustusImpliesHaltedMarlowe:
"final (fContract, ctx, state, env, warnings, payments) ⟹
  wellTypedContract tyCtx fContract ⟹
  wellTypedContext tyCtx ctx ⟹
  wellTypedState tyCtx state ⟹
  finalMarlowe (the (compile fContract ctx state), fStateToMState state, env, warnings, payments)
       "
  subgoal premises ps proof (cases fContract)
    case Close
    then show ?thesis using ps apply (cases "refundOne (accounts state)", auto simp add:
         finalMarlowe_def final_def)
      by blast
  next
    case (Pay x21 x22 x23 x24 x25)
    then show ?thesis using FaustusSemantics.finalD ps(1) ps(2) ps(3) ps(4) by blast
  next
    case (If x31 x32 x33)
    then show ?thesis using FaustusSemantics.finalD ps(1) ps(2) ps(3) ps(4) by blast
  next
    case (When x41 timeout x43)
    then show ?thesis using ps apply (auto simp add: finalMarlowe_def final_def)
      apply (cases "snd (timeInterval env) ≥ timeout" "fst (timeInterval env) ≥ timeout" rule:
           bool.exhaust[case_product bool.exhaust], auto)
      using surjective_pairing apply blast
        apply (cases "compile x43 ctx state", auto)
      using wellTypedCompiles(1) apply fastforce
        apply (cases "compileCases x41 ctx state", auto)
      using wellTypedCompiles(2) apply fastforce
       apply (cases "compile x43 ctx state", auto)
      using wellTypedCompiles(1) apply fastforce
```

```
      apply (cases "compileCases x41 ctx state", auto)
     using wellTypedCompiles(2) apply fastforce
     apply (cases "compile x43 ctx state", auto)
     using wellTypedCompiles(1) apply fastforce
     apply (cases "compileCases x41 ctx state", auto)
     using wellTypedCompiles(2) by fastforce
  next
    case (Let x51 x52 x53)
    then show ?thesis using FaustusSemantics.finalD ps(1) ps(2) ps(3) ps(4) by blast
  next
    case (LetObservation x61 x62 x63)
    then show ?thesis using FaustusSemantics.finalD ps(1) ps(2) ps(3) ps(4) by blast
  next
    case (LetPubKey x71 x72 x73)
    then show ?thesis using FaustusSemantics.finalD ps(1) ps(2) ps(3) ps(4) by blast
  next
    case (Assert x81 x82)
    then show ?thesis using FaustusSemantics.finalD ps(1) ps(2) ps(3) ps(4) by blast
  next
    case (LetC x91 x92 x93 x94)
    then show ?thesis using FaustusSemantics.finalD ps(1) ps(2) ps(3) ps(4) by blast
  next
    case (UseC x101 x102)
    then show ?thesis using FaustusSemantics.finalD ps(1) ps(2) ps(3) ps(4) by blast
  qed
  done

datatype FaustusReduceStepResult = Reduced FContext ReduceWarning ReduceEffect FState FContract
                          | NotReduced
                          | AmbiguousSlotIntervalReductionError
                          | UnboundIdentifierError
                          | ArgumentEvaluationError

fun reduceContractStep :: "FContext ⇒ Environment ⇒ FState ⇒ FContract ⇒
     FaustusReduceStepResult" where
"reduceContractStep ctx _ state Close =
  (case refundOne (accounts state) of
     Some ((party, token, money), newAccount) ⇒
       let newState = state ⦇ accounts := newAccount ⦈ in
       Reduced ctx ReduceNoWarning (ReduceWithPayment (Payment party (SemanticsTypes.Party party)
             token money)) newState Close
   | None ⇒ NotReduced)" |
"reduceContractStep ctx env state (Pay faustusAccId faustusPayee token val cont) =
```

```
  (case (evalFParty state faustusAccId, evalFPayee state faustusPayee, evalFValue env state val)
       of
    (None, _, _) ⇒ UnboundIdentifierError
    | (_, None, _) ⇒ UnboundIdentifierError
    | (_, _, None) ⇒ UnboundIdentifierError
    | (Some accId, Some payee, Some moneyToPay) ⇒
     (if moneyToPay ≤ 0
       then (let warning = ReduceNonPositivePay accId payee token moneyToPay in
             Reduced ctx warning ReduceNoPayment state cont)
       else (let balance = moneyInAccount accId token (accounts state) in
            (let paidMoney = min balance moneyToPay in
             let newBalance = balance - paidMoney in
             let newAccs = updateMoneyInAccount accId token newBalance (accounts state) in
             let warning = (if paidMoney < moneyToPay
                            then ReducePartialPay accId payee token paidMoney moneyToPay
                            else ReduceNoWarning) in
             let (payment, finalAccs) = giveMoney accId payee token paidMoney newAccs in
             Reduced ctx warning payment (state ⦇ accounts := finalAccs ⦈) cont))))" |
"reduceContractStep ctx env state (If obs cont1 cont2) =
  (case evalFObservation env state obs of
    None ⇒ UnboundIdentifierError
    | Some evaluatedObservation ⇒ (let cont = (if evaluatedObservation then cont1 else cont2) in
       Reduced ctx ReduceNoWarning ReduceNoPayment state cont))" |
"reduceContractStep ctx env state (When _ timeout cont) =
  (let (startSlot, endSlot) = timeInterval env in
   if endSlot < timeout
   then NotReduced
   else (if timeout ≤ startSlot
         then Reduced ctx ReduceNoWarning ReduceNoPayment state cont
         else AmbiguousSlotIntervalReductionError))" |
"reduceContractStep ctx env state (Let valId val cont) =
  (case evalFValue env state val of
    None ⇒ UnboundIdentifierError
    | Some evaluatedValue ⇒ (let boundVals = boundValues state in
      let newState = state ⦇ boundValues := MList.insert valId evaluatedValue boundVals⦈ in
      let warn = case lookup valId boundVals of
                  Some oldVal ⇒ ReduceShadowing valId oldVal evaluatedValue
                | None ⇒ ReduceNoWarning in
      Reduced (ValueAbstraction valId#ctx) warn ReduceNoPayment newState cont))" |
"reduceContractStep ctx env state (LetObservation obsId obs cont) =
  (case evalFObservation env state obs of
    None ⇒ UnboundIdentifierError
    | Some evaluatedObservation ⇒ (let boundVals = boundValues state in
```

```
      let newState = state ⦇ boundValues := MList.insert obsId (if evaluatedObservation then 1
           else 0) boundVals ⦈ in
      Reduced (ObservationAbstraction obsId#ctx) ReduceNoWarning ReduceNoPayment newState cont))"
            |
"reduceContractStep ctx env state (LetPubKey pkId pk cont) =
  (case evalFParty state pk of
    None ⇒ UnboundIdentifierError
    | Some evaluatedPubKey ⇒ (let boundPubKeys = boundPubKeys state in
      let newState = state ⦇ boundPubKeys := MList.insert pkId evaluatedPubKey boundPubKeys ⦈ in
      Reduced (PubKeyAbstraction pkId#ctx) ReduceNoWarning ReduceNoPayment newState cont))" |
"reduceContractStep ctx env state (Assert obs cont) =
  (case evalFObservation env state obs of
    None ⇒ UnboundIdentifierError
    | Some evaluatedObservation ⇒ (let warning = if evaluatedObservation
                then ReduceNoWarning
                else ReduceAssertionFailed
   in Reduced ctx warning ReduceNoPayment state cont))" |
"reduceContractStep ctx env state (LetC cid params boundCon cont) = Reduced (ContractAbstraction
     (cid, params, boundCon)#ctx) ReduceNoWarning ReduceNoPayment state cont" |
"reduceContractStep ctx env state (UseC cid args) = (
  case lookupContractIdAbsInformation ctx cid of
    None ⇒ UnboundIdentifierError
    | Some (params, bodyCont, innerCtx) ⇒ (case evalFArguments env state params args of
      None ⇒ ArgumentEvaluationError
      | Some newState ⇒ Reduced ((paramsToFContext params)@innerCtx) ReduceNoWarning
           ReduceNoPayment newState bodyCont))"

(* If reduce step function reduces a contract, then a small step reduction is possible. *)
lemma reduceStepIsSmallStepReduction:
assumes "reduceContractStep ctx env state contract = Reduced newCtx warning paymentReduceResult
     newState continueContract"
shows "∃ appendPayment .(contract, ctx, state, env, initialWarnings, initialPayments) →f (
     continueContract, newCtx, newState, env, initialWarnings @ [warning], initialPayments @
     appendPayment)"
proof (cases contract paymentReduceResult rule: FContract.exhaust[case_product ReduceEffect.
     exhaust])
  case Close_ReduceNoPayment
  then show ?thesis using assms apply auto
    by (cases "refundOne (accounts state)", auto)
next
  case (Close_ReduceWithPayment reducePayment)
  then show ?thesis using assms apply auto
    by (cases "refundOne (accounts state)", auto)
next
```

```
case (Pay_ReduceNoPayment accId payee token val cont)
then show ?thesis using assms apply auto
  apply (cases "evalFParty state accId" "evalFPayee state payee" "evalFValue env state val"
       rule: option.exhaust[case_product option.exhaust[case_product option.exhaust]], auto)
  apply (split if_split_asm, auto)
   apply (metis FaustusSemantics.small_step_reduce.PayNonPositive append.right_neutral)
  apply (split option.split_asm, auto simp add: min_def)
  subgoal premises ps proof -
    obtain evaluatedPubKey where "evalFParty state accId = Some evaluatedPubKey" using ps by
         auto
    moreover obtain evaluatedPayee where "evalFPayee state payee = Some evaluatedPayee" using
         ps by auto
    moreover obtain evaluatedValue where "evalFValue env state val = Some evaluatedValue" using
          ps by auto
    moreover obtain finalAccs where "giveMoney evaluatedPubKey evaluatedPayee token 0 (MList.
         delete (evaluatedPubKey, token) (FState.accounts state)) = (ReduceNoPayment, finalAccs
         )" using ps calculation apply auto
      by (metis FaustusReduceStepResult.inject ReduceEffect.distinct(1))
    moreover have "(FContract.Pay accId payee token val continueContract, newCtx, state, env,
         initialWarnings, initialPayments) →f (continueContract, newCtx, state⦇FState.accounts
          := finalAccs⦈, env, initialWarnings @ [ReducePartialPay evaluatedPubKey
         evaluatedPayee token 0 evaluatedValue], initialPayments)" using calculation ps by auto
    ultimately show ?thesis using ps by auto
  qed
  apply (split if_split_asm, auto)
  subgoal premises ps proof -
    obtain evaluatedPubKey where "evalFParty state accId = Some evaluatedPubKey" using ps by
         auto
    moreover obtain evaluatedPayee where "evalFPayee state payee = Some evaluatedPayee" using
         ps by auto
    moreover obtain evaluatedValue where "evalFValue env state val = Some evaluatedValue" using
          ps by auto
    moreover obtain availableMoney where "lookup (evaluatedPubKey, token) (FState.accounts
         state) = Some availableMoney" using ps calculation by auto
    moreover obtain finalAccs where "giveMoney evaluatedPubKey evaluatedPayee token
         availableMoney (MList.delete (evaluatedPubKey, token) (FState.accounts state)) = (
         ReduceNoPayment, finalAccs)" using ps calculation apply auto
      by (metis FaustusReduceStepResult.inject ReduceEffect.distinct(1))
    moreover have "(FContract.Pay accId payee token val continueContract, newCtx, state, env,
         initialWarnings, initialPayments) →f (continueContract, newCtx, state⦇FState.accounts
          := finalAccs⦈, env, initialWarnings @ [ReducePartialPay evaluatedPubKey
         evaluatedPayee token availableMoney evaluatedValue], initialPayments)" using
         calculation ps by auto
    ultimately show ?thesis using ps by auto
```

```
    qed
    subgoal premises ps proof -
      obtain evaluatedPubKey where "evalFParty state accId = Some evaluatedPubKey" using ps by
           auto
      moreover obtain evaluatedPayee where "evalFPayee state payee = Some evaluatedPayee" using
           ps by auto
      moreover obtain evaluatedValue where "evalFValue env state val = Some evaluatedValue" using
            ps by auto
      moreover obtain availableMoney where "lookup (evaluatedPubKey, token) (FState.accounts
           state) = Some availableMoney" using ps calculation by auto
      moreover obtain finalAccs where "giveMoney evaluatedPubKey evaluatedPayee token
           availableMoney (MList.delete (evaluatedPubKey, token) (FState.accounts state)) = (
           ReduceNoPayment, finalAccs)" using ps calculation apply auto
        by (metis FaustusReduceStepResult.inject ReduceEffect.distinct(1))
      moreover have "(FContract.Pay accId payee token val continueContract, newCtx, state, env,
           initialWarnings, initialPayments) →f (continueContract, newCtx, state(FState.accounts
            := finalAccs), env, initialWarnings @ [ReduceNoWarning], initialPayments)" using
           calculation ps by auto
      ultimately show ?thesis using ps by auto
    qed
    done
next
  case (Pay_ReduceWithPayment accId payee token val cont payment)
  then show ?thesis using assms apply auto
    apply (cases "evalFParty state accId" "evalFPayee state payee" "evalFValue env state val"
         rule: option.exhaust[case_product option.exhaust[case_product option.exhaust]], auto)
    apply (split if_split_asm, auto)
    apply (split option.split_asm, auto simp add: min_def)
     (*apply (auto split: prod.split_asm)*)
    subgoal premises ps proof -
      obtain evaluatedPubKey where "evalFParty state accId = Some evaluatedPubKey" using ps by
           auto
      moreover obtain evaluatedPayee where "evalFPayee state payee = Some evaluatedPayee" using
           ps by auto
      moreover obtain evaluatedValue where "evalFValue env state val = Some evaluatedValue" using
            ps by auto
      moreover obtain finalAccs where "giveMoney evaluatedPubKey evaluatedPayee token 0 (MList.
           delete (evaluatedPubKey, token) (FState.accounts state)) = (ReduceWithPayment payment,
            finalAccs)" using ps calculation apply auto
        by (meson FaustusReduceStepResult.inject ReduceEffect.inject)
      moreover have "(FContract.Pay accId payee token val continueContract, newCtx, state, env,
           initialWarnings, initialPayments) →f (continueContract, newCtx, state(FState.accounts
            := finalAccs), env, initialWarnings @ [ReducePartialPay evaluatedPubKey
           evaluatedPayee token 0 evaluatedValue], initialPayments @ [payment])" using
```

```
        calculation ps PayPositivePartialWithPayment by auto
  ultimately show ?thesis using ps apply auto
    by (metis \<open>(FContract.Pay accId payee token val continueContract, newCtx, state,
          env, initialWarnings, initialPayments) →f (continueContract, newCtx, state ⦇FState.
          accounts := finalAccs⦈, env, initialWarnings @ [ReducePartialPay evaluatedPubKey
          evaluatedPayee token 0 evaluatedValue], initialPayments @ [payment])\<close>)
qed
apply (auto split: if_split_asm)
 (*apply (split prod.split_asm, auto)*)
subgoal premises ps proof -
  obtain evaluatedPubKey where "evalFParty state accId = Some evaluatedPubKey" using ps by
        auto
  moreover obtain evaluatedPayee where "evalFPayee state payee = Some evaluatedPayee" using
        ps by auto
  moreover obtain evaluatedValue where "evalFValue env state val = Some evaluatedValue" using
         ps by auto
  moreover have "evaluatedValue > 0" using ps calculation by auto
  moreover obtain moneyToPay where "moneyInAccount evaluatedPubKey token (FState.accounts
        state) = moneyToPay" using ps calculation by auto
  moreover obtain availableMoney where "lookup (evaluatedPubKey, token) (FState.accounts
        state) = Some availableMoney" using ps calculation by auto
  moreover have "moneyToPay = availableMoney" using calculation ps by auto
  moreover obtain finalAccs where "giveMoney evaluatedPubKey evaluatedPayee token
        availableMoney (MList.delete (evaluatedPubKey, token) (FState.accounts state)) = (
        ReduceWithPayment payment, finalAccs)" using ps calculation apply auto
    by (meson FaustusReduceStepResult.inject ReduceEffect.inject)
  moreover have "newCtx = ctx" using ps calculation apply auto
    by (metis FaustusReduceStepResult.inject)
  moreover have "evaluatedValue > moneyToPay" using ps calculation by auto
  moreover have "(FContract.Pay accId payee token val continueContract, newCtx, state, env,
        initialWarnings, initialPayments) →f (continueContract, newCtx, state⦇FState.accounts
         := finalAccs⦈, env, initialWarnings @ [ReducePartialPay evaluatedPubKey
        evaluatedPayee token availableMoney evaluatedValue], initialPayments @ [payment])"
        using calculation by auto
  ultimately show ?thesis using ps apply auto
    by (metis FaustusReduceStepResult.inject \<open>(FContract.Pay accId payee token val
          continueContract, newCtx, state, env, initialWarnings, initialPayments) →f (
          continueContract, newCtx, state ⦇FState.accounts := finalAccs⦈, env, initialWarnings
           @ [ReducePartialPay evaluatedPubKey evaluatedPayee token availableMoney
          evaluatedValue], initialPayments @ [payment])\<close>)
qed
subgoal premises ps proof -
  obtain evaluatedPubKey where "evalFParty state accId = Some evaluatedPubKey" using ps by
        auto
```

```
    moreover obtain evaluatedPayee where "evalFPayee state payee = Some evaluatedPayee" using
        ps by auto
    moreover obtain evaluatedValue where "evalFValue env state val = Some evaluatedValue" using
         ps by auto
    moreover obtain availableMoney where "lookup (evaluatedPubKey, token) (FState.accounts
        state) = Some availableMoney" using ps calculation by auto
    moreover obtain finalAccs where "giveMoney evaluatedPubKey evaluatedPayee token
        availableMoney (MList.delete (evaluatedPubKey, token) (FState.accounts state)) = (
        ReduceWithPayment payment, finalAccs)" using ps calculation apply auto
      by (meson FaustusReduceStepResult.inject ReduceEffect.inject)
    moreover have "(FContract.Pay accId payee token val continueContract, newCtx, state, env,
        initialWarnings, initialPayments) →f (continueContract, newCtx, state⦇FState.accounts
         := finalAccs⦈, env, initialWarnings @ [ReduceNoWarning], initialPayments @ [payment])
        " using calculation ps PayPositivePartialWithPayment by auto
    ultimately show ?thesis using ps apply auto
      by (metis FaustusReduceStepResult.inject \<open>(FContract.Pay accId payee token val
          continueContract, newCtx, state, env, initialWarnings, initialPayments) →f (
          continueContract, newCtx, state ⦇FState.accounts := finalAccs⦈, env, initialWarnings
           @ [ReduceNoWarning], initialPayments @ [payment])\<close> ps(8))
  qed
  subgoal premises ps proof -
    obtain evaluatedPubKey where "evalFParty state accId = Some evaluatedPubKey" using ps by
        auto
    moreover obtain evaluatedPayee where "evalFPayee state payee = Some evaluatedPayee" using
        ps by auto
    moreover obtain evaluatedValue where "evalFValue env state val = Some evaluatedValue" using
         ps by auto
    moreover obtain availableMoney where "lookup (evaluatedPubKey, token) (FState.accounts
        state) = Some availableMoney" using ps calculation by auto
    moreover obtain finalAccs where "giveMoney evaluatedPubKey evaluatedPayee token
        evaluatedValue (MList.insert (evaluatedPubKey, token) (availableMoney - evaluatedValue
        ) (FState.accounts state)) = (ReduceWithPayment payment, finalAccs)" using ps
        calculation apply auto
      by (meson FaustusReduceStepResult.inject ReduceEffect.inject)
    moreover have "(FContract.Pay accId payee token val continueContract, newCtx, state, env,
        initialWarnings, initialPayments) →f (continueContract, newCtx, state⦇FState.accounts
         := finalAccs⦈, env, initialWarnings @ [ReduceNoWarning], initialPayments @ [payment])
        " using calculation ps by auto
    ultimately show ?thesis using ps apply auto
      by (metis \<open>(FContract.Pay accId payee token val continueContract, newCtx, state,
          env, initialWarnings, initialPayments) →f (continueContract, newCtx, state ⦇FState.
          accounts := finalAccs⦈, env, initialWarnings @ [ReduceNoWarning], initialPayments @
          [payment])\<close>)
  qed
```

```
    done
next
  case (If_ReduceNoPayment obs x32 x33)
  then show ?thesis using assms apply (auto split: option.split_asm)
     apply (metis IfTrue append_Nil2)
    by (metis IfFalse append.right_neutral)
next
  case (If_ReduceWithPayment x31 x32 x33 x2)
  then show ?thesis using assms by (auto split: option.split_asm)
next
  case (When_ReduceNoPayment x41 timeout x43)
  then show ?thesis using assms apply auto
    subgoal premises ps proof (cases "timeInterval env")
      case (Pair ss es)
      then show ?thesis using ps apply auto
        apply (cases "es < timeout", auto)
        apply (cases "timeout ≤ ss", auto)
        by (smt WhenTimeout append_Nil2)
    qed
    done
next
  case (When_ReduceWithPayment x41 x42 x43 x2)
  then show ?thesis using assms by (auto split: if_split_asm prod.split_asm)
next
  case (Let_ReduceNoPayment vid x52 x53)
  then show ?thesis using assms apply (auto split: option.split_asm)
    apply (cases "lookup vid (boundValues state)", auto)
     apply (metis LetNoShadow append_Nil2)
    by (metis (no_types, lifting) LetShadow append_Nil2)
next
  case (Let_ReduceWithPayment vid x52 x53 x2)
  then show ?thesis using assms apply (auto split: option.split_asm)
    by (cases "lookup vid (boundValues state)", auto)
next
  case (LetObservation_ReduceNoPayment obsId x52 x53)
  then show ?thesis using assms apply (simp split: option.split_asm)
    by (metis (no_types, lifting) LetObservation append_Nil2)
next
  case (LetObservation_ReduceWithPayment vid x52 x53 x2)
  then show ?thesis using assms by (auto split: option.split_asm)
next
  case (LetPubKey_ReduceNoPayment pkId x52 x53)
  then show ?thesis using assms apply (simp split: option.split_asm)
    by (metis (no_types, lifting) LetPubKey append_Nil2)
```

```
next
  case (LetPubKey_ReduceWithPayment vid x52 x53 x2)
  then show ?thesis using assms by (auto split: option.split_asm)
next
  case (Assert_ReduceNoPayment x61 x62)
  then show ?thesis using assms apply (auto split: option.split_asm)
     apply (metis AssertTrue append.right_neutral)
    by (metis AssertFalse append_Nil2)
next
  case (Assert_ReduceWithPayment x61 x62 x2)
  then show ?thesis using assms by (auto split: option.split_asm)
next
  case (LetC_ReduceNoPayment cid x72 x73)
  then show ?thesis using assms apply auto
    by (metis LetC append.right_neutral)
next
  case (LetC_ReduceWithPayment cid x72 x73 x2)
  then show ?thesis using assms by auto
next
  case (UseC_ReduceNoPayment cid)
  then show ?thesis using assms apply (auto split: option.split_asm)
    by (metis UseCFound append_Nil2)
next
  case (UseC_ReduceWithPayment cid x2)
  then show ?thesis using assms by (auto split: option.split_asm)
qed

(* If a small step reduction is possible, then the reduce step function will reduce the contract
     *)
lemma smallStepReductionImpReduceStep:
assumes "cs = (contract, ctx, state, env, initialWarnings, initialPayments) ∧
    cs' = (continueContract, newCtx, newState, newEnv, newWarnings, newPayments) ∧
    cs →f cs'"
shows "∃warning paymentReduceResult . reduceContractStep ctx env state contract = Reduced newCtx
     warning paymentReduceResult newState continueContract"
proof (cases contract)
  case Close
  then show ?thesis using assms by auto
next
  case (Pay accId payee token val cont)
  then show ?thesis using assms apply auto
       apply (auto split: option.split)
            apply metis
           apply metis
```

```
          apply metis
         apply metis
        apply metis
       apply metis
      apply metis
     apply metis
    by metis
next
  case (If x31 x32 x33)
  then show ?thesis using assms by auto
next
  case (When cases timeout cont)
  then show ?thesis using assms by auto
next
  case (Let x51 x52 x53)
  then show ?thesis using assms by auto
next
  case (LetObservation x51 x52 x53)
  then show ?thesis using assms by auto
next
  case (LetPubKey x51 x52 x53)
  then show ?thesis using assms by auto
next
  case (Assert x61 x62)
  then show ?thesis using assms by auto
next
  case (LetC x71 x72 x73)
  then show ?thesis using assms by auto
next
  case (UseC x8)
  then show ?thesis using assms by (auto split: option.split)
qed

lemma wellTypedContractNeverGetsUnboundIdentifierError:
"wellTypedContract tyCtx contract ⟹
  wellTypedContext tyCtx ctx ⟹
  wellTypedState tyCtx state ⟹
  reduceContractStep ctx env state contract ≠ UnboundIdentifierError"
  apply (cases contract, auto)
          apply (cases "refundOne (accounts state)", auto)
         apply (auto split: option.split_asm if_split_asm prod.split_asm simp add:
              wellTypedPartyEvaluates wellTypedValueObservationEvaluates wellTypedPayeeEvaluates
              min_def)
       apply (meson FaustusReduceStepResult.distinct(5))
```

```
      apply (meson FaustusReduceStepResult.distinct(5))
     apply (meson FaustusReduceStepResult.distinct(5))
    apply (meson FaustusReduceStepResult.distinct(5))
   apply (meson FaustusReduceStepResult.distinct(5))
  using lookupContractAbstractionWellTypedContext by fastforce

lemma wellTypedContractNeverGetsArgumentEvaluationError:
"wellTypedContract tyCtx contract ⟹
  wellTypedContext tyCtx ctx ⟹
  wellTypedState tyCtx state ⟹
  reduceContractStep ctx env state contract ≠ ArgumentEvaluationError"
  apply (cases contract, auto)
          apply (cases "refundOne (accounts state)", auto)
         apply (auto split: option.split_asm if_split_asm prod.split_asm simp add:
              wellTypedPartyEvaluates wellTypedValueObservationEvaluates wellTypedPayeeEvaluates
              min_def)
       apply (meson FaustusReduceStepResult.distinct)
      apply (meson FaustusReduceStepResult.distinct)
     apply (meson FaustusReduceStepResult.distinct)
    apply (meson FaustusReduceStepResult.distinct)
   apply (meson FaustusReduceStepResult.distinct)
  by (metis lookupContractTypeWellTypedContext old.prod.inject option.inject
       wellTypedArgumentsExecute)


datatype FaustusReduceResult = ContractQuiescent "ReduceWarning list" "Payment list"
                                          FState FContract
                                          | RRAmbiguousSlotIntervalError
                                          | RRUnboundIdentifierError
                                          | RRArgumentEvaluationError

function (sequential) reductionLoop :: "FContext ⇒ Environment ⇒ FState ⇒ FContract ⇒
     ReduceWarning list ⇒
                                        Payment list ⇒ FaustusReduceResult" where
"reductionLoop ctx env state contract warnings payments =
  (case reduceContractStep ctx env state contract of
     Reduced newCtx warning effect newState ncontract ⇒
       let newWarnings = (if warning = ReduceNoWarning
                          then warnings
                          else warning # warnings) in
       let newPayments = (case effect of
                            ReduceWithPayment payment ⇒ payment # payments
                          | ReduceNoPayment ⇒ payments) in
       reductionLoop newCtx env newState ncontract newWarnings newPayments
```

```
  | AmbiguousSlotIntervalReductionError ⇒ RRAmbiguousSlotIntervalError
  | NotReduced ⇒ ContractQuiescent (rev warnings) (rev payments) state contract
  | UnboundIdentifierError ⇒ RRUnboundIdentifierError
  | ArgumentEvaluationError ⇒ RRArgumentEvaluationError)"
 by pat_completeness auto

lemma reductionLoop_termination_aux1:
"FaustusSemantics.reduceContractStep ctx env state contract =
      FaustusReduceStepResult.Reduced newCtx x12 x13 x14 newContract ⟹
      ¬ fContractSize newContract + (fold (+) (map (abstractionInformationSize) newCtx) 0)
         < fContractSize contract + (fold (+) (map (abstractionInformationSize) ctx) 0) ⟹
      fContractSize newContract + (fold (+) (map (abstractionInformationSize) newCtx) 0) =
      fContractSize contract + (fold (+) (map (abstractionInformationSize) ctx) 0)"
 subgoal premises ps proof (cases contract)
   case Close
   then show ?thesis using ps by (cases "refundOne (accounts state)", auto)
 next
   case (Pay x21 x22 x23 x24 x25)
   then show ?thesis using ps apply (auto split: option.split_asm if_split_asm prod.split_asm
        simp add: min_def)
        apply (metis FaustusReduceStepResult.inject lessI)
       apply (metis FaustusReduceStepResult.inject lessI)
      apply (metis FaustusReduceStepResult.inject lessI)
     by (metis FaustusReduceStepResult.inject lessI)
 next
   case (If x31 x32 x33)
   then show ?thesis using ps by (auto split: option.split_asm simp add: le_imp_less_Suc)
 next
 case (When x41 x42 x43)
   then show ?thesis using ps by (cases "timeInterval env" "snd (timeInterval env) < x42" "x42 ≤
         fst (timeInterval env)" rule: prod.exhaust[case_product bool.exhaust[case_product bool.
        exhaust]], auto)
 next
   case (Let x51 x52 x53)
   then show ?thesis using ps by (cases "lookup x51 (boundValues state)", auto split: option.
        split_asm)
 next
   case (LetObservation obsId obs cont)
   then show ?thesis using ps by (auto split: option.split_asm)
 next
   case (LetPubKey pkId pk cont)
   then show ?thesis using ps by (auto split: option.split_asm)
 next
   case (Assert x61 x62)
```

```
    then show ?thesis using ps by (auto split: option.split_asm)
  next
    case (LetC x71 x72 x73)
    then show ?thesis using ps by (auto simp add: fold_plus_sum_list_rev)
  next
    case (UseC x8)
    then show ?thesis using ps apply (auto split: option.split_asm)
      using paramsToFContextNoSize lookupContractCompilerContextSmaller by fastforce
  qed
  done

lemma reductionLoop_termination_aux2:
"FaustusSemantics.reduceContractStep ctx env state contract =
       FaustusReduceStepResult.Reduced newCtx x12 x13 x14 x15 ⟹
       ¬ fContractSize x15 + (fold (+) (map (abstractionInformationSize) newCtx) 0)
          < fContractSize contract + (fold (+) (map (abstractionInformationSize) ctx) 0) ⟹
       ¬ length newCtx < length ctx ⟹ length newCtx = length ctx"
  subgoal premises ps proof (cases contract)
    case Close
    then show ?thesis using ps by (cases "refundOne (accounts state)", auto)
  next
    case (Pay x21 x22 x23 x24 x25)
    then show ?thesis using ps apply (auto split: option.split_asm if_split_asm prod.split_asm
         simp add: min_def)
         apply (metis FaustusReduceStepResult.inject)
        apply (metis FaustusReduceStepResult.inject)
       apply (metis FaustusReduceStepResult.inject)
      by (metis FaustusReduceStepResult.inject)
  next
    case (If x31 x32 x33)
    then show ?thesis using ps by (auto split: option.split_asm)
  next
  case (When x41 x42 x43)
    then show ?thesis using ps by (cases "timeInterval env" "snd (timeInterval env) < x42" "x42 ≤
          fst (timeInterval env)" rule: prod.exhaust[case_product bool.exhaust[case_product bool.
         exhaust]], auto)
  next
    case (Let x51 x52 x53)
    then show ?thesis using ps apply (auto split: option.split_asm)
      by (cases "lookup x51 (boundValues state)", auto)
  next
    case (LetObservation obsId obs cont)
    then show ?thesis using ps by (auto split: option.split_asm)
  next
```

```
    case (LetPubKey pkId pk cont)
    then show ?thesis using ps by (auto split: option.split_asm)
  next
    case (Assert x61 x62)
    then show ?thesis using ps by (auto split: option.split_asm)
  next
    case (LetC x71 x72 x73)
    then show ?thesis using ps by (auto simp add: fold_plus_sum_list_rev)
  next
    case (UseC x8)
    then show ?thesis using ps apply (auto split: option.split_asm)
      using paramsToFContextNoSize lookupContractCompilerContextSmaller by fastforce
  qed
  done

lemma reductionLoop_termination_aux3:
"FaustusSemantics.reduceContractStep ctx env state contract =
       FaustusReduceStepResult.Reduced newCtx x12 x13 x14 x15 ⟹
       ¬ fContractSize x15 + (fold (+) (map (abstractionInformationSize) newCtx) 0)
          < fContractSize contract + (fold (+) (map (abstractionInformationSize) ctx) 0) ⟹
       ¬ length newCtx < length ctx ⟹
       (\<And>ctx env state contract newCtx x12 x13 x14 newContract.
           FaustusSemantics.reduceContractStep ctx env state contract =
           FaustusReduceStepResult.Reduced newCtx x12 x13 x14 newContract ⟹
           ¬ fContractSize newContract + (fold (+) (map (abstractionInformationSize) newCtx) 0)
              < fContractSize contract + (fold (+) (map (abstractionInformationSize) ctx) 0) ⟹
           fContractSize newContract + (fold (+) (map (abstractionInformationSize) newCtx) 0) =
           fContractSize contract + (fold (+) (map (abstractionInformationSize) ctx) 0)) ⟹
       (\<And>ctx env state contract newCtx x12 x13 x14 x15.
           FaustusSemantics.reduceContractStep ctx env state contract =
           FaustusReduceStepResult.Reduced newCtx x12 x13 x14 x15 ⟹
           ¬ fContractSize x15 + (fold (+) (map (abstractionInformationSize) newCtx) 0)
              < fContractSize contract + (fold (+) (map (abstractionInformationSize) ctx) 0) ⟹
           ¬ length newCtx < length ctx ⟹ length newCtx = length ctx) ⟹
       length (accounts x14) < length (accounts state)"
  subgoal premises ps proof (cases contract)
    case Close
    then show ?thesis using ps apply auto
      apply (induction "accounts state", auto)
      by (cases "refundOne (accounts state)", auto simp add: refundOneShortens)
  next
    case (Pay x21 x22 x23 x24 x25)
    then show ?thesis using ps apply (auto split: option.split_asm if_split_asm prod.split_asm
          simp add: min_def)
```

```
        apply (metis FaustusReduceStepResult.inject lessI)
       apply (metis FaustusReduceStepResult.inject lessI)
      apply (metis FaustusReduceStepResult.inject lessI)
     by (metis FaustusReduceStepResult.inject lessI)
  next
    case (If x31 x32 x33)
    then show ?thesis using ps by (auto split: option.split_asm simp add: le_imp_less_Suc)
  next
    case (When x41 x42 x43)
    then show ?thesis using ps by (cases "timeInterval env" "snd (timeInterval env) < x42" "x42 ≤
          fst (timeInterval env)" rule: prod.exhaust[case_product bool.exhaust[case_product bool.
        exhaust]], auto)
  next
    case (Let x51 x52 x53)
    then show ?thesis using ps apply (auto split: option.split_asm)
      by (cases "lookup x51 (boundValues state)", auto)
  next
    case (LetObservation obsId obs cont)
    then show ?thesis using ps by (auto split: option.split_asm)
  next
    case (LetPubKey pkId pk cont)
    then show ?thesis using ps by (auto split: option.split_asm)
  next
    case (Assert x61 x62)
    then show ?thesis using ps by (auto split: option.split_asm)
  next
    case (LetC x71 x72 x73)
    then show ?thesis using ps by (auto simp add: fold_plus_sum_list_rev)
  next
    case (UseC x8)
    then show ?thesis using ps apply (auto split: option.split_asm)
      using paramsToFContextNoSize lookupContractCompilerContextSmaller by fastforce
  qed
  done

termination reductionLoop
  apply (relation "measures [(λ(ctx, env, state, contract, _) . ((fContractSize contract)) + (
       fold (+) (map (abstractionInformationSize) ctx) 0)),
    (λ(ctx, _, state, contract, _) . (length ctx)),
    (λ(ctx, _, state, contract, _) . (length (accounts state)))]", auto)
  using reductionLoop_termination_aux1
    reductionLoop_termination_aux2
    reductionLoop_termination_aux3 by auto
```

```
fun reduceContractUntilQuiescent :: "FContext ⇒ Environment ⇒ FState ⇒ FContract ⇒
     FaustusReduceResult" where
"reduceContractUntilQuiescent ctx env state contract = reductionLoop ctx env state contract []
     []"

datatype ApplyResult = Applied ApplyWarning FState FContract
                     | ApplyNoMatchError

fun applyCases :: "Environment ⇒ FState ⇒ Input ⇒ FCase list ⇒ ApplyResult" where
"applyCases env state (IDeposit accId1 party1 tok1 amount)
            (Case (Deposit accId2 party2 tok2 val) cont # rest) =
  (if (Some accId1 = evalFParty state accId2 ∧ Some party1 = evalFParty state party2 ∧ tok1 =
       tok2
        ∧ Some amount = evalFValue env state val)
   then let warning = if amount > 0
                      then ApplyNoWarning
                      else ApplyNonPositiveDeposit party1 accId1 tok2 amount in
        let newState = state ⦇ accounts := addMoneyToAccount accId1 tok1 amount (accounts state)
             ⦈ in
        Applied warning newState cont
   else applyCases env state (IDeposit accId1 party1 tok1 amount) rest)" |
"applyCases env state (IChoice choId1 choice)
            (Case (Choice choId2 bounds) cont # rest) =
  (if (Some choId1 = evalFChoiceId state choId2 ∧ inBounds choice bounds)
   then let newState = state ⦇ choices := MList.insert choId1 choice (choices state) ⦈ in
        Applied ApplyNoWarning newState cont
   else applyCases env state (IChoice choId1 choice) rest)" |
"applyCases env state INotify (Case (Notify obs) cont # rest) =
  (if evalFObservation env state obs = Some True
   then Applied ApplyNoWarning state cont
   else applyCases env state INotify rest)" |
"applyCases env state (IDeposit accId1 party1 tok1 amount) (Cons _ rest) =
  applyCases env state (IDeposit accId1 party1 tok1 amount) rest" |
"applyCases env state (IChoice choId1 choice) (Cons _ rest) =
  applyCases env state (IChoice choId1 choice) rest" |
"applyCases env state INotify (Cons _ rest) =
  applyCases env state INotify rest" |
"applyCases env state acc Nil = ApplyNoMatchError"

lemma preservationOfApplyCases_FaustusErrorImpliesMarloweError:
"applyCases env state inp cases = ApplyNoMatchError ⟹
  wellTypedCases tyCtx cases ⟹
  wellTypedState tyCtx state ⟹
  compileCases cases ctx state = Some marloweCases ⟹
```

```
  Semantics.applyCases env (fStateToMState state) inp marloweCases = Semantics.ApplyNoMatchError"
proof (induction cases arbitrary: marloweCases)
  case Nil
  then show ?case by auto
next
  case (Cons firstCase cases)
  then show ?case apply auto
    subgoal premises ps proof (cases firstCase inp rule: FCase.exhaust[case_product Input.exhaust
        ])
      case (Case_IDeposit firstAct firstCont x11 x12 x13 x14)
      then show ?thesis using ps apply (cases firstAct, auto split: if_split_asm option.split_asm
          )
         apply (metis compileFParty.elims evalFParty.simps)
        apply (metis compileFParty.elims evalFParty.simps)
       by (simp add: preservationOfValueEvaluation_MarloweImpliesFaustus(1))
    next
      case (Case_IChoice firstAct firstCont x21 x22)
      then show ?thesis using ps by (cases firstAct, auto split: if_split_asm option.split_asm
          simp add: preservationOfChoiceIdEvaluation)
    next
      case (Case_INotify firstAct x2)
      then show ?thesis using ps by (cases firstAct, auto split: if_split_asm option.split_asm
          simp add: preservationOfValueEvaluation_MarloweImpliesFaustus(2))
    qed
    done
qed

lemma preservationOfApplyCases_FaustusAppliedImpliesMarloweApplied:
"applyCases env state inp cases = Applied warn newState newCont ⟹
  wellTypedCases tyCtx cases ⟹
  wellTypedState tyCtx state ⟹
  compileCases cases ctx state = Some marloweCases ⟹
  Semantics.applyCases env (fStateToMState state) inp marloweCases = Semantics.Applied warn (
       fStateToMState newState) (the (compile newCont ctx newState))"
proof (induction cases arbitrary: marloweCases warn newState newCont)
  case Nil
  then show ?case by auto
next
  case (Cons firstCase cases)
  then show ?case apply auto
    subgoal premises ps proof (cases firstCase inp rule: FCase.exhaust[case_product Input.exhaust
        ])
      case (Case_IDeposit firstAct firstCont inPk1 inPk2 inToken inAmt)
      then show ?thesis using ps apply (auto split: option.split_asm)
```

```
subgoal premises ps proof (cases firstAct)
  case (Deposit depPk1 depPk2 depToken depValue)
  then show ?thesis using ps apply (auto split: if_split_asm)
          apply (split option.split_asm, simp)
          apply (split option.split_asm, simp)
          apply (split option.split_asm, simp)
          apply (split option.split_asm, simp)
          apply (auto simp add: compileAccountsArbitrary(1)
               preservationOfPubKeyEvaluation)[1]
          apply (metis fStateToMState.simps option.sel preservationOfValueEvaluation(1))
         apply (split option.split_asm, simp)
         apply (split option.split_asm, simp)
          apply (split option.split_asm, simp)
          apply (split option.split_asm, simp)
          apply (auto simp add: compileAccountsArbitrary(1)
               preservationOfPubKeyEvaluation)[1]
          apply (metis fStateToMState.simps option.sel preservationOfValueEvaluation(1))
         apply (split option.split_asm, simp)
         apply (split option.split_asm, simp)
         apply (split option.split_asm, simp)
         apply (split option.split, simp)
         apply (auto simp add: compileAccountsArbitrary(1) preservationOfPubKeyEvaluation
              )[1]
         apply (metis fStateToMState.simps option.sel preservationOfValueEvaluation(1))
        apply (split option.split_asm, simp)
        apply (split option.split_asm, simp)
        apply (split option.split_asm, simp)
        apply (auto simp add: compileAccountsArbitrary(1) preservationOfPubKeyEvaluation)
             [1]
        apply (metis fStateToMState.simps option.sel preservationOfValueEvaluation(1))
       apply (split option.split_asm, simp)
       apply (split option.split_asm, simp)
       apply (split option.split_asm, simp)
       apply (auto simp add: compileAccountsArbitrary(1) preservationOfPubKeyEvaluation)
            [1]
      apply (split option.split_asm, simp)
      apply (split option.split_asm, simp)
      apply (split option.split_asm, simp)
      apply (auto simp add: compileAccountsArbitrary(1) preservationOfPubKeyEvaluation)
           [1]
     apply (split option.split_asm, simp)
     apply (split option.split_asm, simp)
     apply (split option.split_asm, simp)
     apply (auto simp add: compileAccountsArbitrary(1) preservationOfPubKeyEvaluation)[1]
```

```
        apply (split option.split_asm, simp)
        apply (split option.split_asm, simp)
        apply (split option.split_asm, simp)
        by (auto split: option.split_asm simp add:
             preservationOfValueEvaluation_MarloweImpliesFaustus(1))
    next
      case (Choice choiceId x22)
      then show ?thesis using ps by (auto split: option.split_asm)
    next
      case (Notify x3)
      then show ?thesis using ps by (auto split: option.split_asm)
    qed
    done
next
  case (Case_IChoice firstAct firstCont inChoice inNum)
  then show ?thesis using ps apply (auto split: option.split_asm)
    subgoal premises ps proof (cases firstAct)
      case (Deposit depPk1 depPk2 depToken depValue)
      then show ?thesis using ps by (auto split: option.split_asm)
    next
      case (Choice choiceId choiceBound)
      then show ?thesis using ps apply (auto split: if_split_asm option.split_asm simp add:
           preservationOfChoiceIdEvaluation)
        by (metis compileContractOnlyBoundPubKeysMatter(1) FState.surjective FState.
             update_convs(2) FState.update_convs(4) option.sel)
    next
      case (Notify x3)
      then show ?thesis using ps by (auto split: option.split_asm)
    qed
    done
next
  case (Case_INotify firstAct firstCont)
  then show ?thesis using ps apply (auto split: option.split_asm)
    subgoal premises ps proof (cases firstAct)
      case (Deposit depPk1 depPk2 depToken depValue)
      then show ?thesis using ps by (auto split: option.split_asm)
    next
      case (Choice choiceId choiceBound)
      then show ?thesis using ps by (auto split: option.split_asm)
    next
      case (Notify x3)
      then show ?thesis using ps by (auto split: if_split_asm option.split_asm simp add:
           preservationOfValueEvaluation_MarloweImpliesFaustus(2))
    qed
```

```
        done
    qed
    done
qed

lemma preservationOfApplyCases_MarloweErrorImpliesFaustusError:
"wellTypedCases tyCtx cases ⟹
  wellTypedState tyCtx state ⟹
  wellTypedContext tyCtx ctx ⟹
  Semantics.applyCases env (fStateToMState state) inp (the (compileCases cases ctx state)) =
       Semantics.ApplyNoMatchError ⟹
  applyCases env state inp cases = ApplyNoMatchError"
proof (induction cases)
  case Nil
  then show ?case by auto
next
  case (Cons firstCase cases)
  then show ?case apply auto
    subgoal premises ps proof (cases firstCase inp rule: FCase.exhaust[case_product Input.exhaust
         ])
      case (Case_IDeposit firstAct firstCont x11 x12 x13 x14)
      then show ?thesis using ps apply (cases firstAct, auto split: if_split_asm option.split_asm
            simp add: preservationOfValueEvaluation_MarloweImpliesFaustus(1))
        using wellTypedCompiles(1) apply fastforce
        using wellTypedCompiles(1) apply fastforce
        using wellTypedCompiles(1) apply fastforce
        using wellTypedCompiles(1) apply fastforce
        using wellTypedCompiles(1) apply fastforce
        using wellTypedCompiles(2) apply fastforce
        using wellTypedPartyCompiles apply fastforce
        using wellTypedPartyCompiles apply fastforce
        using wellTypedPartyCompiles apply fastforce
        using wellTypedPartyCompiles apply fastforce
        using wellTypedPartyCompiles apply fastforce
        using wellTypedPartyCompiles apply fastforce
        using wellTypedPartyCompiles apply fastforce
        using wellTypedPartyCompiles apply fastforce
        using wellTypedPartyCompiles apply fastforce
        using wellTypedPartyCompiles apply fastforce
        using wellTypedValueObservationCompiles(1) apply fastforce
        using wellTypedValueObservationCompiles(1) apply fastforce
        using wellTypedValueObservationCompiles(1) apply fastforce
        using wellTypedValueObservationCompiles(1) apply fastforce
        using wellTypedValueObservationCompiles(1) apply fastforce
```

```
            apply (metis compileFParty.elims compileFParty.simps(2) evalFParty.simps option.
                 inject wellTypedParty.simps(1) wellTypedParty.simps(2) wellTypedParty.simps(3)
                 )
           apply (metis compileFParty.elims evalFParty.simps option.inject)
       using wellTypedCompiles(1) apply fastforce
       using wellTypedChoiceIdCompiles apply fastforce
       using wellTypedCompiles(1) apply fastforce
       using wellTypedValueObservationCompiles(2) by fastforce
    next
      case (Case_IChoice firstAct firstCont x21 x22)
      then show ?thesis using ps apply (cases firstAct, auto split: if_split_asm option.split_asm
            simp add: preservationOfChoiceIdEvaluation)
       using wellTypedCompiles(1) apply fastforce
       using wellTypedPartyCompiles apply fastforce
       using wellTypedPartyCompiles apply fastforce
       using wellTypedValueObservationCompiles(1) apply fastforce
       using wellTypedCompiles(1) apply fastforce
       using wellTypedCompiles(1) apply fastforce
       using wellTypedCompiles(1) apply fastforce
       using wellTypedCompiles(2) apply fastforce
       using wellTypedChoiceIdCompiles apply fastforce
       using wellTypedCompiles(1) apply fastforce
       using wellTypedValueObservationCompiles(2) by fastforce
    next
      case (Case_INotify firstAct x2)
      then show ?thesis using ps apply (cases firstAct, auto split: if_split_asm option.split_asm
            simp add: preservationOfValueEvaluation_MarloweImpliesFaustus(2))
       using wellTypedCompiles(1) apply fastforce
       using wellTypedPartyCompiles apply fastforce
       using wellTypedPartyCompiles apply fastforce
       using wellTypedValueObservationCompiles(1) apply fastforce
       using wellTypedCompiles(1) apply fastforce
       using wellTypedChoiceIdCompiles apply fastforce
       using wellTypedCompiles(1) apply fastforce
       using wellTypedCompiles(1) apply fastforce
       using wellTypedCompiles(2) apply fastforce
       using wellTypedValueObservationCompiles(2) apply fastforce
       using wellTypedValueObservationCompiles(2) by fastforce
    qed
    done
qed

lemma preservationOfApplyCases_MarloweAppliedImpliesFaustusApplied:
"(∃newState newCont. wellTypedCases tyCtx cases ⟶
```

```
  wellTypedState tyCtx state ⟶
  wellTypedContext tyCtx ctx ⟶
  Semantics.applyCases env (fStateToMState state) inp (the (compileCases cases ctx state)) =
       Semantics.Applied warn newMarloweState newMarloweCont ⟶
  (applyCases env state inp cases = Applied warn newState newCont ∧
  fStateToMState newState = newMarloweState ∧
  (the (compile newCont ctx newState)) = newMarloweCont))"
proof (induction cases)
  case Nil
  then show ?case by auto
next
  case (Cons firstCase cases)
  then show ?case apply auto
    apply (smt FaustusSemantics.ApplyResult.exhaust Semantics.ApplyResult.distinct(1) Semantics.
         ApplyResult.inject fStateToMState.simps option.sel
         preservationOfApplyCases_FaustusAppliedImpliesMarloweApplied
         preservationOfApplyCases_FaustusErrorImpliesMarloweError wellTypedCompiles(2))
    subgoal premises ps proof (cases firstCase inp rule: FCase.exhaust[case_product Input.exhaust
         ])
      case (Case_IDeposit firstAct firstCont x11 x12 x13 x14)
      then show ?thesis using ps apply (auto split: option.split)
        apply (cases firstAct, auto simp only: wellTypedCases.simps)
        apply (split option.split_asm)
        using wellTypedCompiles(1) apply fastforce
          apply (split option.split_asm)
        using wellTypedCompiles(1) apply fastforce
          apply (split option.split_asm)
           apply (metis (no_types, lifting) case_prod_conv compileCases.simps(1) option.distinct
                (1) option.simps(4) option.simps(5) ps(1) wellTypedCompiles(2))
        apply (cases firstAct, auto)
                             apply (auto split: option.split_asm if_split_asm)
                             apply (simp_all add: compileAccountsArbitrary(1)
                                  preservationOfValueEvaluation_MarloweImpliesFaustus(1))
                        apply (metis compileFParty.elims evalFParty.simps)
                       apply (metis compileFParty.elims evalFParty.simps)
                      apply (metis compileFParty.elims evalFParty.simps)
                     apply (metis compileFParty.elims evalFParty.simps)
                    apply (metis compileFParty.elims evalFParty.simps)
                   apply (metis compileFParty.elims evalFParty.simps)
                  apply (metis compileFParty.elims evalFParty.simps)
                 apply (metis compileFParty.elims evalFParty.simps)
                apply (metis compileFParty.elims evalFParty.simps)
               apply (metis compileFParty.elims evalFParty.simps)
              apply (metis compileFParty.elims evalFParty.simps)
```

```
        apply (metis compileFParty.elims evalFParty.simps)
    using wellTypedCompiles(1) apply fastforce
    using wellTypedChoiceIdCompiles apply fastforce
    using wellTypedCompiles(1) apply fastforce
    using wellTypedValueObservationCompiles(2) by fastforce
next
  case (Case_IChoice firstAct firstCont x21 x22)
  then show ?thesis using ps apply (auto split: option.split)
    apply (auto split: option.split_asm)
    using wellTypedCompiles(2) apply fastforce
    using wellTypedCompiles(2) apply fastforce
    apply (cases firstAct, auto split: option.split_asm simp add:
         preservationOfChoiceIdEvaluation)
    by (metis FState.surjective FState.update_convs(2) FState.update_convs(4)
         compileContractOnlyBoundPubKeysMatter(1) option.sel)
next
  case (Case_INotify firstAct x2)
  then show ?thesis using ps apply (auto split: option.split)
    apply (auto split: option.split_asm)
    using wellTypedCompiles(2) apply fastforce
    using wellTypedCompiles(2) apply fastforce
    by (cases firstAct, auto split: option.split_asm simp add:
         preservationOfValueEvaluation_MarloweImpliesFaustus(2))
qed
subgoal premises ps proof (cases firstCase inp rule: FCase.exhaust[case_product Input.exhaust
     ])
  case (Case_IDeposit firstAct firstCont x11 x12 x13 x14)
  then show ?thesis using ps apply (auto split: option.split)
    apply (auto split: option.split_asm)
       apply (cases firstAct, auto simp only: wellTypedCases.simps)
        apply (metis option.discI wellTypedCompiles(1))
    using wellTypedCompiles(1) apply fastforce
    using wellTypedCompiles(1) apply fastforce
    using wellTypedCompiles(2) apply fastforce
    using wellTypedCompiles(2) apply fastforce
    by (smt FaustusSemantics.ApplyResult.exhaust Semantics.ApplyResult.distinct(1) Semantics.
         ApplyResult.inject State.ext_inject fStateToMState.simps option.sel
         preservationOfApplyCases_FaustusAppliedImpliesMarloweApplied
         preservationOfApplyCases_FaustusErrorImpliesMarloweError ps(4) wellTypedCompiles(2))
next
  case (Case_IChoice firstAct firstCont x21 x22)
  then show ?thesis using ps apply (auto split: option.split)
    apply (auto split: option.split_asm)
    using wellTypedCompiles(2) apply fastforce
```

```
        using wellTypedCompiles(2) apply fastforce
        using wellTypedCompiles(2) apply fastforce
        by (smt FaustusSemantics.ApplyResult.exhaust Semantics.ApplyResult.distinct(1) Semantics.
             ApplyResult.inject State.ext_inject fStateToMState.simps option.sel
             preservationOfApplyCases_FaustusAppliedImpliesMarloweApplied
             preservationOfApplyCases_FaustusErrorImpliesMarloweError ps(4) wellTypedCompiles(2))
    next
      case (Case_INotify firstAct x2)
      then show ?thesis using ps apply (auto split: option.split)
        apply (auto split: option.split_asm)
        using wellTypedCompiles(2) apply fastforce
        using wellTypedCompiles(2) apply fastforce
        using wellTypedCompiles(2) apply fastforce
        by (cases firstAct, auto split: option.split_asm simp add:
             preservationOfValueEvaluation_MarloweImpliesFaustus(2))
    qed
    done
qed

end
```

The Faustus type-checker adding guarded commands is defined in Isabelle with the following code.

Listing C.2: The Faustus type-checker in Isabelle.

```
theory FaustusV2TypeChecker
  imports Main Util.MList Util.SList Core.ListTools Core.Semantics "HOL-Library.Product_Lexorder"
        "HOL.Product_Type" "HOL-IMP.Star" FaustusTypes FaustusSemantics FaustusV2AST
begin

(* Type rules require the identifier is most recently associated with the correct type in the
     stack.*)
fun lookupType :: "TypeContext ⇒ Identifier ⇒ AbstractionTypeInformation option" where
"lookupType [] i = None" |
"lookupType ((boundId, tyInfo)#rest) i = (if i = boundId then Some tyInfo else lookupType rest i)
     "

fun wellTypedStatement :: "TypeContext ⇒ FStatement ⇒ bool" where
"wellTypedStatement tyCtx (Assert obs) = (wellTypedObservation tyCtx obs)" |
"wellTypedStatement tyCtx (Pay accId payee token val) =(wellTypedParty tyCtx accId ∧
     wellTypedPayee tyCtx payee ∧ wellTypedValue tyCtx val)" |
"wellTypedStatement tyCtx (ReassignVal valId val) = (lookupType tyCtx valId = (Some ValueType) ∧
     wellTypedValue tyCtx val)" |
"wellTypedStatement tyCtx (ReassignObservation obsId obs) = (lookupType tyCtx obsId = (Some
```

```
     ObservationType) ∧ wellTypedObservation tyCtx obs)" |
"wellTypedStatement tyCtx (ReassignPubKey pkId pk) = (lookupType tyCtx pkId = (Some PubKeyType) ∧
      wellTypedParty tyCtx pk)"

fun updateTyCtxFromStmt :: "TypeContext ⇒ FStatement ⇒ TypeContext" where
"updateTyCtxFromStmt tyCtx (ReassignVal valId val) = ((valId, ValueType)#tyCtx)" |
"updateTyCtxFromStmt tyCtx (ReassignObservation obsId obs) = ((obsId, ObservationType)#tyCtx)" |
"updateTyCtxFromStmt tyCtx (ReassignPubKey pkId pk) = ((pkId, PubKeyType)#tyCtx)" |
"updateTyCtxFromStmt tyCtx _ = tyCtx"

fun wellTypedStatements :: "TypeContext ⇒ FStatement list ⇒ bool" where
"wellTypedStatements tyCtx [] = True" |
"wellTypedStatements tyCtx (s#rest) = ((wellTypedStatement tyCtx s) ∧ (wellTypedStatements (
     updateTyCtxFromStmt tyCtx s) rest))"

fun updateTyCtxFromStmts :: "TypeContext ⇒ FStatement list ⇒ TypeContext" where
"updateTyCtxFromStmts tyCtx [] = tyCtx" |
"updateTyCtxFromStmts tyCtx (s#rest) = (updateTyCtxFromStmts (updateTyCtxFromStmt tyCtx s) rest)"

fun wellTypedAction :: "TypeContext ⇒ FAction ⇒ bool" where
"wellTypedAction tyCtx (Deposit fromPk toPk token value) = ((wellTypedParty tyCtx fromPk) ∧ (
     wellTypedParty tyCtx toPk) ∧ (wellTypedValue tyCtx value))" |
"wellTypedAction tyCtx (Choice choiceId bounds) = (wellTypedChoiceId tyCtx choiceId)" |
"wellTypedAction tyCtx (Notify observation) = (wellTypedObservation tyCtx observation)"

fun wellTypedGuardExpression :: "TypeContext ⇒ FGuardExpression ⇒ bool" where
"wellTypedGuardExpression tyCtx (ActionGuard a) = (wellTypedAction tyCtx a)" |
"wellTypedGuardExpression tyCtx (GuardThenGuard g1 g2) = ((wellTypedGuardExpression tyCtx g1) ∧ (
     wellTypedGuardExpression tyCtx g2))" |
"wellTypedGuardExpression tyCtx (GuardStmtsGuard g stmts) = ((wellTypedGuardExpression tyCtx g) ∧
      (wellTypedStatements tyCtx stmts))" |
"wellTypedGuardExpression tyCtx (DisjointGuard g1 g2) = ((wellTypedGuardExpression tyCtx g1) ∧ (
     wellTypedGuardExpression tyCtx g2))" |
"wellTypedGuardExpression tyCtx (InterleavedGuard g1 g2) = ((wellTypedGuardExpression tyCtx g1) ∧
      (wellTypedGuardExpression tyCtx g2))"

fun wellTypedContract :: "TypeContext ⇒ FContract ⇒ bool" and
  wellTypedCases :: "TypeContext ⇒ FCase list ⇒ bool" where
"wellTypedContract tyCtx Close = True" |
"wellTypedContract tyCtx (StatementCont stmt cont) = (wellTypedStatement tyCtx stmt ∧ (
     wellTypedContract (updateTyCtxFromStmt tyCtx stmt) cont))" |
"wellTypedContract tyCtx (If obs cont1 cont2) =
  (wellTypedObservation tyCtx obs ∧ wellTypedContract tyCtx cont1 ∧ wellTypedContract tyCtx cont2
       )" |
```

```
"wellTypedContract tyCtx (When cases t cont) = (wellTypedCases tyCtx cases ∧ wellTypedContract
     tyCtx cont)" |
"wellTypedContract tyCtx (Let valId val cont) = (wellTypedValue tyCtx val ∧ wellTypedContract ((
     valId, ValueType)#tyCtx) cont)" |
"wellTypedContract tyCtx (LetObservation obsId obs cont) = (wellTypedObservation tyCtx obs ∧
     wellTypedContract ((obsId, ObservationType)#tyCtx) cont)" |
"wellTypedContract tyCtx (LetPubKey pkId pk cont) = (wellTypedParty tyCtx pk ∧ wellTypedContract
     ((pkId, PubKeyType)#tyCtx) cont)" |
"wellTypedContract tyCtx (LetC cid params body cont) = (wellTypedContract ((paramsToTypeContext
     params)@tyCtx) body ∧ wellTypedContract ((cid, ContractType params)#tyCtx) cont)" |
"wellTypedContract tyCtx (UseC cid args) = (case lookupContractIdParamTypeInformation tyCtx cid
     of
  Some (params, innerTyCtx) ⇒ wellTypedArguments tyCtx params args
  | _ ⇒ False)" |
"wellTypedCases tyCtx [] = True" |
"wellTypedCases tyCtx ((Case g c)#rest) = ((wellTypedGuardExpression tyCtx g) ∧ (
     wellTypedContract tyCtx c) ∧ (wellTypedCases tyCtx rest))"

fun wellTypedContext :: "TypeContext ⇒ FContext ⇒ bool" where
"wellTypedContext [] [] = True" |
"wellTypedContext tyCtx [] = False" |
"wellTypedContext [] ctx = False" |
"wellTypedContext ((tyValId, ValueType)#restTypes) ((ValueAbstraction absValId)#restAbstractions)
      =
  (tyValId = absValId ∧ wellTypedContext restTypes restAbstractions)" |
"wellTypedContext ((tyPkId, PubKeyType)#restTypes) ((PubKeyAbstraction absPkId)#restAbstractions)
      =
  (tyPkId = absPkId ∧ wellTypedContext restTypes restAbstractions)" |
"wellTypedContext ((tyObsId, ObservationType)#restTypes) ((ObservationAbstraction absObsId)#
     restAbstractions) =
  (tyObsId = absObsId ∧ wellTypedContext restTypes restAbstractions)" |
"wellTypedContext ((tyCid, ContractType tyParams)#restTypes) ((ContractAbstraction (absCid,
     absParams, absBody))#restAbstractions) =
  (tyCid = absCid ∧ tyParams = absParams ∧ wellTypedContract ((paramsToTypeContext absParams)
       @restTypes) absBody ∧ wellTypedContext restTypes restAbstractions)" |
"wellTypedContext (someType#restTypes) (someAbs#restAbs) = False"

lemma consValueWellTypedContext:
"wellTypedContext tyCtx ctx ⟷
  wellTypedContext ((valId, ValueType)#tyCtx) (ValueAbstraction valId#ctx)"
  by simp

lemma consObservationWellTypedContext:
"wellTypedContext tyCtx ctx ⟷
```

```
  wellTypedContext ((obsId, ObservationType)#tyCtx) (ObservationAbstraction obsId#ctx)"
  by simp

lemma consPubKeyWellTypedContext:
"wellTypedContext tyCtx ctx ⟷
  wellTypedContext ((pkId, PubKeyType)#tyCtx) (PubKeyAbstraction pkId#ctx)"
  by simp

lemma consContractWellTypedContext:
"wellTypedContext tyCtx ctx ∧ wellTypedContract ((paramsToTypeContext params)@tyCtx) bodyCont ⟷
  wellTypedContext ((cid, ContractType params)#tyCtx) (ContractAbstraction (cid, params, bodyCont
       )#ctx)"
  by auto

lemma paramsWellTypedContext:
"wellTypedContext (paramsToTypeContext params) (paramsToFContext params)"
proof (induction params)
  case Nil
  then show ?case by auto
next
  case (Cons firstParam params)
  then show ?case by (cases firstParam, auto)
qed

lemma wellTypedActionAppendTypes [simp]:
"(∀ tyCtx2 . wellTypedAction tyCtx a ⟶
  wellTypedAction (tyCtx @ tyCtx2) a)"
proof (induction rule: wellTypedAction.induct)
  case (1 tyCtx fromPk toPk token "value")
  then show ?case by (auto simp add: wellTypedPartyAppendTypes wellTypedValueAppendTypes)
next
  case (2 tyCtx choiceId bounds)
  then show ?case by (auto simp add: wellTypedChoiceIdAppendTypes)
next
  case (3 tyCtx observation)
  then show ?case by (auto simp add: wellTypedValueAppendTypes)
qed

lemma lookupTypeAppendTypes:
"(∀ tyCtx2 . lookupType tyCtx i = Some res ⟶
  lookupType (tyCtx @ tyCtx2) i = Some res)"
  by (induction tyCtx, auto)

lemma wellTypedStatementAppendTypes:
```

```
"(∀ tyCtx2 . wellTypedStatement tyCtx s ⟶
  wellTypedStatement (tyCtx @ tyCtx2) s)"
proof (induction rule: wellTypedStatement.induct)
  case (1 tyCtx obs)
  then show ?case by (auto simp add: wellTypedValueAppendTypes)
next
  case (2 tyCtx accId payee token val)
  then show ?case by (auto simp add: wellTypedValueAppendTypes wellTypedPartyAppendTypes
       wellTypedPayeeAppendTypes)
next
  case (3 tyCtx valId val)
  then show ?case by (auto simp add: lookupTypeAppendTypes wellTypedValueAppendTypes)
next
  case (4 tyCtx obsId obs)
  then show ?case by (auto simp add: lookupTypeAppendTypes wellTypedValueAppendTypes)
next
  case (5 tyCtx pkId pk)
  then show ?case by (auto simp add: lookupTypeAppendTypes wellTypedPartyAppendTypes)
qed

lemma wellTypedStatementsAppendTypes:
"(∀ tyCtx2 . wellTypedStatements tyCtx stmts ⟶
  wellTypedStatements (tyCtx @ tyCtx2) stmts)"
proof (induction rule: wellTypedStatements.induct)
  case (1 tyCtx)
  then show ?case by auto
next
  case (2 tyCtx s rest)
  then show ?case by (cases s, auto simp add: lookupTypeAppendTypes wellTypedStatementAppendTypes
        wellTypedPayeeAppendTypes wellTypedValueAppendTypes wellTypedPartyAppendTypes)
qed

lemma wellTypedGuardExpressionAppendTypes:
"(∀ tyCtx2 . wellTypedGuardExpression tyCtx g ⟶
  wellTypedGuardExpression (tyCtx @ tyCtx2) g)"
proof (induction rule: wellTypedGuardExpression.induct)
  case (1 tyCtx a)
  then show ?case by auto
next
  case (2 tyCtx g1 g2)
  then show ?case by auto
next
  case (3 tyCtx g stmts)
  then show ?case by (auto simp add: wellTypedStatementsAppendTypes)
```

```
next
  case (4 tyCtx g1 g2)
  then show ?case by auto
next
  case (5 tyCtx g1 g2)
  then show ?case by auto
qed


lemma wellTypedContractAppendTypes:
"(∀ tyCtx2 . wellTypedContract tyCtx cont ⟶
  wellTypedContract (tyCtx @ tyCtx2) cont)"
"(∀ tyCtx2 . wellTypedCases tyCtx cases ⟶
  wellTypedCases (tyCtx @ tyCtx2) cases)"
proof (induction rule: wellTypedContract_wellTypedCases.induct)
  case (1 tyCtx)
  then show ?case by auto
next
  case (2 tyCtx s cont)
  then show ?case by (cases s, auto simp add: lookupTypeAppendTypes wellTypedStatementAppendTypes
         wellTypedPayeeAppendTypes wellTypedValueAppendTypes wellTypedPartyAppendTypes)
next
  case (3 tyCtx obs cont1 cont2)
  then show ?case by (simp add: wellTypedValueAppendTypes(2))
next
  case (4 tyCtx cases t cont)
  then show ?case by simp
next
  case (5 tyCtx valId val cont)
  then show ?case by (simp add: wellTypedValueAppendTypes(1))
next
  case (6 tyCtx obsId obs cont)
  then show ?case by (simp add: wellTypedValueAppendTypes(2))
next
  case (7 tyCtx pkId pk cont)
  then show ?case by (simp add: wellTypedPartyAppendTypes)
next
  case (8 tyCtx cid params body cont)
  then show ?case by auto
next
  case (9 tyCtx cid args)
  then show ?case by (cases "lookupContractIdParamTypeInformation tyCtx cid", auto simp add:
       lookupCidAppendTypes wellTypedArgumentsAppendTypes)
next
```

```
  case (10 tyCtx)
  then show ?case by simp
next
  case (11 tyCtx g c rest)
  then show ?case by (simp add: wellTypedGuardExpressionAppendTypes)
qed

lemma concatWellTypedContexts:
"wellTypedContext tyCtx1 ctx1 ⟹
  wellTypedContext tyCtx2 ctx2 ⟹
  wellTypedContext (tyCtx2 @ tyCtx1) (ctx2 @ ctx1)"
  apply (induction tyCtx2 ctx2 rule: wellTypedContext.induct, auto)
  using wellTypedContractAppendTypes(1) by fastforce

fun lookupContractIdAbsInformation :: "FContext ⇒ Identifier ⇒ (FParameter list × FContract ×
     FContext) option" where
"lookupContractIdAbsInformation [] cid = None" |
"lookupContractIdAbsInformation (ContractAbstraction (boundCid, params, bodyCont)#restAbs) cid =
     (if boundCid = cid
  then Some (params, bodyCont, restAbs)
  else lookupContractIdAbsInformation restAbs cid)" |
"lookupContractIdAbsInformation (ValueAbstraction i#restAbs) cid = (if i = cid then None else
     lookupContractIdAbsInformation restAbs cid)" |
"lookupContractIdAbsInformation (ObservationAbstraction i#restAbs) cid = (if i = cid then None
     else lookupContractIdAbsInformation restAbs cid)" |
"lookupContractIdAbsInformation (PubKeyAbstraction i#restAbs) cid = (if i = cid then None else
     lookupContractIdAbsInformation restAbs cid)"

lemma lookupContractTypeWellTypedContext:
"(∀cid params bodyCont restCtx. ∃restTy . wellTypedContext tyCtx ctx ⟶
  lookupContractIdAbsInformation ctx cid = Some (params, bodyCont, restCtx) ⟶
  lookupContractIdParamTypeInformation tyCtx cid = Some (params, restTy))"
  by (induction tyCtx ctx rule: wellTypedContext.induct, auto)

lemma lookupContractAbstractionWellTypedContext:
"(∀cid params restTy. ∃bodyCont restCtx . wellTypedContext tyCtx ctx ⟶
  lookupContractIdParamTypeInformation tyCtx cid = Some (params, restTy) ⟶
  lookupContractIdAbsInformation ctx cid = Some (params, bodyCont, restCtx))"
  by (induction tyCtx ctx rule: wellTypedContext.induct, auto)

lemma lookupContractRestIsWellTyped:
"(∀cid params restTy bodyCont restCtx . wellTypedContext tyCtx ctx ⟶
  lookupContractIdParamTypeInformation tyCtx cid = Some (params, restTy) ⟶
  lookupContractIdAbsInformation ctx cid = Some (params, bodyCont, restCtx) ⟶
```

```
  wellTypedContext restTy restCtx)"
  by (induction tyCtx ctx rule: wellTypedContext.induct, auto)

lemma lookupContractFoundContractIsWellTyped:
"(∀cid params restTy bodyCont restCtx . wellTypedContext tyCtx ctx ⟶
  lookupContractIdParamTypeInformation tyCtx cid = Some (params, restTy) ⟶
  lookupContractIdAbsInformation ctx cid = Some (params, bodyCont, restCtx) ⟶
  wellTypedContract (paramsToTypeContext params @ restTy) bodyCont)"
  by (induction tyCtx ctx rule: wellTypedContext.induct, auto)

lemma lookupContractStateStaysWellTyped:
"(∀cid params restTy restCtx .
  wellTypedState tyCtx s ⟶
  lookupContractIdParamTypeInformation tyCtx cid = Some (params, restTy) ⟶
  wellTypedState restTy s)"
  by (induction tyCtx s rule: wellTypedState.induct, auto)

lemma lookupContractCompilerContextSmaller:
"lookupContractIdAbsInformation ctx cid = Some (params, bodyCont, innerCtx) ⟹
  1 + fold (+) (map (abstractionInformationSize) ctx) 0 > fContractSize bodyCont + fold (+) (map
       (abstractionInformationSize) (innerCtx)) 0"
  apply (induction rule: lookupContractIdAbsInformation.induct, auto)
     apply (smt add.commute add.right_neutral fold_plus_sum_list_rev less_add_Suc1 less_trans
          nat_add_left_cancel_less not_less_eq not_less_iff_gr_or_eq option.inject prod.inject)
    apply (meson option.distinct(1))
   apply (meson option.distinct(1))
  by (meson option.distinct(1))


end
```

Appendix D

Faustus Processes and Bisimulation Isabelle Formalization

The following Isabelle theory files formalize Faustus processes on top of the Faustus semantics from Chapter 3 and Appendix C. From there a verification of a transformation is done using the definition of simulation and bisimulation for Faustus processes.

Listing D.1: Formalization of Faustus processes.

```
(*<*)
theory FaustusV2Sim
  imports Main Util.MList Util.SList Core.ListTools "HOL-Library.Product_Lexorder" "HOL.
       Product_Type" "HOL-IMP.Star" FaustusV2Semantics HOL.Relation "HOL-Library.LaTeXsugar"
begin
(*>*)

chapter \<open>Faustus Reaction Semantics\<close>

text \<open>We must first define internal and external reaction rules for Faustus contracts.
  Internal reaction rules are for contract states that do not require external interaction
  for the contract to transition. External reaction rules define how external interactions
  from parties are processed and allow the contract to transition. Like CCS we define an internal
  reaction to be the "\tau" symbol. We prefix symbols with external input information with the
  "\lambda" symbol.\<close>

text \<open>Since time passes for the contract only when transactions are processed by the
     blockchain we
```

```
  define our inputs as a triple of optional input information, and required lower and upper
       bounds
  of time. This means a deposit, choice, or notification isn't required to advance the current
       time
  of the contract. Timeout conditions in the contract are entered through an input that only has
  the new time associated.\<close>

(* Tau transitions = internal_reaction freshChoiceName defined in SmallStep.thy now *)
(*datatype FullSymbol  = Tau ("τ")
  | Lambda InputLabel ("λ_")*)

type_synonym FConfig = "FContract * FContext * FState * Environment * (TransactionWarning list) *
      (Payment list)"

type_synonym Restriction = "Input set"

type_synonym Proc = "FConfig × Restriction"

type_synonym TransitionRelation = "FConfig ⇒ FullSymbol ⇒ FConfig ⇒ bool"

inductive small_step_receive_ccs :: "FConfig ⇒ InputLabel ⇒ FConfig ⇒ bool" ("_ →_ _" [55, 0,
      55]) where
LambdaReceive: "⟦
  fixInterval (lower, upper) state = IntervalTrimmed newTimeEnv newTimeState;
  timeInterval newTimeEnv = (realLower, realUpper);
  realLower < t;
  realUpper < t;
  inp ⊢ (cases, newTimeState, newTimeEnv) →cases_f2 (newC, newState, newWarn);
  i = (Some inp, lower, upper) ⟧ ⟹
  (When cases t c, ctx, state, env, warnings, payments) →i (newC t c, ctx, newState, newTimeEnv,
        warnings @ convertApplyWarning newWarn, payments)" |
CCSTimeout: "⟦
  fixInterval (lower, upper) state = IntervalTrimmed newTimeEnv newTimeState;
  timeInterval newTimeEnv = (realLower, realUpper);
  realLower ≥ t;
  realUpper ≥ t;
  i = (None, lower, upper) ⟧ ⟹
  (When cases t c, ctx, state, env, warnings, payments) →i (c, ctx, newTimeState, newTimeEnv,
       warnings, payments)"

inductive_cases ReceiveE[elim]: "p1 →μ p1'"

lemma deterministicSmallStepReceive:
"f →μ f' ⟹
```

```
  f →μ f'' ⟹
  f' = f''"
proof (induction f μ f' arbitrary: f'' rule: small_step_receive_ccs.induct)
  case (LambdaReceive lower upper state newTimeEnv newTimeState realLower realUpper t inp cases
       newC newState newWarn i c ctx env warnings payments f'')
  show ?case using LambdaReceive(7) LambdaReceive(1-6) apply (induction "(FaustusV2AST.FContract.
       When cases t c, ctx, state, env, warnings, payments)" i f'' rule: small_step_receive_ccs.
       induct)
     apply auto
    by (metis Pair_inject deterministicCasesSemantics)+
qed auto

(*
  These receive semantics represent observable transitions in the LTS. They can be informally
  justified by creating a process that receives an input in front of the contract. That process
       can
  send an specific choice input in front of the original input to transition to an inner when.
*)
inductive small_step_receive_ccs_add_choice :: "ChoiceId ⇒ FConfig ⇒ InputLabel ⇒ FConfig ⇒
     bool" ("_ ⊢ _ →_ _" [55, 0, 55]) where
NoChoiceLambdaReceive: "⟦
  f1 →μ f2;
  ¬((IChoice cho 0) = inp);
  μ = (Some inp, lower, upper) ⟧ ⟹
  cho ⊢ f1 →μ f2" |
ChoiceLambdaReceive: "⟦
  ¬(∃f . f1 →μ f);
  f1 →(Some (IChoice cho 0), lower, upper) f2;
  cho ⊢ f2 →μ f3;
  ¬((IChoice cho 0) = inp);
  μ = (Some inp, lower, upper) ⟧ ⟹
  cho ⊢ f1 →μ f3" |
CCSTimeout: "⟦
  μ = (None, lower, upper);
  f1 →μ f2 ⟧ ⟹
  freshChoice ⊢ f1 →μ f2"

inductive_cases ReceiveAddChoiceE[elim]: "cho ⊢ p1 →μ p1'"
thm ReceiveE
thm ReceiveAddChoiceE

lemma deterministicSmallStepReceiveAddChoice:
"freshChoice ⊢ f →μ f' ⟹
  freshChoice ⊢ f →μ f'' ⟹
```

```
  f' = f''"
proof (induction freshChoice f μ f' arbitrary: f'' rule: small_step_receive_ccs_add_choice.induct
     )
  case (NoChoiceLambdaReceive f1 μ f2 cho inp lower upper)
  then show ?case apply auto
    by (metis deterministicSmallStepReceive small_step_receive_ccs_add_choice.cases)
next
  case (ChoiceLambdaReceive f1 μ cho lower upper f2 f3 inp f4)
  then show ?case by (smt (verit) ReceiveAddChoiceE deterministicSmallStepReceive old.prod.inject
        prod_cases6)
next
  case (CCSTimeout μ lower upper f1 f2 freshChoice)
  then show ?case by (metis deterministicSmallStepReceive small_step_receive_ccs_add_choice.cases
       )
qed

(* Reactions have already been defined in small_step_reduce *)
inductive internal_reaction :: "FConfig ⇒ FConfig ⇒ bool" ("_ →𝒯 _" [55, 55]) where
SmallStepReduce: "(c, ctx, s, env, [], payments) →f2 (newC, newCtx, newS, newEnv, newWarnings,
     newPayments) ⟹
  (c, ctx, s, env, warnings, payments) →𝒯 (newC, newCtx, newS, newEnv, warnings @
       convertReduceWarnings newWarnings, newPayments)"

abbreviation small_step_reacts_ccs :: "FConfig ⇒ FConfig ⇒ bool" ("_ →*𝒯 _" [55, 55]) where
"x →*𝒯 y == star internal_reaction x y"

inductive_cases ReactE: "p1 →𝒯 p1'"
thm ReactE

(*
  These reaction semantics that process a notification input can be informally justified by
       treating this
  as creating a process that can send a notify to the contract any time and hiding the notify.
  This means that INotifies can be treated as a Tau transition
  External parties can still notify. 𝒯\<^sub>n denotes tau transitions induced by INotify.
*)
inductive small_step_react_add_notify :: "FConfig ⇒ FConfig ⇒ bool" ("_ →𝒯n _" [55, 55])
     where
BaseReaction: "c1 →𝒯 c2 ⟹ c1 →𝒯n c2" |
NotifyReaction: "⟦fixInterval (newLower, newUpper) state = IntervalTrimmed newTimeEnv
     newTimeState;
  timeInterval newTimeEnv = (realLower, realUpper);
  realLower < t;
  realUpper < t;
```

```
  INotify ⊢ (cases, newTimeState, newTimeEnv) →cases_f2 (newC, newState, newWarn) ⟧ ⟹
  (When cases t cont, ctx, state, env, warnings, payments) →𝒯n (newC t cont, ctx, newState,
       newTimeEnv, warnings @ convertApplyWarning newWarn, payments)"

inductive internal_reaction_add_choice :: "ChoiceName ⇒ FConfig ⇒ FConfig ⇒ bool" ("_ ⊢ _ →𝒯
      _" [55, 0, 55]) where
BaseReaction: "c1 →𝒯 c2 ⟹ choiceName ⊢ c1 →𝒯 c2" |
ChoiceReaction: "⟦fixInterval (newLower, newUpper) state = IntervalTrimmed newTimeEnv
     newTimeState;
  timeInterval newTimeEnv = (realLower, realUpper);
  realLower < t;
  realUpper < t;
  (IChoice (ChoiceId freshChoiceName party) i) ⊢ (cases, newTimeState, newTimeEnv) →cases_f2 (
       newC, newState, newWarn) ⟧ ⟹
  freshChoiceName ⊢ (When cases t cont, ctx, state, env, warnings, payments) →𝒯 (newC t cont,
       ctx, newState, newTimeEnv, warnings @ convertApplyWarning newWarn, payments)"

inductive full_semantics :: "Restriction ⇒ TransitionRelation" ("_ ⊢ _ −_→ _" [55, 0, 55])
     where
BaseInternalFull: "p1 →𝒯 p2 ⟹ full_semantics A p1 τ p2" |
BaseReceiveFull: "⟦
  i \<notin> A;
  p1 →(Some i, lower, upper) p2⟧ ⟹
  full_semantics A p1 (λ(Some i, lower, upper)) p2" |
RestrictedReceiveFull: "⟦
  i \<notin> A;
  ¬(∃f . p1 →μ f);
  μ = (Some i, lower, upper);
  (∃j . j ∈ A ∧ p1 →(Some j, lower, upper) p3);
  (A ⊢ p3 −λμ→ p4)⟧ ⟹
  full_semantics A p1 (λμ) p4" |
TimeoutFull: "⟦p1 →(None, lower, upper) p2⟧ ⟹ full_semantics A p1 (λ(None, lower, upper)) p2"

inductive full_nd_semantics :: "Restriction ⇒ TransitionRelation" ("_ ⊢ _ ⟝_→ _" [55, 0, 55])
     where
BaseInternalFullND: "p1 →𝒯 p2 ⟹ full_nd_semantics A p1 τ p2" |
BaseReceiveFullND: "⟦
  i \<notin> A;
  p1 →(Some i, lower, upper) p2⟧ ⟹
  full_nd_semantics A p1 (λ(Some i, lower, upper)) p2" |
RestrictedReceiveFullND: "⟦
  i \<notin> A;
  μ = (Some i, lower, upper);
  (∃j . j ∈ A ∧ p1 →(Some j, lower, upper) p3);
```

```
  (A ⊢ p3 ⟶λμ→ p4)⟧ ⟹
  full_nd_semantics A p1 (λμ) p4" |
TimeoutFullND: "⟦p1 →(None, lower, upper) p2⟧ ⟹ full_nd_semantics A p1 (λ(None, lower, upper))
      p2"

lemma full_nd_simulates_original:
"A ⊢ p1 −i→ p2 ⟹
  A ⊢ p1 ⟶i→ p2"
proof (induction "(A)" p1 i p2 rule: full_semantics.induct)
  case (BaseInternalFull p1 p2 A)
  then show ?case by (simp add: BaseInternalFullND)
next
  case (BaseReceiveFull i A p1 lower upper p2)
  then show ?case by (simp add: BaseReceiveFullND)
next
  case (RestrictedReceiveFull i A p1 μ lower upper p3 p4)
  then show ?case by (simp add: RestrictedReceiveFullND)
next
  case (TimeoutFull p1 lower upper p2 A)
  then show ?case by (simp add: TimeoutFullND)
qed

lemma full_semantics_weakening:
"(A ⊢ p1 −λμ→ p1') ⟹
  μ = (Some i, lower, upper) ⟹
  i \<notin> B ⟹
  ((A ∪ B) ⊢ p1 −λμ→ p1')"
proof (induction A p1 "λμ" p1' arbitrary: μ i lower upper rule: full_semantics.induct)
  case (BaseReceiveFull i A p1 lower upper p2)
  then show ?case by (auto simp add: full_semantics.BaseReceiveFull)
next
  case (RestrictedReceiveFull innerI A p1 μ lower' upper' p3 p4)
  moreover have "i = innerI ∧ lower = lower' ∧ upper = upper'" using RestrictedReceiveFull by
       auto
  moreover have "i \<notin> (A ∪ B)" using calculation by auto
  moreover obtain j where "j ∈ A ∧ p1 →(Some j, lower', upper') p3" using calculation by auto
  moreover have "j ∈ (A ∪ B)" using calculation by auto
  ultimately show ?case using RestrictedReceiveFull(6) full_semantics.RestrictedReceiveFull[of i
       "A ∪ B" p1 μ lower upper p3 p4] by auto
next
  case (TimeoutFull p1 lower upper p2 A)
  then show ?case by auto
qed
```

```
lemma full_semantics_union_composition_input:
"(B ⊢ p2 −λ(Some j, lower, upper)→ p2') ⟹
  (A ∪ B) ⊢ p2' −λμ→ p2'' ⟹
  j ∈ A ⟹
  \<nexists>f. B ⊢ p2 −λμ→ f ⟹
  μ = (Some i, lower, upper) ⟹
  i \<notin> B ⟹
  (A ∪ B) ⊢ p2 −λμ→ p2'''"
proof (induction "B" p2 "λ(Some j, lower, upper)" p2' rule: full_semantics.induct)
  case (BaseReceiveFull B p1 p1')
  moreover have "j ∈ A ∪ B" using calculation by auto
  moreover have "i \<notin> A ∪ B" using calculation(6,3) full_semantics.simps [of "A ∪ B" p1' "λ
      μ" p2''] by auto
  moreover have "\<nexists>f. p1 →μ f" by (metis calculation(5,6,7) full_semantics.
      BaseReceiveFull)
  ultimately show ?case using full_semantics.RestrictedReceiveFull[of i "A ∪ B" p1 μ lower upper
      p1' p2''] by auto
next
  case (RestrictedReceiveFull arbj B p1 lower upper p3 p4)
  moreover have "∀ p3' . (B ⊢ p3 −λμ→ p3') ⟶ (B ⊢ p1 −λμ→ p3')" using full_semantics.
      RestrictedReceiveFull[of i B p1 μ lower upper p3] RestrictedReceiveFull apply auto
    by (meson BaseReceiveFull)
  moreover have "(A ∪ B) ⊢ p3 −λμ→ p2'''" using calculation by blast
  moreover have "i \<notin> A ∪ B" using RestrictedReceiveFull(7,11,10) by (cases rule:
      full_semantics.cases, auto)
  ultimately show ?case apply auto
    using full_semantics.RestrictedReceiveFull[of i "A∪B" p1 μ lower upper p3 p2''] apply auto
    by (meson BaseReceiveFull)
qed

lemma full_nd_semantics_union_composition_input:
"(B ⊢ p2 ⟶λ(Some j, lower, upper)→ p2') ⟹
  (A ∪ B) ⊢ p2' ⟶λμ→ p2'' ⟹
  j ∈ A ⟹
  μ = (Some i, lower, upper) ⟹
  i \<notin> B ⟹
  (A ∪ B) ⊢ p2 ⟶λμ→ p2'''"
proof (induction "B" p2 "λ(Some j, lower, upper)" p2' rule: full_nd_semantics.induct)
  case (BaseReceiveFullND B p1 p1')
  moreover have "j ∈ A ∪ B" using calculation by auto
  moreover have "i \<notin> A ∪ B" using calculation full_nd_semantics.simps[of "A ∪ B" p1' "λμ"
      p2''] by auto
  ultimately show ?case using full_nd_semantics.RestrictedReceiveFullND[of i "A ∪ B" μ lower
      upper p1 p1' p2''] by auto
```

```
next
  case (RestrictedReceiveFullND arbj B lower upper p1 p3 p4)
  moreover have "∀ p3' . (B ⊢ p3 ⟶λμ→ p3') ⟶ (B ⊢ p1 ⟶λμ→ p3')" using full_nd_semantics.
       RestrictedReceiveFullND[of i B μ lower upper p1 p3] RestrictedReceiveFullND by auto
  moreover have "(A ∪ B) ⊢ p3 ⟶λμ→ p2''" using calculation by blast
  moreover have "i \<notin> A ∪ B" using RestrictedReceiveFullND(6,8,9) by (cases rule:
       full_nd_semantics.cases, auto)
  ultimately show ?case using full_nd_semantics.RestrictedReceiveFullND[of i "A∪B" μ lower upper
       p1 p3 p2''] by auto
qed

lemma full_semantics_react_is_equivalent_to_internal_reaction:
"p1 →𝒯 p2 ⟷ full_semantics A p1 τ p2"
  apply (auto simp add: BaseInternalFull)
  using full_semantics.cases by blast

lemma full_nd_semantics_react_is_equivalent_to_internal_reaction:
"p1 →𝒯 p2 ⟷ full_nd_semantics A p1 τ p2"
  apply (auto simp add: BaseInternalFullND)
  using full_nd_semantics.cases by blast

(* ADV = (a.(!a.ADV + !c.!a.ADV))
  CON = (c.a...0)
 new a,c (ADV | CON)
*)
lemma full_semantics_receive_empty_set_is_equivalent_to_base_small_step:
"(p1 →μ p2) ⟷ (full_semantics \<emptyset> p1 (λμ) p2)"
  apply auto
   apply (induction p1 μ p2 rule: small_step_receive_ccs.induct, auto)
    apply (simp add: BaseReceiveFull LambdaReceive)
   apply (simp add: TimeoutFull small_step_receive_ccs.CCSTimeout)
  by (smt (verit, best) FullSymbol.distinct(1) FullSymbol.inject empty_iff full_semantics.simps)

lemma full_nd_semantics_receive_empty_set_is_equivalent_to_base_small_step:
"(p1 →μ p2) ⟷ (full_nd_semantics \<emptyset> p1 (λμ) p2)"
  apply auto
   apply (induction p1 μ p2 rule: small_step_receive_ccs.induct, auto)
    apply (simp add: BaseReceiveFullND LambdaReceive)
   apply (simp add: TimeoutFullND small_step_receive_ccs.CCSTimeout)
  by (smt (verit, best) FullSymbol.distinct(1) FullSymbol.inject empty_iff full_nd_semantics.
       simps)

lemma full_semantics_receive_choice_input_is_equivalent_to_small_step_add_choice_forward:
"((ChoiceId cn pt) ⊢ p1 →μ p2) ⟹ (full_semantics {(IChoice (ChoiceId cn pt) 0)} p1 (λμ) p2)"
```

```
  apply (induction "ChoiceId cn pt" "p1" "μ" "p2" rule: small_step_receive_ccs_add_choice.induct)
  using BaseReceiveFull RestrictedReceiveFull TimeoutFull by (auto)

lemma full_semantics_receive_choice_input_is_equivalent_to_small_step_add_choice_backward:
"(full_semantics {(IChoice (ChoiceId cn pt) 0)} p1 (λμ) p2) ⟹ ((ChoiceId cn pt) ⊢ p1 →μ p2)"
  apply (induction "{(IChoice (ChoiceId cn pt) 0)}" p1 "(λμ)" p2 rule: full_semantics.induct)
    using  NoChoiceLambdaReceive small_step_receive_ccs_add_choice.CCSTimeout ChoiceLambdaReceive
          by auto

lemma full_semantics_receive_choice_input_is_equivalent_to_small_step_add_choice:
"((ChoiceId cn pt) ⊢ p1 →μ p2) ⟷ (full_semantics {(IChoice (ChoiceId cn pt) 0)} p1 (λμ) p2)"
  by (meson full_semantics_receive_choice_input_is_equivalent_to_small_step_add_choice_backward
       full_semantics_receive_choice_input_is_equivalent_to_small_step_add_choice_forward)

inductive experiment_relation :: "FConfig ⇒ InputLabel list ⇒ FConfig ⇒ bool" ("_ =_⇒ _" [55,
      0, 55]) where
BaseExperiment: "⟦c1 →⋆𝒯 c2 ⟧ ⟹
  c1 =[]⇒ c2" |
StepExperiment: "⟦c1 →⋆𝒯 c2;
  c2 →i c3;
  c3 →⋆𝒯 c4;
  c4 =rest⇒ c5 ⟧ ⟹
   c1 =(i#rest)⇒ c5"

inductive full_nd_experiment_relation :: "Restriction ⇒ FConfig ⇒ FullSymbol list ⇒ FConfig ⇒
      bool" ("_ ⊢ _ ==_⇒ _" [55, 0, 55]) where
BaseExperimentND: "⟦c1 →⋆𝒯 c2 ⟧ ⟹
  A ⊢ c1 ==[]⇒ c2" |
StepExperimentND: "⟦c1 →⋆𝒯 c2;
  A ⊢ c2 —μ→ c3;
  c3 →⋆𝒯 c4;
  A ⊢ c4 ==rest⇒ c5 ⟧ ⟹
   A ⊢ c1 ==(μ#rest)⇒ c5"

lemma single_full_nd_receive_in_experiment:
"full_nd_semantics A p1 μ p2 ⟹
  A ⊢ p1 ==[μ]⇒ p2"
  using BaseExperimentND StepExperimentND by blast

inductive general_experiment_relation :: "('process ⇒ 'process ⇒ bool) ⇒
  ('process ⇒ 'label ⇒ 'process ⇒ bool) ⇒
  'process => 'label list ⇒ 'process ⇒ bool" ("_, _ ⊢ _ =_⇒ _" [55, 55, 0, 0, 55]) where
GeneralBaseExperiment: "⟦star (general_reaction) p1 p2⟧ ⟹
  general_reaction, general_receive ⊢ p1 =[]⇒ p2" |
```

```
GeneralStepExperiment: "⟦star (general_reaction) p1 p2;
  general_receive p2 inp p3;
  star general_reaction p3 p4;
  general_reaction, general_receive ⊢ p4 =rest⇒ p5 ⟧ ⟹
  general_reaction, general_receive ⊢ p1 =(inp#rest)⇒  p5"

lemma single_receive_in_experiment:
"receive p1 inp p2 ⟹
  reaction, receive ⊢ p1 =[inp]⇒ p2"
  by (meson GeneralBaseExperiment GeneralStepExperiment star.refl)

lemma faustusExperimentInGeneral:
"experiment_relation c1 inp c2 = general_experiment_relation (internal_reaction) (
     small_step_receive_ccs) c1 inp c2"
  apply auto
   apply (induction  c1 inp c2 rule: experiment_relation.induct, auto simp add:
        GeneralBaseExperiment GeneralStepExperiment)
  by (induction internal_reaction small_step_receive_ccs c1 inp c2 rule:
       general_experiment_relation.induct, auto simp add: BaseExperiment StepExperiment)

inductive_cases EmptyExperimentE: "c1 =[]⇒ c2"
inductive_cases GeneralEmptyExperimentE: "general_experiment_relation reaction receive [] c1 c2"
thm EmptyExperimentE

lemma emptyExperimentComposition:
"reaction, receive ⊢ c1 =[]⇒ c2 ⟹
  reaction, receive ⊢ c2 =[]⇒ c3 ⟹
  reaction, receive ⊢ c1 =[]⇒ c3"
  by (smt (verit, best) GeneralBaseExperiment general_experiment_relation.cases list.distinct(1)
       star_trans)

lemma emptyNDExperimentComposition:
"A ⊢ c1 ==[]⇒ c2 ⟹
  A ⊢ c2 ==[]⇒ c3 ⟹
  A ⊢ c1 ==[]⇒ c3"
  by (smt (verit) full_nd_experiment_relation.simps list.discI star_trans)

lemma reactionsPrefixNDExperiment:
"A ⊢ c2 ==e⇒ c3 ⟹
  star internal_reaction c1 c2 ⟹
  A ⊢ c1 ==e⇒ c3"
proof (induction A c2 e c3 arbitrary: c1 rule: full_nd_experiment_relation.induct)
  case (BaseExperimentND c1 c2 A)
  then show ?case by (meson full_nd_experiment_relation.BaseExperimentND star_trans)
```

```
next
  case (StepExperimentND c1' c2 A μ c3 c4 rest c5)
  moreover have "star internal_reaction c1 c2" using calculation star_trans by metis
  ultimately show ?case using full_nd_experiment_relation.StepExperimentND by blast
qed

lemma reactionsPrefixExperiment:
"reaction, receive ⊢ c2 =e⇒ c3 ⟹
  star reaction c1 c2 ⟹
  reaction, receive ⊢ c1 =e⇒ c3"
proof (induction reaction receive c2 e c3 arbitrary: c1 rule: general_experiment_relation.induct)
  case (GeneralBaseExperiment c2 c3)
  then show ?case using emptyExperimentComposition general_experiment_relation.
       GeneralBaseExperiment by metis
next
  case (GeneralStepExperiment reaction c2 c3 receive inp c4 c5 rest c6 c1)
  moreover have "star reaction c1 c3" using calculation star_trans by metis
  ultimately show ?case using general_experiment_relation.GeneralStepExperiment by metis
qed

lemma fullNDExperimentComposition:
"A ⊢ c1 ==e1⇒ c2 ⟹
  A ⊢ c2 ==e2⇒ c3 ⟹
  A ⊢ c1 ==(e1 @ e2)⇒ c3"
proof (induction A c1 e1 c2 arbitrary: c3 rule: full_nd_experiment_relation.induct)
  case (BaseExperimentND c1 c2 A)
  then show ?case proof (induction c1 c2 rule: star.induct[of internal_reaction])
    case (refl c1)
    then show ?case by auto
  next
    case (step c1 c2 c2')
    moreover have "star internal_reaction c1 c2'" using calculation by (simp add: star.step)
    ultimately show ?case using reactionsPrefixNDExperiment by blast
  qed
next
  case (StepExperimentND c1 c2 A μ innerC3 c4 rest c5)
  moreover have "A ⊢ c4 ==(rest @ e2)⇒ c3" using calculation(5,6) by auto
  moreover have "A ⊢ innerC3 ==(rest @ e2)⇒ c3" using calculation reactionsPrefixNDExperiment by
        blast
  moreover have "A ⊢ c2 ==(μ # rest @ e2)⇒ c3" using calculation full_nd_experiment_relation.
       StepExperimentND star.refl by (metis full_nd_experiment_relation.StepExperimentND)
  ultimately show ?case using reactionsPrefixExperiment by (simp add: full_nd_experiment_relation
       .StepExperimentND)
qed
```

```
lemma generalExperimentComposition:
"reaction, receive ⊢ c1 =e1⇒ c2 ⟹
  reaction, receive ⊢ c2 =e2⇒ c3 ⟹
  reaction, receive ⊢ c1 =(e1 @ e2)⇒ c3"
proof (induction reaction receive c1 e1 c2 rule: general_experiment_relation.induct)
  case (GeneralBaseExperiment reaction c1 c2 receive)
  then show ?case proof (induction c1 c2 rule: star.induct[of reaction])
    case (refl c1)
    then show ?case by auto
  next
    case (step c1 c2 c2')
    moreover have "star reaction c1 c2'" using calculation by (simp add: star.step)
    ultimately show ?case by (metis star_step1 reactionsPrefixExperiment)
  qed
next
  case (GeneralStepExperiment reaction c1 c2 receive inp innerC3 c4 rest c5)
  moreover have "general_experiment_relation reaction receive c4 (rest @ e2) c3" using
       calculation by auto
  moreover have "general_experiment_relation reaction receive innerC3 (rest @ e2) c3" using
       calculation reactionsPrefixExperiment by auto
  moreover have "general_experiment_relation reaction receive c2 (inp # rest @ e2) c3" using
       calculation experiment_relation.StepExperiment star.refl by (metis
       general_experiment_relation.GeneralStepExperiment)
  ultimately show ?case using reactionsPrefixExperiment by auto
qed


(*
  The weak_sim and weak_sim_simple definitions are given in
  Robin Milner's book "Communicating and Mobile Systems: The π-calculus".
  They are said to be equivalent, and we show that here.
*)

(* c1 is weakly simulated by c2 if there is a binary relation from c1 to c2 where this property
     holds. *)
definition weak_sim :: "FConfig rel ⇒ bool" where
"weak_sim s ⟷ (∀c1 c2 . (c1, c2) ∈ s ⟶ (∀e c1' . c1 =e⇒ c1' ⟶ (∃c2' . c2 =e⇒ c2' ∧ (c1',
     c2') ∈ s)))"

definition weak_nd_sim :: "Proc rel ⇒ bool" where
"weak_nd_sim s ⟷ (∀c1 A1 c2 A2 . ((c1, A1), (c2, A2)) ∈ s ⟶ (∀e c1' . (A1 ⊢ c1 ==e⇒ c1') ⟶
      (∃c2' . (A2 ⊢ c2 ==e⇒ c2') ∧ ((c1', A1), (c2', A2)) ∈ s)))"
```

```
definition weak_nd_sim_simple :: "Proc rel ⇒ bool" where
"weak_nd_sim_simple s ⟷ (∀c1 A1 c2 A2 . ((c1, A1), (c2, A2)) ∈ s ⟶
  (∀c1' . (A1 ⊢ c1 —τ→ c1') ⟶ (∃c2' . c2 →*𝒯 c2' ∧ ((c1', A1), (c2', A2)) ∈ s)) ∧
  (∀μ c1' . (A1 ⊢ c1 —μ→ c1') ⟶ (∃c2' . (A2 ⊢ c2 ==[μ]⇒ c2') ∧ ((c1', A1), (c2', A2)) ∈ s)))
      "

definition observation_nd_bisimulation :: "Proc rel ⇒ bool" where
"observation_nd_bisimulation s ⟷ weak_nd_sim_simple s ∧ weak_nd_sim_simple (s⁻¹)"

definition observable_equivalence :: "Proc ⇒ Proc ⇒ bool" (infix "\<approx>" 55) where
"observable_equivalence p1 p2 ⟷ (∃r. observation_nd_bisimulation r ∧ (p1, p2) ∈ r)"

(*
  Simulations with arbitrary transition rules.
  Must be provided internal reaction rules and observable transition rules.
*)
definition general_weak_sim :: "('process ⇒ 'process ⇒ bool) ⇒
  ('process ⇒ 'label ⇒ 'process ⇒ bool) ⇒
  'process rel ⇒ bool" where
"general_weak_sim reaction receive s ⟷ (∀c1 c2 . (c1, c2) ∈ s ⟶
  (∀e c1' . reaction, receive ⊢ c1 =e⇒ c1' ⟶
    (∃c2' . reaction, receive ⊢ c2 =e⇒ c2' ∧ (c1', c2') ∈ s)))"

definition weak_sim_simple :: "FConfig rel ⇒ bool" where
"weak_sim_simple s ⟷ (∀c1 c2 . (c1, c2) ∈ s ⟶
  (∀c1' . c1 →𝒯 c1' ⟶ (∃c2' . c2 →*𝒯 c2' ∧ (c1', c2') ∈ s)) ∧
  (∀i c1' . c1 →i c1' ⟶ (∃c2' . c2 =[i]⇒ c2' ∧ (c1', c2') ∈ s)))"

definition general_weak_sim_simple :: "('process ⇒ 'process ⇒ bool) ⇒
  ('process ⇒ 'label ⇒ 'process ⇒ bool) ⇒
  'process rel ⇒ bool" where
"general_weak_sim_simple reaction receive s ⟷ (∀c1 c2 . (c1, c2) ∈ s ⟶
  (∀c1' . reaction c1 c1' ⟶ (∃c2' . star reaction c2 c2' ∧ (c1', c2') ∈ s)) ∧
  (∀i c1' . receive c1 i c1' ⟶ (∃c2' . reaction, receive ⊢ c2 =[i]⇒ c2' ∧ (c1', c2') ∈ s)))"

definition observation_bisimulation :: "('process ⇒ 'process ⇒ bool) ⇒
  ('process ⇒ 'label ⇒ 'process ⇒ bool) ⇒
  'process rel ⇒ bool" where
"observation_bisimulation reaction receive s ⟷ general_weak_sim_simple reaction receive s ∧
    general_weak_sim_simple reaction receive (s⁻¹)"

lemma concreteWeakSimDefInGeneralDef:
"weak_sim s = general_weak_sim internal_reaction small_step_receive_ccs s"
"weak_sim_simple s = general_weak_sim_simple internal_reaction small_step_receive_ccs s"
```

```
   apply (simp_all add: weak_sim_def general_weak_sim_def weak_sim_simple_def
        general_weak_sim_simple_def)
  using faustusExperimentInGeneral by presburger+

lemma initialReactionInEmptyExperimentGeneral:
"reaction c1 c1' ⟹ (reaction, receive ⊢ c1 =[]⇒ c1')"
  by (simp add: GeneralBaseExperiment)

lemma simpleWeakSimulationGeneralReaction:
"star reaction c1 c2 ⟹
  general_weak_sim_simple reaction receive s ⟹
  (c1, d1) ∈ s ⟹
  ∃d2 . star reaction d1 d2 ∧ (c2, d2) ∈ s"
  apply (induction c1 c2 arbitrary: d1 rule: star.induct[of reaction])
   apply blast
  by (smt (verit, best) star_trans general_weak_sim_simple_def)

lemma simpleWeakNDSimulationGeneralReaction:
"star internal_reaction c1 c2 ⟹
  weak_nd_sim_simple s ⟹
  ((c1, A1), (d1, A2)) ∈ s ⟹
  ∃d2 . star internal_reaction d1 d2 ∧ ((c2, A1), (d2, A2)) ∈ s"
proof (induction c1 c2 arbitrary: d1 rule: star.induct[of internal_reaction])
  case (refl x)
  then show ?case by blast
next
  case (step x y z)
  moreover obtain d2' where "((y, A1), d2', A2) ∈ s" using calculation by (metis
       full_nd_semantics_react_is_equivalent_to_internal_reaction weak_nd_sim_simple_def)
  ultimately show ?case by (smt (z3) BaseInternalFullND star_trans weak_nd_sim_simple_def)
qed

lemma simpleWeakSimulationImpliesWeakSimulation_aux:
"reaction, receive ⊢ c1 =e⇒ c1' ⟹
  general_weak_sim_simple reaction receive s ⟹
  (c1, c2) ∈ s ⟹
  ∃c2'. reaction, receive ⊢ c2 =e⇒ c2' ∧ (c1', c2') ∈ s"
proof (induction reaction receive "c1" e "c1'" arbitrary: c2 rule: general_experiment_relation.
     induct)
  case (GeneralBaseExperiment reaction c1 c1' receive)
  then show ?case proof (induction c1 c1' arbitrary: c2 rule: star.induct[of reaction])
    case (refl x)
    then show ?case by (metis star.refl general_experiment_relation.GeneralBaseExperiment)
  next
```

```
    case (step x y z)
    moreover obtain c2y where "star reaction c2 c2y ∧ (y, c2y) ∈ s" using
         general_weak_sim_simple_def calculation by metis
    moreover obtain c2' where "reaction, receive ⊢ c2y =[]⇒ c2' ∧ (z, c2') ∈ s" using
         calculation by blast
    moreover have "reaction, receive ⊢ c2 =[]⇒ c2y" using calculation
         general_experiment_relation.GeneralBaseExperiment by metis
    moreover have "reaction, receive ⊢ c2 =[]⇒ c2'" using calculation emptyExperimentComposition
          by metis
    ultimately show ?case by blast
  qed
next
  case (GeneralStepExperiment reaction c1 c2 receive inp c3 c4 rest c5 d1)
  moreover obtain d2 where "star reaction d1 d2 ∧ (c2, d2) ∈ s" using calculation(1,6,7)
       simpleWeakSimulationGeneralReaction weak_sim_simple_def by metis
  moreover obtain d3 where "general_experiment_relation reaction receive d2 [inp]  d3 ∧ (c3, d3)
       ∈ s" using general_weak_sim_simple_def calculation by metis
  moreover obtain d4 where "star reaction d3 d4 ∧ (c4, d4) ∈ s" using calculation(3,6,9)
       simpleWeakSimulationGeneralReaction weak_sim_simple_def by metis
  moreover obtain d5 where "general_experiment_relation reaction receive d4 rest d5 ∧ (c5, d5) ∈
       s" using calculation by blast
  ultimately show ?case by (metis (mono_tags, opaque_lifting) append_Cons append_Nil
       generalExperimentComposition reactionsPrefixExperiment)
qed

lemma simpleWeakNDSimulationImpliesWeakNDSimulation_aux:
"(A1 ⊢ c1 ==e⇒ c1') ⟹
  weak_nd_sim_simple s ⟹
  ((c1, A1), (c2, A2)) ∈ s ⟹
  ∃c2'. (A2 ⊢ c2 ==e⇒ c2') ∧ ((c1', A1), (c2', A2)) ∈ s"
proof (induction A1 "c1" e "c1'" arbitrary: c2 rule: full_nd_experiment_relation.induct)
  case (BaseExperimentND c1 c1' A)
  then show ?case proof (induction c1 c1' arbitrary: c2 rule: star.induct[])
    case (refl x)
    then show ?case using full_nd_experiment_relation.BaseExperimentND by blast
  next
    case (step x y z)
    moreover obtain c2y where "star internal_reaction c2 c2y ∧ ((y, A), (c2y, A2)) ∈ s" using
         weak_nd_sim_simple_def calculation BaseInternalFullND by presburger
    moreover obtain c2' where "(A2 ⊢ c2y ==[]⇒ c2') ∧ ((z, A), (c2', A2)) ∈ s" using calculation
          by blast
    moreover have "A2 ⊢ c2 ==[]⇒ c2y" using calculation full_nd_experiment_relation.
         BaseExperimentND by blast
    moreover have "A2 ⊢ c2 ==[]⇒ c2'" using calculation emptyNDExperimentComposition by metis
```

```
    ultimately show ?case by blast
  qed
next
  case (StepExperimentND c1 c1' A μ c1'' c1''' rest c1'''')
  moreover obtain c2' where "star internal_reaction c2 c2' ∧ ((c1', A), (c2', A2)) ∈ s" using
       calculation(1,6,7) weak_nd_sim_simple_def by (meson simpleWeakNDSimulationGeneralReaction)
  moreover obtain c2'' where "(A2 ⊢ c2' ==[μ]⇒ c2'') ∧ ((c1'', A), (c2'', A2)) ∈ s" using
       weak_nd_sim_simple_def calculation by metis
  moreover obtain c2''' where "star internal_reaction c2'' c2''' ∧ ((c1''', A), (c2''', A2)) ∈ s"
        using calculation(3,6,9) simpleWeakNDSimulationGeneralReaction weak_nd_sim_simple_def by
       metis
  moreover obtain c2'''' where "(A2 ⊢ c2''' ==rest⇒ c2'''') ∧ ((c1'''', A), c2'''', A2) ∈ s"
       using calculation by blast
  moreover have "A2 ⊢ c2 ==μ#rest⇒ c2'''''" by (metis (full_types) append_Cons append_Nil
       calculation(10,11,8,9) fullNDExperimentComposition reactionsPrefixNDExperiment)
  ultimately show ?case by blast
qed

lemma simpleWeakSimulationImpliesWeakSimulation:
"general_weak_sim_simple reaction receive s ⟶ general_weak_sim reaction receive s"
  by (smt (verit, ccfv_SIG) simpleWeakSimulationImpliesWeakSimulation_aux general_weak_sim_def
       general_weak_sim_simple_def)

lemma simpleWeakNDSimulationImpliesWeakNDSimulation:
"weak_nd_sim_simple s ⟶ weak_nd_sim s"
  using simpleWeakNDSimulationImpliesWeakNDSimulation_aux weak_nd_sim_def by presburger

lemma weakSimulationImpliesSimpleWeakSimulation:
"general_weak_sim reaction receive s ⟶ general_weak_sim_simple reaction receive s"
  apply (auto simp add: general_weak_sim_def general_weak_sim_simple_def)
   apply (smt (verit, best) general_experiment_relation.cases
        initialReactionInEmptyExperimentGeneral neq_Nil_conv)
  by (meson GeneralBaseExperiment GeneralStepExperiment star.refl)

lemma weakNDSimulationImpliesSimpleWeakNDSimulation:
"weak_nd_sim s ⟶ weak_nd_sim_simple s"
  apply (auto simp add: weak_nd_sim_def weak_nd_sim_simple_def)
   apply (metis BaseExperimentND full_nd_experiment_relation.cases
        full_nd_semantics_react_is_equivalent_to_internal_reaction list.distinct(1) star_step1)
  using single_full_nd_receive_in_experiment by presburger

lemma equivalenceIsWeakSimulation:
"general_weak_sim_simple reaction receive Id"
  by (simp add: general_weak_sim_def weakSimulationImpliesSimpleWeakSimulation)
```

```
inductive_cases FullSemanticsLambdaE[elim]: "A ⊢ (c1, ctx1, state1, env2, warnings2, payments2) −
     λ(a, aa, b)→ (c1', ctx1', state1', env1', warnings1', payments1')"
inductive_cases FullSemanticsTauE[elim]: "A ⊢ (c1, ctx1, state1, env2, warnings2, payments2) −τ→
      (c1', ctx1', state1', env1', warnings1', payments1')"
inductive_cases ReceiveWhenSomeElim: "(FaustusV2AST.FContract.When cases t cont, ctx1, state1,
     env1, warnings1, payments1) →(Some inp, lower, upper) (c1', ctx1', state1', env1',
     warnings1', payments1')"

lemma small_step_receive_time_change_before_or_during:
"(When cases t cont, ctx1, newTimeState, newTimeEnv, warnings, payments) →(i, lower, upper) (c1
     ', ctx1', state1', env1', warnings1', payments1') ⟹
  fixInterval (lower, upper) state = IntervalTrimmed newTimeEnv newTimeState ⟹
  (When cases t cont, ctx1, state, env, warnings, payments) →(i, lower, upper) (c1', ctx1',
       state1', env1', warnings1', payments1')"
proof (induction "When cases t cont" ctx1 newTimeState newTimeEnv warnings payments i lower upper
      c1' ctx1' state1' env1' warnings1' payments1' arbitrary: state rule: small_step_receive_ccs
     .induct[split_format(complete)])
  case (LambdaReceive lower upper state1 newTimeEnv newTimeState realLower realUpper inp newC
       newState newWarn i1 lower1 upper1 ctx env warnings payments)
  moreover have "lower1 = lower ∧ upper1 = upper" using calculation by auto
  moreover have "state1 = newTimeState" using calculation apply auto
    apply (cases "upper < lower", auto)
    apply (cases "upper < (FState.minTime state1)", auto)
    apply (cases "lower < (FState.minTime state1)", auto)
    apply (cases "realUpper < FState.minTime state", auto)
    by (cases "realLower < FState.minTime state", auto)
  moreover have "env = newTimeEnv" using calculation apply auto
    apply (cases "upper < lower", auto)
    apply (cases "upper < (FState.minTime state1)", auto)
    apply (cases "lower < (FState.minTime state1)", auto)
    apply (cases "realUpper < FState.minTime state", auto)
    apply (cases "lower < FState.minTime state", auto)
    apply (cases "realUpper < (FState.minTime state)", auto)
    by (cases "realLower < (FState.minTime state)", auto)
  ultimately show ?case apply (auto simp del: fixInterval.simps)
    using small_step_receive_ccs.LambdaReceive by presburger
next
  case (CCSTimeout lower upper state1 newTimeEnv newTimeState realLower realUpper i1 lower1
       upper1 ctx env warnings payments)
  moreover have "lower1 = lower ∧ upper1 = upper" using calculation by auto
  moreover have "state1 = newTimeState" using calculation apply auto
    apply (cases "upper < lower", auto)
    apply (cases "upper < (FState.minTime state1)", auto)
```

```
    apply (cases "lower < (FState.minTime state1)", auto)
    apply (cases "realUpper < FState.minTime state", auto)
    by (cases "realLower < FState.minTime state", auto)
  moreover have "env = newTimeEnv" using calculation apply auto
    apply (cases "upper < lower", auto)
    apply (cases "upper < (FState.minTime state1)", auto)
    apply (cases "lower < (FState.minTime state1)", auto)
    apply (cases "realUpper < FState.minTime state", auto)
    apply (cases "lower < FState.minTime state", auto)
    apply (cases "realUpper < (FState.minTime state)", auto)
    by (cases "realLower < (FState.minTime state)", auto)
  ultimately show ?case using small_step_receive_ccs.CCSTimeout by auto
qed

lemma full_nd_semantics_time_change_before_or_during:
"A ⊢ (When cases t cont, ctx1, newTimeState, newTimeEnv, warnings, payments) —λ(i, lower, upper)
     → (c1', ctx1', state1', env1', warnings1', payments1') ⟹
  fixInterval (lower, upper) state = IntervalTrimmed newTimeEnv newTimeState ⟹
  A ⊢ (When cases t cont, ctx1, state, env, warnings, payments) —λ(i, lower, upper)→ (c1', ctx1
       ', state1', env1', warnings1', payments1')"
proof (induction A "(When cases t cont, ctx1, newTimeState, newTimeEnv, warnings, payments)" "λ(i
     , lower, upper)" "(c1', ctx1', state1', env1', warnings1', payments1')" rule:
     full_nd_semantics.induct)
  case (BaseReceiveFullND i A)
  then show ?case by (meson full_nd_semantics.BaseReceiveFullND
       small_step_receive_time_change_before_or_during)
next
  case (RestrictedReceiveFullND i A lower upper p3)
  moreover obtain j where j_obtained: "j ∈ A ∧ (FaustusV2AST.FContract.When cases t cont, ctx1,
       newTimeState, newTimeEnv, warnings, payments) →(Some j, lower, upper) p3" using
       calculation by auto
  moreover have "(FaustusV2AST.FContract.When cases t cont, ctx1, state, env, warnings, payments)
        →(Some j, lower, upper) p3" using j_obtained RestrictedReceiveFullND(6)
       RestrictedReceiveFullND(2) small_step_receive_time_change_before_or_during[of cases t cont
        ctx1 newTimeState newTimeEnv warnings payments "Some j" lower upper ] apply (cases p3)
    by auto
  ultimately show ?case by (metis full_nd_semantics.RestrictedReceiveFullND)
next
  case (TimeoutFullND A)
  then show ?case by (meson full_nd_semantics.TimeoutFullND
       small_step_receive_time_change_before_or_during)
qed

inductive_cases FullNDInternalE: "A ⊢ c1 —τ→ c2"
```

```
end
```

The next Isabelle theory file proceeds from the formalization of Faustus processes. It verifies the correctness of the transformation on Faustus processes using the definition of simulation and bisimulation for Faustus processes.

Listing D.2: Formalization of Faustus bisimulation and transformation verification.

```
theory FaustusV2SplitTrans
  imports Main Util.MList Util.SList Core.ListTools "HOL-Library.Product_Lexorder" "HOL.
       Product_Type" "HOL-IMP.Star" FaustusV2Semantics HOL.Relation "HOL-Library.LaTeXsugar"
       FaustusV2Sim
begin

fun isFreshChoiceInChoiceId :: "ChoiceName ⇒ FChoiceId ⇒ bool" where
"isFreshChoiceInChoiceId cn (FChoiceId innerChoiceName p) = (cn ≠ innerChoiceName)"

fun isFreshChoiceInValue :: "ChoiceName ⇒ FValue ⇒ bool" and
  isFreshChoiceInObservation :: "ChoiceName ⇒ FObservation ⇒ bool" where
"isFreshChoiceInValue cn (Constant i) = True" |
"isFreshChoiceInValue cn (AvailableMoney accId token) = True" |
"isFreshChoiceInValue cn TimeIntervalStart = True" |
"isFreshChoiceInValue cn TimeIntervalEnd = True" |
"isFreshChoiceInValue cn (UseValue vId) = True" |
"isFreshChoiceInValue cn (NegValue v) = isFreshChoiceInValue cn v" |
"isFreshChoiceInValue cn (AddValue v1 v2) = (isFreshChoiceInValue cn v1 ∧ isFreshChoiceInValue cn
      v2)" |
"isFreshChoiceInValue cn (SubValue v1 v2) = (isFreshChoiceInValue cn v1 ∧ isFreshChoiceInValue cn
      v2)" |
"isFreshChoiceInValue cn (MulValue v1 v2) = (isFreshChoiceInValue cn v1 ∧ isFreshChoiceInValue cn
      v2)" |
"isFreshChoiceInValue cn (DivValue v1 v2) = (isFreshChoiceInValue cn v1 ∧ isFreshChoiceInValue cn
      v2)" |
"isFreshChoiceInValue cn (ChoiceValue choiceId) = isFreshChoiceInChoiceId cn choiceId" |
"isFreshChoiceInValue cn (Cond obs v1 v2) = (isFreshChoiceInObservation cn obs ∧
     isFreshChoiceInValue cn v1 ∧ isFreshChoiceInValue cn v2)" |
"isFreshChoiceInObservation cn TrueObs = True" |
"isFreshChoiceInObservation cn FalseObs = True" |
"isFreshChoiceInObservation cn (UseObservation oId) = True" |
"isFreshChoiceInObservation cn (AndObs obs1 obs2) = (isFreshChoiceInObservation cn obs1 ∧
     isFreshChoiceInObservation cn obs2)" |
"isFreshChoiceInObservation cn (OrObs obs1 obs2) = (isFreshChoiceInObservation cn obs1 ∧
     isFreshChoiceInObservation cn obs2)" |
```

```
"isFreshChoiceInObservation cn (NotObs obs) = isFreshChoiceInObservation cn obs" |
"isFreshChoiceInObservation cn (ChoseSomething choiceId) = isFreshChoiceInChoiceId cn choiceId" |
"isFreshChoiceInObservation cn (ValueGE v1 v2) = (isFreshChoiceInValue cn v1 ∧
     isFreshChoiceInValue cn v2)" |
"isFreshChoiceInObservation cn (ValueGT v1 v2) = (isFreshChoiceInValue cn v1 ∧
     isFreshChoiceInValue cn v2)" |
"isFreshChoiceInObservation cn (ValueLT v1 v2) = (isFreshChoiceInValue cn v1 ∧
     isFreshChoiceInValue cn v2)" |
"isFreshChoiceInObservation cn (ValueLE v1 v2) = (isFreshChoiceInValue cn v1 ∧
     isFreshChoiceInValue cn v2)" |
"isFreshChoiceInObservation cn (ValueEQ v1 v2) = (isFreshChoiceInValue cn v1 ∧
     isFreshChoiceInValue cn v2)"

fun isFreshChoiceInAction :: "ChoiceName ⇒ FAction ⇒ bool" where
"isFreshChoiceInAction cn (Choice (FChoiceId innerCn p) bounds) = (cn ≠ innerCn)" |
"isFreshChoiceInAction cn (Notify obs) = isFreshChoiceInObservation cn obs" |
"isFreshChoiceInAction cn (Deposit p1 p2 t v) = isFreshChoiceInValue cn v"

fun isFreshChoiceInStatement :: "ChoiceName ⇒ FStatement ⇒ bool" where
"isFreshChoiceInStatement cn (Pay accId payee token v) = isFreshChoiceInValue cn v" |
"isFreshChoiceInStatement cn (Assert obs) = isFreshChoiceInObservation cn obs" |
"isFreshChoiceInStatement cn (ReassignVal vId val) = isFreshChoiceInValue cn val" |
"isFreshChoiceInStatement cn (ReassignObservation oId obs) = isFreshChoiceInObservation cn obs" |
"isFreshChoiceInStatement cn (ReassignParty pkId pk) = True"

fun isFreshChoiceInStatements :: "ChoiceName ⇒ FStatement list ⇒ bool" where
"isFreshChoiceInStatements cn [] = True" |
"isFreshChoiceInStatements cn (s#stmts) = (isFreshChoiceInStatement cn s ∧
     isFreshChoiceInStatements cn stmts)"

fun isFreshChoiceInGuard :: "ChoiceName ⇒ FGuardExpression ⇒ bool" where
"isFreshChoiceInGuard cn (ActionGuard a) = isFreshChoiceInAction cn a" |
"isFreshChoiceInGuard cn (GuardThenGuard g1 g2) = (isFreshChoiceInGuard cn g1 ∧
     isFreshChoiceInGuard cn g2)" |
"isFreshChoiceInGuard cn (GuardStmtsGuard g stmts) = (isFreshChoiceInGuard cn g ∧
     isFreshChoiceInStatements cn stmts)" |
"isFreshChoiceInGuard cn (DisjointGuard g1 g2) = (isFreshChoiceInGuard cn g1 ∧
     isFreshChoiceInGuard cn g2)" |
"isFreshChoiceInGuard cn (InterleavedGuard g1 g2) = (isFreshChoiceInGuard cn g1 ∧
     isFreshChoiceInGuard cn g2)"

fun isFreshChoiceInArguments :: "ChoiceName ⇒ FArgument list ⇒ bool" where
"isFreshChoiceInArguments cn [] = True" |
"isFreshChoiceInArguments cn ((ValueArgument val)#args) = (isFreshChoiceInValue cn val ∧
```

```
     isFreshChoiceInArguments cn args)" |
"isFreshChoiceInArguments cn ((ObservationArgument obs)#args) = (isFreshChoiceInObservation cn
     obs ∧ isFreshChoiceInArguments cn args)" |
"isFreshChoiceInArguments cn ((PartyArgument pk)#args) = (isFreshChoiceInArguments cn args)"

fun isFreshChoiceInContract :: "ChoiceName ⇒ FContract ⇒ bool" and isFreshChoiceInCases :: "
     ChoiceName ⇒ FCase list ⇒ bool" where
"isFreshChoiceInContract cn Close = True" |
"isFreshChoiceInContract cn (StatementCont s cont) = (isFreshChoiceInStatement cn s ∧
     isFreshChoiceInContract cn cont)" |
"isFreshChoiceInContract cn (If obs cont1 cont2) = (isFreshChoiceInObservation cn obs ∧
     isFreshChoiceInContract cn cont1 ∧ isFreshChoiceInContract cn cont2)" |
"isFreshChoiceInContract cn (When cases t tcont) = (isFreshChoiceInContract cn tcont ∧
     isFreshChoiceInCases cn cases)" |
"isFreshChoiceInContract cn (Let valId val cont) = (isFreshChoiceInValue cn val ∧
     isFreshChoiceInContract cn cont)" |
"isFreshChoiceInContract cn (LetObservation valId obs cont) = (isFreshChoiceInObservation cn obs
     ∧ isFreshChoiceInContract cn cont)" |
"isFreshChoiceInContract cn (LetParty pkId pk cont) = (isFreshChoiceInContract cn cont)" |
"isFreshChoiceInContract cn (LetC cId params bodyCont cont) = (isFreshChoiceInContract cn
     bodyCont ∧ isFreshChoiceInContract cn cont)" |
"isFreshChoiceInContract cn (UseC cId args) = (isFreshChoiceInArguments cn args)" |
"isFreshChoiceInCases cn [] = True" |
"isFreshChoiceInCases cn (Case g cont#cases) = (isFreshChoiceInGuard cn g ∧
     isFreshChoiceInContract cn cont ∧ isFreshChoiceInCases cn cases)"

fun isFreshChoiceInCasesGuardsShallow :: "ChoiceName ⇒ FCase list ⇒ bool" where
"isFreshChoiceInCasesGuardsShallow cn [] = True" |
"isFreshChoiceInCasesGuardsShallow cn ((Case g cont)#cases) = (isFreshChoiceInGuard cn g ∧
     isFreshChoiceInCasesGuardsShallow cn cases)"

fun isFreshChoiceInCasesGuards :: "ChoiceName ⇒ FCase list ⇒ bool" where
"isFreshChoiceInCasesGuards cn [] = True" |
"isFreshChoiceInCasesGuards cn ((Case g cont)#cases) = (isFreshChoiceInGuard cn g ∧
     isFreshChoiceInCasesGuards cn cases)"

lemma isFreshChoiceInCasesImpliesIsFreshChoiceInCasesGuards:
"isFreshChoiceInCases cn cases ⟹
  isFreshChoiceInCasesGuards cn cases"
  apply (induction cases, auto)
  by (metis isFreshChoiceInCases.simps(2) isFreshChoiceInCasesGuards.elims(3) list.inject)

lemma isFreshChoiceInCasesGuardsImpliesIsFreshChoiceInCasesGuardsShallow:
"isFreshChoiceInCasesGuards cn cases ⟹
```

```
  isFreshChoiceInCasesGuardsShallow cn cases"
  apply (induction cases, auto)
  by (metis FaustusV2AST.FCase.exhaust isFreshChoiceInCasesGuards.simps(2)
       isFreshChoiceInCasesGuardsShallow.simps(2))

lemma isFreshChoiceInCasesPreservedInSlice:
"isFreshChoiceInCases cn cases ⟹
  cases = c1@c2 ⟹
  isFreshChoiceInCases cn c1 ∧ isFreshChoiceInCases cn c2"
proof (induction cases arbitrary: c1 c2)
  case Nil
  then show ?case by auto
next
  case (Cons a cases)
  then show ?case apply (cases a, meson)
     apply (smt (verit, best) Cons_eq_append_conv isFreshChoiceInCases.simps(1)
          isFreshChoiceInCases.simps(2))
    by (smt (verit) append_eq_Cons_conv append_eq_append_conv2 isFreshChoiceInCases.elims(1)
         isFreshChoiceInCases.simps(1) isFreshChoiceInCases.simps(2) list.inject)
qed

lemma isFreshChoiceInCasesGuardsPreservedInSlice:
"isFreshChoiceInCasesGuards cn cases ⟹
  cases = c1@c2 ⟹
  isFreshChoiceInCasesGuards cn c1 ∧ isFreshChoiceInCasesGuards cn c2"
proof (induction cases arbitrary: c1 c2)
  case Nil
  then show ?case by auto
next
  case (Cons a cases)
  then show ?case apply (cases a, meson)
     apply (smt (verit, best) Cons_eq_append_conv isFreshChoiceInCasesGuards.simps(1)
          isFreshChoiceInCasesGuards.simps(2))
    by (smt (verit) append_eq_Cons_conv append_eq_append_conv2 isFreshChoiceInCasesGuards.elims
         (1) isFreshChoiceInCasesGuards.simps list.inject)
qed

lemma isFreshChoiceInCasesGuardsShallowPreservedInSlice:
"isFreshChoiceInCasesGuardsShallow cn cases ⟹
  cases = c1@c2 ⟹
  isFreshChoiceInCasesGuardsShallow cn c1 ∧ isFreshChoiceInCasesGuardsShallow cn c2"
proof (induction cn cases arbitrary: c1 c2 rule: isFreshChoiceInCasesGuardsShallow.induct)
  case (1 cn)
  then show ?case by auto
```

```
next
  case (2 cn g cont cases)
  then show ?case apply meson
     apply (metis Cons_eq_append_conv isFreshChoiceInCasesGuardsShallow.simps(1)
          isFreshChoiceInCasesGuardsShallow.simps(2))
    by (metis Cons_eq_append_conv isFreshChoiceInCasesGuardsShallow.simps(2))
qed

fun isFreshChoiceInContext :: "ChoiceName ⇒ FContext ⇒ bool" where
"isFreshChoiceInContext cn [] = True" |
"isFreshChoiceInContext cn (PartyAbstraction i#ctx) = isFreshChoiceInContext cn ctx" |
"isFreshChoiceInContext cn (ValueAbstraction i#ctx) = isFreshChoiceInContext cn ctx" |
"isFreshChoiceInContext cn (ObservationAbstraction i#ctx) = isFreshChoiceInContext cn ctx" |
"isFreshChoiceInContext cn (ContractAbstraction (i, params, body)#ctx) = (isFreshChoiceInContract
      cn body ∧ isFreshChoiceInContext cn ctx)"

fun isFreshChoiceInChoices :: "ChoiceName ⇒ (ChoiceId × ChosenNum) list ⇒ bool" where
"isFreshChoiceInChoices cn [] = True" |
"isFreshChoiceInChoices cn ((ChoiceId innerCn p, n)#cs) = (cn ≠ innerCn ∧ isFreshChoiceInChoices
     cn cs)"

fun isFreshChoiceInState :: "ChoiceName ⇒ FState ⇒ bool" where
"isFreshChoiceInState cn s = isFreshChoiceInChoices cn (choices s)"

lemma freshChoiceInParamsContext:
"isFreshChoiceInContext cn (paramsToFContext params)"
  by (induction params rule: paramsToFContext.induct, auto)

lemma freshChoicePreserved_append:
"isFreshChoiceInContext cn ctx1 ⟹
  isFreshChoiceInContext cn ctx2 ⟹
  isFreshChoiceInContext cn (ctx1 @ ctx2)"
  by (induction ctx1 rule: isFreshChoiceInContext.induct, auto)

lemma freshChoicePreserved_evalArguments:
"evalFArguments env s params args = Some newState ⟹
  isFreshChoiceInState cn s ⟹
  isFreshChoiceInState cn newState"
  by (induction env s params args rule: evalFArguments.induct, auto split: option.split_asm)

lemma freshChoicePreserved_lookupContractIdAbsInformation:
"isFreshChoiceInContext cn ctx ⟹
  lookupContractIdAbsInformation ctx cid = Some (params, body, innerCtx) ⟹
  isFreshChoiceInContract cn body ∧ isFreshChoiceInContext cn innerCtx"
```

```
  by (induction cn ctx rule: isFreshChoiceInContext.induct, auto split: if_split_asm)

lemma freshChoicePreserved_appendParams:
"isFreshChoiceInContext cn ctx ⟹
  isFreshChoiceInContext cn (paramsToFContext params @ ctx)"
  by (induction cn ctx rule: isFreshChoiceInContext.induct; induction params rule:
       paramsToFContext.induct, auto)

lemma freshChoicePreserved_small_step_reduce_weak:
"(c1, ctx1, state1, env1, warnings1, payments1) →f2 (c2, ctx2, state2, env2, warnings2,
     payments2) ⟹
  isFreshChoiceInContract cn c1 ⟹
  isFreshChoiceInContext cn ctx1 ⟹
  isFreshChoiceInState cn state1 ⟹
  (isFreshChoiceInContract cn c2 ∧ isFreshChoiceInContext cn ctx2 ∧ isFreshChoiceInState cn
       state2)"
proof (induction c1 ctx1 state1 env1 warnings1 payments1 c2 ctx2 state2 env2 warnings2 payments2
     rule: small_step_reduce.induct[split_format(complete)])
  case (UseCFound)
  then show ?case using freshChoicePreserved_appendParams
       freshChoicePreserved_lookupContractIdAbsInformation freshChoicePreserved_evalArguments by
       presburger
qed auto

lemma freshChoicePreserved_small_step_reduce:
"(c1, ctx1, state1, env1, warnings1, payments1) →f2 (c2, ctx2, state2, env2, warnings2,
     payments2) ⟹
  isFreshChoiceInContract cn c1 ⟹
  isFreshChoiceInContext cn ctx1 ⟹
  (isFreshChoiceInContract cn c2 ∧ isFreshChoiceInContext cn ctx2)"
proof (induction c1 ctx1 state1 env1 warnings1 payments1 c2 ctx2 state2 env2 warnings2 payments2
     rule: small_step_reduce.induct[split_format(complete)])
  case (UseCFound ctx cid params bodyCont innerCtx env s args newState warns payments)
  then show ?case proof (induction ctx cid rule: lookupContractIdAbsInformation.induct)
    case (2 boundCid innerParams innerBodyCont restAbs cid)
    then show ?case using freshChoicePreserved_appendParams
         freshChoicePreserved_lookupContractIdAbsInformation by blast
  qed (auto split: if_split_asm)
qed auto

lemma freshChoicePreserved_insert:
"isFreshChoiceInChoices cn cs ⟹
  cn ≠ newCn ⟹
  isFreshChoiceInChoices cn (MList.insert (ChoiceId newCn p) choice cs)"
```

```
proof (induction "(ChoiceId newCn p)" choice cs arbitrary: cn newCn p rule: MList.insert.induct)
  case (2 b x y z)
  then show ?case by (cases x, auto)
qed auto

lemma freshChoicePreserved_action_step:
"ins ⊢ (a, s, env) →a_f2 (newS, newWarn) ⟹
  isFreshChoiceInAction cn a ⟹
  isFreshChoiceInState cn s ⟹
  (isFreshChoiceInState cn newS)"
proof (induction ins a s env newS newWarn rule: small_step_reduce_action.induct[split_format(
     complete)])
  case (ChoiceActionStep choice bounds s fChoiceId fChoiceIdRes mChoiceId env)
  then show ?case by (cases fChoiceId, auto split: option.split_asm simp add:
       freshChoicePreserved_insert)
qed auto

lemma freshChoiceNoMatch_action_step:
"isFreshChoiceInAction cn a ⟹ ¬ (IChoice (ChoiceId cn p) i) ⊢ (a, s, env) →a_f2 (newS, newWarn)
     "
  apply (induction cn a rule: isFreshChoiceInAction.induct, auto)
  by (elim ChoiceActionE, auto split: option.split_asm)

lemma freshChoiceMatchNoChoiceInput_action_step:
"isFreshChoiceInAction cn a ⟹ inp ⊢ (a, s, env) →a_f2 (newS, newWarn) ⟹ (\<nexists>p i . inp
     = (IChoice (ChoiceId cn p) i))"
  using freshChoiceNoMatch_action_step by auto

lemma freshChoicePreserved_statements_append:
"isFreshChoiceInStatements cn stmts1 ⟹
  isFreshChoiceInStatements cn stmts2 ⟹
  isFreshChoiceInStatements cn (stmts1 @ stmts2)"
  by (induction stmts1, auto)

lemma freshChoicePreserved_guard_step_weak:
"ins ⊢ (g, s, env) →g_f2 (stmts, nextG, newS, newWarn) ⟹
  isFreshChoiceInGuard cn g ⟹
  isFreshChoiceInState cn s ⟹
  (isFreshChoiceInStatements cn stmts ∧ isFreshChoiceInState cn newS ∧ (nextG = Some newG ⟶
       isFreshChoiceInGuard cn newG))"
proof (induction ins g s env stmts nextG newS newWarn arbitrary: newG rule:
     small_step_reduce_guard.induct[split_format(complete)])
  case (ActionGuardStep g a ins s env newS)
  then show ?case using freshChoicePreserved_action_step by auto
```

```
next
  case (GuardStmtsGuardCompleteGuardStep g innerG stmts ins s env newStmts newS)
  then show ?case by (auto simp add: freshChoicePreserved_statements_append)
qed auto

lemma freshChoicePreserved_guard_step:
"ins ⊢ (g, s, env) →g_f2 (stmts, nextG, newS, newWarn) ⟹
  isFreshChoiceInGuard cn g ⟹
  (isFreshChoiceInStatements cn stmts ∧ (nextG = Some newG ⟶ isFreshChoiceInGuard cn newG))"
proof (induction ins g s env stmts nextG newS newWarn arbitrary: newG rule:
     small_step_reduce_guard.induct[split_format(complete)])
  case (ActionGuardStep g a ins s env newS)
  then show ?case using freshChoicePreserved_action_step by auto
next
  case (GuardStmtsGuardCompleteGuardStep g innerG stmts ins s env newStmts newS)
  then show ?case by (auto simp add: freshChoicePreserved_statements_append)
qed auto

lemma freshChoiceNoMatch_guard_step:
"isFreshChoiceInGuard cn g ⟹ finalGuard (IChoice (ChoiceId cn p) i) (g, s, env)"
  apply (induction cn g rule: isFreshChoiceInGuard.induct, auto)
  using finalGuard_def freshChoiceNoMatch_action_step apply force
  using finalGuard_def apply force[1]
  using finalGuard_def apply auto[1]
   apply (meson DisjointGuardE finalGuard_def)
  by (metis (no_types, lifting) computeGuardStep.simps(5)
       computeGuardStepHaltedImpliesSmallStepHalted option.case_eq_if)

lemma freshChoiceMatchNoChoiceInput_guard_step:
"isFreshChoiceInGuard cn g ⟹
  inp ⊢ (g, s, env) →g_f2 (stmts, nextG, newS) ⟹
  (\<nexists>p i . inp = IChoice (ChoiceId cn p) i)"
  using finalGuard_def freshChoiceNoMatch_guard_step by blast

lemma freshChoicePreserved_prependStatements:
"isFreshChoiceInContract cn cont ⟹
  isFreshChoiceInStatements cn stmts ⟹
  isFreshChoiceInContract cn (prependStatements stmts cont)"
  by (induction stmts cont rule: prependStatements.induct, auto)

lemma freshChoicePreserved_case_step_weak:
"ins ⊢ (c, s, env) →case_f2 (newC, newS, newW) ⟹
  isFreshChoiceInCases cn [c] ⟹
  isFreshChoiceInState cn s ⟹
```

```
  (isFreshChoiceInState cn newS ∧ (isFreshChoiceInContract cn tc ⟶ isFreshChoiceInContract cn (
      newC t tc)))"
proof (induction ins c s env newC newS newW rule: small_step_reduce_case.induct[split_format(
     complete)])
  case (CaseGuardCompleteStep ins g s env stmts newS c)
  then show ?case apply (auto simp add: freshChoicePreserved_prependStatements
      freshChoicePreserved_guard_step)
    using freshChoicePreserved_guard_step_weak isFreshChoiceInState.simps by blast
next
  case (CaseGuardIncompleteStep ins g s env stmts innerG newS c)
  then show ?case apply (auto simp add: freshChoicePreserved_guard_step_weak
      freshChoicePreserved_prependStatements)
    using freshChoicePreserved_guard_step_weak isFreshChoiceInState.simps by blast
qed

lemma freshChoicePreserved_case_step:
"ins ⊢ (c, s, env) →case_f2 (newC, newS, newW) ⟹
  isFreshChoiceInCases cn [c] ⟹
  ((isFreshChoiceInContract cn tc ⟶ isFreshChoiceInContract cn (newC t tc)))"
proof (induction ins c s env newC newS newW rule: small_step_reduce_case.induct[split_format(
     complete)])
  case (CaseGuardCompleteStep ins g s env stmts newS c)
  then show ?case by (auto simp add: freshChoicePreserved_prependStatements
      freshChoicePreserved_guard_step)
next
  case (CaseGuardIncompleteStep ins g s env stmts innerG newS c)
  then show ?case by (auto simp add: freshChoicePreserved_guard_step
      freshChoicePreserved_prependStatements)
qed

lemma freshChoiceNoMatch_case_step:
"¬ ((IChoice (ChoiceId cn p) i) ⊢ (c, s, env) →case_f2 (newC, newS, newW) ∧
     isFreshChoiceInCasesGuards cn [c])"
  apply (auto)
  apply (induction "(IChoice (ChoiceId cn p) i)" c s env newC newS newW rule:
      small_step_reduce_case.induct[split_format(complete)])
  using finalGuard_def freshChoiceNoMatch_guard_step isFreshChoiceInCasesGuards.simps(2) by blast
      +

lemma freshChoiceMatchNoChoiceInput_case_step:
"isFreshChoiceInCases cn [c] ⟹
  ins ⊢ (c, s, env) →case_f2 (newC, newS, newW) ⟹
  (\<nexists>p i . (IChoice (ChoiceId cn p) i) = ins)"
  using freshChoiceNoMatch_case_step isFreshChoiceInCasesImpliesIsFreshChoiceInCasesGuards by
```

```
      blast

lemma freshChoiceShallowNoMatch_case_step:
"¬ ((IChoice (ChoiceId cn p) i) ⊢ (c, s, env) →case_f2 (newC, newS, newW) ∧
     isFreshChoiceInCasesGuardsShallow cn [c])"
  apply (auto)
  apply (induction "(IChoice (ChoiceId cn p) i)" c s env newC newS newW rule:
       small_step_reduce_case.induct[split_format(complete)])
  using finalGuard_def freshChoiceNoMatch_guard_step isFreshChoiceInCasesGuardsShallow.simps(2)
       by blast+

lemma freshChoiceShallowMatchNoChoiceInput_case_step:
"isFreshChoiceInCasesGuardsShallow cn [c] ⟹
  ins ⊢ (c, s, env) →case_f2 (newC, newS, newW) ⟹
  (\<nexists>p i . (IChoice (ChoiceId cn p) i) = ins)"
  using freshChoiceShallowNoMatch_case_step isFreshChoiceInCasesImpliesIsFreshChoiceInCasesGuards
        by blast

lemma freshChoicePreserved_cases_step_weak:
"ins ⊢ (c, s, env) →cases_f2 (newC, newS, newW) ⟹
  isFreshChoiceInCases cn c ⟹
  isFreshChoiceInContext cn ctx ⟹
  isFreshChoiceInState cn s ⟹
  (isFreshChoiceInState cn newS ∧ (isFreshChoiceInContract cn tc ⟶ isFreshChoiceInContract cn (
       newC t tc)))"
proof (induction ins c s env newC newS newW rule: small_step_reduce_cases.induct[split_format(
     complete)])
  case (CasesHeadMatches ins c s env newC newS rest)
  then show ?case apply auto
     apply (metis FaustusV2AST.FCase.exhaust freshChoicePreserved_case_step_weak
          isFreshChoiceInCases.simps(1) isFreshChoiceInCases.simps(2) isFreshChoiceInState.simps)
    by (metis CasesHeadMatches.prems(3) FaustusV2AST.FCase.exhaust
         freshChoicePreserved_case_step_weak isFreshChoiceInCases.simps(1) isFreshChoiceInCases.
         simps(2))
next
  case (CasesTailMatches ins rest s env newC newS c)
  then show ?case apply auto
    using isFreshChoiceInCases.elims(2) by blast+
qed

lemma freshChoicePreserved_cases_step:
"ins ⊢ (c, s, env) →cases_f2 (newC, newS, newW) ⟹
  isFreshChoiceInCases cn c ⟹
  ((isFreshChoiceInContract cn tc ⟶ isFreshChoiceInContract cn (newC t tc)))"
```

```
proof (induction ins c s env newC newS newW rule: small_step_reduce_cases.induct[split_format(
     complete)])
  case (CasesHeadMatches ins c s env newC newS rest)
  then show ?case apply auto
    by (metis FaustusV2AST.FCase.exhaust freshChoicePreserved_case_step isFreshChoiceInCases.
         simps(1) isFreshChoiceInCases.simps(2))
next
  case (CasesTailMatches ins rest s env newC newS c)
  then show ?case apply auto
    using isFreshChoiceInCases.elims(2) by blast+
qed

lemma freshChoiceNoMatch_cases_step_aux:
"(IChoice (ChoiceId cn p) i) ⊢ (c, s, env) →cases_f2 (newC, newS, newW) ⟹
  isFreshChoiceInCasesGuards cn c ⟹
  False"
  apply (induction "(IChoice (ChoiceId cn p) i)" c s env newC newS newW rule:
       small_step_reduce_cases.induct[split_format(complete)])
   apply (metis FaustusV2AST.FCase.exhaust freshChoiceNoMatch_case_step
        isFreshChoiceInCasesGuards.simps(1) isFreshChoiceInCasesGuards.simps(2))
  by (metis FaustusV2AST.FCase.exhaust isFreshChoiceInCasesGuards.simps(2))

lemma freshChoiceShallowNoMatch_cases_step_aux:
"(IChoice (ChoiceId cn p) i) ⊢ (c, s, env) →cases_f2 (newC, newS, newW) ⟹
  isFreshChoiceInCasesGuardsShallow cn c ⟹
  False"
  apply (induction "(IChoice (ChoiceId cn p) i)" c s env newC newS newW rule:
       small_step_reduce_cases.induct[split_format(complete)])
   apply (metis FaustusV2AST.FCase.exhaust freshChoiceShallowNoMatch_case_step
        isFreshChoiceInCasesGuardsShallow.simps)
  by (metis FaustusV2AST.FCase.exhaust isFreshChoiceInCasesGuardsShallow.simps(2))

lemma freshChoiceNoMatch_cases_step:
"isFreshChoiceInCasesGuards cn c ⟹ finalFCases (IChoice (ChoiceId cn p) i) (c, s, env)"
  using finalFCases_def freshChoiceNoMatch_cases_step_aux by fastforce

lemma freshChoiceMatchNoChoiceInput_cases_step:
"isFreshChoiceInCasesGuards cn c ⟹
  inp ⊢ (c, s, env) →cases_f2 (newC, newS) ⟹
  (\<nexists>p i . (IChoice (ChoiceId cn p) i) = inp)"
  using finalFCases_def freshChoiceNoMatch_cases_step by blast

lemma freshChoiceShallowNoMatch_cases_step:
"isFreshChoiceInCasesGuardsShallow cn c ⟹ finalFCases (IChoice (ChoiceId cn p) i) (c, s, env)"
```

```
  using finalFCases_def freshChoiceShallowNoMatch_cases_step_aux by fastforce

lemma freshChoiceShallowMatchNoChoiceInput_cases_step:
"isFreshChoiceInCasesGuardsShallow cn c ⟹
  inp ⊢ (c, s, env) →cases_f2 (newC, newS) ⟹
  (\<nexists>p i . (IChoice (ChoiceId cn p) i) = inp)"
  using finalFCases_def freshChoiceShallowNoMatch_cases_step by blast

fun isFreshChoiceInProcess :: "ChoiceName ⇒ FConfig ⇒ bool" where
"isFreshChoiceInProcess cn (c, ctx, s, env, warnings, payments) = (isFreshChoiceInContract cn c ∧
      isFreshChoiceInContext cn ctx)"

lemma choice_reaction_same_as_basic_when_fresh:
"isFreshChoiceInProcess cn p ⟹
  (cn ⊢ p →𝒯 p') = p →𝒯 p'"
  apply auto
  subgoal premises ps proof -
    show ?thesis using ps(2) ps(1) proof (induction cn p p' rule: internal_reaction_add_choice.
        induct)
      case (ChoiceReaction newLower newUpper state newTimeEnv newTimeState realLower realUpper t
          freshChoiceName party i cases newC newState cont ctx env warnings payments)
      then show ?case by (meson isFreshChoiceInCasesImpliesIsFreshChoiceInCasesGuards
          freshChoiceNoMatch_cases_step_aux isFreshChoiceInContract.simps(4)
          isFreshChoiceInProcess.simps)
    qed auto
  qed
  subgoal premises ps proof -
    show ?thesis using ps(2) ps(1) proof (induction p p' rule: internal_reaction.induct)
      case (SmallStepReduce c ctx s env warnings payments newC newCtx newS newEnv newWarnings
          newPayments)
      then show ?case by (simp add: internal_reaction_add_choice.BaseReaction internal_reaction.
          SmallStepReduce)
    qed
  qed
  done

lemma choice_receive_same_as_basic_when_fresh:
"isFreshChoiceInProcess cn p ⟹
  ((ChoiceId cn pt) ⊢ p →i p') = p →i p'"
apply auto
  subgoal premises ps proof -
    show ?thesis using ps(2) ps(1) proof (induction "(ChoiceId cn pt)" p i p' rule:
        small_step_receive_ccs_add_choice.induct)
      case (NoChoiceLambdaReceive f1 μ f2 inp lower upper)
```

```
      then show ?case by auto
    next
      case (ChoiceLambdaReceive f1 μ lower upper f2 f3 inp)
      then show ?case apply auto
        apply (elim ReceiveE, auto)
        by (meson freshChoiceNoMatch_cases_step_aux
              isFreshChoiceInCasesImpliesIsFreshChoiceInCasesGuards)
    next
      case (CCSTimeout μ lower upper f1 f2)
      then show ?case by auto
    qed
  qed
  subgoal premises ps proof -
    show ?thesis using ps(2) ps(1) proof (induction p i p' rule: small_step_receive_ccs.induct)
      case (LambdaReceive lower upper state newTimeEnv newTimeState realLower realUpper t inp
           cases newC newState i c ctx env warnings payments)
      then show ?case by (metis NoChoiceLambdaReceive freshChoiceNoMatch_cases_step_aux
           isFreshChoiceInCasesImpliesIsFreshChoiceInCasesGuards isFreshChoiceInContract.simps(4)
            isFreshChoiceInProcess.simps small_step_receive_ccs.LambdaReceive)
    next
      case (CCSTimeout lower upper state newTimeEnv newTimeState realLower realUpper t i cases c
           ctx env warnings payments)
      then show ?case by (meson small_step_receive_ccs.CCSTimeout
           small_step_receive_ccs_add_choice.CCSTimeout)
    qed
  qed
  done


lemma basic_reaction_preserves_fresh_choice:
"isFreshChoiceInProcess cn p ⟹
  p →𝒯 p' ⟹
  isFreshChoiceInProcess cn p'"
  using freshChoicePreserved_small_step_reduce internal_reaction.simps by auto

lemma basic_reaction_star_preserves_fresh_choice:
"p →⋆𝒯 p' ⟹
  isFreshChoiceInProcess cn p ⟹
  isFreshChoiceInProcess cn p'"
  by (induction p p' rule: star.induct[of internal_reaction], auto simp add:
       basic_reaction_preserves_fresh_choice)

lemma choice_reaction_preserves_fresh_choice:
"isFreshChoiceInProcess cn p ⟹
```

```
  (internal_reaction_add_choice cn) p p' ⟹
  isFreshChoiceInProcess cn p'"
  using choice_reaction_same_as_basic_when_fresh freshChoicePreserved_small_step_reduce
       internal_reaction.simps by auto

lemma choice_reaction_star_preserves_fresh_choice:
"star (internal_reaction_add_choice cn) p p' ⟹
  isFreshChoiceInProcess cn p ⟹
  isFreshChoiceInProcess cn p'"
  by (induction p p' rule: star.induct[of "internal_reaction_add_choice cn"], auto simp add:
       choice_reaction_preserves_fresh_choice)

lemma fix_time_preserves_choices:
"fixInterval (lower, upper) state = IntervalTrimmed newTimeEnv newTimeState ⟹
  choices state = choices newTimeState"
  apply (cases "lower < upper", auto)
   apply (cases "upper < FState.minTime state", auto)
   apply (cases "lower < minTime state", auto)
  apply (cases "upper < lower", auto)
  by (cases "upper < FState.minTime state", auto)

lemma basic_receive_preserves_fresh_choice:
"p →i p' ⟹
  isFreshChoiceInProcess cn p ⟹
  isFreshChoiceInProcess cn p'"
proof (induction p i p' rule: small_step_receive_ccs.induct)
  case (LambdaReceive lower upper state newTimeEnv newTimeState realLower realUpper t inp cases
       newC newState i c ctx env warnings payments)
  then show ?case by (metis freshChoicePreserved_cases_step isFreshChoiceInContract.simps(4)
       isFreshChoiceInProcess.simps)
next
  case (CCSTimeout lower upper state newTimeEnv newTimeState realLower realUpper t cases c ctx
       env warnings payments)
  then show ?case by (metis isFreshChoiceInContract.simps(4) isFreshChoiceInProcess.simps)
qed

lemma basic_receive_input_not_fresh_choice:
"p →μ p' ⟹
  isFreshChoiceInProcess cn p ⟹
  (\<nexists>p i lower upper. μ = (Some ((IChoice (ChoiceId cn p) i)), lower, upper))"
proof (induction p μ p' rule: small_step_receive_ccs.induct)
  case (LambdaReceive lower upper state newTimeEnv newTimeState realLower realUpper t inp cases
       newC newState i c ctx env warnings payments)
  then show ?case apply auto
```

```
    using freshChoiceMatchNoChoiceInput_cases_step
         isFreshChoiceInCasesImpliesIsFreshChoiceInCasesGuards by blast
qed auto

lemma choice_receive_preserves_fresh_choice:
"isFreshChoiceInProcess cn p ⟹
  (ChoiceId cn pt) ⊢ p →i p' ⟹
  isFreshChoiceInProcess cn p'"
  by (simp add: basic_receive_preserves_fresh_choice choice_receive_same_as_basic_when_fresh)

lemma choice_receive_input_not_fresh_choice:
"(ChoiceId cn pt) ⊢ p →μ p' ⟹
  isFreshChoiceInProcess cn p ⟹
  (\<nexists>p i lower upper. μ = (Some ((IChoice (ChoiceId cn p) i)), lower, upper))"
  by (simp add: basic_receive_input_not_fresh_choice choice_receive_same_as_basic_when_fresh)

lemma choice_star_reaction_same_as_basic_when_fresh:
"isFreshChoiceInProcess cn p ⟹
  star (internal_reaction_add_choice cn) p p' = p →*𝒯 p'"
  apply auto
  subgoal premises ps proof -
    show ?thesis using ps(2) ps(1) proof (induction p p' rule: star.induct[of "(
         internal_reaction_add_choice cn)"])
      case (step x y z)
      then show ?case by (smt (verit, ccfv_SIG) choice_reaction_same_as_basic_when_fresh
           freshChoicePreserved_small_step_reduce isFreshChoiceInProcess.simps internal_reaction.
           simps star.step)
    qed auto
  qed
  subgoal premises ps proof -
    show ?thesis using ps(2) ps(1) proof (induction p p' rule: star.induct[of "internal_reaction
         "])
      case (step x y z)
      then show ?case by (smt (verit, best) freshChoicePreserved_small_step_reduce
           isFreshChoiceInProcess.simps internal_reaction_add_choice.BaseReaction
           internal_reaction.simps star.step)
    qed auto
  qed
  done

lemma reaction_choice_experiment_same_as_basic_when_fresh:
"isFreshChoiceInProcess cn p ⟹
  general_experiment_relation (internal_reaction_add_choice cn) small_step_receive_ccs p inps p'
       = p =inps⇒ p'"
```

```
  apply auto
  subgoal premises ps proof -
    show ?thesis using ps(2) ps(1) proof (induction "(internal_reaction_add_choice cn)"
         small_step_receive_ccs p inps p' rule: general_experiment_relation.induct)
      case (GeneralBaseExperiment p1 p2)
      then show ?case using BaseExperiment choice_star_reaction_same_as_basic_when_fresh by blast
    next
      case (GeneralStepExperiment p1 p2 inp p3 p4 rest p5)
      then show ?case by (meson StepExperiment basic_receive_preserves_fresh_choice
           choice_reaction_star_preserves_fresh_choice
           choice_star_reaction_same_as_basic_when_fresh)
    qed
  qed
  subgoal premises ps proof -
    show ?thesis using ps(2) ps(1) proof (induction p inps p' rule: experiment_relation.induct)
      case (BaseExperiment c1 c2)
      then show ?case by (simp add: GeneralBaseExperiment
           choice_star_reaction_same_as_basic_when_fresh)
    next
      case (StepExperiment c1 c2 inp c3 c4 rest c5)
      then show ?case by (simp add: GeneralStepExperiment
           basic_reaction_star_preserves_fresh_choice basic_receive_preserves_fresh_choice
           choice_star_reaction_same_as_basic_when_fresh)
    qed
  qed
  done

lemma receive_choice_experiment_same_as_basic_when_fresh:
"isFreshChoiceInProcess cn p ⟹
  internal_reaction, (small_step_receive_ccs_add_choice (ChoiceId cn pt)) ⊢ p =inps⇒ p' = (p =
       inps⇒ p')"
  apply auto
  subgoal premises ps proof -
    show ?thesis using ps(2) ps(1) proof (induction "internal_reaction" "
         small_step_receive_ccs_add_choice (ChoiceId cn pt)" p inps p' rule:
         general_experiment_relation.induct)
      case (GeneralBaseExperiment p1 p2)
      then show ?case using BaseExperiment choice_star_reaction_same_as_basic_when_fresh by blast
    next
      case (GeneralStepExperiment p1 p2 inp p3 p4 rest p5)
      then show ?case by (meson StepExperiment basic_reaction_star_preserves_fresh_choice
           choice_receive_preserves_fresh_choice choice_receive_same_as_basic_when_fresh)
    qed
  qed
```

```
  subgoal premises ps proof -
    show ?thesis using ps(2) ps(1) proof (induction p inps p' rule: experiment_relation.induct)
      case (BaseExperiment c1 c2)
      then show ?case by (simp add: GeneralBaseExperiment
           choice_star_reaction_same_as_basic_when_fresh)
    next
      case (StepExperiment c1 c2 inp c3 c4 rest c5)
      then show ?case by (simp add: GeneralStepExperiment
           basic_reaction_star_preserves_fresh_choice basic_receive_preserves_fresh_choice
           choice_receive_same_as_basic_when_fresh)
    qed
  qed
  done

(*
  split_cases is based on the transformation required for large contracts. Split the When into
       multiple pieces
  https://github.com/input-output-hk/marlowe-cardano/blob/
       bf9eb11b91b24ee4d0495688fdaae15fe7d48d50/marlowe/best-practices.md?plain=1#L136
  https://github.com/input-output-hk/marlowe-cardano/blob/2
       a0db652f01c6c08b3d90b6052c4b84785811d05/marlowe-cli/cookbook/stabilized-collective-loan-
       failure.ipynb
*)

function split_cases :: "TokenName ⇒ ChoiceName ⇒ nat ⇒ FContract ⇒ FContract" where
"split_cases role choiceName k (When cases t cont) = (
  let
    newCases = map (λ(Case g c) ⇒ (Case g (split_cases role choiceName k c))) cases;
    newCont = split_cases role choiceName k cont
  in
  if (k > 1 ∧ length newCases > k)
  then (When (take k newCases @ [Case (ActionGuard (Choice (FChoiceId choiceName (Role role))
       [(0, 0)])) (split_cases role choiceName k (When (drop k cases) t cont))]) t newCont)
  else (When newCases t newCont))" |
"split_cases role choiceName k (Close) = Close" |
"split_cases role choiceName k (StatementCont s c) = (StatementCont s (split_cases role
     choiceName k c))" |
"split_cases role choiceName k (If e c1 c2) = (If e (split_cases role choiceName k c1) (
     split_cases role choiceName k c2))" |
"split_cases role choiceName k (Let i v c) = (Let i v (split_cases role choiceName k c))" |
"split_cases role choiceName k (LetObservation i v c) = (LetObservation i v (split_cases role
     choiceName k c))" |
"split_cases role choiceName k (LetParty i v c) = (LetParty i v (split_cases role choiceName k c)
     )" |
```

```
"split_cases role choiceName k (LetC i p b c) = (LetC i p (split_cases role choiceName k b) (
     split_cases role choiceName k c))" |
"split_cases role choiceName k (UseC i a) = (UseC i a)"
  by pat_completeness auto

lemma split_cases_termination_aux_1:
"FaustusV2AST.FCase.Case x1 x2 ∈ set cases ⟹
  FaustusV2AST.fContractSize x2 < Suc (FaustusV2AST.fContractSize cont + FaustusV2AST.fCasesSize
       cases)"
  apply (induction cases, auto)
  by (smt (verit, ccfv_SIG) FaustusV2AST.fCasesSize.elims add_less_imp_less_left less_trans_Suc
       list.discI list.inject not_add_less2 not_less_eq)

lemma split_cases_termination_aux_2:
"cases ≠ [] ⟹ k > 0 ⟹ FaustusV2AST.fCasesSize (drop k cases) < FaustusV2AST.fCasesSize cases"
proof (induction cases arbitrary: k)
  case Nil
  then show ?case by auto
next
  case (Cons a cases)
  moreover have "cases = [] ⟹ drop k (a # cases) = cases" using calculation by auto
  moreover have "fCasesSize (a # cases) > fCasesSize cases" by (cases a, auto)
  moreover have "k = 1 ⟹ drop k (a # cases) = cases" by auto
  moreover have "k > 1 ∧ cases ≠ [] ⟹ k-1 > 0 ∧ FaustusV2AST.fCasesSize (drop (k-1) cases) <
       FaustusV2AST.fCasesSize cases" using calculation by auto
  ultimately show ?case apply (cases "cases", auto)
    apply (cases "k = 1", auto)
    by (metis bot_nat_0.not_eq_extremum drop_Cons' dual_order.strict_trans)
qed

termination
  apply (relation "measures [(λx::((TokenName × ChoiceName × nat × FContract)) . case x of
    (t, cn, n, contract) ⇒ (fContractSize contract))]")
             apply (auto)
  using split_cases_termination_aux_1 apply presburger
  by (metis Suc_lessD drop_Nil drop_eq_Nil2 leD split_cases_termination_aux_2)

definition advP :: FParty where "advP \<equiv> Role [0 :: 8 word]"
definition rcmdk :: nat where "rcmdk \<equiv> 5"

function sc :: "ChoiceName ⇒ FContract ⇒ FContract" where
"sc cn (When cases t cont) = (
  let
    newCases = map (λ(Case g c) ⇒ (Case g (sc cn c))) cases;
```

```
    newCont = sc cn cont
  in
  if (rcmdk > 1 ∧ length newCases > rcmdk)
  then (When (take rcmdk newCases @
    [Case (ActionGuard (Choice (FChoiceId cn advP) [(0, 0)]))
      (sc cn (When (drop rcmdk cases) t cont))])
    t newCont)
  else (When newCases t newCont))" |
"sc cn (Close) = Close" |
"sc cn (StatementCont s c) = (StatementCont s (sc cn c))" |
"sc cn (If e c1 c2) = (If e (sc cn c1) (sc cn c2))" |
"sc cn (Let i v c) = (Let i v (sc cn c))" |
"sc cn (LetObservation i v c) = (LetObservation i v (sc cn c))" |
"sc cn (LetParty i v c) = (LetParty i v (sc cn c))" |
"sc cn (LetC i p b c) = (LetC i p (sc cn b) (sc cn c))" |
"sc cn (UseC i a) = (UseC i a)"
  by pat_completeness auto

termination apply (relation "measures [(λx::((ChoiceName × FContract)) . case x of
    (cn, contract) ⇒ (fContractSize contract))]")
            apply (auto)
  using split_cases_termination_aux_1 apply presburger
  by (metis Suc_lessD drop_Nil drop_eq_Nil2 leD split_cases_termination_aux_2)

lemma sc_is_split_cases:
"sc cn c = split_cases [0] cn rcmdk c"
proof (induction cn c rule: sc.induct)
  case (1 cn cases t cont)
  moreover have "map (FaustusV2AST.FCase.case_FCase (λg c. FaustusV2AST.FCase.Case g (sc cn c)))
       cases = map (FaustusV2AST.FCase.case_FCase (λg c. FaustusV2AST.FCase.Case g (split_cases
       [0] cn rcmdk c))) cases" using 1(1) apply (induction cases)
    by(auto split: FCase.split FCase.split_asm)
  moreover have "sc cn cont = split_cases [0] cn rcmdk cont" using 1 by auto
  ultimately show ?case by (cases "1 < rcmdk ∧ rcmdk < length cases", auto simp add: advP_def,
       presburger)
qed auto

fun split_cases_context :: "TokenName ⇒ ChoiceName ⇒ nat ⇒ FContext ⇒ FContext" where
"split_cases_context role choiceName k [] = []" |
"split_cases_context role choiceName k ((ContractAbstraction (i, p, body))#rest) = (
     ContractAbstraction (i, p, (split_cases role choiceName k body))#(split_cases_context role
     choiceName k rest))" |
"split_cases_context role choiceName k (h#rest) = (h#(split_cases_context role choiceName k rest)
     )"
```

```
lemma lookup_split_cases_context_preserved_original:
"split_cases_context role choiceName k ctx1 = ctx2 ⟹
  lookupContractIdAbsInformation ctx1 cid = Some (params, body, restCtx) ⟹
  lookupContractIdAbsInformation ctx2 cid = Some (params, split_cases role choiceName k body,
       split_cases_context role choiceName k restCtx)"
proof (induction ctx1 arbitrary: ctx2)
  case Nil
  then show ?case by auto
next
  case (Cons a ctx1)
  then show ?case by (cases a, auto)
qed

lemma lookup_split_cases_context_preserved_split_context:
"split_cases_context role choiceName k ctx1 = ctx2 ⟹
  lookupContractIdAbsInformation ctx2 cid = Some (params, body, restCtx) ⟹
  (∃body1 restCtx1 . lookupContractIdAbsInformation ctx1 cid = Some (params, body1, restCtx1) ∧
    split_cases role choiceName k body1 = body ∧
    split_cases_context role choiceName k restCtx1 = restCtx)"
proof (induction ctx1 arbitrary: ctx2)
  case Nil
  then show ?case by auto
next
  case (Cons a ctx1)
  then show ?case by (cases a, auto)
qed

lemma split_cases_context_append_distribute:
"split_cases_context role choiceName k (ctx1@ctx2) = (split_cases_context role choiceName k ctx1)
     @(split_cases_context role choiceName k ctx2)"
proof (induction ctx1)
  case Nil
  then show ?case by auto
next
  case (Cons a ctx1)
  then show ?case by (cases a, auto)
qed

lemma split_cases_context_append_params_preserved:
"split_cases_context role choiceName k ctx1 = ctx2 ⟹
  split_cases_context role choiceName k (paramsToFContext params @ ctx1) = (paramsToFContext
       params @ ctx2)"
proof (induction ctx1 arbitrary: ctx2)
```

```
  case Nil
  moreover have "split_cases_context role choiceName k (paramsToFContext params) =
       paramsToFContext params" using calculation by (induction params rule: paramsToFContext.
       induct, auto)
  ultimately show ?case by auto
next
  case (Cons a ctx1)
  then show ?case using split_cases_context_append_distribute by force
qed

fun state_minimal_eq :: "ChoiceName ⇒ FState ⇒ FState ⇒ bool" where
"state_minimal_eq choiceName s1 s2 = (accounts s2 = accounts s1 ∧
    boundValues s2 = boundValues s1 ∧
    boundParties s2 = boundParties s1 ∧
    minTime s2 = minTime s1 ∧
    (∀choiceNameMade choicePartyMade choiceMade .
      choiceNameMade ≠ choiceName ⟶
      MList.lookup (ChoiceId choiceNameMade choicePartyMade) (choices s1) = choiceMade ⟶
      MList.lookup (ChoiceId choiceNameMade choicePartyMade) (choices s2) = choiceMade))"

lemma state_minimal_eq_reflexive:
"state_minimal_eq choiceName s s" by auto

lemma state_minimal_eq_commutative:
"state_minimal_eq choiceName s1 s2 ⟹
  state_minimal_eq choiceName s2 s1" by auto

lemma state_minimal_eq_transitive:
"state_minimal_eq choiceName s1 s2 ⟹
  state_minimal_eq choiceName s2 s3 ⟹
  state_minimal_eq choiceName s1 s3" by auto

fun split_cases_relation :: "TokenName ⇒ ChoiceName ⇒ nat ⇒ FConfig ⇒ FConfig ⇒ bool" where
"split_cases_relation role choiceName k
  (c1, ctx1, s1, env1, warnings1, payments1)
  (c2, ctx2, s2, env2, warnings2, payments2) =
    (isFreshChoiceInProcess choiceName (c1, ctx1, s1, env1, warnings1, payments1) ∧
    c2 = split_cases role choiceName k c1 ∧
    ctx2 = split_cases_context role choiceName k ctx1 ∧
    payments1 = payments2 ∧
    warnings1 = warnings2 ∧
    env2 = env1 ∧
    state_minimal_eq choiceName s1 s2)"
```

```
fun split_cases_relation_set :: "TokenName ⇒ ChoiceName ⇒ nat ⇒ FConfig rel" where
"split_cases_relation_set role choiceName k =
  {((c1, ctx1, s1, env1, warnings1, payments1), (c2, ctx2, s2, env2, warnings2, payments2)) |
   c1 ctx1 s1 env1 warnings1 payments1 c2 ctx2 s2 env2 warnings2 payments2 .
    (isFreshChoiceInProcess choiceName (c1, ctx1, s1, env1, warnings1, payments1) ∧
    c2 = split_cases role choiceName k c1 ∧
    ctx2 = split_cases_context role choiceName k ctx1 ∧
    payments1 = payments2 ∧
    warnings1 = warnings2 ∧
    env2 = env1 ∧
    state_minimal_eq choiceName s1 s2)}"

lemma split_cases_relation_gives_set:
"split_cases_relation role cn k p1 p2 ⟷ (p1, p2) ∈ (split_cases_relation_set role cn k)"
  by (cases p1; cases p2; auto)

lemma fresh_choice_state_preserves_party_evaluation:
"evalFParty s1 pk = s1Res ⟹
  boundParties s1 = boundParties s2 ⟹
  evalFParty s2 pk = s1Res"
  by (cases pk, auto)

lemma fresh_choice_state_preserves_payee_evaluation:
"evalFPayee s1 pk = s1Res ⟹
  boundParties s1 = boundParties s2 ⟹
  evalFPayee s2 pk = s1Res"
  apply (cases pk, auto)
  by (metis fresh_choice_state_preserves_party_evaluation)+

lemma fresh_choice_state_preserves_choiceId_evaluation:
"evalFChoiceId s1 choiceId = s1Res ⟹
  boundParties s1 = boundParties s2 ⟹
  evalFChoiceId s2 choiceId = s1Res"
  by (metis evalFChoiceId.elims evalFChoiceId.simps fresh_choice_state_preserves_party_evaluation
       )

lemma fresh_choice_state_preserves_value_evaluation:
"evalFValue env s1 val = s1Res ⟹
  isFreshChoiceInValue choiceName val ⟹
  state_minimal_eq choiceName s1 s2 ⟹
  evalFValue env s2 val = s1Res"
"evalFObservation env s1 obs = s1ResObs ⟹
  isFreshChoiceInObservation choiceName obs ⟹
  state_minimal_eq choiceName s1 s2 ⟹
```

```
  evalFObservation env s2 obs = s1ResObs"
proof (induction env s1 val and env s1 obs arbitrary: s1Res and s1ResObs rule:
     evalFValue_evalFObservation.induct)
  case (1 env state fParty token)
  then show ?case apply (simp only: evalFValue.simps state_minimal_eq.simps)
    by (metis fresh_choice_state_preserves_party_evaluation)
next
  case (8 env state choId)
  moreover have "evalFChoiceId state choId = evalFChoiceId s2 choId" by (metis calculation(3)
       state_minimal_eq.simps fresh_choice_state_preserves_choiceId_evaluation)
  moreover obtain innerChoiceName innerParty where "choId = FChoiceId innerChoiceName innerParty"
        using calculation state_minimal_eq.simps FChoiceId.exhaust by blast
  moreover have "innerChoiceName ≠ choiceName" using calculation by auto
  moreover have "(∀choiceParty . lookup (ChoiceId innerChoiceName choiceParty) (choices state) =
       lookup (ChoiceId innerChoiceName choiceParty) (choices s2))" using calculation(3)
       calculation(6) by force
  moreover have "(∀choiceParty . findWithDefault 0 (ChoiceId innerChoiceName choiceParty) (
       choices state) = findWithDefault 0 (ChoiceId innerChoiceName choiceParty) (choices s2))"
       using calculation(7) by auto
  ultimately show ?case apply (simp only: evalFValue.simps)
    subgoal premises ps
      using ps(1) ps(8) by (cases "evalFParty s2 innerParty", auto)
    done
next
  case (16 env state choId)
  moreover have "evalFChoiceId state choId = evalFChoiceId s2 choId" by (metis calculation(3)
       state_minimal_eq.simps fresh_choice_state_preserves_choiceId_evaluation)
  moreover obtain innerChoiceName innerParty where "choId = FChoiceId innerChoiceName innerParty"
        using calculation FChoiceId.exhaust by blast
  moreover have "innerChoiceName ≠ choiceName" using calculation(2) calculation(5) by auto
  moreover have "(∀choiceParty . lookup (ChoiceId innerChoiceName choiceParty) (choices state) =
       lookup (ChoiceId innerChoiceName choiceParty) (choices s2))" using calculation(3)
       calculation(6) by force
  moreover have "(∀choiceParty . member (ChoiceId innerChoiceName choiceParty) (choices state) =
       member (ChoiceId innerChoiceName choiceParty) (choices s2))" by (simp add: calculation(7))
  ultimately show ?case apply (simp only: evalFObservation.simps)
    subgoal premises ps
      using ps(1) ps(8) by (cases "evalFParty s2 innerParty", auto)
    done
qed (auto split: option.split option.split_asm)

lemma fresh_choice_state_preserves_argument_evaluation:
"evalFArguments env s1 params args = Some newS1 ⟹
  isFreshChoiceInArguments choiceName args ⟹
```

```
  state_minimal_eq choiceName s1 s2 ⟹
  (∃newS2 . evalFArguments env s2 params args = Some newS2 ∧
    state_minimal_eq choiceName newS1 newS2)"
proof (induction env s1 params args arbitrary: newS1 s2 rule: evalFArguments.induct)
  case (2 e s vid restParams val restArgs)
  from 2 have val_same: "evalFValue e s val = evalFValue e s2 val" by (metis
       fresh_choice_state_preserves_value_evaluation(1) isFreshChoiceInArguments.simps(2))
  from 2 val_same show ?case apply auto
    by (cases "evalFValue e s2 val", auto)+
next
  case (3 e s obsId restParams obs restArgs)
  moreover from calculation have "evalFObservation e s obs = evalFObservation e s2 obs" by (metis
        fresh_choice_state_preserves_value_evaluation(2) isFreshChoiceInArguments.simps(3))
  moreover obtain res where "evalFObservation e s2 obs = Some res" using calculation by fastforce
  moreover obtain intermediateS2 where "intermediateS2 = s2⦇FState.boundValues := MList.insert
       obsId (if res then 1 else 0) (FState.boundValues s2)⦈" by auto
  moreover obtain intermediateS1 where "intermediateS1 = s⦇FState.boundValues := MList.insert
       obsId (if res then 1 else 0) (FState.boundValues s)⦈" by auto
  moreover have "evalFArguments e intermediateS1 restParams restArgs = Some newS1" using
       calculation apply meson
    apply (simp_all only: evalFArguments.simps)
    by (metis option.simps(5))
  moreover have "boundParties intermediateS2 = boundParties intermediateS1" using calculation by
       (metis state_minimal_eq.simps FState.select_convs(4) FState.surjective FState.update_convs
       (3))
  moreover have "boundValues intermediateS2 = boundValues intermediateS1" using calculation by (
       metis state_minimal_eq.simps FState.select_convs(3) FState.surjective FState.update_convs
       (3))
  moreover have "accounts intermediateS2 = accounts intermediateS1" using calculation by (metis
       state_minimal_eq.simps FState.select_convs(1) FState.surjective FState.update_convs(3))
  moreover have "∀choiceNameMade choicePartyMade choiceMade. choiceNameMade ≠ choiceName ⟶
       lookup (ChoiceId choiceNameMade choicePartyMade) (FState.choices intermediateS1) =
            choiceMade ⟶
       lookup (ChoiceId choiceNameMade choicePartyMade) (FState.choices intermediateS2) =
            choiceMade" using calculation by (metis state_minimal_eq.simps FState.select_convs(2)
             FState.surjective FState.update_convs(3))
  moreover obtain newS2 where "(evalFArguments e intermediateS2 restParams restArgs) = Some newS2
       " using "3.IH"[of res newS1 intermediateS2] calculation(2-) by auto
  moreover have "boundParties newS1 = boundParties newS2" using "3.IH"[of res newS1
       intermediateS2] calculation(2-) by auto
  moreover have "boundValues newS1 = boundValues newS2"  using "3.IH"[of res newS1 intermediateS2
       ] calculation(2-) by auto
  moreover have "accounts newS1 = accounts newS2" using "3.IH"[of res newS1 intermediateS2]
       calculation(2-) by auto
```

```
  moreover have "minTime newS1 = minTime newS2" using "3.IH"[of res newS1 intermediateS2]
       calculation(2-) by auto
  moreover have "(∀lookupChoiceName lookupChoiceParty . lookupChoiceName ≠ choiceName ⟶ lookup
       (ChoiceId lookupChoiceName lookupChoiceParty) (choices newS1) = lookup (ChoiceId
       lookupChoiceName lookupChoiceParty) (choices newS2))" using calculation(1)[of res newS1
       intermediateS2] calculation(2-) by auto
  ultimately show ?case by force
next
  case (4 e s pkId restParams pk restArgs)
  then show ?case apply (simp only: state_minimal_eq.simps)
    by (smt (verit, ccfv_threshold) FState.select_convs(1-5) FState.surjective FState.
         update_convs(4) evalFArguments.simps(4) fresh_choice_state_preserves_party_evaluation
         isFreshChoiceInArguments.simps(4) option.case_eq_if option.collapse option.discI)
qed auto

lemma small_step_preserves_split_cases_relation:
"(c1, ctx1, state1, env1, [], payments1) →f2 (c1', ctx1', state1', env1', reduceWarnings1',
     payments1') ⟹
  split_cases_relation role choiceName k (c1, ctx1, state1, env1, warnings1, payments1) (c2, ctx2
       , state2, env2, warnings2, payments2) ⟹
  (∃c2' ctx2' state2' env2' reduceWarnings2' payments2' . (c2, ctx2, state2, env2, [], payments2)
        →f2 (c2', ctx2', state2', env2', reduceWarnings2', payments2') ∧
    split_cases_relation role choiceName k (c1', ctx1', state1', env1', warnings1 @
         convertReduceWarnings reduceWarnings1', payments1') (c2', ctx2', state2', env2',
         warnings2 @ convertReduceWarnings reduceWarnings2', payments2'))"
proof (induction c1 ctx1 state1 env1 "[] :: ReduceWarning list" payments1 c1' ctx1' state1' env1'
      reduceWarnings1' payments1'
    arbitrary: c2 ctx2 state2 env2 payments2 warnings2 warnings1
    rule: small_step_reduce_induct)
  case (CloseRefund s party token money newAccount ctx env payments)
  moreover have "refundOne (FState.accounts state2) = Some ((party, token, money), newAccount)"
       using calculation by auto
  moreover have "(Close, split_cases_context role choiceName k ctx, state2, env2, [], payments2)
       →f2 (Close, split_cases_context role choiceName k ctx, state2⦇FState.accounts :=
       newAccount⦈, env2, [ReduceNoWarning], payments2 @ [Payment party (Payee.Party party) token
        money])" using calculation apply (auto simp add: calculation)
    by (metis CloseRefund.hyps FaustusV2Semantics.small_step_reduce.CloseRefund self_append_conv2
         )
  moreover obtain p1 p2 where "p1 = (Close, ctx, s⦇FState.accounts := newAccount⦈, env,
       convertReduceWarnings ([ReduceNoWarning]), payments @ [Payment party (Payee.Party party)
       token money]) ∧
    p2 = (Close, split_cases_context role choiceName k ctx, state2⦇FState.accounts := newAccount⦈
         , env2, convertReduceWarnings ([ReduceNoWarning]), payments2 @ [Payment party (Payee.
         Party party) token money])" by auto
```

```
  moreover have "split_cases_relation role choiceName k p1 p2" using calculation by auto
  ultimately show ?case by (metis isFreshChoiceInProcess.simps self_append_conv2
        split_cases_relation.simps)
next
  case (PayNonPositive env s val res fromAcc accId toPayee payee token cont ctx payments)
  moreover have "evalFValue env s val = evalFValue env state2 val" using calculation(1)
        calculation(5) apply auto
    by (metis (no_types, opaque_lifting) calculation(5)
          fresh_choice_state_preserves_value_evaluation(1) split_cases_relation.simps)
  moreover have "evalFParty s accId = evalFParty state2 accId" using calculation(3) calculation
        (5) apply auto
    using fresh_choice_state_preserves_party_evaluation by blast
  moreover have "evalFPayee s payee = evalFPayee state2 payee" using calculation(4) calculation
        (5) apply auto
    using fresh_choice_state_preserves_payee_evaluation by blast
  moreover have "(StatementCont (FStatement.Pay accId payee token val) (split_cases role
        choiceName k cont), split_cases_context role choiceName k ctx, state2, env2, [], payments2
        ) →f2 (split_cases role choiceName k cont, split_cases_context role choiceName k ctx,
        state2, env2, [] @ [ReduceNonPositivePay fromAcc toPayee token res], payments2)" using
        calculation by (metis FaustusV2Semantics.small_step_reduce.PayNonPositive
        split_cases_relation.simps)
  ultimately show ?case apply auto
    by (metis \<open>(StatementCont (FStatement.Pay accId payee token val) (split_cases role
          choiceName k cont), split_cases_context role choiceName k ctx, state2, env2, [],
          payments2) →f2 (split_cases role choiceName k cont, split_cases_context role choiceName
           k ctx, state2, env2, [] @ [ReduceNonPositivePay fromAcc toPayee token res], payments2)
          \<close> convertReduceWarnings.simps(1) convertReduceWarnings.simps(3) self_append_conv2
          )
next
  case (PayPositivePartialWithPayment env s val res fromAcc accId toPayee payee token newAccs
        moneyToPay payment finalAccs somePayment cont ctx payments)
  moreover have "evalFValue env s val = evalFValue env state2 val" using calculation(1)
        calculation(10) apply auto
    by (metis (no_types, lifting) calculation(10) fresh_choice_state_preserves_value_evaluation
          (1) split_cases_relation.simps)
  moreover have "evalFParty s accId = evalFParty state2 accId" using calculation(3) calculation
        (10) apply auto
    using fresh_choice_state_preserves_party_evaluation by blast
  moreover have "evalFPayee s payee = evalFPayee state2 payee" using calculation(4) calculation
        (10) apply auto
    using fresh_choice_state_preserves_payee_evaluation by blast
  moreover have "lookup (fromAcc, token) (FState.accounts s) = lookup (fromAcc, token) (FState.
        accounts state2)" using calculation by auto
  moreover have "(StatementCont (FStatement.Pay accId payee token val) (split_cases role
```

```
      choiceName k cont), split_cases_context role choiceName k ctx, state2, env2, [], payments2
      ) →f2
    (split_cases role choiceName k cont, split_cases_context role choiceName k ctx, state2⦇FState
        .accounts := finalAccs⦈, env2, [] @ [ReducePartialPay fromAcc toPayee token moneyToPay
        res], payments2 @ [somePayment])" using calculation apply (auto split: option.split_asm
        option.split Payee.split Payee.split_asm)
    by (metis FaustusV2Semantics.small_step_reduce.PayPositivePartialWithPayment calculation
        calculation(7) calculation(8) self_append_conv2)+
  moreover have "split_cases_relation role choiceName k
    (cont, ctx, s⦇FState.accounts := finalAccs⦈, env, ([] @ convertReduceWarnings [
        ReducePartialPay fromAcc toPayee token moneyToPay res]), payments @ [somePayment])
    (split_cases role choiceName k cont, split_cases_context role choiceName k ctx, state2⦇FState
        .accounts := finalAccs⦈, env2, ([] @ convertReduceWarnings [ReducePartialPay fromAcc
        toPayee token moneyToPay res]), payments2 @ [somePayment])" using calculation by auto
  ultimately show ?case by (metis isFreshChoiceInProcess.simps split_cases.simps(3)
      split_cases_relation.simps)
next
  case (PayPositivePartialWithoutPayment env s val res fromAcc accId toPayee payee token newAccs
      moneyToPay payment finalAccs cont ctx payments)
  moreover have "evalFValue env s val = evalFValue env state2 val" using calculation(1)
      calculation(10) apply auto
    by (metis (no_types, lifting) calculation(10) fresh_choice_state_preserves_value_evaluation
        (1) split_cases_relation.simps)
  moreover have "evalFParty s accId = evalFParty state2 accId" using calculation(3) calculation
      (10) apply auto
    using fresh_choice_state_preserves_party_evaluation by blast
  moreover have "evalFPayee s payee = evalFPayee state2 payee" using calculation(4) calculation
      (10) apply auto
    using fresh_choice_state_preserves_payee_evaluation by blast
  moreover have "lookup (fromAcc, token) (FState.accounts s) = lookup (fromAcc, token) (FState.
      accounts state2)" using calculation by auto
  moreover have "(StatementCont (FStatement.Pay accId payee token val) (split_cases role
      choiceName k cont), split_cases_context role choiceName k ctx, state2, env2, [], payments2
      ) →f2
    (split_cases role choiceName k cont, split_cases_context role choiceName k ctx, state2⦇FState
        .accounts := finalAccs⦈, env2, [] @ [ReducePartialPay fromAcc toPayee token moneyToPay
        res], payments2)" using calculation by auto
  moreover have "split_cases_relation role choiceName k
    (cont, ctx, s⦇FState.accounts := finalAccs⦈, env, ([] @ convertReduceWarnings [
        ReducePartialPay fromAcc toPayee token moneyToPay res]), payments)
    (split_cases role choiceName k cont, split_cases_context role choiceName k ctx, state2⦇FState
        .accounts := finalAccs⦈, env2, ([] @ convertReduceWarnings [ReducePartialPay fromAcc
        toPayee token moneyToPay res]), payments2)" using calculation by auto
  ultimately show ?case by blast
```

```
next
  case (PayPositiveFullWithPayment env s val res fromAcc accId toPayee payee token moneyInAcc
       newBalance newAccs payment finalAccs somePayment cont ctx payments)
  moreover have "evalFValue env s val = evalFValue env state2 val" using calculation(1)
       calculation(11) apply auto
    by (metis (no_types, lifting) calculation(11) fresh_choice_state_preserves_value_evaluation
         (1) split_cases_relation.simps)
  moreover have "evalFParty s accId = evalFParty state2 accId" using calculation(3) calculation
       (11) apply auto
    using fresh_choice_state_preserves_party_evaluation by blast
  moreover have "evalFPayee s payee = evalFPayee state2 payee" using calculation(4) calculation
       (11) apply auto
    using fresh_choice_state_preserves_payee_evaluation by blast
  moreover have "lookup (fromAcc, token) (FState.accounts s) = lookup (fromAcc, token) (FState.
       accounts state2)" using calculation by auto
  moreover have "(StatementCont (FStatement.Pay accId payee token val) (split_cases role
       choiceName k cont), split_cases_context role choiceName k ctx, state2, env2, [], payments2
       ) →f2
    (split_cases role choiceName k cont, split_cases_context role choiceName k ctx, state2(FState
         .accounts := finalAccs), env2, [] @ [ReduceNoWarning], payments2 @ [somePayment])" using
          calculation small_step_reduce.PayPositiveFullWithPayment[of env2 state2 val res fromAcc
          accId toPayee payee token moneyInAcc newBalance newAccs payment finalAccs somePayment]
         by (metis (no_types, lifting) split_cases_relation.simps state_minimal_eq.simps)
  moreover have "split_cases_relation role choiceName k
    (cont, ctx, s(FState.accounts := finalAccs), env, ([] @ convertReduceWarnings [
         ReduceNoWarning]), payments @ [somePayment])
    (split_cases role choiceName k cont, split_cases_context role choiceName k ctx, state2(FState
         .accounts := finalAccs), env2, ([] @ convertReduceWarnings [ReduceNoWarning]), payments2
          @ [somePayment])" using calculation by auto
  ultimately show ?case by (metis isFreshChoiceInProcess.simps split_cases.simps(3)
       split_cases_relation.simps)
next
  case (PayPositiveFullWithoutPayment env s val res fromAcc accId toPayee payee token moneyInAcc
       newBalance newAccs payment finalAccs cont ctx payments)
  moreover have "evalFValue env s val = evalFValue env state2 val" using calculation(1)
       calculation(11) apply auto
    by (metis (no_types, lifting) calculation(11) fresh_choice_state_preserves_value_evaluation
         (1) split_cases_relation.simps)
  moreover have "evalFParty s accId = evalFParty state2 accId" using calculation(3) calculation
       (11) apply auto
    using fresh_choice_state_preserves_party_evaluation by blast
  moreover have "evalFPayee s payee = evalFPayee state2 payee" using calculation(4) calculation
       (11) apply auto
    using fresh_choice_state_preserves_payee_evaluation by blast
```

```
  moreover have "lookup (fromAcc, token) (FState.accounts s) = lookup (fromAcc, token) (FState.
      accounts state2)" using calculation by auto
  moreover have "(StatementCont (FStatement.Pay accId payee token val) (split_cases role
      choiceName k cont), split_cases_context role choiceName k ctx, state2, env2, [], payments2
      ) →f2
    (split_cases role choiceName k cont, split_cases_context role choiceName k ctx, state2⦇FState
        .accounts := finalAccs⦈, env2, [] @ [ReduceNoWarning], payments2)" using calculation by
        auto
  moreover have "split_cases_relation role choiceName k
    (cont, ctx, s⦇FState.accounts := finalAccs⦈, env, ([] @ convertReduceWarnings [
        ReduceNoWarning]), payments)
    (split_cases role choiceName k cont, split_cases_context role choiceName k ctx, state2⦇FState
        .accounts := finalAccs⦈, env2, ([] @ convertReduceWarnings [ReduceNoWarning]), payments2
        )" using calculation by auto
  ultimately show ?case by blast
next
  case (IfTrue env s obs cont1 cont2 ctx payments)
  then show ?case apply auto
    by (smt (verit) FaustusV2Semantics.small_step_reduce.IfTrue IfTrue.prems append_self_conv2
        convertReduceWarnings.simps(1) convertReduceWarnings.simps(2)
        fresh_choice_state_preserves_value_evaluation(2) split_cases_relation.simps)
next
  case (IfFalse env s obs cont1 cont2 ctx warns payments)
  then show ?case apply auto
    by (smt (verit) FaustusV2Semantics.small_step_reduce.IfFalse IfFalse.prems append_self_conv2
        convertReduceWarnings.simps(1) convertReduceWarnings.simps(2)
        fresh_choice_state_preserves_value_evaluation(2) split_cases_relation.simps)
next
  case (WhenTimeout env startSlot endSlot timeout cases cont ctx s warns payments)
  then show ?case apply auto
    by (smt (verit) FaustusV2Semantics.small_step_reduce.WhenTimeout append_self_conv2
        convertReduceWarnings.simps(1) convertReduceWarnings.simps(2))
next
  case (LetShadow valId s oldVal env val res cont ctx payments)
  from LetShadow have valEQ: "evalFValue env s val = evalFValue env state2 val" apply auto
    by (metis (no_types, lifting) LetShadow.prems(1)
        fresh_choice_state_preserves_value_evaluation(1) split_cases_relation.simps)
  from LetShadow have valExists: "lookup valId (boundValues state2) = Some oldVal" by auto
  from LetShadow valEQ valExists have step: "(Let valId val (split_cases role choiceName k cont),
       split_cases_context role choiceName k ctx, state2, env, [], payments2) →f2
      (split_cases role choiceName k cont, FaustusV2AST.AbstractionInformation.ValueAbstraction
          valId # split_cases_context role choiceName k ctx, state2⦇FState.boundValues := MList
          .insert valId res (FState.boundValues state2)⦈, env, [] @ [ReduceShadowing valId
          oldVal res], payments2)" apply auto
```

```
    using small_step_reduce.LetShadow by (smt (verit, ccfv_threshold) FState.surjective FState.
          update_convs(3) self_append_conv2)
  from step LetShadow have rel: "split_cases_relation role choiceName k (cont, FaustusV2AST.
       AbstractionInformation.ValueAbstraction valId # ctx, s⦇FState.boundValues := MList.insert
       valId res (FState.boundValues s)⦈, env, ([] @ convertReduceWarnings [ReduceShadowing valId
        oldVal res]), payments)
    (split_cases role choiceName k cont, FaustusV2AST.AbstractionInformation.ValueAbstraction
          valId # split_cases_context role choiceName k ctx, state2⦇FState.boundValues := MList.
          insert valId res (FState.boundValues state2)⦈, env, ([] @ convertReduceWarnings [
          ReduceShadowing valId oldVal res]), payments2)" by auto
  from LetShadow valEQ rel step show ?case by (metis isFreshChoiceInProcess.simps split_cases.
       simps(5) split_cases_relation.simps)
next
  case (ReassignValShadow valId s oldVal env val res cont ctx payments)
  from ReassignValShadow have valEQ: "evalFValue env s val = evalFValue env state2 val" apply
       auto
    by (metis (no_types, lifting) ReassignValShadow.prems(1)
          fresh_choice_state_preserves_value_evaluation(1) split_cases_relation.simps)
  from ReassignValShadow have valExists: "lookup valId (boundValues state2) = Some oldVal" by
       auto
  from ReassignValShadow valEQ valExists have step: "(StatementCont (ReassignVal valId val) (
       split_cases role choiceName k cont), split_cases_context role choiceName k ctx, state2,
       env, [], payments2) →f2
       (split_cases role choiceName k cont, FaustusV2AST.AbstractionInformation.ValueAbstraction
            valId # split_cases_context role choiceName k ctx, state2⦇FState.boundValues := MList
            .insert valId res (FState.boundValues state2)⦈, env, [] @ [ReduceShadowing valId
            oldVal res], payments2)" apply auto
    using small_step_reduce.ReassignValShadow by (smt (verit, ccfv_threshold) FState.surjective
          FState.update_convs(3) self_append_conv2)
  from step ReassignValShadow have rel: "split_cases_relation role choiceName k (cont,
       FaustusV2AST.AbstractionInformation.ValueAbstraction valId # ctx, s⦇FState.boundValues :=
       MList.insert valId res (FState.boundValues s)⦈, env, ([] @ convertReduceWarnings [
       ReduceShadowing valId oldVal res]), payments)
    (split_cases role choiceName k cont, FaustusV2AST.AbstractionInformation.ValueAbstraction
          valId # split_cases_context role choiceName k ctx, state2⦇FState.boundValues := MList.
          insert valId res (FState.boundValues state2)⦈, env, ([] @ convertReduceWarnings [
          ReduceShadowing valId oldVal res]), payments2)" by auto
  from ReassignValShadow valEQ rel step show ?case by (metis isFreshChoiceInProcess.simps
       split_cases.simps(3) split_cases_relation.simps)
next
  case (LetNoShadow valId s env val res cont ctx payments)
  from LetNoShadow have valEQ: "evalFValue env s val = evalFValue env state2 val" apply auto
    by (metis (no_types, lifting) LetNoShadow.prems(1)
          fresh_choice_state_preserves_value_evaluation(1) split_cases_relation.simps)
```

```
  from LetNoShadow have valExists: "lookup valId (boundValues state2) = None" by auto
  from LetNoShadow valEQ valExists have step: "(Let valId val (split_cases role choiceName k cont
       ), split_cases_context role choiceName k ctx, state2, env, [], payments2) →f2
       (split_cases role choiceName k cont, FaustusV2AST.AbstractionInformation.ValueAbstraction
            valId # split_cases_context role choiceName k ctx, state2⦇FState.boundValues := MList
            .insert valId res (FState.boundValues state2)⦈, env, [] @ [ReduceNoWarning],
            payments2)" apply auto
    using small_step_reduce.LetNoShadow by (metis (no_types, lifting) FState.surjective FState.
          update_convs(3) self_append_conv2)
  from step LetNoShadow have rel: "split_cases_relation role choiceName k (cont, FaustusV2AST.
       AbstractionInformation.ValueAbstraction valId # ctx, s⦇FState.boundValues := MList.insert
       valId res (FState.boundValues s)⦈, env, ([] @ convertReduceWarnings [ReduceNoWarning]),
       payments)
    (split_cases role choiceName k cont, FaustusV2AST.AbstractionInformation.ValueAbstraction
          valId # split_cases_context role choiceName k ctx, state2⦇FState.boundValues := MList.
          insert valId res (FState.boundValues state2)⦈, env, ([] @ convertReduceWarnings [
          ReduceNoWarning]), payments2)" by auto
  from LetNoShadow valEQ rel step show ?case by (metis isFreshChoiceInProcess.simps split_cases.
       simps(5) split_cases_relation.simps)
next
  case (ReassignValNoShadow valId s env val res cont ctx payments)
  from ReassignValNoShadow have valEQ: "evalFValue env s val = evalFValue env state2 val" apply
       auto
    by (metis (no_types, lifting) ReassignValNoShadow.prems(1)
          fresh_choice_state_preserves_value_evaluation(1) split_cases_relation.simps)
  from ReassignValNoShadow have valExists: "lookup valId (boundValues state2) = None" by auto
  from ReassignValNoShadow valEQ valExists have step: "(StatementCont (ReassignVal valId val) (
       split_cases role choiceName k cont), split_cases_context role choiceName k ctx, state2,
       env, [], payments2) →f2
       (split_cases role choiceName k cont, FaustusV2AST.AbstractionInformation.ValueAbstraction
            valId # split_cases_context role choiceName k ctx, state2⦇FState.boundValues := MList
            .insert valId res (FState.boundValues state2)⦈, env, [] @ [ReduceNoWarning],
            payments2)" apply auto
    using small_step_reduce.ReassignValNoShadow by (smt (verit, del_insts) FState.surjective
          FState.update_convs(3) self_append_conv2)
  from step ReassignValNoShadow have rel: "split_cases_relation role choiceName k (cont,
       FaustusV2AST.AbstractionInformation.ValueAbstraction valId # ctx, s⦇FState.boundValues :=
       MList.insert valId res (FState.boundValues s)⦈, env, ([] @ convertReduceWarnings [
       ReduceNoWarning]), payments)
    (split_cases role choiceName k cont, FaustusV2AST.AbstractionInformation.ValueAbstraction
          valId # split_cases_context role choiceName k ctx, state2⦇FState.boundValues := MList.
          insert valId res (FState.boundValues state2)⦈, env, ([] @ convertReduceWarnings [
          ReduceNoWarning]), payments2)" by auto
  from ReassignValNoShadow valEQ rel step show ?case by (metis isFreshChoiceInProcess.simps
```

```
      split_cases.simps(3) split_cases_relation.simps)
next
  case (LetObservation e s obs res oId c cx ps)
  from LetObservation have valEQ: "evalFObservation e s obs = evalFObservation e state2 obs"
      apply auto
    by (metis (no_types, opaque_lifting) LetObservation.prems(1)
        fresh_choice_state_preserves_value_evaluation(2) split_cases_relation.simps)
  from LetObservation valEQ have step: "(LetObservation oId obs (split_cases role choiceName k c)
      , split_cases_context role choiceName k cx, state2, e, [], payments2) →f2
      (split_cases role choiceName k c, ObservationAbstraction oId # split_cases_context role
          choiceName k cx, state2⦇FState.boundValues := MList.insert oId (if res then 1 else 0)
           (FState.boundValues state2)⦈, e, [] @ [ReduceNoWarning], payments2)" using
          small_step_reduce.LetObservation by presburger
  from step LetObservation have rel: "split_cases_relation role choiceName k (c,
      ObservationAbstraction oId # cx, s⦇FState.boundValues := MList.insert oId (if res then 1
      else 0) (FState.boundValues s)⦈, e, ([] @ convertReduceWarnings [ReduceNoWarning]), ps)
    (split_cases role choiceName k c, ObservationAbstraction oId # split_cases_context role
        choiceName k cx, state2⦇FState.boundValues := MList.insert oId (if res then 1 else 0) (
        FState.boundValues state2)⦈, e, ([] @ convertReduceWarnings [ReduceNoWarning]), ps)" by
        auto
  from LetObservation valEQ rel step show ?case by (metis isFreshChoiceInProcess.simps
      split_cases.simps(6) split_cases_relation.simps)
next
  case (ReassignObservation e s obs res oId c cx ps)
  from ReassignObservation have valEQ: "evalFObservation e s obs = evalFObservation e state2 obs"
       apply auto
    by (metis (no_types, lifting) ReassignObservation.prems(1)
        fresh_choice_state_preserves_value_evaluation(2) split_cases_relation.simps)
  from ReassignObservation valEQ have step: "(StatementCont (ReassignObservation oId obs) (
      split_cases role choiceName k c), split_cases_context role choiceName k cx, state2, e, [],
       payments2) →f2
      (split_cases role choiceName k c, ObservationAbstraction oId # split_cases_context role
          choiceName k cx, state2⦇FState.boundValues := MList.insert oId (if res then 1 else 0)
           (FState.boundValues state2)⦈, e, [] @ [ReduceNoWarning], payments2)" using
          small_step_reduce.ReassignObservation by presburger
  from step ReassignObservation have rel: "split_cases_relation role choiceName k (c,
      ObservationAbstraction oId # cx, s⦇FState.boundValues := MList.insert oId (if res then 1
      else 0) (FState.boundValues s)⦈, e, ([] @ convertReduceWarnings [ReduceNoWarning]), ps)
    (split_cases role choiceName k c, ObservationAbstraction oId # split_cases_context role
        choiceName k cx, state2⦇FState.boundValues := MList.insert oId (if res then 1 else 0) (
        FState.boundValues state2)⦈, e, ([] @ convertReduceWarnings [ReduceNoWarning]), ps)" by
        auto
  from ReassignObservation valEQ rel step show ?case by (metis isFreshChoiceInProcess.simps
      split_cases.simps(3) split_cases_relation.simps)
```

```
next
  case (LetParty s pk res pId c cx e ps)
  from LetParty have valEQ: "evalFParty s pk = evalFParty state2 pk" apply auto
    using fresh_choice_state_preserves_party_evaluation by presburger
  from LetParty valEQ have step: "(LetParty pId pk (split_cases role choiceName k c),
       split_cases_context role choiceName k cx, state2, e, [], payments2) →f2
       (split_cases role choiceName k c, PartyAbstraction pId # split_cases_context role
            choiceName k cx, state2⦇FState.boundParties := MList.insert pId res (boundParties
            state2)⦈, e, [] @ [ReduceNoWarning], payments2)" using small_step_reduce.LetParty by
            presburger
  from step LetParty have rel: "split_cases_relation role choiceName k (c, PartyAbstraction pId #
        cx, s⦇FState.boundParties := MList.insert pId res (FState.boundParties s)⦈, e, ([] @
       convertReduceWarnings [ReduceNoWarning]), ps)
    (split_cases role choiceName k c, PartyAbstraction pId # split_cases_context role choiceName
         k cx, state2⦇FState.boundParties := MList.insert pId res (boundParties state2)⦈, e, ([]
         @ convertReduceWarnings [ReduceNoWarning]), payments2)" by auto
  from LetParty valEQ rel step show ?case by (metis isFreshChoiceInProcess.simps split_cases.
       simps(7) split_cases_relation.simps)
next
  case (ReassignParty s pk res pId c cx e ps)
  from ReassignParty have valEQ: "evalFParty s pk = evalFParty state2 pk" apply auto
    using fresh_choice_state_preserves_party_evaluation by presburger
  from ReassignParty valEQ have step: "(StatementCont (ReassignParty pId pk) (split_cases role
       choiceName k c), split_cases_context role choiceName k cx, state2, e, [], payments2) →f2
       (split_cases role choiceName k c, PartyAbstraction pId # split_cases_context role
            choiceName k cx, state2⦇FState.boundParties := MList.insert pId res (boundParties
            state2)⦈, e, [] @ [ReduceNoWarning], payments2)" using small_step_reduce.
            ReassignParty by presburger
  from step ReassignParty have rel: "split_cases_relation role choiceName k (c, FaustusV2AST.
       AbstractionInformation.PartyAbstraction pId # cx, s⦇boundParties := MList.insert pId res (
       boundParties s)⦈, e, ([] @ convertReduceWarnings [ReduceNoWarning]), ps)
    (split_cases role choiceName k c, PartyAbstraction pId # split_cases_context role choiceName
         k cx, state2⦇FState.boundParties := MList.insert pId res (boundParties state2)⦈, e, ([]
         @ convertReduceWarnings [ReduceNoWarning]), payments2)" by auto
  from ReassignParty valEQ rel step show ?case by (metis isFreshChoiceInProcess.simps split_cases
       .simps(3) split_cases_relation.simps)
next
  case (AssertTrue env s obs cont ctx warns payments)
  then show ?case apply auto
    by (smt (verit, best) AssertTrue.prems FaustusV2Semantics.small_step_reduce.AssertTrue append
         .left_neutral convertReduceWarnings.simps(1) convertReduceWarnings.simps(2)
         fresh_choice_state_preserves_value_evaluation(2) split_cases_relation.simps)
next
  case (AssertFalse env s obs cont ctx warns payments)
```

```
  then show ?case apply auto
    by (smt (verit, best) AssertFalse.prems FaustusV2Semantics.small_step_reduce.AssertFalse
          append.left_neutral convertReduceWarnings.simps(1) convertReduceWarnings.simps(6)
          fresh_choice_state_preserves_value_evaluation(2) split_cases_relation.simps)
next
  case (UseCFound ctx cid params bodyCont innerCtx env s args newState payments)
  moreover from UseCFound(1) have ctx_split: "lookupContractIdAbsInformation (split_cases_context
         role choiceName k ctx) cid = Some (params, split_cases role choiceName k bodyCont,
        split_cases_context role choiceName k innerCtx)" by (induction ctx cid rule:
        lookupContractIdAbsInformation.induct, auto)
  moreover obtain newState2 where "(evalFArguments env state2 params args) = Some newState2"  by
        (metis (no_types, lifting) calculation(2,3)
        fresh_choice_state_preserves_argument_evaluation isFreshChoiceInContract.simps(9)
        isFreshChoiceInProcess.simps split_cases_relation.simps)
  moreover from UseCFound calculation have accounts_same: "accounts newState = accounts (the (
        evalFArguments env state2 params args)) ∧
    boundParties newState = boundParties newState2 ∧
    boundValues newState = boundValues newState2 ∧
    minTime newState = minTime newState2 ∧
    ((∀choiceNameMade choicePartyMade choiceMade .
      choiceNameMade ≠ choiceName ⟶
      MList.lookup (ChoiceId choiceNameMade choicePartyMade) (choices newState) = choiceMade ⟶
      MList.lookup (ChoiceId choiceNameMade choicePartyMade) (choices newState2) = choiceMade))"
           using fresh_choice_state_preserves_argument_evaluation[of env s params args newState
           choiceName state2] by auto
  moreover have "(FaustusV2AST.FContract.UseC cid args, split_cases_context role choiceName k ctx
        , state2, env, [], payments2) →f2
        (split_cases role choiceName k bodyCont, (paramsToFContext params @ split_cases_context
             role choiceName k innerCtx), newState2, env, [] @ [ReduceNoWarning], payments2)"
             using calculation apply auto
    by (simp add: FaustusV2Semantics.small_step_reduce.simps)
  moreover have "paramsToFContext params @ split_cases_context role choiceName k innerCtx =
        split_cases_context role choiceName k (paramsToFContext params @ innerCtx)" by (induction
        params rule: paramsToFContext.induct, auto)
  moreover have "split_cases_relation role choiceName k
         (bodyCont, FaustusV2AST.paramsToFContext params @ innerCtx, newState, env, ([] @
              convertReduceWarnings [ReduceNoWarning]), payments)
         (split_cases role choiceName k bodyCont, (paramsToFContext params @ split_cases_context
              role choiceName k innerCtx), newState2, env, ([] @ convertReduceWarnings [
              ReduceNoWarning]), payments2)" using calculation apply auto
    by (meson FaustusV2Semantics.small_step_reduce.UseCFound
          freshChoicePreserved_small_step_reduce isFreshChoiceInContract.simps(9)
          isFreshChoiceInState.elims(3)freshChoicePreserved_evalArguments isFreshChoiceInState.
          elims(2) isFreshChoiceInState.elims(3))+
```

```
  ultimately show ?case apply auto
    by (metis \<open>(FaustusV2AST.FContract.UseC cid args, split_cases_context role choiceName k
          ctx, state2, env, [], payments2) →f2 (split_cases role choiceName k bodyCont,
         FaustusV2AST.paramsToFContext params @ split_cases_context role choiceName k innerCtx,
         newState2, env, [] @ [ReduceNoWarning], payments2)\<close> append.left_neutral
         convertReduceWarnings.simps(1) convertReduceWarnings.simps(2))
next
  case (LetC cid params boundCon cont ctx s env payments)
  then show ?case apply auto
    by (metis (no_types, lifting) FaustusV2Semantics.small_step_reduce.LetC append.assoc append.
         right_neutral convertReduceWarnings.simps(1) convertReduceWarnings.simps(2)
         same_append_eq)
qed

lemma small_step_preserves_inverse_split_cases_relation:
"(c2, ctx2, state2, env2, [], payments2) →f2 (c2', ctx2', state2', env2', reduceWarnings2',
     payments2') ⟹
  split_cases_relation role choiceName k (c1, ctx1, state1, env1, warnings1, payments1) (c2, ctx2
       , state2, env2, warnings2, payments2) ⟹
  (∃c1' ctx1' state1' env1' reduceWarnings1' payments1' . (c1, ctx1, state1, env1, [], payments1)
        →f2 (c1', ctx1', state1', env1', reduceWarnings1', payments1') ∧
    split_cases_relation role choiceName k (c1', ctx1', state1', env1', warnings1 @
         convertReduceWarnings reduceWarnings1', payments1') (c2', ctx2', state2', env2',
         warnings2 @ convertReduceWarnings reduceWarnings2', payments2'))"
proof (induction c2 ctx2 state2 env2 "[] :: ReduceWarning list" payments2 c2' ctx2' state2' env2'
      reduceWarnings2' payments2'
    arbitrary: c1 ctx1 state1 env1 payments1 warnings1 warnings2
    rule: small_step_reduce_induct)
  case (CloseRefund s party token money newAccount ctx env payments)
  moreover have "refundOne (FState.accounts state1) = Some ((party, token, money), newAccount)"
       using calculation by auto
  moreover have "c1 = Close" using calculation apply (induction c1, auto)
    by (smt (verit, best) FaustusV2AST.FContract.distinct(5))
  moreover have "(Close, ctx1, state1, env1, [], payments1) →f2 (Close, ctx1, state1⦇accounts :=
        newAccount⦈, env1, [] @ [ReduceNoWarning], payments1 @ [Payment party (Payee.Party party)
        token money])" using calculation apply auto
    by (metis CloseRefund.hyps FaustusV2Semantics.small_step_reduce.CloseRefund append.
         left_neutral)
  moreover have "isFreshChoiceInContract choiceName Close" by auto
  moreover have "state_minimal_eq choiceName (state1⦇accounts := newAccount⦈) (s⦇FState.accounts
       := newAccount⦈)" using calculation by auto
  ultimately show ?case by (metis isFreshChoiceInProcess.simps split_cases_relation.simps)
next
  case (PayNonPositive env s val res fromAcc accId toPayee payee token cont ctx payments)
```

```
  moreover obtain innerC1 where "c1 = (StatementCont (Pay accId payee token val) innerC1) ∧
        split_cases role choiceName k innerC1 = cont" using calculation apply (cases c1, auto)
    by (smt (verit, best) FaustusV2AST.FContract.distinct(19))
  moreover have "isFreshChoiceInValue choiceName val" using calculation by auto
  moreover have "evalFValue env state1 val = Some res" using calculation by (metis (no_types,
        opaque_lifting) calculation(5) fresh_choice_state_preserves_value_evaluation(1)
        split_cases_relation.simps)
  moreover have forwardStep: "(StatementCont (Pay accId payee token val) innerC1, ctx1, state1,
        env1, [], payments1) →f2 (innerC1, ctx1, state1, env1, ([] @ [ReduceNonPositivePay
        fromAcc toPayee token res]), payments)" using calculation apply auto
    by (smt (verit, best) FaustusV2Semantics.small_step_reduce.PayNonPositive
          fresh_choice_state_preserves_payee_evaluation
          fresh_choice_state_preserves_party_evaluation self_append_conv2)
  ultimately show ?case using forwardStep apply auto
    by (metis append_Nil convertReduceWarnings.simps(1) convertReduceWarnings.simps(3)
          forwardStep)
next
  case (PayPositivePartialWithPayment env s val res fromAcc accId toPayee payee token newAccs
        moneyToPay payment finalAccs somePayment cont ctx payments)
  moreover obtain innerC1 where "c1 = (StatementCont (Pay accId payee token val) innerC1) ∧
        split_cases role choiceName k innerC1 = cont" using calculation apply (cases c1, auto)
    by (smt (verit, best) FaustusV2AST.FContract.distinct(19))
  moreover have "isFreshChoiceInValue choiceName val" using calculation by auto
  moreover have "evalFValue env state1 val = Some res" using calculation by (metis (no_types,
        opaque_lifting) fresh_choice_state_preserves_value_evaluation(1) split_cases_relation.
        simps)
  moreover have "Some fromAcc = evalFParty state1 accId ∧
    Some toPayee = evalFPayee state1 payee ∧
    moneyInAccount fromAcc token (FState.accounts state1) < res ∧
    updateMoneyInAccount fromAcc token 0 (FState.accounts state1) = newAccs ∧
    moneyInAccount fromAcc token (FState.accounts state1) = moneyToPay" using calculation apply (
          auto)
     apply (metis fresh_choice_state_preserves_party_evaluation)
    by (metis fresh_choice_state_preserves_payee_evaluation)
  moreover have forwardStep: "(StatementCont (Pay accId payee token val) innerC1, ctx1, state1,
        env1, [], payments1) →f2 (innerC1, ctx1, state1(FState.accounts := finalAccs), env1, ([]
        @ [ReducePartialPay fromAcc toPayee token moneyToPay res]), payments @ [somePayment])"
        using calculation
      small_step_reduce.PayPositivePartialWithPayment[of env1 state1 val res fromAcc accId
          toPayee payee token newAccs moneyToPay payment finalAccs somePayment innerC1 ctx1 "[]"
           payments1] by auto
  moreover have "state_minimal_eq choiceName (state1(FState.accounts := finalAccs)) (s(FState.
        accounts := finalAccs))" using calculation by auto
  ultimately show ?case by (metis freshChoicePreserved_small_step_reduce isFreshChoiceInProcess.
```

```
      simps split_cases_relation.simps)
next
  case (PayPositivePartialWithoutPayment env s val res fromAcc accId toPayee payee token newAccs
      moneyToPay payment finalAccs cont ctx payments)
  moreover obtain innerC1 where "c1 = (StatementCont (Pay accId payee token val) innerC1) ∧
      split_cases role choiceName k innerC1 = cont" using calculation by (cases c1, auto)
  moreover have "isFreshChoiceInValue choiceName val" using calculation by auto
  moreover have "evalFValue env state1 val = Some res" using calculation by (metis (no_types,
      opaque_lifting) fresh_choice_state_preserves_value_evaluation(1) split_cases_relation.
      simps)
  moreover have "Some fromAcc = evalFParty state1 accId ∧
    Some toPayee = evalFPayee state1 payee ∧
    moneyInAccount fromAcc token (FState.accounts state1) < res ∧
    updateMoneyInAccount fromAcc token 0 (FState.accounts state1) = newAccs ∧
    moneyInAccount fromAcc token (FState.accounts state1) = moneyToPay" using calculation by auto
  moreover have forwardStep: "(StatementCont (Pay accId payee token val) innerC1, ctx1, state1,
      env1, [], payments1) →f2 (innerC1, ctx1, state1⦇FState.accounts := finalAccs⦈, env1, ([]
      @ [ReducePartialPay fromAcc toPayee token moneyToPay res]), payments)" using calculation
      by auto
  moreover have "state_minimal_eq choiceName (state1⦇FState.accounts := finalAccs⦈) (s⦇FState.
      accounts := finalAccs⦈)" using calculation by auto
  ultimately show ?case by (metis freshChoicePreserved_small_step_reduce isFreshChoiceInProcess.
      simps split_cases_relation.simps)
next
  case (PayPositiveFullWithPayment env s val res fromAcc accId toPayee payee token moneyInAcc
      newBalance newAccs payment finalAccs somePayment cont ctx payments)
  moreover obtain innerC1 where "c1 = (StatementCont (Pay accId payee token val) innerC1) ∧
      split_cases role choiceName k innerC1 = cont" using calculation apply (cases c1, auto)
    by (smt (verit, best) FaustusV2AST.FContract.distinct(19))
  moreover have "isFreshChoiceInValue choiceName val" using calculation by auto
  moreover have "evalFValue env state1 val = Some res" using calculation by (metis (no_types,
      opaque_lifting) fresh_choice_state_preserves_value_evaluation(1) split_cases_relation.
      simps)
  moreover have "Some fromAcc = evalFParty state1 accId ∧
    Some toPayee = evalFPayee state1 payee ∧
    moneyInAccount fromAcc token (FState.accounts state1) ≥ res ∧
    updateMoneyInAccount fromAcc token newBalance (FState.accounts state1) = newAccs ∧
    moneyInAccount fromAcc token (FState.accounts state1) = moneyInAcc" using calculation apply (
        auto)
      using fresh_choice_state_preserves_party_evaluation
          fresh_choice_state_preserves_payee_evaluation by metis+
    moreover have forwardStep: "(StatementCont (Pay accId payee token val) innerC1, ctx1, state1
        , env1, [], payments1) →f2 (innerC1, ctx1, state1⦇FState.accounts := finalAccs⦈, env1,
         ([] @ [ReduceNoWarning]), payments @ [somePayment])" using calculation
```

```
      small_step_reduce.PayPositiveFullWithPayment[of env1 state1 val res fromAcc accId toPayee
          payee token moneyInAcc newBalance newAccs payment finalAccs somePayment innerC1 ctx1
          "[]" payments1] by auto
  moreover have "state_minimal_eq choiceName (state1⦇FState.accounts := finalAccs⦈) (s⦇FState.
      accounts := finalAccs⦈)" using calculation by auto
  ultimately show ?case by (metis freshChoicePreserved_small_step_reduce isFreshChoiceInProcess.
      simps split_cases_relation.simps)
next
  case (PayPositiveFullWithoutPayment env s val res fromAcc accId toPayee payee token moneyInAcc
      newBalance newAccs payment finalAccs cont ctx payments)
  moreover obtain innerC1 where "c1 = (StatementCont (Pay accId payee token val) innerC1) ∧
      split_cases role choiceName k innerC1 = cont" using calculation by (cases c1, auto)
  moreover have "isFreshChoiceInValue choiceName val" using calculation by auto
  moreover have "evalFValue env state1 val = Some res" using calculation by (metis (no_types,
      opaque_lifting) fresh_choice_state_preserves_value_evaluation(1) split_cases_relation.
      simps)
  moreover have "Some fromAcc = evalFParty state1 accId ∧
    Some toPayee = evalFPayee state1 payee ∧
    moneyInAccount fromAcc token (FState.accounts state1) ≥ res ∧
    updateMoneyInAccount fromAcc token newBalance (FState.accounts state1) = newAccs ∧
    moneyInAccount fromAcc token (FState.accounts state1) = moneyInAcc" using calculation by auto
  moreover have forwardStep: "(StatementCont (Pay accId payee token val) innerC1, ctx1, state1,
      env1, [], payments1) →f2 (innerC1, ctx1, state1⦇FState.accounts := finalAccs⦈, env1, ([]
      @ [ReduceNoWarning]), payments)" using calculation by auto
  moreover have "state_minimal_eq choiceName (state1⦇FState.accounts := finalAccs⦈) (s⦇FState.
      accounts := finalAccs⦈)" using calculation by auto
  ultimately show ?case by (metis freshChoicePreserved_small_step_reduce isFreshChoiceInProcess.
      simps split_cases_relation.simps)
next
  case (IfTrue env s obs cont1 cont2 ctx payments)
  moreover obtain innerTrueCont innerFalseCont where "c1 = (If obs innerTrueCont innerFalseCont)
      ∧ split_cases role choiceName k innerTrueCont = cont1 ∧ split_cases role choiceName k
      innerFalseCont = cont2" using calculation(2) apply (cases c1, auto)
    by (smt (verit) FaustusV2AST.FContract.distinct(31))
  moreover have "isFreshChoiceInObservation choiceName obs" using calculation by auto
  moreover have "evalFObservation env state1 obs = Some True" using calculation by (metis (
      no_types, opaque_lifting) fresh_choice_state_preserves_value_evaluation(2)
      split_cases_relation.simps)
  moreover have forwardStep: "(If obs innerTrueCont innerFalseCont, ctx1, state1, env1, [],
      payments1) →f2 (innerTrueCont, ctx1, state1, env1, ([] @ [ReduceNoWarning]), payments)"
      using calculation apply auto
    by (metis FaustusV2Semantics.small_step_reduce.IfTrue self_append_conv2)
  ultimately show ?case by (metis freshChoicePreserved_small_step_reduce isFreshChoiceInProcess.
      simps split_cases_relation.simps)
```

```
next
  case (IfFalse env s obs cont1 cont2 ctx payments)
  moreover obtain innerTrueCont innerFalseCont where "c1 = (If obs innerTrueCont innerFalseCont)
      ∧ split_cases role choiceName k innerTrueCont = cont1 ∧ split_cases role choiceName k
      innerFalseCont = cont2" using calculation(2) apply (cases c1, auto)
    by (smt (verit) FaustusV2AST.FContract.distinct(31))
  moreover have "isFreshChoiceInObservation choiceName obs" using calculation by auto
  moreover have "evalFObservation env state1 obs = Some False" using calculation by (metis (
      no_types, opaque_lifting) fresh_choice_state_preserves_value_evaluation(2)
      split_cases_relation.simps)
  moreover have forwardStep: "(If obs innerTrueCont innerFalseCont, ctx1, state1, env1, [],
      payments1) →f2 (innerFalseCont, ctx1, state1, env1, ([] @ [ReduceNoWarning]), payments)"
      using calculation apply auto
    by (metis FaustusV2Semantics.small_step_reduce.IfFalse self_append_conv2)
  ultimately show ?case by (metis freshChoicePreserved_small_step_reduce isFreshChoiceInProcess.
      simps split_cases_relation.simps)
next
  case (WhenTimeout env startSlot endSlot timeout cases cont ctx s payments)
  moreover obtain innerCases innerTCont where "c1 = (When innerCases timeout innerTCont) ∧
      split_cases role choiceName k innerTCont = cont" using calculation apply (cases c1, auto)
    by (smt (verit, best) FaustusV2AST.FContract.inject(3))
  moreover have "timeInterval env1 = (startSlot, endSlot)" using calculation by auto
  moreover have forwardStep: "(When innerCases timeout innerTCont, ctx1, state1, env1, [],
      payments1) →f2 (innerTCont, ctx1, state1, env1, ([] @ [ReduceNoWarning]), payments)"
      using calculation apply auto
    by (metis FaustusV2Semantics.small_step_reduce.WhenTimeout self_append_conv2)
  ultimately show ?case by (metis freshChoicePreserved_small_step_reduce isFreshChoiceInProcess.
      simps split_cases_relation.simps)
next
  case (LetShadow valId s oldVal env val res cont ctx payments)
  moreover obtain innerCont where "c1 = (Let valId val innerCont) ∧ split_cases role choiceName k
       innerCont = cont" using calculation apply (cases c1, auto)
    by (smt (verit) FaustusV2AST.FContract.distinct)
  moreover have "isFreshChoiceInValue choiceName val" using calculation by auto
  moreover have "evalFValue env state1 val = Some res" using calculation by (metis (no_types,
      opaque_lifting) fresh_choice_state_preserves_value_evaluation(1) split_cases_relation.
      simps)
  moreover have "split_cases_context role choiceName k (ValueAbstraction valId#ctx1) = (
      ValueAbstraction valId#ctx)" using calculation by auto
  moreover have "state_minimal_eq choiceName (state1⦇boundValues := MList.insert valId res (
      boundValues state1)⦈) (s⦇boundValues := MList.insert valId res (boundValues state1)⦈)"
      using calculation by auto
  moreover have forwardStep: "(Let valId val innerCont, ctx1, state1, env1, [], payments1) →f2 (
      innerCont, ValueAbstraction valId#ctx1, state1⦇boundValues := MList.insert valId res (
```

```
      boundValues state1)⦈, env1, ([] @ [ReduceShadowing valId oldVal res]), payments)" using
      calculation apply auto
    by (simp add: FaustusV2Semantics.small_step_reduce.simps)
  moreover have "split_cases_relation role choiceName k (innerCont, ValueAbstraction valId#ctx1,
      state1⦇boundValues := MList.insert valId res (boundValues state1)⦈, env1, ([] @
      convertReduceWarnings [ReduceShadowing valId oldVal res]), payments) (cont,
      ValueAbstraction valId # ctx, s⦇FState.boundValues := MList.insert valId res (FState.
      boundValues s)⦈, env, ([] @ convertReduceWarnings [ReduceShadowing valId oldVal res]),
      payments)" using calculation by auto
  ultimately show ?case by (metis isFreshChoiceInProcess.simps split_cases_relation.simps)
next
  case (ReassignValShadow valId s oldVal env val res cont ctx payments)
  moreover obtain innerCont where "c1 = (StatementCont (ReassignVal valId val) innerCont) ∧
      split_cases role choiceName k innerCont = cont" using calculation apply (cases c1, auto)
    by (smt (verit) FaustusV2AST.FContract.distinct)
  moreover have "isFreshChoiceInValue choiceName val" using calculation by auto
  moreover have "evalFValue env state1 val = Some res" using calculation by (metis (no_types,
      opaque_lifting) fresh_choice_state_preserves_value_evaluation(1) split_cases_relation.
      simps)
  moreover have "split_cases_context role choiceName k (ValueAbstraction valId#ctx1) = (
      ValueAbstraction valId#ctx)" using calculation by auto
  moreover have "state_minimal_eq choiceName (state1⦇boundValues := MList.insert valId res (
      boundValues state1)⦈) (s⦇boundValues := MList.insert valId res (boundValues state1)⦈)"
      using calculation by auto
  moreover have forwardStep: "(StatementCont (ReassignVal valId val) innerCont, ctx1, state1,
      env1, [], payments1) →f2 (innerCont, ValueAbstraction valId#ctx1, state1⦇boundValues :=
      MList.insert valId res (boundValues state1)⦈, env1, ([] @ [ReduceShadowing valId oldVal
      res]), payments)" using calculation apply auto
    by (simp add: FaustusV2Semantics.small_step_reduce.simps)
  moreover have "split_cases_relation role choiceName k (innerCont, ValueAbstraction valId#ctx1,
      state1⦇boundValues := MList.insert valId res (boundValues state1)⦈, env1,
      convertReduceWarnings ([] @ [ReduceShadowing valId oldVal res]), payments) (cont,
      ValueAbstraction valId # ctx, s⦇FState.boundValues := MList.insert valId res (FState.
      boundValues s)⦈, env, convertReduceWarnings ([] @ [ReduceShadowing valId oldVal res]),
      payments)" using calculation by auto
  ultimately show ?case by (metis isFreshChoiceInProcess.simps split_cases_relation.simps)
next
  case (LetNoShadow valId s env val res cont ctx payments)
  moreover obtain innerCont where "c1 = (Let valId val innerCont) ∧ split_cases role choiceName k
       innerCont = cont" using calculation apply (cases c1, auto)
    by (smt (verit) FaustusV2AST.FContract.distinct)
  moreover have "isFreshChoiceInValue choiceName val" using calculation by auto
  moreover have "evalFValue env state1 val = Some res" using calculation by (metis (no_types,
      opaque_lifting) fresh_choice_state_preserves_value_evaluation(1) split_cases_relation.
```

```
      simps)
  moreover have "split_cases_context role choiceName k (ValueAbstraction valId#ctx1) = (
      ValueAbstraction valId#ctx)" using calculation by auto
  moreover have "state_minimal_eq choiceName (state1⦇boundValues := MList.insert valId res (
      boundValues state1)⦈) (s⦇boundValues := MList.insert valId res (boundValues state1)⦈)"
      using calculation by auto
  moreover have forwardStep: "(Let valId val innerCont, ctx1, state1, env1, [], payments1) →f2 (
      innerCont, ValueAbstraction valId#ctx1, state1⦇boundValues := MList.insert valId res (
      boundValues state1)⦈, env1, ([] @ [ReduceNoWarning]), payments)" using calculation apply
      auto
    by (simp add: FaustusV2Semantics.small_step_reduce.simps)
  moreover have "split_cases_relation role choiceName k (innerCont, ValueAbstraction valId#ctx1,
      state1⦇boundValues := MList.insert valId res (boundValues state1)⦈, env1,
      convertReduceWarnings ([] @ [ReduceNoWarning]), payments) (cont, ValueAbstraction valId #
      ctx, s⦇FState.boundValues := MList.insert valId res (FState.boundValues s)⦈, env,
      convertReduceWarnings ([] @ [ReduceNoWarning]), payments)" using calculation by auto
  ultimately show ?case by (metis isFreshChoiceInProcess.simps split_cases_relation.simps)
next
  case (ReassignValNoShadow valId s env val res cont ctx payments)
  moreover obtain innerCont where "c1 = (StatementCont (ReassignVal valId val) innerCont) ∧
      split_cases role choiceName k innerCont = cont" using calculation apply (cases c1, auto)
    by (smt (verit) FaustusV2AST.FContract.distinct)
  moreover have "isFreshChoiceInValue choiceName val" using calculation by auto
  moreover have "evalFValue env state1 val = Some res" using calculation by (metis (no_types,
      opaque_lifting) fresh_choice_state_preserves_value_evaluation(1) split_cases_relation.
      simps)
  moreover have "split_cases_context role choiceName k (ValueAbstraction valId#ctx1) = (
      ValueAbstraction valId#ctx)" using calculation by auto
  moreover have "state_minimal_eq choiceName (state1⦇boundValues := MList.insert valId res (
      boundValues state1)⦈) (s⦇boundValues := MList.insert valId res (boundValues state1)⦈)"
      using calculation by auto
  moreover have forwardStep: "(StatementCont (ReassignVal valId val) innerCont, ctx1, state1,
      env1, [], payments1) →f2 (innerCont, ValueAbstraction valId#ctx1, state1⦇boundValues :=
      MList.insert valId res (boundValues state1)⦈, env1, ([] @ [ReduceNoWarning]), payments)"
      using calculation by (auto simp add: FaustusV2Semantics.small_step_reduce.simps)
  moreover have "split_cases_relation role choiceName k (innerCont, ValueAbstraction valId#ctx1,
      state1⦇boundValues := MList.insert valId res (boundValues state1)⦈, env1,
      convertReduceWarnings ([] @ [ReduceNoWarning]), payments) (cont, ValueAbstraction valId #
      ctx, s⦇FState.boundValues := MList.insert valId res (FState.boundValues s)⦈, env,
      convertReduceWarnings ([] @ [ReduceNoWarning]), payments)" using calculation by auto
  ultimately show ?case by (metis isFreshChoiceInProcess.simps split_cases_relation.simps)
next
  case (LetObservation e s obs res oId c cx ps)
  moreover obtain innerCont where "c1 = (LetObservation oId obs innerCont) ∧ split_cases role
```

```
      choiceName k innerCont = c" using calculation apply (cases c1, auto)
    by (smt (verit) FaustusV2AST.FContract.distinct)
  moreover have "isFreshChoiceInObservation choiceName obs" using calculation by auto
  moreover have "evalFObservation e state1 obs = Some res" using calculation by (metis (no_types,
       opaque_lifting) fresh_choice_state_preserves_value_evaluation(2) split_cases_relation.
      simps)
  moreover have "split_cases_context role choiceName k (ObservationAbstraction oId#ctx1) = (
      ObservationAbstraction oId#cx)" using calculation by auto
  moreover have "state_minimal_eq choiceName (state1⦇boundValues := MList.insert oId (if res then
       1 else 0) (boundValues state1)⦈) (s⦇boundValues := MList.insert oId (if res then 1 else
      0) (boundValues state1)⦈)" using calculation by auto
  moreover have forwardStep: "(LetObservation oId obs innerCont, ctx1, state1, env1, [],
      payments1) →f2 (innerCont, ObservationAbstraction oId#ctx1, state1⦇boundValues := MList.
      insert oId (if res then 1 else 0) (boundValues state1)⦈, env1, ([] @ [ReduceNoWarning]),
      ps)" using calculation by (auto simp add: FaustusV2Semantics.small_step_reduce.simps)
  moreover have "split_cases_relation role choiceName k (innerCont, ObservationAbstraction oId#
      ctx1, state1⦇boundValues := MList.insert oId (if res then 1 else 0) (boundValues state1)⦈,
       env1, convertReduceWarnings ([] @ [ReduceNoWarning]), ps) (c, ObservationAbstraction oId
      # cx, s⦇FState.boundValues := MList.insert oId (if res then 1 else 0) (FState.boundValues
      s)⦈, e, convertReduceWarnings ([] @ [ReduceNoWarning]), ps)" using calculation by auto
  ultimately show ?case by (metis isFreshChoiceInProcess.simps split_cases_relation.simps)
next
  case (ReassignObservation e s obs res oId c cx ps)
  moreover obtain innerCont where "c1 = (StatementCont (ReassignObservation oId obs) innerCont) ∧
       split_cases role choiceName k innerCont = c" using calculation apply (cases c1, auto)
    by (smt (verit) FaustusV2AST.FContract.distinct)
  moreover have "isFreshChoiceInObservation choiceName obs" using calculation by auto
  moreover have "evalFObservation e state1 obs = Some res" using calculation by (metis (no_types,
       opaque_lifting) fresh_choice_state_preserves_value_evaluation(2) split_cases_relation.
      simps)
  moreover have "split_cases_context role choiceName k (ObservationAbstraction oId#ctx1) = (
      ObservationAbstraction oId#cx)" using calculation by auto
  moreover have "state_minimal_eq choiceName (state1⦇boundValues := MList.insert oId (if res then
       1 else 0) (boundValues state1)⦈) (s⦇boundValues := MList.insert oId (if res then 1 else
      0) (boundValues state1)⦈)" using calculation by auto
  moreover have forwardStep: "(StatementCont (ReassignObservation oId obs) innerCont, ctx1,
      state1, env1, [], payments1) →f2 (innerCont, ObservationAbstraction oId#ctx1, state1⦇
      boundValues := MList.insert oId (if res then 1 else 0) (boundValues state1)⦈, env1, ([] @
      [ReduceNoWarning]), ps)" using calculation by (auto simp add: FaustusV2Semantics.
      small_step_reduce.simps)
  moreover have "split_cases_relation role choiceName k (innerCont, ObservationAbstraction oId#
      ctx1, state1⦇boundValues := MList.insert oId (if res then 1 else 0) (boundValues state1)⦈,
       env1, convertReduceWarnings ([] @ [ReduceNoWarning]), ps) (c, ObservationAbstraction oId
      # cx, s⦇FState.boundValues := MList.insert oId (if res then 1 else 0) (FState.boundValues
```

```
      s)⦈, e, convertReduceWarnings ([] @ [ReduceNoWarning]), ps)" using calculation by auto
  ultimately show ?case by (metis isFreshChoiceInProcess.simps split_cases_relation.simps)
next
  case (LetParty s pk res pId c cx e ps)
  moreover obtain innerCont where "c1 = (LetParty pId pk innerCont) ∧ split_cases role choiceName
       k innerCont = c" using calculation apply (cases c1, auto)
    by (smt (verit) FaustusV2AST.FContract.distinct)
  moreover have "evalFParty state1 pk = Some res" using calculation by (metis (no_types, lifting)
       fresh_choice_state_preserves_party_evaluation split_cases_relation.simps state_minimal_eq
      .simps)
  moreover have "split_cases_context role choiceName k (PartyAbstraction pId#ctx1) = (
      PartyAbstraction pId#cx)" using calculation by auto
  moreover have "state_minimal_eq choiceName (state1⦇boundParties := MList.insert pId res (
      boundParties state1)⦈) (s⦇boundParties := MList.insert pId res (boundParties s)⦈)" using
      calculation by auto
  moreover have forwardStep: "(LetParty pId pk innerCont, ctx1, state1, env1, [], payments1) →f2
       (innerCont, PartyAbstraction pId#ctx1, state1⦇boundParties := MList.insert pId res (
      boundParties state1)⦈, env1, ([] @ [ReduceNoWarning]), ps)" using calculation by (auto
      simp add: FaustusV2Semantics.small_step_reduce.simps)
  moreover have "split_cases_relation role choiceName k (innerCont, PartyAbstraction pId#ctx1,
      state1⦇boundParties := MList.insert pId res (boundParties state1)⦈, env1,
      convertReduceWarnings ([] @ [ReduceNoWarning]), ps) (c, PartyAbstraction pId # cx, s⦇
      boundParties := MList.insert pId res (boundParties s)⦈, e, convertReduceWarnings ([] @ [
      ReduceNoWarning]), ps)" using calculation by auto
  ultimately show ?case by (metis isFreshChoiceInProcess.simps split_cases_relation.simps)
next
  case (ReassignParty s pk res pId c cx e ps)
  moreover obtain innerCont where "c1 = (StatementCont (ReassignParty pId pk) innerCont) ∧
      split_cases role choiceName k innerCont = c" using calculation apply (cases c1, auto)
    by (smt (verit) FaustusV2AST.FContract.distinct)
  moreover have "evalFParty state1 pk = Some res" using calculation by (metis (no_types, lifting)
       fresh_choice_state_preserves_party_evaluation split_cases_relation.simps state_minimal_eq
      .simps)
  moreover have "split_cases_context role choiceName k (PartyAbstraction pId#ctx1) = (
      PartyAbstraction pId#cx)" using calculation by auto
  moreover have "state_minimal_eq choiceName (state1⦇boundParties := MList.insert pId res (
      boundParties state1)⦈) (s⦇boundParties := MList.insert pId res (boundParties s)⦈)" using
      calculation by auto
  moreover have forwardStep: "(StatementCont (ReassignParty pId pk) innerCont, ctx1, state1, env1
      , [], payments1) →f2 (innerCont, PartyAbstraction pId#ctx1, state1⦇boundParties := MList.
      insert pId res (boundParties state1)⦈, env1, ([] @ [ReduceNoWarning]), ps)" using
      calculation by (auto simp add: FaustusV2Semantics.small_step_reduce.simps)
  moreover have "split_cases_relation role choiceName k (innerCont, PartyAbstraction pId#ctx1,
      state1⦇boundParties := MList.insert pId res (boundParties state1)⦈, env1,
```

```
      convertReduceWarnings ([] @ [ReduceNoWarning]), ps) (c, PartyAbstraction pId # cx, s⦇
      boundParties := MList.insert pId res (boundParties s)⦈, e, convertReduceWarnings ([] @ [
      ReduceNoWarning]), ps)" using calculation by auto
  ultimately show ?case by (metis isFreshChoiceInProcess.simps split_cases_relation.simps)
next
  case (AssertTrue env s obs cont ctx payments)
  moreover obtain innerCont where "c1 = (StatementCont (Assert obs) innerCont) ∧ split_cases role
       choiceName k innerCont = cont" using calculation apply (cases c1, auto)
    by (smt (verit) FaustusV2AST.FContract.distinct)
  moreover have "isFreshChoiceInObservation choiceName obs" using calculation by auto
  moreover have "evalFObservation env1 state1 obs = Some True" using calculation by (metis (
      no_types, lifting) fresh_choice_state_preserves_value_evaluation(2) split_cases_relation.
      simps)
  moreover have forwardStep: "(StatementCont (Assert obs) innerCont, ctx1, state1, env1, [],
      payments1) →f2 (innerCont, ctx1, state1, env1, ([] @ [ReduceNoWarning]), payments)" using
       calculation by (auto simp add: FaustusV2Semantics.small_step_reduce.simps)
  moreover have "split_cases_relation role choiceName k (innerCont, ctx1, state1, env1,
      convertReduceWarnings ([] @ [ReduceNoWarning]), payments) (cont, ctx, s, env,
      convertReduceWarnings ([] @ [ReduceNoWarning]), payments)" using calculation by auto
  ultimately show ?case by (metis isFreshChoiceInProcess.simps split_cases_relation.simps)
next
  case (AssertFalse env s obs cont ctx payments)
  moreover obtain innerCont where "c1 = (StatementCont (Assert obs) innerCont) ∧ split_cases role
       choiceName k innerCont = cont" using calculation apply (cases c1, auto)
    by (smt (verit) FaustusV2AST.FContract.distinct)
  moreover have "isFreshChoiceInObservation choiceName obs" using calculation by auto
  moreover have "evalFObservation env1 state1 obs = Some False" using calculation by (metis (
      no_types, lifting) fresh_choice_state_preserves_value_evaluation(2) split_cases_relation.
      simps)
  moreover have forwardStep: "(StatementCont (Assert obs) innerCont, ctx1, state1, env1, [],
      payments1) →f2 (innerCont, ctx1, state1, env1, ([] @ [ReduceAssertionFailed]), payments)"
       using calculation by (auto simp add: FaustusV2Semantics.small_step_reduce.simps)
  moreover have "split_cases_relation role choiceName k (innerCont, ctx1, state1, env1,
      convertReduceWarnings ([] @ [ReduceAssertionFailed]), payments) (cont, ctx, s, env,
      convertReduceWarnings ([] @ [ReduceAssertionFailed]), payments)" using calculation by auto
  ultimately show ?case by (metis isFreshChoiceInProcess.simps split_cases_relation.simps)
next
  case (LetC cid params boundCon cont ctx s env payments)
  moreover obtain innerBodyCont innerCont where "c1 = (LetC cid params innerBodyCont innerCont) ∧
       split_cases role choiceName k innerCont = cont ∧ split_cases role choiceName k
      innerBodyCont = boundCon" using calculation apply (cases c1, auto)
    by (smt (verit) FaustusV2AST.FContract.distinct)
  moreover have "split_cases_context role choiceName k (ContractAbstraction (cid, params,
      innerBodyCont)#ctx1) = (ContractAbstraction (cid, params, boundCon)#ctx)" using
```

```
      calculation by auto
  moreover have forwardStep: "(LetC cid params innerBodyCont innerCont, ctx1, state1, env1, [],
      payments1) →f2 (innerCont, ContractAbstraction (cid, params, innerBodyCont)#ctx1, state1,
       env1, ([] @ [ReduceNoWarning]), payments)" using calculation by (auto simp add:
      FaustusV2Semantics.small_step_reduce.simps)
  ultimately show ?case by fastforce
next
  case (UseCFound ctx cid params bodyCont innerCtx env s args newState payments)
  moreover have "c1 = (UseC cid args)" using calculation apply (cases c1, auto)
    by (smt (verit) FaustusV2AST.FContract.distinct)
  moreover obtain bodyCont1 innerCtx1 where "lookupContractIdAbsInformation ctx1 cid = Some (
      params, bodyCont1, innerCtx1) ∧ split_cases role choiceName k bodyCont1 = bodyCont ∧
      split_cases_context role choiceName k innerCtx1 = innerCtx" using calculation(1,3)
      lookup_split_cases_context_preserved_split_context by fastforce
  moreover have "isFreshChoiceInArguments choiceName args" using calculation by auto
  moreover obtain newState1 where "evalFArguments env1 state1 params args = Some newState1 ∧
      state_minimal_eq choiceName newState1 newState" using calculation
      fresh_choice_state_preserves_argument_evaluation by (metis split_cases_relation.simps
      state_minimal_eq_commutative)
  moreover have forwardStep: "(UseC cid args, ctx1, state1, env1, [], payments1) →f2 (bodyCont1,
       paramsToFContext params @ innerCtx1, newState1, env1, ([] @ [ReduceNoWarning]), payments)
      " using calculation by (auto simp add: FaustusV2Semantics.small_step_reduce.simps)
  moreover have "split_cases_context role choiceName k (paramsToFContext params @ innerCtx1) = (
      paramsToFContext params @ innerCtx)" using calculation(5)
      split_cases_context_append_params_preserved by presburger
  ultimately show ?case by (metis freshChoicePreserved_small_step_reduce isFreshChoiceInProcess.
      simps split_cases_relation.simps)
qed

lemma react_step_strongly_preserves_split_cases_relation:
"internal_reaction p1 p1' ⟹
  split_cases_relation role choiceName k p1 p2 ⟹
  ∃p2'.
    internal_reaction p2 p2' ∧
    split_cases_relation role choiceName k p1' p2'"
proof (induction p1 p1' arbitrary: p2 rule: internal_reaction.induct)
  case (SmallStepReduce c ctx s env payments newC newCtx newS newEnv newWarnings newPayments)
  then show ?case by (smt (z3) internal_reaction.SmallStepReduce
      small_step_preserves_split_cases_relation split_cases_relation.elims(2)
      split_cases_relation.simps)
qed

lemma react_step_preserves_split_cases_relation:
"internal_reaction p1 p1' ⟹
```

```
  split_cases_relation role choiceName k p1 p2 ⟹
  ∃p2'.
    star internal_reaction p2 p2' ∧
    split_cases_relation role choiceName k p1' p2'"
proof (induction p1 p1' arbitrary: p2 rule: internal_reaction.induct)
  case (SmallStepReduce c ctx s env warnings payments newC newCtx newS newEnv newWarnings
       newPayments)
  then show ?case by (meson internal_reaction.SmallStepReduce
       react_step_strongly_preserves_split_cases_relation star_step1)
qed

lemma split_cases_relation_preserves_halted_action:
"computeActionStep inp (a1, state1, env1) = None ⟹
  isFreshChoiceInAction choiceName a1 ⟹
  state_minimal_eq choiceName state1 state2 ⟹
  computeActionStep inp (a1, state2, env1) = None"
proof (induction inp a1 state1 env1 rule: computeActionStep.induct[split_format(complete)])
  case (1 accId mParty mToken mMoney fAccId fParty fToken fValue s env)
  then show ?case apply (auto split: option.split_asm option.split if_split_asm)
         apply (metis "1.prems"(3) fresh_choice_state_preserves_value_evaluation(1) option.discI)
        apply (metis fresh_choice_state_preserves_party_evaluation option.discI)
       apply (metis fresh_choice_state_preserves_party_evaluation option.discI)
      apply (metis fresh_choice_state_preserves_party_evaluation option.inject)
     apply (metis fresh_choice_state_preserves_party_evaluation option.inject)
    by (metis "1.prems"(3) fresh_choice_state_preserves_value_evaluation(1) option.inject)
next
  case (2 mChoiceId choice fChoiceId bounds s env)
  then show ?case apply (auto split: option.split_asm option.split if_split_asm)
    using fresh_choice_state_preserves_choiceId_evaluation by blast
next
  case (3 obs s env)
  then show ?case apply auto
    by (metis 3(3) fresh_choice_state_preserves_value_evaluation(2) option.distinct(1))
qed auto

lemma small_step_action_preserves_split_cases_relation:
"inp ⊢ (a, state1, env) →a_f2 (newS1, newW1) ⟹
  isFreshChoiceInAction choiceName a ⟹
  state_minimal_eq choiceName state1 state2 ⟹
  (∃newS2 . inp ⊢ (a, state2, env) →a_f2 (newS2, newW1) ∧
    state_minimal_eq choiceName newS1 newS2)"
proof (induction inp a state1 env newS1 newW1 arbitrary: state2 rule: small_step_reduce_action.
     induct[split_format(complete)])
  case (DepositPositiveActionStep env s fValue valRes fAccId fAccIdRes fParty fPartyRes token)
```

```
  moreover have "evalFValue env state2 fValue = Some valRes" using
        fresh_choice_state_preserves_value_evaluation(1)[of env s fValue "Some valRes" choiceName
        state2] calculation by auto
  moreover have "evalFParty state2 fAccId = Some fAccIdRes" using
        fresh_choice_state_preserves_party_evaluation[of s fAccId "Some fAccIdRes" state2]
        calculation by auto
  moreover have "evalFParty state2 fParty = Some fPartyRes" using
        fresh_choice_state_preserves_party_evaluation[of s fParty "Some fPartyRes" state2]
        calculation by auto
  moreover have "valRes > 0" by (simp add: calculation(2))
  moreover have "(IDeposit fAccIdRes fPartyRes token valRes) ⊢ (Deposit fAccId fParty token
        fValue, state2, env) →a_f2 (state2⦇FState.accounts := addMoneyToAccount fAccIdRes token
        valRes (FState.accounts s)⦈, ApplyNoWarning)" using calculation small_step_reduce_action.
        simps by fastforce
  moreover have "state_minimal_eq choiceName (s⦇FState.accounts := addMoneyToAccount fAccIdRes
        token valRes (FState.accounts s)⦈) (state2⦇FState.accounts := addMoneyToAccount fAccIdRes
        token valRes (FState.accounts s)⦈)" using calculation by auto
  ultimately show ?case by auto
next
  case (DepositNonPositiveActionStep env s fValue valRes fAccId fAccIdRes fParty fPartyRes token)
  moreover have "evalFValue env state2 fValue = Some valRes" using
        fresh_choice_state_preserves_value_evaluation(1)[of env s fValue "Some valRes" choiceName
        state2] calculation by auto
  moreover have "evalFParty state2 fAccId = Some fAccIdRes" using
        fresh_choice_state_preserves_party_evaluation[of s fAccId "Some fAccIdRes" state2]
        calculation by auto
  moreover have "evalFParty state2 fParty = Some fPartyRes" using
        fresh_choice_state_preserves_party_evaluation[of s fParty "Some fPartyRes" state2]
        calculation by auto
  moreover have "valRes ≤ 0" by (simp add: calculation(2))
  moreover have "(IDeposit fAccIdRes fPartyRes token valRes) ⊢ (Deposit fAccId fParty token
        fValue, state2, env) →a_f2 (state2⦇FState.accounts := addMoneyToAccount fAccIdRes token
        valRes (FState.accounts s)⦈, ApplyNonPositiveDeposit fPartyRes fAccIdRes token valRes)"
        using calculation small_step_reduce_action.simps by fastforce
  moreover have "state_minimal_eq choiceName (s⦇FState.accounts := addMoneyToAccount fAccIdRes
        token valRes (FState.accounts s)⦈) (state2⦇FState.accounts := addMoneyToAccount fAccIdRes
        token valRes (FState.accounts s)⦈)" using calculation by auto
  ultimately show ?case by auto
next
  case (ChoiceActionStep choice bounds s fChoiceId fChoiceIdRes mChoiceId ctx env)
  then show ?case apply auto
    by (smt (verit, best) ChoiceActionStep.hyps(1) FState.select_convs(1-5) FState.surjective
          FState.update_convs(2) MList.insert_lookup_Some computeActionStep.simps(2)
          computeActionStepIsSmallStep fresh_choice_state_preserves_choiceId_evaluation
```

```
        insert_lookup_different)
next
  case (NotifyActionStep env s obs ctx)
  then show ?case using fresh_choice_state_preserves_value_evaluation(2) isFreshChoiceInAction.
       simps(2) small_step_reduce_action.NotifyActionStep by blast
qed

lemma split_cases_relation_preserves_halted_guard:
"computeGuardStep inp (g1, state1, env1) = None ⟹
  isFreshChoiceInGuard choiceName g1 ⟹
  state_minimal_eq choiceName state1 state2 ⟹
  computeGuardStep inp (g1, state2, env1) = None"
  apply (induction inp g1 state1 env1 rule: computeGuardStep.induct[split_format(complete)])
  by (auto split: option.split_asm option.split simp add:
       split_cases_relation_preserves_halted_action)

lemma small_step_guard_preserves_split_cases_relation:
"inp ⊢ (g1, state1, env1) →gf2 (stmts, g1', state1', warn) ⟹
  isFreshChoiceInGuard choiceName g1 ⟹
  state_minimal_eq choiceName state1 state2 ⟹
  (∃ state2' . inp ⊢ (g1, state2, env1) →gf2 (stmts, g1', state2', warn) ∧
    state_minimal_eq choiceName state1' state2')"
proof (induction inp g1 state1 env1 stmts g1' state1' warn
      arbitrary: state2 rule: small_step_reduce_guard.induct[split_format(complete)])
  case (ActionGuardStep g a ins s env newS newW)
  moreover obtain newS2 where "ins ⊢ (a, state2, env) →af2 (newS2, newW) ∧ state_minimal_eq
       choiceName newS newS2" using calculation isFreshChoiceInGuard.simps(1)
       small_step_action_preserves_split_cases_relation by blast
  moreover have "ins ⊢ (ActionGuard a, state2, env) →gf2 ([], None, newS2, newW)" using
       calculation small_step_reduce_guard.ActionGuardStep by blast
  ultimately show ?case by auto
next
  case (GuardStmtsGuardCompleteGuardStep g g1 stmts ins s env newStmts newS)
  then show ?case by (meson isFreshChoiceInGuard.simps(3) small_step_reduce_guard.
       GuardStmtsGuardCompleteGuardStep)
next
  case (GuardStmtsGuardIncompleteGuard g g1 stmts ins s env newStmts nextG1 newS)
  then show ?case by (meson isFreshChoiceInGuard.simps(3) small_step_reduce_guard.
       GuardStmtsGuardIncompleteGuard)
next
  case (DisjointGuardMatchesG1)
  then show ?case by (meson isFreshChoiceInGuard.simps(4) small_step_reduce_guard.
       DisjointGuardMatchesG1)
next
```

```
  case (DisjointGuardMatchesG2)
  then show ?case by (meson isFreshChoiceInGuard.simps(4) small_step_reduce_guard.
       DisjointGuardMatchesG2 split_cases_relation_preserves_halted_guard)
next
  case (InterleavedGuardMatchesG1Complete ins g1 s env stmts newS g2)
  then show ?case by (meson isFreshChoiceInGuard.simps(5) small_step_reduce_guard.
       InterleavedGuardMatchesG1Complete)
next
  case (InterleavedGuardMatchesG1Incomplete ins g1 s env stmts innerG1 newS g2)
  then show ?case by (meson isFreshChoiceInGuard.simps(5) small_step_reduce_guard.
       InterleavedGuardMatchesG1Incomplete)
next
  case (InterleavedGuardMatchesG2Complete ins g2 s env stmts newS g1)
  then show ?case by (meson isFreshChoiceInGuard.simps(5) small_step_reduce_guard.
       InterleavedGuardMatchesG2Complete split_cases_relation_preserves_halted_guard)
next
  case (InterleavedGuardMatchesG2Incomplete ins g2 s env stmts innerG2 newS g1)
  then show ?case by (meson isFreshChoiceInGuard.simps(5) small_step_reduce_guard.
       InterleavedGuardMatchesG2Incomplete split_cases_relation_preserves_halted_guard)
qed (auto; meson small_step_reduce_guard.GuardThenGuardCompleteFirstGuardStep
     GuardThenGuardIncompleteGuard)+

lemma split_cases_distributes_past_prepend_statements:
"prependStatements stmts (split_cases role choiceName k c) = split_cases role choiceName k (
     prependStatements stmts c)" by (induction stmts c rule: prependStatements.induct, auto)

lemma split_cases_relation_preserves_halted_case:
"computeCaseStep inp (c1, state1, env1) = None ⟹
  isFreshChoiceInCasesGuards choiceName [c1] ⟹
  state_minimal_eq choiceName state1 state2 ⟹
  computeCaseStep inp ((λ(Case g c) ⇒ (Case g (split_cases role choiceName k c))) c1, state2,
       env1) = None"
proof (induction inp c1 state1 env1 arbitrary: state2 rule: computeCaseStep.induct[split_format(
     complete)])
  case (1 ins g c s env)
  then show ?case apply auto
    apply (cases "computeGuardStep ins (g, s, env)", auto)
     apply (cases "computeGuardStep ins (g, state2, env)", auto)
     apply (metis 1(3) option.distinct(1) split_cases_relation_preserves_halted_guard)
    apply (cases "computeGuardStep ins (g, state2, env)", auto)
    by (split option.split_asm option.split, auto)
qed

lemma small_step_case_preserves_split_cases_relation:
```

```
"inp ⊢ (c1, state1, env1) →case_f2 (c1', state1', newW) ⟹
  isFreshChoiceInCasesGuards choiceName [c1] ⟹
  state_minimal_eq choiceName state1 state2 ⟹
  (∃c2' state2' . inp ⊢ ((λ(Case g c) ⇒ (Case g (split_cases role choiceName k c))) c1, state2,
       env1) →case_f2 (c2', state2', newW) ∧
    state_minimal_eq choiceName state1' state2' ∧
    (c2' t (split_cases role choiceName k tc1) = split_cases role choiceName k (c1' t tc1)))"
proof (induction inp c1 state1 env1 c1' state1' newW
      arbitrary: state2 rule: small_step_reduce_case.induct[split_format(complete)])
  case (CaseGuardCompleteStep ins g s env stmts newS newW c)
  moreover obtain state2' where "ins ⊢ (g, state2, env) →g_f2 (stmts, None, state2', newW) ∧
       state_minimal_eq choiceName newS state2'" using
       small_step_guard_preserves_split_cases_relation[of ins g s env stmts None newS newW
       choiceName state2] calculation by auto
  moreover have "ins ⊢ (Case g (split_cases role choiceName k c), state2, env) →case_f2 ((λt tc .
        prependStatements stmts (split_cases role choiceName k c)), state2', newW)" by (simp add:
        calculation small_step_reduce_case.CaseGuardCompleteStep)
  moreover have "prependStatements stmts (split_cases role choiceName k c) = split_cases role
       choiceName k (prependStatements stmts c)" by (induction stmts c rule: prependStatements.
       induct, auto)
  ultimately show ?case by auto
next
  case (CaseGuardIncompleteStep ins g s env stmts innerG newS newW c)
  moreover obtain state2' where "ins ⊢ (g, state2, env) →g_f2 (stmts, Some innerG, state2', newW)
        ∧ state_minimal_eq choiceName newS state2'" using
       small_step_guard_preserves_split_cases_relation[of ins g s env stmts "Some innerG" newS
       newW choiceName state2] calculation by auto
  moreover have "ins ⊢ (Case g (split_cases role choiceName k c), state2, env) →case_f2 ((λt tc .
        prependStatements stmts (When [Case innerG  (split_cases role choiceName k c)] t tc)),
       state2', newW)" by (simp add: calculation small_step_reduce_case.CaseGuardIncompleteStep)
  moreover have "When [Case innerG  (split_cases role choiceName k c)] t (split_cases role
       choiceName k tc1) = (split_cases role choiceName k (When [Case innerG c] t tc1))" by (
       induction k, auto)
  moreover have "(λt tc . prependStatements stmts (When [Case innerG  (split_cases role
       choiceName k c)] t tc)) t (split_cases role choiceName k tc1) = prependStatements stmts (
       When [Case innerG  (split_cases role choiceName k c)] t (split_cases role choiceName k tc1
       ))" by auto
  moreover have "(λt tc . prependStatements stmts (When [Case innerG  (split_cases role
       choiceName k c)] t tc)) t (split_cases role choiceName k tc1) = split_cases role
       choiceName k (prependStatements stmts (When [Case innerG c] t tc1))" by (simp only:
       calculation split_cases_distributes_past_prepend_statements)
  ultimately show ?case by (metis FaustusV2AST.FCase.case)
qed
```

```
lemma minimal_state_eq_preserves_small_step_action:
"inp ⊢ (a, state1, env1) →a_f2 (newS, newW) ⟹
  isFreshChoiceInAction choiceName a ⟹
  state_minimal_eq choiceName state1 state2 ⟹
  (∃newS2 . inp ⊢ (a, state2, env1) →a_f2 (newS2, newW) ∧
    state_minimal_eq choiceName newS newS2)"
proof (induction inp a state1 env1 newS newW rule: small_step_reduce_action.induct[split_format(
     complete)])
  case (DepositPositiveActionStep env s fValue valRes fAccId fAccIdRes fParty fPartyRes token)
  moreover have "evalFValue env state2 fValue = Some valRes" using
       fresh_choice_state_preserves_value_evaluation(1)[of env s fValue "Some valRes" choiceName
       state2] calculation by auto
  moreover have "evalFParty state2 fAccId = Some fAccIdRes" using
       fresh_choice_state_preserves_party_evaluation[of s fAccId "Some fAccIdRes" state2]
       calculation by auto
  moreover have "evalFParty s fParty = Some fPartyRes" using
       fresh_choice_state_preserves_party_evaluation[of s fParty "Some fPartyRes" state2]
       calculation by auto
  moreover have "valRes > 0" using calculation by auto
  moreover have "(IDeposit fAccIdRes fPartyRes token valRes) ⊢ (Deposit fAccId fParty token
       fValue, state2, env) →a_f2 (state2⦇FState.accounts := addMoneyToAccount fAccIdRes token
       valRes (FState.accounts state2)⦈, ApplyNoWarning)" using small_step_reduce_action.
       DepositPositiveActionStep[of env state2 fValue valRes fAccId fAccIdRes fParty fPartyRes]
       calculation apply auto
    using fresh_choice_state_preserves_party_evaluation by (smt (verit, del_insts) FState.
         surjective FState.update_convs(1))
  ultimately show ?case by auto
next
  case (DepositNonPositiveActionStep env s fValue valRes fAccId fAccIdRes fParty fPartyRes token)
  moreover have "evalFValue env state2 fValue = Some valRes" using
       fresh_choice_state_preserves_value_evaluation(1)[of env s fValue "Some valRes" choiceName
       state2] calculation by auto
  moreover have "evalFParty state2 fAccId = Some fAccIdRes" using
       fresh_choice_state_preserves_party_evaluation[of s fAccId "Some fAccIdRes" state2]
       calculation by auto
  moreover have "evalFParty s fParty = Some fPartyRes" using
       fresh_choice_state_preserves_party_evaluation[of s fParty "Some fPartyRes" state2]
       calculation by auto
  moreover have "valRes ≤ 0" using calculation by auto
  moreover have "(IDeposit fAccIdRes fPartyRes token valRes) ⊢ (Deposit fAccId fParty token
       fValue, state2, env) →a_f2 (state2⦇FState.accounts := addMoneyToAccount fAccIdRes token
       valRes (FState.accounts state2)⦈, ApplyNonPositiveDeposit fPartyRes fAccIdRes token valRes
       )" using small_step_reduce_action.DepositNonPositiveActionStep[of env state2 fValue valRes
        fAccId fAccIdRes fParty fPartyRes] calculation apply auto
```

```
    using fresh_choice_state_preserves_party_evaluation by (smt (verit, del_insts) FState.
          surjective FState.update_convs(1))
  ultimately show ?case by auto
next
  case (ChoiceActionStep choice bounds s fChoiceId fChoiceIdRes mChoiceId env)
  moreover have "evalFChoiceId state2 fChoiceId = Some fChoiceIdRes" using
        fresh_choice_state_preserves_choiceId_evaluation[of s fChoiceId "Some fChoiceIdRes" state2
        ] calculation by auto
  moreover have "(IChoice mChoiceId choice) ⊢ (FAction.Choice fChoiceId bounds, state2, env) →af
        2 (state2⦇FState.choices := MList.insert mChoiceId choice (FState.choices state2)⦈,
        ApplyNoWarning)" using calculation small_step_reduce_action.ChoiceActionStep by auto
  moreover have "state_minimal_eq choiceName (s⦇FState.choices := MList.insert mChoiceId choice (
        FState.choices s)⦈) (state2⦇FState.choices := MList.insert mChoiceId choice (FState.
        choices state2)⦈)" using calculation apply auto
    by (metis MList.insert_lookup_Some insert_lookup_different)
  ultimately show ?case by auto
next
  case (NotifyActionStep env s obs)
  moreover have "evalFObservation env state2 obs = Some True" using NotifyActionStep calculation
        (2,3) fresh_choice_state_preserves_value_evaluation(2) isFreshChoiceInAction.simps(2) by
        blast
  ultimately show ?case using small_step_reduce_action.NotifyActionStep by blast
qed

lemma minimal_state_eq_preserves_small_step_action_halted:
"computeActionStep inp (a, state1, env1) = None ⟹
  isFreshChoiceInAction choiceName a ⟹
  state_minimal_eq choiceName state1 state2 ⟹
  computeActionStep inp (a, state2, env1) = None"
  apply (induction inp a state1 env1 rule: computeActionStep.induct[split_format(complete)])
           apply (auto split: option.split_asm option.split)
       apply (metis evalFValue_InsertDifferentValueStillExecutes(1) option.distinct(1))
      apply ((metis fresh_choice_state_preserves_party_evaluation option.distinct(1))+)[2]
    apply (auto split: if_split if_split_asm)
      apply ((metis fresh_choice_state_preserves_party_evaluation option.inject)+)[2]
    apply (smt (verit, ccfv_threshold) fresh_choice_state_preserves_value_evaluation(1) option.
          inject state_minimal_eq.simps)
   apply (metis fresh_choice_state_preserves_choiceId_evaluation)
  by (smt (verit, ccfv_SIG) fresh_choice_state_preserves_value_evaluation(2) state_minimal_eq.
        simps)

lemma minimal_state_eq_preserves_small_step_guard_halted:
"computeGuardStep inp (g, state1, env1) = None ⟹
  isFreshChoiceInGuard choiceName g ⟹
```

```
  state_minimal_eq choiceName state1 state2 ⟹
  computeGuardStep inp (g, state2, env1) = None"
  apply (induction inp g state1 env1 arbitrary: state2 rule: computeGuardStep.induct[split_format
       (complete)])
  using minimal_state_eq_preserves_small_step_action_halted by (auto split: option.split option.
       split_asm)

lemma minimal_state_eq_preserves_small_step_guard:
"inp ⊢ (g, state1, env1) →g_f2 (stmts, nextG, state1', newW) ⟹
  isFreshChoiceInGuard choiceName g ⟹
  state_minimal_eq choiceName state1 state2 ⟹
  (∃state2' . inp ⊢ (g, state2, env1) →g_f2 (stmts, nextG, state2', newW) ∧
    state_minimal_eq choiceName state1' state2')"
proof (induction inp g state1 env1 stmts nextG state1' newW arbitrary: state2 rule:
     small_step_reduce_guard.induct[split_format(complete)])
  case (ActionGuardStep g a ins s env newS newW)
  moreover obtain newS2 where "ins ⊢ (a, state2, env) →a_f2 (newS2, newW) ∧ state_minimal_eq
       choiceName newS newS2" using minimal_state_eq_preserves_small_step_action[of ins a s env
       newS newW choiceName state2] calculation by auto
  ultimately show ?case using small_step_reduce_guard.ActionGuardStep by blast
qed (meson isFreshChoiceInGuard.simps minimal_state_eq_preserves_small_step_guard_halted
    GuardThenGuardCompleteFirstGuardStep GuardThenGuardIncompleteGuard
         GuardStmtsGuardCompleteGuardStep
    GuardStmtsGuardIncompleteGuard DisjointGuardMatchesG1 DisjointGuardMatchesG2
         InterleavedGuardMatchesG1Complete
    InterleavedGuardMatchesG1Incomplete InterleavedGuardMatchesG2Complete
         InterleavedGuardMatchesG2Incomplete)+

lemma minimal_state_eq_preserves_small_step_case_halted:
"computeCaseStep inp (c1, state1, env1) = None ⟹
  isFreshChoiceInCasesGuards choiceName [c1] ⟹
  state_minimal_eq choiceName state1 state2 ⟹
  computeCaseStep inp (c1, state2, env1) = None"
  apply (induction inp c1 state1 env1 rule: computeCaseStep.induct[split_format(complete)])
  using minimal_state_eq_preserves_small_step_guard_halted by (auto split: option.split_asm
       option.split)

lemma minimal_state_eq_preserves_small_step_case:
"inp ⊢ (c1, state1, env1) →case_f2 (c1', state1', newW) ⟹
  isFreshChoiceInCasesGuards choiceName [c1] ⟹
  state_minimal_eq choiceName state1 state2 ⟹
  (∃state2' . inp ⊢ (c1, state2, env1) →case_f2 (c1', state2', newW) ∧
    state_minimal_eq choiceName state1' state2')"
proof (induction inp c1 state1 env1 c1' state1' newW
```

```
      arbitrary: state2 rule: small_step_reduce_case.induct[split_format(complete)])
  case (CaseGuardCompleteStep ins g s env stmts newS c)
  then show ?case by (metis isFreshChoiceInCasesGuards.simps(2)
       minimal_state_eq_preserves_small_step_guard small_step_reduce_case.CaseGuardCompleteStep)
next
  case (CaseGuardIncompleteStep ins g s env stmts innerG newS c)
  then show ?case by (metis isFreshChoiceInCasesGuards.simps(2)
       minimal_state_eq_preserves_small_step_guard small_step_reduce_case.CaseGuardIncompleteStep
       )
qed

lemma minimal_state_eq_preserves_small_step_cases_halted:
"computeCasesStep inp (c1, state1, env1) = None ⟹
  isFreshChoiceInCasesGuards choiceName c1 ⟹
  state_minimal_eq choiceName state1 state2 ⟹
  computeCasesStep inp (c1, state2, env1) = None"
  apply (induction inp c1 state1 env1 rule: computeCasesStep.induct[split_format(complete)])
   apply (auto split: option.split_asm option.split)
  using isFreshChoiceInCasesGuards.elims(2) apply blast
  by (metis (no_types, lifting) isFreshChoiceInCasesGuards.elims(2) isFreshChoiceInCasesGuards.
       simps(1) isFreshChoiceInCasesGuards.simps(2) list.distinct(1) list.inject
       minimal_state_eq_preserves_small_step_case_halted option.distinct(1) state_minimal_eq.
       elims(3))

lemma minimal_state_eq_preserves_small_step_cases:
"inp ⊢ (c1, state1, env1) →cases_f2 (c1', state1', newW) ⟹
  isFreshChoiceInCasesGuards choiceName c1 ⟹
  state_minimal_eq choiceName state1 state2 ⟹
  (∃state2' . inp ⊢ (c1, state2, env1) →cases_f2 (c1', state2', newW) ∧
    state_minimal_eq choiceName state1' state2')"
proof (induction inp c1 state1 env1 c1' state1' newW
      arbitrary: state2 rule: small_step_reduce_cases.induct[split_format(complete)])
  case (CasesHeadMatches ins c s env newC newS rest)
  then show ?case by (metis FaustusV2AST.FCase.exhaust isFreshChoiceInCasesGuards.simps(1)
       isFreshChoiceInCasesGuards.simps(2) minimal_state_eq_preserves_small_step_case
       small_step_reduce_cases.CasesHeadMatches)
next
  case (CasesTailMatches ins rest s env newC newS newW c)
  moreover have "isFreshChoiceInCasesGuards choiceName rest" using calculation by (cases c, auto)
  moreover obtain state2' where "ins ⊢ (rest, state2, env) →cases_f2 (newC, state2')" using
       calculation(3)[of state2] calculation(1,2,4,5,6) by auto
  moreover have "computeCaseStep ins (c, s, env) = None" by (meson calculation(2)
       computeCaseStepResImpliesReduceStep option.exhaust)
  moreover have "computeCaseStep ins (c, state2, env) = None" by (metis (no_types, lifting)
```

```
      calculation(4) calculation(5) calculation(8) isFreshChoiceInCasesGuards.elims(2)
      isFreshChoiceInCasesGuards.simps(1) isFreshChoiceInCasesGuards.simps(2) list.distinct(1)
      list.inject minimal_state_eq_preserves_small_step_case_halted)
  ultimately show ?case by (metis (no_types, lifting) computeCasesStep.simps(2)
      computeCasesStepResImpliesReduceStep option.simps(4)
      reduceCasesStepImpliesComputeCaseStepRes)
qed

lemma small_step_cases_preserves_split_cases_relation:
"inp ⊢ (c1, state1, env1) →cases_f2 (c1', state1', newW) ⟹
  isFreshChoiceInCasesGuards choiceName c1 ⟹
  state_minimal_eq choiceName state1 state2 ⟹
  (∃c2' state2' . inp ⊢ (map (λ(Case g c) ⇒ (Case g (split_cases role choiceName k c))) c1,
      state2, env1) →cases_f2 (c2', state2', newW) ∧
    state_minimal_eq choiceName state1' state2' ∧
    (c2' t (split_cases role choiceName k tc1) = split_cases role choiceName k (c1' t tc1)))"
proof (induction inp c1 state1 env1 c1' state1' newW
     arbitrary: state2 rule: small_step_reduce_cases.induct[split_format(complete)])
  case (CasesHeadMatches ins c s env newC newS newW rest)
  moreover have "isFreshChoiceInCasesGuards choiceName [c]" using calculation(2) by (cases c,
      auto)
  moreover obtain newC2 newS2 where "ins ⊢ ((λ(Case g c) ⇒ (Case g (split_cases role choiceName
      k c))) c, state2, env) →case_f2 (newC2, newS2, newW) ∧
    state_minimal_eq choiceName newS newS2 ∧
    (newC2 t (split_cases role choiceName k tc1) = split_cases role choiceName k (newC t tc1))"
    using small_step_case_preserves_split_cases_relation[of ins c s env newC newS newW choiceName
        state2 role k t tc1] calculation by auto
  ultimately show ?case by (metis (mono_tags, lifting) list.simps(9) small_step_reduce_cases.
      CasesHeadMatches)
next
  case (CasesTailMatches ins rest s env newC newS newW c)
  moreover have "computeCaseStep ins (c, s, env) = None" by (meson calculation(2)
      computeCaseStepResImpliesReduceStep option.exhaust)
  moreover have "isFreshChoiceInCasesGuards choiceName [(λ(Case g c) ⇒ (Case g (split_cases role
       choiceName k c))) c]" using calculation by (cases c, auto)
  moreover have "computeCaseStep ins ((λ(Case g c) ⇒ (Case g (split_cases role choiceName k c)))
       c, state2, env) = None" by (metis FaustusV2AST.FCase.exhaust calculation(4-7)
      isFreshChoiceInCasesGuards.simps(2)  split_cases_relation_preserves_halted_case)
  moreover have "∀res . ¬ ins ⊢ ((λ(Case g c) ⇒ (Case g (split_cases role choiceName k c))) c,
      state2, env) →case_f2 res" using calculation reduceCaseStepImpliesComputeCaseStepRes by
      force
  ultimately show ?case by (smt (verit, best) computeCasesStep.simps(2)
      computeCasesStepResImpliesReduceStep isFreshChoiceInCasesGuards.elims(2)
      isFreshChoiceInCasesImpliesIsFreshChoiceInCasesGuards list.distinct(1) list.inject list.
```

```
      simps(9) option.simps(4) reduceCasesStepImpliesComputeCaseStepRes)
qed

lemma small_step_action_preserved_inverse_split_cases_relation:
"ins ⊢ (a, s, env) →a_f2 (newS, newW) ⟹
  isFreshChoiceInAction choiceName a ⟹
  state_minimal_eq choiceName s1 s ⟹
  (∃newS1 . ins ⊢ (a, s1, env) →a_f2 (newS1, newW) ∧
    state_minimal_eq choiceName newS1 newS)"
proof (induction ins a s env newS newW
    arbitrary: s1 rule: small_step_reduce_action.induct[split_format(complete)])
  case (DepositPositiveActionStep env s fValue valRes fAccId fAccIdRes fParty fPartyRes token)
  moreover have "evalFValue env s1 fValue = Some valRes ∧
    evalFParty s1 fAccId = Some fAccIdRes ∧
    evalFParty s1 fParty = Some fPartyRes" using calculation by (metis
         fresh_choice_state_preserves_party_evaluation
         fresh_choice_state_preserves_value_evaluation(1) isFreshChoiceInAction.simps(3)
         state_minimal_eq.elims(1))
  moreover have "(IDeposit fAccIdRes fPartyRes token valRes) ⊢ (Deposit fAccId fParty token
       fValue, s1, env) →a_f2 (s1⦇FState.accounts := addMoneyToAccount fAccIdRes token valRes (
       FState.accounts s)⦈, ApplyNoWarning)" using calculation small_step_reduce_action.simps by
       auto
  ultimately show ?case by auto
next
  case (DepositNonPositiveActionStep env s fValue valRes fAccId fAccIdRes fParty fPartyRes token)
  moreover have "evalFValue env s1 fValue = Some valRes ∧
    evalFParty s1 fAccId = Some fAccIdRes ∧
    evalFParty s1 fParty = Some fPartyRes" using calculation by (metis
         fresh_choice_state_preserves_party_evaluation
         fresh_choice_state_preserves_value_evaluation(1) isFreshChoiceInAction.simps(3)
         state_minimal_eq.elims(1))
  moreover have "(IDeposit fAccIdRes fPartyRes token valRes) ⊢ (Deposit fAccId fParty token
       fValue, s1, env) →a_f2 (s1⦇FState.accounts := addMoneyToAccount fAccIdRes token valRes (
       FState.accounts s)⦈, ApplyNonPositiveDeposit fPartyRes fAccIdRes token valRes)" using
       calculation small_step_reduce_action.simps by auto
  ultimately show ?case by auto
next
  case (ChoiceActionStep choice bounds s fChoiceId fChoiceIdRes mChoiceId env)
  moreover have "evalFChoiceId s1 fChoiceId = Some fChoiceIdRes" using calculation(2,5)
       fresh_choice_state_preserves_choiceId_evaluation state_minimal_eq.simps by blast
  moreover have "(IChoice mChoiceId choice) ⊢ (Choice fChoiceId bounds, s1, env) →a_f2 (s1⦇FState
       .choices := MList.insert mChoiceId choice (FState.choices s1)⦈, ApplyNoWarning)" using
       small_step_reduce_action.ChoiceActionStep[of choice bounds s1 fChoiceId fChoiceIdRes
       mChoiceId env] calculation by simp
```

```
  ultimately show ?case by (smt (verit, ccfv_threshold) FState.select_convs(3) FState.surjective
        FState.update_convs(2) computeActionStep.simps(2) computeActionStepIsSmallStep
        minimal_state_eq_preserves_small_step_action option.inject state_minimal_eq_commutative)
next
  case (NotifyActionStep env s obs)
  moreover have "evalFObservation env s1 obs = Some True" by (metis NotifyActionStep.hyps
        calculation(2) calculation(3) fresh_choice_state_preserves_value_evaluation(2)
        isFreshChoiceInAction.simps(2))
  ultimately show ?case by (simp add: small_step_reduce_action.simps)
qed

lemma small_step_guard_preserved_inverse_split_cases_relation:
"ins ⊢ (g, s, env) →g_f2 (stmts, innerG, newS, newW) ⟹
  isFreshChoiceInGuard choiceName g ⟹
  state_minimal_eq choiceName s1 s ⟹
  (∃newCtx1 newS1 . ins ⊢ (g, s1, env) →g_f2 (stmts, innerG, newS1, newW) ∧
    state_minimal_eq choiceName newS1 newS)"
proof (induction ins g s env stmts innerG newS newW
   arbitrary: s1
   rule: small_step_reduce_guard.induct[split_format(complete)])
  case (ActionGuardStep g a ins s env newS newW)
  moreover obtain newS1 where "ins ⊢ (a, s1, env) →a_f2 (newS1, newW) ∧ state_minimal_eq
        choiceName newS1 newS" using calculation
        small_step_action_preserved_inverse_split_cases_relation by fastforce
  moreover have "ins ⊢ (g, s1, env) →g_f2 ([], None, newS1, newW)" using calculation
        small_step_reduce_guard.ActionGuardStep by auto
  ultimately show ?case by auto
next
  case (GuardThenGuardCompleteFirstGuardStep)
  then show ?case by (metis isFreshChoiceInGuard.simps(2) small_step_reduce_guard.
        GuardThenGuardCompleteFirstGuardStep)
next
  case (GuardThenGuardIncompleteGuard)
  then show ?case by (metis isFreshChoiceInGuard.simps(2) small_step_reduce_guard.
        GuardThenGuardIncompleteGuard)
next
  case (GuardStmtsGuardCompleteGuardStep)
  then show ?case by (metis isFreshChoiceInGuard.simps(3) small_step_reduce_guard.
        GuardStmtsGuardCompleteGuardStep)
next
  case (GuardStmtsGuardIncompleteGuard)
  then show ?case by (metis isFreshChoiceInGuard.simps(3) small_step_reduce_guard.
        GuardStmtsGuardIncompleteGuard)
next
```

```
  case (DisjointGuardMatchesG1)
  then show ?case by (metis isFreshChoiceInGuard.simps(4) small_step_reduce_guard.
       DisjointGuardMatchesG1)
next
  case (DisjointGuardMatchesG2)
  then show ?case by (metis isFreshChoiceInGuard.simps(4)
       minimal_state_eq_preserves_small_step_guard_halted small_step_reduce_guard.
       DisjointGuardMatchesG2 state_minimal_eq_commutative)
next
  case (InterleavedGuardMatchesG1Complete)
  then show ?case by (metis isFreshChoiceInGuard.simps(5) small_step_reduce_guard.
       InterleavedGuardMatchesG1Complete)
next
  case (InterleavedGuardMatchesG1Incomplete)
  then show ?case by (metis isFreshChoiceInGuard.simps(5) small_step_reduce_guard.
       InterleavedGuardMatchesG1Incomplete)
next
  case (InterleavedGuardMatchesG2Complete)
  then show ?case by (metis (no_types, lifting) isFreshChoiceInGuard.simps(5)
       minimal_state_eq_preserves_small_step_guard_halted small_step_reduce_guard.
       InterleavedGuardMatchesG2Complete state_minimal_eq_commutative)
next
  case (InterleavedGuardMatchesG2Incomplete)
  then show ?case by (metis (no_types, lifting) isFreshChoiceInGuard.simps(5)
       minimal_state_eq_preserves_small_step_guard_halted small_step_reduce_guard.
       InterleavedGuardMatchesG2Incomplete state_minimal_eq_commutative)
qed

lemma small_step_case_preserved_inverse_split_cases_relation:
"inp ⊢ ((λ(Case g c) ⇒ (Case g (split_cases role choiceName k c))) c1, state2, env1) →case_f2 (
     c2', state2', newW) ⟹
  isFreshChoiceInCasesGuards choiceName [c1] ⟹
  state_minimal_eq choiceName state1 state2 ⟹
  (∃c1' ctx1' state1' . inp ⊢ (c1, state1, env1) →case_f2 (c1', state1', newW) ∧
    state_minimal_eq choiceName state1' state2' ∧
    (c2' t (split_cases role choiceName k tc1) = split_cases role choiceName k (c1' t tc1)))"
proof (induction inp "(λ(Case g c) ⇒ (Case g (split_cases role choiceName k c))) c1" state2 env1
      c2' state2' newW
      arbitrary: c1 state1  rule: small_step_reduce_case.induct[split_format(complete)])
  case (CaseGuardCompleteStep ins g s env stmts newS newW c)
  moreover obtain innerCont1 where "c1 = Case g innerCont1 ∧ split_cases role choiceName k
       innerCont1 = c" using calculation by (cases c1, auto)
  moreover obtain newS1 where "ins ⊢ (g, state1, env) →g_f2 (stmts, None, newS1, newW) ∧
    state_minimal_eq choiceName newS1 newS"  using
```

```
        small_step_guard_preserved_inverse_split_cases_relation calculation apply auto
    by (smt (verit, best))
  moreover have "ins ⊢ (c1, state1, env) →case_f2 ((λt tcont . prependStatements stmts innerCont1
      ), newS1, newW)" using calculation small_step_reduce_case.CaseGuardCompleteStep by
      presburger
  moreover have "split_cases role choiceName k ((λt tcont . prependStatements stmts innerCont1) t
       tc1) = prependStatements stmts c" using calculation
      split_cases_distributes_past_prepend_statements by auto
  ultimately show ?case by metis
next
  case (CaseGuardIncompleteStep ins g s env stmts innerG newS newW c)
  moreover obtain innerCont1 where "c1 = Case g innerCont1 ∧ split_cases role choiceName k
      innerCont1 = c" using calculation by (cases c1, auto)
  moreover obtain newS1 where "ins ⊢ (g, state1, env) →g_f2 (stmts, Some innerG, newS1, newW) ∧
    state_minimal_eq choiceName newS1 newS"  using
        small_step_guard_preserved_inverse_split_cases_relation calculation apply auto
    by (smt (verit, best))
  moreover have "ins ⊢ (c1, state1, env) →case_f2 ((λt tcont . prependStatements stmts (
      FaustusV2AST.FContract.When [Case innerG innerCont1] t tcont)), newS1, newW)" using
      calculation small_step_reduce_case.CaseGuardIncompleteStep by auto
  moreover have "split_cases role choiceName k (When [FaustusV2AST.FCase.Case innerG innerCont1]
      t tc1) = (When [FaustusV2AST.FCase.Case innerG (split_cases role choiceName k innerCont1)]
       t (split_cases role choiceName k tc1))" by (cases "k = 0", auto)
  moreover have "split_cases role choiceName k ((λt tcont . prependStatements stmts (When [Case
      innerG innerCont1] t tcont)) t tc1) = (prependStatements stmts (When [FaustusV2AST.FCase.
      Case innerG c] t (split_cases role choiceName k tc1)))" using calculation apply auto
    using split_cases_distributes_past_prepend_statements[of stmts role choiceName k "(
        FaustusV2AST.FContract.When [FaustusV2AST.FCase.Case innerG innerCont1] t tc1)"] by (
        metis calculation(8))
  ultimately show ?case by metis
qed

lemma small_step_cases_preserved_inverse_split_cases_relation:
"inp ⊢ (map (λ(Case g c) ⇒ (Case g (split_cases role choiceName k c))) c1, state2, env1) →cases
     _f2 (c2', state2', newW) ⟹
  isFreshChoiceInCasesGuards choiceName c1 ⟹
  state_minimal_eq choiceName state1 state2 ⟹
  (∃c1' state1' . inp ⊢ (c1, state1, env1) →cases_f2 (c1', state1', newW) ∧
    state_minimal_eq choiceName state1' state2' ∧
    (c2' t (split_cases role choiceName k tc1) = split_cases role choiceName k (c1' t tc1)))"
proof (induction inp "map (λ(Case g c) ⇒ (Case g (split_cases role choiceName k c))) c1" state2
     env1 c2' state2' newW
      arbitrary: c1 state1  rule: small_step_reduce_cases.induct[split_format(complete)])
  case (CasesHeadMatches ins c s env newC newS newW rest)
```

```
  moreover obtain c1Head c1Rest where "c1 = c1Head#c1Rest ∧
    isFreshChoiceInCasesGuards choiceName [c1Head] ∧
    (λ(Case g c) ⇒ (Case g (split_cases role choiceName k c))) c1Head = c" using calculation
         apply (cases c1, auto)
    by (metis FaustusV2AST.FCase.exhaust isFreshChoiceInCasesGuards.simps(1)
         isFreshChoiceInCasesGuards.simps(2))
  moreover obtain newC1 newS1 where "ins ⊢ (c1Head, state1, env) →case_f2 (newC1, newS1, newW) ∧
    state_minimal_eq choiceName newS1 newS ∧
    split_cases role choiceName k (newC1 t tc1) = newC t (split_cases role choiceName k tc1)"
         using small_step_case_preserved_inverse_split_cases_relation[of ins role choiceName k
         c1Head s env newC newS newW state1 t tc1] calculation apply (cases c1Head)
    by (auto, blast)
  ultimately show ?case by (metis small_step_reduce_cases.CasesHeadMatches)
next
  case (CasesTailMatches ins rest s env newC newS newW c)
  moreover obtain c1Head c1Rest where "c1 = c1Head#c1Rest ∧
    isFreshChoiceInCasesGuards choiceName [c1Head] ∧
    isFreshChoiceInCasesGuards choiceName c1Rest ∧
    (λ(Case g c) ⇒ (Case g (split_cases role choiceName k c))) c1Head = c ∧
    map (λ(Case g c) ⇒ (Case g (split_cases role choiceName k c))) c1Rest = rest" using
         calculation apply (cases c1, auto)
    by (metis FaustusV2AST.FCase.exhaust isFreshChoiceInCasesGuards.simps(1)
         isFreshChoiceInCasesGuards.simps(2))
  moreover obtain newC1 newS1 where "ins ⊢ (c1Rest, state1, env) →cases_f2 (newC1, newS1, newW) ∧
       state_minimal_eq choiceName newS1 newS ∧
       newC t (split_cases role choiceName k tc1) = split_cases role choiceName k (newC1 t tc1)"
            using calculation by blast
  moreover have c1CaseStepImpC2CaseStep: "∀ res . ins ⊢ (c1Head, state1, env) →case_f2 res ⟶ (∃
       res2 . ins ⊢ (c, s, env) →case_f2 res2)" using
       small_step_case_preserves_split_cases_relation[of ins c1Head state1 env] calculation apply
        auto
    by meson
  ultimately show ?case by (metis (no_types, opaque_lifting) small_step_reduce_cases.
       CasesTailMatches)
qed

lemma small_step_cases_split_list_matches_one_side:
"inp ⊢ (c, state1, env1) →cases_f2 (c', state1', newW) ⟹
  c = c1@c2 ⟹
  inp ⊢ (c1, state1, env1) →cases_f2 (c', state1', newW) ∨
  (inp ⊢ (c2, state1, env1) →cases_f2 (c', state1', newW) ∧ ¬ (∀c' state1'. inp ⊢ (c1, state1,
       env1) →cases_f2 (c', state1', newW)))"
proof (induction inp c state1 env1 c' state1' newW arbitrary: c1 c2 rule: small_step_reduce_cases
     .induct[split_format(complete)])
```

```
  case (CasesHeadMatches ins c s env newC newS rest)
  then show ?case by (induction c1, auto simp add: small_step_reduce_cases.CasesHeadMatches)
next
  case (CasesTailMatches ins rest s env newC newS c)
  then show ?case apply auto
     apply (induction c1, auto simp add: small_step_reduce_cases.CasesTailMatches)
    by (metis CasesTailMatches.hyps(2) small_step_reduce_cases.CasesTailMatches)
qed

lemma small_step_action_fresh_choice_state_eq:
"(IChoice (ChoiceId choiceName (Party.Role role)) 0) ⊢ (Choice (FChoiceId choiceName (Role role))
      [(0, 0)], state, env) →a_f2 (state⦇ choices := MList.insert (ChoiceId choiceName (Party.
     Role role)) 0 (choices state) ⦈, ApplyNoWarning) ∧
  state_minimal_eq choiceName state (state⦇ choices := MList.insert (ChoiceId choiceName (Party.
       Role role)) 0 (choices state) ⦈)"
proof -
  have "inBounds 0 [(0,0)]" by auto
  moreover have "evalFChoiceId state (FChoiceId choiceName (Role role)) = Some (ChoiceId
       choiceName (Party.Role role))" by auto
  moreover have "state_minimal_eq choiceName state (state⦇ choices := MList.insert (ChoiceId
       choiceName (Party.Role role)) 0 (choices state) ⦈)" by (auto simp add:
       insert_lookup_different)
  ultimately show ?thesis by (auto simp add: ChoiceActionStep)
qed

lemma react_choice_maintains_split_cases:
"isFreshChoiceInCasesGuards choiceName cases ⟹
  (IChoice (ChoiceId choiceName (Party.Role role)) 0) ⊢ (cases @ [Case (ActionGuard (Choice (
       FChoiceId choiceName (Role role)) [(0, 0)])) innerC], state, env) →cases_f2 (λinnerT
       innerTc . innerC, state⦇ choices := MList.insert (ChoiceId choiceName (Party.Role role)) 0
        (choices state) ⦈, ApplyNoWarning)"
proof (induction cases)
  case Nil
  moreover have "(IChoice (ChoiceId choiceName (Party.Role role)) 0) ⊢ (Choice (FChoiceId
       choiceName (Role role)) [(0, 0)], state, env) →a_f2 (state⦇ choices := MList.insert (
       ChoiceId choiceName (Party.Role role)) 0 (choices state) ⦈, ApplyNoWarning)" by (simp add:
        small_step_action_fresh_choice_state_eq)
  moreover have "(IChoice (ChoiceId choiceName (Party.Role role)) 0) ⊢ (ActionGuard (Choice (
       FChoiceId choiceName (Role role)) [(0, 0)]), state, env) →g_f2 ([], None, state⦇ choices
       := MList.insert (ChoiceId choiceName (Party.Role role)) 0 (choices state) ⦈,
       ApplyNoWarning)" using ActionGuardStep calculation(2) by blast
  moreover have "(IChoice (ChoiceId choiceName (Party.Role role)) 0) ⊢ (Case (ActionGuard (Choice
        (FChoiceId choiceName (Role role)) [(0, 0)])) innerC, state, env) →case_f2 (λinnerT
       innerTc . (prependStatements [] innerC), state⦇ choices := MList.insert (ChoiceId
```

```
      choiceName (Party.Role role)) 0 (choices state) ⦈, ApplyNoWarning)" using
      CaseGuardCompleteStep calculation(3) by presburger
  moreover have "prependStatements [] innerC = innerC" by auto
  moreover have "[] @ [Case (ActionGuard (Choice (FChoiceId choiceName (Role role)) [(0, 0)]))
      innerC] = [Case (ActionGuard (Choice (FChoiceId choiceName (Role role)) [(0, 0)])) innerC
      ]" by auto
  ultimately show ?case using CasesHeadMatches by force
next
  case (Cons a cases)
  moreover have "finalFCases (IChoice (ChoiceId choiceName (Party.Role role)) 0) (a#cases, state,
       env)" by (simp add: Cons.prems freshChoiceNoMatch_cases_step)
  moreover have "(IChoice (ChoiceId choiceName (Party.Role role)) 0) ⊢ (Choice (FChoiceId
      choiceName (Role role)) [(0, 0)], state, env) →a_f2 (state⦇ choices := MList.insert (
      ChoiceId choiceName (Party.Role role)) 0 (choices state) ⦈, ApplyNoWarning)" by (simp add:
       small_step_action_fresh_choice_state_eq)
  moreover have "(IChoice (ChoiceId choiceName (Party.Role role)) 0) ⊢ (ActionGuard (Choice (
      FChoiceId choiceName (Role role)) [(0, 0)]), state, env) →g_f2 ([], None, state⦇ choices
      := MList.insert (ChoiceId choiceName (Party.Role role)) 0 (choices state) ⦈,
      ApplyNoWarning)" using ActionGuardStep calculation by blast
  moreover have "(IChoice (ChoiceId choiceName (Party.Role role)) 0) ⊢ (Case (ActionGuard (Choice
       (FChoiceId choiceName (Role role)) [(0, 0)])) innerC, state, env) →case_f2 (λinnerT
      innerTc . (prependStatements [] innerC), state⦇ choices := MList.insert (ChoiceId
      choiceName (Party.Role role)) 0 (choices state) ⦈, ApplyNoWarning)" using
      CaseGuardCompleteStep calculation by presburger
  moreover have "(IChoice (ChoiceId choiceName (Party.Role role)) 0) ⊢ ([Case (ActionGuard (
      Choice (FChoiceId choiceName (Role role)) [(0, 0)])) innerC], state, env) →cases_f2 (λ
      innerT innerTc . (prependStatements [] innerC), state⦇ choices := MList.insert (ChoiceId
      choiceName (Party.Role role)) 0 (choices state) ⦈, ApplyNoWarning)" using CasesHeadMatches
       calculation(6) by auto
  ultimately show ?case by (metis (no_types, lifting) casesStepAppendFinalCases
      freshChoiceNoMatch_cases_step_aux isFreshChoiceInCasesGuards.elims(2) list.distinct(1)
      list.inject small_step_cases_split_list_matches_one_side)
qed

lemma react_choice_map_split_maintains_split_cases:
"isFreshChoiceInCasesGuards choiceName cases ⟹
  (IChoice (ChoiceId choiceName (Party.Role role)) 0) ⊢ (take l (map (λCase g c ⇒ Case g (
      split_cases role choiceName k c)) cases) @ [Case (ActionGuard (Choice (FChoiceId
      choiceName (Role role)) [(0, 0)])) innerC], state, env) →cases_f2 (λinnerT innerTc .
      innerC, state⦇ choices := MList.insert (ChoiceId choiceName (Party.Role role)) 0 (choices
      state) ⦈, ApplyNoWarning)"
proof (induction cases arbitrary: l)
  case Nil
  moreover have "(IChoice (ChoiceId choiceName (Party.Role role)) 0) ⊢ (Choice (FChoiceId
```

```
      choiceName (Role role)) [(0, 0)], state, env) →a_f2 (state⦇ choices := MList.insert (
      ChoiceId choiceName (Party.Role role)) 0 (choices state) ⦈, ApplyNoWarning)" by (simp add:
       small_step_action_fresh_choice_state_eq)
  moreover have "(IChoice (ChoiceId choiceName (Party.Role role)) 0) ⊢ (ActionGuard (Choice (
      FChoiceId choiceName (Role role)) [(0, 0)]), state, env) →g_f2 ([], None, state⦇ choices
      := MList.insert (ChoiceId choiceName (Party.Role role)) 0 (choices state) ⦈,
      ApplyNoWarning)" using ActionGuardStep calculation(2) by blast
  moreover have "(IChoice (ChoiceId choiceName (Party.Role role)) 0) ⊢ (Case (ActionGuard (Choice
       (FChoiceId choiceName (Role role)) [(0, 0)])) innerC, state, env) →case_f2 (λinnerT
      innerTc . (prependStatements [] innerC), state⦇ choices := MList.insert (ChoiceId
      choiceName (Party.Role role)) 0 (choices state) ⦈, ApplyNoWarning)" using
      CaseGuardCompleteStep calculation(3) by presburger
  moreover have "prependStatements [] innerC = innerC" by auto
  moreover have "[] @ [Case (ActionGuard (Choice (FChoiceId choiceName (Role role)) [(0, 0)]))
      innerC] = [Case (ActionGuard (Choice (FChoiceId choiceName (Role role)) [(0, 0)])) innerC
      ]" by auto
  ultimately show ?case using CasesHeadMatches by force
next
  case (Cons a cases)
  moreover have "finalFCases (IChoice (ChoiceId choiceName (Party.Role role)) 0) (a#cases, state,
       env)" by (simp add: Cons.prems freshChoiceNoMatch_cases_step)
  moreover have "(IChoice (ChoiceId choiceName (Party.Role role)) 0) ⊢ (Choice (FChoiceId
      choiceName (Role role)) [(0, 0)], state, env) →a_f2 (state⦇ choices := MList.insert (
      ChoiceId choiceName (Party.Role role)) 0 (choices state) ⦈, ApplyNoWarning)" by (simp add:
       small_step_action_fresh_choice_state_eq)
  moreover have "(IChoice (ChoiceId choiceName (Party.Role role)) 0) ⊢ (ActionGuard (Choice (
      FChoiceId choiceName (Role role)) [(0, 0)]), state, env) →g_f2 ([], None, state⦇ choices
      := MList.insert (ChoiceId choiceName (Party.Role role)) 0 (choices state) ⦈,
      ApplyNoWarning)" using ActionGuardStep calculation by blast
  moreover have "(IChoice (ChoiceId choiceName (Party.Role role)) 0) ⊢ (Case (ActionGuard (Choice
       (FChoiceId choiceName (Role role)) [(0, 0)])) innerC, state, env) →case_f2 (λinnerT
      innerTc . (prependStatements [] innerC), state⦇ choices := MList.insert (ChoiceId
      choiceName (Party.Role role)) 0 (choices state) ⦈, ApplyNoWarning)" using
      CaseGuardCompleteStep calculation by presburger
  moreover have "(IChoice (ChoiceId choiceName (Party.Role role)) 0) ⊢ ([Case (ActionGuard (
      Choice (FChoiceId choiceName (Role role)) [(0, 0)])) innerC], state, env) →cases_f2 (λ
      innerT innerTc . (prependStatements [] innerC), state⦇ choices := MList.insert (ChoiceId
      choiceName (Party.Role role)) 0 (choices state) ⦈, ApplyNoWarning)" using CasesHeadMatches
       calculation(6) by auto
  moreover have "(IChoice (ChoiceId choiceName (Party.Role role)) 0) ⊢ (take (l-1) (map (λa. case
       a of Case g c ⇒ Case g (split_cases role choiceName k c)) cases) @ [FaustusV2AST.FCase.
      Case (ActionGuard (FAction.Choice (FChoiceId choiceName (FParty.Role role)) [(0, 0)]))
      innerC], state, env) →cases_f2 (λinnerT innerTc. innerC, state ⦇FState.choices := MList.
      insert (ChoiceId choiceName (Party.Role role)) 0 (FState.choices state)⦈, ApplyNoWarning)"
```

```
        using Cons.IH Cons.prems isFreshChoiceInCasesGuards.elims by (smt (verit, ccfv_SIG)
       isFreshChoiceInCasesImpliesIsFreshChoiceInCasesGuards list.inject neq_Nil_conv)
  moreover have "l > 0 ⟶ take l (map (λa. case a of Case g c ⇒ Case g (split_cases role
       choiceName k c)) (a # cases)) = ((λa. case a of Case g c ⇒ Case g (split_cases role
       choiceName k c)) a) # (take (l-1) (map (λa. case a of Case g c ⇒ Case g (split_cases role
        choiceName k c)) (cases)))" by (cases a, auto simp add: take_Cons')
  moreover have "l ≤ 0 ⟶ take l (map (λa. case a of Case g c ⇒ Case g (split_cases role
       choiceName k c)) (a # cases)) = []" by auto
  moreover have "∀ res . ¬ (IChoice (ChoiceId choiceName (Party.Role role)) 0) ⊢ ((λa. case a of
       Case g c ⇒ Case g (split_cases role choiceName k c)) a, state, env) →case_f2 res" using
       calculation apply (cases a, auto)
    by (metis (no_types, lifting) computeCaseStep.simps
         computeGuardStepHaltedImpliesSmallStepHalted freshChoiceNoMatch_guard_step option.
         distinct(1) option.simps(4) reduceCaseStepImpliesComputeCaseStepRes)+
  ultimately show ?case using CasesTailMatches by auto
qed

lemma fresh_choice_no_match_small_step_receive:
"isFreshChoiceInProcess choiceName p1 ⟹
  (\<nexists>p1' . p1 →(Some (((IChoice ((ChoiceId choiceName (SemanticsTypes.Role role))) 0))),
        lower, upper) p1')"
  by (meson basic_receive_input_not_fresh_choice)

lemma fresh_choice_no_match_full_semantics:
"(A ⊢ p1 −λ(Some (((IChoice ((ChoiceId choiceName (SemanticsTypes.Role role))) 0))), lower, upper
     )→ p1') ⟹
  isFreshChoiceInProcess choiceName p1 ⟹
  False"
  apply (induction A p1 "λ(Some (((IChoice ((ChoiceId choiceName (SemanticsTypes.Role role))) 0))
       ), lower, upper)" p1' rule: full_semantics.induct)
  by (meson fresh_choice_no_match_small_step_receive basic_receive_preserves_fresh_choice)+

lemma fresh_choice_no_match_full_nd_semantics:
"(A ⊢ p1 −−λ(Some (((IChoice ((ChoiceId choiceName (SemanticsTypes.Role role))) 0))), lower,
     upper)→ p1') ⟹
  isFreshChoiceInProcess choiceName p1 ⟹
  False"
  apply (induction A p1 "λ(Some (((IChoice ((ChoiceId choiceName (SemanticsTypes.Role role))) 0))
       ), lower, upper)" p1' rule: full_nd_semantics.induct)
  by (meson fresh_choice_no_match_small_step_receive basic_receive_preserves_fresh_choice)+

lemma receive_semantics_strongly_maintains_split_cases_NoChoiceLambdaReceive:
"split_cases role choiceName k c1 = c2 ⟹
  split_cases_context role choiceName k ctx1 = ctx2 ⟹
```

```
  state_minimal_eq choiceName state1 state2 ⟹
  env1 = env2 ⟹
  warnings1 = warnings2 ⟹
  payments1 = payments2 ⟹
  isFreshChoiceInContract choiceName c1 ⟹
  isFreshChoiceInContext choiceName ctx1 ⟹
  (c1, ctx1, state1, env1, warnings1, payments1) →(Some inp, lower, upper) (c1', ctx1', state1',
        env1', warnings1', payments1') ⟹
  split_cases_relation role choiceName k (c1, ctx1, state1, env1, warnings1, payments1) (c2, ctx2
       , state2, env2, warnings2, payments2) ⟹
  ∃c2' ctx2' state2' env2' warnings2' payments2'.
    ({(IChoice (ChoiceId choiceName (Party.Role role)) 0)} ⊢ (c2, ctx2, state2, env2, warnings2,
          payments2) −λ(Some inp, lower, upper)→ (c2', ctx2', state2', env2', warnings2',
          payments2')) ∧
    split_cases_relation role choiceName k (c1', ctx1', state1', env1', warnings1', payments1') (
          c2', ctx2', state2', env2', warnings2', payments2')"
proof (induction role choiceName k c1 arbitrary: c2 ctx1 ctx2 state1 state2 env1 env2 warnings1
     warnings2 payments1 payments2 inp lower upper c1' ctx1' state1' env1' warnings1' payments1'
     rule: split_cases.induct)
  case (1 role choiceName k cases t cont)
  moreover obtain cases2 tCont2 where decompile_c2: "c2 = When cases2 t tCont2" using 1(4) by (
       cases "1 < k ∧ k < length cases", auto)
  moreover have small_k_cases2_is_take_k: "1 < k ∧ k < length cases ⟶ cases2 = (take k (map (λ(
       Case g c) ⇒ (Case g (split_cases role choiceName k c))) cases)) @ [Case (ActionGuard (
       Choice (FChoiceId choiceName (Role role)) [(0, 0)])) (split_cases role choiceName k (When
       (drop k cases) t cont))]" using 1(4) decompile_c2 apply (auto)
    by metis
  moreover have large_k_cases2_is_map: "¬(1 < k ∧ k < length cases) ⟶ cases2 = (map (λ(Case g c
       ) ⇒ (Case g (split_cases role choiceName k c))) cases)" using 1(4) decompile_c2 by auto
  moreover obtain newTimeState1 newTimeState2 newTimeEnv1 newTimeEnv2 where new_time_states: "
       fixInterval (lower, upper) state1 = IntervalTrimmed newTimeEnv1 newTimeState1 ∧
    fixInterval (lower, upper) state2 = IntervalTrimmed newTimeEnv2 newTimeState2 ∧
    state_minimal_eq choiceName newTimeState1 newTimeState2 ∧
    newTimeEnv1 = newTimeEnv2" using 1(6,7,12) by (cases "upper < lower"; cases "upper < FState.
          minTime state1"; cases "lower < FState.minTime state1",  auto)
  moreover obtain realLower realUpper where real_lower_upper_obtained: "timeInterval newTimeEnv1
       = (realLower, realUpper) ∧ realLower < t ∧ realUpper < t" using 1(12) new_time_states
       ReceiveWhenSomeElim by auto
  moreover have is_fresh_in_map_split: "isFreshChoiceInCasesGuards choiceName (map (λ(Case g c)
       ⇒ (Case g (split_cases role choiceName k c))) cases)" using 1(10) by (induction cases,
       auto split: FCase.split)
  moreover have is_fresh_in_take_k_map_split: "isFreshChoiceInCasesGuards choiceName (take k (map
        (λ(Case g c) ⇒ (Case g (split_cases role choiceName k c))) cases))" using
       is_fresh_in_map_split apply (induction cases arbitrary: k, auto split: FCase.split)
```

```
  by (metis append_take_drop_id isFreshChoiceInCasesGuards.simps(2)
        isFreshChoiceInCasesGuardsPreservedInSlice)
moreover have "¬(∀c2' . (IChoice (ChoiceId choiceName (Party.Role role)) 0) ⊢ ((map (λ(Case g c
     ) ⇒ (Case g (split_cases role choiceName k c))) cases), newTimeState2, newTimeEnv2) →cas
     es_f2 c2')" using is_fresh_in_map_split freshChoiceNoMatch_cases_step_aux by blast
moreover have "¬(∀c2' . (IChoice (ChoiceId choiceName (Party.Role role)) 0) ⊢ (take k (map (λ(
     Case g c) ⇒ (Case g (split_cases role choiceName k c))) cases), newTimeState2,
     newTimeEnv2) →cases_f2 c2')" using is_fresh_in_take_k_map_split
     freshChoiceNoMatch_cases_step_aux by blast
moreover have choice_step_take_k_map_cases:"(IChoice (ChoiceId choiceName (Party.Role role)) 0)
      ⊢ ((take k (map (λ(Case g c) ⇒ (Case g (split_cases role choiceName k c))) cases)) @ [
     Case (ActionGuard (Choice (FChoiceId choiceName (Role role)) [(0, 0)])) (split_cases role
     choiceName k (When (drop k cases) t cont))], newTimeState2, newTimeEnv2) →cases_f2 (λ
     passedT tCont. (split_cases role choiceName k (When (drop k cases) t cont)), newTimeState2
     ⦇ choices := MList.insert (ChoiceId choiceName (Party.Role role)) 0 (choices newTimeState2
     ) ⦈, ApplyNoWarning)"
  using is_fresh_in_take_k_map_split react_choice_maintains_split_cases by presburger
moreover have small_k_cases_step_choice:"1 < k ∧ k < length cases ⟶ (IChoice (ChoiceId
     choiceName (Party.Role role)) 0) ⊢ (cases2, newTimeState2, newTimeEnv2) →cases_f2 (λ
     passedT passedTCont. (split_cases role choiceName k (When (drop k cases) t cont)),
     newTimeState2⦇ choices := MList.insert (ChoiceId choiceName (Party.Role role)) 0 (choices
     newTimeState2) ⦈, ApplyNoWarning)"
  using choice_step_take_k_map_cases small_k_cases2_is_take_k by auto
moreover obtain newC1' newW where cases_step: "inp ⊢ (cases, newTimeState1, newTimeEnv1) →case
     s_f2 (newC1', state1', newW) ∧ newC1' t cont = c1' ∧ warnings1' = warnings1 @
     convertApplyWarning newW" using 1(12) new_time_states by (auto elim: ReceiveWhenSomeElim)
moreover have cases_step_constants: "ctx1 = ctx1' ∧ newTimeEnv1 = env1' ∧ payments1 = payments1
     '" using cases_step new_time_states calculation(12) by (auto simp add:)
moreover obtain newC2' newTimeState2' where no_split_cases2_step: "inp ⊢ ((map (λ(Case g c) ⇒
     (Case g (split_cases role choiceName k c))) cases), newTimeState2, newTimeEnv2) →cases_f2
     (newC2', newTimeState2', newW) ∧ (∀t tCont. newC2' t (split_cases role choiceName k tCont)
      = split_cases role choiceName k (newC1' t tCont)) ∧ state_minimal_eq choiceName state1'
     newTimeState2'"
  using cases_step small_step_cases_preserves_split_cases_relation[of inp cases newTimeState1
        newTimeEnv1 newC1' state1' newW choiceName newTimeState2 role k] new_time_states 1(4-)
        isFreshChoiceInCasesImpliesIsFreshChoiceInCasesGuards apply auto
  by (metis deterministicCasesSemantics prod.inject)
moreover have no_split_lemma_proved: "split_cases_relation role choiceName k (c1', ctx1',
     state1', env1', warnings1', payments1') (newC2' t (split_cases role choiceName k cont),
     split_cases_context role choiceName k ctx1', newTimeState2', env1', warnings1', payments1
     ')" using no_split_cases2_step cases_step_constants 1(10,11,12) cases_step
     freshChoicePreserved_cases_step by auto
moreover have drop_in_split_cases_relation: "split_cases_relation role choiceName k (When (drop
      k cases) t cont, ctx1, newTimeState1, newTimeEnv1, warnings1, payments1) (split_cases
```

```
      role choiceName k (When (drop k cases) t cont), ctx2, newTimeState2⦇ choices := MList.
      insert (ChoiceId choiceName (Party.Role role)) 0 (choices newTimeState2) ⦈, newTimeEnv2,
      warnings2, payments2)" using 1(4-) new_time_states apply auto
  using 1(4-) apply (metis append_take_drop_id isFreshChoiceInCasesPreservedInSlice)
  by (metis ChoiceId.inject insert_lookup_different)
moreover have is_fresh_choice_in_take_k_cases: "isFreshChoiceInCases choiceName (take k cases)"
       using 1(10) by (metis append_take_drop_id isFreshChoiceInCasesPreservedInSlice
      isFreshChoiceInContract.simps(4))
moreover obtain takeKNewC2' takeKNewTimeState2' where take_k_cases_step_imp_step_in_relation: "
      inp ⊢ (take k cases, newTimeState1, newTimeEnv1) →cases_f2 (newC1', state1', newW) ⟶ inp
       ⊢ (take k (map (λ(Case g c) ⇒ (Case g (split_cases role choiceName k c))) cases),
      newTimeState2, newTimeEnv2) →cases_f2 (takeKNewC2', takeKNewTimeState2', newW) ∧ (∀t tCont
      . takeKNewC2' t (split_cases role choiceName k tCont) = split_cases role choiceName k (
      newC1' t tCont)) ∧ state_minimal_eq choiceName state1' takeKNewTimeState2'"
  using cases_step small_step_cases_preserves_split_cases_relation[of inp "take k cases"
        newTimeState1 newTimeEnv1 newC1' state1' newW choiceName newTimeState2 role k]
        new_time_states is_fresh_choice_in_take_k_cases 1(4-)
        isFreshChoiceInCasesImpliesIsFreshChoiceInCasesGuards take_map[of k "(λ(Case g c) ⇒ (
        Case g (split_cases role choiceName k c)))" cases] by (metis (no_types, lifting)
        Pair_inject deterministicCasesSemantics)
moreover have no_match_take_k_match_drop_k: "finalFCases inp (take k cases, newTimeState1,
      newTimeEnv1) ⟶ inp ⊢ (drop k cases, newTimeState1, newTimeEnv1) →cases_f2 (newC1',
      state1', newW)" by (metis append_take_drop_id cases_step finalFCases_def
      small_step_cases_split_list_matches_one_side)
moreover have finalFCases_cases2: "finalFCases inp (take k cases, newTimeState1, newTimeEnv1)
      ⟶ finalFCases inp (take k (map (λ(Case g c) ⇒ (Case g (split_cases role choiceName k c)
      )) cases), newTimeState2, newTimeEnv2)" using
      small_step_cases_preserved_inverse_split_cases_relation[of inp role choiceName k "take k
      cases" newTimeState2 newTimeEnv2 _ _ _ newTimeState1 t cont] new_time_states
      is_fresh_choice_in_take_k_cases 1(4-)
      isFreshChoiceInCasesImpliesIsFreshChoiceInCasesGuards take_map[of k "(λ(Case g c) ⇒ (Case
       g (split_cases role choiceName k c)))" cases] finalFCases_def apply auto
  by presburger
moreover have fixFInterval_fixed_point: "fixInterval (lower, upper) newTimeState1 =
      IntervalTrimmed newTimeEnv1 newTimeState1" using new_time_states apply (cases "upper <
      lower",  meson)
   apply auto
  by (cases "upper < minTime state1"; cases "lower < minTime state1", auto)
moreover have final_take_k_implies_drop_k_transition_to_final: "finalFCases inp (take k cases,
      newTimeState1, newTimeEnv1) ⟶ (When (drop k cases) t cont, ctx1, newTimeState1,
      newTimeEnv1, warnings1, payments1) →(Some inp, lower, upper) (c1', ctx1, state1',
      newTimeEnv1, warnings1 @ convertApplyWarning newW, payments1)" by (metis LambdaReceive
      real_lower_upper_obtained fixFInterval_fixed_point cases_step no_match_take_k_match_drop_k
      )
```

```
moreover have fixFInterval_fixed_point2: "fixInterval (lower, upper) (newTimeState2⦇ choices :=
      MList.insert (ChoiceId choiceName (Party.Role role)) 0 (choices newTimeState2) ⦈) =
     IntervalTrimmed newTimeEnv2 (newTimeState2⦇ choices := MList.insert (ChoiceId choiceName (
     Party.Role role)) 0 (choices newTimeState2) ⦈)" using new_time_states apply auto
  apply (cases "upper < lower"; cases "upper < minTime state1"; cases "lower < minTime state1";
        cases "upper < minTime state2"; cases "lower < minTime state2", auto)
  by (metis (no_types, lifting) FState.surjective FState.update_convs(2) FState.update_convs(5)
       )
moreover obtain newCases newTCont where newCases_newTCont_obtained: "newCases = map (λ(Case g c
     ) ⇒ (Case g (split_cases role choiceName k c))) cases ∧ newTCont = split_cases role
     choiceName k cont" by auto
moreover obtain dropState2' where dropState2'_obtained: "1 < k ∧ k < length cases ∧ finalFCases
      inp (take k cases, newTimeState1, newTimeEnv1) ⟶ ({IChoice (ChoiceId choiceName (Party.
     Role role)) 0} ⊢ (split_cases role choiceName k (When (drop k cases) t cont), ctx2,
     newTimeState2⦇ choices := MList.insert (ChoiceId choiceName (Party.Role role)) 0 (choices
     newTimeState2) ⦈, newTimeEnv2, warnings2, payments2) −λ(Some inp, lower, upper)→ (
     split_cases role choiceName k c1', ctx2, dropState2', newTimeEnv2, warnings2 @
     convertApplyWarning newW, payments2)) ∧ split_cases_relation role choiceName k (c1', ctx1,
      state1', newTimeEnv1, warnings1, payments1) (split_cases role choiceName k c1', ctx2,
     dropState2', newTimeEnv2, warnings2, payments2)"
  using 1(3)[of newCases newTCont "split_cases role choiceName k (When (drop k cases) t cont)"
       ctx1 ctx2 newTimeState1 "newTimeState2⦇ choices := MList.insert (ChoiceId choiceName (
       Party.Role role)) 0 (choices newTimeState2) ⦈" newTimeEnv1 newTimeEnv2 warnings1
       warnings2 payments1 payments2 inp lower upper c1' ctx1' state1' env1' warnings1'
       payments1'] drop_in_split_cases_relation final_take_k_implies_drop_k_transition_to_final
        newCases_newTCont_obtained by (smt (z3) LambdaReceive calculation(12) cases_step
       deterministicSmallStepReceive isFreshChoiceInProcess.simps length_map new_time_states
       real_lower_upper_obtained split_cases_relation.simps)
moreover have inp_not_choice: "inp ≠ (IChoice (ChoiceId choiceName (Party.Role role)) 0)" using
      freshChoiceMatchNoChoiceInput_cases_step is_fresh_in_map_split no_split_cases2_step by
     blast
moreover have computeCasesStep_inp_choice_None: "computeCasesStep inp ([Case (ActionGuard (
     Choice (FChoiceId choiceName (Role role)) [(0, 0)])) (split_cases role choiceName k (When
     (drop k cases) t cont))], newTimeState2, newTimeEnv2) = None" using inp_not_choice apply (
     auto split: option.split)
  by (cases inp, auto split: if_split_asm)
moreover have finalFCases_inp_choice: "finalFCases inp ([Case (ActionGuard (Choice (FChoiceId
     choiceName (Role role)) [(0, 0)])) (split_cases role choiceName k (When (drop k cases) t
     cont))], newTimeState2, newTimeEnv2)" by (metis (no_types, lifting)
     computeCasesStep_inp_choice_None finalFCases_def option.distinct(1)
     reduceCasesStepImpliesComputeCaseStepRes)
moreover have finalFCases_take_append_choice: "finalFCases inp (take k cases, newTimeState1,
     newTimeEnv1) ⟶ finalFCases inp ((take k (map (λ(Case g c) ⇒ (Case g (split_cases role
     choiceName k c))) cases)) @ [Case (ActionGuard (Choice (FChoiceId choiceName (Role role))
```

```
     [(0, 0)])) (split_cases role choiceName k (When (drop k cases) t cont))], newTimeState2,
     newTimeEnv2)" using finalFCases_cases2 choice_step_take_k_map_cases
     small_k_cases2_is_take_k finalFCases_inp_choice finalFCasesAppend by auto
moreover have finalFCases2_no_match_take_split: "1 < k ∧ k < length cases ∧ finalFCases inp (
     take k cases, newTimeState1, newTimeEnv1) ⟶ finalFCases inp (cases2, newTimeState2,
     newTimeEnv2)" using small_k_cases2_is_take_k finalFCases_take_append_choice by (auto simp
     del: split_cases.simps)
moreover have split_halted_level_1: "1 < k ∧ k < length cases ∧ finalFCases inp (take k cases,
     newTimeState1, newTimeEnv1) ⟶ (\<nexists>f . (c2, ctx2, state2, env2, warnings2,
     payments2) →(Some inp, lower, upper) f)" using decompile_c2 finalFCases_cases2
     choice_step_take_k_map_cases small_k_cases2_is_take_k finalFCases2_no_match_take_split
     new_time_states real_lower_upper_obtained by (auto simp only: small_step_receive_ccs.simps
      finalFCases_def)
moreover have split_cases_choice_step_lambda_receive: "1 < k ∧ k < length cases ⟶ (c2, ctx2,
     state2, env2, warnings2, payments2) →(Some (IChoice (ChoiceId choiceName (Party.Role role
     )) 0), lower, upper) (split_cases role choiceName k (When (drop k cases) t cont), ctx2,
     newTimeState2⦇ choices := MList.insert (ChoiceId choiceName (Party.Role role)) 0 (choices
     newTimeState2) ⦈, newTimeEnv2, warnings2, payments2)" using small_step_receive_ccs.
     LambdaReceive[of lower upper state2 newTimeEnv2 newTimeState2 realLower realUpper t "
     IChoice (ChoiceId choiceName (Party.Role role)) 0" cases2 "λpassedT passedTCont.
     split_cases role choiceName k (FaustusV2AST.FContract.When (drop k cases) t cont)" "
     newTimeState2⦇FState.choices := MList.insert (ChoiceId choiceName (Party.Role role)) 0 (
     FState.choices newTimeState2)⦈" ApplyNoWarning "(Some (IChoice (ChoiceId choiceName (Party
     .Role role)) 0), lower, upper)" tCont2] small_k_cases2_is_take_k decompile_c2
     new_time_states small_k_cases_step_choice real_lower_upper_obtained
     newCases_newTCont_obtained by (auto simp del: split_cases.simps fixInterval.simps
     state_minimal_eq.simps)
moreover have full_hidden_step_lemma_proved: "1 < k ∧ k < length cases ∧ finalFCases inp (take
     k cases, newTimeState1, newTimeEnv1) ⟶ ({IChoice (ChoiceId choiceName (Party.Role role))
      0} ⊢ (c2, ctx2, state2, env2, warnings2, payments2) −λ(Some inp, lower, upper)→ (
     split_cases role choiceName k c1', split_cases_context role choiceName k ctx1', dropState2
     ', env1', warnings1', payments1'))"
  using full_semantics.RestrictedReceiveFull[of inp "{IChoice (ChoiceId choiceName (Party.Role
       role)) 0}" "(c2, ctx2, state2, env2, warnings2, payments2)" "(Some inp, lower, upper)"
       lower upper "(split_cases role choiceName k (When (drop k cases) t cont), ctx2,
       newTimeState2⦇ choices := MList.insert (ChoiceId choiceName (Party.Role role)) 0 (
       choices newTimeState2) ⦈, newTimeEnv2, warnings2, payments2)" "(split_cases role
       choiceName k c1', split_cases_context role choiceName k ctx1', dropState2', env1',
       warnings1', payments1')"] split_halted_level_1 small_k_cases2_is_take_k decompile_c2
       dropState2'_obtained new_time_states real_lower_upper_obtained singletonD singletonI
       small_k_cases_step_choice fixFInterval_fixed_point2
       split_cases_choice_step_lambda_receive apply (auto simp del: split_cases.simps
       state_minimal_eq.simps fixInterval.simps)
  by (smt (verit) LambdaReceive calculation(12) calculation(7) calculation(8) calculation(9)
```

```
        cases_step deterministicSmallStepReceive no_split_lemma_proved split_cases_relation.
        simps)
moreover have "inp ⊢ (take k cases, newTimeState1, newTimeEnv1) →cases_f2 (newC1', state1',
     newW) ⟶ (When ((take k (map (λ(Case g c) ⇒ (Case g (split_cases role choiceName k c)))
     cases)) @ [Case (ActionGuard (Choice (FChoiceId choiceName (Role role)) [(0, 0)])) (
     split_cases role choiceName k (When (drop k cases) t cont))]) t tCont2, ctx2, state2, env2
     , warnings2, payments2) →(Some inp, lower, upper) (takeKNewC2' t tCont2,
     split_cases_context role choiceName k ctx1', takeKNewTimeState2', env1', warnings2 @
     convertApplyWarning newW, payments1')" using decompile_c2 new_time_states
     take_k_cases_step_imp_step_in_relation casesStepImpliesAppendCasesStep
     small_step_receive_ccs.LambdaReceive real_lower_upper_obtained cases_step
     "1"(5,6,7,8,12,9) cases_step_constants by auto
moreover have "¬finalFCases inp (take k cases, newTimeState1, newTimeEnv1) ⟶ inp ⊢ (take k
     cases, newTimeState1, newTimeEnv1) →cases_f2 (newC1', state1', newW)" using cases_step
     append_take_drop_id[of k cases] deterministicCasesSemantics
     casesStepImpliesAppendCasesStep[of inp "take k cases" newTimeState1 newTimeEnv1 _ _ newW "
     drop k cases"] finalFCases_def apply auto
  by (metis FaustusV2Semantics.casesStepImpliesAppendCasesStep \<open>take k cases @ drop k
        cases = cases\<close>)
moreover have c1_constants: "ctx1 = ctx1' ∧ payments1 = payments1' ∧ (warnings1 @
     convertApplyWarning newW) = warnings1'" using 1(12) apply auto
  using cases_step by presburger
moreover have c2_step_no_split: "¬(1 < k ∧ k < length cases) ⟶ (When cases2 t tCont2, ctx2,
     state2, env2, warnings2, payments2) →(Some inp, lower, upper) (newC2' t (split_cases role
      choiceName k cont), ctx2, newTimeState2', newTimeEnv2, warnings2 @ convertApplyWarning
     newW, payments2)" using large_k_cases2_is_map no_split_cases2_step new_time_states
     real_lower_upper_obtained no_split_cases2_step small_step_receive_ccs.LambdaReceive[of
     lower upper state2 newTimeEnv2 newTimeState2 realLower realUpper t inp cases2 newC2'
     newTimeState2' newW _ tCont2 ctx2 env2 warnings2 payments2] by (smt (verit, ccfv_threshold
     ) FaustusV2AST.FContract.inject(3) calculation(4) decompile_c2 split_cases.simps(1))
moreover have c1_final_env_time: "newTimeEnv1 = env1'" using 1(12) real_lower_upper_obtained
     new_time_states cases_step deterministicSmallStepReceive LambdaReceive[of lower upper
     state1 newTimeEnv1 newTimeState1 realLower realUpper t inp cases newC1' state1' newW _
     cont ctx1 env1 warnings1 payments1] by auto
moreover have "¬(1 < k ∧ k < length cases) ⟶ (When cases2 t tCont2, ctx2, state2, env2,
     warnings2, payments2) →(Some inp, lower, upper) (newC2' t (split_cases role choiceName k
     cont), split_cases_context role choiceName k ctx1', newTimeState2', env1', warnings1',
     payments1')"
  using 1(5,8,9) decompile_c2 c1_constants c2_step_no_split real_lower_upper_obtained
        new_time_states c1_final_env_time by auto
moreover show ?case apply (cases "1 < k ∧ k < length cases")
   apply (cases "finalFCases inp (take k cases, newTimeState1, newTimeEnv1)")
  using c1_constants c1_final_env_time 1(5,8,9) dropState2'_obtained
        full_hidden_step_lemma_proved new_time_states
```

```
      apply auto[1]
    apply (smt (z3) BaseReceiveFull FaustusV2AST.FContract.inject(3) calculation(4) calculation
         (47) calculation(48) calculation(8) cases_step decompile_c2 inp_not_choice
         no_split_lemma_proved singletonD small_k_cases2_is_take_k split_cases.simps(1)
         split_cases_relation.simps take_k_cases_step_imp_step_in_relation)
    by (metis NoChoiceLambdaReceive c1_constants c1_final_env_time c2_step_no_split calculation
         (5) calculation(8) calculation(9) decompile_c2
         full_semantics_receive_choice_input_is_equivalent_to_small_step_add_choice_forward
         inp_not_choice new_time_states no_split_lemma_proved)
qed auto

inductive_cases ReceiveInpE: "(c2, ctx2, state2, env2, warnings2, payments2) →(inp, lower, upper
     ) (c2', ctx2', state2', env2', warnings2', payments2')"
thm ReceiveInpE

lemma split_cases_relation_backward_simulation_small_step_receive_add_choice:
"(ChoiceId choiceName (SemanticsTypes.Role role)) ⊢ (c2, ctx2, state2, env2, warnings2, payments2
     ) →(inp, lower, upper) (c2', ctx2', state2', env2', warnings2', payments2') ⟹
  split_cases_relation role choiceName k (c1, ctx1, state1, env1, warnings1, payments1) (c2, ctx2
       , state2, env2, warnings2, payments2) ⟹
       ∃c1' ctx1' state1' env1' warnings1' payments1'.
          ((ChoiceId choiceName (SemanticsTypes.Role role)) ⊢ (c1, ctx1, state1, env1, warnings1,
                payments1) →(inp, lower, upper) (c1', ctx1', state1', env1', warnings1',
               payments1')) ∧
          split_cases_relation role choiceName k (c1', ctx1', state1', env1', warnings1',
               payments1') (c2', ctx2', state2', env2', warnings2', payments2')"
proof (induction "(ChoiceId choiceName (SemanticsTypes.Role role))" c2 ctx2 state2 env2 warnings2
      payments2 inp lower upper c2' ctx2' state2' env2' warnings2' payments2'
    arbitrary: c1 ctx1 state1 env1 warnings1 payments1
    rule: small_step_receive_ccs_add_choice.induct[split_format(complete)])
  case (NoChoiceLambdaReceive c2 ctx2 state2 env2 warnings2 payments2 inp lower upper c2' ctx2'
       state2' env2' warnings2' payments2' innerInp innerLower innerUpper)
  moreover obtain cases2 t tCont2 where decompile_c2: "c2 = When cases2 t tCont2" using
       calculation by auto
  moreover obtain cases1 tCont1 where decompile: "c1 = When cases1 t tCont1 ∧
    split_cases role choiceName k tCont1 = tCont2" using calculation apply (cases c1, auto)
    by (smt (verit) FaustusV2AST.FContract.inject(3))
  moreover have base_state_eq: "state_minimal_eq choiceName state2 state1" using calculation by
       auto
  moreover obtain newTimeEnv1 newTimeState1 newTimeEnv2 newTimeState2 where new_time_states: "
       fixInterval (lower, upper) state1 = IntervalTrimmed newTimeEnv1 newTimeState1 ∧
    fixInterval (lower, upper) state2 = IntervalTrimmed newTimeEnv2 newTimeState2" using
         NoChoiceLambdaReceive(1,4) by (cases "upper < lower"; cases "upper < minTime state1";
         cases "lower < minTime state1", auto)
```

```
moreover have "newTimeEnv1 = newTimeEnv2 ∧ state_minimal_eq choiceName newTimeState2
     newTimeState1" using NoChoiceLambdaReceive(1,4) new_time_states base_state_eq apply (cases
      "upper < lower"; cases "upper < minTime state1"; cases "lower < minTime state1", auto)
       apply (meson FIntervalResult.inject(1))
  by (metis FIntervalResult.inject(1) FState.surjective FState.update_convs(5))+
moreover have "choices newTimeState1 = choices state1" using calculation apply (cases "upper <
     lower"; auto)
  by (cases "upper < minTime state1"; cases "lower < minTime state1", auto)
moreover have "choices newTimeState2 = choices state2" using new_time_states apply (cases "
     upper < lower"; auto)
  apply (cases "upper < minTime state1"; cases "lower < minTime state1", auto)
  by (cases "upper < minTime state2"; cases "lower < minTime state2", auto)+
moreover have "isFreshChoiceInCasesGuards choiceName cases1" using calculation
     isFreshChoiceInCasesImpliesIsFreshChoiceInCasesGuards by auto
moreover have "split_cases_context role choiceName k ctx1 = ctx2" using calculation by auto
moreover obtain newCases where "newCases = map (λ(Case g c) ⇒ (Case g (split_cases role
     choiceName k c))) cases1" using calculation by auto
moreover obtain takeNewCases dropNewCases where "take k newCases = takeNewCases ∧ drop k
     newCases = dropNewCases" by auto
moreover have "¬(k > 1 ∧ length cases1 > k) ⟶ cases2 = newCases" using calculation by auto
moreover have small_k_cases2_split: "(k > 1 ∧ length cases1 > k) ⟶ cases2 = (takeNewCases @ [
     Case (ActionGuard (Choice (FChoiceId choiceName (Role role)) [(0, 0)])) (split_cases role
     choiceName k (When (drop k cases1) t tCont1))])" using calculation by auto
moreover have "computeActionStep innerInp (Choice (FChoiceId choiceName (Role role)) [(0, 0)],
     newTimeState2, newTimeEnv2) = None" using calculation by (cases innerInp, auto)
moreover have "finalFCases innerInp ([Case (ActionGuard (Choice (FChoiceId choiceName (Role
     role)) [(0, 0)])) (split_cases role choiceName k (When (drop k cases1) t tCont1))],
     newTimeState2, newTimeEnv2)" by (metis (no_types, lifting) ActionGuardE calculation
     computeActionStepIsSmallStep computeCaseStep.simps computeCasesStep.simps(1)
     computeCasesStep.simps(2) computeGuardStepHaltedImpliesSmallStepHalted finalFCases_def
     finalGuard_def option.distinct(1) option.simps(4) reduceCasesStepImpliesComputeCaseStepRes
     )
moreover obtain newC2 newW where newC2_obtained: "innerInp ⊢ (cases2, newTimeState2,
     newTimeEnv2) →cases_f2 (newC2, state2', newW) ∧ warnings2' = warnings1 @
     convertApplyWarning newW" using NoChoiceLambdaReceive new_time_states decompile_c2 by (
     induction c2 ctx2 state2 env2 warnings2 payments2 inp lower upper c2' ctx2' state2' env2'
     warnings2' payments2' rule: small_step_receive_ccs.induct[split_format(complete)], auto)
moreover have "(k > 1 ∧ length cases1 > k) ⟶ innerInp ⊢ (takeNewCases, newTimeState2,
     newTimeEnv2) →cases_f2 (newC2, state2', newW)" using newC2_obtained
     casesStepDropFinalCases small_k_cases2_split calculation by meson
moreover have "¬(k > 1 ∧ length cases1 > k) ⟶ innerInp ⊢ (newCases, newTimeState2,
     newTimeEnv2) →cases_f2 (newC2, state2', newW)" using calculation by auto
moreover obtain takeCases1 dropCases1 where "take k cases1 = takeCases1 ∧ drop k cases1 =
     dropCases1" by auto
```

```
moreover have "takeNewCases = map (λ(Case g c) ⇒ (Case g (split_cases role choiceName k c)))
     takeCases1" using calculation take_map by blast
moreover have "isFreshChoiceInCasesGuards choiceName takeCases1" by (metis append_take_drop_id
     calculation isFreshChoiceInCasesGuardsPreservedInSlice)
moreover obtain newC1 state1' where takeCasesStep: "(k > 1 ∧ length cases1 > k) ⟶ innerInp ⊢
     (takeCases1, newTimeState1, newTimeEnv1) →cases_f2 (newC1, state1', newW) ∧
  split_cases role choiceName k (newC1 t tCont1) = newC2 t tCont2 ∧
  state_minimal_eq choiceName state1' state2'" using
       small_step_cases_preserved_inverse_split_cases_relation[of innerInp role choiceName k
       takeCases1 newTimeState2 newTimeEnv1 newC2 state2' newW newTimeState1 t tCont1] by (
       metis NoChoiceLambdaReceive.prems calculation(21,24,25,9) decompile split_cases_relation
       .simps state_minimal_eq_commutative)
moreover have fullTakeCasesStep: "(k > 1 ∧ length cases1 > k) ⟶ innerInp ⊢ (cases1,
     newTimeState1, newTimeEnv1) →cases_f2 (newC1, state1', newW)" by (metis
     append_take_drop_id casesStepImpliesAppendCasesStep calculation)
moreover obtain fullNewC1 fullNewState1 where fullCasesStep: "¬(k > 1 ∧ length cases1 > k) ⟶
     innerInp ⊢ (cases1, newTimeState1, newTimeEnv1) →cases_f2 (fullNewC1, fullNewState1, newW)
       ∧
  split_cases role choiceName k (fullNewC1 t tCont1) = newC2 t tCont2 ∧
  state_minimal_eq choiceName fullNewState1 state2'" using
       small_step_cases_preserved_inverse_split_cases_relation[of innerInp role choiceName k
       cases1 newTimeState2 newTimeEnv1 newC2 state2' newW newTimeState1 t tCont1]
  by (metis (no_types, lifting) NoChoiceLambdaReceive.prems calculation(12,14,22,9) decompile
       split_cases_relation.simps state_minimal_eq_commutative)
moreover obtain realLower realUpper where "timeInterval newTimeEnv2 = (realLower, realUpper) ∧
     realLower < t ∧ realUpper < t" using NoChoiceLambdaReceive new_time_states decompile_c2
     new_time_states by (induction c2 ctx2 state2 env2 warnings2 payments2 inp lower upper c2'
     ctx2' state2' env2' warnings2' payments2' rule: small_step_receive_ccs.induct[split_format
     (complete)], auto)
moreover have c2_constants: "ctx2 = ctx2' ∧ payments2 = payments2'" using NoChoiceLambdaReceive
     (1) NoChoiceLambdaReceive(3) by auto
moreover have "(k > 1 ∧ length cases1 > k) ⟶ (c1, ctx1, state1, env1, warnings1, payments1)
     →(Some innerInp, lower, upper) (newC1 t tCont1, ctx1, state1', newTimeEnv1, warnings2',
     payments2')" using LambdaReceive[of lower upper state1 newTimeEnv1 newTimeState1 realLower
      realUpper t innerInp cases1 newC1 state1' newW _ tCont1 ctx1 env1 warnings1 payments1]
     using calculation(29) calculation(9) decompile fullTakeCasesStep new_time_states
     c2_constants by (metis NoChoiceLambdaReceive.prems newC2_obtained split_cases_relation.
     simps)
moreover have split_transition: "(k > 1 ∧ length cases1 > k) ⟶ (ChoiceId choiceName (
     SemanticsTypes.Role role)) ⊢ (c1, ctx1, state1, env1, warnings1, payments1) →(Some
     innerInp, lower, upper) (newC1 t tCont1, ctx1, state1', newTimeEnv1, warnings2', payments2
     ')" using calculation(2) calculation(30,31) small_step_receive_ccs_add_choice.
     NoChoiceLambdaReceive by auto
moreover have no_split_transition: "¬(k > 1 ∧ length cases1 > k) ⟶ (ChoiceId choiceName (
```

```
      SemanticsTypes.Role role)) ⊢ (c1, ctx1, state1, env1, warnings1, payments1) →(Some
      innerInp, lower, upper) (fullNewC1 t tCont1, ctx1, fullNewState1, newTimeEnv1, warnings2',
       payments2')" using LambdaReceive NoChoiceLambdaReceive.prems calculation
      choice_receive_same_as_basic_when_fresh decompile fullCasesStep by auto
  moreover have "newC2 t tCont2 = c2'" using NoChoiceLambdaReceive new_time_states decompile_c2
      newC2_obtained apply (induction c2 ctx2 state2 env2 warnings2 payments2 inp lower upper c2
      ' ctx2' state2' env2' warnings2' payments2' rule: small_step_receive_ccs.induct[
      split_format(complete)], auto)
    by (metis FaustusV2AST.FContract.inject(3) Pair_inject deterministicCasesSemantics)
  moreover have c2_final_env_time: "env2' = newTimeEnv2" using NoChoiceLambdaReceive
      new_time_states small_step_receive_ccs.simps by auto
  moreover have split_relation: "(k > 1 ∧ length cases1 > k) ⟶ split_cases_relation role
      choiceName k (newC1 t tCont1, ctx1, state1', newTimeEnv1, warnings2', payments2') (c2',
      ctx2', state2', env2', warnings2', payments2')" by (metis NoChoiceLambdaReceive.prems
      c2_constants c2_final_env_time calculation(34) calculation(9)
      choice_receive_preserves_fresh_choice split_cases_relation.simps split_transition
      takeCasesStep)
  moreover have no_split_relation: "¬(k > 1 ∧ length cases1 > k) ⟶ split_cases_relation role
      choiceName k (fullNewC1 t tCont1, ctx1, fullNewState1, newTimeEnv1, warnings2', payments2
      ') (c2', ctx2', state2', env2', warnings2', payments2')" by (metis NoChoiceLambdaReceive.
      prems c2_constants c2_final_env_time calculation(34) calculation(9)
      choice_receive_preserves_fresh_choice fullCasesStep no_split_transition
      split_cases_relation.simps)
  show ?case apply (cases "(k > 1 ∧ length cases1 > k)") using NoChoiceLambdaReceive
      split_transition split_relation no_split_transition no_split_relation by blast+
next
  case (ChoiceLambdaReceive c2 ctx2 state2 env2 warnings2 payments2 inp1 lower1 upper1 lower2
      upper2 innerC2 innerCtx2 innerState2 innerEnv2 innerWarnings2 innerPayments2 c2' ctx2'
      state2' env2' warnings2' payments2' innerInp)
  moreover have "lower1 = lower2 ∧ upper1 = upper2" using ChoiceLambdaReceive by auto
  moreover obtain tCont2 cases2 t where decompile_c2: "c2 = When cases2 t tCont2" using
      ChoiceLambdaReceive by auto
  moreover obtain cases1 tCont1 where decompile: "c1 = When cases1 t tCont1 ∧
    split_cases role choiceName k tCont1 = tCont2" using calculation apply (cases c1, auto)
    by (smt (verit) FaustusV2AST.FContract.inject(3))
  moreover obtain newTimeEnv1 newTimeState1 newTimeState2 newTimeEnv2 where new_states: "
      fixInterval (lower1, upper1) state1 = IntervalTrimmed newTimeEnv1 newTimeState1 ∧
      fixInterval (lower1, upper1) state2 = IntervalTrimmed newTimeEnv2 newTimeState2" using
      calculation apply (cases "upper2 < lower2", auto)
    by (cases "upper2 < minTime state1"; cases "lower2 < minTime state1", auto)
  moreover have times_equal: "minTime state1 = minTime state2" using ChoiceLambdaReceive by auto
  moreover have new_times_equal: "newTimeEnv1 = newTimeEnv2 ∧ minTime newTimeState1 = minTime
      newTimeState2" using times_equal new_states fixFIntervalOnlyTimeDependent by metis
  moreover have time_state_1_only_time_changes: "choices newTimeState1 = choices state1 ∧
```

```
  accounts newTimeState1 = accounts state1 ∧
  boundValues newTimeState1 = boundValues state1 ∧
  boundParties newTimeState1 = boundParties state1" using new_states apply (cases "upper1 <
       lower1", auto)
  by (cases "upper1 < minTime state1"; cases "lower1 < minTime state1", auto)+
moreover have time_state_2_only_time_changes: "choices newTimeState2 = choices state2 ∧
  accounts newTimeState2 = accounts state2 ∧
  boundValues newTimeState2 = boundValues state2 ∧
  boundParties newTimeState2 = boundParties state2" using new_states apply (cases "upper1 <
       lower1", auto)
     apply (cases "upper1 < minTime state1"; cases "lower1 < minTime state1"; cases "upper1 <
          minTime state2"; auto; metis fix_time_preserves_choices new_states)
    apply (cases "upper1 < minTime state1"; cases "lower1 < minTime state1"; cases "upper1 <
         minTime state2"; auto; metis FIntervalResult.inject(1) FState.select_convs(1) FState.
         surjective FState.update_convs(5))
   apply (cases "upper1 < minTime state1"; cases "lower1 < minTime state1"; cases "upper1 <
        minTime state2"; auto; metis FIntervalResult.inject(1) FState.select_convs(3) FState.
        surjective FState.update_convs(5))
  apply (cases "upper1 < minTime state1"; cases "lower1 < minTime state1"; cases "upper1 <
       minTime state2"; auto)
  by (metis FIntervalResult.inject(1) FState.ext_inject FState.surjective FState.update_convs
       (5))+
moreover have newTimeStatesEq: "state_minimal_eq choiceName newTimeState2 newTimeState1" using
     ChoiceLambdaReceive(7) new_states times_equal new_times_equal
     time_state_1_only_time_changes time_state_2_only_time_changes by auto
moreover have "isFreshChoiceInCasesGuards choiceName cases1" using calculation
     isFreshChoiceInCasesImpliesIsFreshChoiceInCasesGuards by auto
moreover have "split_cases_context role choiceName k ctx1 = ctx2" using calculation by auto
moreover obtain newCases where "newCases = map (λ(Case g c) ⇒ (Case g (split_cases role
     choiceName k c))) cases1" using calculation by auto
moreover obtain takeNewCases dropNewCases where "take k newCases = takeNewCases ∧ drop k
     newCases = dropNewCases" by auto
moreover have cases2_is_newCases_k_size: "¬(k > 1 ∧ length cases1 > k) ⟶ cases2 = newCases"
     using calculation by auto
moreover have cases2_is_takeNewCases_k_size: "(k > 1 ∧ length cases1 > k) ⟶ cases2 = (
     takeNewCases @ [Case (ActionGuard (Choice (FChoiceId choiceName (Role role)) [(0, 0)])) (
     split_cases role choiceName k (When (drop k cases1) t tCont1))])" using calculation by
     auto
moreover have "isFreshChoiceInCasesGuardsShallow choiceName (map (λ(Case g c) ⇒ (Case g (
     split_cases role choiceName k c))) cases1)" using calculation(17) by (induction cases1,
     auto split: FCase.split)
moreover have "(∀ party i . finalFCases (IChoice (ChoiceId choiceName party) i) (takeNewCases,
     newTimeState2, newTimeEnv2))" using calculation by (metis append_take_drop_id
     freshChoiceShallowNoMatch_cases_step isFreshChoiceInCasesGuardsShallowPreservedInSlice)
```

```
moreover have choiceActionStep: "(IChoice (ChoiceId choiceName (Party.Role role)) 0) ⊢ (Choice
     (FChoiceId choiceName (Role role)) [(0, 0)], newTimeState2, newTimeEnv2) →af2 (
     newTimeState2⦇choices := MList.insert (ChoiceId choiceName (Party.Role role)) 0 (choices
     newTimeState2) ⦈, ApplyNoWarning)" by (simp add: small_step_action_fresh_choice_state_eq)
moreover have choiceActionStep: "computeActionStep (IChoice (ChoiceId choiceName party) i) (
     Choice (FChoiceId choiceName (Role role)) [(0, 0)], newTimeState2, newTimeEnv2) = Some (
     newTimeState2⦇choices := MList.insert (ChoiceId choiceName (Party.Role role)) 0 (choices
     newTimeState2) ⦈, ApplyNoWarning) ⟹ party = (Party.Role role) ∧ i = 0" by (auto split:
     if_split_asm)
moreover obtain newChoiceState where new_choice_state: "newTimeState2⦇choices := MList.insert (
     ChoiceId choiceName (Party.Role role)) 0 (choices newTimeState2) ⦈ = newChoiceState ∧
     state_minimal_eq choiceName newTimeState2 newChoiceState" using
     small_step_action_fresh_choice_state_eq by auto
moreover have choiceGuardStep: "(IChoice (ChoiceId choiceName (Party.Role role)) 0) ⊢ (
     ActionGuard (Choice (FChoiceId choiceName (Role role)) [(0, 0)]), newTimeState2,
     newTimeEnv2) →gf2 ([], None, newChoiceState, ApplyNoWarning)"  by (metis ActionGuardStep
     append.right_neutral calculation(25,27) convertApplyWarning.simps(1))
moreover have choiceCaseStep: "(IChoice (ChoiceId choiceName (Party.Role role)) 0) ⊢ (Case (
     ActionGuard (Choice (FChoiceId choiceName (Role role)) [(0, 0)])) (split_cases role
     choiceName k (When (drop k cases1) t tCont1)), newTimeState2, newTimeEnv2) →casef2 (λ
     innerT innerTCont . prependStatements [] (split_cases role choiceName k (When (drop k
     cases1) t tCont1)), newChoiceState, ApplyNoWarning)" using CaseGuardCompleteStep
     choiceGuardStep by meson
moreover have choiceCasesStep: "(IChoice (ChoiceId choiceName (Party.Role role)) 0) ⊢ ([Case (
     ActionGuard (Choice (FChoiceId choiceName (Role role)) [(0, 0)])) (split_cases role
     choiceName k (When (drop k cases1) t tCont1))], newTimeState2, newTimeEnv2) →casesf2 (λ
     innerT innerTCont . prependStatements [] (split_cases role choiceName k (When (drop k
     cases1) t tCont1)), newChoiceState, ApplyNoWarning)" using CasesHeadMatches choiceCaseStep
      by force
moreover have partyRoleIZero: "∀res . computeCasesStep (IChoice (ChoiceId choiceName party) i)
     ([Case (ActionGuard (Choice (FChoiceId choiceName (Role role)) [(0, 0)])) (split_cases
     role choiceName k (When (drop k cases1) t tCont1))], newTimeState2, newTimeEnv2) = Some
     res ⟶ party = (Party.Role role) ∧ i = 0" by auto
moreover have final_new_cases: "finalFCases (IChoice (ChoiceId choiceName (Party.Role role)) 0)
      (newCases, newTimeState2, newTimeEnv2)" by (simp add: calculation(19) calculation(23)
     freshChoiceShallowNoMatch_cases_step)
moreover have newCases_fails: "((When newCases t tCont2, ctx2, state2, env2, warnings2,
     payments2) →(Some (IChoice (ChoiceId choiceName (Party.Role role)) 0), lower2, upper2) f)
      ⟹ False" apply (induction "(When newCases t tCont2, ctx2, state2, env2, warnings2,
     payments2)" "(Some (IChoice (ChoiceId choiceName (Party.Role role)) 0), lower2, upper2)" "
     f" rule: small_step_receive_ccs.induct)
  using final_new_cases apply auto
  using calculation(19) calculation(23) freshChoiceShallowMatchNoChoiceInput_cases_step by
       blast
```

```
moreover have cases2_not_newCases: "cases2 ≠ newCases" using final_new_cases newCases_fails[of
     ] decompile_c2 ChoiceLambdaReceive(2) small_step_receive_ccs.simps apply auto
  by (metis calculation(19) calculation(23) freshChoiceShallowNoMatch_cases_step_aux)
moreover have cases2_is_takeNewCases: "cases2 = (takeNewCases @ [Case (ActionGuard (Choice (
     FChoiceId choiceName (Role role)) [(0, 0)])) (split_cases role choiceName k (When (drop k
     cases1) t tCont1))])" using cases2_is_newCases_k_size cases2_not_newCases
     cases2_is_takeNewCases_k_size by auto
moreover have take_cases_cases_step: "(IChoice (ChoiceId choiceName (Party.Role role)) 0) ⊢ (
     takeNewCases @ [Case (ActionGuard (Choice (FChoiceId choiceName (Role role)) [(0, 0)])) (
     split_cases role choiceName k (When (drop k cases1) t tCont1))], newTimeState2,
     newTimeEnv2) →cases_f2 (λ innerT innerTCont . prependStatements [] (split_cases role
     choiceName k (When (drop k cases1) t tCont1)), newChoiceState, ApplyNoWarning)" using
     calculation(24,32) casesStepAppendFinalCases choiceCasesStep by presburger
moreover have "(λ innerT innerTCont . prependStatements [] (split_cases role choiceName k (When
      (drop k cases1) t tCont1))) t tCont2 = (split_cases role choiceName k (When (drop k
     cases1) t tCont1))" by simp
moreover have inner_choice_step: "(When cases2 t tCont2, ctx2, state2, env2, warnings2,
     payments2) →(Some (IChoice (ChoiceId choiceName (Party.Role role)) 0), lower2, upper2) (
     innerC2, innerCtx2, innerState2, innerEnv2, innerWarnings2, innerPayments2)" using
     calculation(2) decompile_c2 by auto
moreover have decompile_innerC2: "innerC2 = (split_cases role choiceName k (When (drop k cases1
     ) t tCont1))" using inner_choice_step take_cases_cases_step cases2_is_takeNewCases  apply
     (elim ReceiveE)
  using new_states deterministicCasesSemantics calculation(8) decompile_c2 decompile by (auto,
       blast)
moreover have decompile_innerCtx2: "innerCtx2 = ctx2" using inner_choice_step
     take_cases_cases_step cases2_is_takeNewCases decompile_c2 decompile  apply (elim ReceiveE)
  using new_states deterministicCasesSemantics calculation(8) decompile_c2 decompile by (auto)
moreover have decompile_innerState2: "innerState2 = newChoiceState" using inner_choice_step
     take_cases_cases_step cases2_is_takeNewCases decompile_c2 decompile  apply (elim ReceiveE)
  using new_states deterministicCasesSemantics calculation(8) decompile_c2 decompile by (auto)
moreover have decompile_innerEnv2: "innerEnv2 = newTimeEnv2" using inner_choice_step
     take_cases_cases_step cases2_is_takeNewCases decompile_c2 decompile  apply (elim ReceiveE)
  using new_states deterministicCasesSemantics calculation(8) decompile_c2 decompile by (auto)
moreover have decompile_innerWarnings2: "innerWarnings2 = warnings2" using inner_choice_step
     take_cases_cases_step cases2_is_takeNewCases decompile_c2 decompile  apply (elim ReceiveE)
  using new_states deterministicCasesSemantics calculation(8) decompile_c2 decompile apply (
       auto)
  using convertApplyWarning.simps(1) by blast
moreover have decompile_innerPayments2: "innerPayments2 = payments2" using inner_choice_step
     take_cases_cases_step cases2_is_takeNewCases decompile_c2 decompile  apply (elim ReceiveE)
  using new_states deterministicCasesSemantics calculation(8) decompile_c2 decompile by (auto)
moreover obtain dropCases1 where dropCasesEq: "dropCases1 = drop k cases1 ∧ map (λ(Case g c) ⇒
      (Case g (split_cases role choiceName k c))) dropCases1 = dropNewCases" by (metis
```

```
     calculation(20) calculation(19) drop_map)
moreover have fresh_choice_drop_cases: "isFreshChoiceInCasesGuards choiceName dropCases1" by (
     metis append_take_drop_id calculation(17) dropCasesEq
     isFreshChoiceInCasesGuardsPreservedInSlice)
moreover have state1_eq_choice_state: "state_minimal_eq choiceName newTimeState1 newChoiceState
     " using calculation(27) newTimeStatesEq state_minimal_eq_commutative
     state_minimal_eq_transitive by blast
moreover have dropCases1_in_relation_inner: "split_cases_relation role choiceName k (When
     dropCases1 t tCont1, ctx1, newTimeState1, newTimeEnv1, warnings1, payments1) (innerC2,
     innerCtx2, innerState2, innerEnv2, innerWarnings2, innerPayments2)" by (metis
     ChoiceLambdaReceive.prems append_take_drop_id decompile decompile_innerC2
     decompile_innerCtx2 decompile_innerEnv2 decompile_innerPayments2 decompile_innerState2
     decompile_innerWarnings2 dropCasesEq isFreshChoiceInCasesPreservedInSlice
     isFreshChoiceInContract.simps(4) isFreshChoiceInProcess.simps new_times_equal
     split_cases_relation.simps state1_eq_choice_state)
moreover obtain c1' ctx1' state1' env1' warnings1' payments1' where dropCases1_step_in_relation
     : "(ChoiceId choiceName (Party.Role role) ⊢ (When dropCases1 t tCont1, ctx1, newTimeState1
     , newTimeEnv1, warnings1, payments1) →(inp1, lower1, upper1) (c1', ctx1', state1', env1',
      warnings1', payments1')) ∧
     split_cases_relation role choiceName k (c1', ctx1', state1', env1', warnings1', payments1
          ') (c2', ctx2', state2', env2', warnings2', payments2')"
  using calculation(4) dropCases1_in_relation_inner by presburger
moreover obtain takeCases1 where takeCasesEq: "takeCases1 = take k cases1 ∧ map (λ(Case g c) ⇒
      (Case g (split_cases role choiceName k c))) takeCases1 = takeNewCases" by (metis
     calculation(19,20) take_map)
moreover obtain realLower realUpper where real_time_bound: "timeInterval newTimeEnv2 = (
     realLower, realUpper) ∧ realLower < t ∧ realUpper < t" using ChoiceLambdaReceive
     new_states decompile_c2 by (auto simp add: small_step_receive_ccs.simps)
moreover have finalFCases_innerInp: "finalFCases innerInp (cases2, newTimeState2, newTimeEnv2)"
      using ChoiceLambdaReceive(2) apply (simp add: small_step_receive_ccs.simps del:
     fixInterval.simps)
  using decompile_c2 real_time_bound new_states calculation(8) apply (simp del: fixInterval.
       simps)
  using small_step_receive_ccs.LambdaReceive[of lower2 upper2 state2 newTimeEnv2 newTimeState2
       realLower realUpper t innerInp cases2] finalFCases_def apply (simp del: fixInterval.
       simps)
  apply (auto simp del: fixInterval.simps)
  using ChoiceLambdaReceive(1,6) by blast
moreover have finalFCases_innerInp_takeNewCases: "finalFCases innerInp (takeNewCases,
     newTimeState2, newTimeEnv2)" using cases2_is_takeNewCases finalFCases_innerInp
     casesStepImpliesAppendCasesStep finalFCases_def by fastforce
moreover have fresh_choice_take_cases: "isFreshChoiceInCasesGuards choiceName takeCases1" by (
     metis append_take_drop_id calculation(17) calculation(50)
     isFreshChoiceInCasesGuardsPreservedInSlice)
```

```
  moreover have fresh_choice_drop_cases: "isFreshChoiceInCases choiceName dropCases1" using
       dropCases1_in_relation_inner by auto
  moreover have final_take_k_cases1: "finalFCases innerInp (take k cases1, newTimeState1,
       newTimeEnv1)" using small_step_cases_preserves_split_cases_relation[of innerInp takeCases1
        newTimeState1 newTimeEnv1 _ _ _ choiceName newTimeState2 role k t tCont1] takeCasesEq
       ChoiceLambdaReceive(7) newTimeStatesEq new_choice_state state1_eq_choice_state
       state_minimal_eq_transitive fresh_choice_take_cases finalFCases_def new_times_equal
       finalFCases_innerInp_takeNewCases by (auto, presburger)
  moreover have fixFInterval_fixed_point: "fixInterval (lower1, upper1) newTimeState1 =
       IntervalTrimmed newTimeEnv1 newTimeState1" using new_states by (cases "upper1 < lower1";
       cases "upper1 < minTime state1"; cases "lower1 < minTime state1", auto)
  moreover have When_dropCases1_basic_step: "(When dropCases1 t tCont1, ctx1, newTimeState1,
       newTimeEnv1, warnings1, payments1) →(inp1, lower1, upper1) (c1', ctx1', state1', env1',
       warnings1', payments1')" by (meson dropCases1_step_in_relation
       choice_receive_same_as_basic_when_fresh dropCases1_in_relation_inner split_cases_relation.
       simps)
  moreover obtain newC1' newW where "innerInp ⊢ (dropCases1, newTimeState1, newTimeEnv1) →cases_f
       2 (newC1', state1', newW) ∧ newC1' t tCont1 = c1'" using When_dropCases1_basic_step
       deterministicCasesSemantics ChoiceLambdaReceive(6) fixFInterval_fixed_point by (auto simp
       add: small_step_receive_ccs.simps)
  moreover have "innerInp ⊢ (cases1, newTimeState1, newTimeEnv1) →cases_f2 (newC1', state1', newW
       )" by (metis append_take_drop_id calculation(59) casesStepAppendFinalCases dropCasesEq
       final_take_k_cases1)
  moreover have "(When cases1 t tCont1, ctx1, state1, env1, warnings1, payments1) →(inp1, lower1
       , upper1) (c1', ctx1', state1', env1', warnings1', payments1')" by (smt (verit)
       LambdaReceive When_dropCases1_basic_step calculation(59) calculation(6) calculation(60)
       calculation(8) deterministicSmallStepReceive fixFInterval_fixed_point new_states
       new_times_equal real_time_bound)
  moreover have "(ChoiceId choiceName (Party.Role role) ⊢ (When cases1 t tCont1, ctx1, state1,
       env1, warnings1, payments1) →(inp1, lower1, upper1) (c1', ctx1', state1', env1',
       warnings1', payments1'))" using NoChoiceLambdaReceive calculation(5) calculation(6)
       calculation(61) by presburger
  ultimately show ?case by meson
next
  case (CCSTimeout inp lower1 upper1 lower2 upper2 c2 ctx2 state2 env2 warnings2 payments2 c2'
       ctx2' state2' env2' warnings2' payments2')
   moreover have "lower1 = lower2 ∧ upper1 = upper2" using CCSTimeout by auto
   moreover obtain tCont2 cases2 t where decompile_c2: "c2 = When cases2 t tCont2" using
        CCSTimeout by auto
   moreover obtain cases1 tCont1 where decompile: "c1 = When cases1 t tCont1 ∧
    split_cases role choiceName k tCont1 = tCont2" using calculation apply (cases c1, auto)
    by (smt (verit) FaustusV2AST.FContract.inject(3))
  moreover obtain newTimeEnv1 newTimeState1 where new_state_1: "fixInterval (lower1, upper1)
       state1 = IntervalTrimmed newTimeEnv1 newTimeState1" using calculation apply (cases "upper2
```

```
      < lower2", auto)
  by (cases "upper2 < minTime state1"; cases "lower2 < minTime state1", auto)
moreover obtain newTimeEnv2 newTimeState2 where new_state_2: "fixInterval (lower1, upper1)
     state2 = IntervalTrimmed newTimeEnv2 newTimeState2" using calculation by (cases "lower1 <
     (minTime state1)", auto)
moreover have "newTimeEnv1 = newTimeEnv2" using calculation apply auto
  apply (cases "upper2 < lower2"; cases "upper2 < minTime state1", auto)
  apply (cases "lower2 < minTime state1", auto)
  by (meson FIntervalResult.inject(1))
moreover have times_equal: "minTime state1 = minTime state2" using CCSTimeout by auto
moreover have new_times_equal: "newTimeEnv1 = newTimeEnv2 ∧ minTime newTimeState1 = minTime
     newTimeState2" using times_equal new_state_1 new_state_2 fixFIntervalOnlyTimeDependent by
     metis
moreover have time_state_1_only_time_changes: "choices newTimeState1 = choices state1 ∧
  accounts newTimeState1 = accounts state1 ∧
  boundValues newTimeState1 = boundValues state1 ∧
  boundParties newTimeState1 = boundParties state1" using new_state_1 by (cases "upper1 <
       lower1"; cases "upper1 < minTime state1"; cases "lower1 < minTime state1", auto)
moreover have time_state_2_only_time_changes: "choices newTimeState2 = choices state2 ∧
  accounts newTimeState2 = accounts state2 ∧
  boundValues newTimeState2 = boundValues state2 ∧
  boundParties newTimeState2 = boundParties state2" using new_state_2 by (cases "upper1 <
       lower1"; cases "upper1 < minTime state2"; cases "lower1 < minTime state2", auto)
moreover have newTimeStatesEq: "state_minimal_eq choiceName newTimeState2 newTimeState1" using
     CCSTimeout new_state_1 new_state_2 times_equal new_times_equal
     time_state_1_only_time_changes time_state_2_only_time_changes by auto
moreover have "choices newTimeState1 = choices state1" using new_state_1 by (cases "upper1 <
     lower1"; cases "upper1 < minTime state1"; cases "lower1 < minTime state1", auto)
moreover have "choices newTimeState2 = choices state2" using new_state_2 by (cases "upper1 <
     lower1"; cases "upper1 < minTime state2"; cases "lower1 < minTime state2", auto)
moreover have decompiled_step: "(When cases2 t tCont2, ctx2, state2, env2, warnings2, payments2
     ) →(inp, lower1, upper1) (tCont2, ctx2, newTimeState2, newTimeEnv2, warnings2, payments2)
     " using CCSTimeout(1,2) decompile_c2 small_step_receive_ccs.simps[of "(When cases2 t
     tCont2, ctx2, state2, env2, warnings2, payments2)" "(inp, lower1, upper1)" "(c2', ctx2',
     state2', env2', warnings2', payments2')"] apply (simp del: fixInterval.simps)
  using CCSTimeout(2) new_state_2 by force
moreover obtain realLower realUpper where real_time_bound: "timeInterval newTimeEnv2 = (
     realLower, realUpper) ∧ realLower ≥ t ∧ realUpper ≥ t" using CCSTimeout new_state_2
     decompile_c2 by (auto simp add: small_step_receive_ccs.simps)
moreover have "(c1, ctx1, state1, env1, warnings1, payments1) →(None, lower1, upper1) (tCont1,
      ctx1, newTimeState1, newTimeEnv1, warnings1, payments1)" using small_step_receive_ccs.
     CCSTimeout[of lower1 upper1 state1 newTimeEnv1 newTimeState1 realLower realUpper t _
     cases1 tCont1 ctx1 env1 warnings1 payments1] calculation decompile by auto
moreover have "(ChoiceId choiceName (Party.Role role)) ⊢ (c1, ctx1, state1, env1, warnings1,
```

```
      payments1) →(None, lower1, upper1) (tCont1, ctx1, newTimeState1, newTimeEnv1, warnings1,
      payments1)" using small_step_receive_ccs_add_choice.CCSTimeout calculation decompile by
      fastforce
  moreover have "split_cases_relation role choiceName k (tCont1, ctx1, newTimeState1, newTimeEnv1
      , warnings1, payments1) (tCont2, ctx2, newTimeState2, newTimeEnv2, warnings2, payments2)"
      using calculation by auto
  ultimately show ?case by (metis deterministicSmallStepReceive)
qed

lemma full_semantics_maintains_split_cases:
"A ⊢ (c1, ctx1, state1, env1, warnings1, payments1) −μ→ (c1', ctx1', state1', env1', warnings1',
     payments1') ⟹
split_cases_relation role choiceName k (c1, ctx1, state1, env1, warnings1, payments1) (c2, ctx2,
     state2, env2, warnings2, payments2) ⟹
  ∃c2' ctx2' state2' env2' warnings2' payments2'.
    ((A ∪ {(IChoice (ChoiceId choiceName (Party.Role role)) 0)}) ⊢ (c2, ctx2, state2, env2,
        warnings2, payments2) −μ→ (c2', ctx2', state2', env2', warnings2', payments2')) ∧
    split_cases_relation role choiceName k (c1', ctx1', state1', env1', warnings1', payments1') (
        c2', ctx2', state2', env2', warnings2', payments2')"
proof (induction A "(c1, ctx1, state1, env1, warnings1, payments1)" μ "(c1', ctx1', state1', env1
     ', warnings1', payments1')"
    arbitrary: c1 ctx1 state1 env1 warnings1 payments1 c1' ctx1' state1' env1' warnings1'
        payments1' c2 ctx2 state2 env2 warnings2 payments2
    rule: full_semantics.induct)
  case (BaseInternalFull A)
  then show ?case using full_semantics_react_is_equivalent_to_internal_reaction
      react_step_strongly_preserves_split_cases_relation by force
next
  case (BaseReceiveFull i A lower upper)
  moreover obtain c2' ctx2' state2' env2' warnings2' payments2' where "({(IChoice (ChoiceId
      choiceName (Party.Role role)) 0)} ⊢ (c2, ctx2, state2, env2, warnings2, payments2) −λ(Some
       i, lower, upper)→ (c2', ctx2', state2', env2', warnings2', payments2')) ∧
      split_cases_relation role choiceName k (c1', ctx1', state1', env1', warnings1', payments1
      ') (c2', ctx2', state2', env2', warnings2', payments2')"
    using receive_semantics_strongly_maintains_split_cases_NoChoiceLambdaReceive[of role
        choiceName k c1 c2 ctx1 ctx2 state1 state2 env1 env2 warnings1 warnings2 payments1
        payments2 i lower upper c1' ctx1' state1' env1' warnings1' payments1'] BaseReceiveFull
        by auto
  ultimately show ?case by (metis BaseReceiveFull(1) full_semantics_weakening inf_sup_aci(5))
next
  case (RestrictedReceiveFull i A μ lower upper p3)
  moreover obtain j innerC1 innerCtx1 innerState1 innerEnv1 innerWarnings1 innerPayments1 where
      innerP3_obtained: "j ∈ A ∧ (c1, ctx1, state1, env1, warnings1, payments1) →(Some j, lower
      , upper) p3 ∧ p3 = (innerC1, innerCtx1, innerState1, innerEnv1, innerWarnings1,
```

```
      innerPayments1)" using RestrictedReceiveFull by (cases p3, auto)
moreover have j_not_choice: "j ≠ (IChoice (ChoiceId choiceName (Party.Role role)) 0)" by (metis
       basic_receive_input_not_fresh_choice innerP3_obtained local.RestrictedReceiveFull(7)
      split_cases_relation.simps)
moreover obtain innerC2 innerCtx2 innerState2 innerEnv2 innerWarnings2 innerPayments2 where
      innerC2_obtained: "({IChoice (ChoiceId choiceName (Party.Role role)) 0} ⊢ (c2, ctx2,
      state2, env2, warnings2, payments2) −λ(Some j, lower, upper)→ (innerC2, innerCtx2,
      innerState2, innerEnv2, innerWarnings2, innerPayments2)) ∧ split_cases_relation role
      choiceName k p3 (innerC2, innerCtx2, innerState2, innerEnv2, innerWarnings2,
      innerPayments2)"
  using  receive_semantics_strongly_maintains_split_cases_NoChoiceLambdaReceive[of role
        choiceName k c1 c2 ctx1 ctx2 state1 state2 env1 env2 warnings1 warnings2 payments1
        payments2 j lower upper innerC1 innerCtx1 innerState1 innerEnv1 innerWarnings1
        innerPayments1] RestrictedReceiveFull j_not_choice innerP3_obtained by (auto simp del:
        state_minimal_eq.simps)
moreover obtain c2' ctx2' state2' env2' warnings2' payments2' where final_c2_step: "((A ∪ {
      IChoice (ChoiceId choiceName (Party.Role role)) 0}) ⊢ (innerC2, innerCtx2, innerState2,
      innerEnv2, innerWarnings2, innerPayments2) −λμ→ (c2', ctx2', state2', env2', warnings2',
      payments2')) ∧
      split_cases_relation role choiceName k (c1', ctx1', state1', env1', warnings1', payments1
           ') (c2', ctx2', state2', env2', warnings2', payments2')"
  using RestrictedReceiveFull(6)[of innerC1 innerCtx1 innerState1 innerEnv1 innerWarnings1
        innerPayments1 innerC2 innerCtx2 innerState2 innerEnv2 innerWarnings2 innerPayments2]
        innerC2_obtained innerP3_obtained by auto
moreover have "i ≠ (IChoice (ChoiceId choiceName (Party.Role role)) 0)" by (metis
      fresh_choice_no_match_full_semantics innerC2_obtained innerP3_obtained local.
      RestrictedReceiveFull(3) local.RestrictedReceiveFull(5) split_cases_relation.simps)
moreover have "\<nexists>f. (ChoiceId choiceName (Party.Role role)) ⊢ (c2, ctx2, state2, env2,
      warnings2, payments2) →(Some i, lower, upper) f" using
      split_cases_relation_backward_simulation_small_step_receive_add_choice
      choice_receive_same_as_basic_when_fresh RestrictedReceiveFull apply auto
  by meson
moreover have "(A ∪ {IChoice (ChoiceId choiceName (Party.Role role)) 0}) ⊢ (c2, ctx2, state2,
      env2, warnings2, payments2) −λ(Some i, lower, upper)→ (c2', ctx2', state2', env2',
      warnings2', payments2')"
    using full_semantics_union_composition_input[of "{IChoice (ChoiceId choiceName (Party.Role
          role)) 0}" "(c2, ctx2, state2, env2, warnings2, payments2)" j lower upper "(innerC2,
          innerCtx2, innerState2, innerEnv2, innerWarnings2, innerPayments2)" A "(Some i, lower,
           upper)" "(c2', ctx2', state2', env2', warnings2', payments2')" i]
    innerC2_obtained
    final_c2_step
    RestrictedReceiveFull(2)
    by (metis calculation(12) calculation(13) calculation(3) empty_iff
          full_semantics_receive_choice_input_is_equivalent_to_small_step_add_choice
```

```
          innerP3_obtained insert_iff)
  then show ?case using final_c2_step RestrictedReceiveFull by blast
next
  case (TimeoutFull lower upper A)
  moreover obtain cases1 t tCont1 where decompile_c1: "c1 = When cases1 t tCont1" using
       TimeoutFull by auto
  moreover obtain cases2 tCont2 where decompile_c2: "c2 = When cases2 t tCont2 ∧ tCont2 =
       split_cases role choiceName k tCont1" using decompile_c1 TimeoutFull(2) apply auto
    by (cases "1 < k ∧ k < length cases1", auto)
  moreover obtain newTimeState1 newTimeEnv1 newTimeState2 newTimeEnv2 where new_time_states: "
       fixInterval (lower, upper) state1 = IntervalTrimmed newTimeEnv1 newTimeState1 ∧
    fixInterval (lower, upper) state2 = IntervalTrimmed newTimeEnv2 newTimeState2 ∧
    state_minimal_eq choiceName newTimeState1 newTimeState2 ∧
    newTimeEnv1 = newTimeEnv2" using TimeoutFull apply auto
    apply (cases "upper < lower"; cases "upper < minTime state1"; cases "lower < minTime state1",
          auto)
    by (smt (verit, best) FState.select_convs(1) FState.select_convs(2) FState.select_convs(3)
         FState.select_convs(4) FState.select_convs(5) FState.surjective FState.update_convs(5))
  moreover obtain realLower realUpper where realLower_realUpper_obtained: "timeInterval
       newTimeEnv1 = (realLower, realUpper) ∧ realLower ≥ t ∧ realUpper ≥ t" using decompile_c1
       new_time_states TimeoutFull(1) by (auto simp add: small_step_receive_ccs.simps)
  moreover have "(c2, ctx2, state2, env2, warnings2, payments2) →(None, lower, upper) (tCont2,
       ctx2, newTimeState2, newTimeEnv2, warnings2, payments2)" using
       realLower_realUpper_obtained decompile_c2 new_time_states by (auto simp add:
       small_step_receive_ccs.simps)
  ultimately show ?case by (smt (z3) choice_receive_same_as_basic_when_fresh
       deterministicSmallStepReceive full_semantics.TimeoutFull small_step_receive_ccs_add_choice
       .CCSTimeout split_cases_relation.simps
       split_cases_relation_backward_simulation_small_step_receive_add_choice)
qed

lemma full_nd_semantics_maintains_split_cases:
"A ⊢ (c1, ctx1, state1, env1, warnings1, payments1) —μ→ (c1', ctx1', state1', env1', warnings1
     ', payments1') ⟹
split_cases_relation role choiceName k (c1, ctx1, state1, env1, warnings1, payments1) (c2, ctx2,
     state2, env2, warnings2, payments2) ⟹
  ∃c2' ctx2' state2' env2' warnings2' payments2'.
    ((A ∪ {(IChoice (ChoiceId choiceName (Party.Role role)) 0)}) ⊢ (c2, ctx2, state2, env2,
         warnings2, payments2) —μ→ (c2', ctx2', state2', env2', warnings2', payments2')) ∧
    split_cases_relation role choiceName k (c1', ctx1', state1', env1', warnings1', payments1') (
         c2', ctx2', state2', env2', warnings2', payments2')"
proof (induction A "(c1, ctx1, state1, env1, warnings1, payments1)" μ "(c1', ctx1', state1', env1
     ', warnings1', payments1')"
    arbitrary: c1 ctx1 state1 env1 warnings1 payments1 c1' ctx1' state1' env1' warnings1'
```

```
        payments1' c2 ctx2 state2 env2 warnings2 payments2
    rule: full_nd_semantics.induct)
  case (BaseInternalFullND A)
  then show ?case by (smt (verit, ccfv_SIG) full_nd_semantics.BaseInternalFullND
      internal_reaction.simps react_step_strongly_preserves_split_cases_relation)
next
  case (BaseReceiveFullND i A lower upper)
  moreover obtain c2' ctx2' state2' env2' warnings2' payments2' where "((ChoiceId choiceName (
      Party.Role role)) ⊢ (c2, ctx2, state2, env2, warnings2, payments2) →(Some i, lower, upper
      ) (c2', ctx2', state2', env2', warnings2', payments2')) ∧
    split_cases_relation role choiceName k (c1', ctx1', state1', env1', warnings1', payments1') (
        c2', ctx2', state2', env2', warnings2', payments2')"
    using BaseReceiveFullND
        receive_semantics_strongly_maintains_split_cases_NoChoiceLambdaReceive by (smt (verit,
        ccfv_SIG) full_semantics_maintains_split_cases
        full_semantics_receive_choice_input_is_equivalent_to_small_step_add_choice_backward
        full_semantics_receive_empty_set_is_equivalent_to_base_small_step sup_bot_left)
  ultimately show ?case by (metis full_nd_simulates_original
      full_semantics_receive_choice_input_is_equivalent_to_small_step_add_choice_forward
      full_semantics_weakening inf_sup_aci(5))
next
  case (RestrictedReceiveFullND i A μ lower upper p3)
  moreover obtain j innerC1 innerCtx1 innerState1 innerEnv1 innerWarnings1 innerPayments1 where
      innerP3_obtained: "j ∈ A ∧ (c1, ctx1, state1, env1, warnings1, payments1) →(Some j, lower
      , upper) p3 ∧ p3 = (innerC1, innerCtx1, innerState1, innerEnv1, innerWarnings1,
      innerPayments1)" using RestrictedReceiveFullND by (cases p3, auto)
  moreover have j_not_choice: "j ≠ (IChoice (ChoiceId choiceName (Party.Role role)) 0)" by (metis
       basic_receive_input_not_fresh_choice innerP3_obtained local.RestrictedReceiveFullND(6)
      split_cases_relation.simps)
  moreover obtain innerC2 innerCtx2 innerState2 innerEnv2 innerWarnings2 innerPayments2 where
      innerC2_obtained: "({IChoice (ChoiceId choiceName (Party.Role role)) 0} ⊢ (c2, ctx2,
      state2, env2, warnings2, payments2) −λ(Some j, lower, upper)→ (innerC2, innerCtx2,
      innerState2, innerEnv2, innerWarnings2, innerPayments2)) ∧ split_cases_relation role
      choiceName k p3 (innerC2, innerCtx2, innerState2, innerEnv2, innerWarnings2,
      innerPayments2)"
    using  receive_semantics_strongly_maintains_split_cases_NoChoiceLambdaReceive[of role
        choiceName k c1 c2 ctx1 ctx2 state1 state2 env1 env2 warnings1 warnings2 payments1
        payments2 j lower upper innerC1 innerCtx1 innerState1 innerEnv1 innerWarnings1
        innerPayments1] RestrictedReceiveFullND j_not_choice innerP3_obtained by (auto simp del:
         state_minimal_eq.simps)
  moreover obtain c2' ctx2' state2' env2' warnings2' payments2' where final_c2_step: "((A ∪ {
      IChoice (ChoiceId choiceName (Party.Role role)) 0}) ⊢ (innerC2, innerCtx2, innerState2,
      innerEnv2, innerWarnings2, innerPayments2) —λμ→ (c2', ctx2', state2', env2', warnings2',
       payments2')) ∧
```

```
      split_cases_relation role choiceName k (c1', ctx1', state1', env1', warnings1', payments1
           ') (c2', ctx2', state2', env2', warnings2', payments2')"
    using RestrictedReceiveFullND(5)[of innerC1 innerCtx1 innerState1 innerEnv1 innerWarnings1
         innerPayments1 innerC2 innerCtx2 innerState2 innerEnv2 innerWarnings2 innerPayments2]
         innerC2_obtained innerP3_obtained by auto
  moreover have "i ≠ (IChoice (ChoiceId choiceName (Party.Role role)) 0)" by (metis calculation
       (2) fresh_choice_no_match_full_nd_semantics innerC2_obtained innerP3_obtained local.
       RestrictedReceiveFullND(4) split_cases_relation.simps)
  moreover have "(A ∪ {IChoice (ChoiceId choiceName (Party.Role role)) 0}) ⊢ (c2, ctx2, state2,
       env2, warnings2, payments2) ⟶λ(Some i, lower, upper)→ (c2', ctx2', state2', env2',
       warnings2', payments2')"
      using full_nd_semantics_union_composition_input[of "{IChoice (ChoiceId choiceName (Party.
           Role role)) 0}" "(c2, ctx2, state2, env2, warnings2, payments2)" j lower upper "(
           innerC2, innerCtx2, innerState2, innerEnv2, innerWarnings2, innerPayments2)" A "(Some
           i, lower, upper)" "(c2', ctx2', state2', env2', warnings2', payments2')" i]
      innerC2_obtained
      final_c2_step
      RestrictedReceiveFullND(2)
      by (meson calculation(11) full_nd_simulates_original innerP3_obtained singletonD)
  then show ?case using final_c2_step RestrictedReceiveFullND by blast
next
  case (TimeoutFullND lower upper A)
  then show ?case by (smt (z3) TimeoutFull full_nd_simulates_original
       full_semantics_maintains_split_cases)
qed

(* == Backward simulation work == *)
definition disjoint_cases :: "FCase list ⇒ bool" where
"disjoint_cases cases ⟷ (∀inp state env res .
  inp ⊢ (cases, state, env) →cases_f2 res ⟶
  (∀ cases1 cases2 . cases = cases1 @ cases2 ⟶
    inp ⊢ (cases1, state, env) →cases_f2 res ∧ ¬(∃otherRes . inp ⊢ (cases2, state, env) →cases_f2
          otherRes) ∨
    inp ⊢ (cases2, state, env) →cases_f2 res ∧ ¬(∃otherRes . inp ⊢ (cases1, state, env) →cases_f2
          otherRes)))"

lemma single_case_disjoint:
"disjoint_cases [c]"
  apply (auto simp add: disjoint_cases_def)
  by (metis (no_types, lifting) append_eq_Cons_conv append_is_Nil_conv computeCasesStep.simps(1)
       option.distinct(1) reduceCasesStepImpliesComputeCaseStepRes)+

lemma slice_cases_preserves_disjoint_cases:
"disjoint_cases (cases1 @ cases2) ⟹
```

```
  ¬disjoint_cases cases2 ⟹
  False"
  subgoal premises ps proof -
    obtain cases21 cases22 state env res1 res2 inp where "cases2 = cases21 @ cases22 ∧
      (inp ⊢ (cases21, state, env) →cases_f2 res1 ∧ (inp ⊢ (cases22, state, env) →cases_f2 res2))
           " using ps(2) apply (auto simp add: disjoint_cases_def)
        apply (meson small_step_cases_split_list_matches_one_side)
       apply (metis small_step_cases_split_list_matches_one_side)
      by (metis small_step_cases_split_list_matches_one_side)
    moreover obtain res3 where "inp ⊢ (cases1@cases21, state, env) →cases_f2 res3" apply auto
      by (metis calculation casesStepAppendFinalCases casesStepImpliesAppendCasesStep
           finalFCases_def surj_pair)
    moreover obtain cases121 where "cases121 = cases1@cases21" by auto
    moreover have new_slice: "cases1 @ cases2 = cases121 @ cases22" using calculation by auto
    moreover have early_step: "inp ⊢ (cases121, state, env) →cases_f2 res3" using calculation by
         auto
    moreover have late_step: "(inp ⊢ (cases22, state, env) →cases_f2 res2)" using calculation by
         auto
    moreover obtain cases where full_cases: "cases = cases121@cases22" using ps by auto
    moreover have full_cases_disjoint: "disjoint_cases cases" using new_slice ps(1) full_cases by
          auto
    moreover show ?thesis using full_cases_disjoint full_cases early_step late_step apply (auto
         simp add: disjoint_cases_def)
      by (smt (verit, best) FaustusV2Semantics.casesStepImpliesAppendCasesStep prod_cases3)
  qed
  done

fun disjoint_cases_contract :: "FContract ⇒ bool" and disjoint_cases_in_cases :: "FCase list ⇒
     bool" where
"disjoint_cases_contract (When cases t cont) = (disjoint_cases_in_cases cases ∧ disjoint_cases
     cases ∧ disjoint_cases_contract cont)" |
"disjoint_cases_contract Close = True" |
"disjoint_cases_contract (StatementCont s cont) = disjoint_cases_contract cont" |
"disjoint_cases_contract (If obs trueCont falseCont) = (disjoint_cases_contract trueCont ∧
     disjoint_cases_contract falseCont)" |
"disjoint_cases_contract (Let vid val cont) = disjoint_cases_contract cont" |
"disjoint_cases_contract (LetObservation obsId obs cont) = disjoint_cases_contract cont" |
"disjoint_cases_contract (LetParty pkId pk cont) = disjoint_cases_contract cont" |
"disjoint_cases_contract (LetC cid params bodyCont nextCont) = (disjoint_cases_contract bodyCont
     ∧ disjoint_cases_contract nextCont)" |
"disjoint_cases_contract (UseC cid args) = True" |
"disjoint_cases_in_cases [] = True" |
"disjoint_cases_in_cases ((Case g c)#cases) = (disjoint_cases_contract c ∧
     disjoint_cases_in_cases cases)"
```

```
fun disjoint_cases_context :: "FContext ⇒ bool" where
"disjoint_cases_context [] = True" |
"disjoint_cases_context ((ContractAbstraction (cid, params, bodyCont))#rest) = (
     disjoint_cases_contract bodyCont ∧ disjoint_cases_context rest)" |
"disjoint_cases_context ((ValueAbstraction vid)#rest) = disjoint_cases_context rest" |
"disjoint_cases_context ((ObservationAbstraction obsId)#rest) = disjoint_cases_context rest" |
"disjoint_cases_context ((PartyAbstraction pkId)#rest) = disjoint_cases_context rest"

fun disjoint_cases_process :: "FConfig ⇒ bool" where
"disjoint_cases_process (cont, ctx, state, env, warnings, payments) = (disjoint_cases_contract
     cont ∧ disjoint_cases_context ctx)"

fun split_cases_relation_disjoint_cases :: "TokenName ⇒ ChoiceName ⇒ nat ⇒ FConfig ⇒ FConfig
     ⇒ bool" where
"split_cases_relation_disjoint_cases role choiceName k
  (c1, ctx1, s1, env1, warnings1, payments1)
  (c2, ctx2, s2, env2, warnings2, payments2) =
    (disjoint_cases_process (c1, ctx1, s1, env1, warnings1, payments1) ∧
    split_cases_relation role choiceName k (c1, ctx1, s1, env1, warnings1, payments1) (c2, ctx2,
         s2, env2, warnings2, payments2))"

fun split_cases_relation_disjoint_cases_set :: "TokenName ⇒ ChoiceName ⇒ nat ⇒ FConfig rel"
     where
"split_cases_relation_disjoint_cases_set role choiceName k =  {(p, q) | p q .
     disjoint_cases_process p} \<inter> (split_cases_relation_set role choiceName k)"

lemma split_cases_relation_disjoint_cases_gives_set:
"split_cases_relation_disjoint_cases role choiceName k p q ⟷ (p, q) ∈ (
     split_cases_relation_disjoint_cases_set role choiceName k)"
  by (cases p; cases q; auto)

lemma disjoint_cases_context_append:
"disjoint_cases_context ctx1 ⟹
  disjoint_cases_context ctx2 ⟹
  disjoint_cases_context (ctx1 @ ctx2)"
  by (induction ctx1 rule: disjoint_cases_context.induct, auto)

lemma small_step_reduce_preserves_disjoint_cases:
"(cont1, ctx1, state1, env1, warnings1, payments1) →f2 (cont2, ctx2, state2, env2, warnings2,
     payments2) ⟹
  disjoint_cases_process (cont1, ctx1, state1, env1, convertReduceWarnings warnings1, payments1)
       ⟹
  disjoint_cases_process (cont2, ctx2, state2, env2, convertReduceWarnings warnings2, payments2)"
```

```
proof (induction cont1 ctx1 state1 env1 warnings1 payments1 cont2 ctx2 state2 env2 warnings2
     payments2 rule: small_step_reduce.induct[split_format(complete)])
  case (UseCFound ctx cid params bodyCont innerCtx env s args newState warns payments)
  moreover have "disjoint_cases_contract bodyCont ∧ disjoint_cases_context innerCtx" using
       calculation(1,3) by (induction ctx cid rule: lookupContractIdAbsInformation.induct, auto
       split: if_split if_split_asm)
  moreover have "disjoint_cases_context (paramsToFContext params)" by (induction params rule:
       paramsToFContext.induct, auto)
  ultimately show ?case using disjoint_cases_context_append by auto
qed auto

lemma prependStatements_preserves_disjoint_cases:
"disjoint_cases_contract c ⟹
  disjoint_cases_contract (prependStatements stmts c)"
  by (induction stmts c rule: prependStatements.induct, auto)

lemma small_step_case_preserves_disjoint_cases_relation:
"inp ⊢ (c1, state1, env1) →case_f2 (c1', state1', newW) ⟹
  disjoint_cases_in_cases [c1] ⟹
  disjoint_cases_contract tc ⟹
  disjoint_cases_contract (c1' t tc)"
proof (induction inp c1 state1 env1 c1' state1' newW rule: small_step_reduce_case.induct[
     split_format(complete)])
  case (CaseGuardCompleteStep)
  then show ?case using prependStatements_preserves_disjoint_cases by simp
next
  case (CaseGuardIncompleteStep ins g s env stmts innerG newS newW c)
  moreover have "disjoint_cases_in_cases [FaustusV2AST.FCase.Case innerG c]" using calculation by
        auto
  moreover have "disjoint_cases_contract (When [FaustusV2AST.FCase.Case innerG c] t tc)" using
       calculation single_case_disjoint by auto
  moreover have "disjoint_cases_contract (prependStatements stmts (When [FaustusV2AST.FCase.Case
       innerG c] t tc))" using calculation prependStatements_preserves_disjoint_cases by auto
  ultimately show ?case by blast
qed

lemma small_step_cases_preserves_disjoint_cases_relation:
"inp ⊢ (c1, state1, env1) →cases_f2 (c1', state1', newW) ⟹
  disjoint_cases_in_cases c1 ⟹
  disjoint_cases_context ctx1 ⟹
  disjoint_cases_contract tc ⟹
  disjoint_cases_contract (c1' t tc)"
proof (induction inp c1 state1 env1 c1' state1' newW rule: small_step_reduce_cases.induct[
     split_format(complete)])
```

```
  case (CasesHeadMatches )
  then show ?case by (metis FaustusV2AST.FCase.exhaust disjoint_cases_in_cases.simps(1)
       disjoint_cases_in_cases.simps(2) small_step_case_preserves_disjoint_cases_relation)
next
  case (CasesTailMatches)
  then show ?case using disjoint_cases_in_cases.elims(2) by blast
qed

lemma receive_preserves_disjoint_cases_relation:
"(c1, ctx1, state1, env1, reduceWarnings1, payments1) →(inp, lower, upper) (c1', ctx1', state1',
      env1', reduceWarnings1', payments1') ⟹
  disjoint_cases_contract c1 ⟹
  disjoint_cases_context ctx1 ⟹
  disjoint_cases_contract c1' ∧ disjoint_cases_context ctx1'"
proof (induction c1 ctx1 state1 env1 reduceWarnings1 payments1 inp lower upper c1' ctx1' state1'
     env1' reduceWarnings1' payments1' rule: small_step_receive_ccs.induct[split_format(complete)
     ])
  case (LambdaReceive )
  then show ?case using disjoint_cases_contract.simps(1)
       small_step_cases_preserves_disjoint_cases_relation by blast
next
  case (CCSTimeout lower upper state newTimeEnv newTimeState realLower realUpper t
       freshChoiceName cases c ctx env warnings payments)
  then show ?case by auto
qed

lemma receive_choice_preserves_disjoint_cases_relation:
"choiceName ⊢ (c1, ctx1, state1, env1, reduceWarnings1, payments1) →(inp, lower, upper) (c1',
     ctx1', state1', env1', reduceWarnings1', payments1') ⟹
  disjoint_cases_contract c1 ⟹
  disjoint_cases_context ctx1 ⟹
  disjoint_cases_contract c1' ∧ disjoint_cases_context ctx1'"
proof (induction choiceName c1 ctx1 state1 env1 reduceWarnings1 payments1 inp lower upper c1'
     ctx1' state1' env1' reduceWarnings1' payments1' rule: small_step_receive_ccs_add_choice.
     induct[split_format(complete)])
  case (NoChoiceLambdaReceive )
  then show ?case using disjoint_cases_contract.simps(1)
       small_step_cases_preserves_disjoint_cases_relation by blast
next
  case (ChoiceLambdaReceive)
  then show ?case by (metis receive_preserves_disjoint_cases_relation)
next
  case (CCSTimeout lower upper state newTimeEnv newTimeState realLower realUpper t
       freshChoiceName cases c ctx env warnings payments)
```

```
  then show ?case by auto
qed

lemma receive_disjoint_cases_append:
"(When cases t cont, ctx1, state, env, warnings, payments) →(i, lower, upper) (c1', ctx1',
     state1', env1', warnings1', payments1') ⟹
  disjoint_cases (cases2 @ cases) ⟹
  (When (cases2 @ cases) t cont, ctx1, state, env, warnings, payments) →(i, lower, upper) (c1',
       ctx1', state1', env1', warnings1', payments1')"
proof (induction "When cases t cont" ctx1 state env warnings payments i lower upper c1' ctx1'
     state1' env1' warnings1' payments1' rule: small_step_receive_ccs.induct[split_format(
     complete)])
  case (LambdaReceive lower upper state newTimeEnv newTimeState realLower realUpper inp newC
       newState i1 lower1 upper1 ctx env warnings payments)
  moreover have "finalFCases inp (cases2, newTimeState, newTimeEnv)" using LambdaReceive(5,7)
       finalFCases_def disjoint_cases_def apply auto
    by (metis (no_types, opaque_lifting) casesStepImpliesAppendCasesStep)
  ultimately show ?case using casesStepAppendFinalCases small_step_receive_ccs.LambdaReceive by
       presburger
next
  case (CCSTimeout)
  then show ?case using small_step_receive_ccs.CCSTimeout by blast
qed

lemma full_nd_semantics_disjoint_cases_append:
"A ⊢ (When cases t cont, ctx1, state, env, warnings, payments) —μ→ (c1', ctx1', state1', env1',
      warnings1', payments1') ⟹
  disjoint_cases (cases2 @ cases) ⟹
  A ⊢ (When (cases2 @ cases) t cont, ctx1, state, env, warnings, payments) —μ→ (c1', ctx1',
       state1', env1', warnings1', payments1')"
proof (induction "When cases t cont" ctx1 state env warnings payments μ c1' ctx1' state1' env1'
     warnings1' payments1' rule: full_nd_semantics.induct[split_format(complete)])
  case (BaseInternalFullND ctx1 state1 env1 warnings1 payments1 c1' ctx1' state1' env1' warnings1
       ' payments1' A)
  moreover have "(FaustusV2AST.FContract.When (cases2 @ cases) t cont, ctx1, state1, env1,
       warnings1, payments1) →𝒯 (c1', ctx1', state1', env1', warnings1', payments1')" using
       BaseInternalFullND(1) internal_reaction.simps by auto
  ultimately show ?case using full_nd_semantics.BaseInternalFullND by blast
next
  case (BaseReceiveFullND i A ctx1 state1 env1 warnings1 payments1 lower upper c1' ctx1' state1'
       env1' warnings1' payments1')
  then show ?case using receive_disjoint_cases_append by (simp add: full_nd_semantics.
       BaseReceiveFullND)
next
```

```
  case (RestrictedReceiveFullND i A a a b lower upper a a a a b a a a a a b a a a a a b)
  then show ?case by (metis (no_types, lifting) full_nd_semantics.RestrictedReceiveFullND
       receive_disjoint_cases_append)
next
  case (TimeoutFullND a a a a b lower upper a a a a a b A)
  then show ?case by (simp add: full_nd_semantics.TimeoutFullND receive_disjoint_cases_append)
qed

lemma full_semantics_preserves_disjoint_cases_relation:
"A ⊢ (c1, ctx1, state1, env1, reduceWarnings1, payments1) −μ→ (c1', ctx1', state1', env1',
     reduceWarnings1', payments1') ⟹
  disjoint_cases_contract c1 ⟹
  disjoint_cases_context ctx1 ⟹
  disjoint_cases_contract c1' ∧ disjoint_cases_context ctx1'"
proof (induction A c1 ctx1 state1 env1 reduceWarnings1 payments1 μ c1' ctx1' state1' env1'
     reduceWarnings1' payments1' rule: full_semantics.induct[split_format(complete)])
  case (BaseInternalFull)
  then show ?case using internal_reaction.simps small_step_reduce_preserves_disjoint_cases by
       auto
next
  case (BaseReceiveFull)
  then show ?case using receive_preserves_disjoint_cases_relation by force
next
  case (RestrictedReceiveFull)
  then show ?case by (meson receive_preserves_disjoint_cases_relation)
next
  case (TimeoutFull)
  then show ?case by auto
qed

lemma full_nd_semantics_preserves_disjoint_cases_relation:
"A ⊢ (c1, ctx1, state1, env1, reduceWarnings1, payments1) ⟜μ→ (c1', ctx1', state1', env1',
     reduceWarnings1', payments1') ⟹
  disjoint_cases_contract c1 ⟹
  disjoint_cases_context ctx1 ⟹
  disjoint_cases_contract c1' ∧ disjoint_cases_context ctx1'"
proof (induction A c1 ctx1 state1 env1 reduceWarnings1 payments1 μ c1' ctx1' state1' env1'
     reduceWarnings1' payments1' rule: full_nd_semantics.induct[split_format(complete)])
  case (BaseInternalFullND)
  then show ?case using internal_reaction.simps small_step_reduce_preserves_disjoint_cases by
       auto
next
  case (BaseReceiveFullND)
  then show ?case using receive_preserves_disjoint_cases_relation by force
```

```
next
  case (RestrictedReceiveFullND)
  then show ?case by (meson receive_preserves_disjoint_cases_relation)
next
  case (TimeoutFullND)
  then show ?case by auto
qed

lemma computeGuardStepChoiceNoAddedWarnings:
"computeGuardStep (IChoice choiceId val) (g, newTimeState, env2) = Some (stmts, gOption, newS,
     newW) ⟹
  newW = ApplyNoWarning"
proof (induction "(IChoice choiceId val)" "(g, newTimeState, env2)" arbitrary: g gOption stmts
     rule: computeGuardStep.induct)
  case (1 a)
  then show ?case by (cases a, auto split: option.split_asm if_split_asm)
qed (auto split: option.split_asm if_split_asm)

lemma receiveChoiceNoAddedWarnings:
"(c1, ctx1, state1, env1, warnings1, payments1) →(Some (IChoice choiceId val), lower, upper) (c2
     , ctx2, state2, env2, warnings2, payments2) ⟹
  warnings1 = warnings2"
proof (induction "(c1, ctx1, state1, env1, warnings1, payments1)" "(Some (IChoice choiceId val),
     lower, upper)" "(c2, ctx2, state2, env2, warnings2, payments2)" rule: small_step_receive_ccs
     .induct)
  case (LambdaReceive lower upper newTimeState realLower realUpper t inp cases newC newWarn c)
  then show ?case apply auto
    subgoal premises ps proof -
      have "computeCasesStep (IChoice choiceId val) (cases, newTimeState, env2) = Some (newC,
           state2, newWarn)" using FaustusV2Semantics.reduceCasesStepImpliesComputeCaseStepRes ps
           (5) by blast
      moreover have "newWarn = ApplyNoWarning" using calculation proof (induction "(IChoice
           choiceId val)" "(cases, newTimeState, env2)" arbitrary: cases rule: computeCasesStep.
           induct)
        case (2 c cs)
        then show ?case apply (auto split: option.split_asm)
          subgoal premises ps proof (cases c)
            case (Case g cont)
            then show ?thesis using ps by (auto split: option.split_asm simp add:
                 computeGuardStepChoiceNoAddedWarnings)
          qed
          done
      qed auto
      ultimately show ?thesis by force
```

```
    qed
    done
qed auto

lemma full_nd_semantics_preserves_split_cases_disjoint_backwards:
"((A ∪ {(IChoice (ChoiceId choiceName (Party.Role role)) 0)}) ⊢ (c2, ctx2, state2, env2,
     warnings2, payments2) —μ→ (c2', ctx2', state2', env2', warnings2', payments2')) ⟹
split_cases_relation_disjoint_cases role choiceName k (c1, ctx1, state1, env1, warnings1,
     payments1) (c2, ctx2, state2, env2, warnings2, payments2) ⟹
  ∃c1' ctx1' state1' env1' warnings1' payments1'.
    (A ⊢ (c1, ctx1, state1, env1, warnings1, payments1) —μ→ (c1', ctx1', state1', env1',
         warnings1', payments1')) ∧
    split_cases_relation_disjoint_cases role choiceName k (c1', ctx1', state1', env1', warnings1
         ', payments1') (c2', ctx2', state2', env2', warnings2', payments2')"
proof (induction "A ∪ {(IChoice (ChoiceId choiceName (Party.Role role)) 0)}" "(c2, ctx2, state2,
     env2, warnings2, payments2)" μ "(c2', ctx2', state2', env2', warnings2', payments2')"
    arbitrary: c1 ctx1 state1 env1 warnings1 payments1 c2' ctx2' state2' env2' warnings2'
         payments2' c2 ctx2 state2 env2 warnings2 payments2
    rule: full_nd_semantics.induct)
  case BaseInternalFullND
  then show ?case by (smt (z3) FaustusV2Sim.internal_reaction.simps disjoint_cases_process.simps
       full_nd_semantics.BaseInternalFullND small_step_preserves_inverse_split_cases_relation
       small_step_reduce_preserves_disjoint_cases split_cases_relation_disjoint_cases.simps)
next
  case (BaseReceiveFullND i lower upper)
  moreover have "i ≠ IChoice (ChoiceId choiceName (Party.Role role)) 0" using calculation(1) by
       fastforce
  moreover have "(ChoiceId choiceName (Party.Role role)) ⊢ (c2, ctx2, state2, env2, warnings2,
       payments2) →(Some i, lower, upper) (c2', ctx2', state2', env2', warnings2', payments2')"
       using NoChoiceLambdaReceive calculation(2) calculation(4) by presburger
  moreover obtain c1' ctx1' state1' env1' warnings1' payments1' where "((ChoiceId choiceName (
       Party.Role role)) ⊢ (c1, ctx1, state1, env1, warnings1, payments1) →(Some i, lower, upper
       ) (c1', ctx1', state1', env1', warnings1', payments1')) ∧
    split_cases_relation_disjoint_cases role choiceName k (c1', ctx1', state1', env1', warnings1
         ', payments1') (c2', ctx2', state2', env2', warnings2', payments2')" using
         split_cases_relation_backward_simulation_small_step_receive_add_choice BaseReceiveFull
         calculation by (metis disjoint_cases_process.simps
         receive_choice_preserves_disjoint_cases_relation split_cases_relation_disjoint_cases.
         simps)
  moreover have "(c1, ctx1, state1, env1, warnings1, payments1) →(Some i, lower, upper) (c1',
       ctx1', state1', env1', warnings1', payments1')" by (meson BaseReceiveFullND.prems
       calculation(6) choice_receive_same_as_basic_when_fresh split_cases_relation.simps
       split_cases_relation_disjoint_cases.simps)
  ultimately show ?case by (meson UnI1 full_nd_semantics.BaseReceiveFullND)
```

```
next
  case (RestrictedReceiveFullND i μ lower upper p3)
  moreover obtain j where j_obtained: "j ∈ A ∪ {IChoice (ChoiceId choiceName (Party.Role role))
       0} ∧ (c2, ctx2, state2, env2, warnings2, payments2) →(Some j, lower, upper) p3" using
       RestrictedReceiveFullND(3) by auto
  moreover have j_not_choice_small_step_receive_add_choice_to_p3: "j ≠ IChoice (ChoiceId
       choiceName (Party.Role role)) 0 ⟶ ((ChoiceId choiceName (Party.Role role)) ⊢ (c2, ctx2,
       state2, env2, warnings2, payments2) →(Some j, lower, upper) p3)" using
       NoChoiceLambdaReceive j_obtained by auto
  moreover obtain c1' ctx1' state1' env1' warnings1' payments1' where j_not_choice_c1_inner_step:
        "j ≠ IChoice (ChoiceId choiceName (Party.Role role)) 0 ⟶ ((ChoiceId choiceName (Party.
       Role role)) ⊢ (c1, ctx1, state1, env1, warnings1, payments1) →(Some j, lower, upper) (c1
       ', ctx1', state1', env1', warnings1', payments1')) ∧
      split_cases_relation role choiceName k (c1', ctx1', state1', env1', warnings1', payments1')
            p3"
    using split_cases_relation_backward_simulation_small_step_receive_add_choice[of choiceName
         role c2 ctx2 state2 env2 warnings2 payments2 "Some j" lower upper _ _ _ _ _ _ k c1 ctx1
         state1 env1 warnings1 payments1] j_obtained
         j_not_choice_small_step_receive_add_choice_to_p3 RestrictedReceiveFullND(6) by (smt (
         verit, del_insts) disjoint_cases_process.cases split_cases_relation_disjoint_cases.simps
         )
  moreover have j_not_choice_inner_step_disjoint: "j ≠ IChoice (ChoiceId choiceName (Party.Role
       role)) 0 ⟶ disjoint_cases_contract c1' ∧ disjoint_cases_context ctx1'" using
       receive_choice_preserves_disjoint_cases_relation[of "(ChoiceId choiceName (Party.Role role
       ))" c1 ctx1 state1 env1 warnings1 payments1 "Some j" lower upper c1' ctx1' state1' env1'
       warnings1' payments1'] j_not_choice_c1_inner_step local.RestrictedReceiveFullND(6)
    by (auto simp only: split_cases_relation_disjoint_cases.simps disjoint_cases_process.simps)
  moreover obtain finalC1' finalCtx1' finalState1' finalEnv1' finalWarnings1' finalPayments1'
       where j_not_choice_c1_final_step: "j ≠ IChoice (ChoiceId choiceName (Party.Role role)) 0
       ⟶ A ⊢ (c1', ctx1', state1', env1', warnings1', payments1') ⟶λμ→ (finalC1', finalCtx1',
        finalState1', finalEnv1', finalWarnings1', finalPayments1') ∧
    split_cases_relation_disjoint_cases role choiceName k (finalC1', finalCtx1', finalState1',
         finalEnv1', finalWarnings1', finalPayments1') (c2', ctx2', state2', env2', warnings2',
         payments2')"
    using RestrictedReceiveFullND(5)[of _ _ _ _ _ _ c1' ctx1' state1' env1' warnings1' payments1
         '] j_not_choice_c1_inner_step j_not_choice_inner_step_disjoint apply (cases p3)
    by (metis j_not_choice_inner_step_disjoint disjoint_cases_process.simps
         split_cases_relation_disjoint_cases.simps)
  moreover have "j ≠ IChoice (ChoiceId choiceName (Party.Role role)) 0 ⟶ A ⊢ (c1, ctx1, state1,
        env1, warnings1, payments1) ⟶λμ→ (finalC1', finalCtx1', finalState1', finalEnv1',
       finalWarnings1', finalPayments1')" using j_not_choice_c1_final_step full_nd_semantics.
       RestrictedReceiveFullND RestrictedReceiveFullND by (metis (no_types, lifting)
       Un_insert_right choice_receive_same_as_basic_when_fresh insert_iff
       j_not_choice_c1_inner_step j_obtained split_cases_relation.simps
```

```
      split_cases_relation_disjoint_cases.simps sup_bot_right)
moreover obtain tCont2 cases2 t where decompile_c2: "c2 = When cases2 t tCont2" using
      RestrictedReceiveFullND by auto
moreover obtain cases1 tCont1 where decompile: "c1 = When cases1 t tCont1 ∧
  split_cases role choiceName k tCont1 = tCont2" using RestrictedReceiveFullND(6) decompile_c2
        apply (cases c1, auto)
  by (smt (verit) FaustusV2AST.FContract.inject(3))
moreover obtain newTimeEnv1 newTimeState1 newTimeState2 newTimeEnv2 where new_states: "
      fixInterval (lower, upper) state1 = IntervalTrimmed newTimeEnv1 newTimeState1 ∧
      fixInterval (lower, upper) state2 = IntervalTrimmed newTimeEnv2 newTimeState2" using
      RestrictedReceiveFullND apply auto
  by (cases "upper < lower"; cases "upper < minTime state1"; cases "lower < minTime state1",
        auto)+
moreover have times_equal: "minTime state1 = minTime state2" using RestrictedReceiveFullND by
      auto
moreover have new_times_equal: "newTimeEnv1 = newTimeEnv2 ∧ minTime newTimeState1 = minTime
      newTimeState2" using times_equal new_states fixFIntervalOnlyTimeDependent by metis
moreover have time_state_1_only_time_changes: "choices newTimeState1 = choices state1 ∧
  accounts newTimeState1 = accounts state1 ∧
  boundValues newTimeState1 = boundValues state1 ∧
  boundParties newTimeState1 = boundParties state1" using new_states apply auto
  by (cases "upper < lower"; cases "upper < minTime state1"; cases "lower < minTime state1",
        auto)+
moreover have time_state_2_only_time_changes: "choices newTimeState2 = choices state2 ∧
  accounts newTimeState2 = accounts state2 ∧
  boundValues newTimeState2 = boundValues state2 ∧
  boundParties newTimeState2 = boundParties state2" using new_states apply auto
  by (cases "upper < lower"; cases "upper < minTime state1"; cases "lower < minTime state1";
        cases "upper < minTime state2"; cases "lower < minTime state2", auto)+
moreover have newTimeStatesEq: "state_minimal_eq choiceName newTimeState2 newTimeState1" using
      RestrictedReceiveFullND new_states times_equal new_times_equal
      time_state_1_only_time_changes time_state_2_only_time_changes by auto
moreover have fresh_choice_in_cases1: "isFreshChoiceInCasesGuards choiceName cases1" using
      calculation isFreshChoiceInCasesImpliesIsFreshChoiceInCasesGuards by auto
moreover have "split_cases_context role choiceName k ctx1 = ctx2" using calculation by auto
moreover obtain newCases where newCases_obtained: "newCases = map (λ(Case g c) ⇒ (Case g (
      split_cases role choiceName k c))) cases1" using calculation by auto
moreover obtain takeNewCases dropNewCases where split_newCases_obtained: "take k newCases =
      takeNewCases ∧ drop k newCases = dropNewCases" by auto
moreover have cases2_is_takeNewCases_k_size: "(k > 1 ∧ length cases1 > k) ⟶ cases2 = (
      takeNewCases @ [Case (ActionGuard (Choice (FChoiceId choiceName (Role role)) [(0, 0)])) (
      split_cases role choiceName k (When (drop k cases1) t tCont1))])" using
      RestrictedReceiveFullND(6) decompile_c2 decompile split_newCases_obtained
      newCases_obtained apply auto
```

```
  by metis
moreover have "isFreshChoiceInCasesGuardsShallow choiceName (map (λ(Case g c) ⇒ (Case g (
     split_cases role choiceName k c))) cases1)" using fresh_choice_in_cases1 by (induction
     cases1, auto split: FCase.split)
moreover have "(∀ party i . finalFCases (IChoice (ChoiceId choiceName party) i) (takeNewCases,
     newTimeState2, newTimeEnv2))" using calculation by (metis append_take_drop_id
     freshChoiceShallowNoMatch_cases_step isFreshChoiceInCasesGuardsShallowPreservedInSlice)
moreover have choiceActionStep: "(IChoice (ChoiceId choiceName (Party.Role role)) 0) ⊢ (Choice
     (FChoiceId choiceName (Role role)) [(0, 0)], newTimeState2, newTimeEnv2) →a_f2 (
     newTimeState2⦇choices := MList.insert (ChoiceId choiceName (Party.Role role)) 0 (choices
     newTimeState2) ⦈, ApplyNoWarning)" by (simp add: small_step_action_fresh_choice_state_eq)
moreover obtain newChoiceState where new_choice_state: "newTimeState2⦇choices := MList.insert (
     ChoiceId choiceName (Party.Role role)) 0 (choices newTimeState2) ⦈ = newChoiceState ∧
     state_minimal_eq choiceName newTimeState2 newChoiceState" using
     small_step_action_fresh_choice_state_eq by auto
moreover have choiceGuardStep: "(IChoice (ChoiceId choiceName (Party.Role role)) 0) ⊢ (
     ActionGuard (Choice (FChoiceId choiceName (Role role)) [(0, 0)]), newTimeState2,
     newTimeEnv2) →g_f2 ([], None, newChoiceState, ApplyNoWarning)" by (metis ActionGuardStep
     append.right_neutral choiceActionStep convertApplyWarning.simps(1) new_choice_state)
moreover have choiceCaseStep: "(IChoice (ChoiceId choiceName (Party.Role role)) 0) ⊢ (Case (
     ActionGuard (Choice (FChoiceId choiceName (Role role)) [(0, 0)])) (split_cases role
     choiceName k (When (drop k cases1) t tCont1)), newTimeState2, newTimeEnv2) →case_f2 (λ
     innerT innerTCont . prependStatements [] (split_cases role choiceName k (When (drop k
     cases1) t tCont1)), newChoiceState, ApplyNoWarning)" using CaseGuardCompleteStep
     choiceGuardStep by meson
moreover have choiceCasesStep: "(IChoice (ChoiceId choiceName (Party.Role role)) 0) ⊢ ([Case (
     ActionGuard (Choice (FChoiceId choiceName (Role role)) [(0, 0)])) (split_cases role
     choiceName k (When (drop k cases1) t tCont1))], newTimeState2, newTimeEnv2) →cases_f2 (λ
     innerT innerTCont . prependStatements [] (split_cases role choiceName k (When (drop k
     cases1) t tCont1)), newChoiceState, ApplyNoWarning)" using CasesHeadMatches choiceCaseStep
      by force
moreover have partyRoleIZero: "∀res . computeCasesStep (IChoice (ChoiceId choiceName party) l)
     ([Case (ActionGuard (Choice (FChoiceId choiceName (Role role)) [(0, 0)])) (split_cases
     role choiceName k (When (drop k cases1) t tCont1))], newTimeState2, newTimeEnv2) = Some
     res ⟶ party = (Party.Role role) ∧ l = 0" by auto
moreover have final_new_cases: "finalFCases (IChoice (ChoiceId choiceName (Party.Role role)) 0)
      (newCases, newTimeState2, newTimeEnv2)" by (simp add: calculation
     freshChoiceShallowNoMatch_cases_step)
moreover have newCases_fails: "((When newCases t tCont2, ctx2, state2, env2, warnings2,
     payments2) →(Some (IChoice (ChoiceId choiceName (Party.Role role)) 0), lower, upper) f)
     ⟹ False" apply (induction "(When newCases t tCont2, ctx2, state2, env2, warnings2,
     payments2)" "(Some (IChoice (ChoiceId choiceName (Party.Role role)) 0), lower, upper)" "f"
      rule: small_step_receive_ccs.induct)
  using final_new_cases apply auto
```

```
  using calculation(19) calculation(23) freshChoiceShallowMatchNoChoiceInput_cases_step
        calculation(26) by blast
moreover have cases2_not_newCases: "j = IChoice (ChoiceId choiceName (Party.Role role)) 0 ⟶
      cases2 ≠ newCases" using final_new_cases newCases_fails[of ] decompile_c2
      RestrictedReceiveFullND(2) small_step_receive_ccs.simps apply auto
  by (metis ReceiveWhenSomeElim calculation(26) freshChoiceShallowNoMatch_cases_step_aux
        j_obtained newCases_obtained prod_cases6)
moreover have "cases2 = newCases ∨ cases2 = (takeNewCases @ [Case (ActionGuard (Choice (
      FChoiceId choiceName (Role role)) [(0, 0)])) (split_cases role choiceName k (When (drop k
      cases1) t tCont1))])" using decompile_c2 decompile RestrictedReceiveFullND(6)
      newCases_obtained split_newCases_obtained apply (auto)
  apply (cases "1 < k ∧ k < length cases1")
   apply auto
  by metis
moreover have cases2_is_takeNewCases: "j = IChoice (ChoiceId choiceName (Party.Role role)) 0
      ⟶ cases2 = (takeNewCases @ [Case (ActionGuard (Choice (FChoiceId choiceName (Role role))
       [(0, 0)])) (split_cases role choiceName k (When (drop k cases1) t tCont1))])" using
      calculation(37) cases2_not_newCases by linarith
moreover have take_cases_cases_step: "(IChoice (ChoiceId choiceName (Party.Role role)) 0) ⊢ (
      takeNewCases @ [Case (ActionGuard (Choice (FChoiceId choiceName (Role role)) [(0, 0)])) (
      split_cases role choiceName k (When (drop k cases1) t tCont1))], newTimeState2,
      newTimeEnv2) →cases_f2 (λ innerT innerTCont . prependStatements [] (split_cases role
      choiceName k (When (drop k cases1) t tCont1)), newChoiceState, ApplyNoWarning)" using
      calculation casesStepAppendFinalCases choiceCasesStep by presburger
moreover have "(λ innerT innerTCont . prependStatements [] (split_cases role choiceName k (When
       (drop k cases1) t tCont1))) t tCont2 = (split_cases role choiceName k (When (drop k
      cases1) t tCont1))" by simp
moreover obtain innerC2 innerCtx2 innerState2 innerEnv2 innerWarnings2 innerPayments2 where
      decompile_p3: "p3 = (innerC2, innerCtx2, innerState2, innerEnv2, innerWarnings2,
      innerPayments2)" by (cases p3, auto)
moreover have inner_choice_step: "j = IChoice (ChoiceId choiceName (Party.Role role)) 0 ⟶ (
      When cases2 t tCont2, ctx2, state2, env2, warnings2, payments2) →(Some (IChoice (ChoiceId
       choiceName (Party.Role role)) 0), lower, upper) (innerC2, innerCtx2, innerState2,
      innerEnv2, innerWarnings2, innerPayments2)" using calculation(2) j_obtained decompile_p3
      decompile_c2 by auto
moreover have decompile_innerC2: "j = IChoice (ChoiceId choiceName (Party.Role role)) 0 ⟶
      innerC2 = (split_cases role choiceName k (When (drop k cases1) t tCont1))" using
      inner_choice_step take_cases_cases_step cases2_is_takeNewCases  apply (auto simp only:)
  apply (elim ReceiveE)
  using new_states deterministicCasesSemantics decompile_c2 decompile by (auto, blast)
moreover have decompile_innerCtx2: "j = IChoice (ChoiceId choiceName (Party.Role role)) 0 ⟶
      innerCtx2 = ctx2" using inner_choice_step take_cases_cases_step cases2_is_takeNewCases
      decompile_c2 decompile  by (auto simp only:)
moreover have decompile_innerState2: "j = IChoice (ChoiceId choiceName (Party.Role role)) 0 ⟶
```

```
      innerState2 = newChoiceState" using inner_choice_step take_cases_cases_step
     cases2_is_takeNewCases decompile_c2 decompile apply (auto simp only:)
  apply (elim ReceiveE)
  using new_states deterministicCasesSemantics by (auto simp del: split_cases.simps)
moreover have decompile_innerEnv2: "j = IChoice (ChoiceId choiceName (Party.Role role)) 0 ⟶
     innerEnv2 = newTimeEnv2" using inner_choice_step take_cases_cases_step
     cases2_is_takeNewCases decompile_c2 decompile apply (auto simp only:)
  apply (elim ReceiveE)
  using new_states deterministicCasesSemantics by (auto simp del: split_cases.simps)
moreover have decompile_innerWarnings2: "j = IChoice (ChoiceId choiceName (Party.Role role)) 0
     ⟶ innerWarnings2 = warnings2" using inner_choice_step receiveChoiceNoAddedWarnings apply
      auto
  by blast
moreover have decompile_innerPayments2: "j = IChoice (ChoiceId choiceName (Party.Role role)) 0
     ⟶ innerPayments2 = payments2" using inner_choice_step take_cases_cases_step
     cases2_is_takeNewCases decompile_c2 decompile by (auto simp only:)
moreover obtain dropCases1 where dropCasesEq: "dropCases1 = drop k cases1 ∧ map (λ(Case g c) ⇒
      (Case g (split_cases role choiceName k c))) dropCases1 = dropNewCases" by (metis drop_map
      newCases_obtained split_newCases_obtained)
moreover have fresh_choice_drop_cases: "isFreshChoiceInCasesGuards choiceName dropCases1" by (
     metis append_take_drop_id dropCasesEq fresh_choice_in_cases1
     isFreshChoiceInCasesGuardsPreservedInSlice)
moreover have state1_eq_choice_state: "state_minimal_eq choiceName newTimeState1 newChoiceState
     " using calculation(27) newTimeStatesEq state_minimal_eq_commutative
     state_minimal_eq_transitive by (meson new_choice_state)
moreover have dropCases1_in_relation_inner: "j = IChoice (ChoiceId choiceName (Party.Role role)
     ) 0 ⟶ split_cases_relation role choiceName k (When dropCases1 t tCont1, ctx1,
     newTimeState1, newTimeEnv1, warnings1, payments1) (innerC2, innerCtx2, innerState2,
     innerEnv2, innerWarnings2, innerPayments2)" apply (auto simp only:
     split_cases_relation_disjoint_cases.simps)
  by (metis RestrictedReceiveFullND.prems append_take_drop_id decompile decompile_innerC2
       decompile_innerCtx2 decompile_innerEnv2 decompile_innerPayments2 decompile_innerState2
       decompile_innerWarnings2 dropCasesEq isFreshChoiceInCasesPreservedInSlice
       isFreshChoiceInContract.simps(4) isFreshChoiceInProcess.simps new_times_equal
       split_cases_relation.simps split_cases_relation_disjoint_cases.simps
       state1_eq_choice_state)
moreover have "disjoint_cases (drop k cases1)" by (metis append_take_drop_id calculation(6)
     decompile disjoint_cases_contract.simps(1) disjoint_cases_process.simps
     slice_cases_preserves_disjoint_cases split_cases_relation_disjoint_cases.simps)
moreover have disjoint_in_cases: "disjoint_cases_in_cases cases1" using RestrictedReceiveFullND
     .prems decompile by force
moreover have "disjoint_cases_in_cases (drop k cases1)" using disjoint_in_cases apply (
     induction cases1 arbitrary: k, auto)
  by (metis disjoint_cases_in_cases.elims(2) drop_Cons' list.distinct(1) list.inject)
```

```
  moreover have dropCases1_in_complete_relation_inner: "j = IChoice (ChoiceId choiceName (Party.
      Role role)) 0 ⟶ split_cases_relation_disjoint_cases role choiceName k (When dropCases1 t
       tCont1, ctx1, newTimeState1, newTimeEnv1, warnings1, payments1) (innerC2, innerCtx2,
      innerState2, innerEnv2, innerWarnings2, innerPayments2)" by (metis RestrictedReceiveFullND
      .prems calculation(53) calculation(55) decompile disjoint_cases_contract.simps(1)
      disjoint_cases_process.simps dropCases1_in_relation_inner dropCasesEq
      split_cases_relation_disjoint_cases.simps)
  moreover obtain c1' ctx1' state1' env1' warnings1' payments1' where dropCases1_step_in_relation
      : "j = IChoice (ChoiceId choiceName (Party.Role role)) 0 ⟶ (A ⊢ (When dropCases1 t
      tCont1, ctx1, newTimeState1, newTimeEnv1, warnings1, payments1) ⟶λμ→ (c1', ctx1', state1
      ', env1', warnings1', payments1')) ∧
      split_cases_relation_disjoint_cases role choiceName k (c1', ctx1', state1', env1',
           warnings1', payments1') (c2', ctx2', state2', env2', warnings2', payments2')"
    using RestrictedReceiveFullND(5) dropCases1_in_complete_relation_inner decompile_p3 by
        presburger
  moreover obtain takeCases1 where takeCasesEq: "takeCases1 = take k cases1 ∧ map (λ(Case g c) ⇒
       (Case g (split_cases role choiceName k c))) takeCases1 = takeNewCases" by (metis
      newCases_obtained split_newCases_obtained take_map)
  moreover obtain realLower realUpper where real_time_bound: "timeInterval newTimeEnv2 = (
      realLower, realUpper) ∧ realLower < t ∧ realUpper < t" using RestrictedReceiveFullND
      new_states decompile_c2 by (auto simp add: small_step_receive_ccs.simps)
  moreover have j_is_choice_transition_time_before: "j = IChoice (ChoiceId choiceName (Party.Role
       role)) 0 ⟶ (A ⊢ (When cases1 t tCont1, ctx1, newTimeState1, newTimeEnv1, warnings1,
      payments1) ⟶λμ→ (c1', ctx1', state1', env1', warnings1', payments1')) ∧
      split_cases_relation_disjoint_cases role choiceName k (c1', ctx1', state1', env1',
           warnings1', payments1') (c2', ctx2', state2', env2', warnings2', payments2')"
    using takeCasesEq dropCasesEq by (metis RestrictedReceiveFullND.prems append_take_drop_id
        decompile disjoint_cases_contract.simps(1) disjoint_cases_process.simps
        dropCases1_step_in_relation full_nd_semantics_disjoint_cases_append
        split_cases_relation_disjoint_cases.simps)
  moreover show ?case apply (cases "j = IChoice (ChoiceId choiceName (Party.Role role)) 0")
    using j_is_choice_transition_time_before decompile j_obtained apply (metis calculation(2)
        full_nd_semantics_time_change_before_or_during new_states)
    using j_not_choice_c1_inner_step j_not_choice_c1_final_step j_obtained by (meson calculation
        (12))
next
  case (TimeoutFullND lower upper)
  moreover obtain cases2 t tCont2 where decompile_c2: "c2 = When cases2 t tCont2" using
      TimeoutFullND by auto
  moreover obtain cases1 t tCont1 where "c1 = When cases1 t tCont1 ∧ tCont2 = split_cases role
      choiceName k tCont1" using TimeoutFullND(2) decompile_c2 apply (cases c1, auto)
    by (smt (verit, del_insts) FaustusV2AST.FContract.inject(3))
  moreover obtain newTimeEnv1 newTimeState1 newTimeState2 newTimeEnv2 where new_states: "
      fixInterval (lower, upper) state1 = IntervalTrimmed newTimeEnv1 newTimeState1 ∧
```

```
      fixInterval (lower, upper) state2 = IntervalTrimmed newTimeEnv2 newTimeState2" using
      TimeoutFullND apply auto
    by (cases "upper < lower"; cases "upper < minTime state1"; cases "lower < minTime state1",
        auto)
  moreover have times_equal: "minTime state1 = minTime state2" using TimeoutFullND by auto
  moreover have new_times_equal: "newTimeEnv1 = newTimeEnv2 ∧ minTime newTimeState1 = minTime
      newTimeState2" using times_equal new_states fixFIntervalOnlyTimeDependent by metis
  ultimately show ?case by (smt (z3) choice_receive_same_as_basic_when_fresh
      disjoint_cases_process.simps full_nd_semantics.TimeoutFullND
      receive_choice_preserves_disjoint_cases_relation small_step_receive_ccs_add_choice.
      CCSTimeout split_cases_relation.simps
      split_cases_relation_backward_simulation_small_step_receive_add_choice
      split_cases_relation_disjoint_cases.simps)
qed

fun SC_set :: "ChoiceName ⇒ Proc rel" where
"SC_set choiceName =  {((p, A), (q, B)) | p q A B . disjoint_cases_process p ∧ B = (A ∪ {(IChoice
      (ChoiceId choiceName (Party.Role [0 :: 8 word])) 0)}) ∧ (p, q) ∈ split_cases_relation_set
     ([0 :: 8 word]) choiceName rcmdk}"

lemma split_cases_relation_disjoint_cases_is_weak_sim:
"weak_nd_sim_simple (SC_set choiceName)"
  apply (auto simp only: SC_set.simps weak_nd_sim_simple_def disjoint_cases_process.simps
      split_cases_relation_gives_set)
  using full_nd_semantics_maintains_split_cases
      full_nd_semantics_preserves_disjoint_cases_relation single_full_nd_receive_in_experiment
      split_cases_relation_gives_set full_nd_semantics_react_is_equivalent_to_internal_reaction
      mem_Collect_eq apply (smt (z3) disjoint_cases_process.simps
      react_step_preserves_split_cases_relation)
  using full_nd_semantics_maintains_split_cases
      full_nd_semantics_preserves_disjoint_cases_relation single_full_nd_receive_in_experiment
      split_cases_relation_gives_set mem_Collect_eq by (smt disjoint_cases_process.simps)

lemma split_cases_relation_disjoint_cases_inv_is_weak_sim_aux:
"((aj, ak, al, am, an, bb), ao, ap, aq, ar, as, bc) ∈ split_cases_relation_set [0] choiceName
     rcmdk ⟹
      A ∪ {IChoice (ChoiceId choiceName (Party.Role [0])) 0} ⊢ (ao, ap, aq, ar, as, bc) ⟶μ⟶ (
           at, au, av, aw, ax, bd) ⟹
      disjoint_cases_contract aj ⟹
      disjoint_cases_context ak ⟹
      ∃c2'. A ⊢ (aj, ak, al, am, an, bb) ⟶μ⟶ c2' ∧
            (((at, au, av, aw, ax, bd), A ∪ {IChoice (ChoiceId choiceName (Party.Role [0])) 0}),
                  c2', A) ∈ { ((p, A), q, B)| p A q B . (disjoint_cases_process p ∧ B = A ∪ {
                 IChoice (ChoiceId choiceName (Party.Role [0])) 0} ∧ (p, q) ∈
```

```
                 split_cases_relation_set [0] choiceName rcmdk)}⁻¹"
  subgoal premises ps proof -
    have "split_cases_relation_disjoint_cases [0] choiceName rcmdk (aj, ak, al, am, an, bb) (ao,
         ap, aq, ar, as, bc)" using disjoint_cases_process.simps ps(1) ps(3) ps(4)
         split_cases_relation_disjoint_cases.simps split_cases_relation_gives_set by presburger
    moreover obtain c2' where "A ⊢ (aj, ak, al, am, an, bb) —μ→ c2' ∧
         split_cases_relation_disjoint_cases [0] choiceName rcmdk c2' (at, au, av, aw, ax, bd)"
         using calculation full_nd_semantics_preserves_split_cases_disjoint_backwards ps(2) by
         fastforce
    moreover have "(((at, au, av, aw, ax, bd), A ∪ {IChoice (ChoiceId choiceName (Party.Role [0])
         ) 0}), c2', A) ∈ { ((p, A), q, B)| p A q B . (disjoint_cases_process p ∧ B = A ∪ {
         IChoice (ChoiceId choiceName (Party.Role [0])) 0} ∧ (p, q) ∈ split_cases_relation_set
         [0] choiceName rcmdk)}⁻¹" using calculation(2)
         split_cases_relation_disjoint_cases_gives_set by auto
    ultimately show ?thesis by blast
  qed
  done

lemma split_cases_relation_disjoint_cases_inv_is_weak_sim_aux_2:
"A ⊢ c1 —τ→ c2 ⟹
  c1 →⋆𝒯 c2"
  apply (elim FullNDInternalE)
  using star_step1[of internal_reaction c1 c2] by force

lemma split_cases_relation_disjoint_cases_inv_is_weak_sim:
"weak_nd_sim_simple ((SC_set choiceName)⁻¹)"
  apply (auto simp only: weak_nd_sim_simple_def SC_set.simps disjoint_cases_process.simps)
  using split_cases_relation_disjoint_cases_inv_is_weak_sim_aux
       split_cases_relation_disjoint_cases_inv_is_weak_sim_aux_2
   apply (smt (z3) converseD converseI mem_Collect_eq)
  using split_cases_relation_disjoint_cases_inv_is_weak_sim_aux
       single_full_nd_receive_in_experiment by (smt (z3) Collect_cong)

lemma split_cases_relation_disjoint_cases_is_bisimulation:
"observation_nd_bisimulation (SC_set choiceName)"
  using observation_nd_bisimulation_def split_cases_relation_disjoint_cases_inv_is_weak_sim
       split_cases_relation_disjoint_cases_is_weak_sim by presburger

lemma observational_equivalence_of_transformation:
"isFreshChoiceInProcess c (c1, ctx1, s1, e, w, ps) ⟹
  disjoint_cases_process (c1, ctx1, s1, e, w, ps) ⟹
  split_cases [0] c rcmdk c1 = c2 ⟹
  split_cases_context [0] c rcmdk ctx1 = ctx2 ⟹
  state_minimal_eq c s1 s2 ⟹
```

```
  ((c1, ctx1, s1, e, w, ps), A) \<approx> ((c2, ctx2, s2, e, w, ps), A ∪ {IChoice (ChoiceId c (
      Party.Role [0])) 0})"
  subgoal premises ps proof -
    have "(((c1, ctx1, s1, e, w, ps), A), ((c2, ctx2, s2, e, w, ps), A ∪ {IChoice (ChoiceId c (
        Party.Role [0])) 0})) ∈ (SC_set c)" using ps by auto
    then show ?thesis using observable_equivalence_def
        split_cases_relation_disjoint_cases_is_bisimulation by blast
  qed
  done

lemma observational_equivalence_of_transformation_same_state:
"isFreshChoiceInProcess c (c1, ctx1, s, e, w, ps) ⟹
  disjoint_cases_process (c1, ctx1, s, e, w, ps) ⟹
  split_cases [0] c rcmdk c1 = c2 ⟹
  split_cases_context [0] c rcmdk ctx1 = ctx2 ⟹
  ((c1, ctx1, s, e, w, ps), A) \<approx> ((c2, ctx2, s, e, w, ps), A ∪ {IChoice (ChoiceId c (
      Party.Role [0])) 0})"
  subgoal premises ps proof -
    have "(((c1, ctx1, s, e, w, ps), A), ((c2, ctx2, s, e, w, ps), A ∪ {IChoice (ChoiceId c (
        Party.Role [0])) 0})) ∈ (SC_set c)" using ps by auto
    then show ?thesis using observable_equivalence_def
        split_cases_relation_disjoint_cases_is_bisimulation by blast
  qed
  done

end
```

Appendix E

STILL Example Proofs

The following proof scripts are provided in the STILL source code that demonstrate its features and help assess the faithfulness of the implementation of the theory.

Listing E.1: The verification of an English auction protocol in the STILL theorem prover.

```
module Auction begin

stype englishSession = "forall ItemType:stype.
    forall bid:Type 1.
    forall zeroB:bid.
    forall validNewB:(Pi b1:bid. Pi b2:bid. Type 1).
    forall currency:(Pi b:bid.Type 1).
    ItemType -o
    (forall b:bid. $currency b -o 1) -o
    forall firstB:bid.
    forall firstValid:validNewB zeroB firstB.
    $currency firstB -o
    (ItemType + $currency firstB -o 1) -o
    nu Y. (forall newB:bid.
        forall nextValid:validNewB firstB newB.
        $currency newB -o
        (ItemType + $currency newB -o 1) -o Y) & 1"

theorem english: "englishSession"
apply Intros
apply NuR AUC
    (item, ownerC, curB, winnerChan, oldB, lastB, curValid)
    (a, c, d, e, zeroB, firstB, firstValid)
apply WithR
```

```
apply ForallLA
apply Exact "firstB"
apply ImpliesLA
apply UnitLA
apply ImpliesLA
apply UnitLA
apply UnitR
apply PlusR1
apply IdA
apply IdA
apply ImpliesLA
apply UnitLA
apply Intros
defer
apply PlusR2
apply IdA
apply Weaken firstValid
apply Weaken zeroB
apply TyVar AUC (a, c, i, j, firstB, newB, nextValid)
done
theorem english2: "englishSession"
apply Intros
apply NuR AUC
    (item, ownerC, curB, winnerC, oldB, lastB, curValid)
    (a, c, d, e, zeroB, firstB, firstValid)
apply WithR
apply ForallLA
apply Exact "firstB"
apply ImpliesLA
apply UnitLA
apply ImpliesLA
apply UnitLA
apply UnitR
apply PlusR1
apply IdA
apply IdA
apply Intros
apply ImpliesLA
apply UnitLA
defer
apply PlusR2
apply IdA
apply Weaken firstValid
apply Weaken zeroB
```

```
apply TyVar AUC (a, c, h, i, firstB, newB, nextValid)
done

assume delivery is "Pi itemType:Type 1.Pi item:itemType.Type 1"

stype postOfficeSession = "forall itemObjectType:Type 1.
    forall item:itemObjectType.$delivery itemObjectType item"

assume book is "Type 1"
assume int is "Type 1"
assume usd is "Pi b:int.Type 1"

stype bookOwnerSession = "exists startPrice:int.exists bidValid:(Pi b1:int. Pi b2:int. Type 1).
     exists bookForSale:book.(forall b:int.$usd b -o 1)"
stype bookBuyerSession = "forall validator:(Pi b1:int. Pi b2:int. Type 1).
    exists buyerAmount:int.
        forall curPrice:int.
        exists buyerValid:validator curPrice buyerAmount.
        $(usd buyerAmount) * (forall bookBought:book.($usd buyerAmount + $delivery book
             bookBought) -o 1)"

theorem bookAuction consumes "bookOwnerSession" "bookBuyerSession" "englishSession" "
     postOfficeSession": "1"
apply ExistsLA+
apply Forall2L d "$delivery book bookForSale"
apply ForallL d
apply Exact "int"
apply ForallL d
apply Exact "startPrice"
apply ForallL d
apply Exact "bidValid"
apply ForallL d
apply Exact "usd"
apply ForallL e
apply Exact "book"
apply ForallL e
apply Exact "bookForSale"
apply ImpliesL d
defer
apply IdA
apply ImpliesL d
defer
apply IdA
apply ForallL c
```

```
apply Exact "bidValid"
apply ExistsLA
apply ForallL d
apply Exact "buyerAmount"
apply ForallL c
apply Exact "startPrice"
apply ExistsLA
apply ForallL d
apply Exact "buyerValid"
apply TensorL c
apply ImpliesL d
defer
apply IdA
apply ForallL c
apply Exact "bookForSale"
apply ImpliesL d
defer
apply ImpliesR
apply ImpliesL c
defer
apply PlusLA
apply PlusR1
apply Id k
apply PlusR2
apply Id k
defer
apply UnitL c
apply UnitR
apply NuL d
apply WithL2 d
apply UnitL d
apply UnitR
done
```

Listing E.2: The verification of a bank and ATM client protocol in the STILL theorem prover.

```
module Bank begin

theorem bank: "forall string:Type 1.
    forall nat:Type 1.
    forall uid:(Pi s:string.Type 1).
    forall deposit:(Pi s:string. Pi n:nat.Type 1).
    forall receipt:(Pi s:string. Pi n:nat.Type 1).
```

```
    forall balance:(Pi s:string. Pi n:nat.Type 1).
    forall dep:(Pi s:string. Pi n:nat. Pi m:deposit s n.receipt s n).
    forall bal:(Pi s : string. Sigma m : nat . balance s m).
    !(forall userId:string. $uid userId -o
    (forall n:nat.$deposit userId n -o (($receipt userId n) * 1)) &
    (exists m:nat.$balance userId m))"
apply Intros
apply BangR
apply Intros
apply FTermLA
apply WithR
apply ExistsR
apply Exact "fst (bal userId)"
apply FTermR
apply Exact "snd (bal userId)"
apply Intros
apply FTermLA
apply TensorR
apply UnitR
apply FTermR
apply Exact "dep userId n c"
done

theorem atmClient: "(!(forall userId:string. $uid userId -o
    (forall n:nat.$deposit userId n -o (($receipt userId n) * 1)) &
    (exists m:nat.$balance userId m))) -- Bank service
    -o
    (forall string:Type 1.
    forall nat:Type 1.
    forall uid:(Pi s:string.Type 1).
    forall deposit:(Pi s:string. Pi n:nat.Type 1).
    forall receipt:(Pi s:string. Pi n:nat.Type 1).
    forall boundedBy:(Pi m:nat. Pi n:nat.Type 1).
    forall charge:(Pi s : string. Pi n : nat. Pi d: deposit s n.  Sigma m : nat. Sigma p :
         boundedBy m n.deposit s m).
    forall s : string.$uid(s) -o
    (forall n : nat.$deposit s n -o exists m : nat.
    exists p : boundedBy m n.($receipt s m * 1)))"
apply Intros
apply FTermLA+
apply ExistsR
apply Exact "fst (charge s n c)"
apply ExistsR
apply Exact "fst (snd (charge s n c))"
```

```
apply TensorR
apply UnitR
apply BangLA
apply Copy f
apply ForallLA
apply Exact "s"
apply ImpliesLA
defer
apply FTermR
apply Exact "b"
apply WithL1A
apply ForallLA
apply Exact "fst (charge s n c)"
apply ImpliesLA
defer
apply FTermR
apply Exact "snd (snd (charge s n c))"
apply TensorLA
apply UnitLA
apply IdA
done
```

Listing E.3: The verification of a cloud server protocol and implementation in the STILL theorem prover.

```
module CloudServer begin

theorem CS: "forall X:stype.!API -o !(API -o X) -o !X"
apply Intros
apply BangLA+
apply BangR
apply Copy d
apply ImpliesLA
apply IdA
apply Copy c
apply IdA
done
theorem Conv: "forall file:stype.forall pdf:stype.!API -o
    (forall X:stype.!API -o !(API -o X) -o !X) -o
    (!(API -o file -o (pdf * 1))) -o
    !(file -o (pdf * 1))"
apply Intros
apply Forall2L b "(file -o (pdf * 1))"
apply BangLA
```

```
apply ImpliesL b
defer
apply BangR
apply Copy d
apply IdA
apply ImpliesL b
defer
apply IdA
apply IdA
done
```

Listing E.4: The verification of a protocol with one thousand tensor applications for stress testing the STILL theorem prover.

```
module LargeMul begin

theorem thousandTensor: "(a * b * c * d * e * f * g * h * i * j * k * l * m * n * o * p * q * r *
      s * t * u * v * w * x * y * z * a1 * b1 * c1 * d1 * e1 * f1 * g1 * h1 * i1 * j1 * k1 * l1 *
      m1 * n1 * o1 * p1 * q1 * r1 * s1 * t1 * u1 * v1 * w1 * x1 * y1 * z1 * a2 * b2 * c2 * d2 *
     e2 * f2 * g2 * h2 * i2 * j2 * k2 * l2 * m2 * n2 * o2 * p2 * q2 * r2 * s2 * t2 * u2 * v2 * w2
      * x2 * y2 * z2 * a3 * b3 * c3 * d3 * e3 * f3 * g3 * h3 * i3 * j3 * k3 * l3 * m3 * n3 * o3 *
      p3 * q3 * r3 * s3 * t3 * u3 * v3 * w3 * x3 * y3 * z3 * a4 * b4 * c4 * d4 * e4 * f4 * g4 *
     h4 * i4 * j4 * k4 * l4 * m4 * n4 * o4 * p4 * q4 * r4 * s4 * t4 * u4 * v4 * w4 * x4 * y4 * z4
      * a5 * b5 * c5 * d5 * e5 * f5 * g5 * h5 * i5 * j5 * k5 * l5 * m5 * n5 * o5 * p5 * q5 * r5 *
      s5 * t5 * u5 * v5 * w5 * x5 * y5 * z5 * a6 * b6 * c6 * d6 * e6 * f6 * g6 * h6 * i6 * j6 *
     k6 * l6 * m6 * n6 * o6 * p6 * q6 * r6 * s6 * t6 * u6 * v6 * w6 * x6 * y6 * z6 * a7 * b7 * c7
      * d7 * e7 * f7 * g7 * h7 * i7 * j7 * k7 * l7 * m7 * n7 * o7 * p7 * q7 * r7 * s7 * t7 * u7 *
      v7 * w7 * x7 * y7 * z7 * a8 * b8 * c8 * d8 * e8 * f8 * g8 * h8 * i8 * j8 * k8 * l8 * m8 *
     n8 * o8 * p8 * q8 * r8 * s8 * t8 * u8 * v8 * w8 * x8 * y8 * z8 * a9 * b9 * c9 * d9 * e9 * f9
      * g9 * h9 * i9 * j9 * k9 * l9 * m9 * n9 * o9 * p9 * q9 * r9 * s9 * t9 * u9 * v9 * w9 * x9 *
      y9 * z9 * a10 * b10 * c10 * d10 * e10 * f10 * g10 * h10 * i10 * j10 * k10 * l10 * m10 * n10
      * o10 * p10 * q10 * r10 * s10 * t10 * u10 * v10 * w10 * x10 * y10 * z10 * a11 * b11 * c11 *
      d11 * e11 * f11 * g11 * h11 * i11 * j11 * k11 * l11 * m11 * n11 * o11 * p11 * q11 * r11 *
     s11 * t11 * u11 * v11 * w11 * x11 * y11 * z11 * a12 * b12 * c12 * d12 * e12 * f12 * g12 *
     h12 * i12 * j12 * k12 * l12 * m12 * n12 * o12 * p12 * q12 * r12 * s12 * t12 * u12 * v12 *
     w12 * x12 * y12 * z12 * a13 * b13 * c13 * d13 * e13 * f13 * g13 * h13 * i13 * j13 * k13 *
     l13 * m13 * n13 * o13 * p13 * q13 * r13 * s13 * t13 * u13 * v13 * w13 * x13 * y13 * z13 *
     a14 * b14 * c14 * d14 * e14 * f14 * g14 * h14 * i14 * j14 * k14 * l14 * m14 * n14 * o14 *
     p14 * q14 * r14 * s14 * t14 * u14 * v14 * w14 * x14 * y14 * z14 * a15 * b15 * c15 * d15 *
     e15 * f15 * g15 * h15 * i15 * j15 * k15 * l15 * m15 * n15 * o15 * p15 * q15 * r15 * s15 *
     t15 * u15 * v15 * w15 * x15 * y15 * z15 * a16 * b16 * c16 * d16 * e16 * f16 * g16 * h16 *
     i16 * j16 * k16 * l16 * m16 * n16 * o16 * p16 * q16 * r16 * s16 * t16 * u16 * v16 * w16 *
     x16 * y16 * z16 * a17 * b17 * c17 * d17 * e17 * f17 * g17 * h17 * i17 * j17 * k17 * l17 *
     m17 * n17 * o17 * p17 * q17 * r17 * s17 * t17 * u17 * v17 * w17 * x17 * y17 * z17 * a18 *
```

```
b18 * c18 * d18 * e18 * f18 * g18 * h18 * i18 * j18 * k18 * l18 * m18 * n18 * o18 * p18 *
q18 * r18 * s18 * t18 * u18 * v18 * w18 * x18 * y18 * z18 * a19 * b19 * c19 * d19 * e19 *
f19 * g19 * h19 * i19 * j19 * k19 * l19 * m19 * n19 * o19 * p19 * q19 * r19 * s19 * t19 *
u19 * v19 * w19 * x19 * y19 * z19 * a20 * b20 * c20 * d20 * e20 * f20 * g20 * h20 * i20 *
j20 * k20 * l20 * m20 * n20 * o20 * p20 * q20 * r20 * s20 * t20 * u20 * v20 * w20 * x20 *
y20 * z20 * a21 * b21 * c21 * d21 * e21 * f21 * g21 * h21 * i21 * j21 * k21 * l21 * m21 *
n21 * o21 * p21 * q21 * r21 * s21 * t21 * u21 * v21 * w21 * x21 * y21 * z21 * a22 * b22 *
c22 * d22 * e22 * f22 * g22 * h22 * i22 * j22 * k22 * l22 * m22 * n22 * o22 * p22 * q22 *
r22 * s22 * t22 * u22 * v22 * w22 * x22 * y22 * z22 * a23 * b23 * c23 * d23 * e23 * f23 *
g23 * h23 * i23 * j23 * k23 * l23 * m23 * n23 * o23 * p23 * q23 * r23 * s23 * t23 * u23 *
v23 * w23 * x23 * y23 * z23 * a24 * b24 * c24 * d24 * e24 * f24 * g24 * h24 * i24 * j24 *
k24 * l24 * m24 * n24 * o24 * p24 * q24 * r24 * s24 * t24 * u24 * v24 * w24 * x24 * y24 *
z24 * a25 * b25 * c25 * d25 * e25 * f25 * g25 * h25 * i25 * j25 * k25 * l25 * m25 * n25 *
o25 * p25 * q25 * r25 * s25 * t25 * u25 * v25 * w25 * x25 * y25 * z25 * a26 * b26 * c26 *
d26 * e26 * f26 * g26 * h26 * i26 * j26 * k26 * l26 * m26 * n26 * o26 * p26 * q26 * r26 *
s26 * t26 * u26 * v26 * w26 * x26 * y26 * z26 * a27 * b27 * c27 * d27 * e27 * f27 * g27 *
h27 * i27 * j27 * k27 * l27 * m27 * n27 * o27 * p27 * q27 * r27 * s27 * t27 * u27 * v27 *
w27 * x27 * y27 * z27 * a28 * b28 * c28 * d28 * e28 * f28 * g28 * h28 * i28 * j28 * k28 *
l28 * m28 * n28 * o28 * p28 * q28 * r28 * s28 * t28 * u28 * v28 * w28 * x28 * y28 * z28 *
a29 * b29 * c29 * d29 * e29 * f29 * g29 * h29 * i29 * j29 * k29 * l29 * m29 * n29 * o29 *
p29 * q29 * r29 * s29 * t29 * u29 * v29 * w29 * x29 * y29 * z29 * a30 * b30 * c30 * d30 *
e30 * f30 * g30 * h30 * i30 * j30 * k30 * l30 * m30 * n30 * o30 * p30 * q30 * r30 * s30 *
t30 * u30 * v30 * w30 * x30 * y30 * z30 * a31 * b31 * c31 * d31 * e31 * f31 * g31 * h31 *
i31 * j31 * k31 * l31 * m31 * n31 * o31 * p31 * q31 * r31 * s31 * t31 * u31 * v31 * w31 *
x31 * y31 * z31 * a32 * b32 * c32 * d32 * e32 * f32 * g32 * h32 * i32 * j32 * k32 * l32 *
m32 * n32 * o32 * p32 * q32 * r32 * s32 * t32 * u32 * v32 * w32 * x32 * y32 * z32 * a33 *
b33 * c33 * d33 * e33 * f33 * g33 * h33 * i33 * j33 * k33 * l33 * m33 * n33 * o33 * p33 *
q33 * r33 * s33 * t33 * u33 * v33 * w33 * x33 * y33 * z33 * a34 * b34 * c34 * d34 * e34 *
f34 * g34 * h34 * i34 * j34 * k34 * l34 * m34 * n34 * o34 * p34 * q34 * r34 * s34 * t34 *
u34 * v34 * w34 * x34 * y34 * z34 * a35 * b35 * c35 * d35 * e35 * f35 * g35 * h35 * i35 *
j35 * k35 * l35 * m35 * n35 * o35 * p35 * q35 * r35 * s35 * t35 * u35 * v35 * w35 * x35 *
y35 * z35 * a36 * b36 * c36 * d36 * e36 * f36 * g36 * h36 * i36 * j36 * k36 * l36 * m36 *
n36 * o36 * p36 * q36 * r36 * s36 * t36 * u36 * v36 * w36 * x36 * y36 * z36 * a37 * b37 *
c37 * d37 * e37 * f37 * g37 * h37 * i37 * j37 * k37 * l37 * m37 * n37 * o37 * p37 * q37 *
r37 * s37 * t37 * u37 * v37 * w37 * x37 * y37 * z37 * a38 * b38 * c38 * d38 * e38 * f38 *
g38 * h38 * i38 * j38 * k38 * l38 ) -o ( l38 * k38 * j38 * i38 * h38 * g38 * f38 * e38 * d38
 * c38 * b38 * a38 * z37 * y37 * x37 * w37 * v37 * u37 * t37 * s37 * r37 * q37 * p37 * o37 *
 n37 * m37 * l37 * k37 * j37 * i37 * h37 * g37 * f37 * e37 * d37 * c37 * b37 * a37 * z36 *
y36 * x36 * w36 * v36 * u36 * t36 * s36 * r36 * q36 * p36 * o36 * n36 * m36 * l36 * k36 *
j36 * i36 * h36 * g36 * f36 * e36 * d36 * c36 * b36 * a36 * z35 * y35 * x35 * w35 * v35 *
u35 * t35 * s35 * r35 * q35 * p35 * o35 * n35 * m35 * l35 * k35 * j35 * i35 * h35 * g35 *
f35 * e35 * d35 * c35 * b35 * a35 * z34 * y34 * x34 * w34 * v34 * u34 * t34 * s34 * r34 *
q34 * p34 * o34 * n34 * m34 * l34 * k34 * j34 * i34 * h34 * g34 * f34 * e34 * d34 * c34 *
b34 * a34 * z33 * y33 * x33 * w33 * v33 * u33 * t33 * s33 * r33 * q33 * p33 * o33 * n33 *
```

```
m33 * l33 * k33 * j33 * i33 * h33 * g33 * f33 * e33 * d33 * c33 * b33 * a33 * z32 * y32 *
x32 * w32 * v32 * u32 * t32 * s32 * r32 * q32 * p32 * o32 * n32 * m32 * l32 * k32 * j32 *
i32 * h32 * g32 * f32 * e32 * d32 * c32 * b32 * a32 * z31 * y31 * x31 * w31 * v31 * u31 *
t31 * s31 * r31 * q31 * p31 * o31 * n31 * m31 * l31 * k31 * j31 * i31 * h31 * g31 * f31 *
e31 * d31 * c31 * b31 * a31 * z30 * y30 * x30 * w30 * v30 * u30 * t30 * s30 * r30 * q30 *
p30 * o30 * n30 * m30 * l30 * k30 * j30 * i30 * h30 * g30 * f30 * e30 * d30 * c30 * b30 *
a30 * z29 * y29 * x29 * w29 * v29 * u29 * t29 * s29 * r29 * q29 * p29 * o29 * n29 * m29 *
l29 * k29 * j29 * i29 * h29 * g29 * f29 * e29 * d29 * c29 * b29 * a29 * z28 * y28 * x28 *
w28 * v28 * u28 * t28 * s28 * r28 * q28 * p28 * o28 * n28 * m28 * l28 * k28 * j28 * i28 *
h28 * g28 * f28 * e28 * d28 * c28 * b28 * a28 * z27 * y27 * x27 * w27 * v27 * u27 * t27 *
s27 * r27 * q27 * p27 * o27 * n27 * m27 * l27 * k27 * j27 * i27 * h27 * g27 * f27 * e27 *
d27 * c27 * b27 * a27 * z26 * y26 * x26 * w26 * v26 * u26 * t26 * s26 * r26 * q26 * p26 *
o26 * n26 * m26 * l26 * k26 * j26 * i26 * h26 * g26 * f26 * e26 * d26 * c26 * b26 * a26 *
z25 * y25 * x25 * w25 * v25 * u25 * t25 * s25 * r25 * q25 * p25 * o25 * n25 * m25 * l25 *
k25 * j25 * i25 * h25 * g25 * f25 * e25 * d25 * c25 * b25 * a25 * z24 * y24 * x24 * w24 *
v24 * u24 * t24 * s24 * r24 * q24 * p24 * o24 * n24 * m24 * l24 * k24 * j24 * i24 * h24 *
g24 * f24 * e24 * d24 * c24 * b24 * a24 * z23 * y23 * x23 * w23 * v23 * u23 * t23 * s23 *
r23 * q23 * p23 * o23 * n23 * m23 * l23 * k23 * j23 * i23 * h23 * g23 * f23 * e23 * d23 *
c23 * b23 * a23 * z22 * y22 * x22 * w22 * v22 * u22 * t22 * s22 * r22 * q22 * p22 * o22 *
n22 * m22 * l22 * k22 * j22 * i22 * h22 * g22 * f22 * e22 * d22 * c22 * b22 * a22 * z21 *
y21 * x21 * w21 * v21 * u21 * t21 * s21 * r21 * q21 * p21 * o21 * n21 * m21 * l21 * k21 *
j21 * i21 * h21 * g21 * f21 * e21 * d21 * c21 * b21 * a21 * z20 * y20 * x20 * w20 * v20 *
u20 * t20 * s20 * r20 * q20 * p20 * o20 * n20 * m20 * l20 * k20 * j20 * i20 * h20 * g20 *
f20 * e20 * d20 * c20 * b20 * a20 * z19 * y19 * x19 * w19 * v19 * u19 * t19 * s19 * r19 *
q19 * p19 * o19 * n19 * m19 * l19 * k19 * j19 * i19 * h19 * g19 * f19 * e19 * d19 * c19 *
b19 * a19 * z18 * y18 * x18 * w18 * v18 * u18 * t18 * s18 * r18 * q18 * p18 * o18 * n18 *
m18 * l18 * k18 * j18 * i18 * h18 * g18 * f18 * e18 * d18 * c18 * b18 * a18 * z17 * y17 *
x17 * w17 * v17 * u17 * t17 * s17 * r17 * q17 * p17 * o17 * n17 * m17 * l17 * k17 * j17 *
i17 * h17 * g17 * f17 * e17 * d17 * c17 * b17 * a17 * z16 * y16 * x16 * w16 * v16 * u16 *
t16 * s16 * r16 * q16 * p16 * o16 * n16 * m16 * l16 * k16 * j16 * i16 * h16 * g16 * f16 *
e16 * d16 * c16 * b16 * a16 * z15 * y15 * x15 * w15 * v15 * u15 * t15 * s15 * r15 * q15 *
p15 * o15 * n15 * m15 * l15 * k15 * j15 * i15 * h15 * g15 * f15 * e15 * d15 * c15 * b15 *
a15 * z14 * y14 * x14 * w14 * v14 * u14 * t14 * s14 * r14 * q14 * p14 * o14 * n14 * m14 *
l14 * k14 * j14 * i14 * h14 * g14 * f14 * e14 * d14 * c14 * b14 * a14 * z13 * y13 * x13 *
w13 * v13 * u13 * t13 * s13 * r13 * q13 * p13 * o13 * n13 * m13 * l13 * k13 * j13 * i13 *
h13 * g13 * f13 * e13 * d13 * c13 * b13 * a13 * z12 * y12 * x12 * w12 * v12 * u12 * t12 *
s12 * r12 * q12 * p12 * o12 * n12 * m12 * l12 * k12 * j12 * i12 * h12 * g12 * f12 * e12 *
d12 * c12 * b12 * a12 * z11 * y11 * x11 * w11 * v11 * u11 * t11 * s11 * r11 * q11 * p11 *
o11 * n11 * m11 * l11 * k11 * j11 * i11 * h11 * g11 * f11 * e11 * d11 * c11 * b11 * a11 *
z10 * y10 * x10 * w10 * v10 * u10 * t10 * s10 * r10 * q10 * p10 * o10 * n10 * m10 * l10 *
k10 * j10 * i10 * h10 * g10 * f10 * e10 * d10 * c10 * b10 * a10 * z9 * y9 * x9 * w9 * v9 *
u9 * t9 * s9 * r9 * q9 * p9 * o9 * n9 * m9 * l9 * k9 * j9 * i9 * h9 * g9 * f9 * e9 * d9 * c9
 * b9 * a9 * z8 * y8 * x8 * w8 * v8 * u8 * t8 * s8 * r8 * q8 * p8 * o8 * n8 * m8 * l8 * k8 *
 j8 * i8 * h8 * g8 * f8 * e8 * d8 * c8 * b8 * a8 * z7 * y7 * x7 * w7 * v7 * u7 * t7 * s7 *
```

```
     r7 * q7 * p7 * o7 * n7 * m7 * l7 * k7 * j7 * i7 * h7 * g7 * f7 * e7 * d7 * c7 * b7 * a7 * z6
      * y6 * x6 * w6 * v6 * u6 * t6 * s6 * r6 * q6 * p6 * o6 * n6 * m6 * l6 * k6 * j6 * i6 * h6 *
      g6 * f6 * e6 * d6 * c6 * b6 * a6 * z5 * y5 * x5 * w5 * v5 * u5 * t5 * s5 * r5 * q5 * p5 *
     o5 * n5 * m5 * l5 * k5 * j5 * i5 * h5 * g5 * f5 * e5 * d5 * c5 * b5 * a5 * z4 * y4 * x4 * w4
      * v4 * u4 * t4 * s4 * r4 * q4 * p4 * o4 * n4 * m4 * l4 * k4 * j4 * i4 * h4 * g4 * f4 * e4 *
      d4 * c4 * b4 * a4 * z3 * y3 * x3 * w3 * v3 * u3 * t3 * s3 * r3 * q3 * p3 * o3 * n3 * m3 *
     l3 * k3 * j3 * i3 * h3 * g3 * f3 * e3 * d3 * c3 * b3 * a3 * z2 * y2 * x2 * w2 * v2 * u2 * t2
      * s2 * r2 * q2 * p2 * o2 * n2 * m2 * l2 * k2 * j2 * i2 * h2 * g2 * f2 * e2 * d2 * c2 * b2 *
      a2 * z1 * y1 * x1 * w1 * v1 * u1 * t1 * s1 * r1 * q1 * p1 * o1 * n1 * m1 * l1 * k1 * j1 *
     i1 * h1 * g1 * f1 * e1 * d1 * c1 * b1 * a1 * z * y * x * w * v * u * t * s * r * q * p * o *
      n * m * l * k * j * i * h * g * f * e * d * c * b * a )"
apply ImpliesR; TensorLA+; (TensorR; (IdA <|> Skip))+
done
```

References

[1] P. Lamela Seijas and S. Thompson, "Marlowe: Financial contracts on blockchain," in *Leveraging Applications of Formal Methods, Verification and Validation. Industrial Practice*, T. Margaria and B. Steffen, Eds. Cham: Springer International Publishing, 2018, pp. 356–375.

[2] N. Szabo. (1994) Smart Contracts. [Online]. Available: https://www.fon.hum.uva.nl/rob/Courses/InformationInSpeech/CDROM/Literature/LOTwinterschool2006/szabo.best.vwh.net/smart.contracts.html

[3] N. Atzei, M. Bartoletti, and T. Cimoli, "A survey of attacks on ethereum smart contracts (sok)," in *Principles of Security and Trust*, M. Maffei and M. Ryan, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2017, pp. 164–186.

[4] O. Kharif, "Crypto-Bridge Hacks Reach Over $1 Billion in Little Over a Year," *Bloomberg.com*, Mar. 2022. [Online]. Available: https://www.bloomberg.com/news/articles/2022-03-30/crypto-bridge-hacks-reach-over-1-billion-in-little-over-a-year

[5] "Internet Crime Complaint Center (IC3) | Cyber Criminals Increasingly Exploit Vulnerabilities in Decentralized Finance Platforms to Obtain Cryptocurrency, Causing Investors to Lose Money." [Online]. Available: https://www.ic3.gov/PSA/2022/PSA220829

[6] R. Milner, "An algebraic definition of simulation between programs," in *Proceedings of the 2nd International Joint Conference on Artificial Intelligence. London, UK, September 1-3, 1971*, D. C. Cooper, Ed. William Kaufmann, 1971, pp. 481–489. [Online]. Available: http://ijcai.org/Proceedings/71/Papers/044.pdf

[7] D. Park, "Concurrency and automata on infinite sequences," in *Theoretical Computer Science*, P. Deussen, Ed. Berlin, Heidelberg: Springer, 1981, pp. 167–183.

[8] S. Peyton Jones, J.-M. Eber, and J. Seward, "Composing contracts: an adventure in financial engineering (functional pearl)," in *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming*, ser. ICFP '00. New York, NY, USA: Association for Computing Machinery, 2000, p. 280–292. [Online]. Available: https://doi.org/10.1145/351240.351267

[9] J. Andersen, E. Elsborg, F. Henglein, J. G. Simonsen, and C. Stefansen, "Compositional specification of commercial contracts," *International Journal on Software Tools for Technology Transfer*, vol. 8, no. 6, pp. 485–516, Nov. 2006. [Online]. Available: https://doi.org/10.1007/s10009-006-0010-1

[10] F. Henglein, C. K. Larsen, and A. Murawska, "A formally verified static analysis framework for compositional contracts," in *Financial Cryptography and Data Security*, M. Bernhard, A. Bracciali, L. J. Camp, S. Matsuo, A. Maurushat, P. B. Rønne, and M. Sala, Eds. Cham: Springer International Publishing, 2020, pp. 599–619.

[11] [Online]. Available: https://cardano.org/

[12] P. Lamela Seijas, A. Nemish, D. Smith, and S. Thompson, "Marlowe: Implementing and Analysing Financial Contracts on Blockchain," in *Financial Cryptography and Data Security*, ser. Lecture Notes in Computer Science, M. Bernhard, A. Bracciali, L. J. Camp, S. Matsuo, A. Maurushat, P. B. Rønne, and M. Sala, Eds. Cham: Springer International Publishing, 2020, pp. 496–511.

[13] P. Lamela Seijas, D. Smith, and S. Thompson, "Efficient Static Analysis of Marlowe Contracts," in *Leveraging Applications of Formal Methods, Verification and Validation: Applications*, T. Margaria and B. Steffen, Eds. Cham: Springer International Publishing, 2020, pp. 161–177.

[14] E. Hildenbrandt, M. Saxena, N. Rodrigues, X. Zhu, P. Daian, D. Guth, B. Moore, D. Park, Y. Zhang, A. Stefanescu, and G. Rosu, "Kevm: A complete formal semantics of the ethereum virtual machine," in *2018 IEEE 31st Computer Security Foundations Symposium (CSF)*. Oxford: IEEE, July 2018, p. 204–217. [Online]. Available: https://ieeexplore.ieee.org/document/8429306/

[15] G. Wood *et al.*, "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum project yellow paper*, vol. 151, no. 2014, pp. 1–32, 2014.

[16] G. Rosu, "K - a semantic framework for programming languages and formal analysis tools," in *Dependable Software Systems Engineering*, ser. NATO Science for Peace and Security, D. Peled and A. Pretschner, Eds. IOS Press, 2017.

[17] B. Bernardo, R. Cauderlier, Z. Hu, B. Pesin, and J. Tesson, "Mi-cho-coq, a framework for certifying tezos smart contracts," in *Formal Methods. FM 2019 International Workshops*, E. Sekerinski, N. Moreira, J. N. Oliveira, D. Ratiu, R. Guidotti, M. Farrell, M. Luckcuck, D. Marmsoler, J. Campos, T. Astarte, L. Gonnord, A. Cerone, L. Couto, B. Dongol, M. Kutrib, P. Monteiro, and D. Delmas, Eds. Cham: Springer International Publishing, 2020, p. 368–379.

[18] L. M. Goodman, L. Faire, L. Propriétaires, and P. J. Proudhon, "Tezos : A self-amending crypto-ledger position paper," 2014. [Online]. Available: https://api.semanticscholar.org/CorpusID:16555326

[19] A. Arusoaie, "Certifying findel derivatives for blockchain," *Journal of Logical and Algebraic Methods in Programming*, vol. 121, p. 100665, June 2021.

[20] T. S. Authors, "Solidity — Solidity 0.8.29 documentation," 2024. [Online]. Available: https://docs.soliditylang.org/en/latest/

[21] D. Marmsoler and A. D. Brucker, "A Denotational Semantics of Solidity in Isabelle/HOL," in *Software Engineering and Formal Methods*, R. Calinescu and C. S. Păsăreanu, Eds. Cham: Springer International Publishing, 2021, vol. 13085, pp. 403–422, series Title: Lecture Notes in Computer Science. [Online]. Available: https://link.springer.com/10.1007/978-3-030-92124-8_23

[22] D. Marmsoler, https://orcid.org/0000 0003-2859-7673, V. Profile, A. D. Brucker, https://orcid.org/0000 0002-6355-1200, and V. Profile, "Isabelle/solidity: A deep embedding of solidity in isabelle/hol," *Formal Aspects of Computing*, vol. 37, no. 2, p. 1–56, Mar. 2025.

[23] X. Leroy, "A formally verified compiler back-end," *Journal of Automated Reasoning*, vol. 43, no. 4, p. 363–446, Dec. 2009.

[24] T. Nipkow and G. Klein, *Concrete Semantics - With Isabelle/HOL*. Springer, 2014. [Online]. Available: https://doi.org/10.1007/978-3-319-10542-0

[25] R. Milner, *A Calculus of Communicating Systems*, ser. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 1980, vol. 92. [Online]. Available: http://link.springer.com/10.1007/3-540-10235-3

[26] J. Camilleri and G. Winskel, "Ccs with priority choice," in *Proceedings Sixth Annual IEEE Symposium on Logic in Computer Science*, Jul. 1991, p. 246–255. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/151649

[27] R. Milner, *Communicating and mobile systems: the π-calculus*. USA: Cambridge University Press, 1999.

[28] E. W. Dijkstra, "Guarded commands, nondeterminacy and formal derivation of programs," *Communications of the ACM*, vol. 18, no. 8, p. 453–457, Aug. 1975.

[29] P. Tolmach, Y. Li, S.-W. Lin, Y. Liu, and Z. Li, "A survey of smart contract formal specification and verification," *ACM Comput. Surv.*, vol. 54, no. 7, Jul. 2021. [Online]. Available: https://doi-org.libproxy.uwyo.edu/10.1145/3464421

[30] S. M. Beillahi, G. Ciocarlie, M. Emmi, and C. Enea, "Behavioral simulation for smart contracts," in *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI 2020. New York, NY, USA: Association for Computing Machinery, 2020, p. 470–486. [Online]. Available: https://doi-org.uwyo.idm.oclc.org/10.1145/3385412.3386022

[31] M. Bartoletti and R. Zunino, "BitML: A Calculus for Bitcoin Smart Contracts," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '18. New York, NY, USA: Association for Computing Machinery, Oct. 2018, pp. 83–100. [Online]. Available: https://dl.acm.org/doi/10.1145/3243734.3243795

[32] ——, "Verifying liquidity of bitcoin contracts," in *Principles of Security and Trust*, F. Nielson and D. Sands, Eds. Cham: Springer International Publishing, 2019, pp. 222–247.

[33] M. Bartoletti, A. Ferrando, E. Lipparini, and V. Malvone, "Solvent: Liquidity Verification of Smart Contracts," in *Integrated Formal Methods*, N. Kosmatov and L. Kovács, Eds. Cham: Springer Nature Switzerland, 2025, pp. 256–266.

[34] E. Albert, P. Gordillo, A. Hernández-Cerezo, A. Rubio, and M. A. Schett, "Superoptimization of smart contracts," *ACM Trans. Softw. Eng. Methodol.*, vol. 31, no. 4, pp. 70:1–70:29, Jul. 2022.

[35] K. Nelaturu, S. M. Beillahi, F. Long, and A. Veneris, "Smart contracts refinement for gas optimization," in *2021 3rd Conference on Blockchain Research & Applications for Innovative Networks and Services (BRAINS)*, 2021, pp. 229–236.

[36] I. Sergey and A. Hobor, "A concurrent perspective on smart contracts," in *Financial Cryptography and Data Security*, ser. Lecture Notes in Computer Science, M. Brenner, K. Rohloff, J. Bonneau, A. Miller, P. Y. Ryan, V. Teague, A. Bracciali, M. Sala, F. Pintore, and M. Jakobsson, Eds. Cham: Springer International Publishing, 2017, p. 478–493.

[37] M. Qu, X. Huang, X. Chen, Y. Wang, X. Ma, and D. Liu, "Formal Verification of Smart Contracts from the Perspective of Concurrency," in *Smart Blockchain*, M. Qiu, Ed. Cham: Springer International Publishing, 2018, pp. 32–43.

[38] C. A. R. Hoare, *Communicating sequential processes*, ser. Prentice-Hall International series in computer science. Englewood Cliffs, N.J: Prentice/Hall International, 1985.

[39] R. Cooperative, "Contract Design — RChain Architecture 0.9.0 documentation," 2017. [Online]. Available: https://architecture-docs.readthedocs.io/contracts/contract-design.html

[40] M. Bartoletti, R. Marchesin, and R. Zunino, "Secure compilation of rich smart contracts on poor utxo blockchains," in *2024 IEEE 9th European Symposium on Security and Privacy (EuroS&P)*, 2024, pp. 235–267.

[41] L. Ciccone and L. Padovani, "A dependently typed linear π-calculus in agda," in *Proceedings of the 22nd International Symposium on Principles and Practice of Declarative Programming*, ser. PPDP '20. New York, NY, USA: Association for Computing Machinery, 2020. [Online]. Available: https://doi.org/10.1145/3414080.3414109

[42] B. Toninho, L. Caires, and F. Pfenning, "Dependent session types via intuitionistic linear type theory," in *Proceedings of the 13th international ACM SIGPLAN symposium on Principles and practices of declarative programming*. Odense Denmark: ACM, July 2011, p. 161–172. [Online]. Available: https://dl.acm.org/doi/10.1145/2003476.2003499

[43] A. Das and F. Pfenning, "Rast: A language for resource-aware session types," *Logical Methods in Computer Science*, vol. Volume 18, Issue 1, p. 7024, Jan. 2022.

[44] D. Marshall and D. Orchard, "Non-linear communication via graded modal session types," *Information and Computation*, vol. 301, p. 105234, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0890540124000993

[45] S. Kalvala and V. De Paiva, "Mechanizing linear logic in isabelle," in *In 10th International Congress of Logic, Philosophy and Methodology of Science*, vol. 24, 1995.

[46] F. Smola and J. D. Fleuriot, "Linear resources in isabelle/hol," *J. Autom. Reason.*, vol. 68, no. 2, May 2024. [Online]. Available: https://doi.org/10.1007/s10817-024-09698-2

[47] M. Trifunovski, "An interactive proof assistant for linear logic," Bachelor of Arts with Departmental Honors, Wesleyan University, Middletown, CT, 2017. [Online]. Available: https://digitalcollections.wesleyan.edu/object/ir-682

[48] E. Callies and O. Laurent, "Click and coLLecT An Interactive Linear Logic Prover," in *5th International Workshop on Trends in Linear Logic and Applications (TLLA 2021)*, Rome (virtual), Italy, Jun. 2021. [Online]. Available: https://hal-lirmm.ccsd.cnrs.fr/lirmm-03271501

[49] E. Brady, "Idris 2: Quantitative Type Theory in Practice," in *35th European Conference on Object-Oriented Programming (ECOOP 2021)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), A. Møller and M. Sridharan, Eds., vol. 194. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021, pp. 9:1–9:26. [Online]. Available: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2021.9

[50] C. McBride, *I Got Plenty o' Nuttin'*. Cham: Springer International Publishing, 2016, pp. 207–233. [Online]. Available: https://doi.org/10.1007/978-3-319-30936-1_12

[51] R. Atkey, "Syntax and semantics of quantitative type theory," in *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science*, ser. LICS '18. New York, NY, USA: Association for Computing Machinery, July 2018, p. 56–65. [Online]. Available: https://dl.acm.org/doi/10.1145/3209108.3209189

[52] J. S. Hodas and D. Miller, "Logic programming in a fragment of intuitionistic linear logic," *Information and Computation*, vol. 110, no. 2, p. 327–365, May 1994.

[53] I. Cervesato, J. S. Hodas, and F. Pfenning, "Efficient resource management for linear logic proof search," in *Extensions of Logic Programming*, R. Dyckhoff, H. Herre, and P. Schroeder-Heister, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 1996, pp. 67–81.

[54] J. Harland, D. Pym, and M. Winikoff, "Programming in lygon: An overview," in *Algebraic Methodology and Software Technology*, M. Wirsing and M. Nivat, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 1996, pp. 391–405.

[55] G. Allais, "Typing with leftovers - a mechanization of intuitionistic multiplicative-additive linear logic," in *23rd International Conference on Types for Proofs and Programs (TYPES 2017)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), A. Abel, F. Nordvall Forsberg, and A. Kaposi, Eds., vol. 104. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019, pp. 1:1–1:22. [Online]. Available: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.TYPES.2017.1

[56] U. Zalakain and O. Dardha, "π with leftovers: A mechanisation in agda," in *Formal Techniques for Distributed Objects, Components, and Systems: 41st IFIP WG 6.1 International Conference, FORTE 2021, Held as Part of the 16th International Federated Conference on Distributed Computing Techniques, DisCoTec 2021, Valletta, Malta, June 14–18, 2021, Proceedings*. Berlin, Heidelberg: Springer-Verlag, June 2021, p. 157–174. [Online]. Available: https://doi.org/10.1007/978-3-030-78089-0_9

[57] L. Caires, J. A. Pérez, F. Pfenning, and B. Toninho, "Behavioral polymorphism and parametricity in session-based communication," in *Programming Languages and Systems*, M. Felleisen and P. Gardner, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 330–349.

[58] B. Toninho, L. Caires, and F. Pfenning, "Corecursion and non-divergence in session-typed processes," in *Trustworthy Global Computing*, M. Maffei and E. Tuosto, Eds. Berlin, Heidelberg: Springer, 2014, p. 159–175.

[59] Z. Luo, "Extended calculus of constructions," Ph.D. dissertation, The University of Edinburgh, 1990, accepted: 2016-01-19T16:28:16Z publisher: The University of Edinburgh. [Online]. Available: https://era.ed.ac.uk/handle/1842/12487

[60] T. Nipkow, L. C. Paulson, and M. Wenzel, *Isabelle/HOL - A Proof Assistant for Higher-Order Logic*, ser. Lecture Notes in Computer Science. Springer, 2002, vol. 2283. [Online]. Available: https://doi.org/10.1007/3-540-45949-9

[61] Feb. 2026. [Online]. Available: https://github.com/marlowe-lang/marlowe

[62] L. Brünjes and M. J. Gabbay, "UTxO- vs Account-Based Smart Contract Blockchain Programming Paradigms," in *Leveraging Applications of Formal Methods, Verification and Validation: Applications: 9th International Symposium on Leveraging Applications of Formal Methods, ISoLA 2020, Rhodes, Greece, October 20–30, 2020, Proceedings,*

Part III. Berlin, Heidelberg: Springer-Verlag, Oct. 2020, pp. 73–88. [Online]. Available: https://doi.org/10.1007/978-3-030-61467-6_6

[63] D. Kondratiuk, P. L. Seijas, A. Nemish, and S. Thompson, "Standardized crypto-loans on the cardano blockchain," in *Financial Cryptography and Data Security. FC 2021 International Workshops*, M. Bernhard, A. Bracciali, L. Gudgeon, T. Haines, A. Klages-Mundt, S. Matsuo, D. Perez, M. Sala, and S. Werner, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2021, p. 579–594.

[64] D. A. Schmidt, *The structure of typed programming languages*. The Mit Press, 1994.

[65] K. McIlwaine, "Secure systems collaborative / public / wabl / faustus v2 · gitlab," Mar. 2026. [Online]. Available: https://gitlab.com/UWyo-SSC/public/wabl/faustus-v2

[66] A. Hunt and D. Thomas, *The Pragmatic programmer : from journeyman to master*. Boston [etc.]: Addison-Wesley, 2000. [Online]. Available: http://www.amazon.com/The-Pragmatic-Programmer-Journeyman-Master/dp/020161622X

[67] I. Pavlov, "7-zip," 2026. [Online]. Available: https://www.7-zip.org/

[68] R. Milner, *Communication and concurrency*. USA: Prentice-Hall, Inc., 1989.

[69] ——, "Operational and algebraic semantics of concurrent processes," in *Handbook of Theoretical Computer Science, Volume B: Formal Models and Semantics*, J. van Leeuwen, Ed. Elsevier and MIT Press, 1990, pp. 1201–1242. [Online]. Available: https://doi.org/10.1016/b978-0-444-88074-1.50024-x

[70] ——, "Processes: A mathematical model of computing agents," in *Logic Colloquium '73*, ser. Studies in Logic and the Foundations of Mathematics, H. Rose and J. Shepherdson, Eds. Elsevier, 1975, vol. 80, pp. 157–173. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0049237X08719487

[71] C. A. R. Hoare, "Communicating sequential processes," *Commun. ACM*, vol. 21, no. 8, pp. 666–677, 1978. [Online]. Available: https://doi.org/10.1145/359576.359585

[72] "isabelle · master · Secure Systems Collaborative / Public / WABL / Faustus V2 · GitLab," Feb. 2025. [Online]. Available: https://gitlab.com/UWyo-SSC/public/wabl/faustus-v2/-/tree/master/isabelle

[73] M. M. T. Chakravarty, J. Chapman, K. MacKenzie, O. Melkonian, M. Peyton Jones, and P. Wadler, "The extended utxo model," in *Financial Cryptography and Data Security: FC 2020 International Workshops, AsiaUSEC, CoDeFi, VOTING, and WTSC, Kota Kinabalu, Malaysia, February 14, 2020, Revised Selected Papers*. Berlin, Heidelberg: Springer-Verlag, 2020, p. 525–539. [Online]. Available: https://doi.org/10.1007/978-3-030-54455-3_37

[74] "Cardano protocol parameters reference guide | Cardano Docs." [Online]. Available: https://docs.cardano.org/about-cardano/explore-more/parameter-guide

[75] B. Bush, "marlowe-cardano/marlowe/best-practices.md at main · input-output-hk/marlowe-cardano · GitHub," Jan. 2023. [Online]. Available: https://github.com/input-output-hk/marlowe-cardano/blob/main/marlowe/best-practices.md#avoid-or-break-up-complex-logic-in-the-contract

[76] H. R. Andersen and M. Mendler, "An asynchronous process algebra with multiple clocks," in *Programming Languages and Systems — ESOP '94*, D. Sannella, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 1994, pp. 58–73.

[77] L. Liquori and M. Mendler, "Strong Priority and Determinacy in Timed CCS," Inria ; University of Bamberg, Tech. Rep., Dec. 2023. [Online]. Available: https://inria.hal.science/hal-04367635

[78] L. Caires and F. Pfenning, "Session types as intuitionistic linear propositions," in *CONCUR 2010 - Concurrency Theory*, P. Gastin and F. Laroussinie, Eds. Berlin, Heidelberg: Springer, 2010, p. 222–236.

[79] J.-Y. Girard, "Linear logic," *Theoretical Computer Science*, vol. 50, no. 1, p. 1–101, Jan. 1987.

[80] J. Y. Girard and Y. Lafont, "Linear logic and lazy computation," in *TAPSOFT '87*, H. Ehrig, R. Kowalski, G. Levi, and U. Montanari, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 1987, pp. 52–66.

[81] G. M. Bierman and W. College, "On intuitionistic linear logic," Ph.D. dissertation, University of Cambridge, Dec. 1993.

[82] P. Wadler, "A taste of linear logic," in *Proceedings of the 18th International Symposium on Mathematical Foundations of Computer Science*, ser. MFCS '93. Berlin, Heidelberg: Springer-Verlag, 1993, p. 185–210.

[83] K. Honda, "Types for dyadic interaction," in *CONCUR'93*, E. Best, Ed. Berlin, Heidelberg: Springer, 1993, p. 509–523.

[84] P. Wadler, "Propositions as sessions," in *Proceedings of the 17th ACM SIGPLAN International Conference on Functional Programming*, ser. ICFP '12. New York, NY, USA: Association for Computing Machinery, 2012, p. 273–286. [Online]. Available: https://doi.org/10.1145/2364527.2364568

[85] P. Martin-Löf, "An intuitionistic theory of types: Predicative part," in *Logic Colloquium '73*, ser. Studies in Logic and the Foundations of Mathematics, H. Rose and J. Shepherdson, Eds. Elsevier, 1975, vol. 80, pp. 73–118. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0049237X08719451

[86] T. Coquand and G. Huet, "The calculus of constructions," *Information and Computation*, vol. 76, no. 2, p. 95–120, Feb. 1988.

[87] L. C. Paulson, Ed., *Isabelle*, ser. Lecture Notes in Computer Science. Berlin/Heidelberg: Springer-Verlag, 1994, vol. 828. [Online]. Available: http://link.springer.com/10.1007/BFb0030541

[88] P. Martin-Löf and G. Sambin, *Intuitionistic type theory.* Bibliopolis Naples, 1984, vol. 9.

[89] R. L. Constable, S. F. Allen, H. M. Bromley, W. R. Cleaveland, J. F. Cremer, R. W. Harper, D. J. Howe, T. B. Knoblock, N. P. Mendler, P. Panangaden, J. T. Sasaki, and S. F. Smith, *Implementing mathematics with the Nuprl proof development system.* USA: Prentice-Hall, Inc., 1986.

[90] *The Rocq Prover*, The Rocq Development Team, Sep. 2025. [Online]. Available: https://doi.org/10.5281/zenodo.17473943

[91] M. J. Gordon, A. J. Milner, and C. P. Wadsworth, *Edinburgh LCF*, ser. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 1979, vol. 78. [Online]. Available: http://link.springer.com/10.1007/3-540-09724-4

[92] A. Gacek, "The abella interactive theorem prover (system description)," in *Automated Reasoning*, A. Armando, P. Baumgartner, and G. Dowek, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2008, pp. 154–161.

[93] A. Felty and A. Momigliano, "Hybrid," *Journal of Automated Reasoning*, vol. 48, no. 1, p. 43–105, Jan. 2012.

[94] B. Toninho, L. Caires, and F. Pfenning, "A decade of dependent session types," in *Proceedings of the 23rd International Symposium on Principles and Practice of Declarative Programming*, ser. PPDP '21. New York, NY, USA: Association for Computing Machinery, Oct. 2021, p. 1–3. [Online]. Available: https://dl.acm.org/doi/10.1145/3479394.3479398

[95] M. J. Gordon, A. J. Milner, and C. P. Wadsworth, *Edinburgh LCF*, ser. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 1979, vol. 78. [Online]. Available: http://link.springer.com/10.1007/3-540-09724-4

[96] B. Nordström, K. Petersson, and J. M. Smith, *Programming in Martin-Löf's type theory.* Oxford University Press Oxford, 1990, vol. 200.

[97] Z. Luo and R. Pollack, *LEGO proof development system: User's manual.* LFCS, Department of Computer Science, University of Edinburgh, 1992.

[98] L. C. Paulson, "Isabelle: The next 700 theorem provers," *CoRR*, vol. cs.LO/9301106, 1993. [Online]. Available: https://arxiv.org/abs/cs/9301106

[99] U. Norell, "Towards a practical programming language based on dependent type theory," Ph.D. dissertation, Chalmers University of Technology, Göteborg, Sweden,, 2007. [Online]. Available: https://www.cse.chalmers.se/~ulfn/papers/thesis.pdf

[100] L. d. Moura and S. Ullrich, "The lean 4 theorem prover and programming language," in *Automated Deduction – CADE 28: 28th International Conference on Automated Deduction, Virtual Event, July 12–15, 2021, Proceedings*. Berlin, Heidelberg: Springer-Verlag, 2021, p. 625–635. [Online]. Available: https://doi.org/10.1007/978-3-030-79876-5_37

[101] B. Toninho and N. Yoshida, *Polymorphic Session Processes as Morphisms*. Cham: Springer International Publishing, 2019, pp. 101–117. [Online]. Available: https://doi.org/10.1007/978-3-030-31175-9_7

[102] J. S. Hodas, P. López, J. Polakow, L. Stoilova, and E. Pimentel, "A tag-frame system of resource management for proof search in linear-logic programming," in *Computer Science Logic*, J. Bradfield, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 2002, pp. 167–182.

[103] A. Martin, R. Nickson, and M. Utting, "A tactic language for ergo," in *Formal Methods Pacific '97*, 1997.

[104] A. Martin and J. Gibbons, "A monadic interpretation of tactics," 2002, accessed from https://www.cs.ox.ac.uk/jeremy.gibbons/publications/tactics.pdf.

[105] T. S. D. Team, Feb. 2026. [Online]. Available: https://github.com/STILL-prover/STILL

[106] D. Griffith and E. L. Gunter, "Liquidpi: Inferrable dependent session types," in *NASA Formal Methods*, G. Brat, N. Rungta, and A. Venet, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 185–197.

[107] L. Caires and B. Toninho, "The session abstract machine," in *Programming Languages and Systems*, S. Weirich, Ed. Cham: Springer Nature Switzerland, 2024, pp. 206–235.

[108] B. Blanchet, M. Abadi, and C. Fournet, "Automated verification of selected equivalences for security protocols," *The Journal of Logic and Algebraic Programming*, vol. 75, no. 1, p. 3–51, Feb. 2008.

[109] A. Das, S. Balzer, J. Hoffmann, F. Pfenning, and I. Santurkar, "Resource-aware session types for digital contracts," in *2021 IEEE 34th Computer Security Foundations Symposium (CSF)*, 2021, pp. 1–16.

[110] K. McIlwaine and J. Caldwell, "Verifying Smart Contract Transformations Using Bisimulations," in *6th International Workshop on Formal Methods for Blockchains (FMBC 2025)*, ser. Open Access Series in Informatics (OASIcs), D. Marmsoler and M. Xu, Eds., vol. 129. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, pp. 9:1–9:19. [Online]. Available: https://drops.dagstuhl.de/entities/document/10.4230/OASIcs.FMBC.2025.9

[111] T. Coquand and G. Huet, *Concepts Mathématiques et Informatiques Formalisés dans le Calcul des Constructions*. Elsevier, 1987, vol. 122, p. 123–146. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0049237X09705518

[112] J.-Y. Girard, "Interprétation fonctionnelle et élimination des coupures de l'arithmétique d'ordre supérieur," PhD Thesis, University of Paris, June 1972.

[113] J. C. Reynolds, "Towards a theory of type structure," in *Programming Symposium*, B. Robinet, Ed. Berlin, Heidelberg: Springer, 1974, p. 408–425.

[114] G. Wraith, "A note on categorical data types, category theory and computer science (dh pitt, de rydeheard, p. dybjer, am pitts, a. poignb, eds.), lncs 389," 1989.

[115] P. Wadler, "Recursive types for free," 1990.

[116] B. Toninho and N. Yoshida, "On polymorphic sessions and functions: A tale of two (fully abstract) encodings," *ACM Transactions on Programming Languages and Systems*, vol. 43, no. 2, p. 1–55, June 2021.

[117] OpenAI, "Introducing chatgpt," Mar. 2024. [Online]. Available: https://openai.com/index/chatgpt/

[118] G. Lowe, "Breaking and fixing the needham-schroeder public-key protocol using fdr," in *Tools and Algorithms for the Construction and Analysis of Systems*, T. Margaria and B. Steffen, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 1996, pp. 147–166.

[119] F. Haftmann and T. Nipkow, "Code generation via higher-order rewrite systems," in *Functional and Logic Programming*, M. Blume, N. Kobayashi, and G. Vidal, Eds. Berlin, Heidelberg: Springer, 2010, p. 103–117.